Welcome

Thank you for choosing Freenove products!

Getting Started

When reading this, you should have downloaded the ZIP file for this product.
Unzip it and you will get a folder containing tutorials and related files. Please start with this PDF tutorial.

! Unzip the ZIP file instead of opening the file in the ZIP file directly.
! Do not move, delete or rename files in the folder just unzipped.

Get Support

Encounter problems? Don't worry! Refer to “TroubleShooting.pdf” or contact us.

When there are packaging damage, quality problems, questions encountering in use, etc., just send us an email. We will reply to you within one working day and provide a solution.

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be used only when there is adult supervision present as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.
- When the product is turned ON, activated or tested, some parts will move or rotate. To avoid injuries to hands and fingers, keep them away from any moving parts!
- It is possible that an improperly connected or shorted circuit may cause overheating. Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down! When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

http://www.freenove.com

Copyright

All the files, materials and instructional guides provided are released under Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License. A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.



This means you can use these resource in your own derived works, in part or completely, but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.

FREENOVE™

Other registered trademarks and their owners appearing in this document:
Arduino® is a trademark of Arduino LLC (https://www.arduino.cc/).
Raspberry Pi® is a trademark of Raspberry Pi Foundation (https://www.raspberrypi.org/).
Raspberry Pi Pico® is a trademark of Raspberry Pi Foundation (https://www.raspberrypi.org/).
micro:bit® is a trademark of Micro:bit Educational Foundation (https://www.microbit.org/).
ESPRESSIF® and ESP32® are trademarks of ESPRESSIF Systems (Shanghai) Co, Ltd (https://www.espressif.com/).

Any concerns? ✉ support@freenove.com

Contents

Welcome 1
Contents 1
Preface 5
Raspberry Pi Pico 6
Raspberry Pi Pico W 10
Raspberry Pi Pico 2 13
Raspberry Pi Pico 2W 16
Chapter 0 Getting Ready (Important) 19
0.1 Installing Thonny (Important) 19
0.2 Basic Configuration of Thonny 24
0.3 Burning Micropython Firmware (Important) 26
0.4 Thonny Connected to Raspberry Pi Pico 30
0.5 Testing codes (Important) 34
0.6 Thonny Common Operation 45
0.7 Paste the Sticker on the Breadboard 50
Chapter 1 LED (Important) 51
Project 1.1 Blink 51
Project 1.2 Blink 61
Chapter 2 Button & LED 69
Project 2.1 Button & LED 70
Project 2.2 MINI table lamp 74
Chapter 3 LED Bar 77
Project 3.1 Flowing Light 77
Chapter 4 Analog & PWM 83
Project 4.1 Breathing LED 83
Project 4.2 Meteor Flowing Light 89
How to import a custom python module 93
Chapter 5 RGBLED 95
Project 5.1 Random Color Light 95
Project 5.2 Gradient Color Light 101

Chapter 6 NeoPixel 104
Project 6.1 NeoPixel 104
Project 6.2 Rainbow Light 110
Chapter 7 Buzzer 113
Project 7.1 Doorbell 113
Project 7.2 Alertor 119
Chapter 8 Serial Communication 123
Project 8.1 Serial Print 123
Project 8.2 Serial Read and Write 128
Chapter 9 AD Converter 132
Project 9.1 Read the Voltage of Potentiometer 132
Chapter 10 Potentiometer & LED 138
Project 10.1 Soft Light 138
Project 10.2 Soft Colorful Light 142
Project 10.3 Soft Rainbow Light 146
Chapter 11 Photoresistor & LED 150
Project 11.1 Control LED through Photoresistor 150
Chapter 12 Thermistor 155
Project 12.1 Thermometer 155
Chapter 13 Joystick 160
Project 13.1 Joystick 160
Chapter 14 74HC595 & LED Bar Graph 165
Project 14.1 Flowing Water Light 165
Chapter 15 74HC595 & 7-Segment Display. 171
Project 15.1 7-Segment Display. 171
Project 15.2 4-Digit 7-Segment Display 177
Chapter 16 74HC595 & LED Matrix 183
Project 16.1 LED Matrix 183
Chapter 17 Relay & Motor 191
Project 17.1 Relay & Motor 191
Chapter 18 L293D & Motor 199

Project 18.1 Control Motor with Potentiometer 199
Chapter 19 Servo 208
Project 19.1 Servo Sweep 208
Project 19.2 Servo Knob 213
Chapter 20 Stepper Motor 217
Project 20.1 Stepper Motor 217
Chapter 21 LCD1602 226
Project 21.1 LCD1602 226
Chapter 22 Ultrasonic Ranging 233
Project 22.1 Ultrasonic Ranging 233
Project 22.2 Ultrasonic Ranging 239
Chapter 23 Matrix Keypad 241
Project 23.1 Matrix Keypad 241
Project 23.2 Keypad Door 246
Chapter 24 Infrared Remote 251
Project 24.1 Infrared Remote Control 251
Project 24.2 Control LED through Infrared Remote 257
Chapter 25 Hygrothermograph DHT11 262
Project 25.1 Hygrothermograph 262
Project 25.2 Hygrothermograph 267
Chapter 26 Infrared Motion Sensor 272
Project 26.1 Infrared Motion Detector with LED Indicator 272
Chapter 27 Attitude Sensor MPU6050 278
Project 27.1 Read a MPU6050 Sensor Module 278
Chapter 28 RFID 284
Project 28.1 RFID read UID 284
Project 28.2 RFID Read and Write 294
Chapter 29 WiFi Working Modes (Only for Pico W/Pico 2W) 299
Project 29.1 Station mode 299
Project 29.2 AP mode 304
Project 29.3 AP+Station mode 308
Chapter 30 TCP/IP (Only for Pico W/Pico 2W) 312

Project 30.1 as Client 312
Project 30.2 as Server 325
Chapter 31 Control LED with Web (Only for Pico W/Pico 2W) 331
Project 31.1 Control_LED_through_Web 331
Chapter 32 Bluetooth (Only for Pico W/Pico 2W) 338
Project 32.1 Bluetooth Low Energy Data Passthrough 338
Project 32.2 Bluetooth Low Energy Control LED 350
What's Next? 358

Preface

Raspberry Pi Pico is a tiny, fast, and versatile board built using RP2040, a brand new microcontroller chip designed by Raspberry Pi in the UK. Getting started is as easy as dragging and dropping a file, it is suitable for beginners and makers to develop, design and research.

Raspberry Pi Pico is programmable in C/C++ and MicroPython. In this tutorial, we use Micropython to develop, which is a very easy to learn language with lean and simple code, hence it is very suitable for beginners to learn and for secondary development.

If you have not downloaded the related material for Raspberry Pi Pico tutorial, you can download it from this link:
https://github.com/Freenove/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico

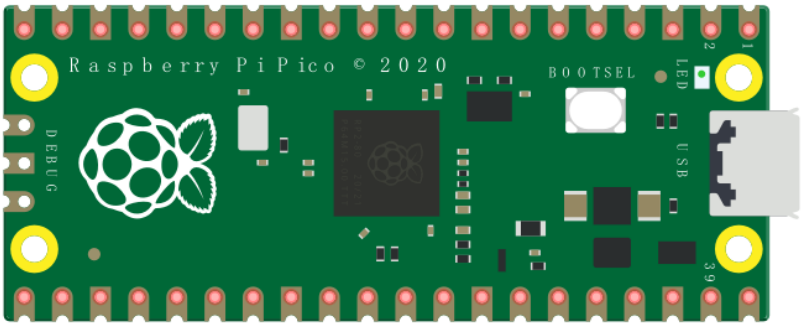
In this tutorial, we divide each project into 4 sections:
1, Component list: helps users to learn and find what components needed in each project.
2, Component knowledge: allows you to learn the features and usage of the components.
3, Circuit: assists to build circuit for each project.
4, Code and annotation: makes it easier for users to learn to use Raspberry Pi Pico and make secondary development.

After completing the projects in this tutorial, you can also combine the components in different projects to make your own smart homes, smart car, robot, etc., bringing your imagination and creativity to life with Raspberry Pi Pico.

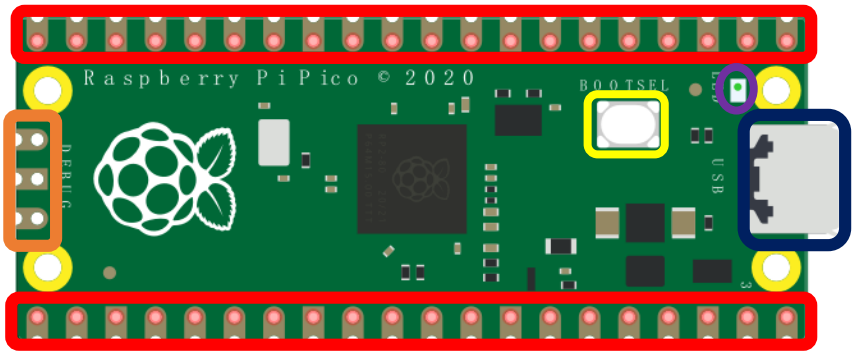
If you have any problems or difficulties using this product, please contact us for quick and free technical support: support@freenove.com

Raspberry Pi Pico

Raspberry Pi Pico is a light-weight electronic product with tiny size and low price. From the picture below, we can see that its onboard resources have been connected to the edge interface, which is very suitable for electronic enthusiasts to use in DIY.

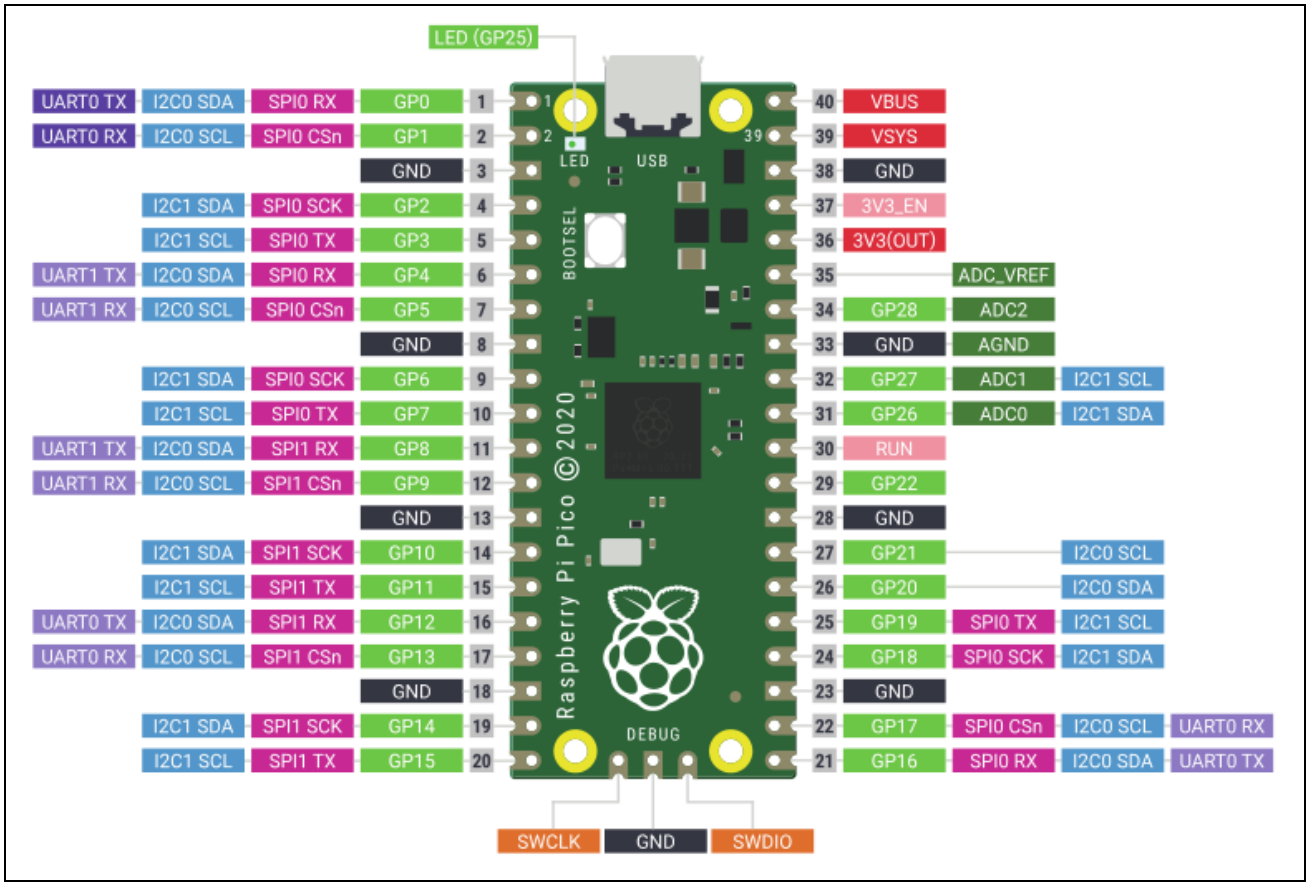


The hardware interfaces are distributed as follows:



Frame color	Description
	Pins
	BOOTSE button
	USB port
	LED
	Debugging

Function definition of pins:



Color	Pins	Color	Pins
	GND		Power
	GPIO		ADC
	UART(default)		UART
	SPI		I2C
	System Control		Debugging

For details: https://datasheets.raspberrypi.org/pico/pico-datasheet.pdf

GND	Ground Pin
Power	VBUS(microUSB Voltage)、VSYS(2-5VDC Input)、3V3(3.3V OUT)、3V3_EN(Enables Pico)
System Control	Run(Start or disable RP2040 microcontroller or reset)
ADC	Raspberry Pi Pico has a total of 5 ADC with a resolution of 12 bits, which are ADC0(GP26), ADC1(GP27), ADC2(GP28), ADC3(GP29), ADC4 respectively. Among them, ADC3(GP29) is used to measure the VSYS on Pico board; ADC4 is directly connected to the RP2040's built-in temperature sensor. ADC_VREF can connect to external accurate voltmeter as ADC reference. ADC_GND pin is used as the reference point for grounding.
PWM	There are 16 PWM channels on Raspberry Pi Pico, each of which can control frequency and duty cycle independently. The GPIO pins are switched to PWM function.
UART	There are 2 UART: UART0, UART1.
SPI	There are 2 SPI: SPI0, SPI1.
I2C	2 I2C: I2C0, I2C1.
Debugging	It is used when debugging code.

UART, I2C, SPI Default Pin

UART

Function	Default
UART_BAUDRATE	115200
UART_BITS	8
UART_STOP	1
UART0_TX	Pin 0
UART0_RX	Pin 1
UART1_TX	Pin 4
UART1_RX	Pin 5

I2C

Function	Default
I2C Frequency	400000
I2C0 SCL	Pin 9
I2C0 SDA	Pin 8
I2C1 SCL	Pin 7
I2C1 SDA	Pin 6

SPI

Function	Default
SPI_BAUDRATE	1000000
SPI_POLARITY	0
SPI_PHASE	0
SPI_BITS	8
SPI_FIRSTBIT	MSB
SPI0_SCK	Pin 6
SPI0_MOSI	Pin 7
SPI0_MISO	Pin 4
SPI1_SCK	Pin 10
SPI1_MOSI	Pin 11
SPI1_MISO	Pin 8

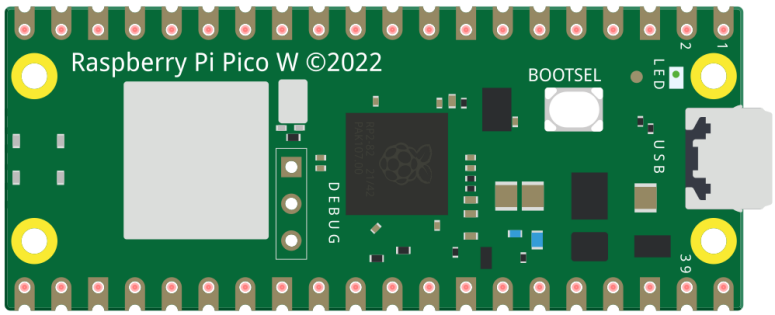
For more detailed information, please refer to:

https://datasheets.raspberrypi.org/pico/raspberry-pi-pico-python-sdk.pdf

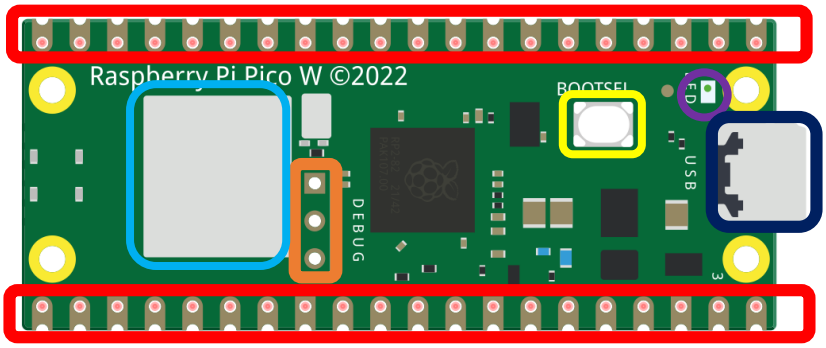
Raspberry Pi Pico W

Raspberry Pi Pico W applies to all chapters in this tutorial.

Raspberry Pi Pico W adds CYW43439 as the WiFi function based on Raspberry Pi Pico. It is connected to RP2040 chip through SPI interface.

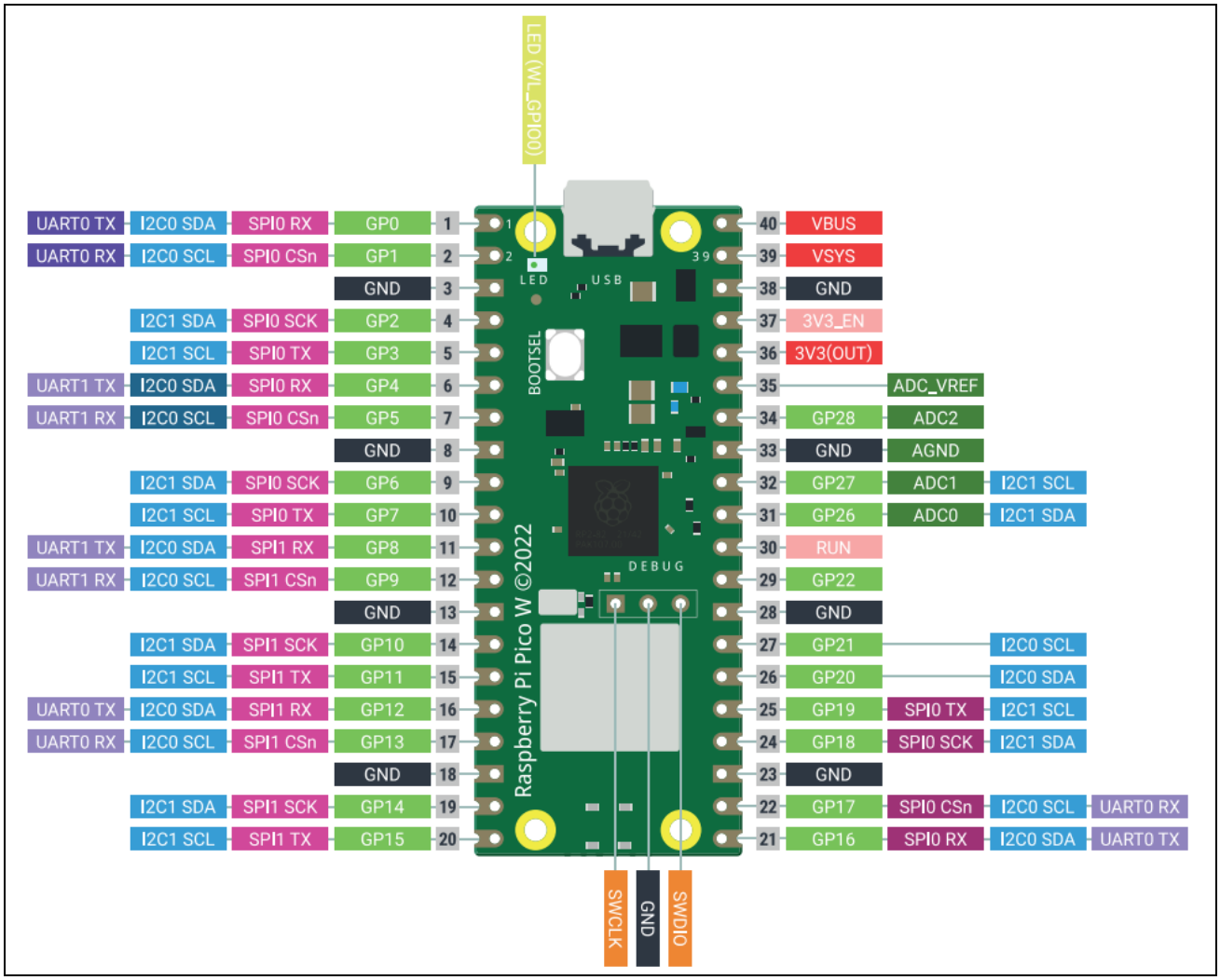


The hardware interfaces are distributed as follows:



Frame color	Description
	Pins
	BOOTSE button
	USB port
	LED
	Debugging
	Wireless

Function definition of pins:



Color	Pins	Color	Pins
	GND		Power
	GPIO		ADC
	UART(default)		UART
	SPI		I2C
	System Control		Debugging

For details: https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf

UART, I2C, SPI, Wireless Default Pin

The default pins of serial port are Pin0 and Pin1.
Note: Serial port is virtualized by RP2040. Therefore, when using the serial port, please enable the verification function of DTR. It can work under any baud rate.

UART

Function	Default
UART_BAUDRATE	X
UART_BITS	8
UART_STOP	1
UART_TX	Pin 0
UART_RX	Pin 1

I2C

Function	Default
I2C Frequency	400000
I2C_SDA	Pin 4
I2C_SCL	Pin 5

SPI

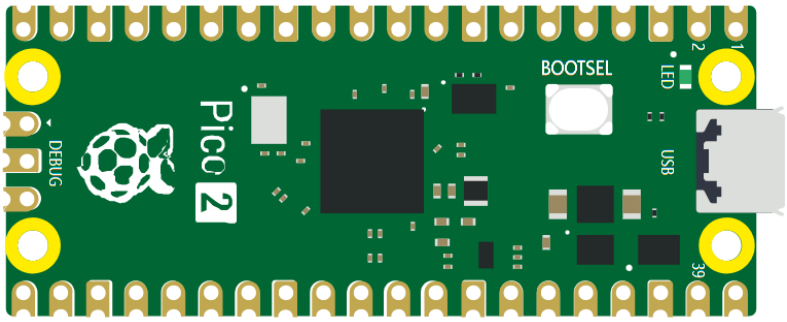
Function	Default
SPI_BAUDRATE	1000000
SPI_POLARITY	0
SPI_PHASE	0
SPI_BITS	8
SPI_FIRSTBIT	MSB
SPI_SCK	Pin 18
SPI_MOSI	Pin 19
SPI_MISO	Pin 16
SPI_SS	Pin 17

Wireless

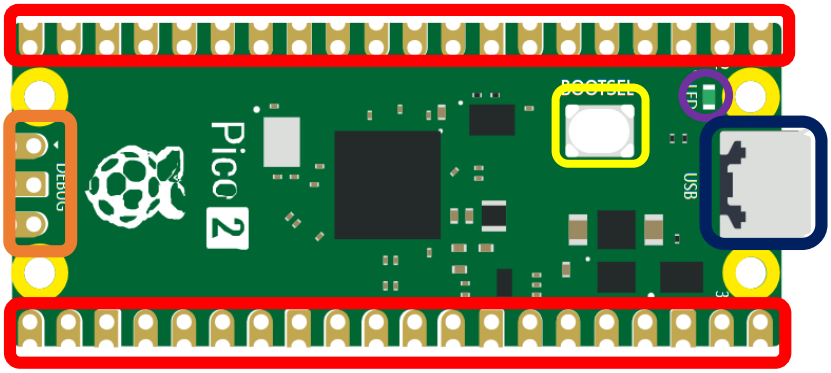
Function	Default
WL_ON	GPIO23
WL_D	GPIO24
WL_CLK	GPIO29_ADC
WL_CS	GPIO25

Raspberry Pi Pico 2

Raspberry Pi Pico 2 is applicable to all chapters in this tutorial except RFID and those involving WiFi.
Raspberry Pi Pico 2 uses RP2350 chip as the main controller, which equipped with dual Cortex-M33 or Hazard3 processors, capable of running up to 150 MHz, providing a significant boost in processing power, compared with the original Pico. It also doubles the memory with 520KB of SRAM and 4MB of onboard flash memory, with the ADC sampling frequency increasing to up to 500ksps. In addition, it adds 8 more PWM channels, and features additional interfaces like 2× Timer with 4 alarms, 1× AON Timer and 4 x PIO.

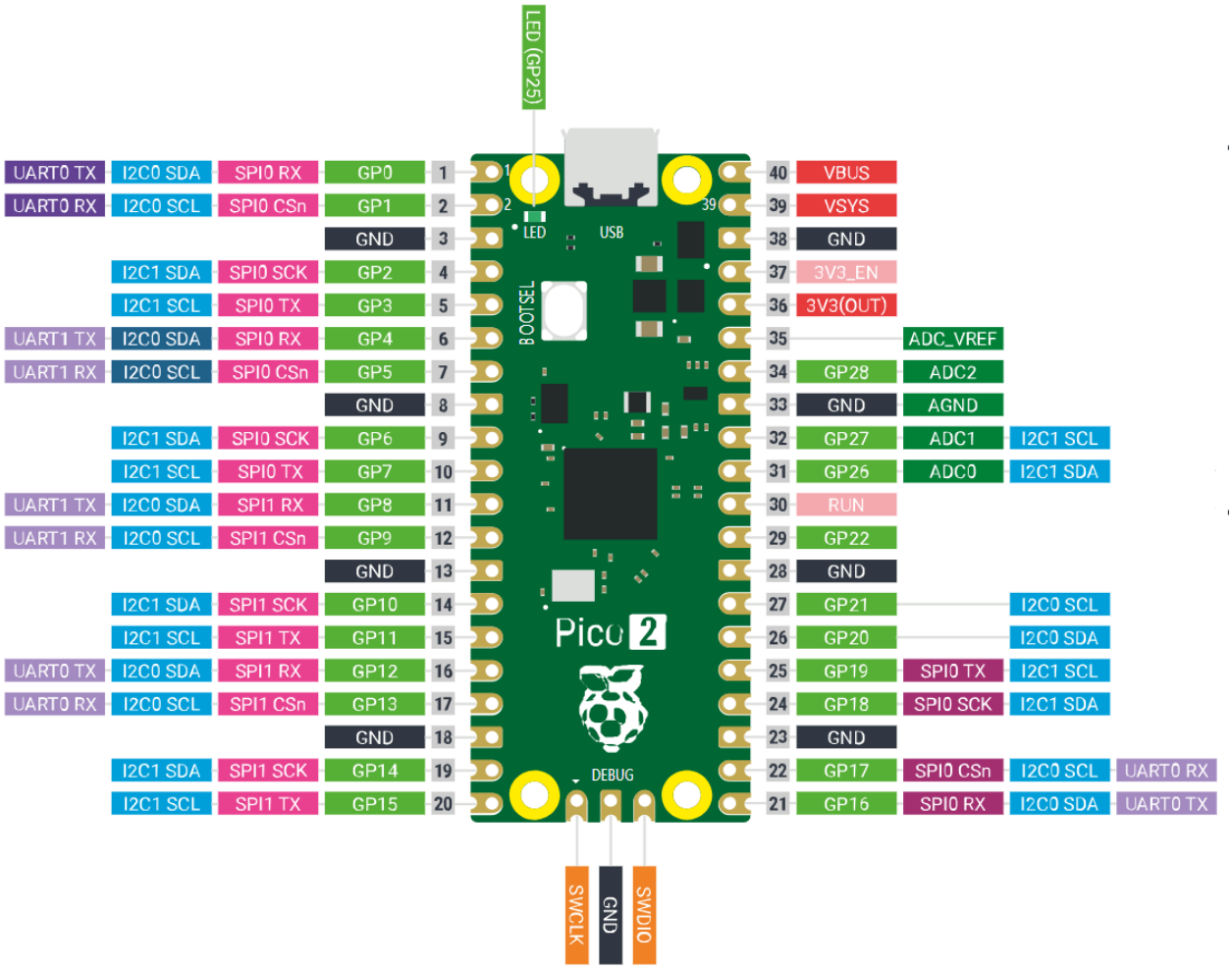


The hardware interfaces are distributed as follows:



Frame color	Description
	Pins
	BOOTSE button
	USB port
	LED
	Debugging

Function definition of pins:



Color	Pins	Color	Pins
	GND		Power
	GPIO		ADC
	UART(default)		UART
	SPI		I2C
	System Control		Debugging

For details: https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf

UART, I2C, SPI Default Pin

The default pins of serial port are Pin0 and Pin1.
Note: Serial port is virtualized by RP2350. Therefore, when using the serial port, please enable the verification function of DTR. It can work under any baud rate.

UART

Function	Default
UART_BAUDRATE	X
UART_BITS	8
UART_STOP	1
UART_TX	Pin 0
UART_RX	Pin 1

I2C

Function	Default
I2C Frequency	400000
I2C_SDA	Pin 4
I2C_SCL	Pin 5

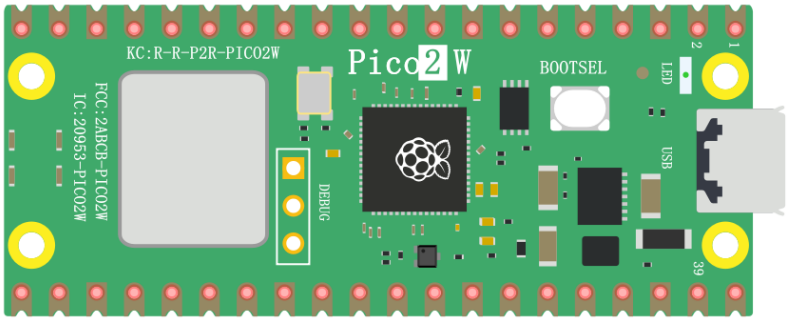
SPI

Function	Default
SPI_BAUDRATE	1000000
SPI_POLARITY	0
SPI_PHASE	0
SPI_BITS	8
SPI_FIRSTBIT	MSB
SPI_SCK	Pin 18
SPI_MOSI	Pin 19
SPI_MISO	Pin 16
SPI_SS	Pin 17

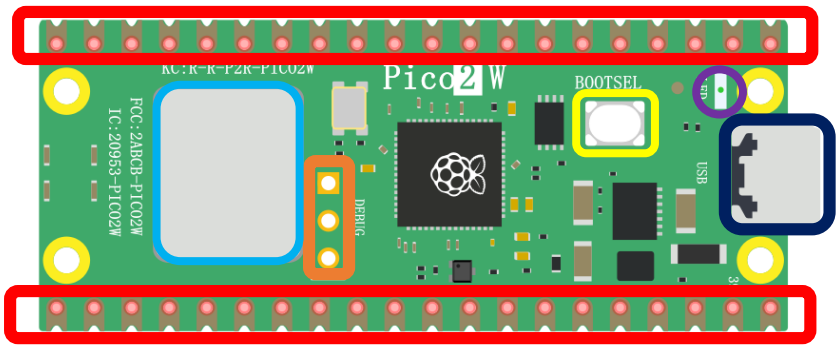
Raspberry Pi Pico 2W

Raspberry Pi Pico 2W applies to all chapters in this tutorial.

The Raspberry Pi Pico 2W adds WiFi functionality based on the Raspberry Pi Pico 2 by incorporating the CYW43439 module, which connects to the RP2350 chip via an SPI interface.

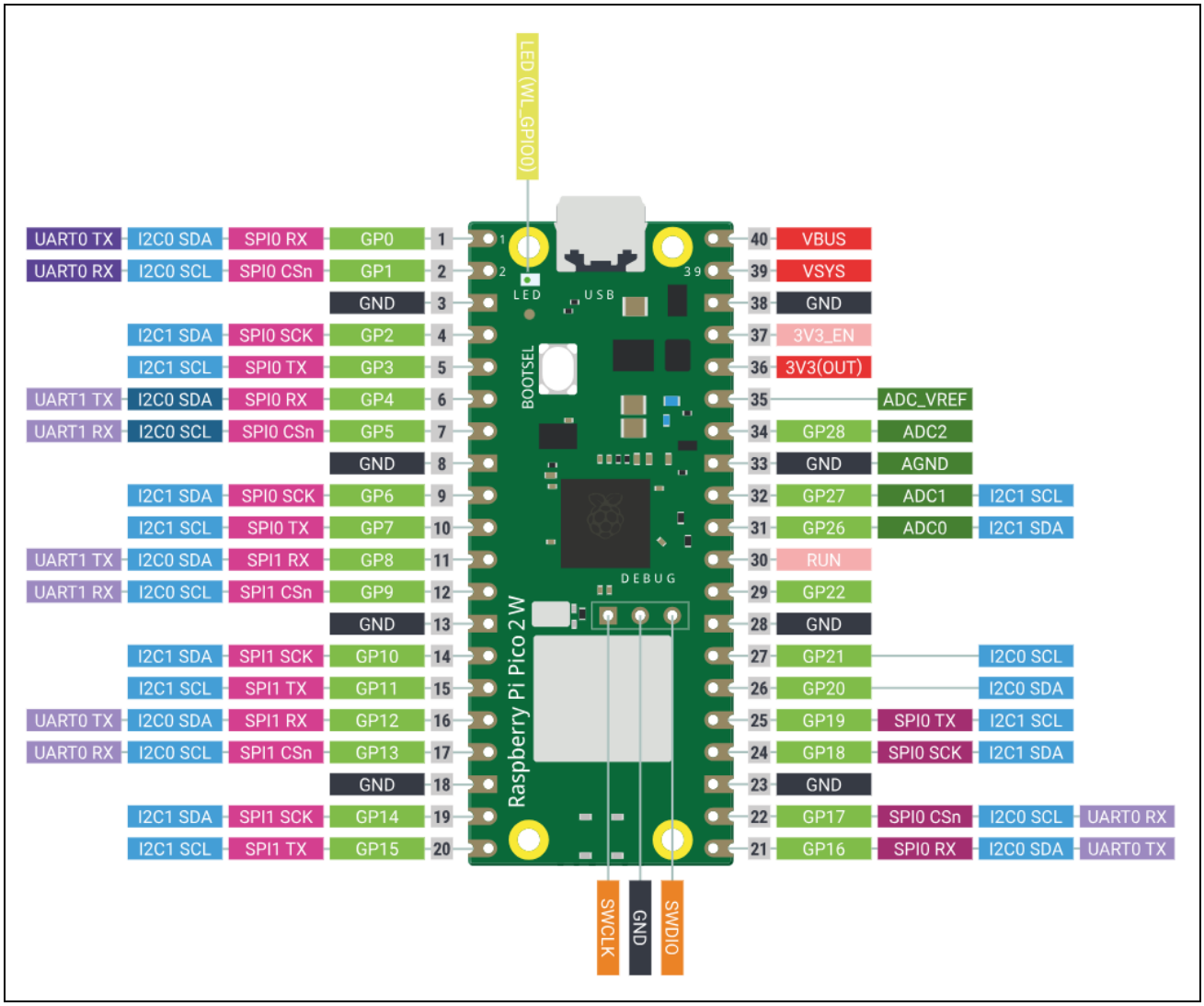


The hardware interfaces are distributed as follows:



Frame color	Description
	Pins
	BOOTSE button
	USB port
	LED
	Debugging
	Wireless

Function definition of pins:



Color	Pins	Color	Pins
	GND		Power
	GPIO		ADC
	UART(default)		UART
	SPI		I2C
	System Control		Debugging

For details: https://datasheets.raspberrypi.com/picow/pico-2-w-datasheet.pdf

UART, I2C, SPI, Wireless Default Pin

The default pins of serial port are Pin0 and Pin1.
Note: Serial port is virtualized by RP2350. Therefore, when using the serial port, please enable the verification function of DTR. It can work under any baud rate.

UART

Function	Default
UART_BAUDRATE	X
UART_BITS	8
UART_STOP	1
UART_TX	Pin 0
UART_RX	Pin 1

I2C

Function	Default
I2C Frequency	400000
I2C_SDA	Pin 4
I2C_SCL	Pin 5

SPI

Function	Default
SPI_BAUDRATE	1000000
SPI_POLARITY	0
SPI_PHASE	0
SPI_BITS	8
SPI_FIRSTBIT	MSB
SPI_SCK	Pin 18
SPI_MOSI	Pin 19
SPI_MISO	Pin 16
SPI_SS	Pin 17

Wireless

Function	Default
WL_ON	GPIO23
WL_D	GPIO24
WL_CLK	GPIO29_ADC
WL_CS	GPIO25

Chapter 0 Getting Ready (Important)

Before starting building the projects, you need to make some preparation first, which is so crucial that you must not skip.

0.1 Installing Thonny (Important)

Thonny is a free, open-source software platform with compact size, simple interface, simple operation and rich functions, making it a Python IDE for beginners. In this tutorial, we use this IDE to develop Raspberry Pi Pico during the whole process.
Thonny supports various operating system, including Windows、Mac OS、Linux.

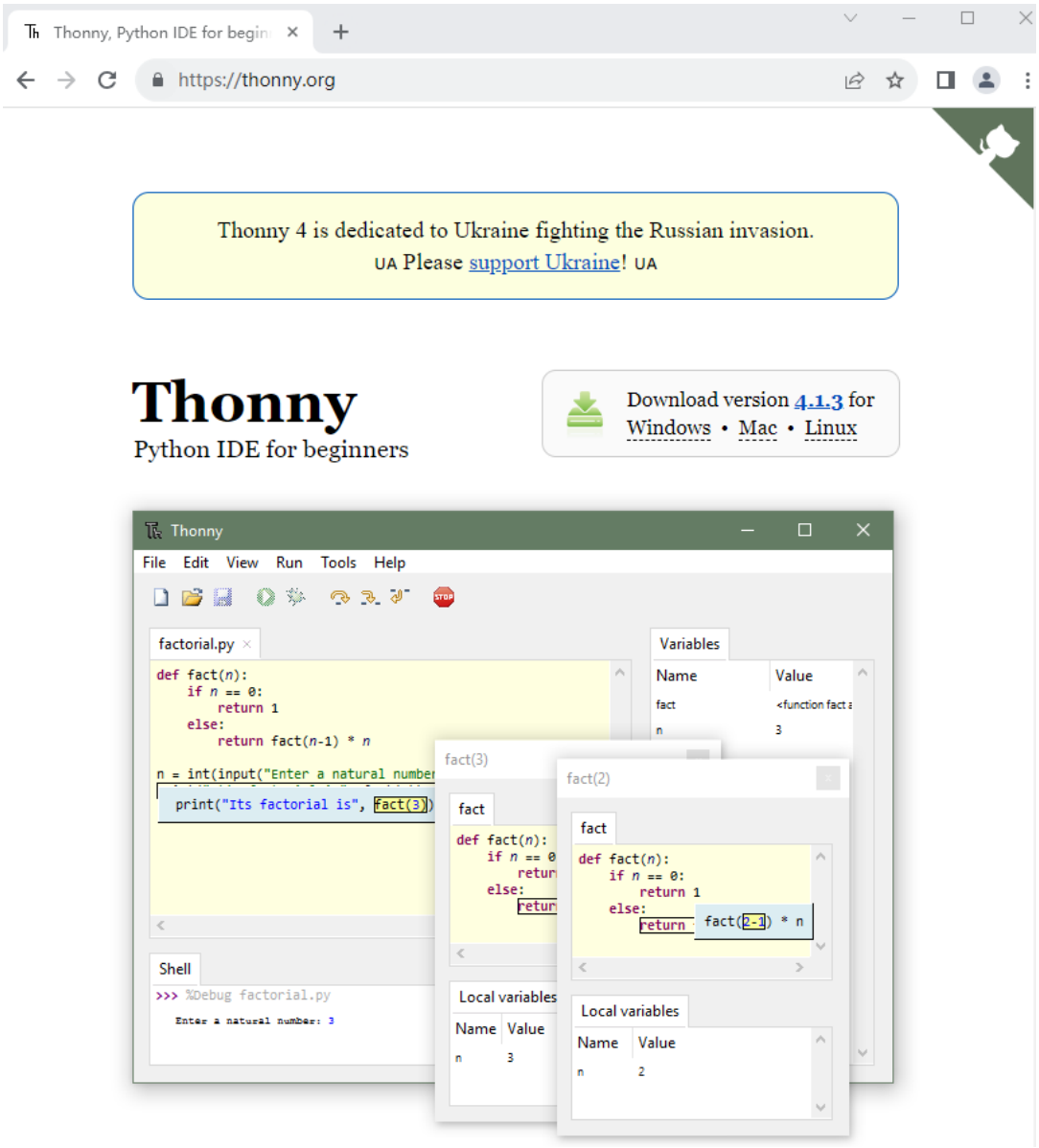
Downloading Thonny

Official website of Thonny: https://thonny.org
Open-source code repositories of Thonny: https://github.com/thonny/thonny

Follow the instruction of official website to install Thonny or click the links below to download and install. (Select the appropriate one based on your operating system.)

Operating System	Download links/methods
Windows	https://github.com/thonny/thonny/releases/download/v4.1.3/thonny-4.1.3.exe
Mac OS	https://github.com/thonny/thonny/releases/download/v4.1.3/thonny-4.1.3.pkg
Linux	**The latest version:** **Binary bundle for PC (Thonny+Python):** bash <(wget -O - https://thonny.org/installer-for-linux) **With pip:** pip3 install thonny **Distro packages (may not be the latest version):** **Debian, Rasbian, Ubuntu, Mint and others:** sudo apt install thonny **Fedora:** sudo dnf install thonny

You can also open "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Software**", we have prepared it in advance.

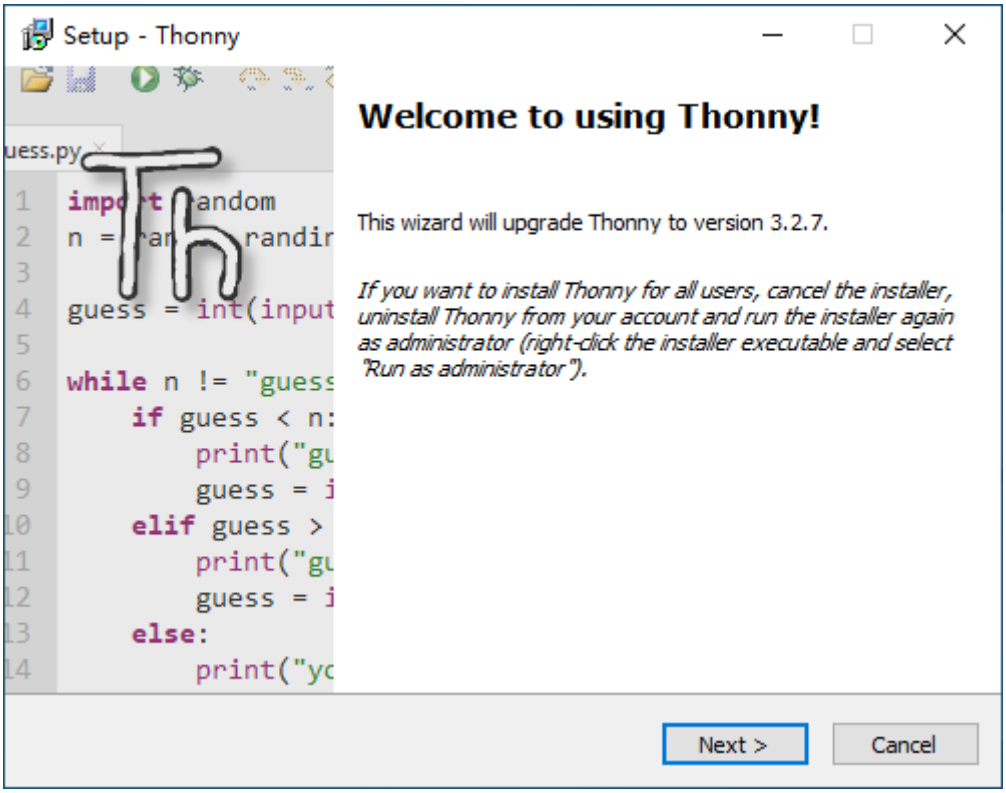


Installing on Windows

The icon of Thonny after downloading is as below. Double click “thonny-4.1.3.exe”.

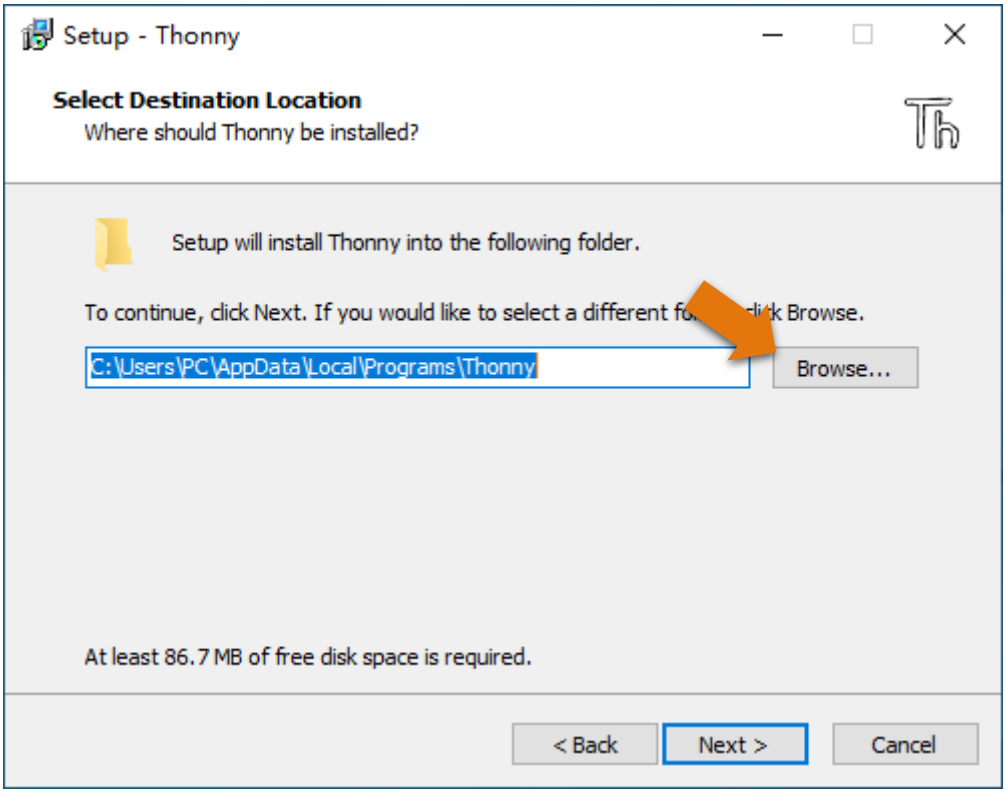


If you are not familiar with computer software installation, you can simply keep clicking “Next” until the installation completes.

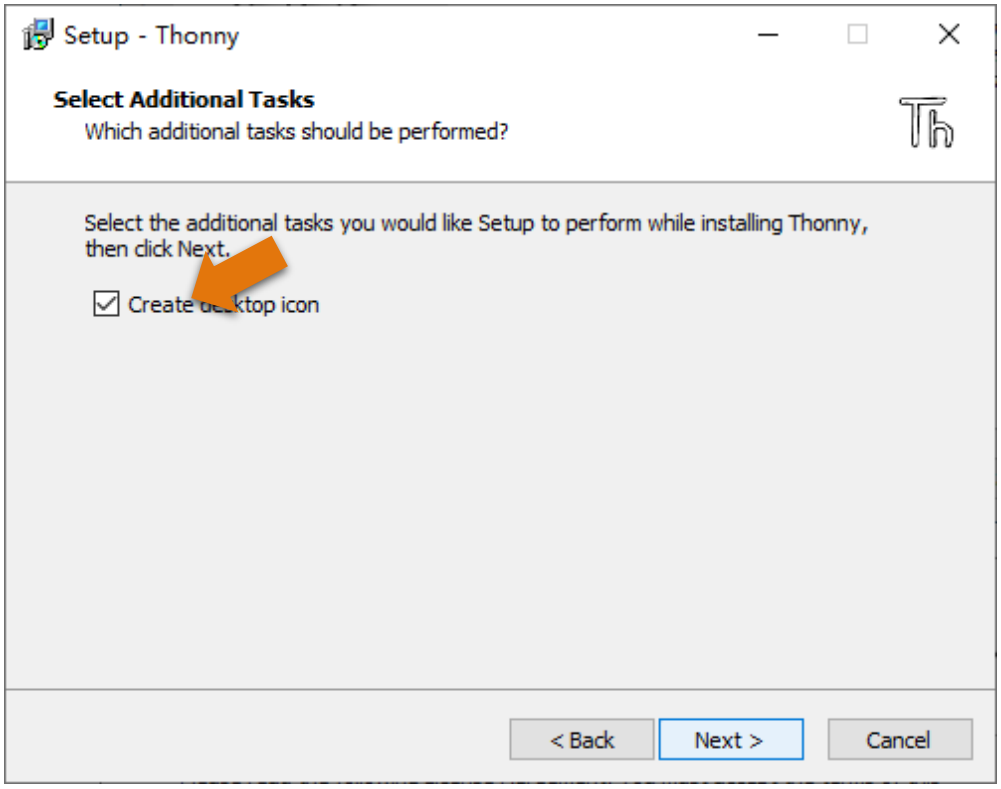


If you want to change Thonny’s installation path, you can click “Browse” to modify it. After selecting installation path, click “OK”.

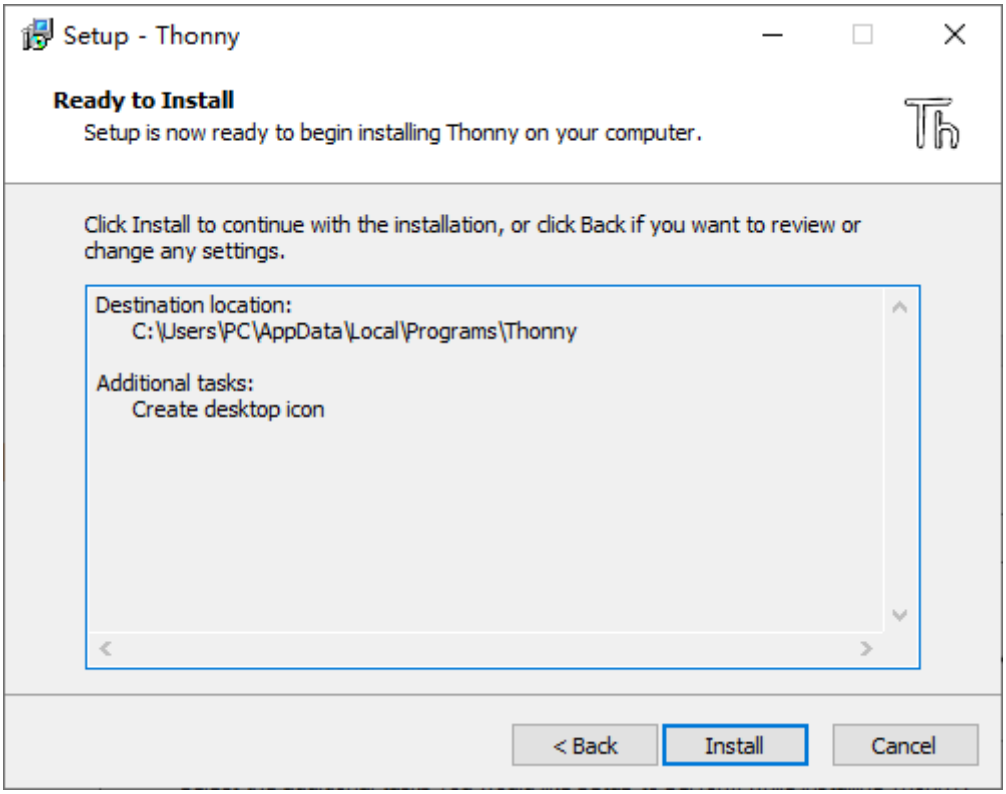
If you do not want to change it, just click “Next”.



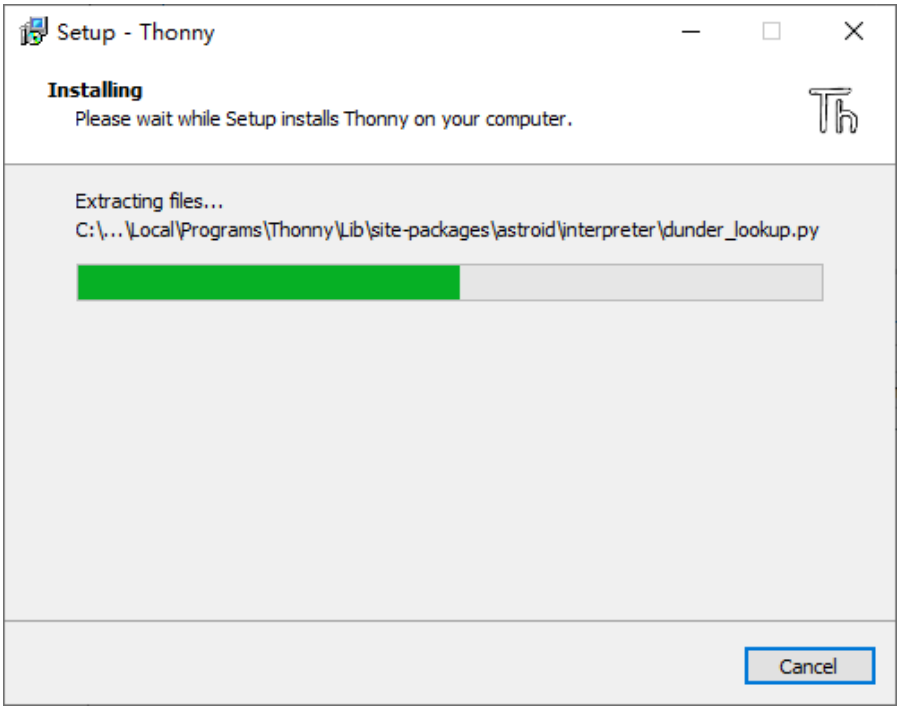
Check "Create desktop icon" and then it will generate a shortcut on your desktop to facilitate you to open Thonny later.



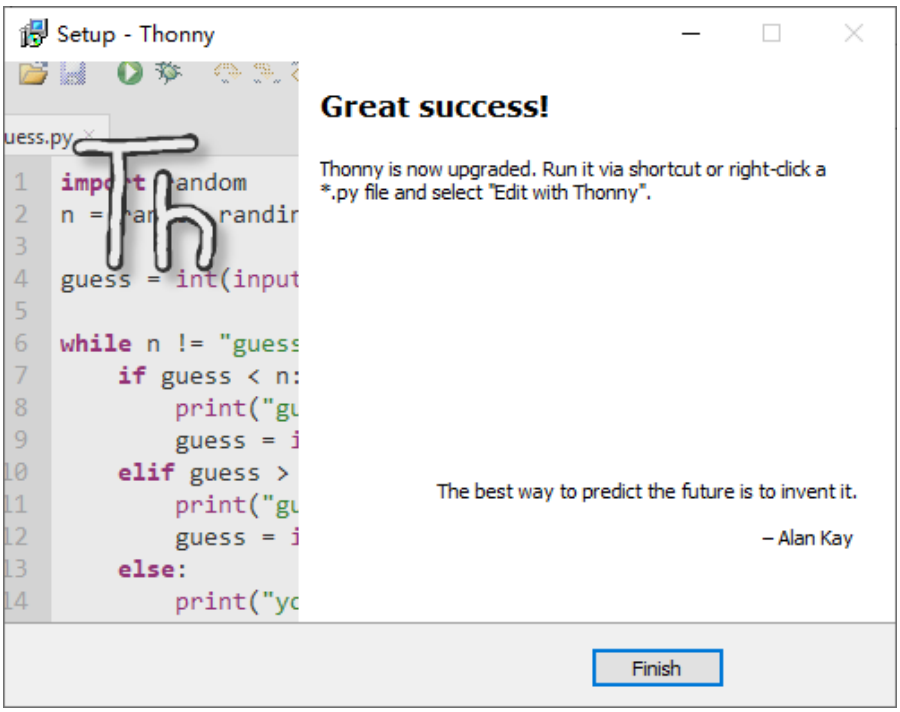
Click "install" to install the software.



During the installation process, you only need to wait for the installation to complete, and you msut not click "Cancel", otherwise Thonny will fail to be installed.



Once you see the interface as below, Thonny has been installed successfully.

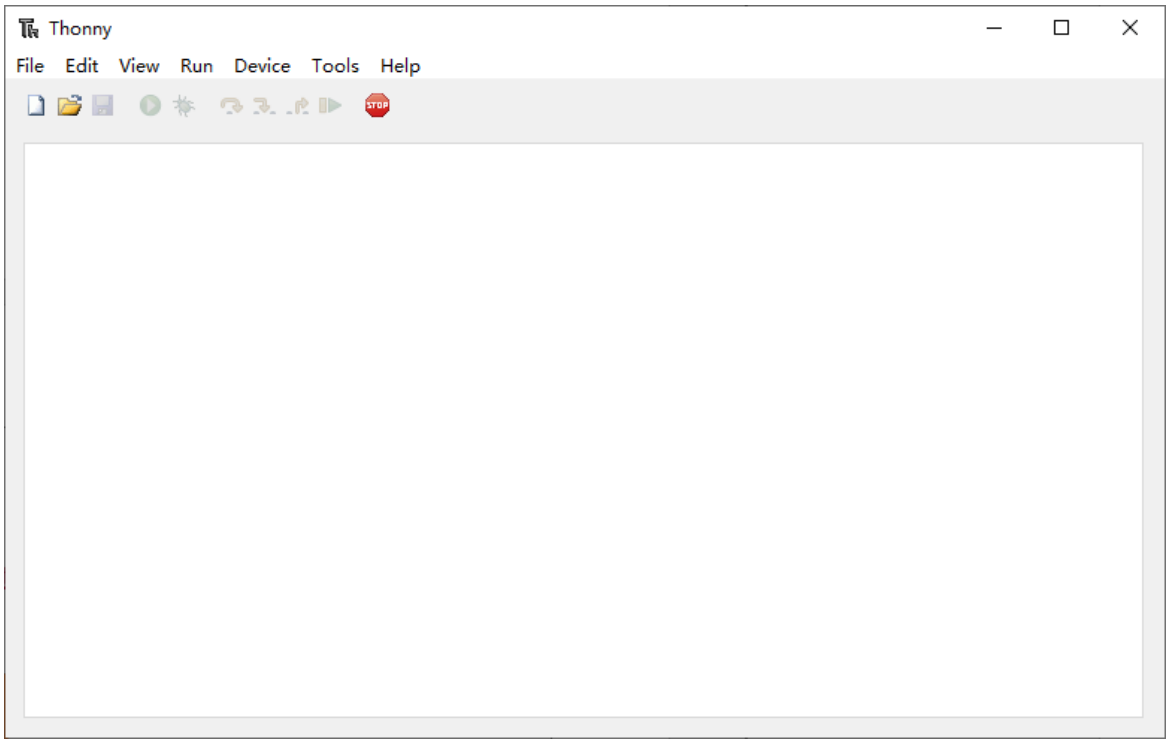


If you've check "Create desktop icon" during the installation process, you can see the below icon on your desktop.

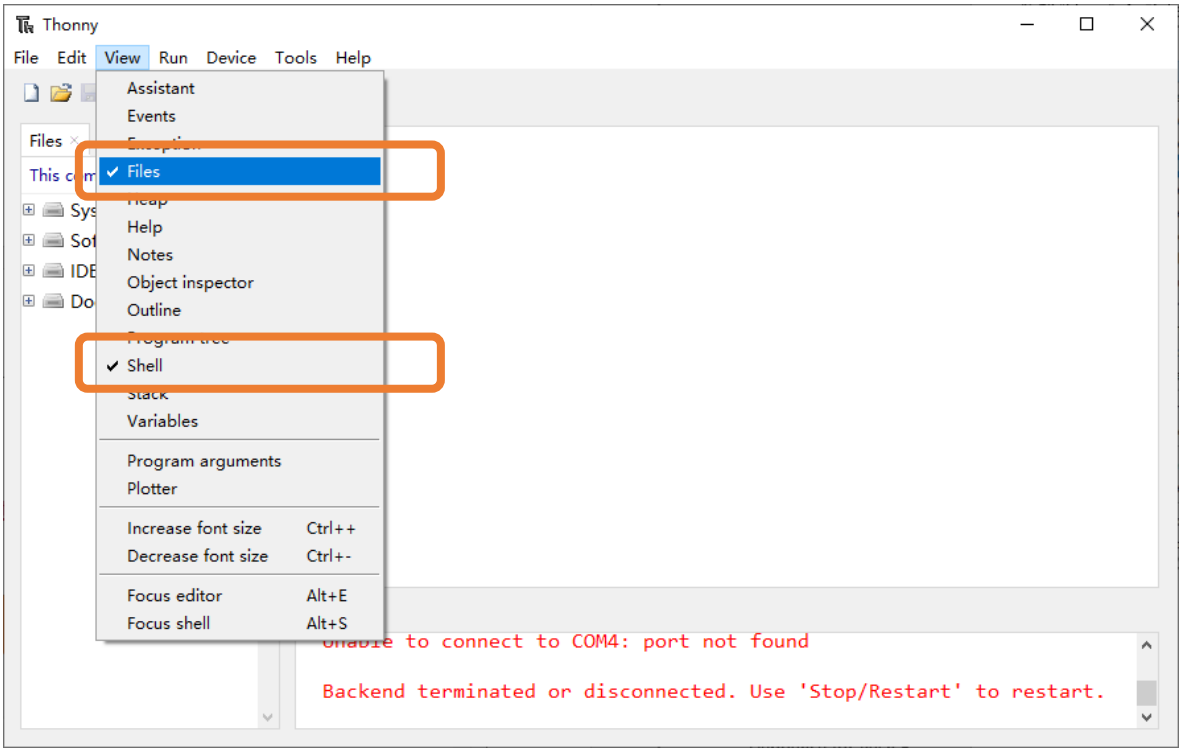


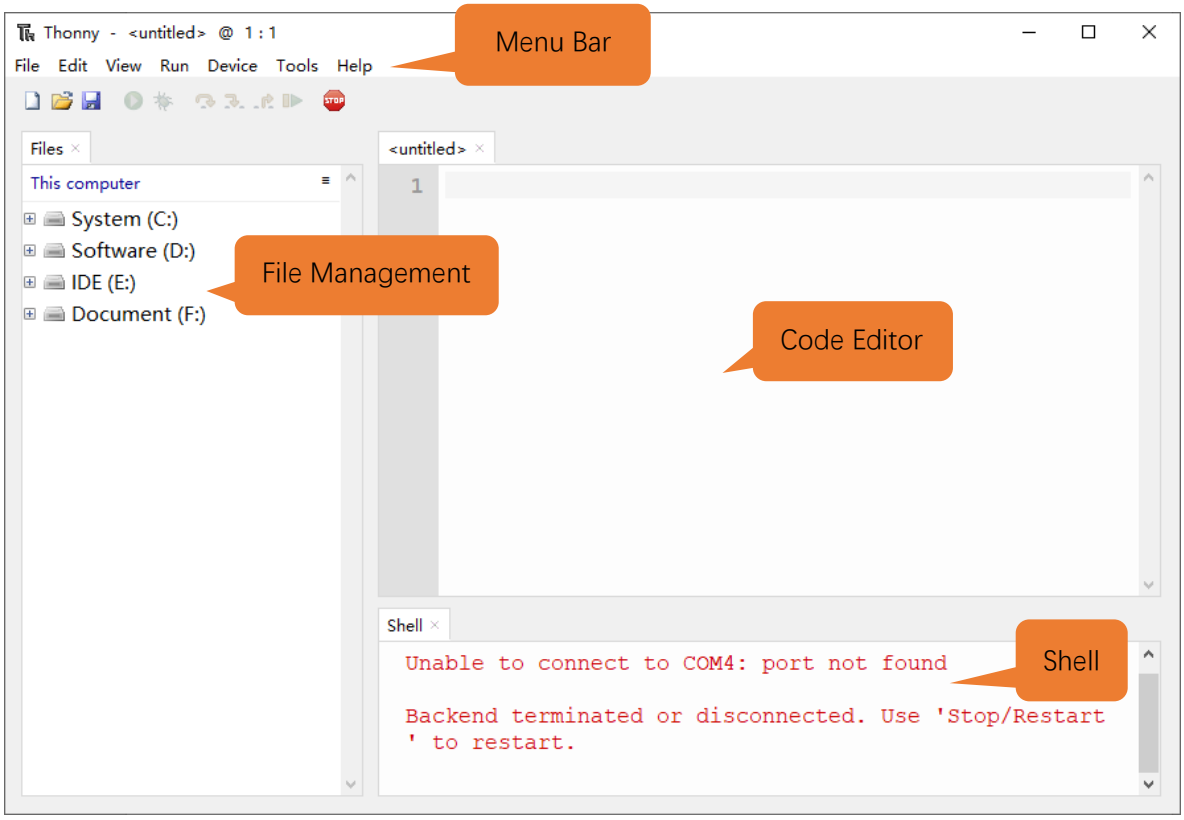
0.2 Basic Configuration of Thonny

Click the desktop icon of Thonny and you can see the interface of it as follows:



Select "View"→ "Files" and "Shell".




Thonny - <untitled> @ 1 : 1
File Edit View Run Device Tools Help
Menu Bar
Files
This computer
System (C:)
Software (D:)
IDE (E:)
Document (F:)
File Management
<untitled>
1
Code Editor
Shell
Unable to connect to COM4: port not found
Backend terminated or disconnected. Use 'Stop/Restart ' to restart.
Shell

0.3 Burning Micropython Firmware (Important)

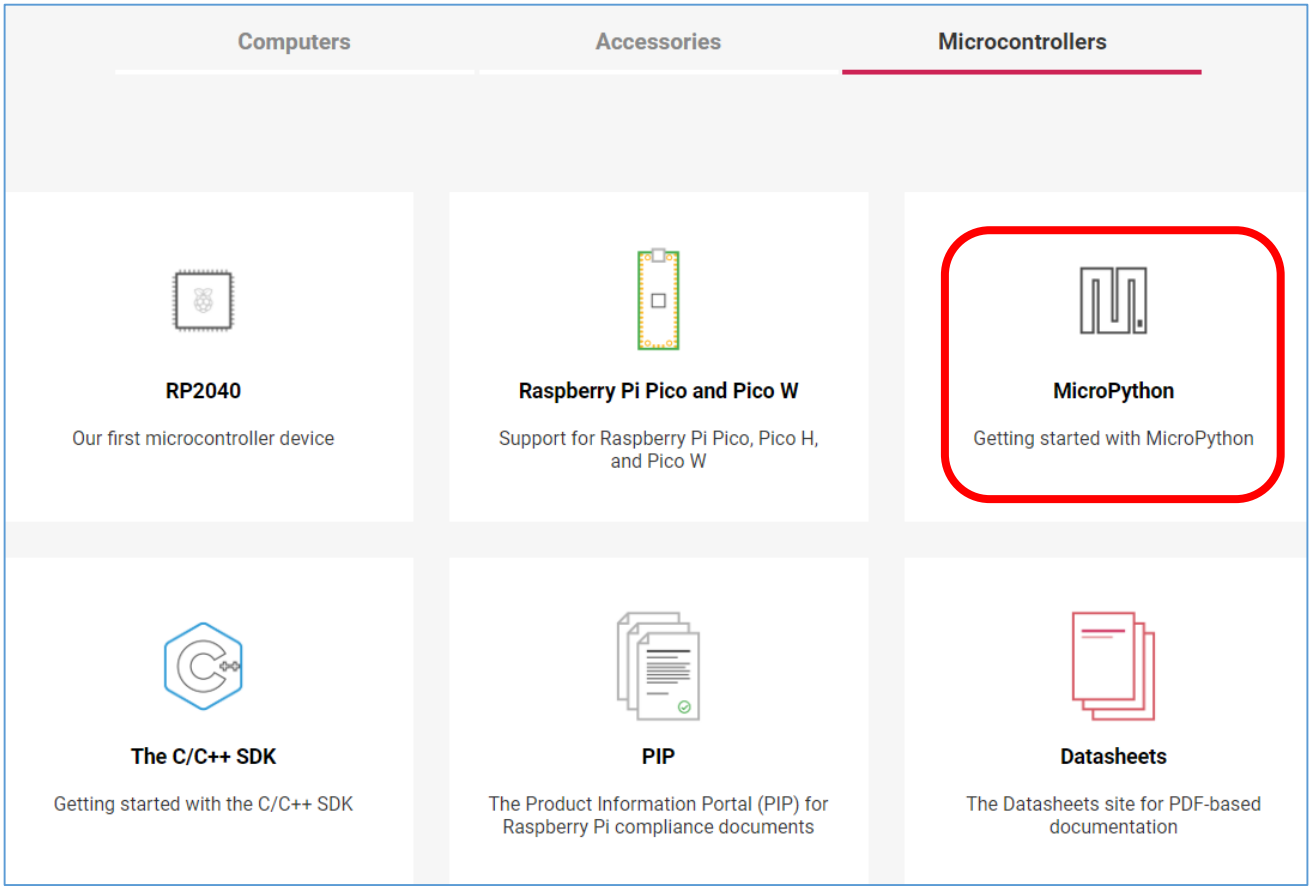
To run Python programs on Raspberry Pi Pico, we need to burn a firmware to Raspberry Pi Pico first.

Downloading Micropython Firmware

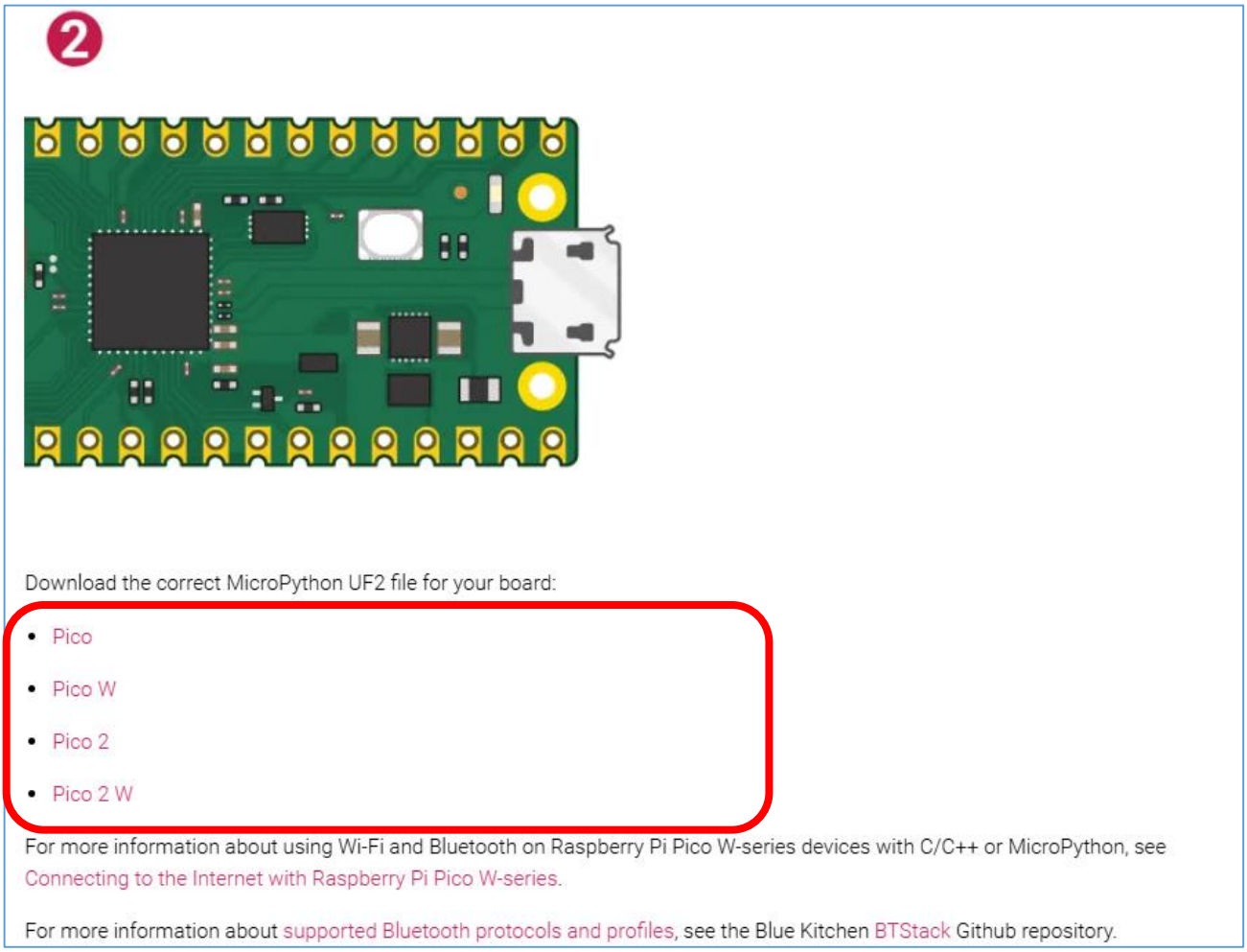
Option 1

Raspberry Pi Pico official website: https://www.raspberrypi.com/documentation/microcontrollers/

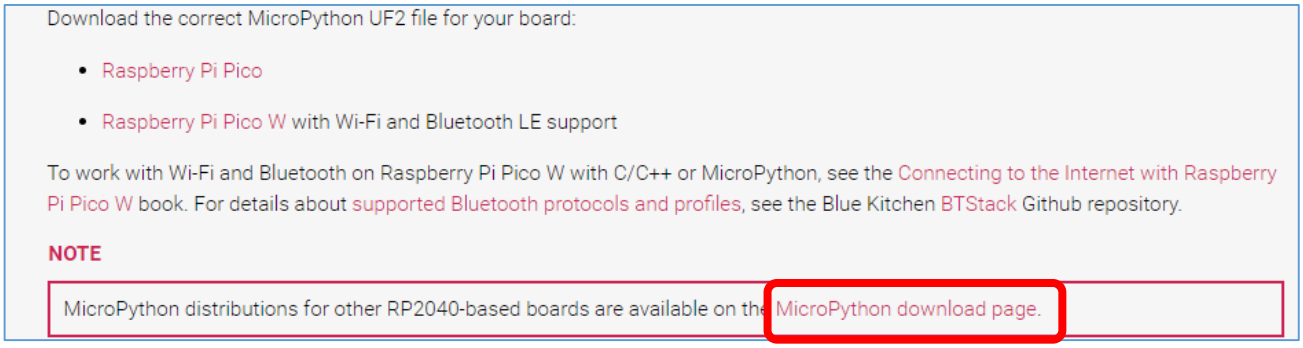
1. Click Micropython.

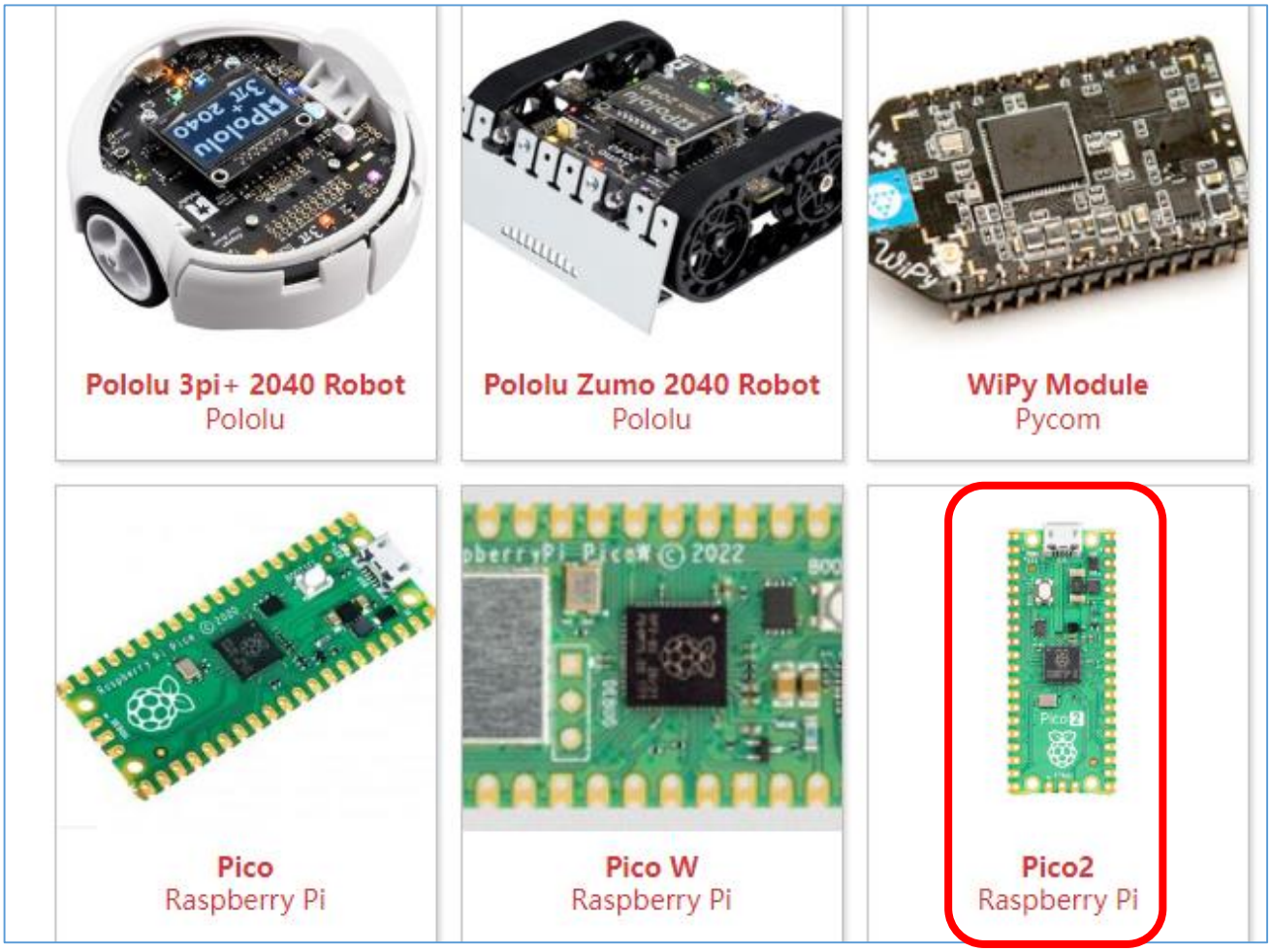


2. In the new interface, find the following content and download the corresponding UF2 according to your Pico version.



If your board is a standard Pico board, please click Raspberry Pi Pico to download it.
If it is a Pico W board, click Raspberry Pi Pico W.
If it is a Pico 2 board, click Raspberry Pi Pico 2.
If it is a Pico 2 W board, click Raspberry Pi Pico 2 W.
If it is Pico 2, click the MicroPython download page, or click the following link:
https://micropython.org/download/RPI_PICO2/







As Raspberry Pi Foundation has specially added two Cortex-M33 and two RISC-V cores in RP2350, both of the above two uf2 files can work.

Option 2

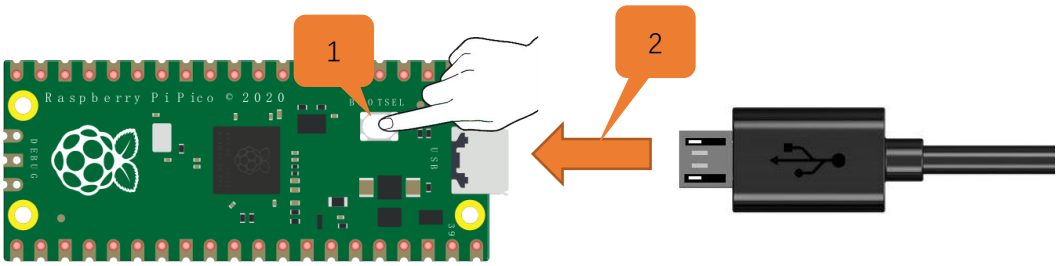
If you cannot download it due to network issue or other reasons, you can use the one we have prepared, which locates at the following file path:

Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Firmware

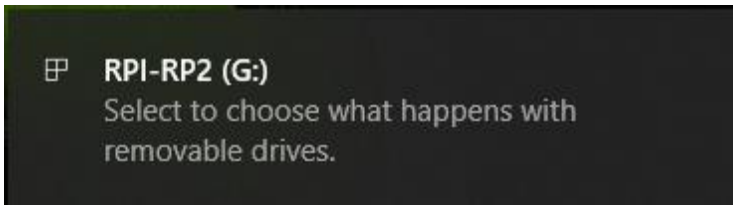
Burning a Micropython Firmware

Note: Pico, Pico W, Pico 2 and Pico 2W burn firmware in the same way. Pico's map is used here as an introduction.

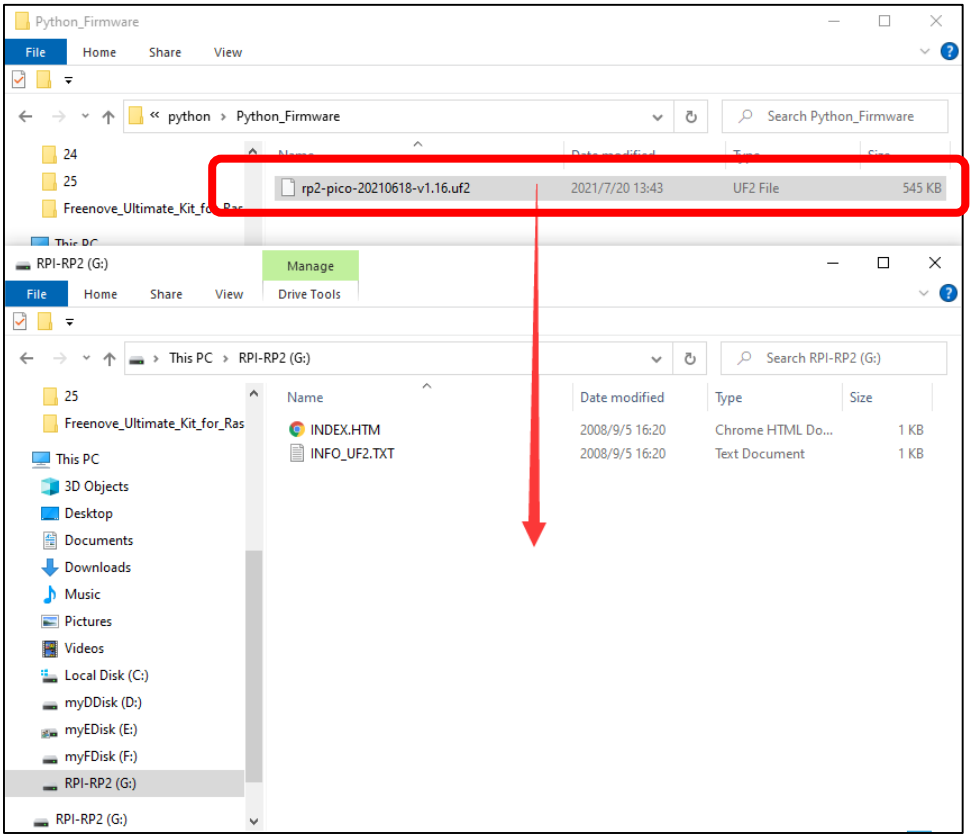
1. Connect a USB cable to your computer.
2. Long press BOOTSEL button on Raspberry Pi Pico and connect it to your computer with the USB cable.



3. When the connection succeeds, the following information will pop up on your computer.



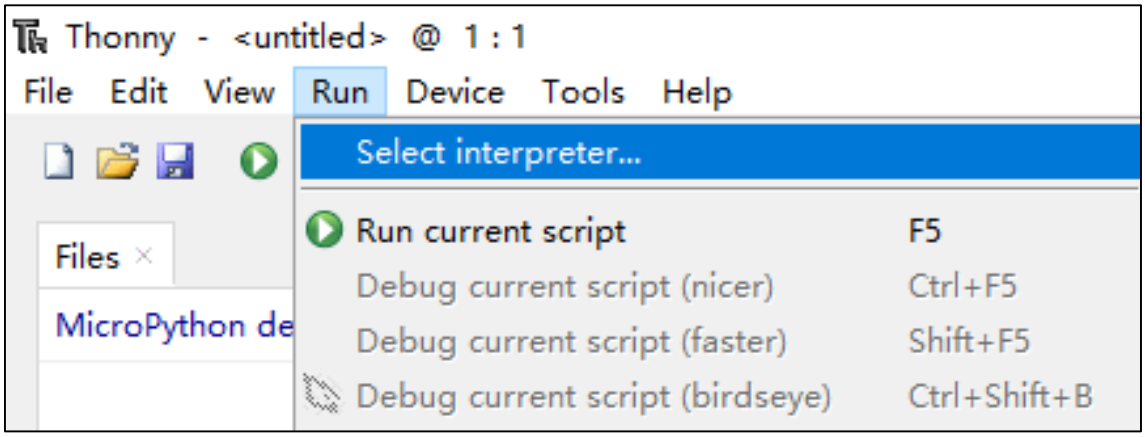
4. Copy the file(**rp2-pico-20210618-v1.16.uf2**) to RPI-RP2 and wait for it to finish, just like copy file to a U disk.



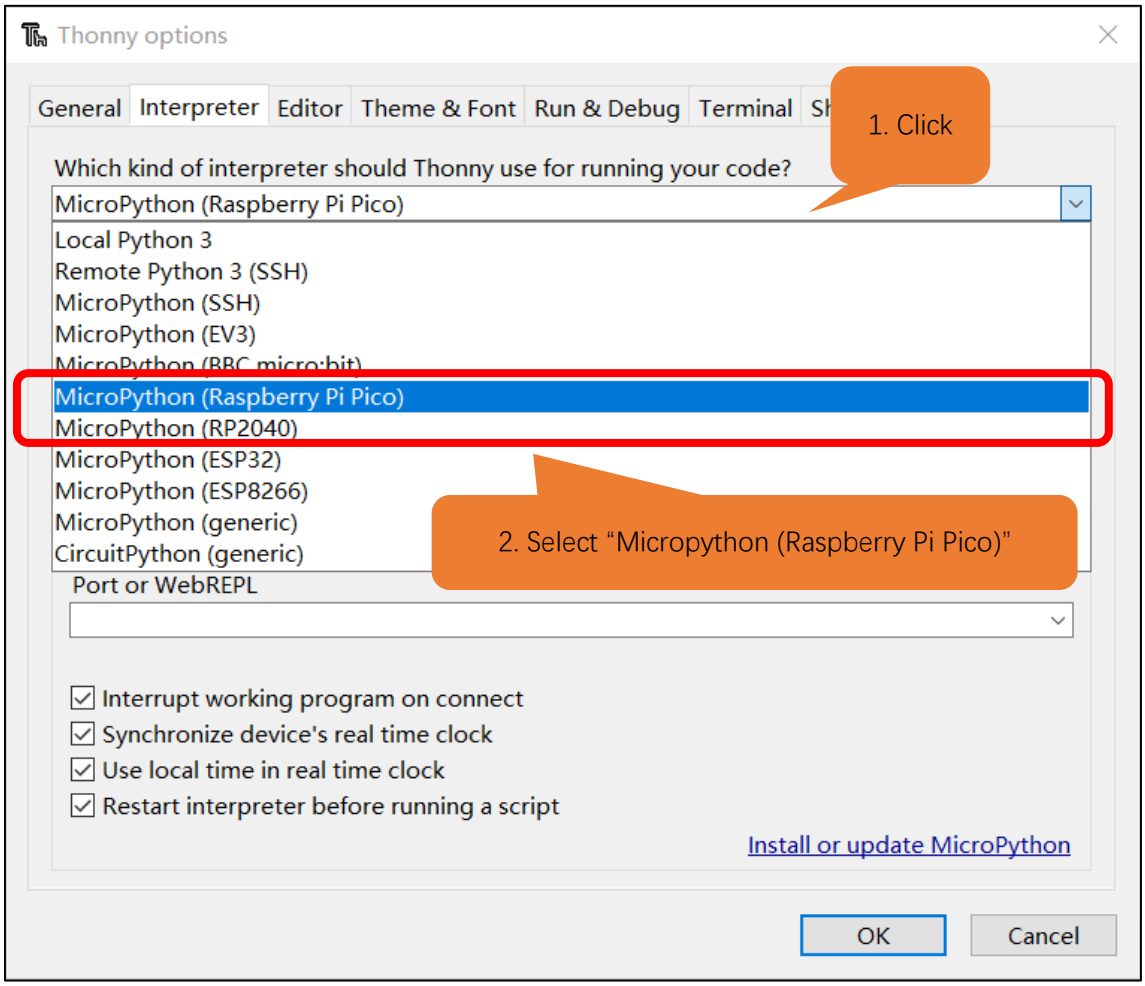
5. When the firmware finishes programming, Raspberry Pi Pico will reboot automatically. After that, you can run Micropython.

0.4 Thonny Connected to Raspberry Pi Pico

1. Open Thonny, click "run" and select "Select interpreter..."



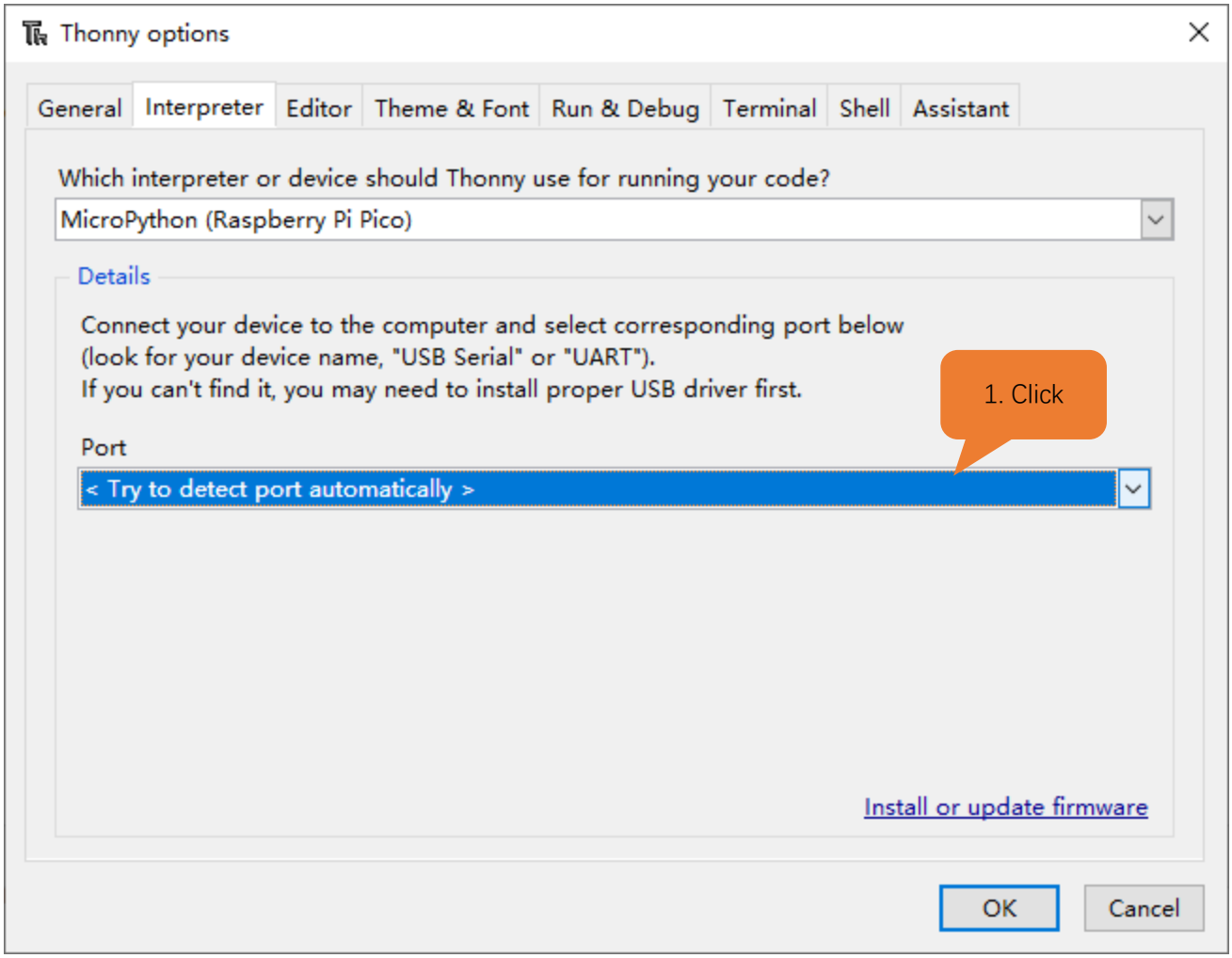
2. Select "Micropython (Raspberry Pi Pico)". If you are using Raspberry Pi Pico (W), select "Micropython (RP2040)";If Raspberry Pi Pico 2 (W) is used, select "Micropython (RP2350)".



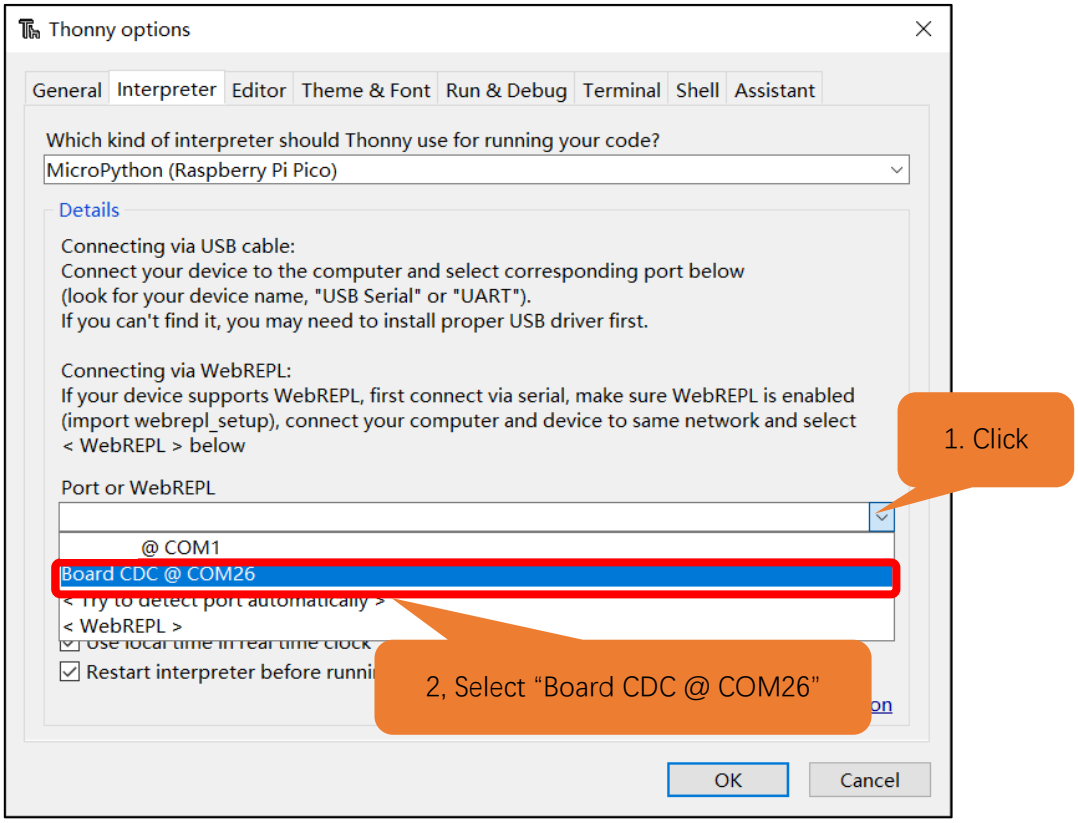
3. Select "Board CDC @ COMx", The number x of COMx may vary among different computers. You only need to make sure selecting Board CDC @ COMx

How to determine the port on which your Raspberry Pi Pico communicates with your computer?

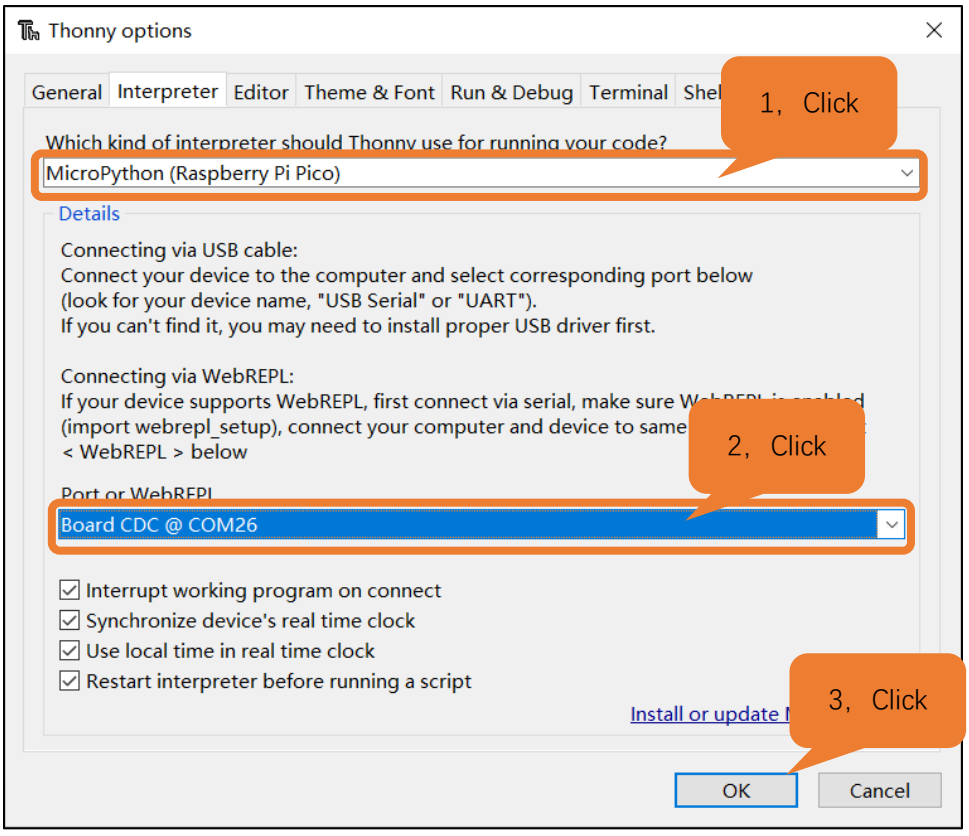
Step 1: When Pico **doesn't** connect to computer, open Thonny, click "Run", select "Select interpreter" and then a dialog box will pop up, click "Port" and you can check the ports currently connected to your computer, as shown below:



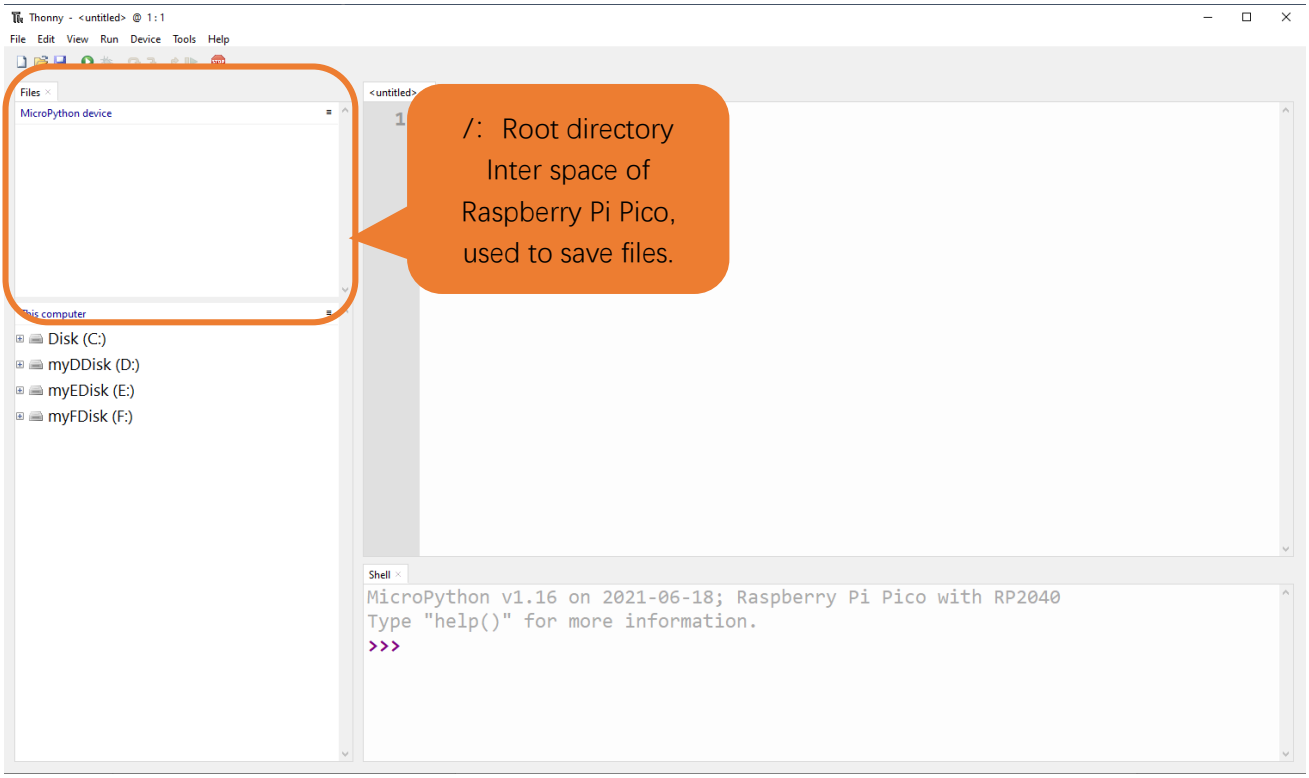
Step 2: Close the dialog box. Connect Pico to your computer, click "Run" again and select "Select interpreter". Click "Port" on the pop-up window and check the current ports. Now there is a newly added port, with which Pico communicates with the computer



4. After selecting "Micropython (Raspberry Pi Pico)" and port, click "OK"



5. When the following message displays on Thonny, it indicates Thonny has successfully connected to Pico.



So far, all the preparations have been made.

0.5 Testing codes (Important)

Testing Shell Command

Enter “print(“hello world!”)” in “Shell” and press Enter.



Running Online

To run Raspberry Pi Pico online, you need to connect it to computer. Users can use Thonny to compile or debug programs.

Advantages:

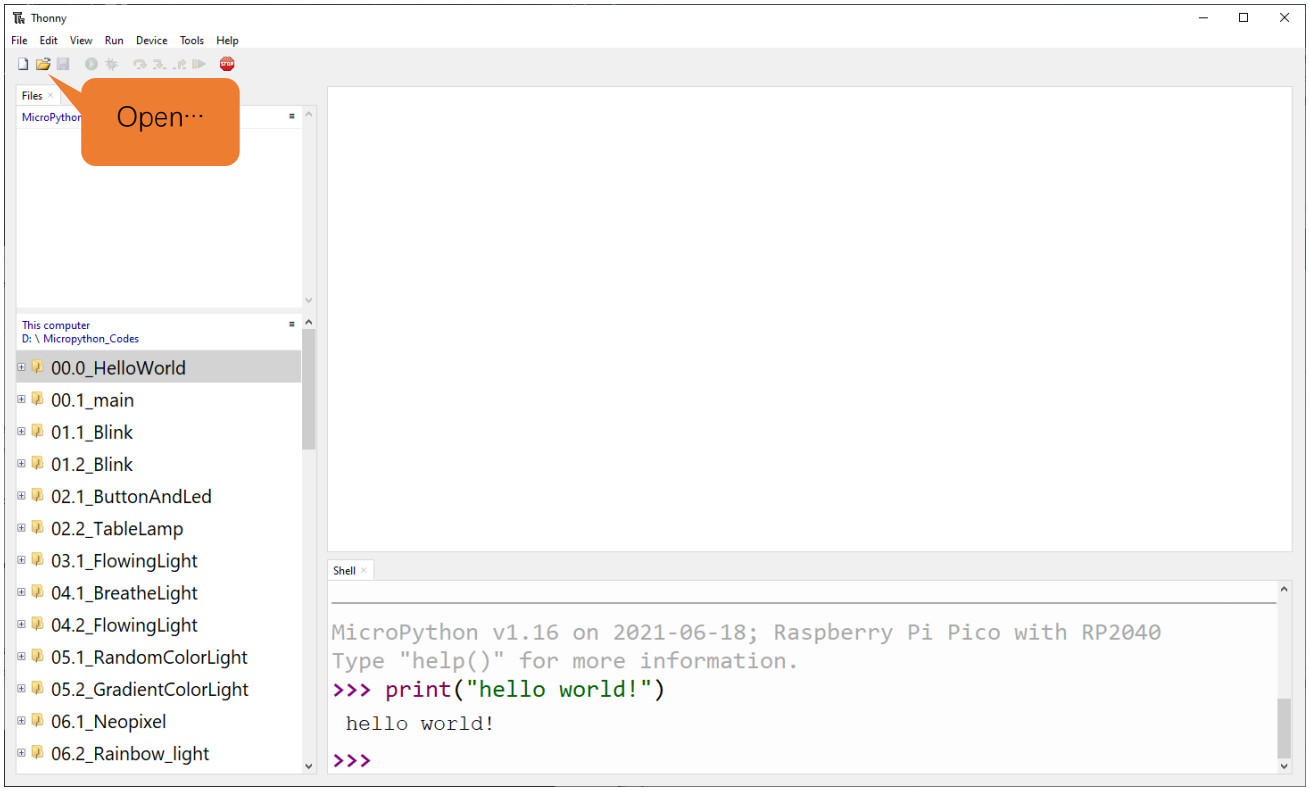
1. Users can use Thonny to compile or debug programs.
2. Through the "Shell" window, users can read the error information and output results generated during the running of the program and query related function information online to help improve the program.

Disadvantages:

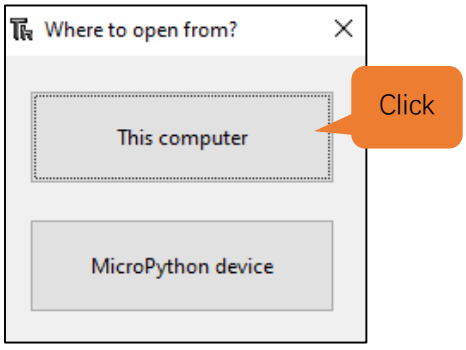
1. To run Raspberry Pi Pico online, you have to be connected to a computer and run with Thonny.
2. If Raspberry Pi Pico disconnects from computer, the program will not run again when they reconnect to each other.

Operation

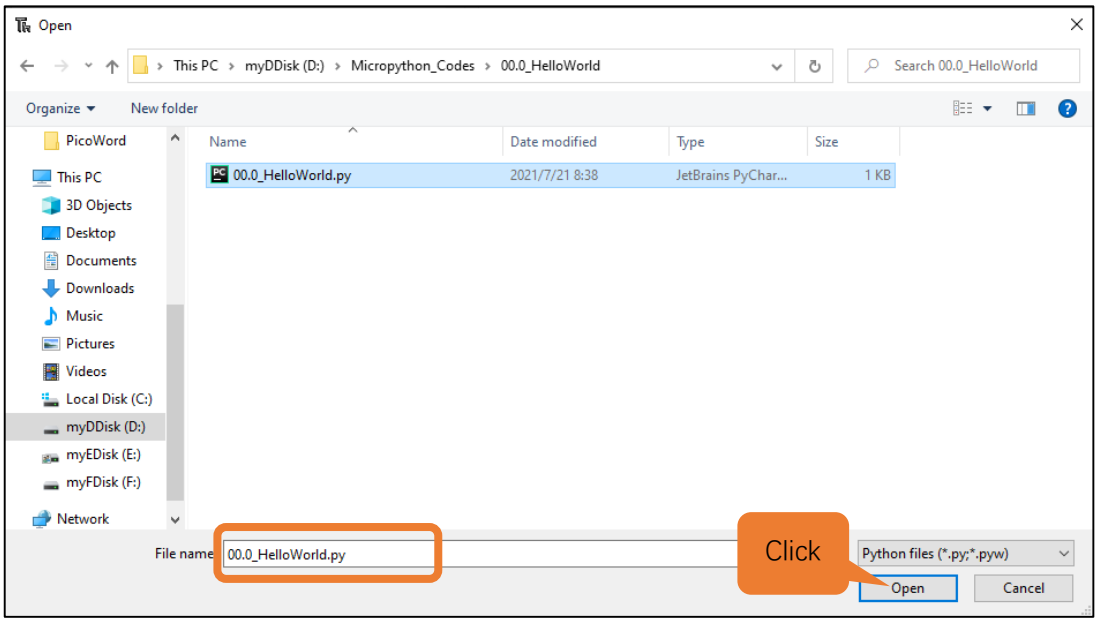
1. Open Thonny and click “Open···”.



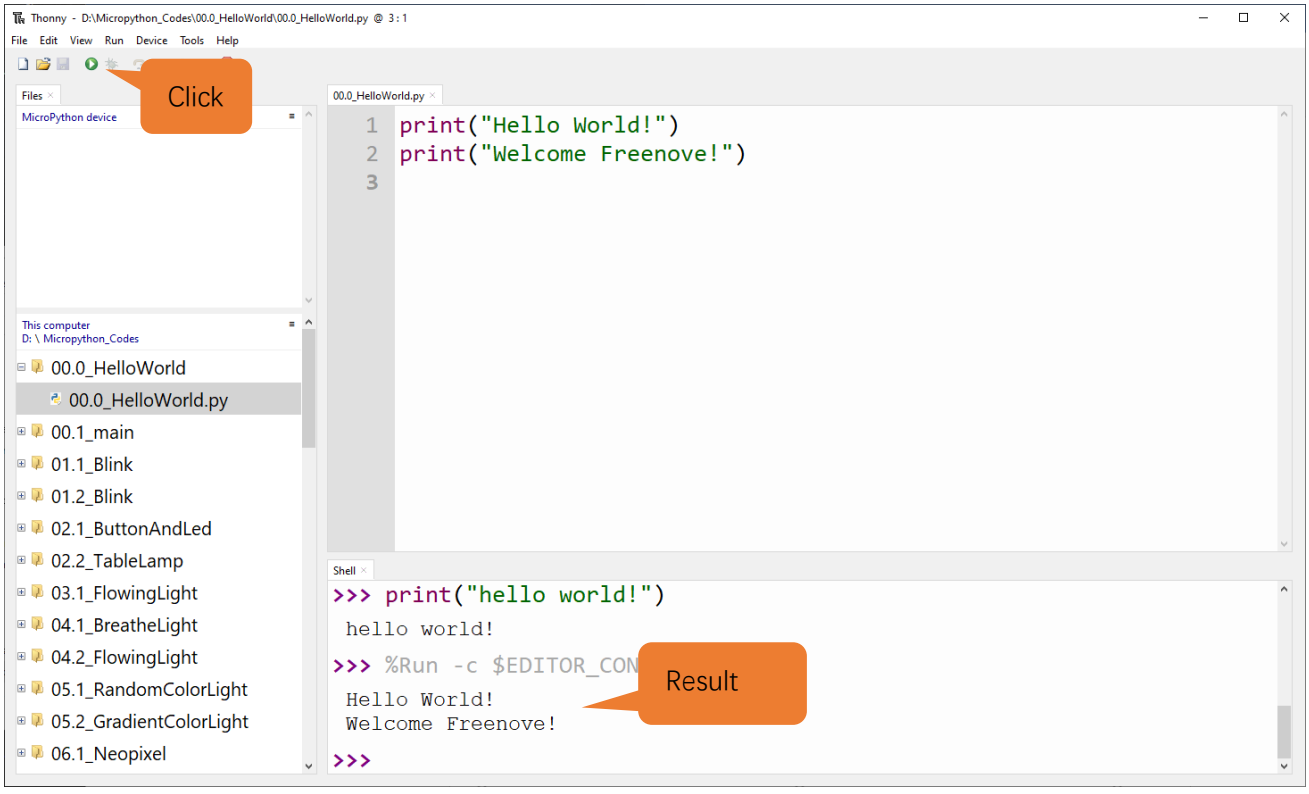
2. On the newly pop-up window, click “This computer”.



In the new dialog box, select "**00.0_HelloWorld.py**" in "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/00.0_HelloWorld**" folder.

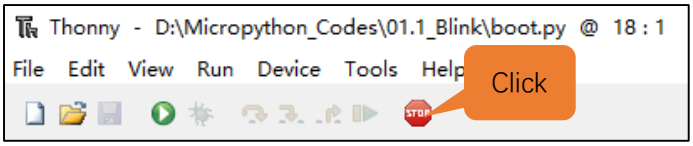


Click "Run current script" to execute the program and "Hello World!", "Welcome Freenove" will be printed in "Shell".



Exiting Running Online

When running online, click "Stop /Restart backend" on Thonny or press Ctrl+C to exit the program.

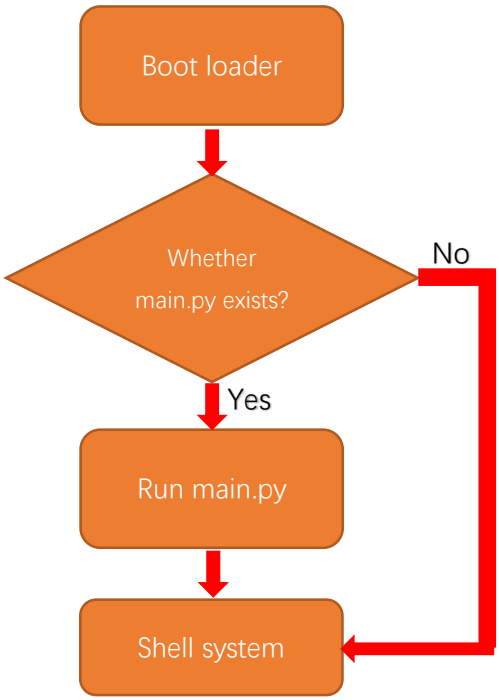


Running Offline

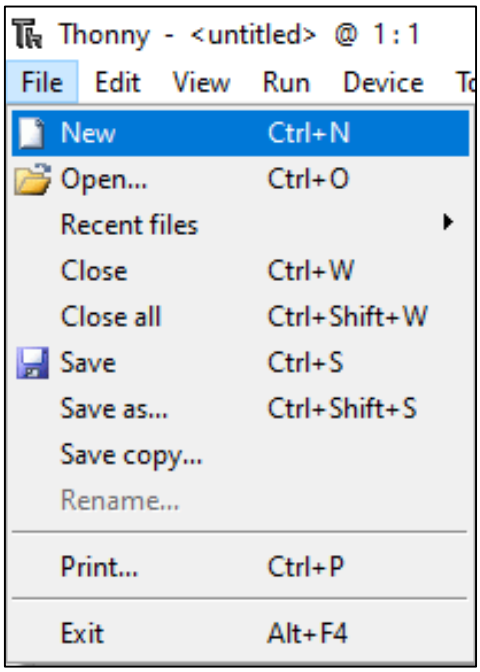
When running offline, Raspberry Pi Pico does not need to connect to computer and Thonny. It can run the programs stored in main.py on the device once powered up.
Advantage: It can run programs when powered up without connected to computer and Thonny.
Disadvantage: The program will stop automatically when error occurs or Raspberry Pi Pico is out of power. Code cannot be changed easily.

Operation

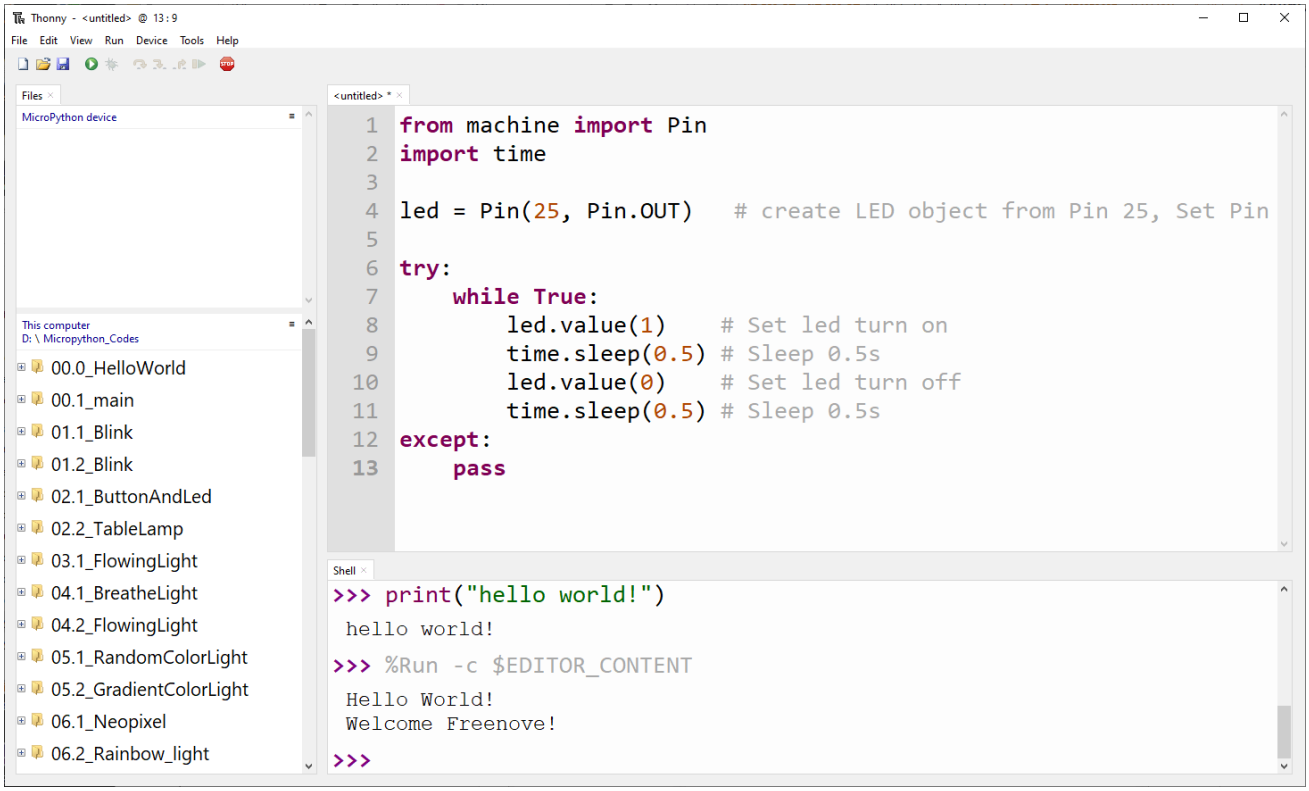
Once powered up, Raspberry Pi Pico will automatically check whether there is main.py existing on the device. If there is, it runs the programs in main.py and then enter shell command system. (If you want the code to run offline, you can save it as main.py); if main.py does not exist, it will enter shell command system directly.



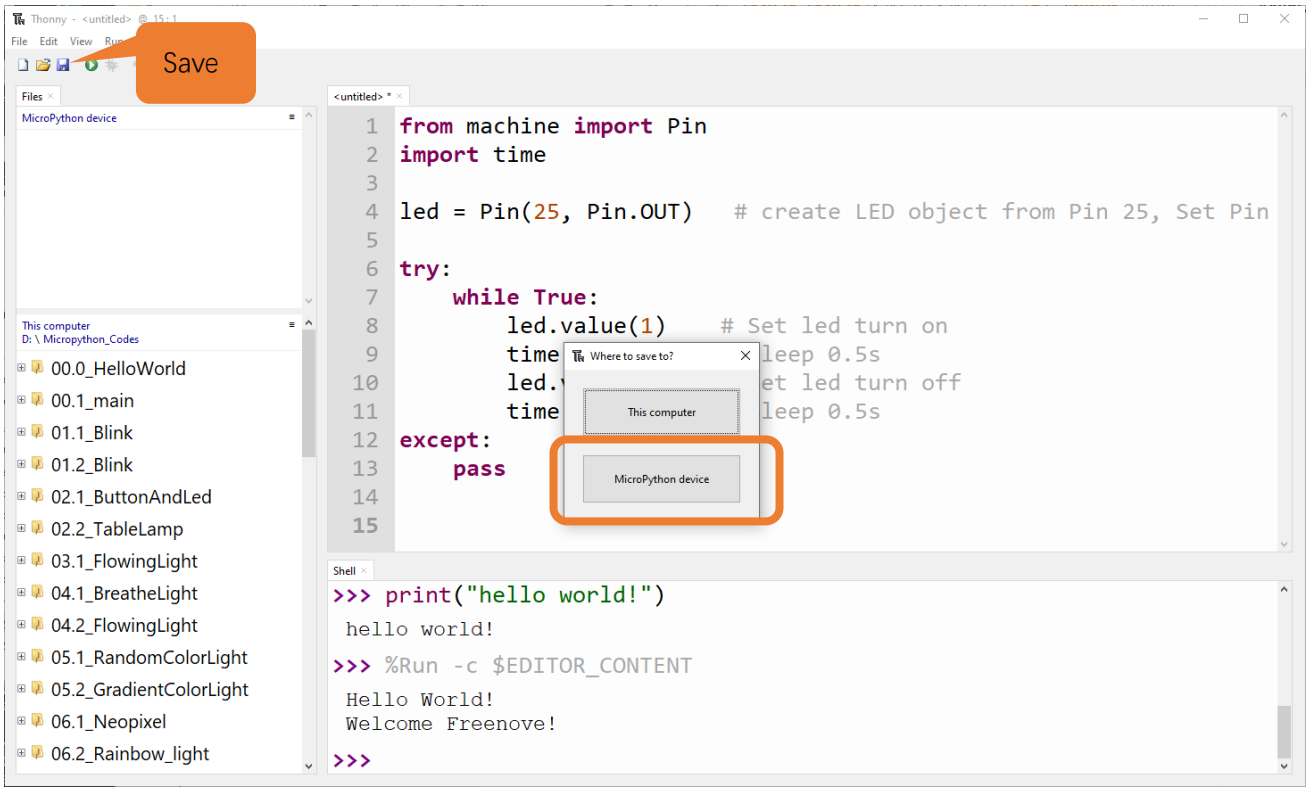
1. Click "File"→"New" to create and write codes.



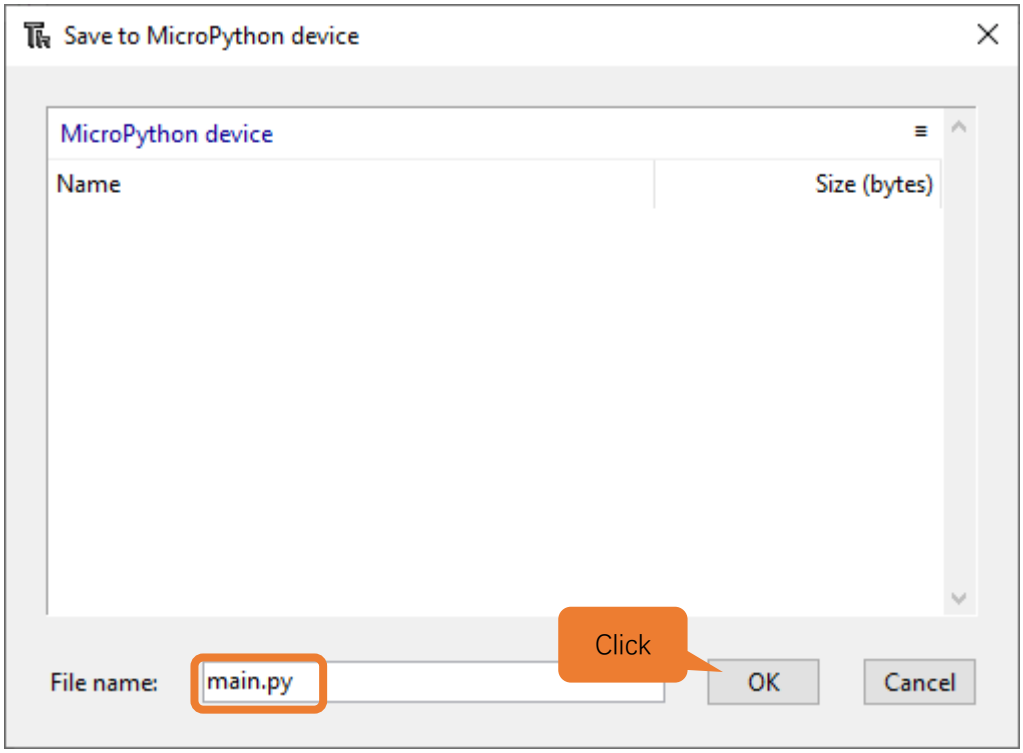
2. Enter codes in the newly opened file. Here we use codes of "01.1_Blink.py" as an example.



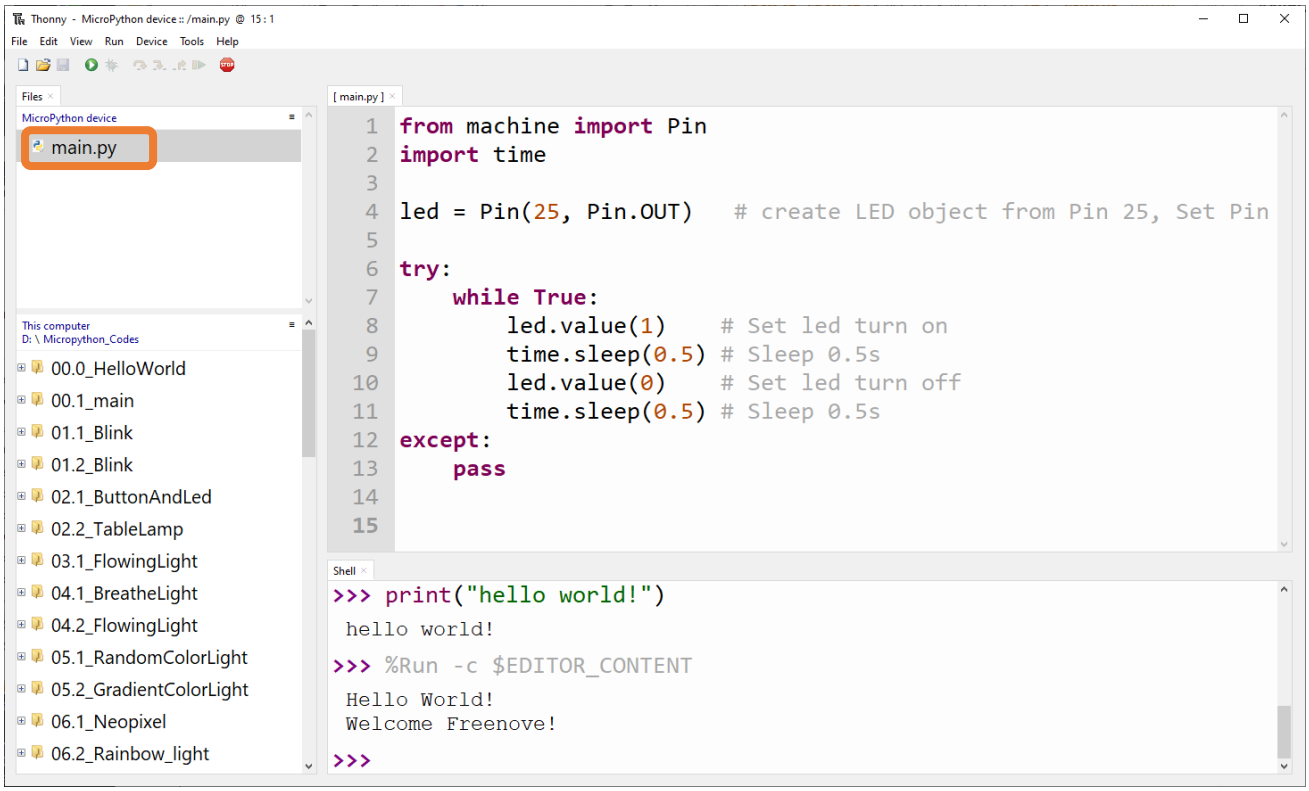
3. Click "Save" on the menu bar. You can save the codes either to your computer or to Raspberry Pi Pico.



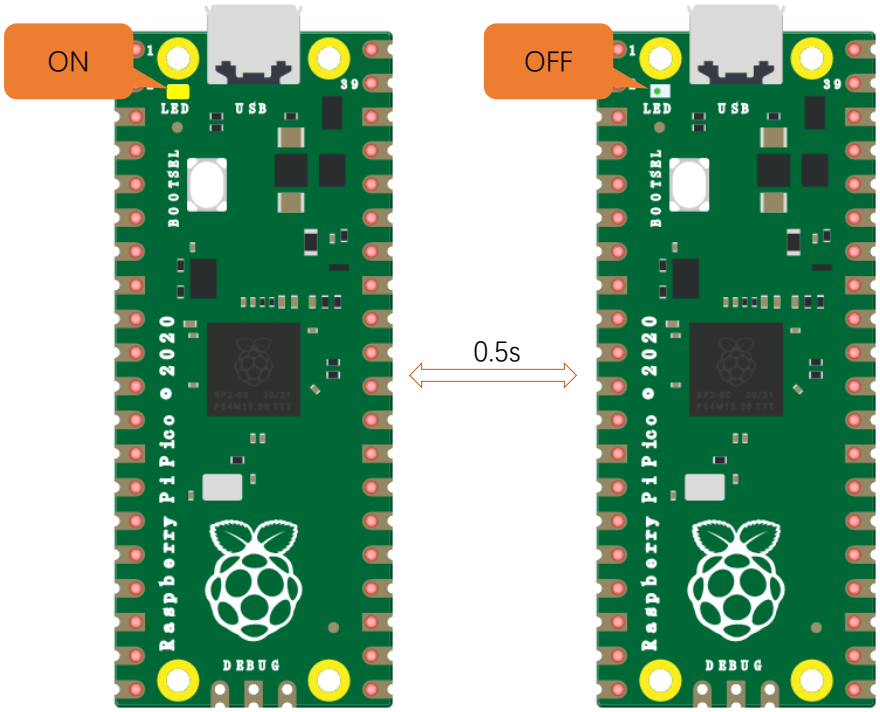
4. Select "MicroPython device", enter "main.py" in the newly pop-up window and click "OK".



5. You can see that codes have been uploaded to Raspberry Pi Pico.

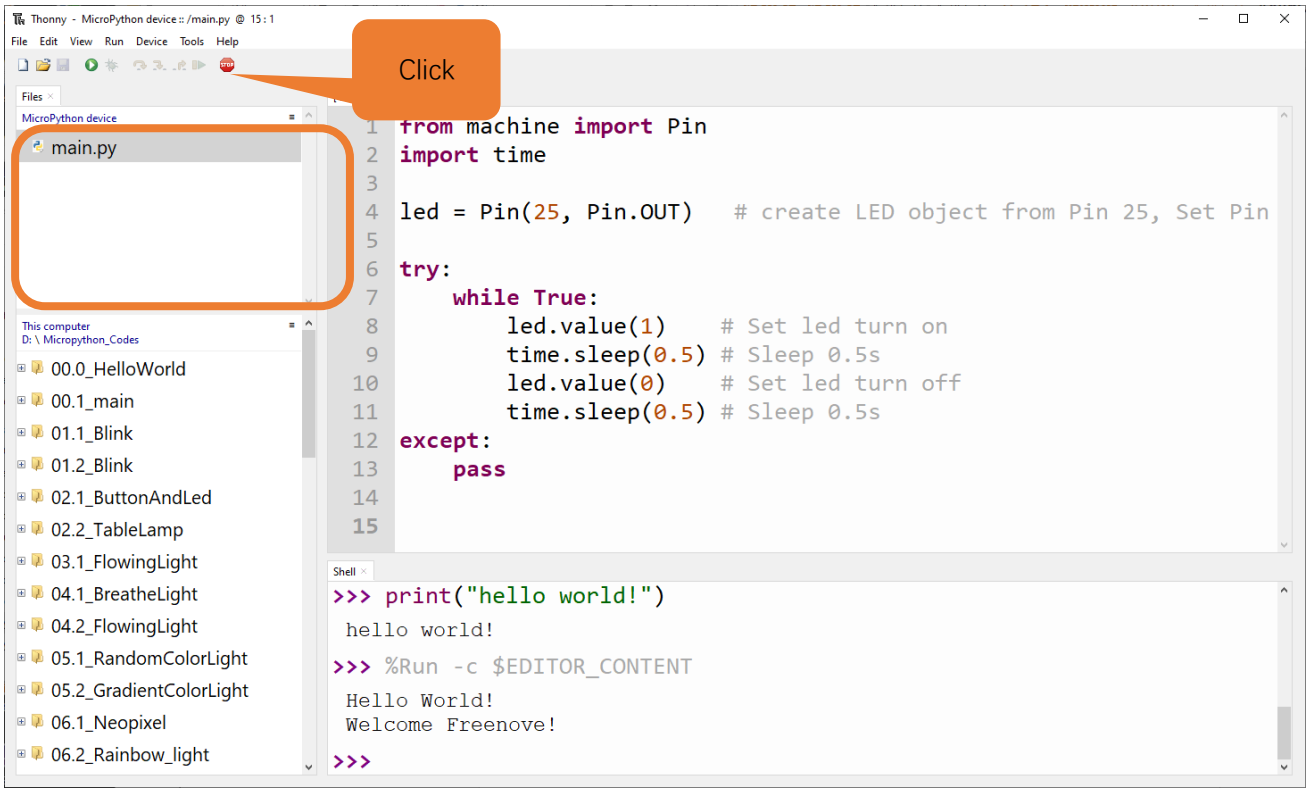


6. Disconnect Raspberry Pi Pico USB cable and then reconnect it, the LED on Raspberry Pi Pico will blink repeatedly.



Exiting Offline Running

Connect Raspberry Pi Pico to computer, click “stop/restart backend” on Thonny to end running offline.

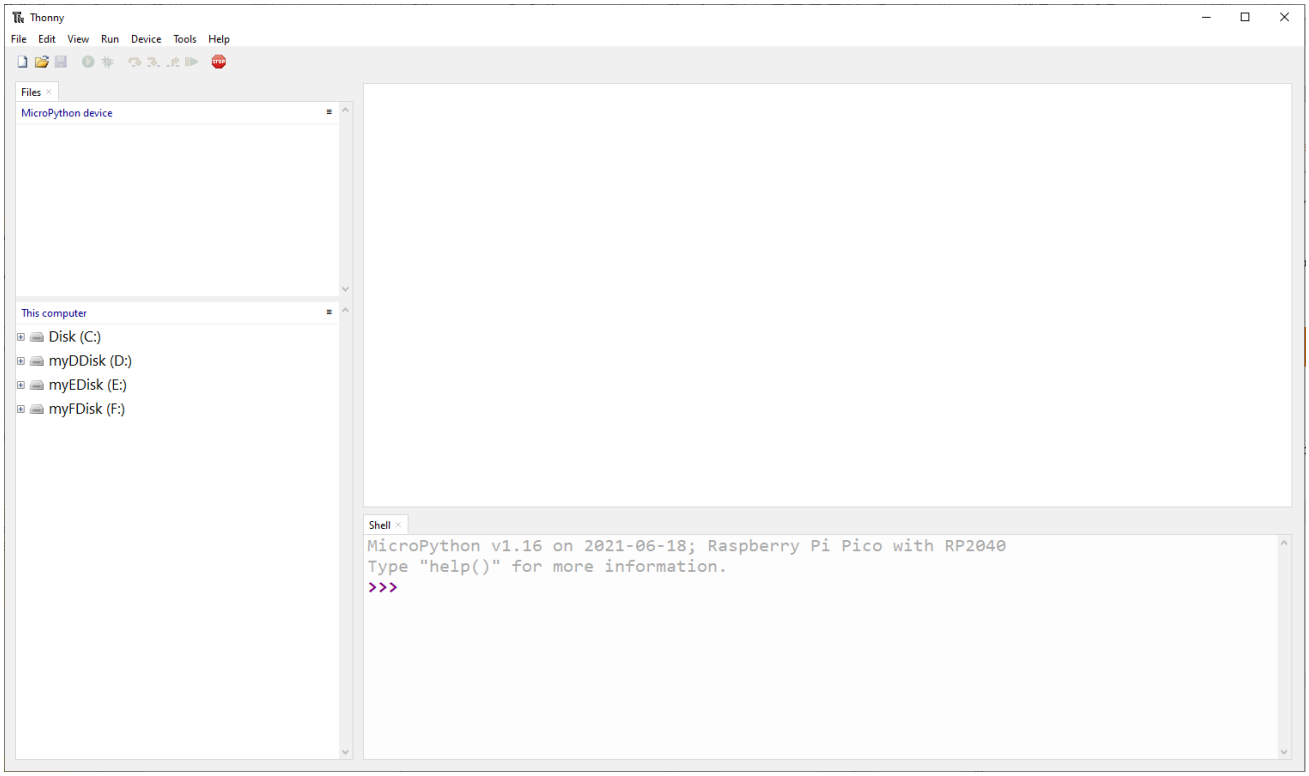


If it does not work, please click on “stop/restart backend” for more times or reconnect Pico.

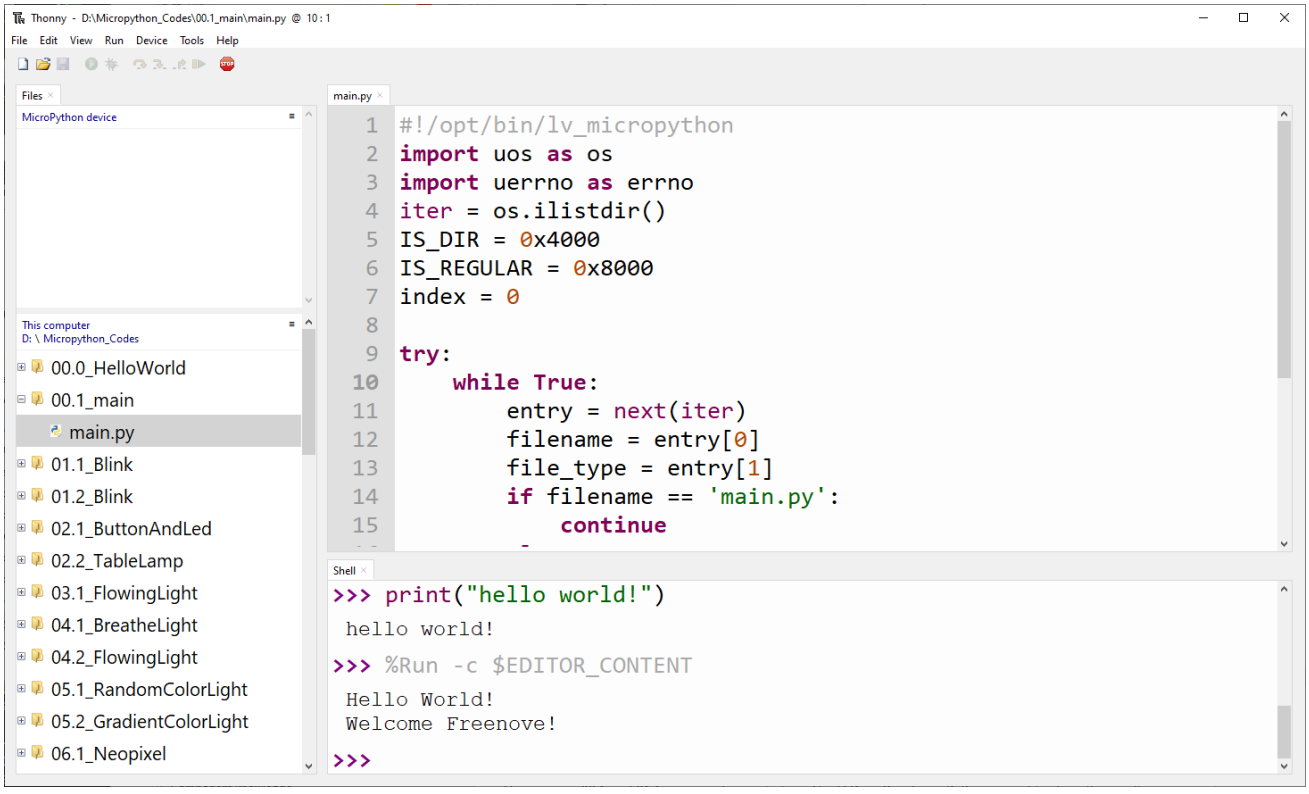


We provide a main.py file for running offline. The code added to main.py is a bootstrap that executes the user's code file. All you need to do is upload the offline project's code file (.py) to the Raspberry Pi Pico device.

1. Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**". Open "Thonny"。

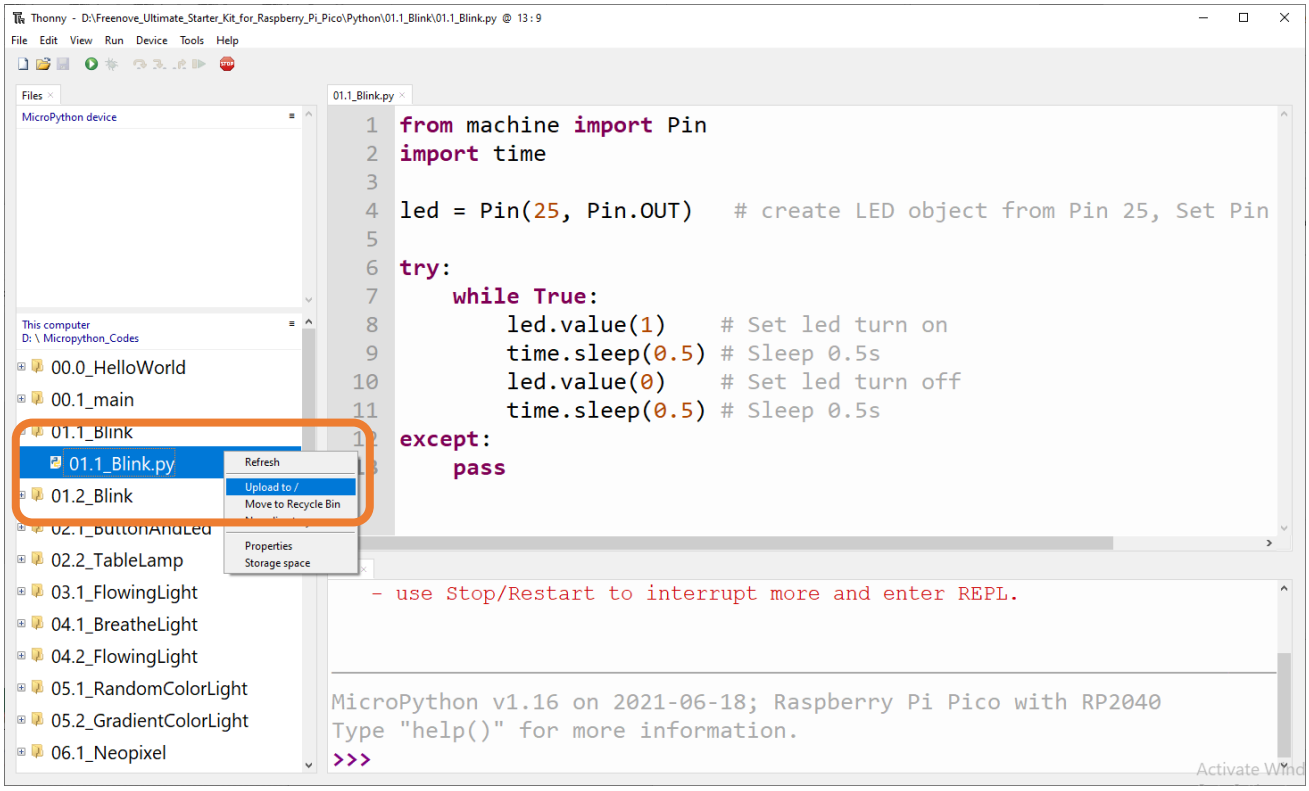


2. Expand "00.1_main" in the "Micropython_Codes" in the directory of disk(D), and double-click main.py, which is provided by us to enable programs in "MicroPython device" to run offline.

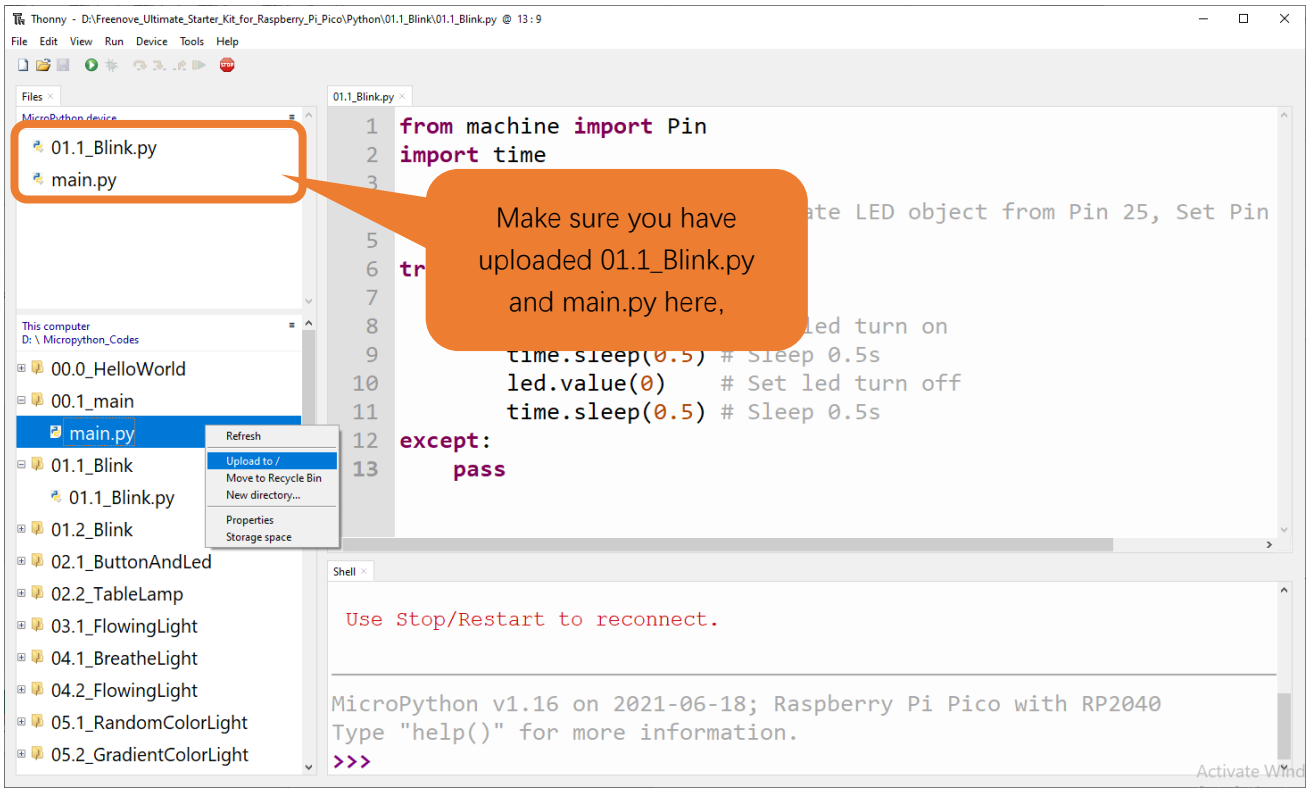


Here we use 00.1 and 01.1 cases as demonstration. The LED on Raspberry Pi Pico is used to show the result, which uses GP25 pin. If you have modified 01.1_Blick.py file, you need to change it accordingly.

As shown in the following illustration, right-click the file 01.1_Blink.py and select "Upload to /" to upload code to Raspberry Pi Pico.



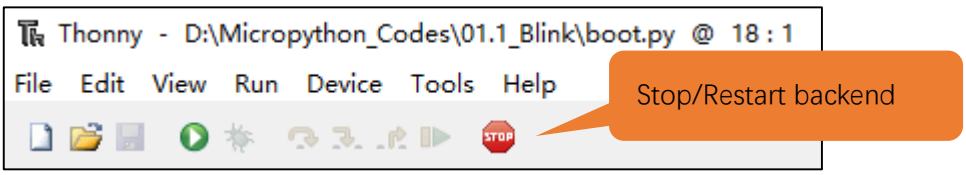
Upload main.py in the same way.



Disconnect Raspberry Pi Pico USB cable and reconnect it, the LED on pico will blink repeatedly.

Note:

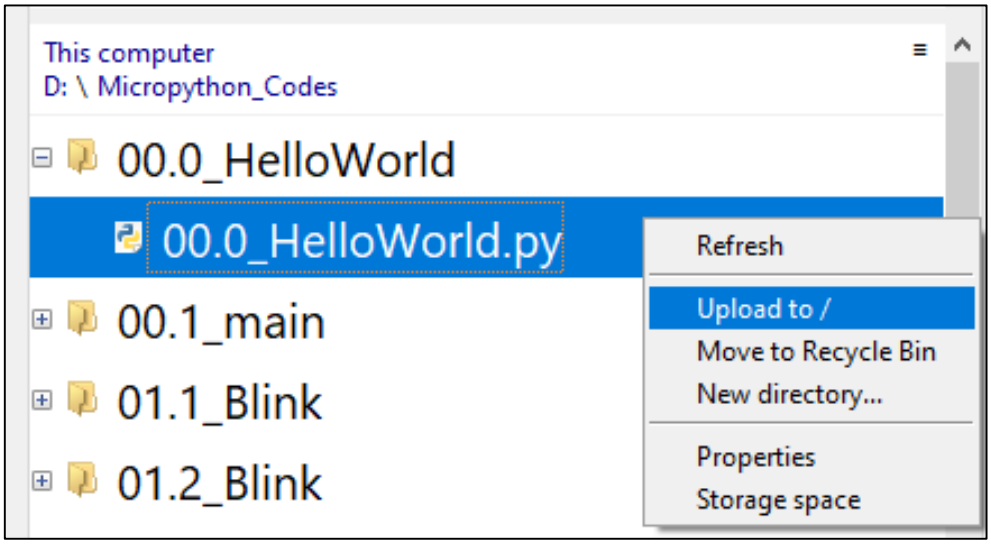
Codes here are run offline. If you want to stop running offline and enter Shell, just click "Stop" in Thonny.



0.6 Thonny Common Operation

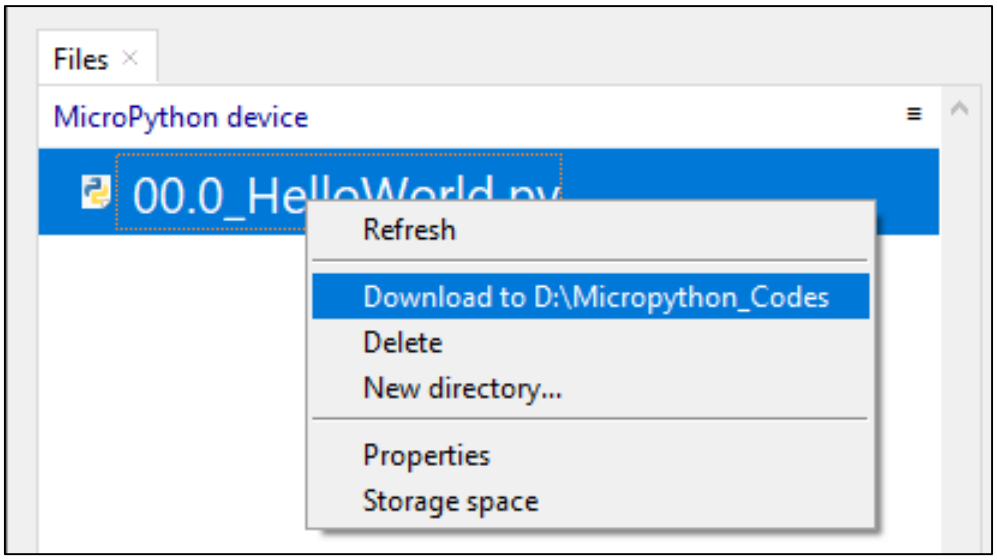
Uploading Code to Raspberry Pi Pico

Select "00.0_HelloWorld.py" in "00.0_HelloWorld", right-click your mouse and select "Upload to /" to upload code to Raspberry Pi Pico's root directory.



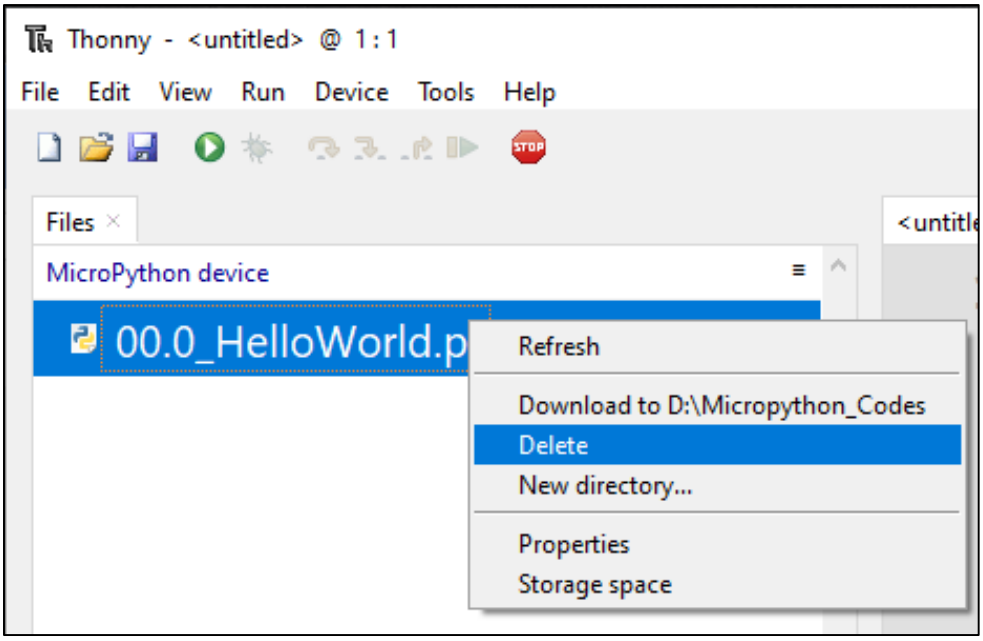
Downloading Code to Computer

Select "00.0_HelloWorld.py" in "MicroPython device", right-click to select "Download to ..." to download the code to your computer.



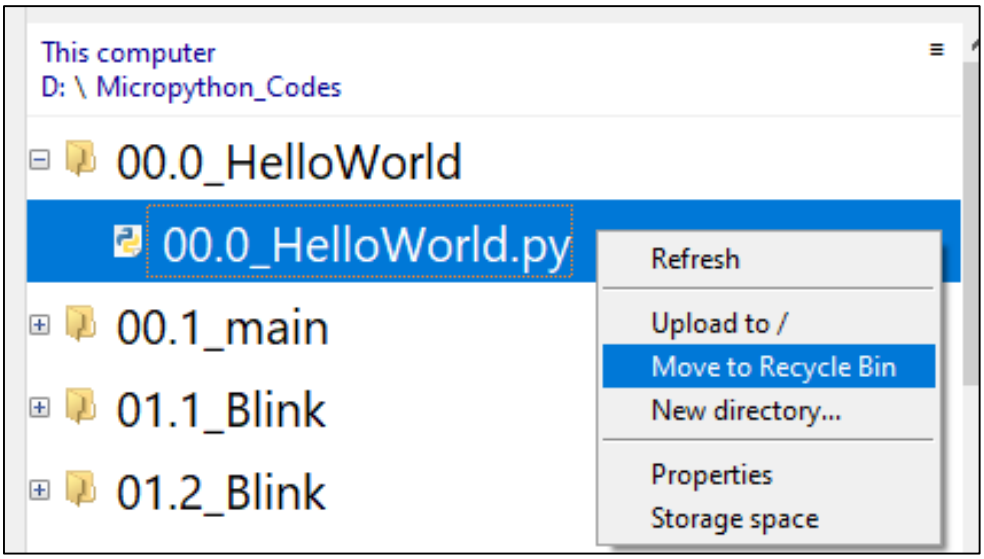
Deleting Files from Raspberry Pi Pico's Root Directory

Select "00.0_HelloWorld.py" in "MicroPython device", right-click it and select "Delete" to delete "00.0_HelloWorld.py" from Raspberry Pi Pico's root directory.



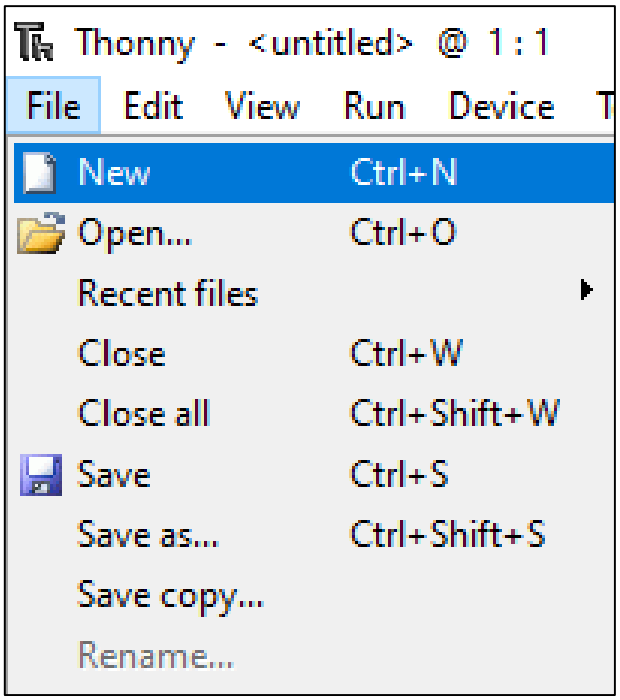
Deleting Files from your Computer Directory

Select "00.0_HelloWorld.py" in "00.0_HelloWorld", right-click it and select "Move to Recycle Bin" to delete it from "00.0_HelloWorld".

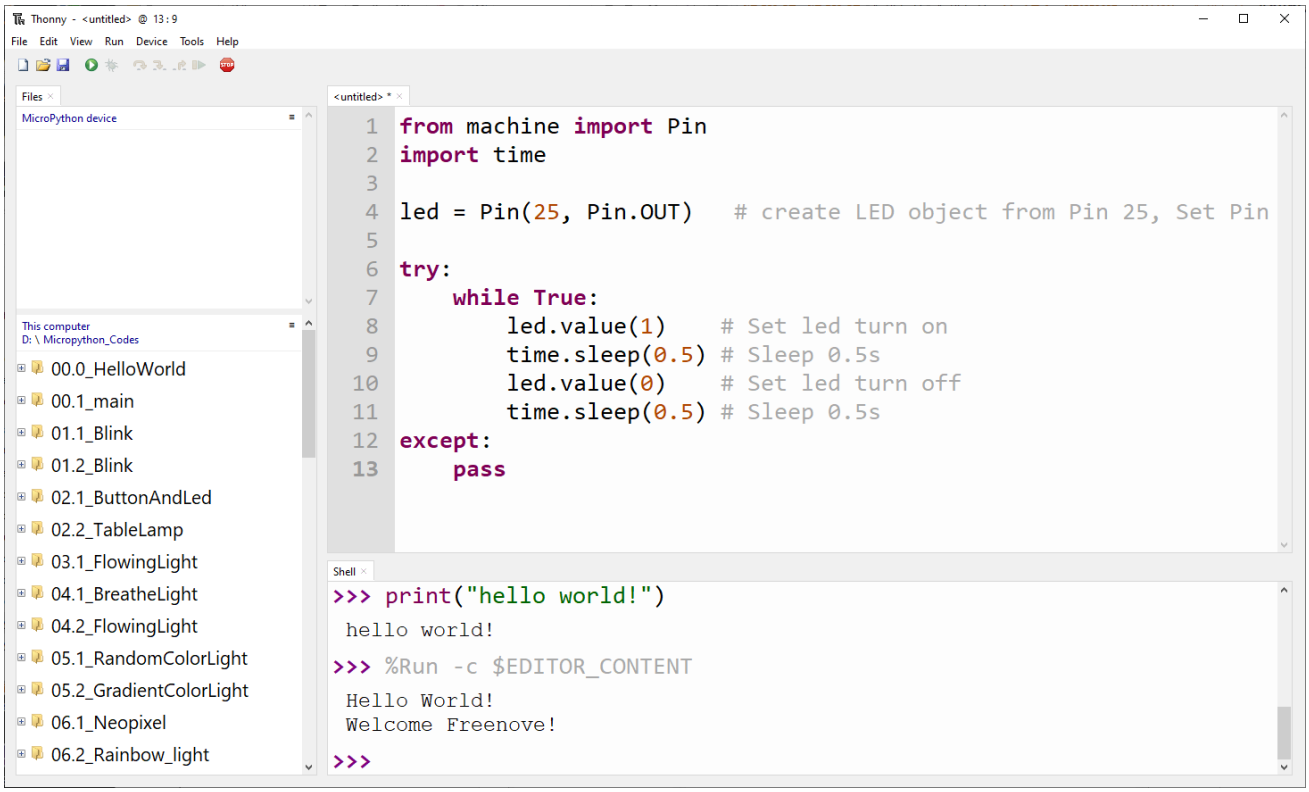


Creating and Saving the code

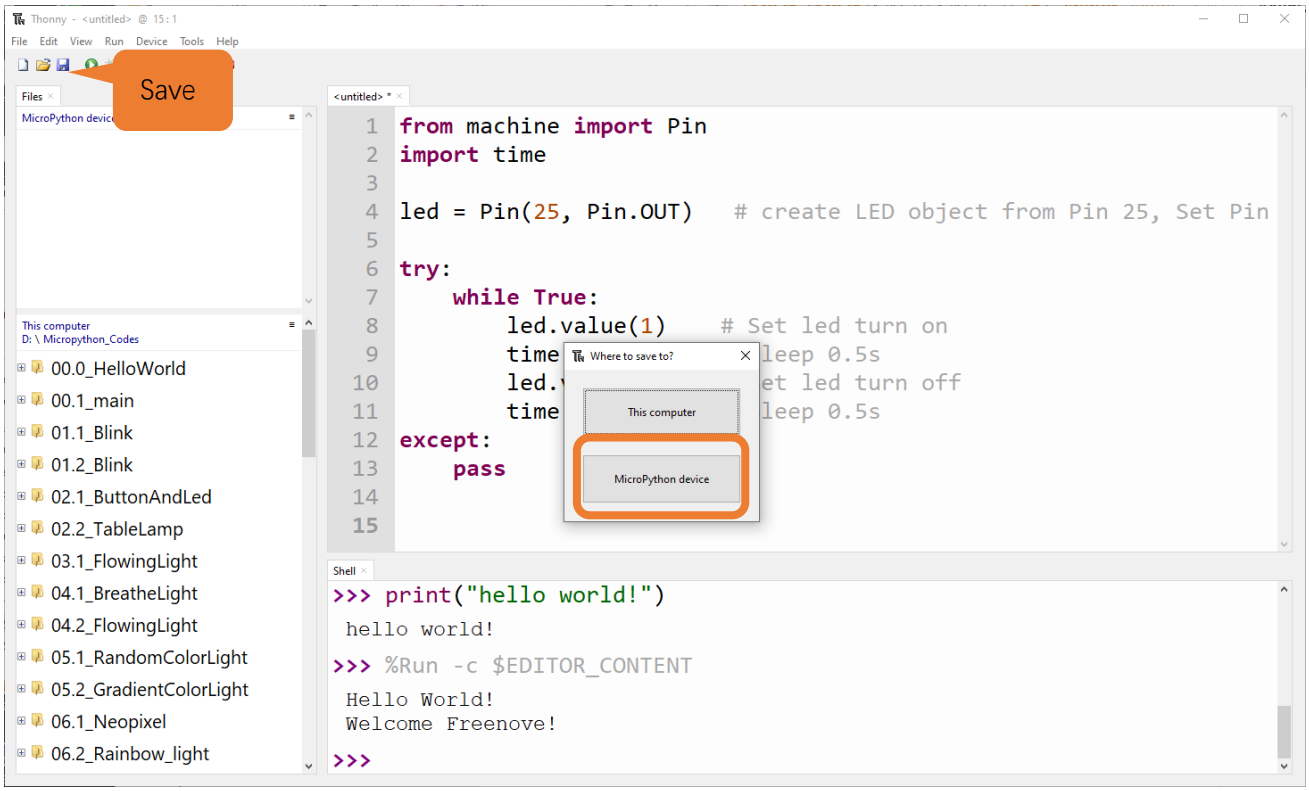
Click "File"→"New" to create and write codes.



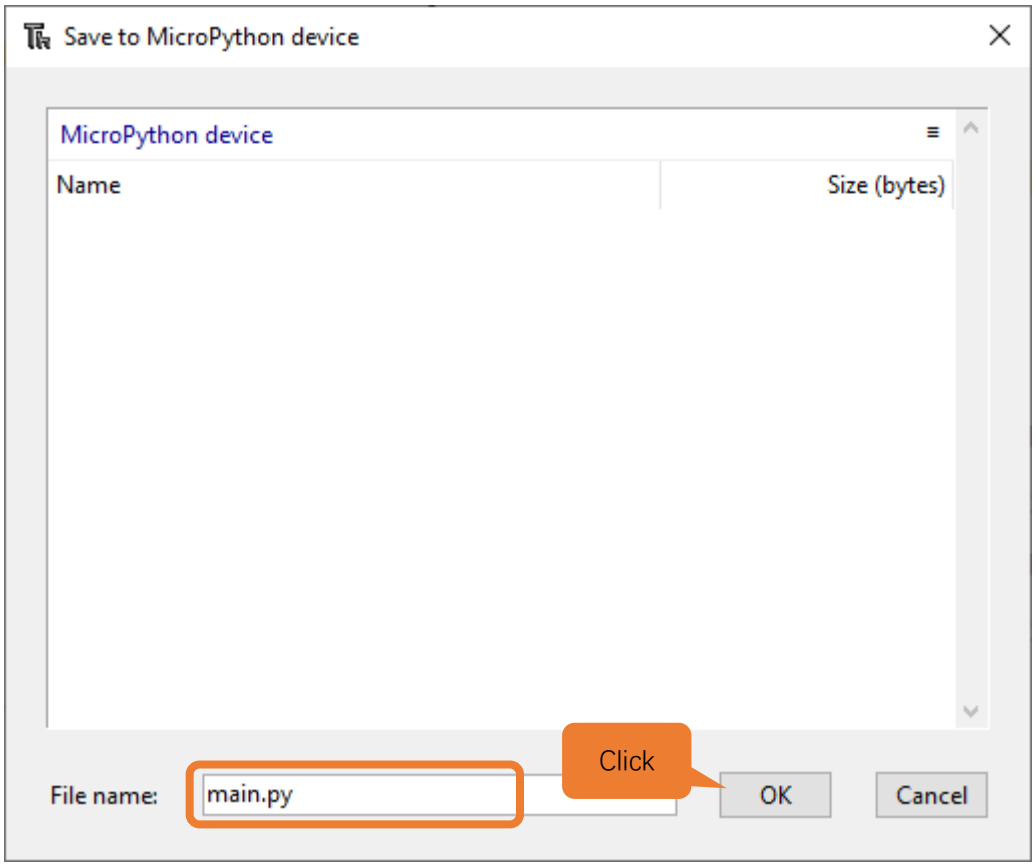
Enter codes in the newly opened file. Here we use codes of "01.1_Blink.py" as an example.



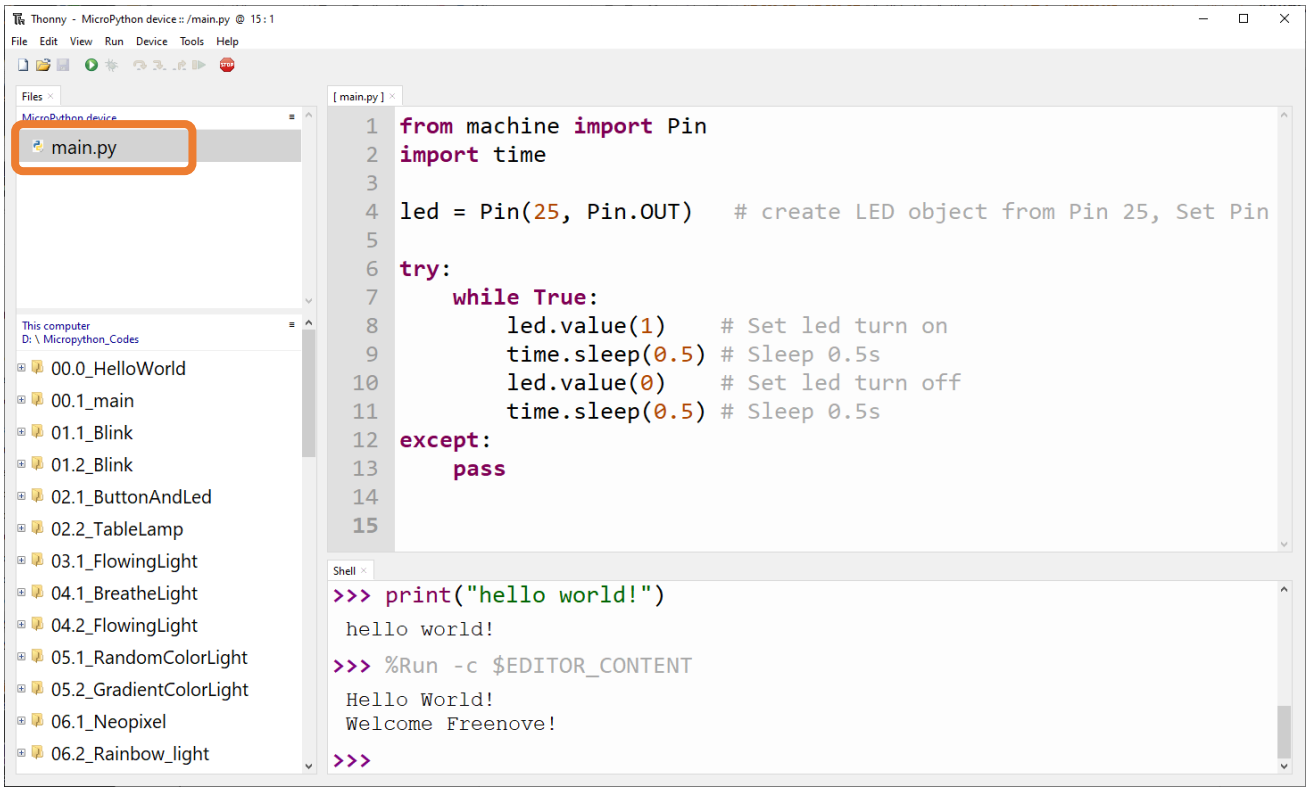
Click "Save" on the menu bar. You can save the codes either to your computer or to Raspberry Pi Pico.



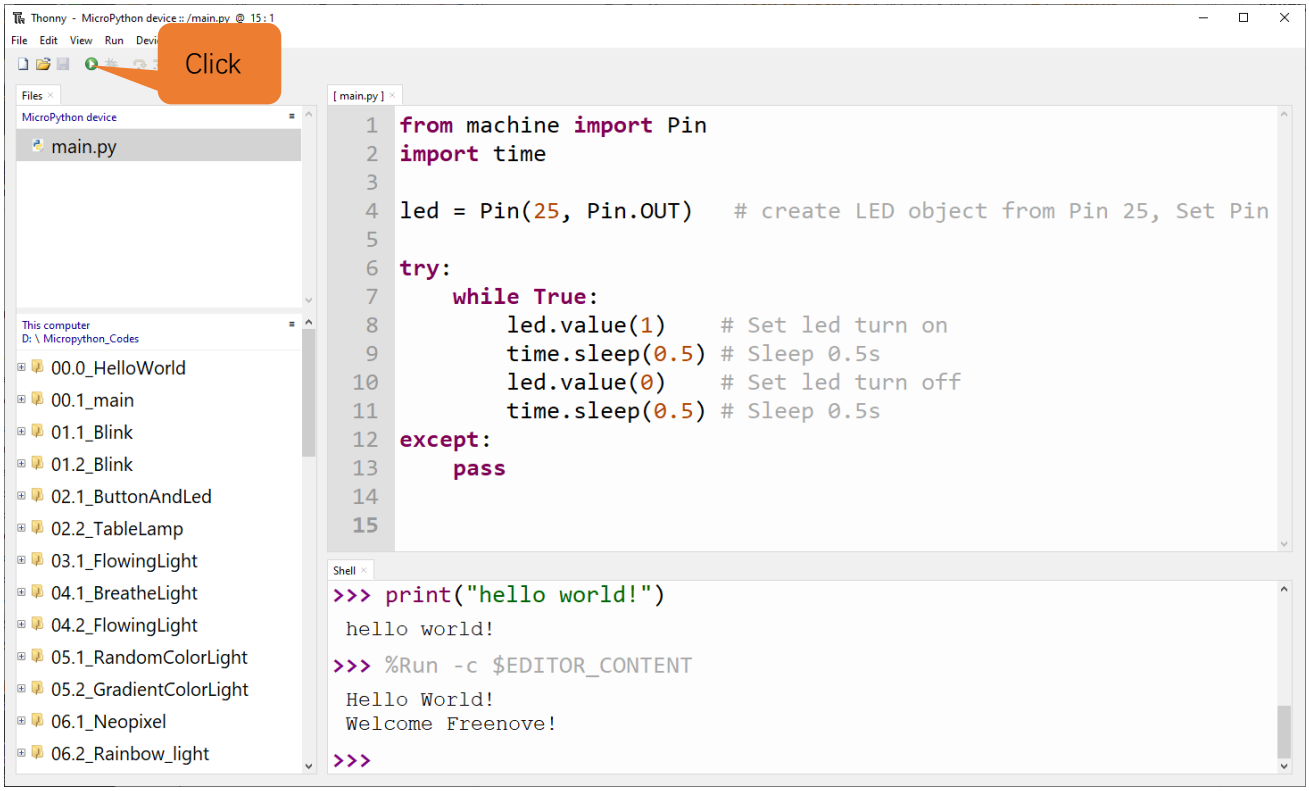
Select "MicroPython device", enter "main.py" in the newly pop-up window and click "OK".



You can see that codes have been uploaded to Raspberry Pi Pico.

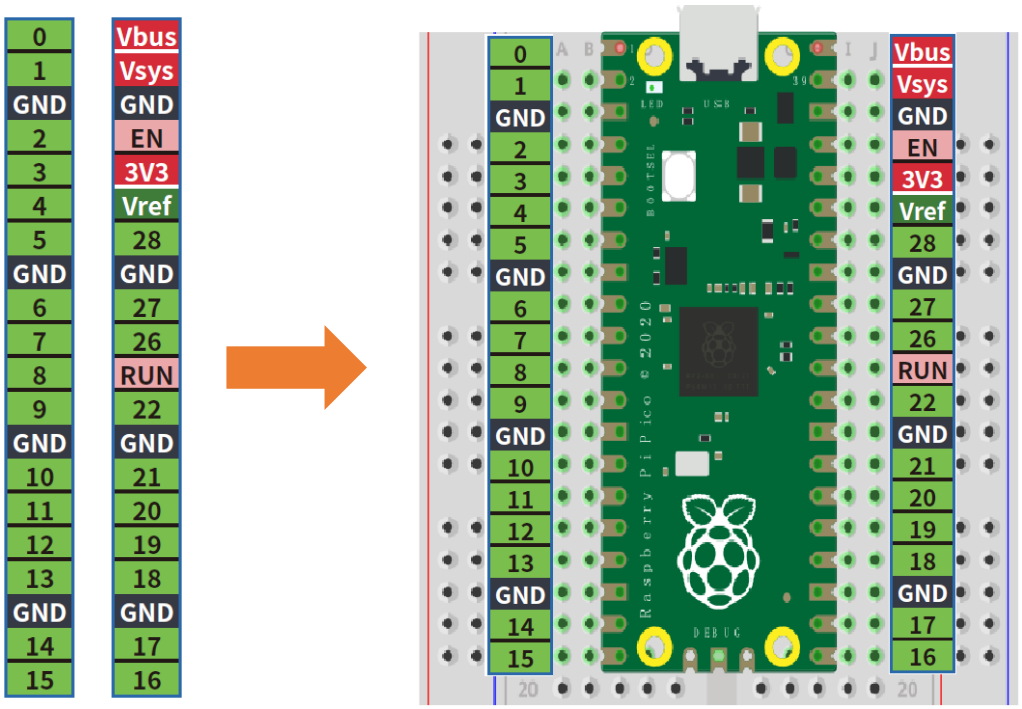


Click "Run" and the LED on Raspberry Pi Pico will blink periodically.



0.7 Paste the Sticker on the Breadboard

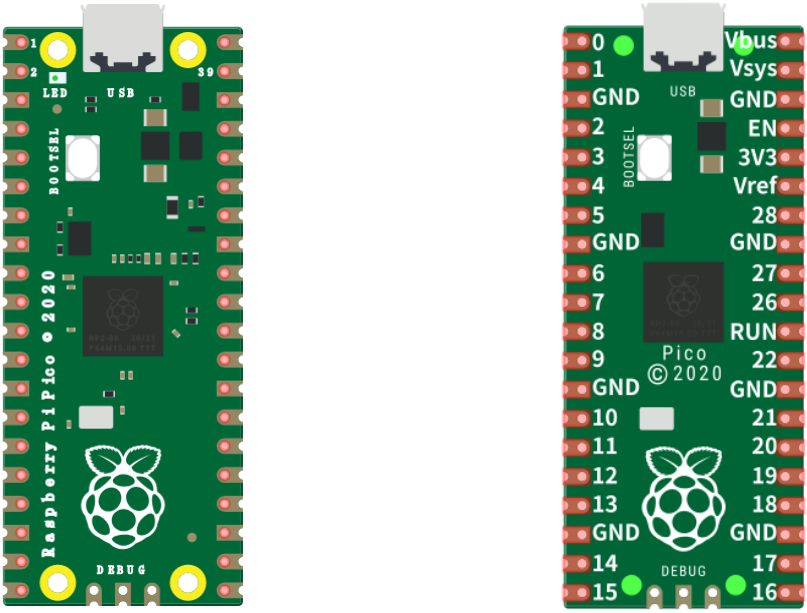
It is not difficult to use the Pico. However, officially, the pin functions are printed on the back of the board, which makes it inconvenient to use. To help users finish each project in the tutorial faster and easier, we provide stickers of the pin functions as follows:



You can paste the sticker on the blank area of the breadboard as above.

Note: The functional pin sequence of Pico, Pico W, Pico 2 and Pico 2W is the same. Therefore, even if your kit is Pico W, Pico 2 and Pico 2W, it is also applicable to the sticker above.

To make the tutorial more intuitive, we have made some changes to the simulation diagram as below. The left one is the actual Pico and the right one is its simulation diagram. Please note that to avoid misunderstanding.



Chapter 1 LED (Important)

Note: Raspberry Pi Pico, Raspberry Pi Pico W, Raspberry Pi Pico 2 and Raspberry Pi Pico 2W only differ by one wireless function, and are almost identical in other aspects. In this tutorial, except for the wireless function, other parts use Raspberry Pi Pico's map for tutorial demonstration.

This chapter is the Start Point in the journey to build and explore Raspberry Pi Pico electronic projects. We will start with simple "Blink" project.

Project 1.1 Blink

In this project, we will use Raspberry Pi Pico to control blinking a common LED.

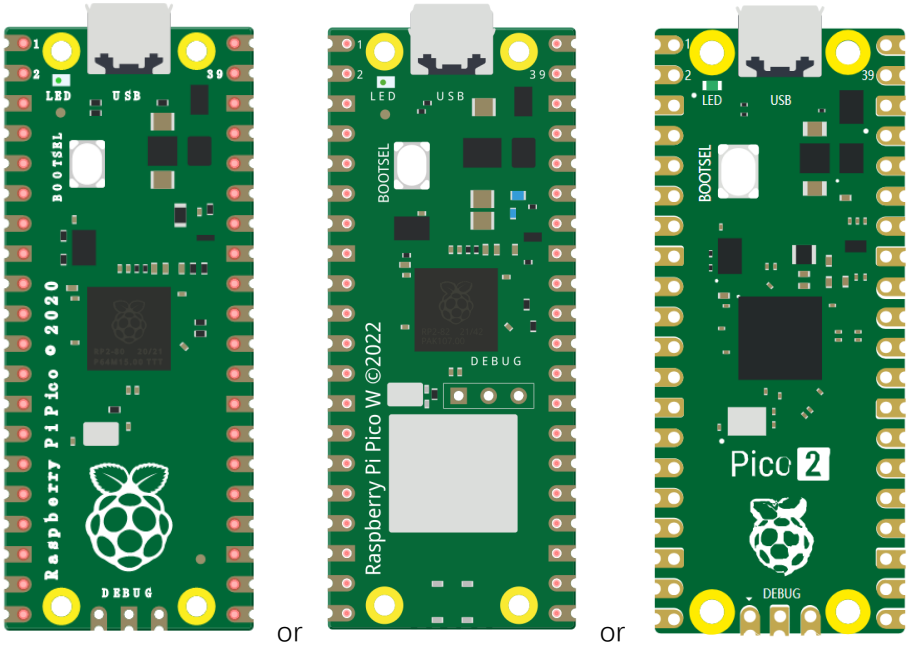
If you have not yet installed Thonny, click here.

If you have not yet downloaded Micropython Firmware, click here.

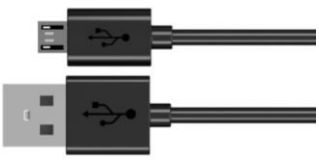
If you have not yet loaded Micropython Firmware, click here.

Component List

Raspberry Pi Pico（or Pico W or Pico 2）x1

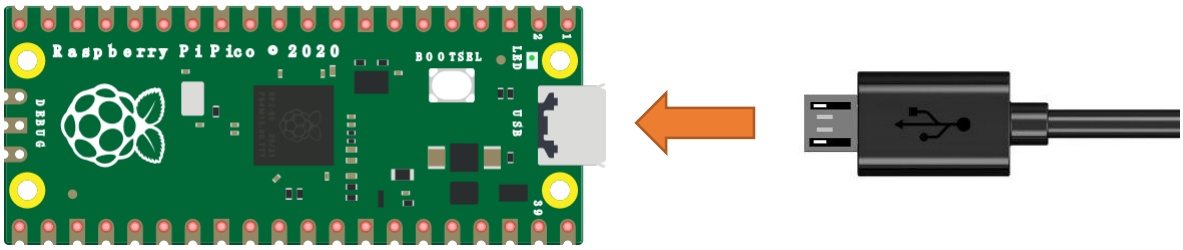


USB cable x1



Power

In this tutorial, we connect Raspberry Pi Pico and computer with a USB cable.

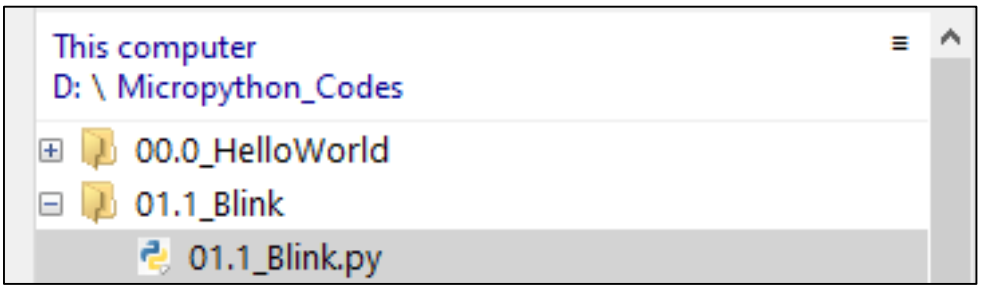


Code

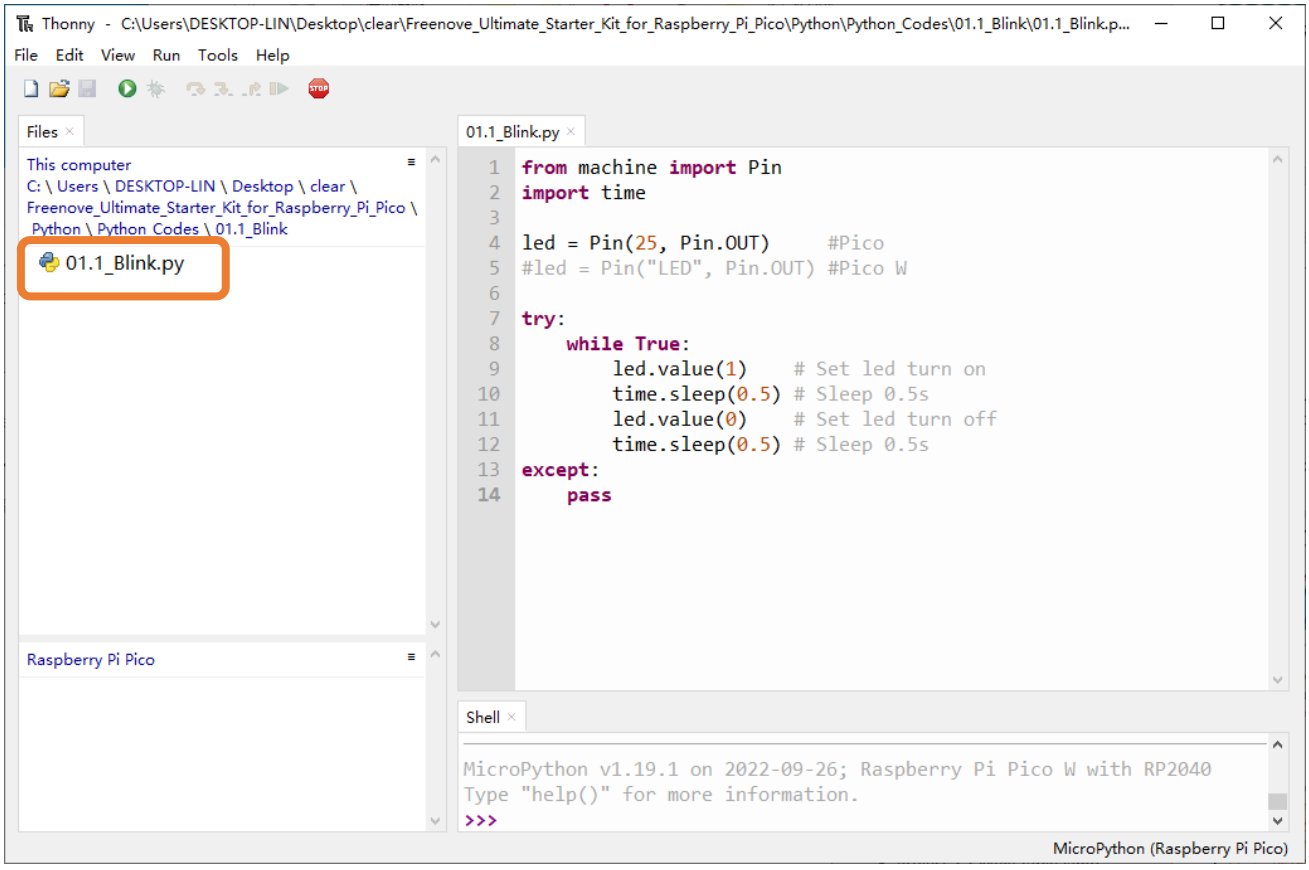
Codes used in this tutorial are saved in "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Codes**". You can move the codes to any location. For example, we save the codes in Disk(D) with the path of "**D:/Micropython_Codes**".

01.1_Blink

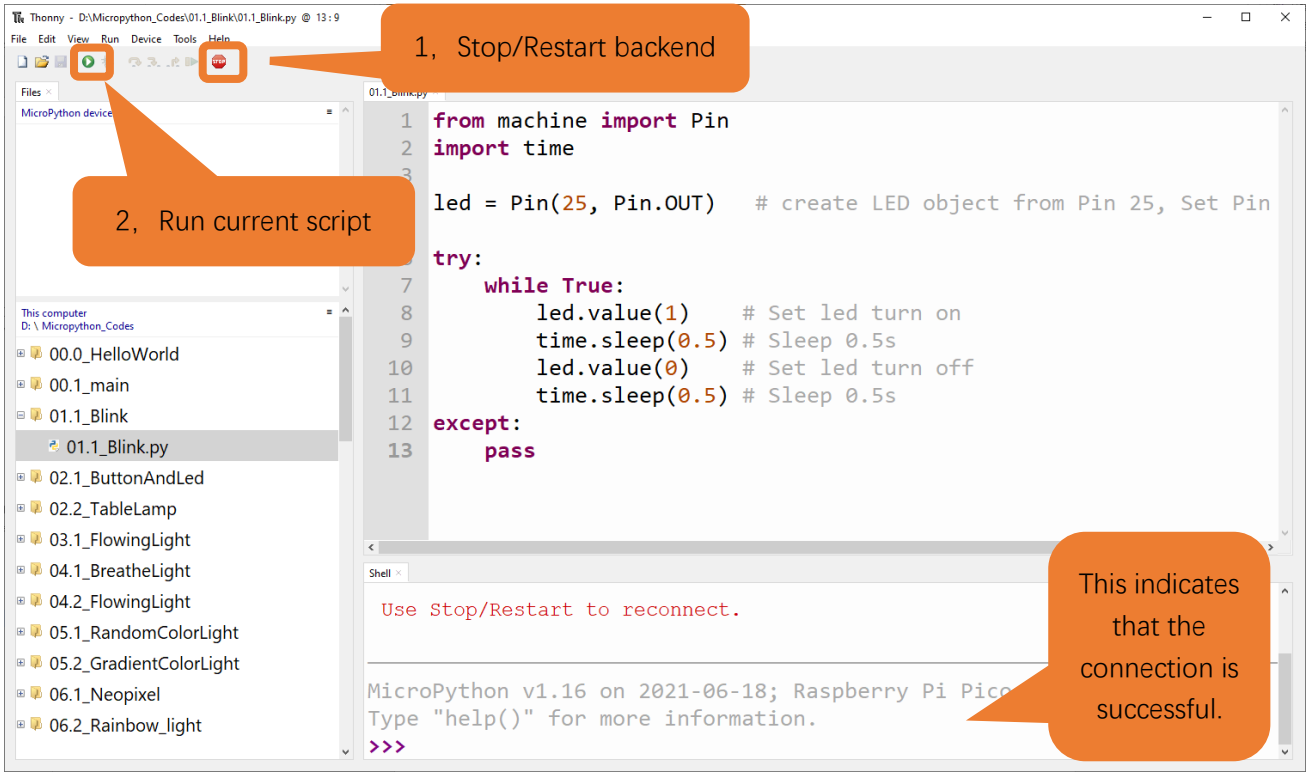
Open "Thonny", click "This computer"→"D:"→"Micropython_Codes".



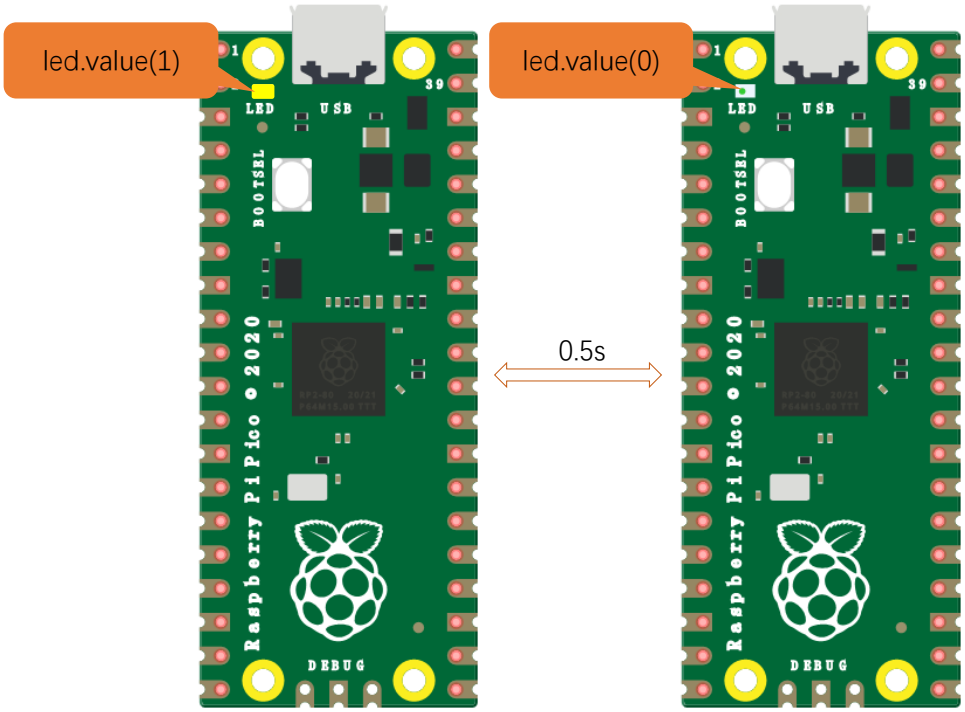
Expand folder "01.1_Blink" and double click "01.1_Blink.py" to open it. As shown in the illustration below.



Make sure Raspberry Pi Pico has been connected with the computer. Click “Stop/Restart backend”, and then wait to see what interface will show up.

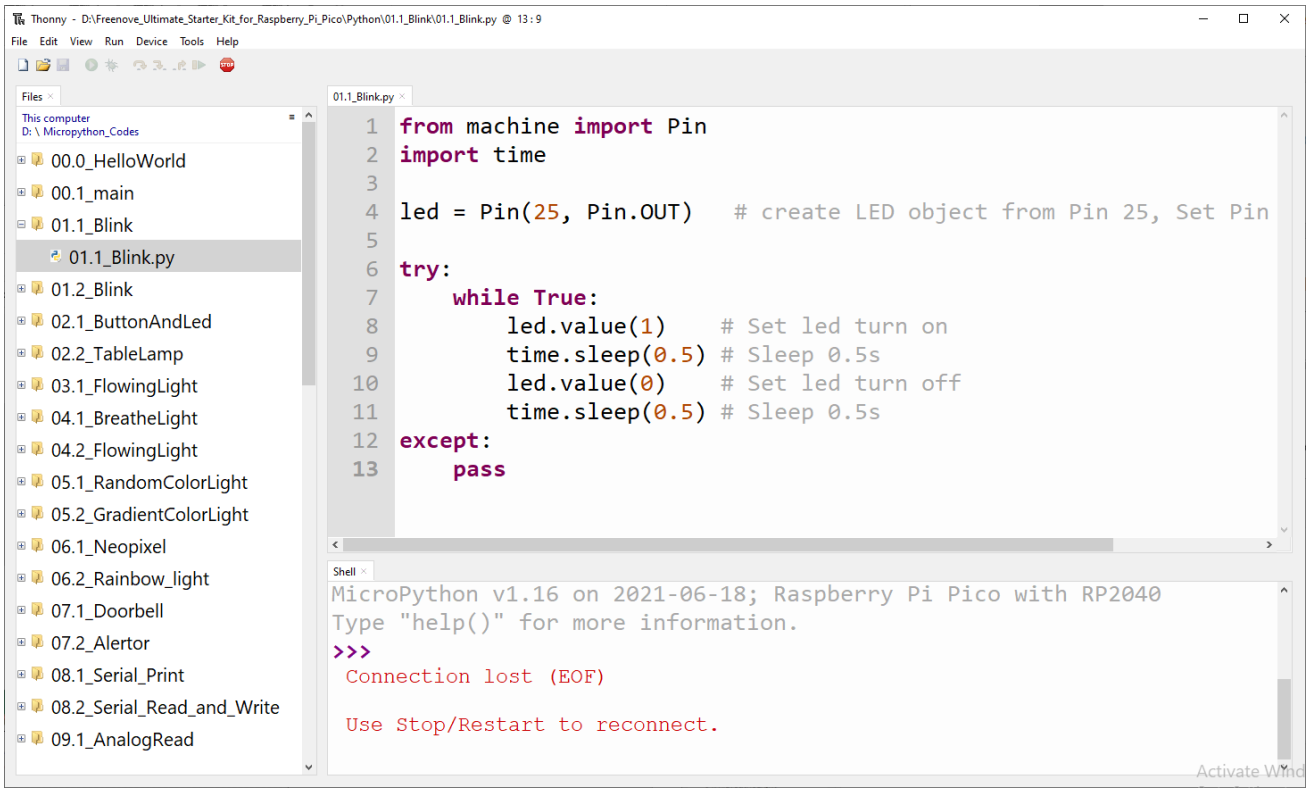


Click “Run current script” shown in the box above, the code starts to be executed and the LED in the circuit starts to blink. Press Ctrl+C or click “Stop/Restart backend” to exit the program.



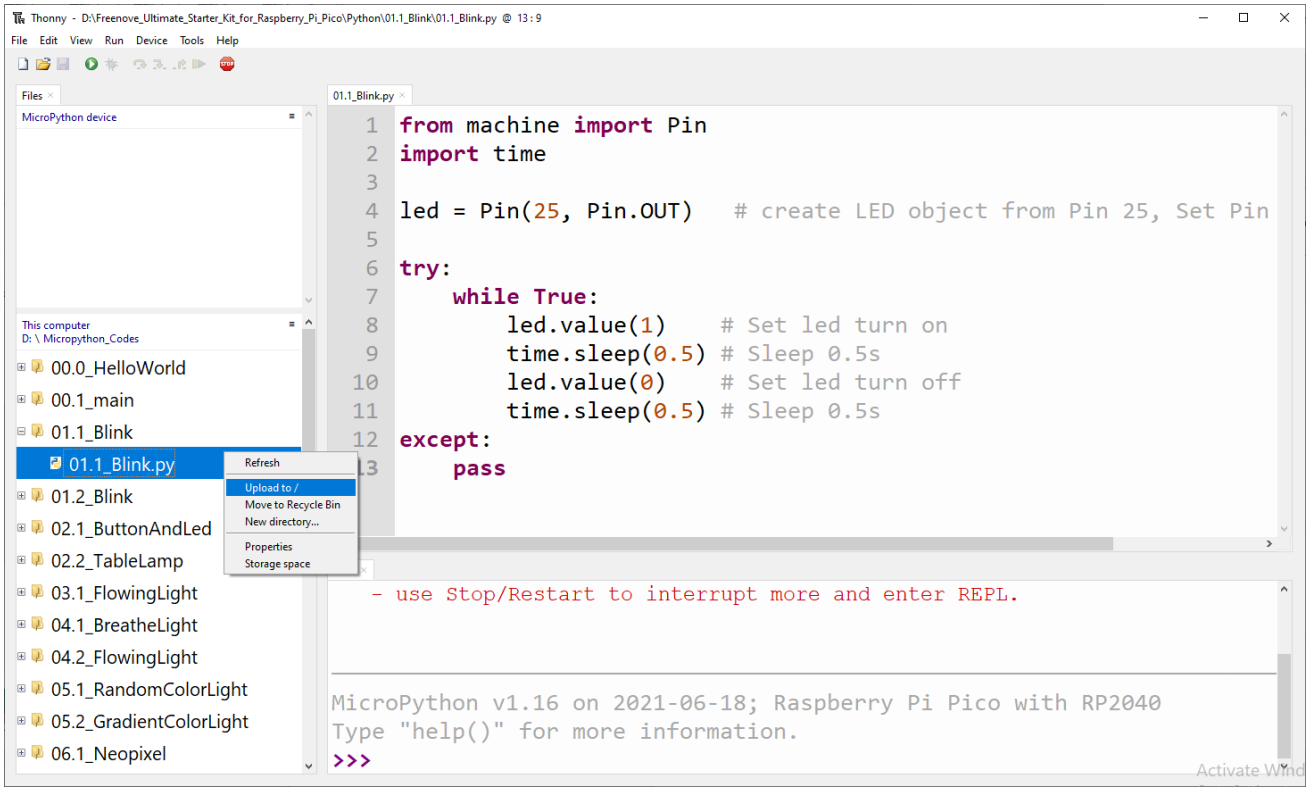
Note:

This is the code running online. If you disconnect USB cable and repower Raspberry Pi Pico, LED stops blinking and the following messages will display in Thonny.

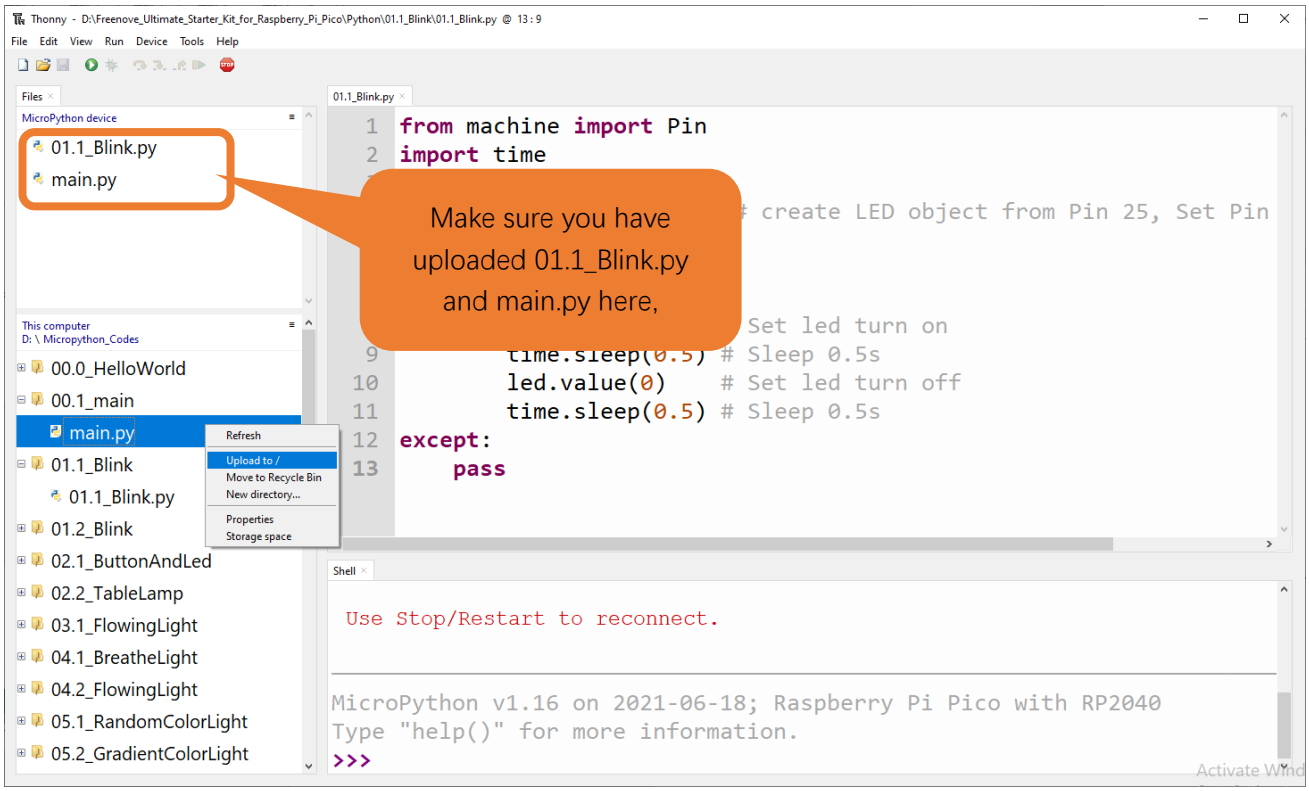


Uploading code to Raspberry Pi Pico

As shown in the following illustration, right-click the file 01.1_Blink.py and select “Upload to /” to upload code to Raspberry Pi Pico.



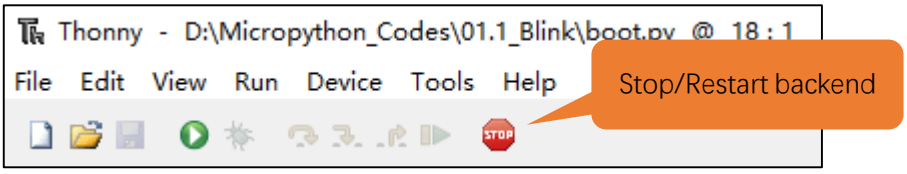
Upload main.py in the same way.



Disconnect Raspberry Pi Pico USB cable and reconnect it, LED on Pico will blink repeatedly.

Note:

Codes here is run offline. If you want to stop running offline and enter Shell, just click "Stop" in Thonny.

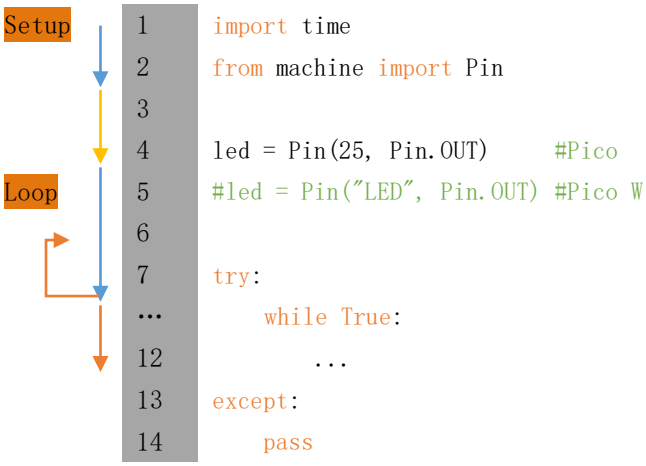


If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
1  import time
2  from machine import Pin
3
4  led = Pin(25, Pin.OUT)      #Pico
5  #led = Pin("LED", Pin.OUT) #Pico W
6
7  try:
8      while True:
9          led.value(1)     #Set led turn on
10         time.sleep(0.5) #Sleep 0.5s
11         led.value(0)     #Set led turn off
12         time.sleep(0.5) #Sleep 0.5s
13 except:
14     pass
```

Each time a new file is opened, the program will be executed from top to bottom. When encountering a loop construction, it will execute the loop statement according to the loop condition.




```
1  import time
2  from machine import Pin
3
4  led = Pin(25, Pin.OUT)      #Pico
5  #led = Pin("LED", Pin.OUT) #Pico W
6
7  try:
…      while True:
12         ...
13 except:
14     pass
```

Print() function is used to print data to Terminal. It can be executed in Terminal directly or be written in a Python file and executed by running the file.

```
print("Hello world!")
```

Each time when using the functions of Raspberry Pi Pico, you need to import modules corresponding to those functions: Import time module and Pin module of machine module.

```
1  import time
2  from machine import Pin
```

Configure GP25 of Raspberry Pi Pico to output mode and assign it to an object named "led".

Configure "LED" of Raspberry Pi Pico W to output mode and assign it to an object named "led".

```
4  led = Pin(25, Pin.OUT)      #Pico
5  #led = Pin("LED", Pin.OUT) #Pico W
```

It means that from now on, LED representing GP25 is in output mode.

Set the value of LED to 1 and GP25 will output high level.

```
8  led.value(1) #Set led turn on
```

Set the value of LED to 0 and led pin will output low level.

```
led.value(0) #Set led turn on
```

Execute codes in a while loop.

```
    while True:
        ...
```

Put statements that may cause an error in "try" block and the executing statements when an error occurs in "except" block. In general, when the program executes statements, it will execute those in "try" block. However, when an error occurs to Raspberry Pi Pico due to some interference or other reasons, it will execute statements in "except" block.

"Pass" is an empty statement. When it is executed, nothing happens. It is useful as a placeholder to make the structure of a program look better.

```
try:
    ...
except:
    pass
```

The single-line comment of Micropython starts with a "#" and continues to the end of the line. Comments help us to understand code. When programs are running, Thonny will skip comments.

```
#Set led turn on
```

MicroPython uses indentations to distinguish different blocks of code instead of braces. The number of indentations is changeable, but it must be consistent throughout one block. If the indentation of the same code block is inconsistent, it will cause errors when the program runs.

```
    while True:
        led.value(1)      #Set led turn on
        time.sleep(0.5)  #Sleep 0.5s
        led.value(0)      #Set led turn off
        time.sleep(0.5)  #Sleep 0.5s
```

How to import python files

If you import the module directly, you should indicate the module to which the function or attribute belongs when using the function or attribute (constant, variable) in the module. The format should be <module name>.<function or attribute>, otherwise, an error will occur.

```
import random
num = random.randint(1, 100)
print(num)
```

If you only want to import a certain function or attribute in the module, use the "from...import" statement. The format is as follows

```
from random import randint
num = randint(1, 100)
print(num)
```

When using "from...import" statement to import function, to avoid conflicts and for easy understanding, you can use "as" statement to rename the imported function, as follows

```
from random import randint as rand
num = rand(1, 100)
print(num)
```

Reference

Class machine

Before each use of the **machine** module, please add the statement "**import machine**" to the top of python file.

machine.freq(freq_val): When "freq_val" is not specified, it is to return to the current CPU frequency; Otherwise, it is to set the current CPU frequency.

freq_val: 125000000Hz (125MHz).

machine.reset(): A reset function. When it is called, the program will be reset.

machine.unique_id(): Obtains MAC address of the device.

machine.idle(): Turns off any temporarily unused functions on the chip and its clock, which is useful to reduce power consumption at any time during short or long periods.

machine.disable_irq(): Disables interrupt requests and return the previous IRQ state. The disable_irq () function and enable_irq () function need to be used together; Otherwise the machine will crash and restart.

machine.enable_irq(state): To re-enable interrupt requests. The parameter **state** should be the value that was returned from the most recent call to the disable_irq() function.

machine.time_pulse_us(pin, pulse_level, timeout_us=1000000):

Tests the duration of the external pulse level on the given pin and returns the duration of the external pulse level in microseconds. When pulse level = 1, it tests the high level duration; When pulse level = 0, it tests the low level duration.

If the setting level is not consistent with the current pulse level, it will wait until they are consistent, and then start timing. If the set level is consistent with the current pulse level, it will start timing immediately.

When the pin level is opposite to the set level, it will wait for timeout and return "-2". When the pin level and the set level is the same, it will also wait timeout but return "-1". **timeout_us** is the duration of timeout.

For more information about class and function, please refer to:

https://docs.micropython.org/en/latest/rp2/quickref.html

Class time

Before each use of the **time** module, please add the statement "**import time**" to the top of python file

time.sleep(sec): Sleeps for the given number of seconds.

sec: This argument should be either an int or a float.

time.sleep_ms(ms): Sleeps for the given number of milliseconds, ms should be an int.

time.sleep_us(us): Sleeps for the given number of microseconds, us should be an int.

time.time(): Obtains the timestamp of CPU, with second as its unit.

time.ticks_ms(): Returns the incrementing millisecond counter value, which recounts after some values.

time.ticks_us(): Returns microsecond.

time.ticks_cpu(): Similar to ticks_ms() and ticks_us(), but it is more accurate(return clock of CPU).

time.ticks_add(ticks, delta): Gets the timestamp after the offset.

ticks: ticks_ms(), ticks_us(), ticks_cpu().

delta: Delta can be an arbitrary integer number or numeric expression.

time.ticks_diff(old_t, new_t): Calculates the interval between two timestamps, such as ticks_ms(), ticks_us() or ticks_cpu().

old_t: Starting time.

new_t: Ending time.

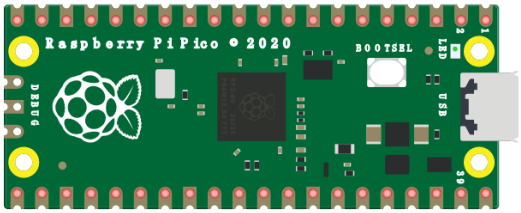
Class Pin(id, mode, pull, value)	
Before each use of the **Pin** module, please add the statement "**from machine import Pin**" to the top of python file. **id:** Arbitrary pin number. **mode**: Mode of pins. **Pin.IN**: Input Mode. **Pin.OUT**: Output Mode. **Pin.OPEN_DRAIN**: Open-drain Mode. **Pull**: Whether to enable the internal pull up and down mode. **None**: No pull up or pull down resistors. **Pin.PULL_UP**: Pull-up Mode, outputting high level by default. **Pin.PULL_DOWN**: Pull-down Mode, outputting low level by default. **Value**: State of the pin level, 0/1. **Pin.init(mode, pull)**: Initialize pins. **Pin.value([value])**: Obtain or set state of the pin level, return 0 or 1 according to the logic level of pins. Without parameter, it reads input level. With parameter given, it is to set output level. **value**: It can be either True/False or 1/0. **Pin.irq(trigger, handler)**: Configures an interrupt handler to be called when the pin level meets a condition. **trigger**: **Pin.IRQ_FALLING**: interrupt on falling edge. **Pin.IRQ_RISING**: interrupt on rising edge. **Handler**: callback function.	

Project 1.2 Blink

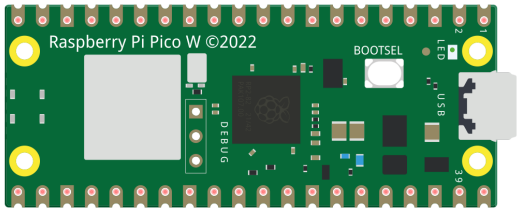
In this project, we will use Raspberry Pi Pico to control blinking a common LED.

Component List

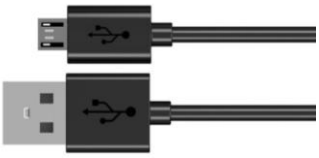
Raspberry Pi Pico（or Pico W）x1



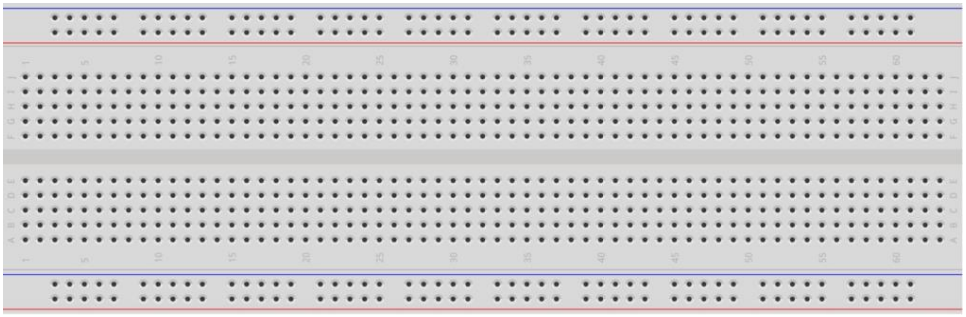
or



USB cable x1



Breadboard x1



LED x1



Resistor 220Ω x1



Jumper

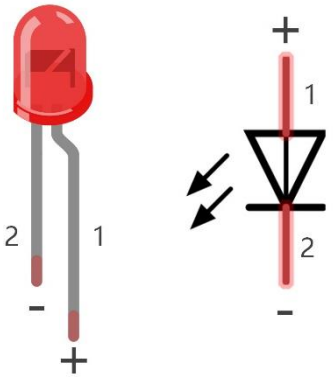


Component knowledge

LED

An LED is a type of diode. All diodes only work if current is flowing in the correct direction and have two Poles. An LED will only work (light up) if the longer pin (+) of LED is connected to the positive output from a power source and the shorter pin is connected to the negative (-). Negative output is also referred to as Ground (GND). This type of component is known as "Polar" (think One-Way Street).

All common two-lead diodes are the same in this respect. Diodes work only if the voltage of its positive electrode is higher than its negative electrode and there is a narrow range of operating voltage for most all common diodes of 1.9 and 3.4V. If you use much more than 3.3V the LED will be damaged and burn out.



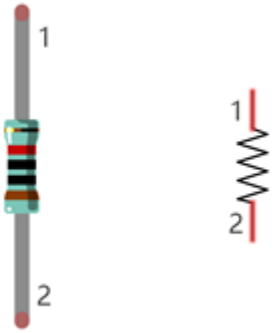
LED	Voltage	Maximum current	Recommended current
Red	1.9-2.2V	20mA	10mA
Green	2.9-3.4V	10mA	5mA
Blue	2.9-3.4V	10mA	5mA
Volt ampere characteristics conform to diode			

Note: LEDs cannot be directly connected to a power supply, which usually ends in a damaged component. A resistor with a specified resistance value must be connected in series to the LED you plan to use.

Resistor

Resistors use Ohms (Ω) as the unit of measurement of their resistance (R). 1MΩ=1000kΩ, 1kΩ=1000Ω.

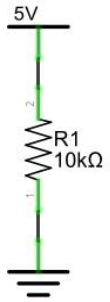
A resistor is a passive electrical component that limits or regulates the flow of current in an electronic circuit. On the left, we see a physical representation of a resistor, and the right is the symbol used to represent the presence of a resistor in a circuit diagram or schematic.



The bands of color on a resistor is a shorthand code used to identify its resistance value. For more details of resistor color codes, please refer to the appendix of this tutorial.

With a fixed voltage, there will be less current output with greater resistance added to the circuit. The relationship between Current, Voltage and Resistance can be expressed by this formula: I=V/R known as Ohm's Law where I = Current, V = Voltage and R = Resistance. Knowing the values of any two of these allows you to solve the value of the third.

In the following diagram, the current through R1 is: I=U/R=5V/10kΩ=0.0005A=0.5mA.

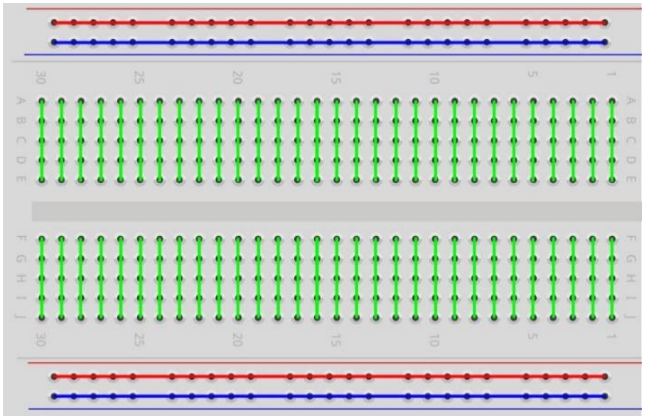


WARNING: Never connect the two poles of a power supply with anything of low resistance value (i.e. a metal object or bare wire) this is a Short and results in high current that may damage the power supply and electronic components.

Note: Unlike LEDs and Diodes, Resistors have no poles and re non-polar (it does not matter which direction you insert them into a circuit, it will work the same)

Breadboard

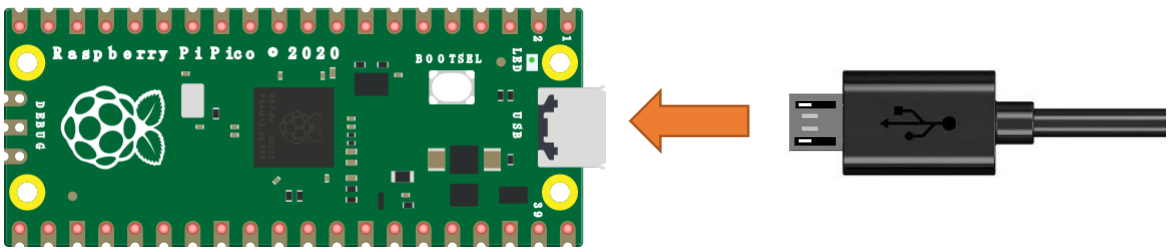
Here we have a small breadboard as an example of how the rows of holes (sockets) are electrically attached. The left picture shows the way to connect pins. The right picture shows the practical internal structure.





Power

In this tutorial, we connect Raspberry Pi Pico and computer with a USB cable.

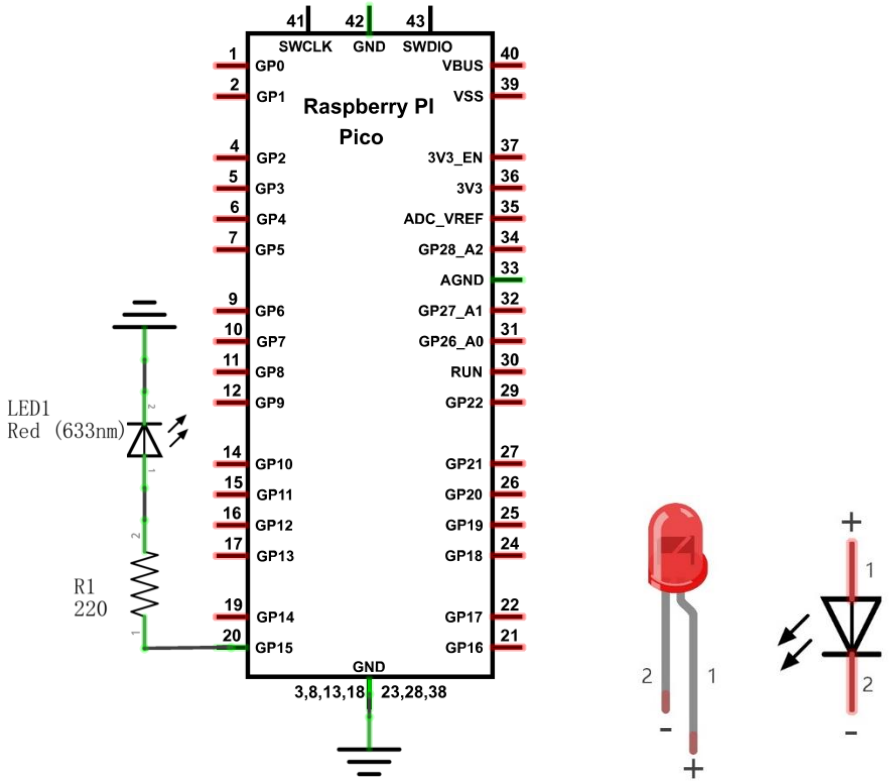


Circuit

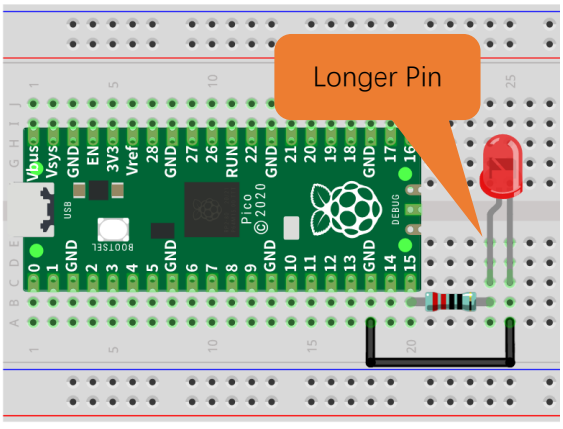
First, disconnect all power from the Raspberry Pi Pico. Then build the circuit according to the circuit and hardware diagrams. After the circuit is built and verified correct, connect the PC to Raspberry Pi Pico.

CAUTION: Avoid any possible short circuits (especially connecting 3.3V and GND)! WARNING: A short circuit can cause high current in your circuit, create excessive component heat and cause permanent damage to your hardware!

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Note: To help users have a better experience when doing the projects, we have made some modifications to Pico's simulation diagram. Please note that there are certain differences between the simulation diagram and the actual board to avoid misunderstanding.

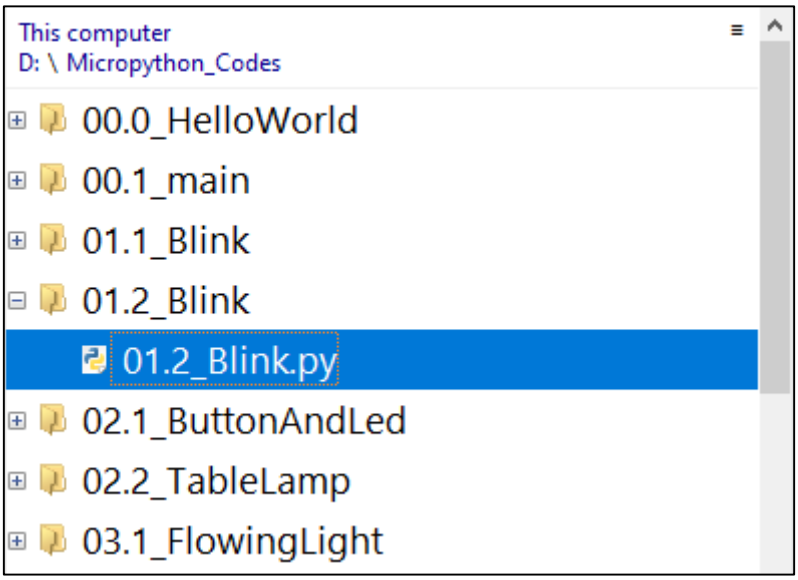
Any concerns? ✉ support@freenove.com

Code

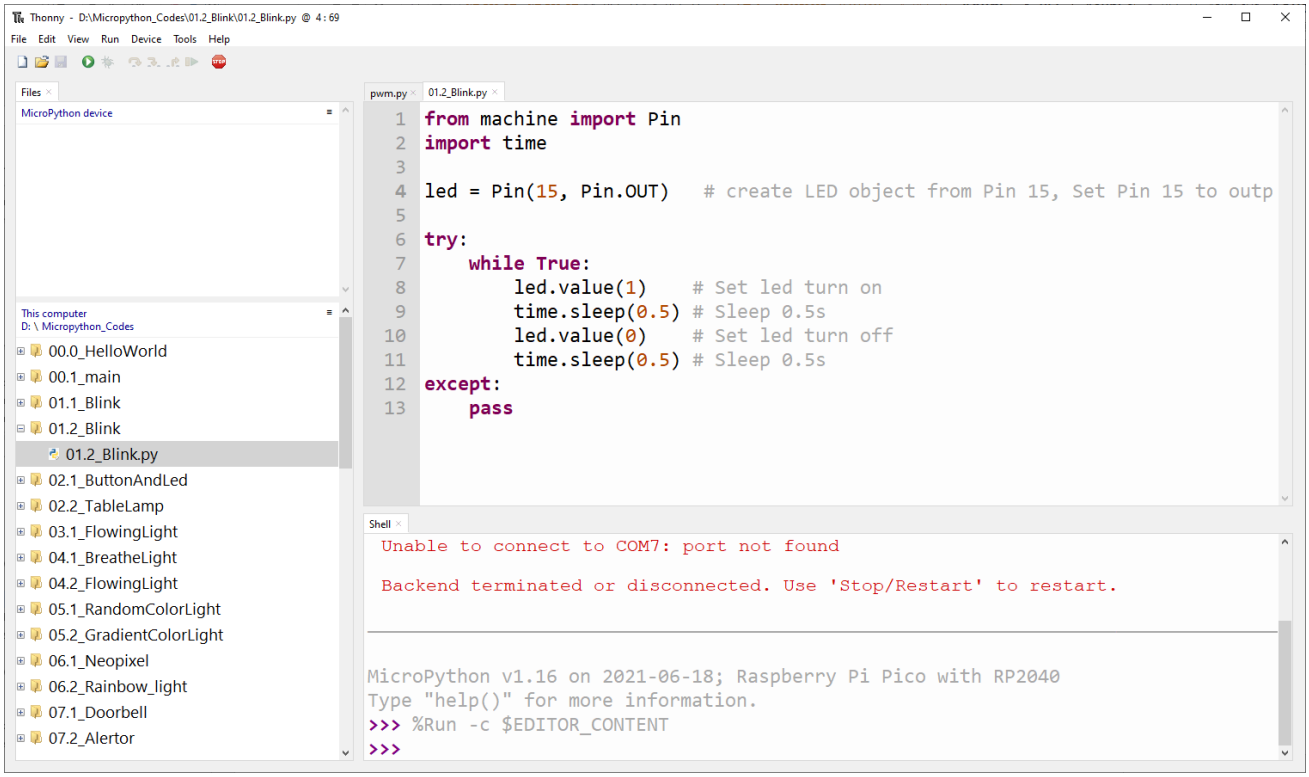
Codes used in this tutorial are saved in "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Codes**". You can move the codes to any location. For example, we save the codes in Disk(D) with the path of "**D:/Micropython_Codes**".

01.2_Blink

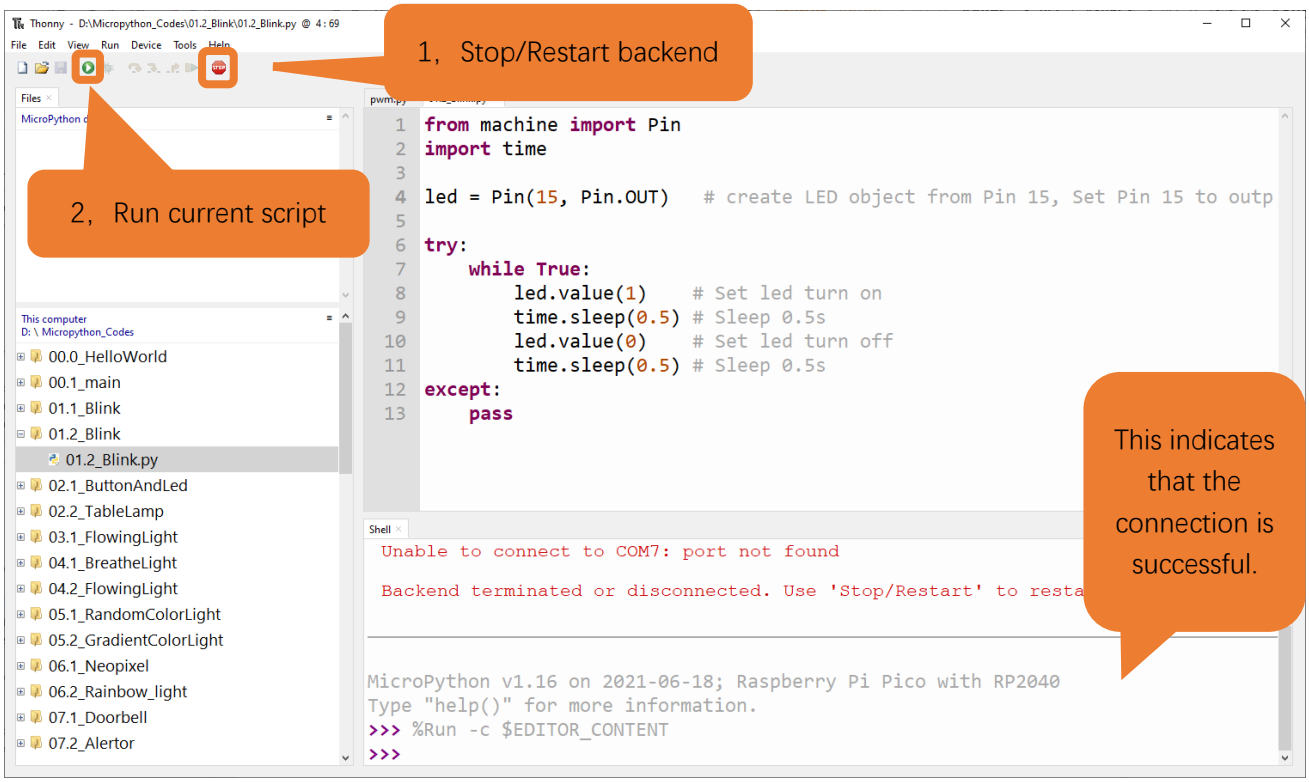
Open "Thonny", click "This computer"→"D:"→"Micropython_Codes".



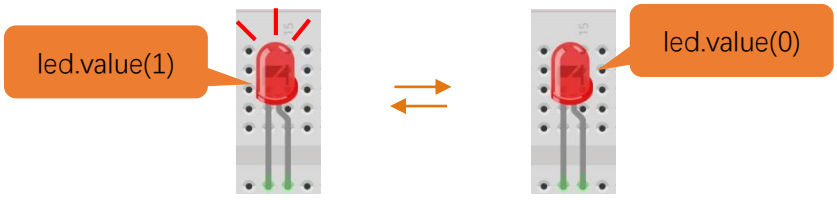
Expand folder "01.2_Blink" and double click "01.2_Blink.py" to open it. As shown in the illustration below.



Make sure Raspberry Pi Pico has been connected with the computer. Click “Stop/Restart backend”, and then wait to see what interface will show up.

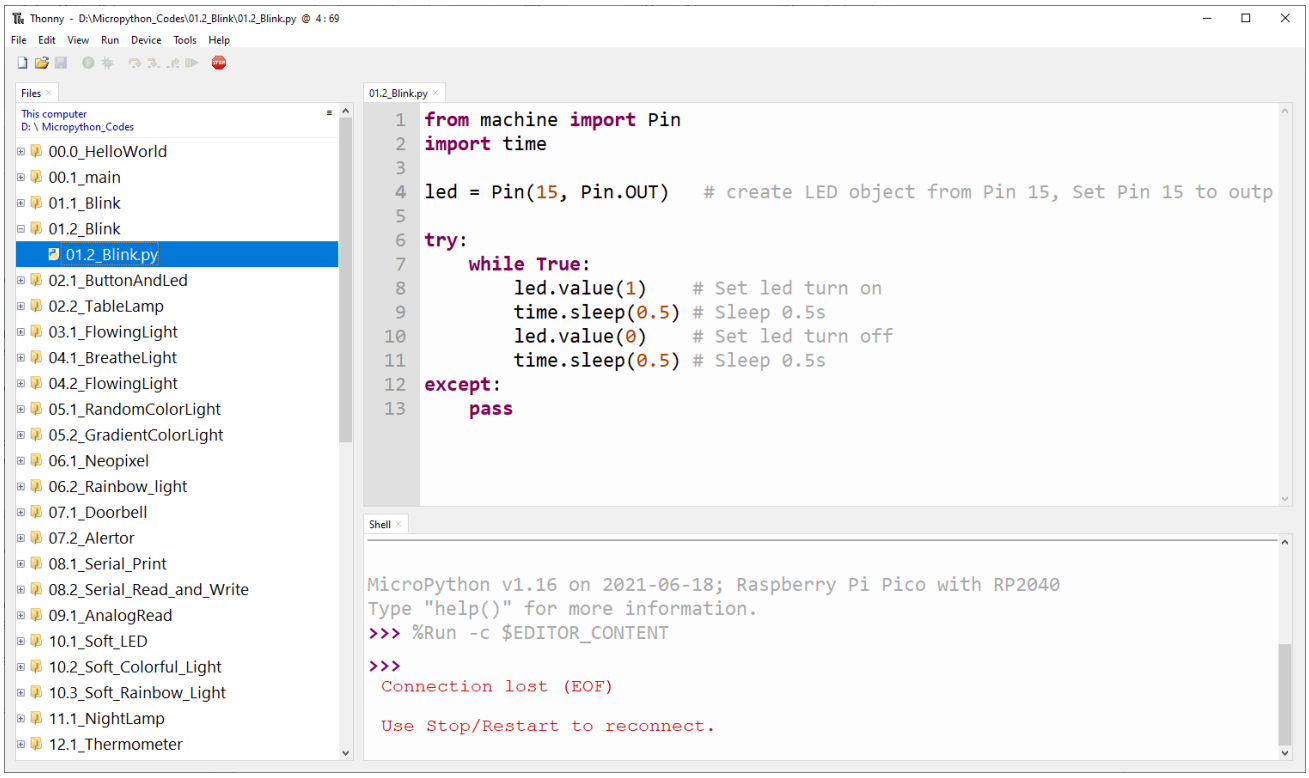


Click “Run current script” shown in the box above, the code starts to be executed and the LED in the circuit starts to blink. Press Ctrl+C or click “Stop/Restart backend” to exit the program.



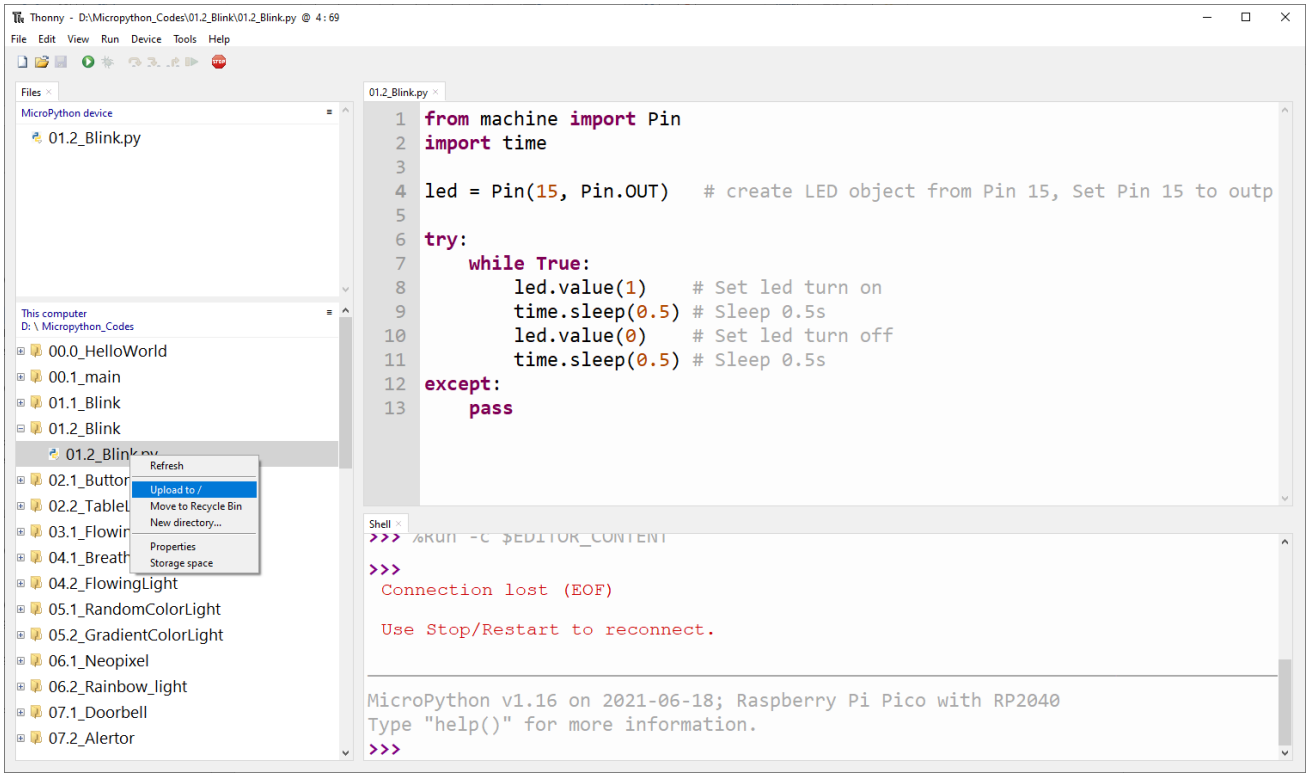
Note:

This is the code running online. If you disconnect USB cable and repower Raspberry Pi Pico, LED stops blinking and the following messages will be displayed in Thonny.

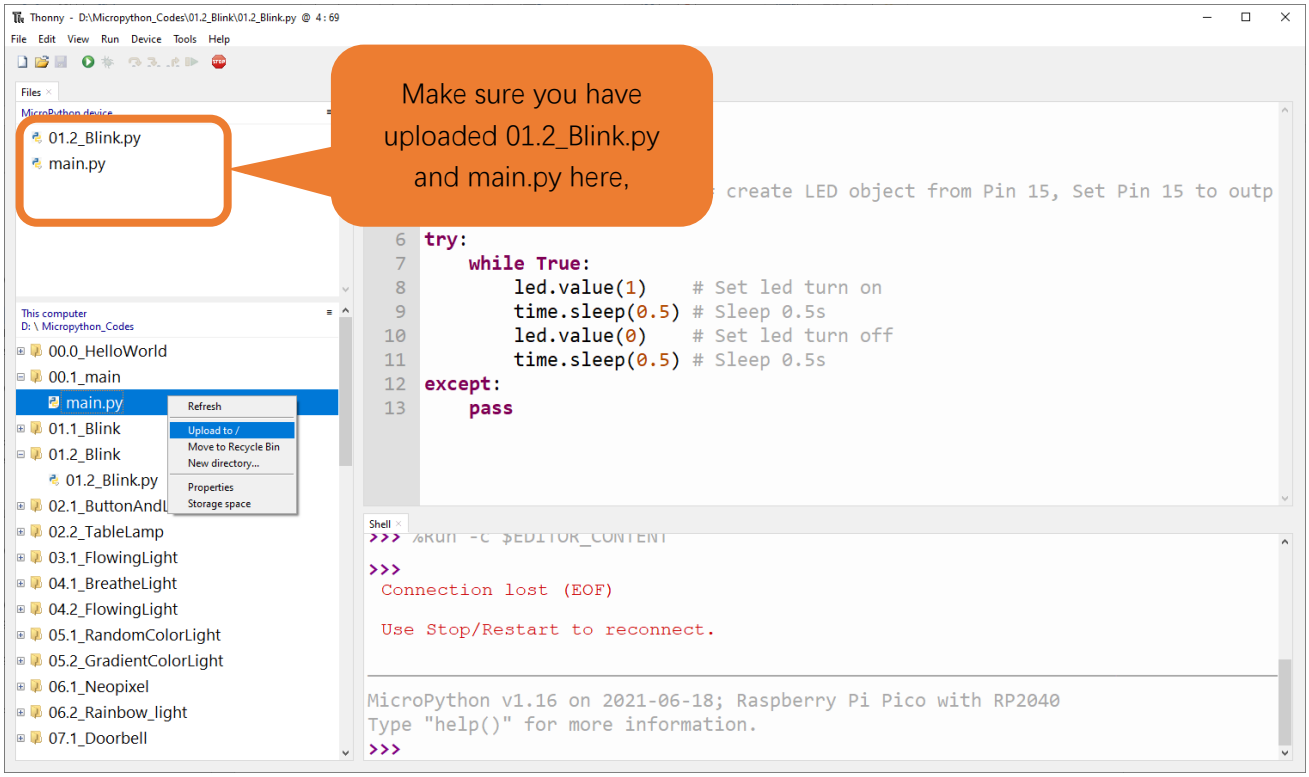


Uploading code to Raspberry Pi Pico

As shown in the following illustration, right-click the file 01.2_Blink.py and select “Upload to /” to upload code to Raspberry Pi Pico.



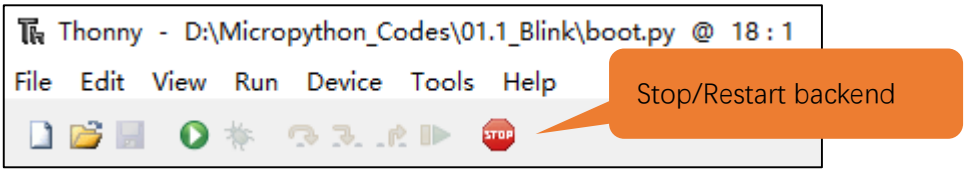
Upload main.py in the same way.



Disconnect Raspberry Pi Pico USB cable and reconnect it, LED on Pico will blink repeatedly.

Note:

Codes here is run offline. If you want to stop running offline and enter Shell, just click "Stop" in Thonny.

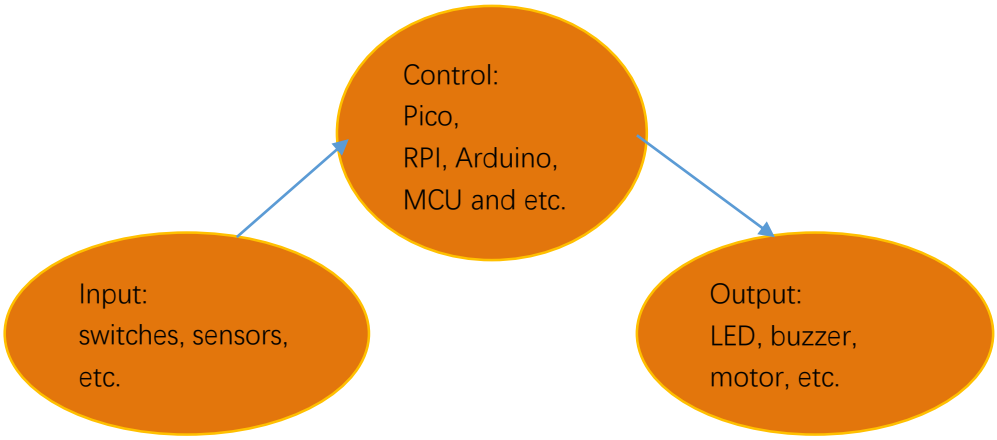


If you have any concerns, please contact us via: support@freenove.com

Chapter 2 Button & LED

Note: Raspberry Pi Pico, Raspberry Pi Pico W, Raspberry Pi Pico 2 and Raspberry Pi Pico 2W only differ by one wireless function, and are almost identical in other aspects. In this tutorial, except for the wireless function, other parts use Raspberry Pi Pico's map for tutorial demonstration.

Usually, there are three essential parts in a complete automatic control device: INPUT, OUTPUT, and CONTROL. In last section, the LED module was the output part and Raspberry Pi Pico was the control part. In practical applications, we not only make LEDs blink, but also make a device sense the surrounding environment, receive instructions and then take the appropriate action such as LEDs light up, make a buzzer turn ON and so on.

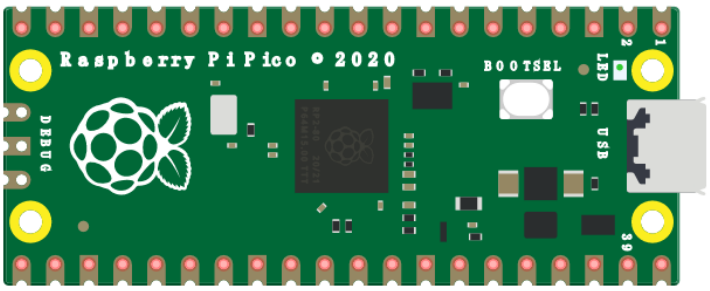
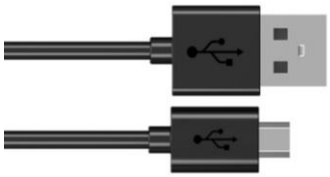
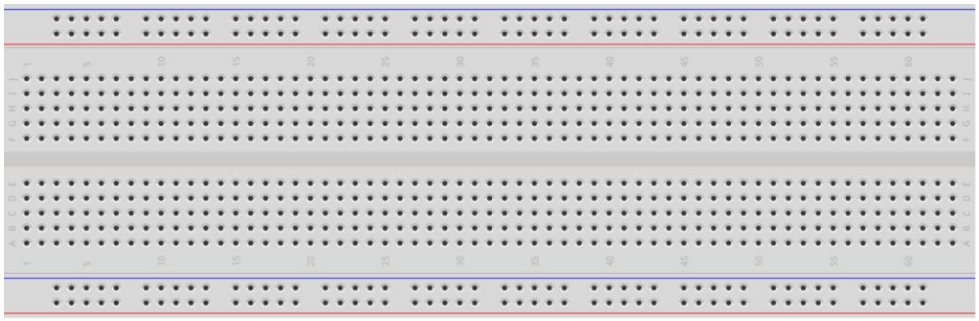


Next, we make a simple project: build a control system with button, LED and Raspberry Pi Pico.
Input: Button
Control: Raspberry Pi Pico
Output: LED

Project 2.1 Button & LED

In the project, we will control the LED state through a Push Button Switch. When the button is pressed, our LED will turn ON, and when it is released, the LED will turn OFF. This describes a Momentary Switch.

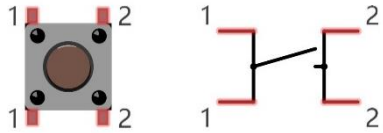
Component List

<table>
<tr><td colspan="3">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="5">Breadboard x1
  </td></tr>
<tr><td>Jumper
  </td><td>LED x1
  </td><td>Resistor
220Ω x1
  </td><td>Resistor
10kΩ x2
  </td><td>Push button
x1
  </td></tr>
</table>

Component knowledge

Push button

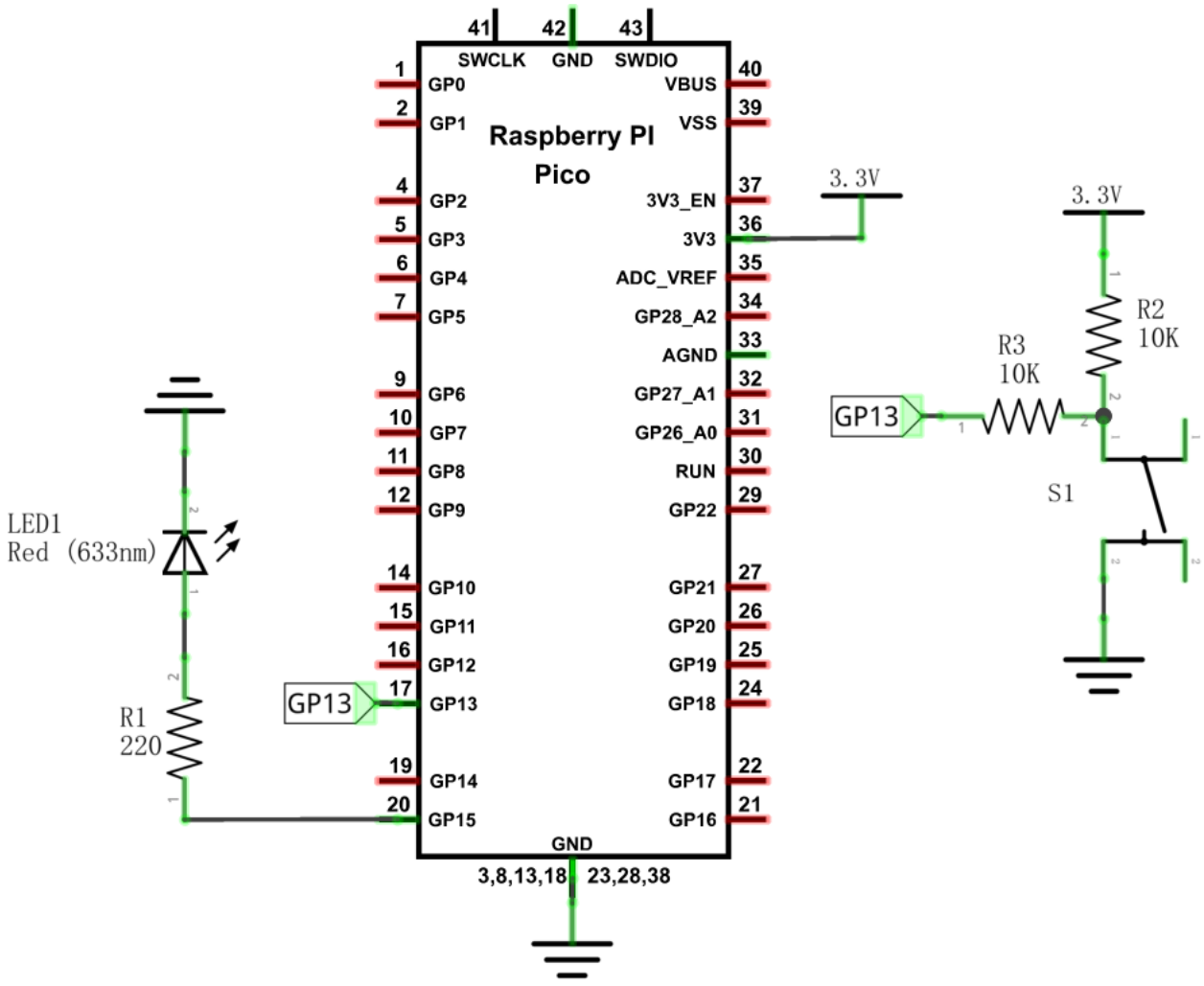
This type of Push Button Switch has four pins (2 Pole Switch). Two pins on the left are connected, and both left and right sides are the same per the illustration:



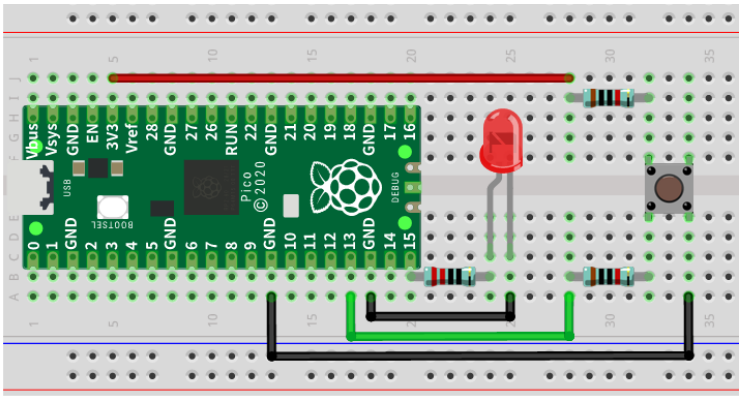
When the button on the switch is pressed, the circuit is completed (your project is Powered ON).

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Note: To help users have a better experience when doing the projects, we have made some modifications to Pico's simulation diagram. Please note that there are certain differences between the simulation diagram and the actual board to avoid misunderstanding.

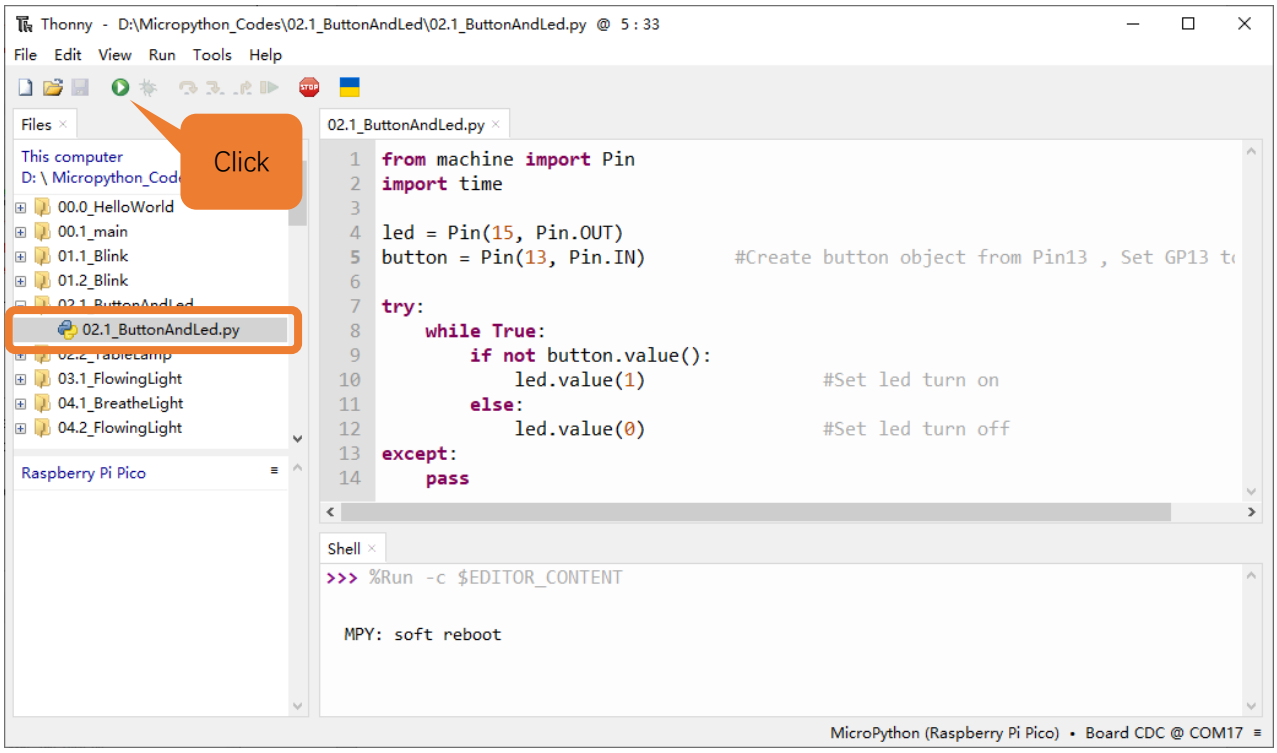
Code

This project is designed to learn to control an LED with a push button switch. First, we need to read the state of the switch and then decide whether the LED is turned on or not based on it.

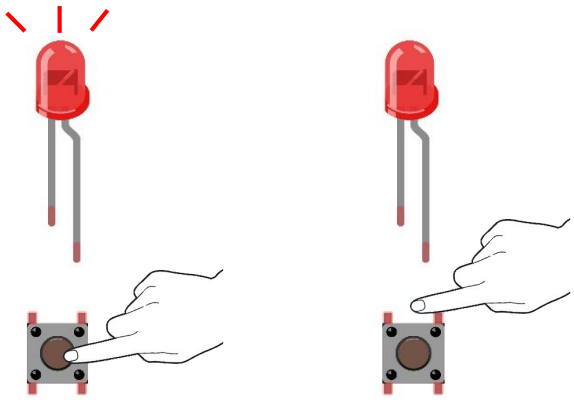
Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "02.1_ButtonAndLed" and double click "02.1_ButtonAndLed.py".

02.1_ButtonAndLed



Click "Run current script" shown in the box of the above illustration, press the push button switch, LED turns ON; release the switch, LED turns OFF. Press Ctrl+C or click "Stop/Restart backend" to exit program.



The following is the program code:

```
from machine import Pin
import time

led = Pin(15, Pin.OUT)
button = Pin(13, Pin.IN)    #Create button object from Pin13 , Pin13 to input

try:
    while True:
        if not button.value():
            led.value(1)                #Set led turn on
        else:
            led.value(0)                #Set led turn off
except:
    pass
```

In this project, we use the Pin module of the machine, so before initializing the Pin, we need to import this module first.

```
from machine import Pin
```

In the circuit connection, LED and Button are connected with GP15 and GP13 respectively, so define LED and button as 0 and 3 respectively.

```
led = Pin(15, Pin.OUT)
button = Pin(13, Pin.IN)    #Create button object from Pin13, Set Pin13 to input
```

Read the pin state of button with value() function. Press the button switch, the function returns low level and the result of "if" is true, and then LED will be turned ON; Otherwise, LED is turned OFF.

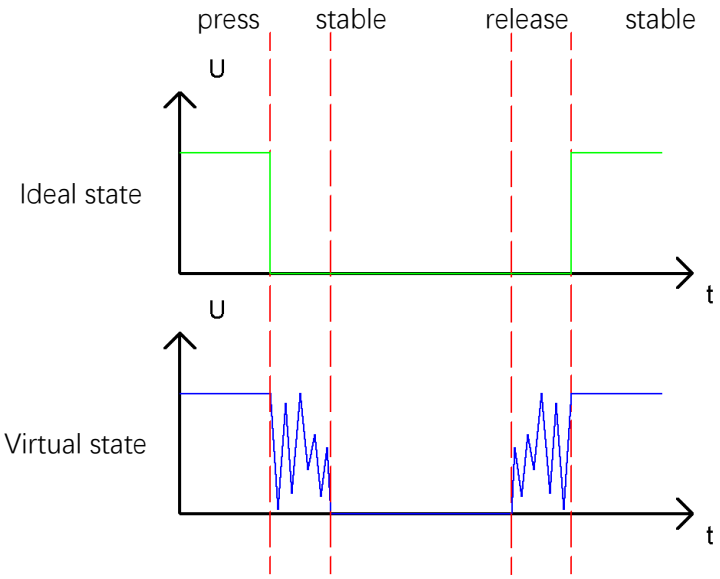
```
        if not button.value():
            led.value(1)                #Set led turn on
        else:
            led.value(0)                #Set led turn off
```

Project 2.2 MINI table lamp

We will also use a Push Button Switch, LED and Raspberry Pi Pico to make a MINI Table Lamp but this will function differently: Press the button, the LED will turn ON, and pressing the button again, the LED turns OFF. The ON switch action is no longer momentary (like a door bell) but remains ON without needing to continually press on the Button Switch.
First, let us learn something about the push button switch.

Debounce for Push Button

When a Momentary Push Button Switch is pressed, it will not change from one state to another state immediately. Due to tiny mechanical vibrations, there will be a short period of continuous buffeting before it completely reaches another state too fast for Humans to detect but not for computer microcontrollers. The same is true when the push button switch is released. This unwanted phenomenon is known as “bounce”.



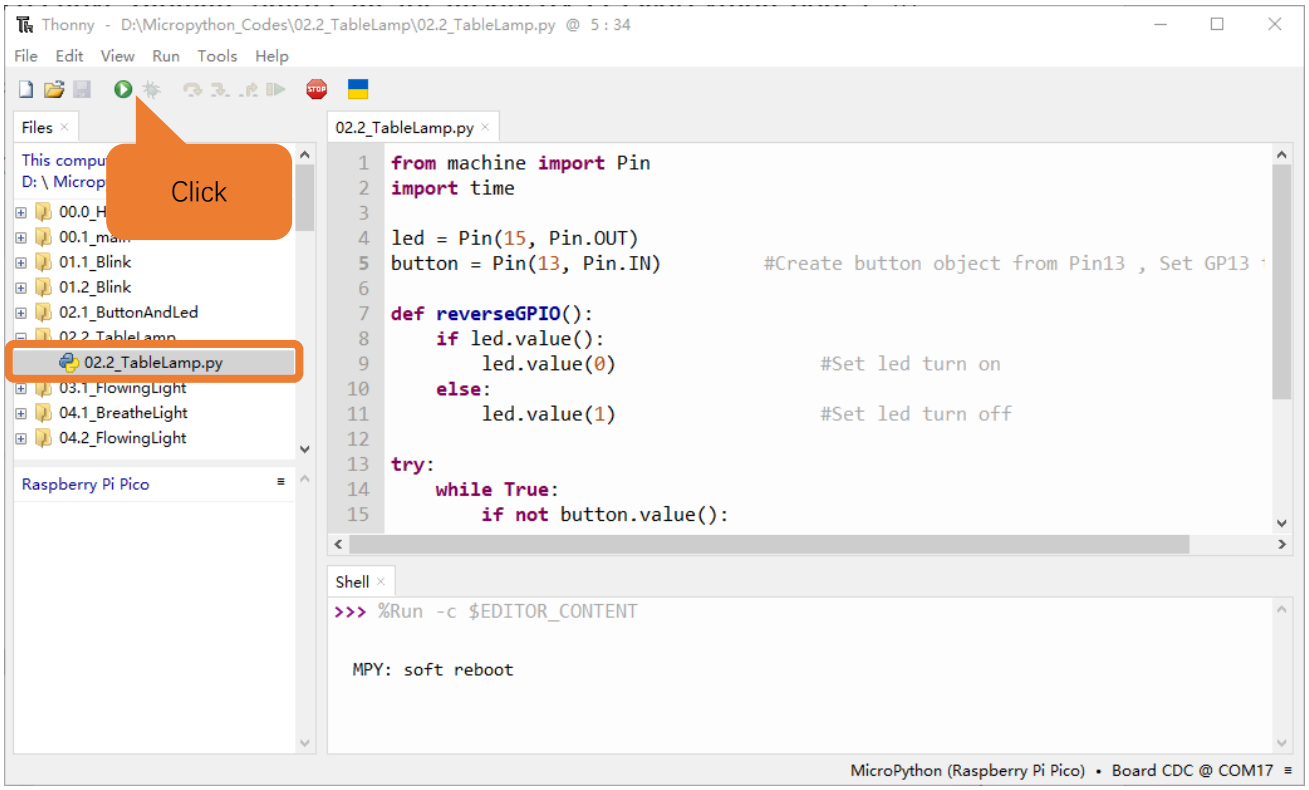
Therefore, if we can directly detect the state of the Push Button Switch, there are multiple pressing and releasing actions in one pressing cycle. This buffeting will mislead the high-speed operation of the microcontroller to cause many false decisions. Therefore, we need to eliminate the impact of buffeting. Our solution: to judge the state of the button multiple times. Only when the button state is stable (consistent) over a period of time, can it indicate that the button is actually in the ON state (being pressed).
This project needs the same components and circuits as we used in the previous section.

Code

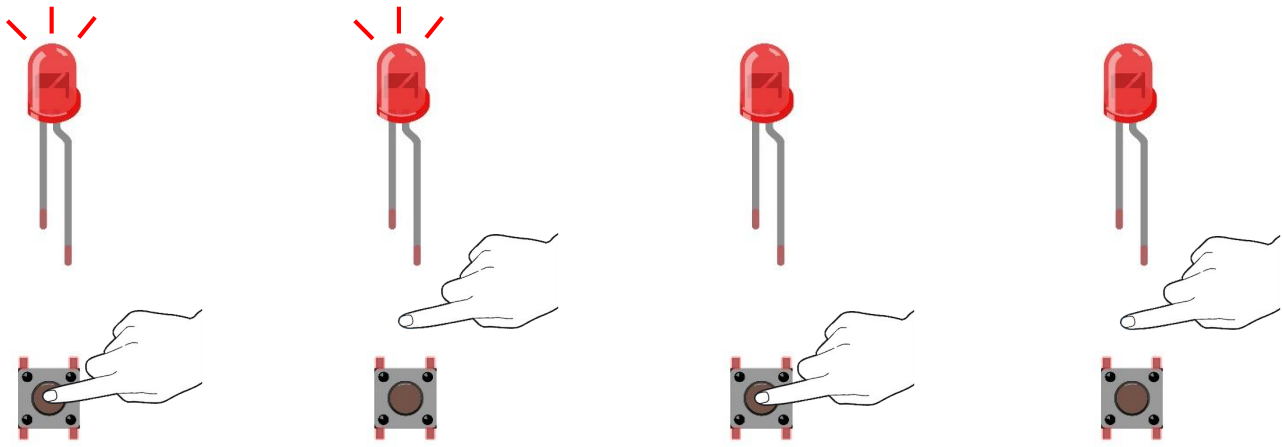
02.2_Tablelamp

Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "02.2_TableLamp"and double click "02.2_TableLamp.py".



Click "Run current script" shown in the box of the above illustration, press the push button switch, LED turns ON; press it again, LED turns OFF. Press Ctrl+C or click "Stop/Restart backend" to exit file.



If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
from machine import Pin
import time

led = Pin(15, Pin.OUT)
button = Pin(13, Pin.IN)    #Create button object from Pin13 , Set Pin13 to input

def reverseGPIO():
    if led.value():
        led.value(0)                    #Set led turn on
    else:
        led.value(1)                    #Set led turn off

try:
    while True:
        if not button.value():
            time.sleep_ms(20)
            if not button.value():
                reverseGPIO()
                while not button.value():
                    time.sleep_ms(20)
except:
    pass
```

When the button is detected to be pressed, delay 20ms to avoid the effect of bounce, and then check whether the button has been pressed again. If so, the conditional statement will be executed, otherwise it will not be executed.

```
while True:
  if not button.value():
      time.sleep_ms(20)
      if not button.value():
          reverseGPIO()
          while not button.value():
              time.sleep_ms(20)
```

Customize a function and name it reverseGPIO(), which reverses the output level of the LED.

```
def reverseGPIO():
    if led.value():
        led.value(0)                    #Set led turn on
    else:
        led.value(1)                    #Set led turn off
```

Chapter 3 LED Bar

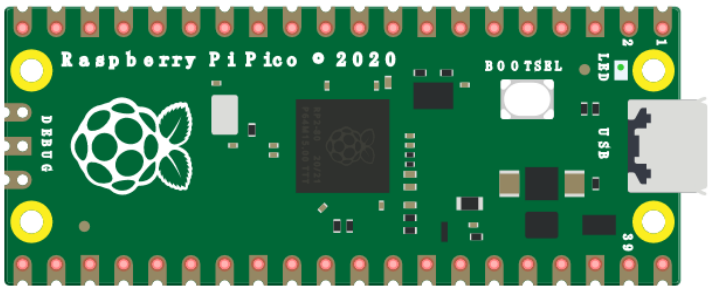
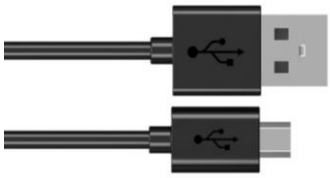
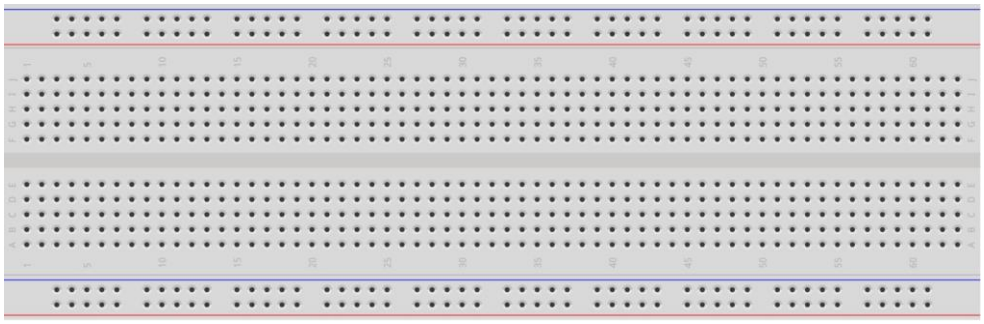
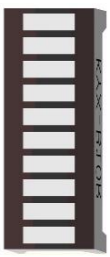
Note: Raspberry Pi Pico, Raspberry Pi Pico W, Raspberry Pi Pico 2 and Raspberry Pi Pico 2W only differ by one wireless function, and are almost identical in other aspects. In this tutorial, except for the wireless function, other parts use Raspberry Pi Pico's map for tutorial demonstration.

We have learned how to control an LED blinking; next, we will learn how to control a number of LEDs.

Project 3.1 Flowing Light

In this project, we use a number of LEDs to make a flowing light.

Component List

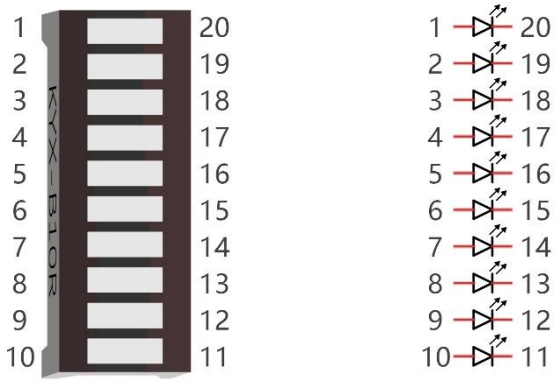
<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td>USB cable x1
  </td></tr>
<tr><td colspan="3">Breadboard x1
  </td></tr>
<tr><td>Jumper
  </td><td>LED bar graph x1
  </td><td>Resistor 220Ω x10
  </td></tr>
</table>

Component knowledge

Let us learn about the basic features of these components to use and understand them better.

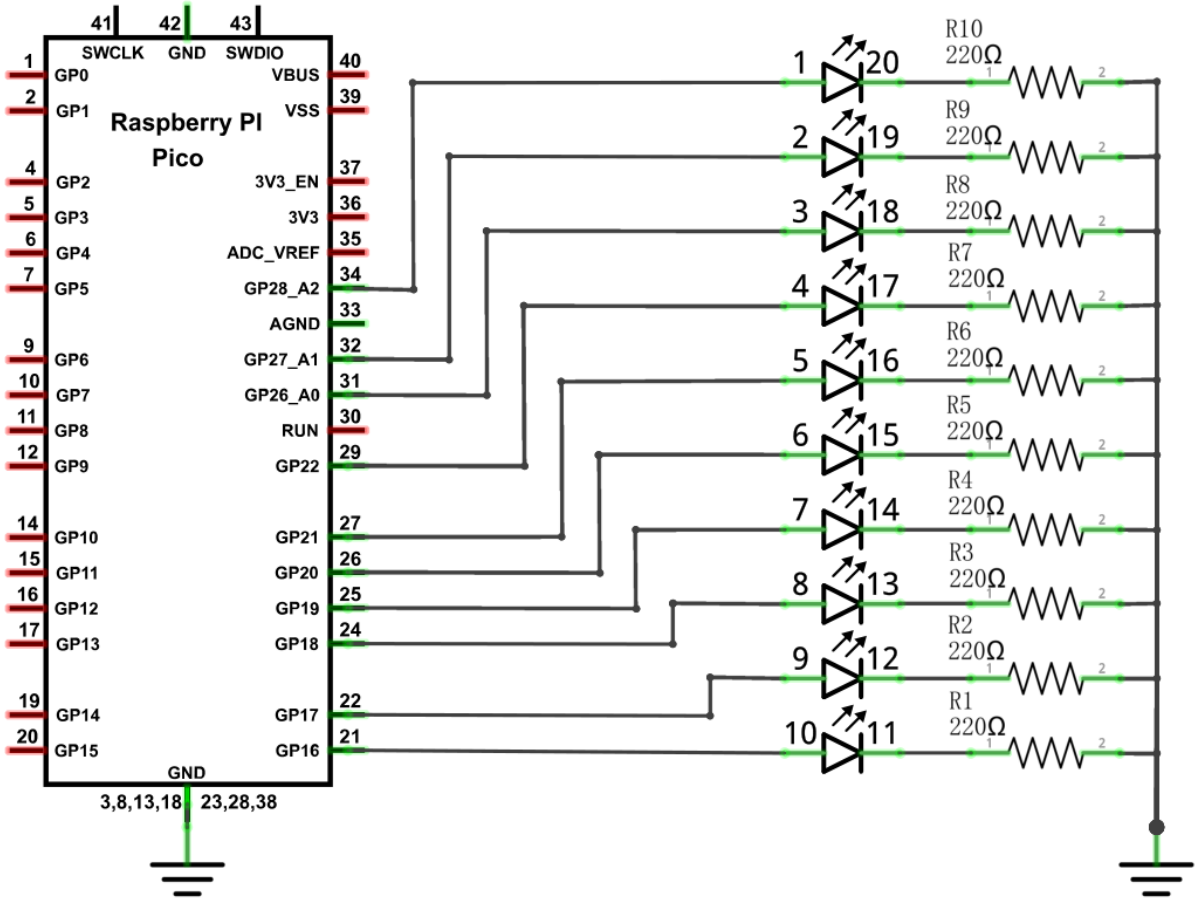
LED bar

A Bar Graph LED has 10 LEDs integrated into one compact component. The two rows of pins at its bottom are paired to identify each LED like the single LED used earlier.

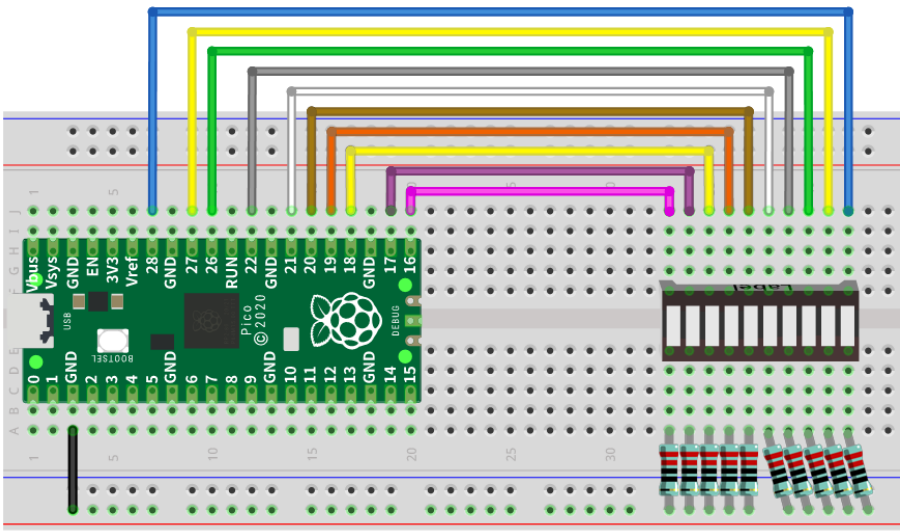


Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Note: To help users have a better experience when doing the projects, we have made some modifications to Pico's simulation diagram. Please note that there are certain differences between the simulation diagram and the actual board to avoid misunderstanding.

If LED bar does not work, try to rotate LED bar for 180°. The label is random.

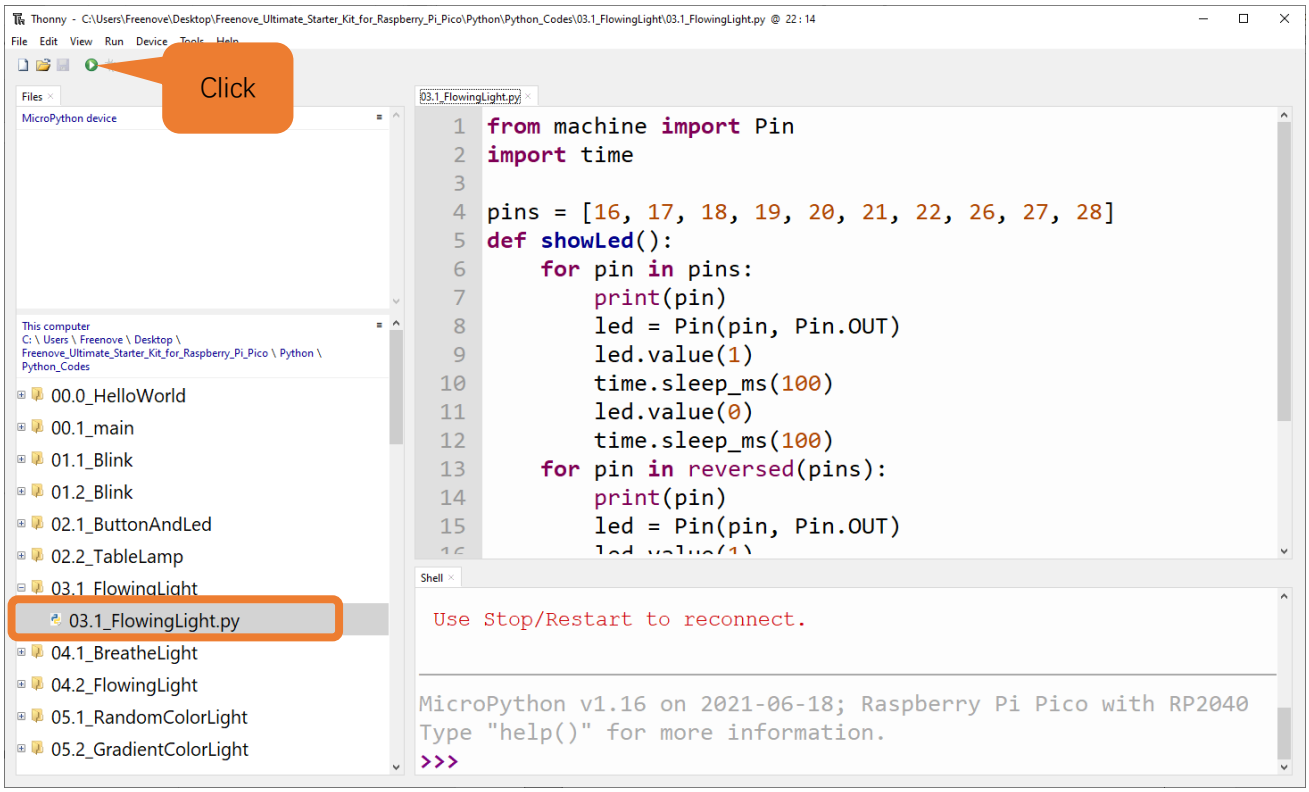
Code

This project is designed to make a flowing water lamp. Which are these actions: First turn LED #1 ON, and then turn it OFF. Then turn LED #2 ON, and then turn it OFF... and repeat the same to all 10 LEDs until the last LED is turns OFF. This process is repeated to achieve the "movements" of flowing water.

03.1_FlowingLight

Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "03.1_FlowingLight" and double click "03.1_FlowingLight.py".



Click "Run current script" shown in the box above, LED Bar Graph will light up from left to right and then back from right to left. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
1  from machine import Pin
2  import time
3
4  pins = [16, 17, 18, 19, 20, 21, 22, 26, 27, 28]
5  def showLed():
6      for pin in pins:
7          led = Pin(pin, Pin.OUT)
8          led.value(1)
9          time.sleep_ms(100)
10         led.value(0)
11         time.sleep_ms(100)
12     for pin in reversed(pins):
13         led = Pin(pin, Pin.OUT)
14         led.value(1)
15         time.sleep_ms(100)
16         led.value(0)
17         time.sleep_ms(100)
18
19 while True:
20     showLed()
```

Use an array to define 10 GPIO ports connected to LED Bar Graph for easier operation.

```
4  pins = [16, 17, 18, 19, 20, 21, 22, 26, 27, 28]
```

Use two for loops to turn on LEDs separately from left to right and then back from right to left.

```
5  def showLed():
6      for pin in pins:
7          led = Pin(pin, Pin.OUT)
8          led.value(1)
9          time.sleep_ms(100)
10         led.value(0)
11         time.sleep_ms(100)
12     for pin in reversed(pins):
13         led = Pin(pin, Pin.OUT)
14         led.value(1)
15         time.sleep_ms(100)
16         led.value(0)
17         time.sleep_ms(100)
```

Reference

for	
For loop is used to execute a program endlessly and iterate in the order of items (a list or a string) in the sequence. Commonly used: “for pin in pins” Pins is a list of elements that are iterated over by a for loop and assigned to pin each time. “for i in range(start, end, num: 1)” start: initial value, from which the for loop starts counting. The default initial value is 0. For example, range(5) equals to range(0, 5). end: the value with which the function ends. For loop counts till it arrives at this value, but this value isn’t included in the counting. num: Num is automatically added each time to the data. The default value is 1. range() function returns a sequence number which is assigned to I by for loop.	

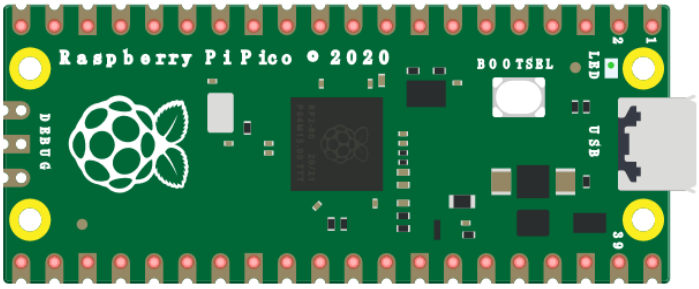
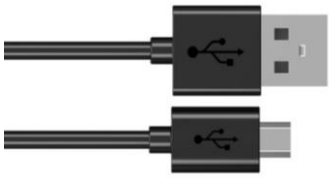
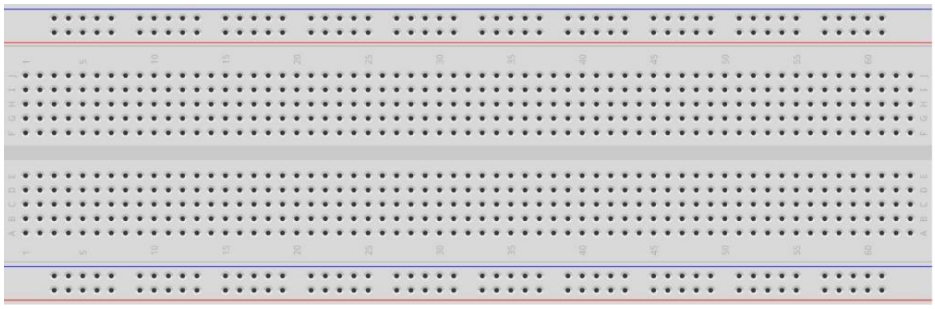
Chapter 4 Analog & PWM

In previous study, we have known that one button has two states: pressed and released, and LED has light-on/off state, then how to enter a middle state? How to output an intermediate state to let LED "semi bright"? That is what we are going to learn.
First, let us learn how to control the brightness of an LED.

Project 4.1 Breathing LED

Breathing light, that is, LED is turned from off to on gradually and gradually from on to off, just like "breathing". So, how to control the brightness of an LED? We will use PWM to achieve this target.

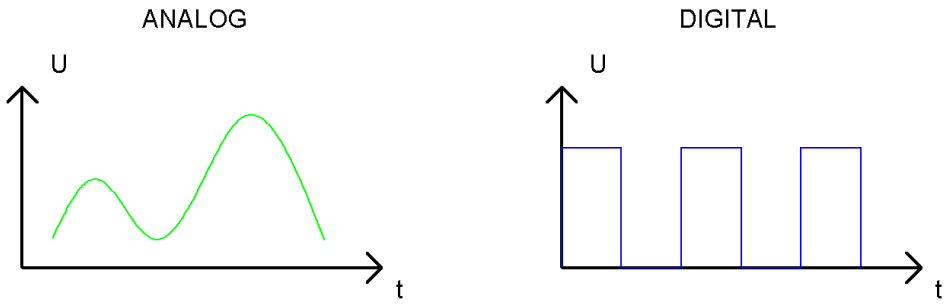
Component List

<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td>USB cable x1
  </td></tr>
<tr><td colspan="3">Breadboard x1
  </td></tr>
<tr><td>LED x1
  </td><td>Resistor 220Ω x1
  </td><td>Jumper
  </td></tr>
</table>

Related knowledge

Analog & Digital

An Analog Signal is a continuous signal in both time and value. On the contrary, a Digital Signal or discrete-time signal is a time series consisting of a sequence of quantities. Most signals in life are analog signals. A familiar example of an Analog Signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0℃ to 10℃. However, Digital Signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.



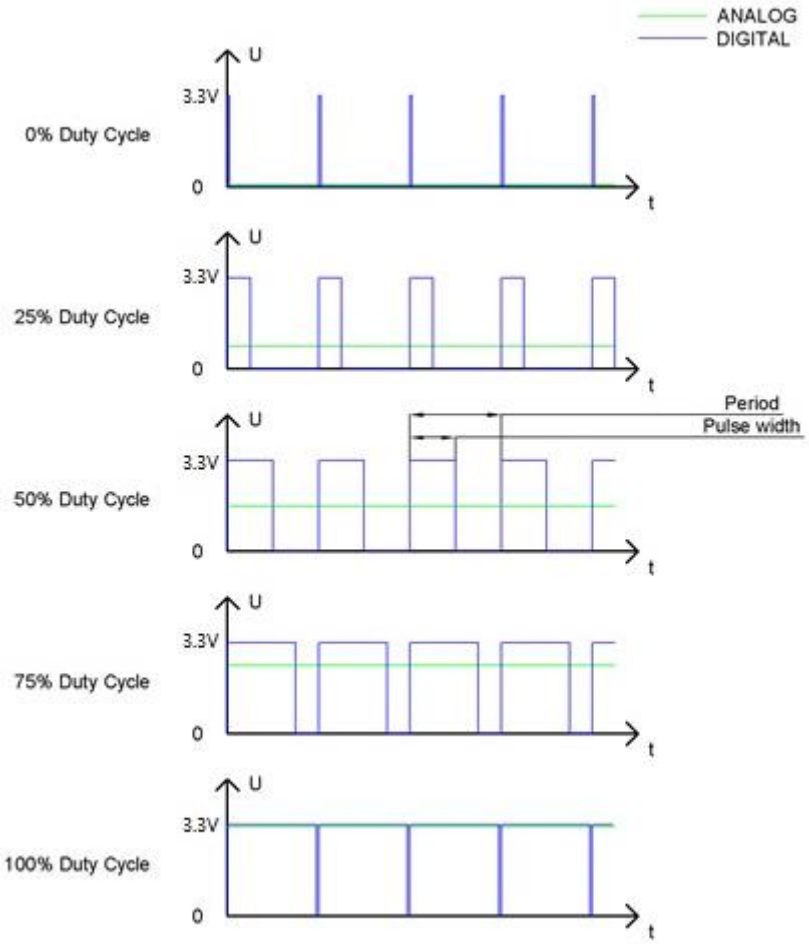
In practical application, we often use binary as the digital signal, which is a series of 0's and 1's. Since a binary signal only has two values (0 or 1), it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse-Width Modulation, is a very effective method for using digital signals to control analog circuits. Common processors cannot directly output analog signals. PWM technology makes it very convenient to achieve this conversion (translation of digital to analog signals)

PWM technology uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (Note: the reciprocal of the period is frequency). The time of high level outputs are generally called "pulse width", and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform.

The longer the output of high levels last, the longer the duty cycle and the higher the corresponding voltage in the analog signal will be. The following figures show how the analog signal voltages vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.
It is evident from the above that PWM is not real analog, and the effective value of the voltage is equivalent to the corresponding analog. Therefore, we can control the output power of the LED and other output modules to achieve different effects.

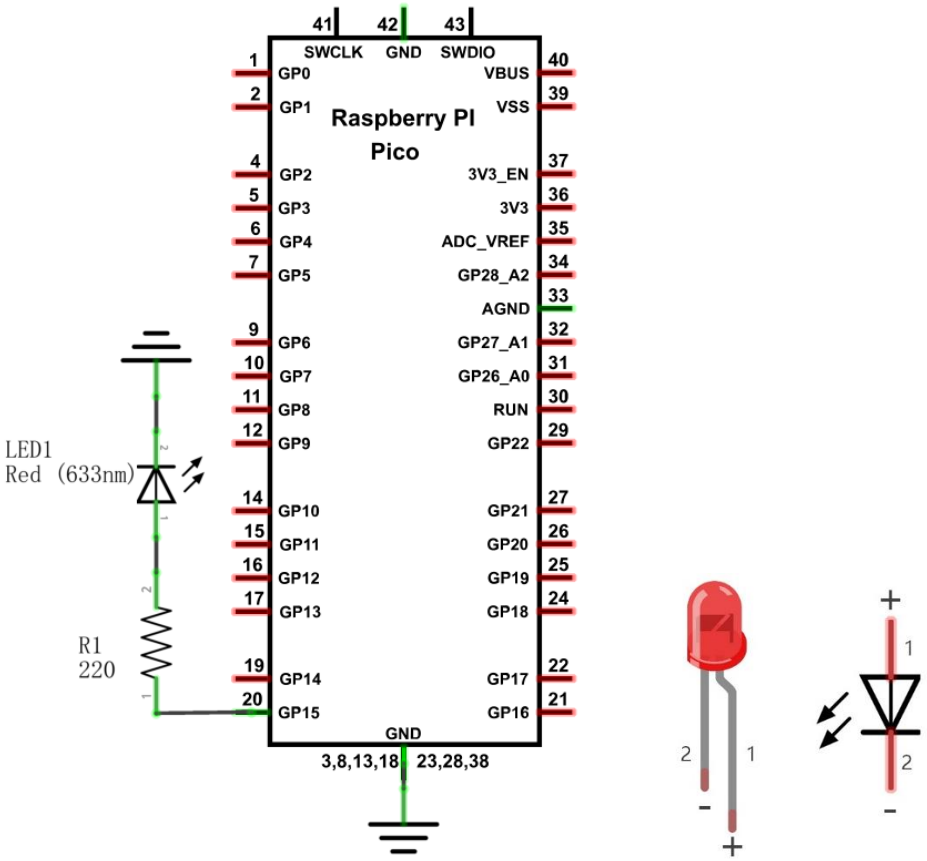
Raspberry Pi Pico and PWM

Raspberry Pi Pico has 16 PWM channels, each of which can control frequency and duty cycle independently, with the clock frequency ranges from 7Hz to 125MHz. Every pin on Raspberry Pi Pico can be configured as PWM output.

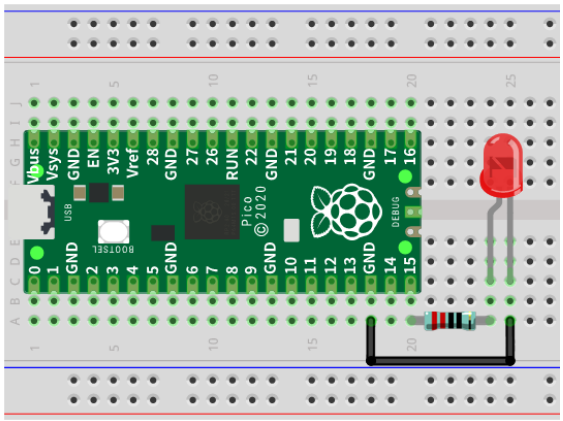
Circuit

This circuit is the same as the one in project Blink.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



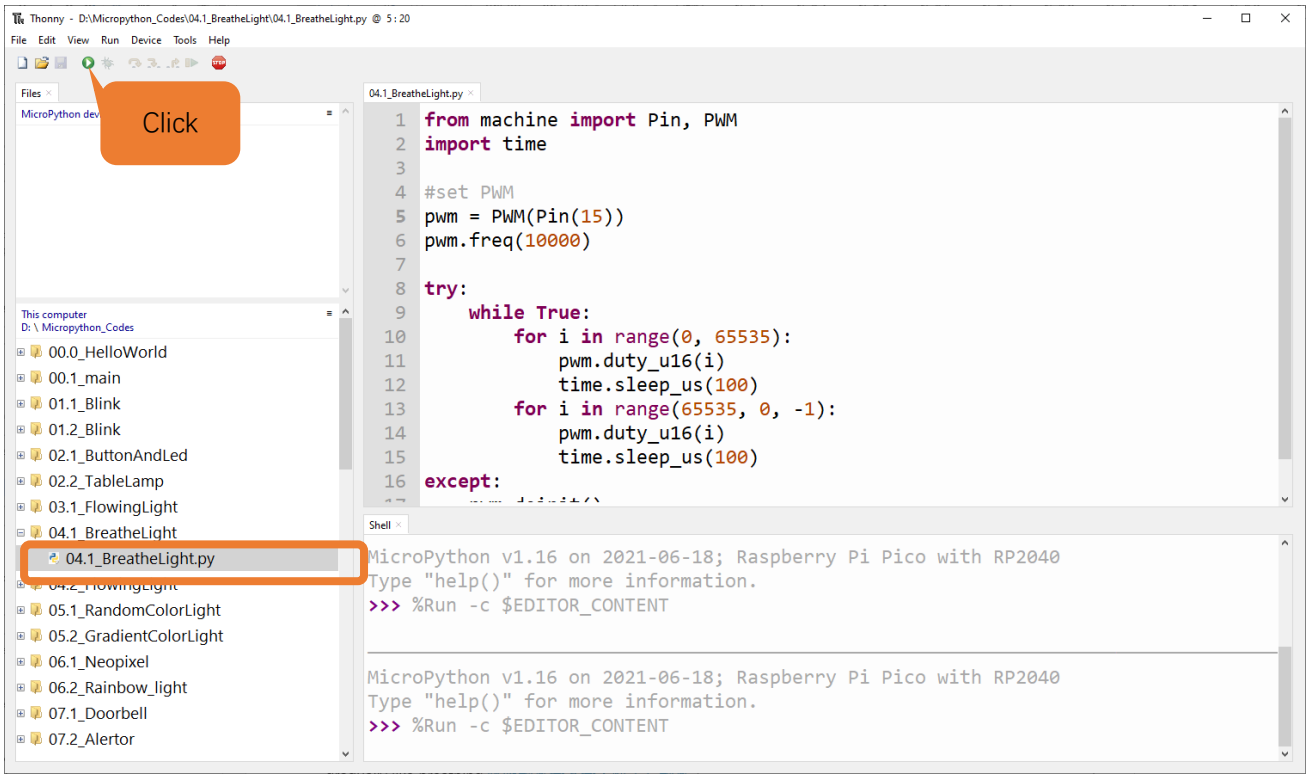
Note: To help users have a better experience when doing the projects, we have made some modifications to Pico's simulation diagram. Please note that there are certain differences between the simulation diagram and the actual board to avoid misunderstanding.

Code

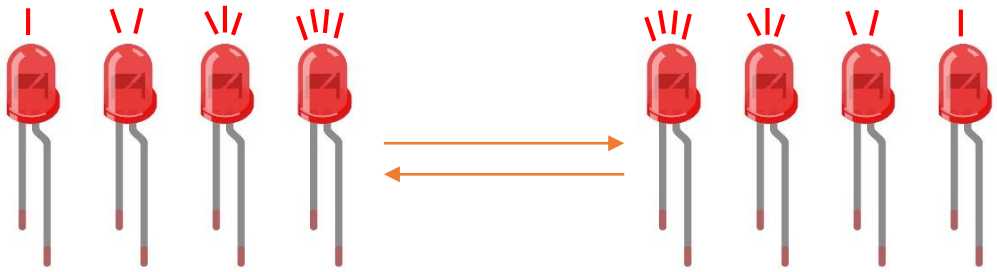
This project is designed to make PWM output GP15 with pulse width increasing from 0% to 100%, and then reducing from 100% to 0% gradually.

Open "Thonny", click"This computer" → "D:" → "Micropython_Codes" → "04.1_BreatheLight" and double click "04.1_BreatheLight.py".

04.1_BreatheLight



Click "Run current script" and you will see that LED is turned from ON to OFF and then back from OFF to ON gradually like breathing. Press Ctrl+C or click "Stop/Restart backend" to exit program.



The following is the program code:

```
from machine import Pin, PWM
import time

#set PWM
pwm = PWM(Pin(15))
pwm.freq(10000)

try:
    while True:
        for i in range(0, 65535):
            pwm.duty_u16(i)
            time.sleep_us(100)
        for i in range(65535, 0, -1):
            pwm.duty_u16(i)
            time.sleep_us(100)
except:
    pwm.deinit()
```

Create a PWM object and configure GP15 as PWM output pin. Call freq() to set PWM output frequency of GP15 to 10000Hz.

```
pwm = PWM(Pin(15))
pwm.freq(10000)
```

The range of duty cycle is 0-65535, so we use the first for loop to control PWM to change the duty cycle value, making PWM output 0% -100%; Use the second for loop to make PWM output 100%-0%.

```
        for i in range(0, 65535):
            pwm.duty_u16(i)
            time.sleep_us(100)
        for i in range(65535, 0, -1):
            pwm.duty_u16(i)
            time.sleep_us(100)
```

Each time PWM is used, the hardware Timer will be turned ON to cooperate it. Therefore, after each use of PWM, deinit() needs to be called to turned OFF the timer. Otherwise, the PWM may fail to work next time.

```
    pwm.deinit()
```

Reference

Class PWM(pin)

Before each use of PWM module, please add the statement "**from machine import PWM**" to the top of the python file.

pin: PWM pins are supported, such as GP(0~22), GP(25), GP(26~28).

PWM.freq(freq_val): the function is used to set PWM frequency and returns nothing; when there is no parameter, the function obtains and returns PWM frequency.

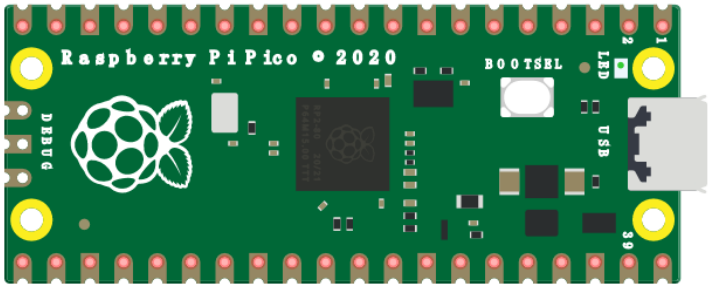
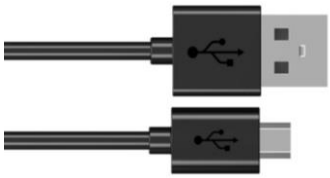
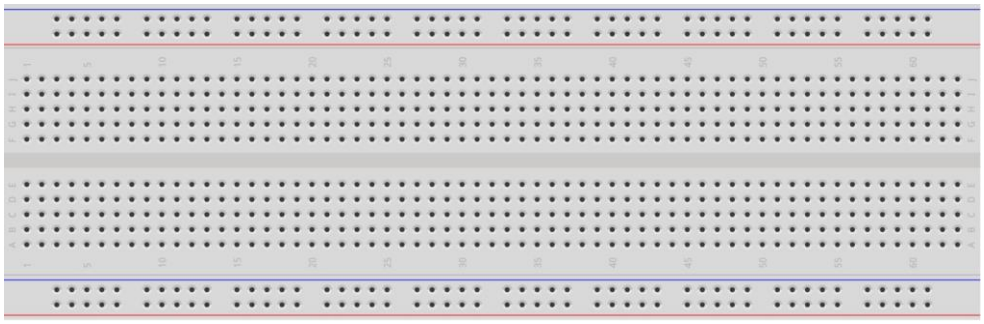
PWM.duty_u16(duty_val): the function is used to set PWM duty cycle, among which, duty_val ranges from 0 to 65535. If there is no parameter, the function returns to currently set duty cycle. If duty cycle has not yet been set, it returns 0.

PWM.deinit(): Turn OFF PWM.

Project 4.2 Meteor Flowing Light

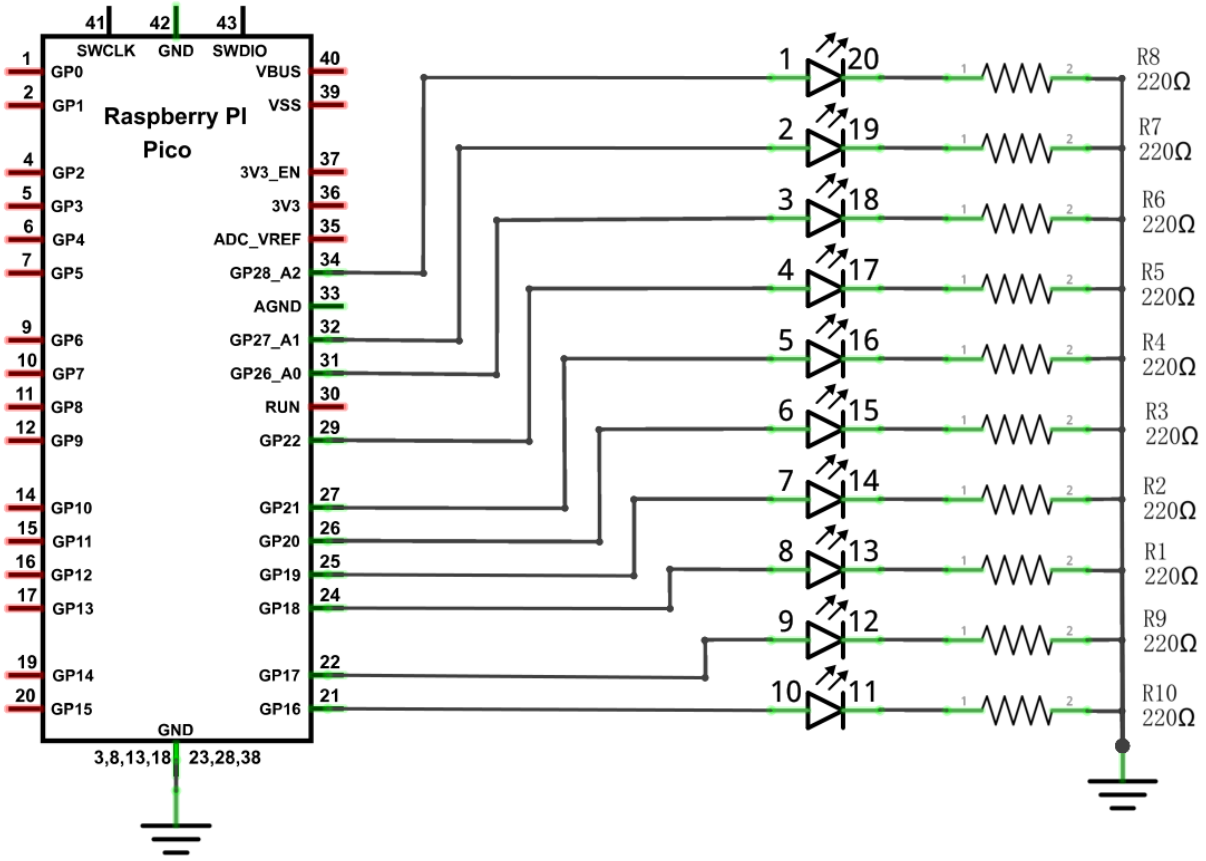
Having learned about PWM, we can use it to control LED Bar Graph and realize a cooler Flowing Light.

Component List

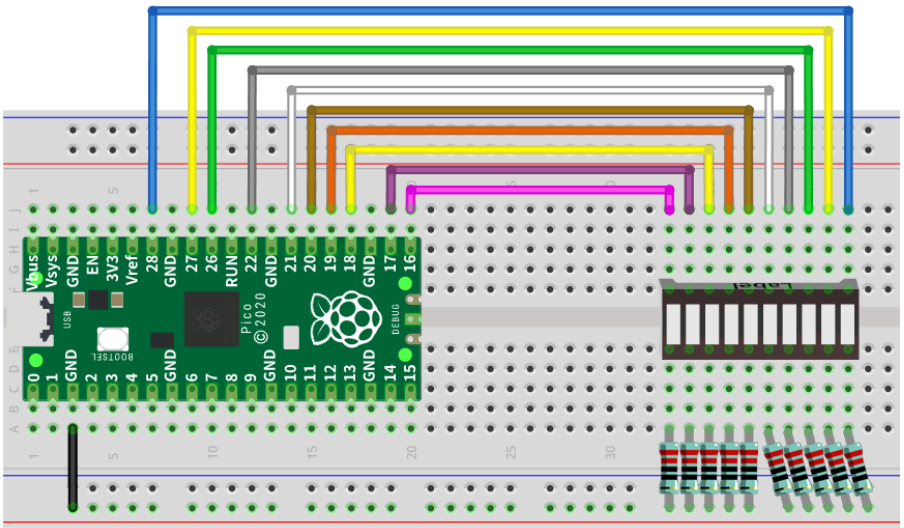
Raspberry Pi Pico x1 	USB cable x1 	
Breadboard x1 		
Jumper 	LED bar graph x1 	Resistor 220Ω x10 

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Note: To help users have a better experience when doing the projects, we have made some modifications to Pico's simulation diagram. Please note that there are certain differences between the simulation diagram and the actual board to avoid misunderstanding.

If LED bar does not work, try rotating it for 180°. The label is random.

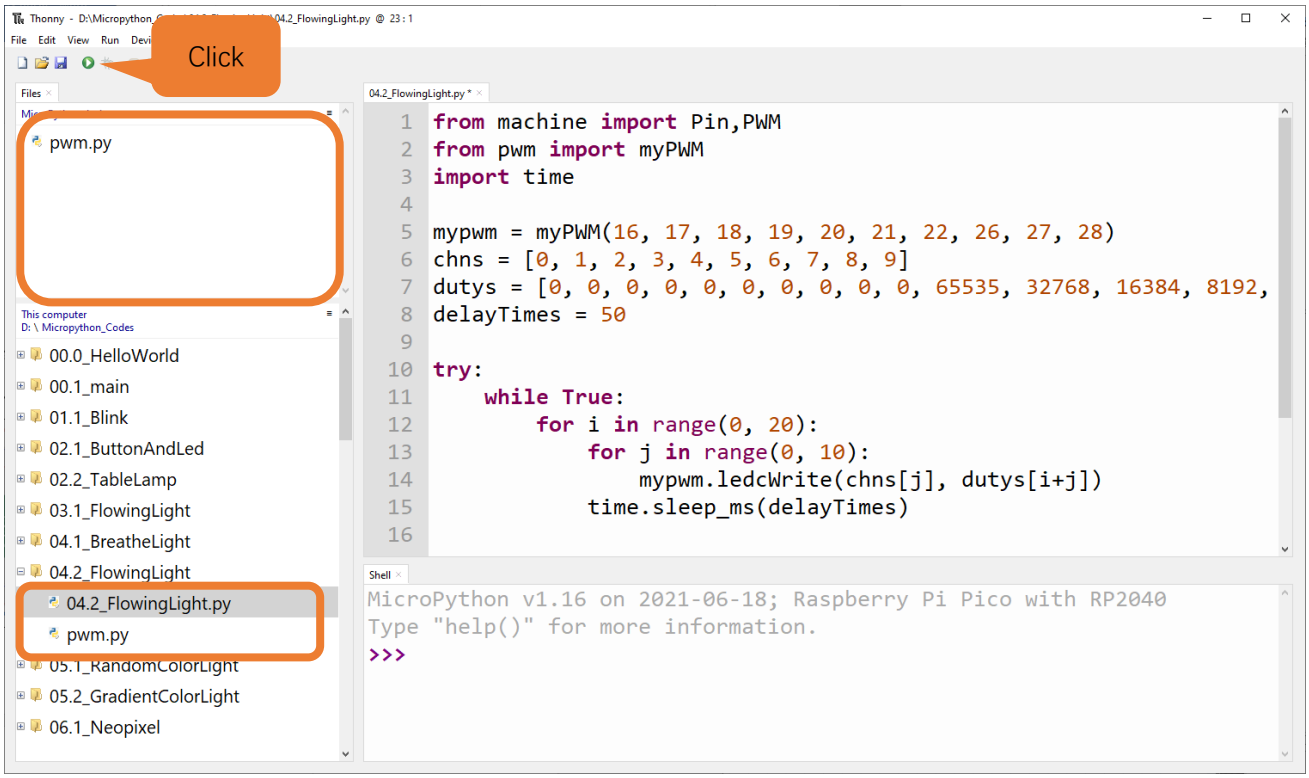
Any concerns? ✉ support@freenove.com

Code

Flowing Light with tail was implemented with PWM.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "04.2_FlowingLight". Select "pwm.py", right click to select "Upload to /", wait for "pwm.py" to be uploaded to Raspberry Pi Pico and then double click "04.2_FlowingLight.py".

04.2_FlowingLight



Click "Run current script", and LED Bar Graph will gradually light up and out from left to right, then light up and out from right to left. Press Ctrl+C or click "Stop/Restart backend" to exit program.

The following is the program code:

```
1  from machine import Pin,PWM
2  from pwm import myPWM
3  import time
4
5  mypwm = myPWM(16, 17, 18, 19, 20, 21, 22, 26, 27, 28)
6  chns = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
7  dutys = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 65535, 32768, 16384, 8192, 4096, 2048, 1024, 512, 256,
   128, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
8  delayTimes = 50
9
10 try:
11     while True:
12         for i in range(0, 20):
```

```
            for j in range(0, 10):
                mypwm.ledcWrite(chns[j], dutys[i+j])
            time.sleep_ms(delayTimes)

        for i in range(0, 20):
            for j in range(0, 10):
                mypwm.ledcWrite(chns[9-j], dutys[i+j])
            time.sleep_ms(delayTimes)
except:
    mypwm.deinit()
```

Import the object myPWM from pwm.py and set corresponding pins for PWM channel.

```
2   from pwm import myPWM
…   …
5   mypwm = myPWM(16, 17, 18, 19, 20, 21, 22, 26, 27, 28)
```

Create an object for myPWM and configure 10 PWM output pins. Define 10 PWM channels and 30 pulse width values.

```
5   mypwm = myPWM(16, 17, 18, 19, 20, 21, 22, 26, 27, 28)
6   chns = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
7   dutys = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 65535, 32768, 16384, 8192, 4096, 2048, 1024, 512, 256,
    128, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Call ledcWrite() to set duty cycle dutys[i+j] for the chns[j] channel of PWM.

```
14  mypwm.ledcWrite(chns[j], dutys[i+j])
```

Turn OFF the PWM of the object myPWM.

```
22  mypwm.deinit()
```

In the code, a nesting of two for loops are used to achieve this effect.

```
12      for i in range(0, 20):
13          for j in range(0, 10):
14              mypwm.ledcWrite(chns[j], dutys[i+j])
15          time.sleep_ms(delayTimes)
16
17      for i in range(0, 20):
18          for j in range(0, 10):
19              mypwm.ledcWrite(chns[9 -j], dutys[i+j])
20          time.sleep_ms(delayTimes)
```

In the main function, a nested for loop is used to control the pulse width of the PWM. Every time i in the first for loop increases by 1, the LED Bar Graph will move one grid, and gradually change according to the value in the array dutys. As shown in the following table, the value in the second row is the value of the array dutys, and the 10 green grids in each row below represent the 10 LEDs on the LED Bar Graph. Each time i increases by 1, the value of the LED Bar Graph will move to the right by one grid, and when it reaches the end, it will move from the end to the starting point, achieving the desired effect.

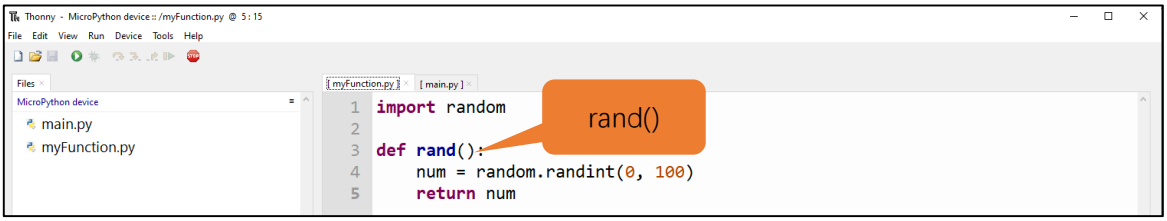
0	1	2	3	…	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	…	28	29	30
di	0	0	0	…	0	0	0	0	65535	32768	16384	8192	4096	2048	1024	512	256	128	0	0	0	0	0	0
0																								
1																								
…																								
18																								
19																								
20																								

How to import a custom python module

Each Python file, as long as it is stored on the file system of Raspberry Pi Pico, is a module. To import a custom module, the module file needs to be located in the MicroPython environment variable path or in the same path as the currently running program.

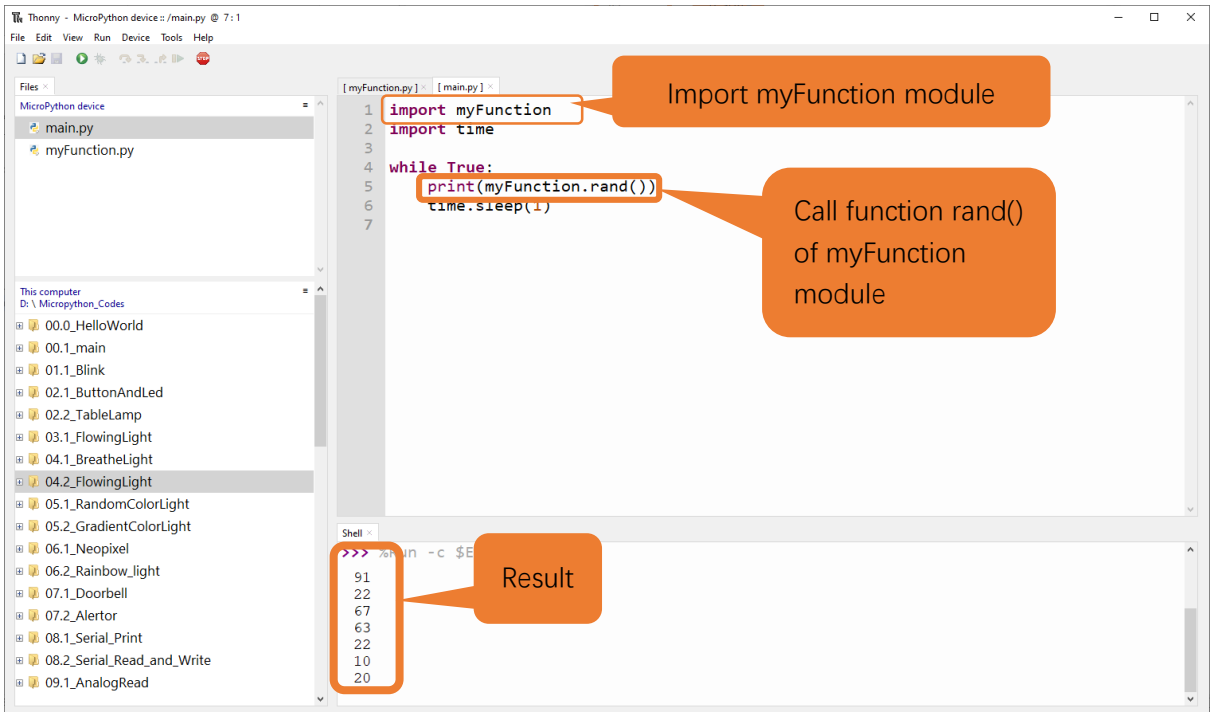
Code

First, customize a python module "myFunction.py". Create a new py file and name it "myFunction.py". Write code to it and save it to Raspberry Pi Pico.



rand() function randomly creates an integer ranging from 0 to 99.

Second, import myFunction module "myFunction" to main.py.



The following is the program code:

1. myFunction.py

```
1  import random
2
3  def rand():
4      num = random.randint(0, 100)
5      return num
```

Import random module.

```
1  import random
```

Call randit() function in random module to randomly generate an integer at the range of 0-99 and assign it to num variable.

```
4      num = random.randint(0, 100)
```

2. main.py

```
1  import myFunction
2  import time
3
4  while True:
5      print(myFunction.rand())
6      time.sleep(1)
```

Import myFunction module "myFunction" to main.py.

```
1  import myFunction
```

Call rand() in myFunction module.

```
5      print(myFunction.rand())
```

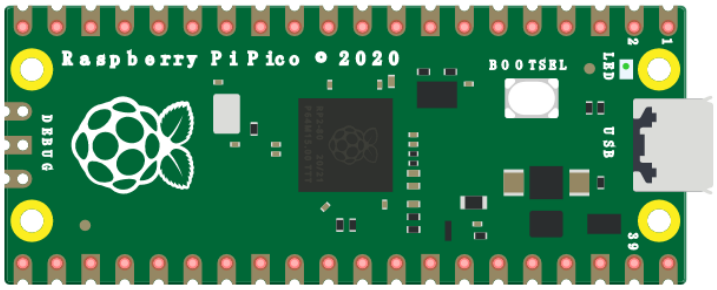
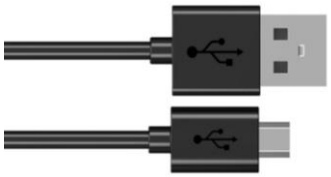
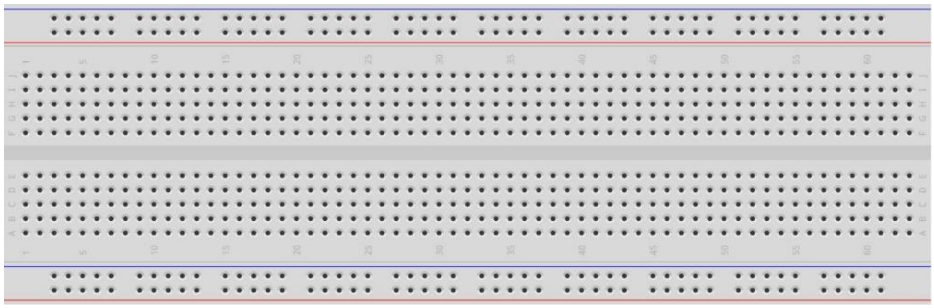
Chapter 5 RGBLED

In this chapter, we will learn how to control an RGBLED. It can emit different colors of light. Next, we will use RGBLED to make a multicolored light.

Project 5.1 Random Color Light

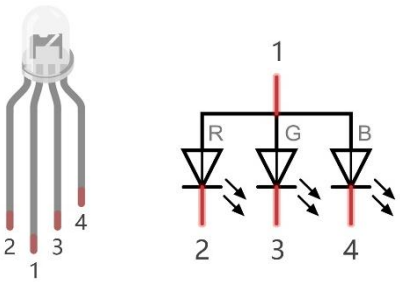
In this project, we will make a multicolored LED and control it to switch different colors automatically.

Component List

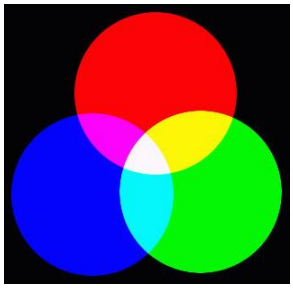
Raspberry Pi Pico x1 		USB cable x1 
Breadboard x1 		
RGBLED x1 	Resistor 220Ω x3 	Jumper 

Related knowledge

RGB LED has integrated three LEDs that can respectively emit red, green and blue light, and it has four pins. The long pin (1) is the common port, that is, 3 LED's positive or negative port. The RGB LED with common positive port and its symbol is shown below. We can make RGB LED emit various colors of light by controlling these three LEDs to emit light with different brightness.



Red, green, and blue light are known as three primary colors. When you combine these three primary-color lights with different brightness, it can produce almost all kinds of visible lights. Computer screens, single pixel of cell phone screen, neon, and etc. are working under this principle.

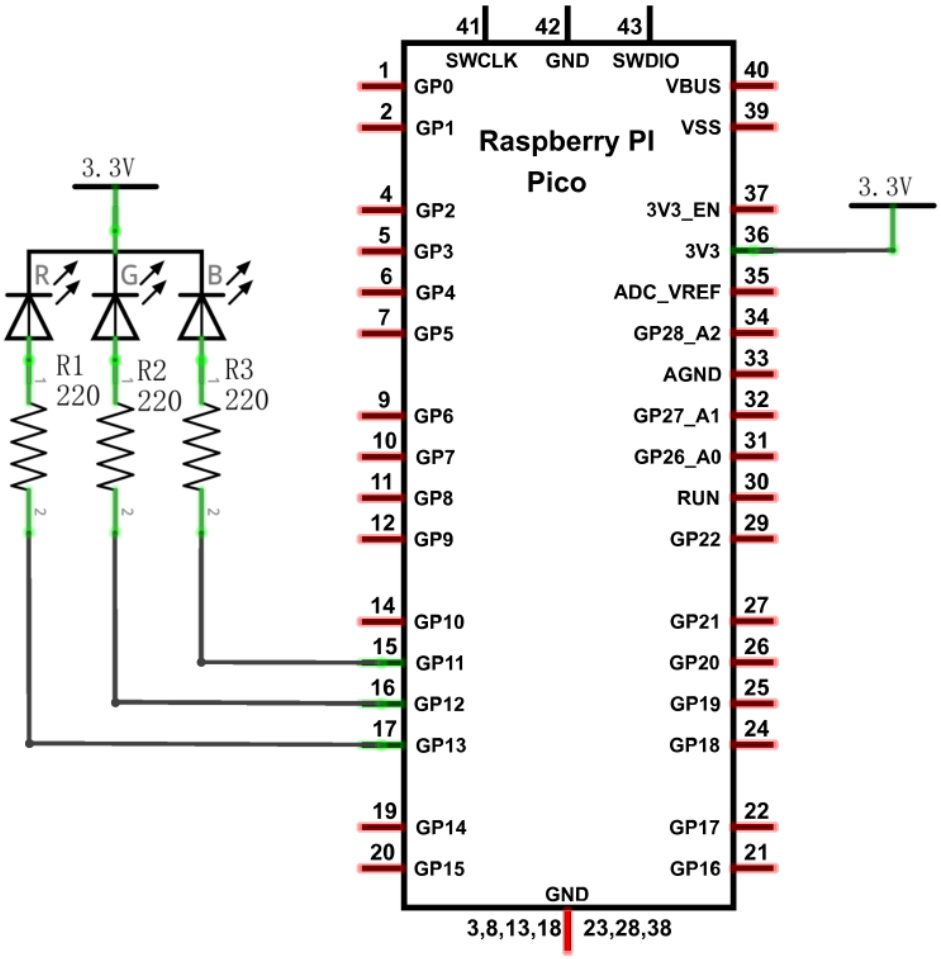


RGB

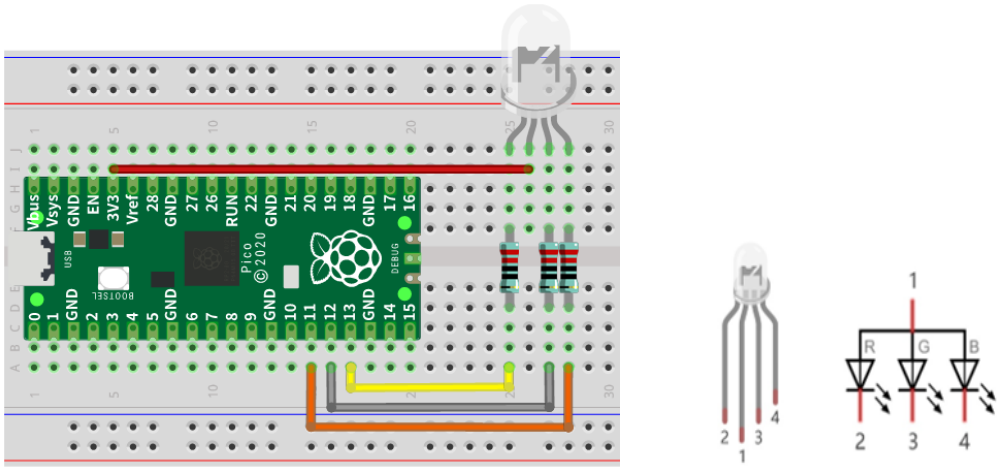
If we use three 10-bit PWM to control the RGBLED, in theory, we can create $2^{10}*2^{10}*2^{10}$= 1,073,741,824(1 billion) colors through different combinations.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



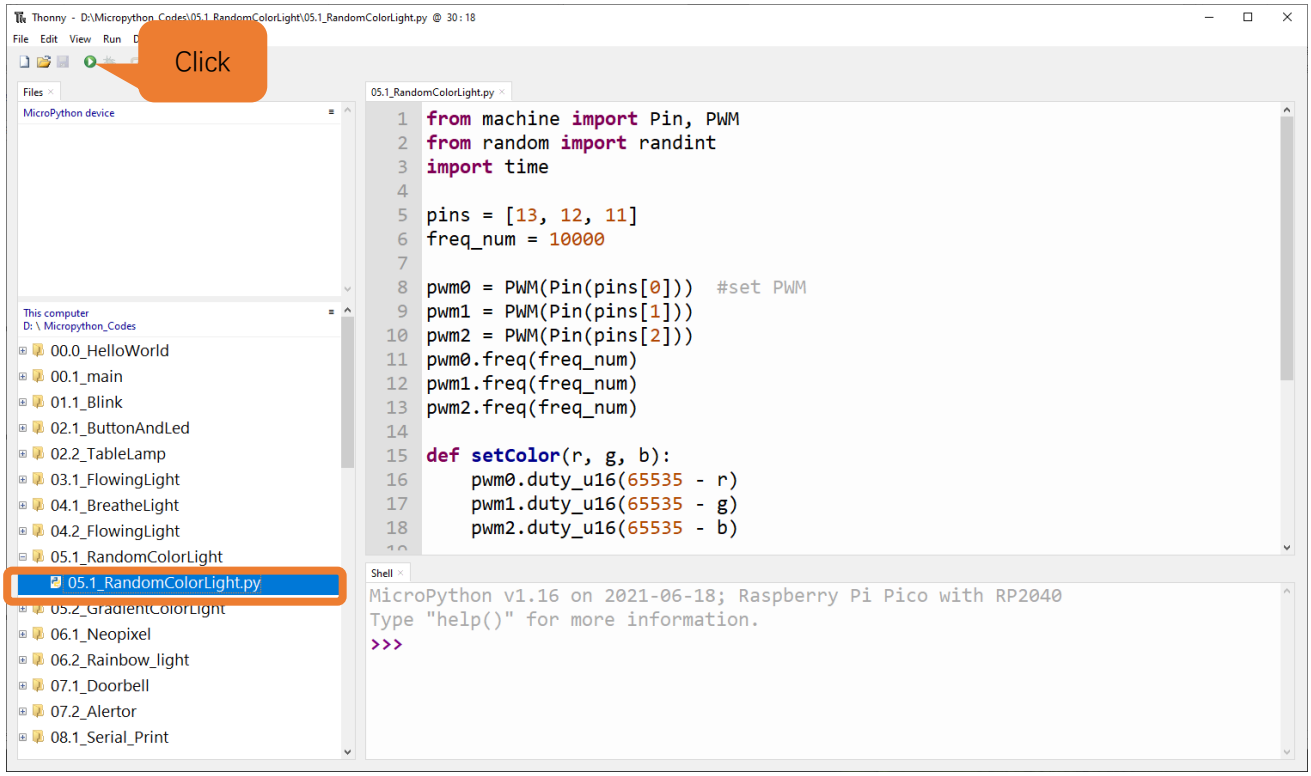
Note: To help users have a better experience when doing the projects, we have made some modifications to Pico's simulation diagram. Please note that there are certain differences between the simulation diagram and the actual board to avoid misunderstanding.

Code

We need to create three PWM channels and use random duty cycle to make random RGBLED color.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "05.1_RandomColorLight"and double click "05.1_RandomColorLight.py".

05.1_RandomColorLight



Click "Run current script", RGBLED begins to display random colors. Press Ctrl+C or click "Stop/Restart backend" to exit program.

If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
from machine import Pin, PWM
from random import randint
import time

pins = [13, 12, 11]
freq_num = 10000

pwm0 = PWM(Pin(pins[0]))  #set PWM
pwm1 = PWM(Pin(pins[1]))
pwm2 = PWM(Pin(pins[2]))
pwm0.freq(freq_num)
pwm1.freq(freq_num)
pwm2.freq(freq_num)

def setColor(r, g, b):
    pwm0.duty_u16(65535 - r)
    pwm1.duty_u16(65535 - g)
    pwm2.duty_u16(65535 - b)

try:
    while True:
        red   = randint(0, 65535)
        green = randint(0, 65535)
        blue  = randint(0, 65535)
        setColor(red, green, blue)
        time.sleep_ms(200)
except:
    pwm0.deinit()
    pwm1.deinit()
    pwm2.deinit()
```

Import Pin, PWM and Random function modules.

```
from machine import Pin, PWM
from random import randint
import time
```

Configure ouput mode of GP11, GP12 and GP13 as PWM output and PWM frequency as 10000Hz.

```
pins = [13, 12, 11]
freq_num = 10000

pwm0 = PWM(Pin(pins[0])) #set PWM
pwm1 = PWM(Pin(pins[1]))
pwm2 = PWM(Pin(pins[2]))
pwm0.freq(freq_num)
pwm1.freq(freq_num)
```

```
13  pwm2.freq(freq_num)
```

Define a function to set the color of RGBLED.

```
15  def setColor(r, g, b):
16      pwm0.duty_u16(65535 - r)
17      pwm1.duty_u16(65535 - g)
18      pwm2.duty_u16(65535 - b)
```

Call random function randint() to generate a random number in the range of 0-65535 and assign the value to red.

```
22      red = randint(0, 65535)
```

Obtain 3 random numbers every 200 milliseconds and call function setColor to make RGBLED display dazzling colors.

```
21      while True:
22          red   = randint(0, 65535)
23          green = randint(0, 65535)
24          blue  = randint(0, 65535)
25          setColor(red, green, blue)
26          time.sleep_ms(200)
```

Reference

Class random

Before each use of the module **random,** please add the statement "**import random**" to the top of Python file.

randint(start, end): Randomly generates an integer between the value of start and end.

start: Starting value in the specified range, which would be included in the range.

end: Ending value in the specified range, which would be included in the range.

random(): Randomly generates a floating point number between 0 and 1.

random.uniform(start, end): Randomly generates a floating point number between the value of start and end.

start: Starting value in the specified range, which would be included in the range.

end: Ending value in the specified range, which would be included in the range.

random.getrandbits(size): Generates an integer with size random bits.

For example:

size = 4, it generates an integer in the range of 0 to 0b1111.

size = 8, it generates an integer in the range of 0 to 0b11111111.

random.randrange(start, end, step): Randomly generates a positive integer in the range from start to end and increment to step.

start: Starting value in the specified range, which would be included in the range.

end: Ending value in the specified range, which would be included in the range.

step: An integer specifying the incrementation.

random.seed(sed): Specifies a random seed, usually being applied in conjunction with other random number generators.

sed: Random seed, a starting point in generating random numbers.

random.choice(obj): Randomly generates an element from the object obj.

obj: list of elements.

Project 5.2 Gradient Color Light

In the previous project, we have mastered the usage of RGBLED, but the random color display is rather stiff. This project will realize a fashionable Light with soft color changes.
Component list, the circuit is the same as the project random color light.
Using a color model, the color changes from 0 to 255 as shown below.

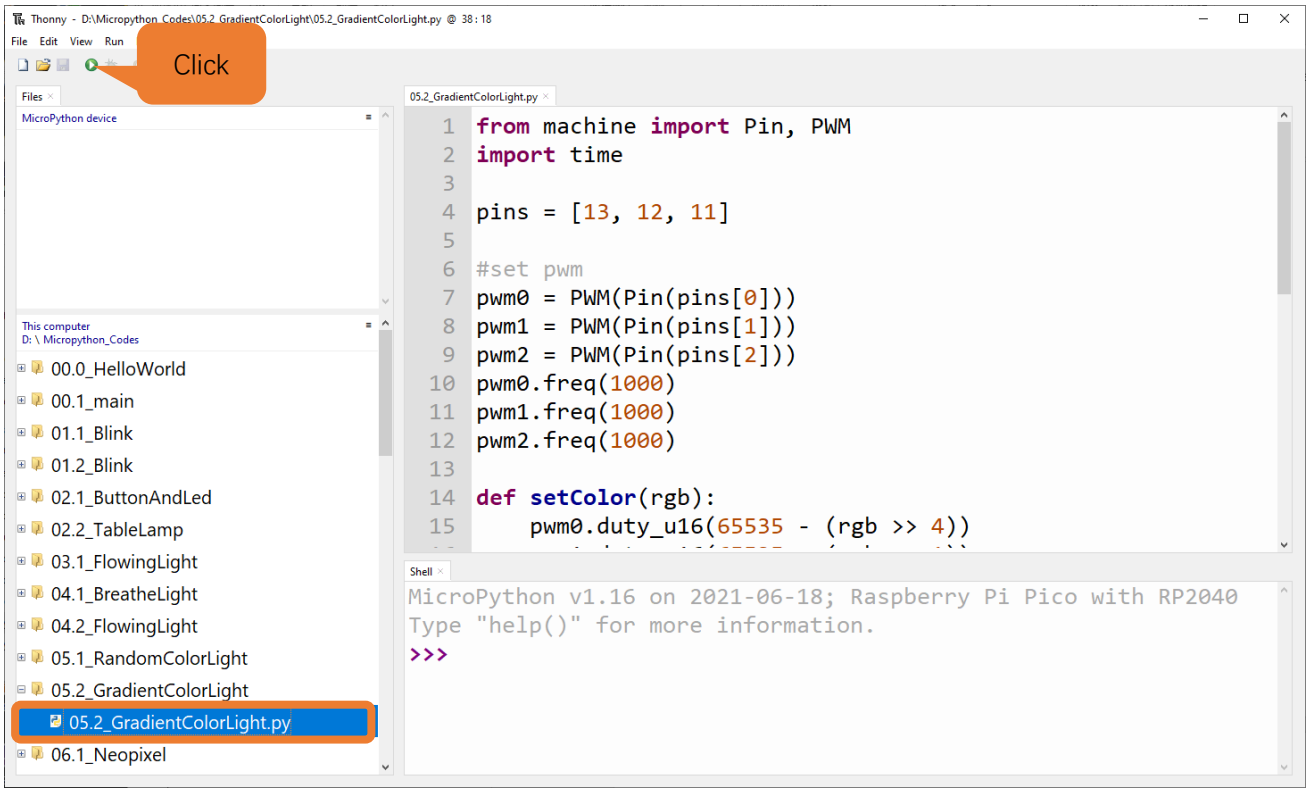


Code

In this code, the color model will be implemented and RGBLED will change colors along the model.

Open "Thonny", click"This computer" → "D:" → "Micropython_Codes" → "05.2_GradientColorLight" and double click "05.2_GradientColorLight.py".

05.2_GradientColorLight



Click "Run current script", and the light emitted by RGBLED will change gradually. Press Ctrl+C or click "Stop/Restart backend" to exit program.

The following is the program code:

```
from machine import Pin, PWM
import time

pins = [13, 12, 11]

#set pwm
pwm0 = PWM(Pin(pins[0]))
pwm1 = PWM(Pin(pins[1]))
pwm2 = PWM(Pin(pins[2]))
pwm0.freq(1000)
pwm1.freq(1000)
pwm2.freq(1000)

def setColor(rgb):
    pwm0.duty_u16(65535 - (rgb >> 4))
    pwm1.duty_u16(65535 - (rgb >> 1))
    pwm2.duty_u16(65535 - (rgb >> 0))

def wheel(pos):
    WheelPos = pos % 65535
    if WheelPos < 21845:
        return (((65535 - WheelPos*3) << 4) | ((WheelPos*3) << 1))
    elif WheelPos >= 21845 and WheelPos < 43690:
        WheelPos -= 21845
        return (((65535 - WheelPos*3) << 1) | (WheelPos*3))
    else :
        WheelPos -= 43690
        return (((WheelPos*3) << 4) | (65535 - WheelPos*3))

try:
    while True:
        for i in range(0, 65535):
            setColor(wheel(i))
            time.sleep_ms(10)
except:
    pwm0.deinit()
    pwm1.deinit()
    pwm2.deinit()
```

In the function **setColor()**, we use a variable to represent the value of RGB, making it more convenient for the passing of parameters. As the range of PWM's duty cycle is 0-65535, which is two to the sixteen power, when spilt, the value of each color channel can be obtained with a simple bitwise operation.

```
14  def setColor(rgb):
15      pwm0.duty_u16(65535 - (rgb >> 4))
16      pwm1.duty_u16(65535 - (rgb >> 1))
17      pwm2.duty_u16(65535 - (rgb >> 0))
```

The function wheel() is a color selection method of the color model introduced earlier. The value range of the parameter pos is 0-65535. The function will return a data containing the duty cycle values of three pins.

```
19  def wheel(pos):
20      WheelPos = pos % 65535
21      if WheelPos < 21845:
22          return (((65535 - WheelPos*3) << 4) | ((WheelPos*3) << 1))
23      elif WheelPos >= 21845 and WheelPos < 43690:
24          WheelPos -= 21845
25          return (((65535 - WheelPos*3) << 1) | (WheelPos*3))
26      else :
27          WheelPos -= 43690
28          return (((WheelPos*3) << 4) | (65535 - WheelPos*3))
```

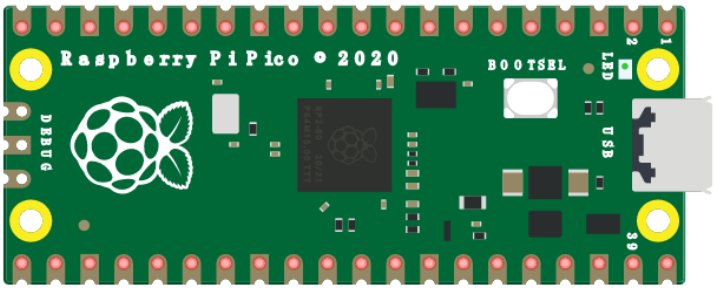
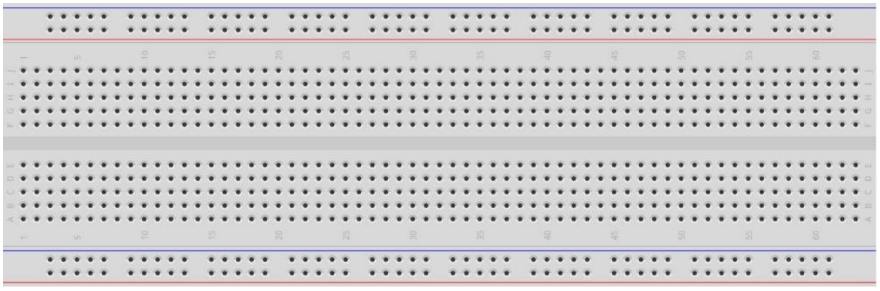
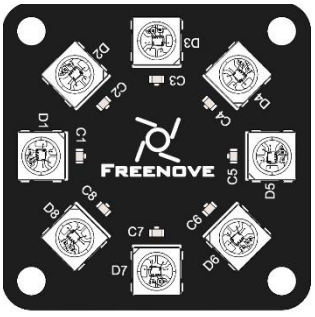
Chapter 6 NeoPixel

This chapter will help you learn to use a more convenient RGBLED lamp, which requires only one GPIO control and can be connected in infinite series in theory. Each LED can be controlled independently.

Project 6.1 NeoPixel

Learn the basic usage of NeoPixel and use it to blink red, green, blue and white.

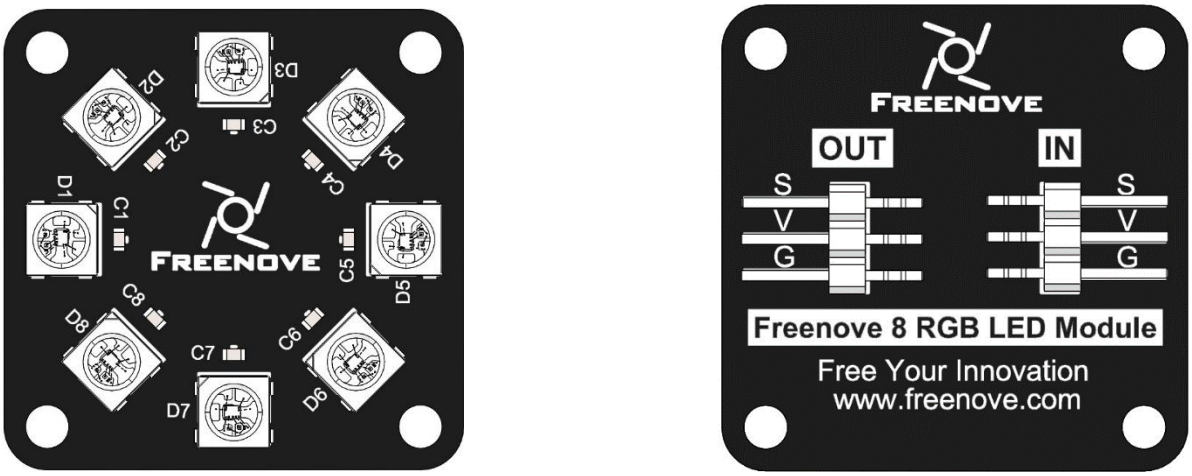
Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Freenove 8 RGB LED Module x1 	Jumper 

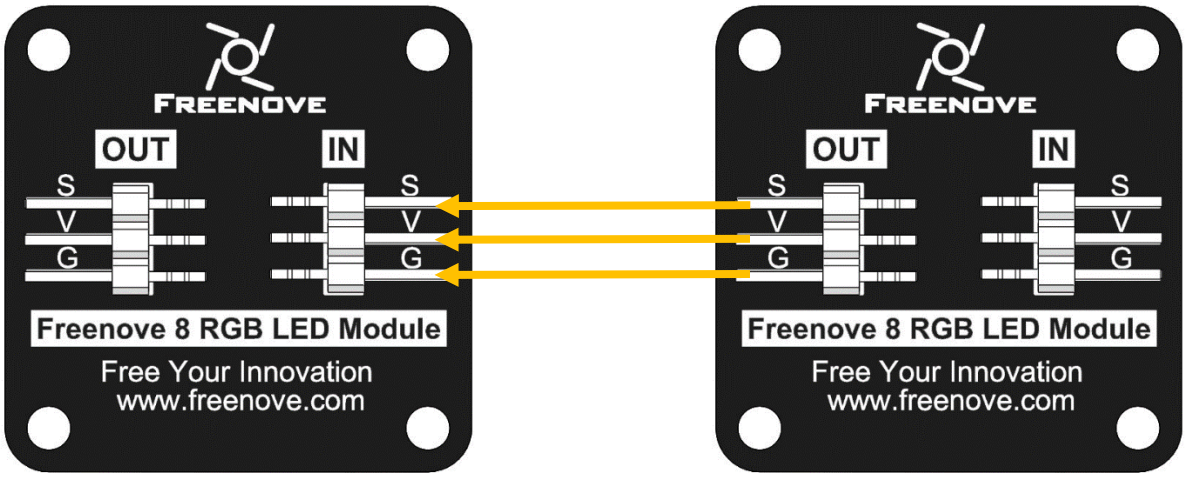
Related knowledge

Freenove 8 RGB LED Module

The Freenove 8 RGB LED Module is as below. You can use only one data pin to control eight LEDs on the module, as shown below:



You can also control many modules at the same time. Just connect OUT pin of one module to IN pin of another module. In this way, you can use one data pin to control 8, 16, 32 ··· LEDs.

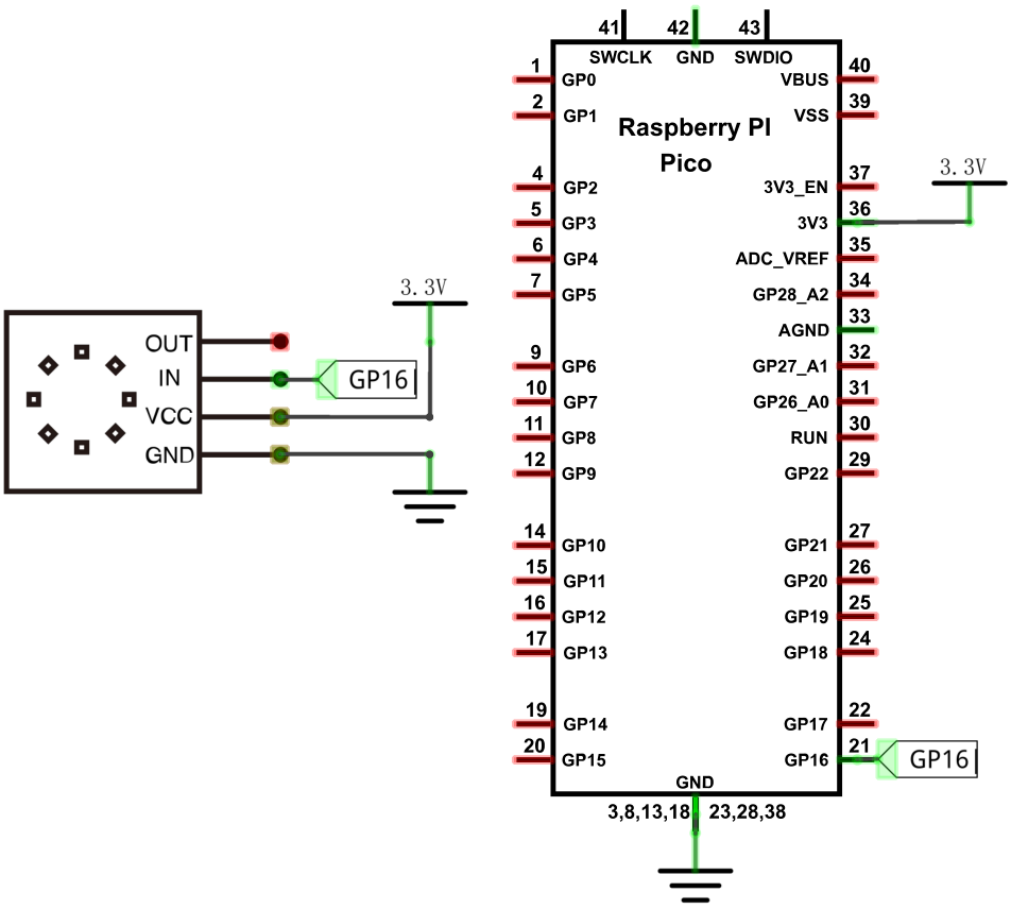


Pin description:

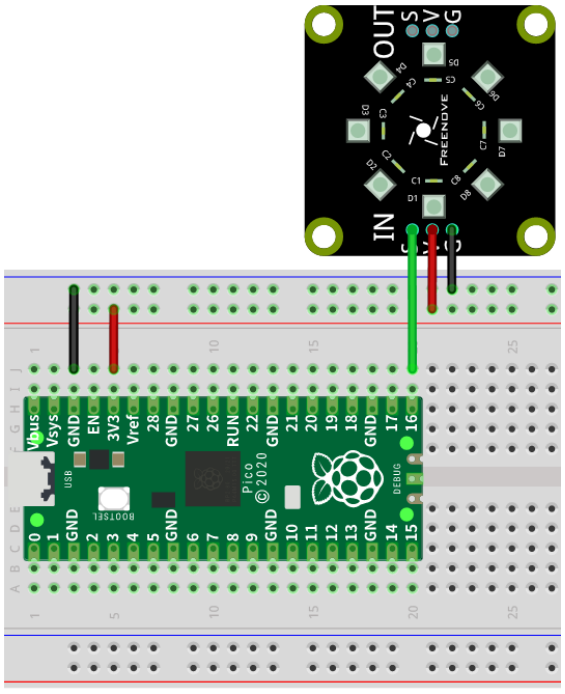
(IN)		(OUT)	
symbol	Function	symbol	Function
S	Input control signal	S	Output control signal
V	Power supply pin, +3.3V~5.5V	V	Power supply pin, +3.3V~5.5V
G	GND	G	GND

Circuit

Schematic diagram



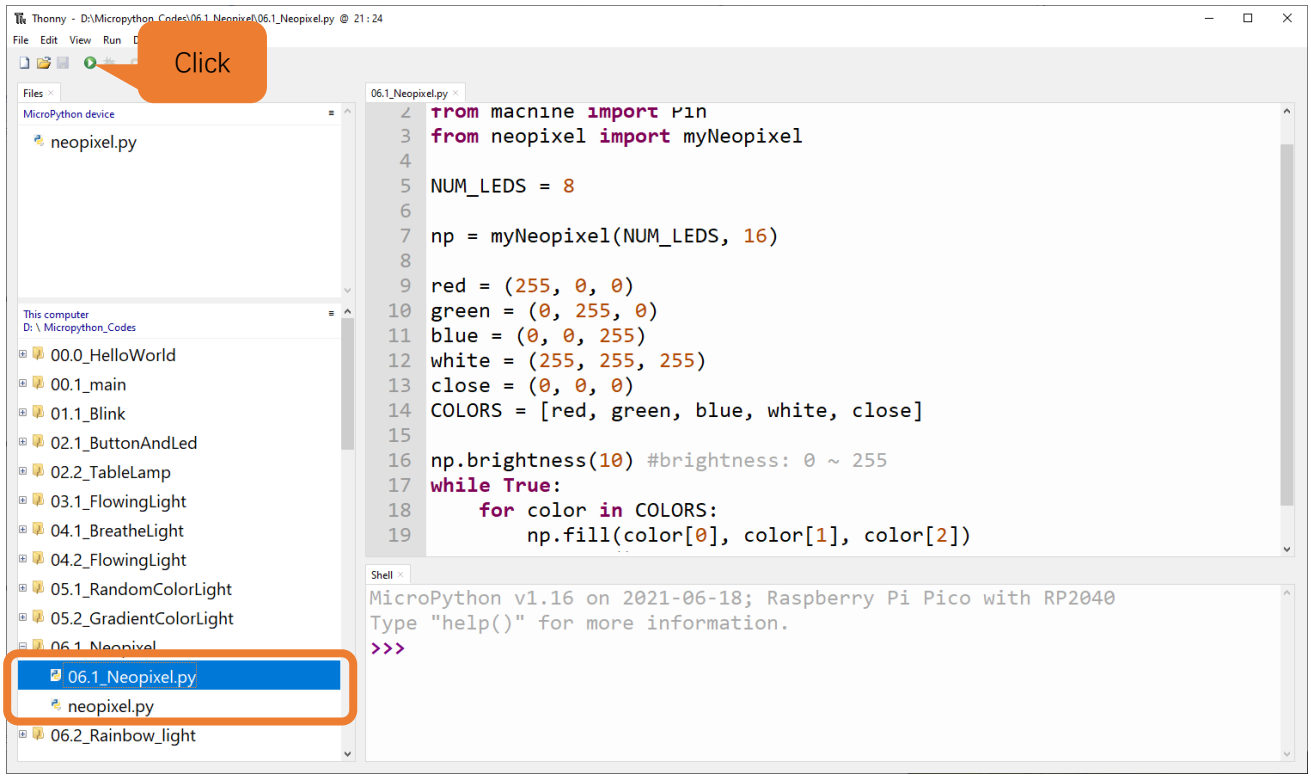
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



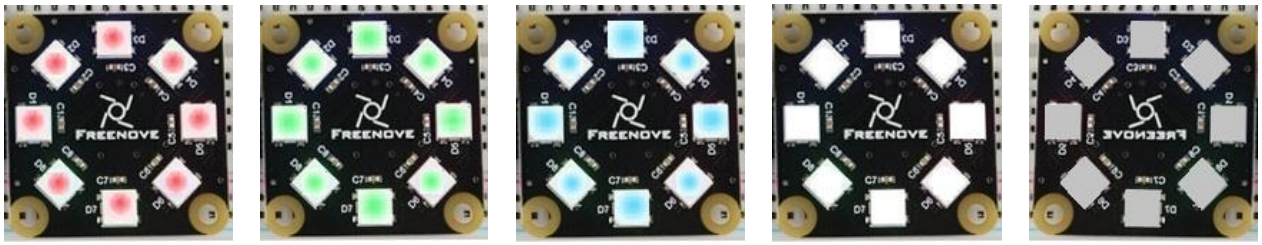
Code

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “06.1_Neopixel” .Select “neopixel.py”, right click to select “Upload to /”, wait for “neopixel.py” to be uploaded to Raspberry Pi Pico and double click “06.1_Neopixel.py”.

06.1_Neopixel



Click “Run current script” and Neopixel begins to light up in red, green, blue, white and close. Press Ctrl+C or click “Stop/Restart backend” to end program.



The following is the program code:

```
1  import time
2  from machine import Pin
3  from neopixel import myNeopixel
4
5  NUM_LEDS = 8
6
7  np = myNeopixel(NUM_LEDS, 16)
8
9  red = (255, 0, 0)
10 green = (0, 255, 0)
11 blue = (0, 0, 255)
12 white = (255, 255, 255)
13 close = (0, 0, 0)
14 COLORS = [red, green, blue, white, close]
15
16 np.brightness(10) #brightness: 0 ~ 255
17 while True:
18     for color in COLORS:
19         np.fill(color[0], color[1], color[2])
20         np.show()
21         time.sleep(0.5)
```

Import Pin, neopiexl and time modules.

```
1  import time
2  from machine import Pin
3  from neopixel import myNeopixel
```

Define the number of LEDs: NUM_LEDS = 8, create an object for myNeopixel and set GP16 pin to connect with neopixel.

```
5  NUM_LEDS = 8
6
7  np = myNeopixel(NUM_LEDS, 16)
```

Define five status for the LED, namely, red, green, blue, white and close and write them to COLORS list.

```
9  red = (255, 0, 0)
10 green = (0, 255, 0)
11 blue = (0, 0, 255)
12 white = (255, 255, 255)
13 close = (0, 0, 0)
14 COLORS = [red, green, blue, white, close]
```

Call brightness () function to set the brightness of LED, ranging from 0-255.

```
16 np.brightness(10) #brightness: 0 ~ 255
```

Call fill () function to set color for all LEDs at once. Only when show() function is called will the LEDs be lighted up to show the previously setting color.

```
19         np.fill(color[0], color[1], color[2])
20         np.show()
```

Define COLORS list through for loop to make neopixel module repeatedly emit red, green, blue, white and close, a total of five status.

```
17  while True:
18      for color in COLORS:
19          np.fill(color[0], color[1], color[2])
20          np.show()
21          time.sleep(0.5)
```

Reference

Class neopixel

Before each use of **neopixel** module**,** please add the statement "**import neopixel**" to the top of Python file.

myNeopixel(num_leds, pin): Define the number of output pins and LEDs of neopixel module

num_leds: The number of LEDs.

pin: Output pins.

myNeopixel.brightness(brightness): Set brightness for LEDs of neopixel module. Brightness range 0-255.

myNeopixel.set_pixel(pixel_num, r, g, b): Specify the color data of a single LED in the neopixel module.

pixel_num: neopixel: The sequence of LED in the module.

r: data of red channel.

g: data of green channel.

b: data of blue channel.

myNeopixel.fill (r, g, b): Set color data for all LEDs at once.

myNeopixel.show(): Make LED module show the setting color.

Project 6.2 Rainbow Light

In the previous project, we have mastered the usage of NeoPixel. This project will realize a slightly complicated Rainbow Light. The component list and the circuit are the same as the project fashionable Light.

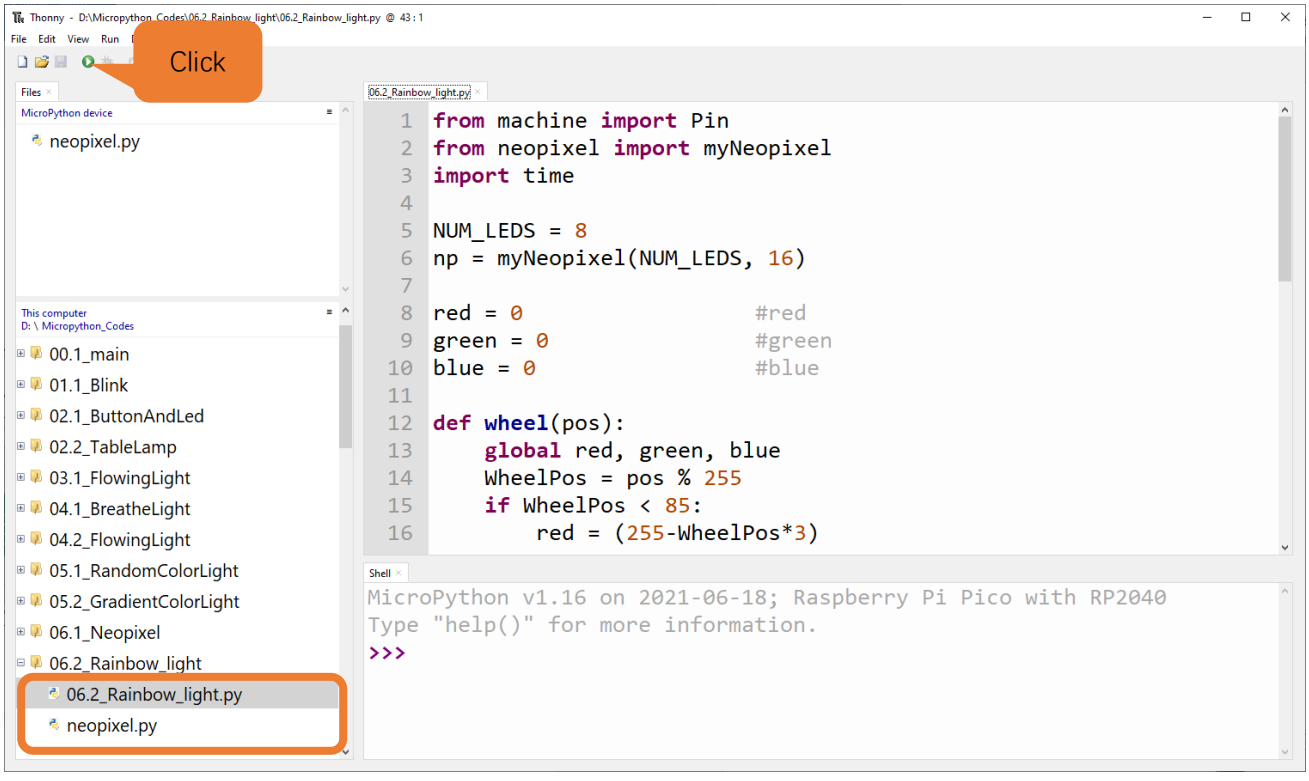
Code

Continue to use the following color model to equalize the color distribution of the eight LEDs and gradually change.



Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "06.2_Rainbow_light", Select "neopixel.py", right click to select "Upload to /", wait for "neopixel.py" to be uploaded to Raspberry Pi Pico and double click "06.2_Rainbow_light.py".

06.2_Rainbow_light



Click "Run current script", and the Freenove 8 RGB LED Strip displays different colors and the color changes gradually. Press Ctrl+C or click "Stop/Restart backend" to exit program.



The following is the program code:

```
from machine import Pin
from neopixel import myNeopixel
import time

NUM_LEDS = 8
np = myNeopixel(NUM_LEDS, 16)

red = 0                    #red
green = 0                  #green
blue = 0                   #blue

def wheel(pos):
    global red, green, blue
    WheelPos = pos % 255
    if WheelPos < 85:
        red = (255 - WheelPos*3)
        green = (WheelPos*3)
        blue=0
    elif WheelPos >= 85 and WheelPos < 170:
        WheelPos -= 85;
        red = 0
        green = (255 - WheelPos*3)
        blue = (WheelPos*3)
    else :
        WheelPos -= 170;
        red = (WheelPos*3)
        green = 0
        blue = (255 - WheelPos*3)

np.brightness(20)
```

```
while True:
    for i in range(0, 255):
        for j in range(0, 8):
            wheel(i + j*255 // 8)
            np.set_pixel(j, red, green, blue)
        np.show()
        time.sleep_ms(1)
```

Define a wheel() function to process the color data of neopixel module.

```
def wheel(pos):
    global red, green, blue
    WheelPos = pos % 255
    if WheelPos < 85:
        red = (255 - WheelPos*3)
        green = (WheelPos*3)
        blue=0
    elif WheelPos >= 85 and WheelPos < 170:
        WheelPos -= 85;
        red = 0
        green = (255 - WheelPos*3)
        blue = (WheelPos*3)
    else :
        WheelPos -= 170;
        red = (WheelPos*3)
        green = 0
        blue = (255 - WheelPos*3)
```

We use np.set_pixel(pixel_num, r, g, b) function to set data for each LED of neopixel separately, among which, parameter pixel_num presents the sequence of LED, parameters r, g, b indicate data of different color channels.

```
    np.set_pixel(j, red, green, blue)
```

Use a nesting of two for loops. The first for loop makes the value of i increase from 0 to 255 automatically and the wheel() function processes the value of i into data of the module's three colors; the second for loop writes the color data to the module.

```
while True:
    for i in range(0, 255):
        for j in range(0, 8):
            wheel(i + j*255 // 8)
            np.set_pixel(j, red, green, blue)
        np.show()
        time.sleep_ms(1)
```

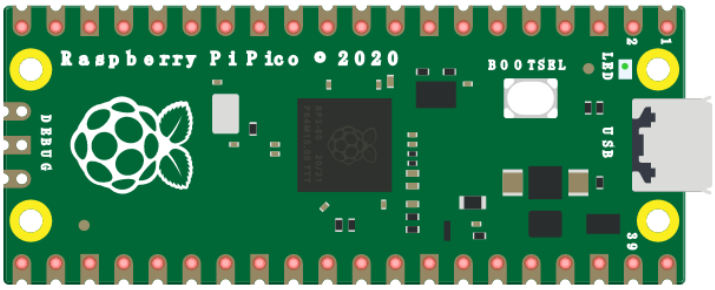
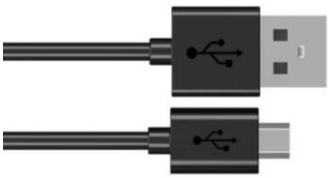
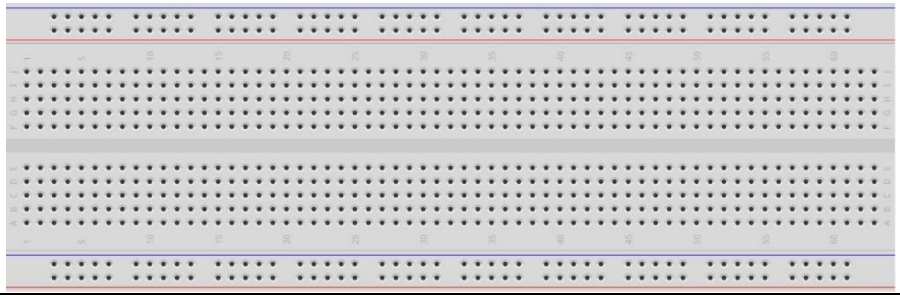
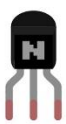
Chapter 7 Buzzer

In this chapter, we will learn about buzzers and the sounds they make.

Project 7.1 Doorbell

We will make this kind of doorbell: when the button is pressed, the buzzer sounds; and when the button is released, the buzzer stops sounding.

Component List

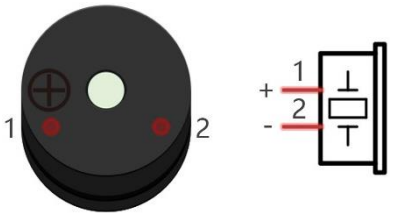
<table>
<tr><td colspan="3">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="5">Breadboard x1
  </td></tr>
<tr><td colspan="5">Jumper
  </td></tr>
<tr><td>NPN transistorx1
(S8050)
  </td><td>Active buzzer x1
  </td><td>Push button x1
  </td><td>Resistor
1kΩ x1
  </td><td>Resistor
10kΩ x2
  </td></tr>
</table>

Component knowledge

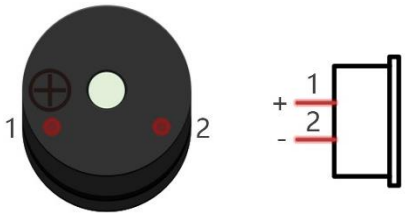
Buzzer

Buzzer is a sounding component, which is widely used in electronic devices such as calculator, electronic warning clock and alarm. Buzzer has two types: active and passive. Active buzzer has oscillator inside, which will sound as long as it is supplied with power. Passive buzzer requires external oscillator signal (generally use PWM with different frequency) to make a sound.

Active buzzer



Passive buzzer



Active buzzer is easy to use. Generally, it can only make a specific frequency of sound. Passive buzzer requires an external circuit to make a sound, but it can be controlled to make a sound with different frequency. The resonant frequency of the passive buzzer is 2 kHz, which means the passive buzzer is loudest when its resonant frequency is 2 kHz.

Next, we will use an active buzzer to make a doorbell and a passive buzzer to make an alarm.

How to identify active and passive buzzer?

1. Usually, there is a label on the surface of active buzzer covering the vocal hole, but this is not an absolute judgment method.
2. Active buzzers are more complex than passive buzzers in their manufacture. There are many circuits and crystal oscillator elements inside active buzzers; all of this is usually protected with a waterproof coating (and a housing) exposing only its pins from the underside. On the other hand, passive buzzers do not have protective coatings on their underside. From the pin holes viewing of a passive buzzer, you can see the circuit board, coils, and a permanent magnet (all or any combination of these components depending on the model.

Active buzzer



Passive buzzer



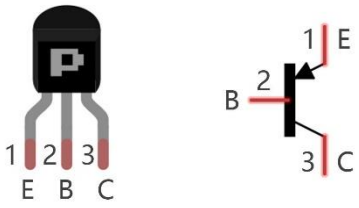
Transistor

Because the buzzer requires such large current that GP of Raspberry Pi Pico output capability cannot meet the requirement, a transistor of NPN type is needed here to amplify the current.

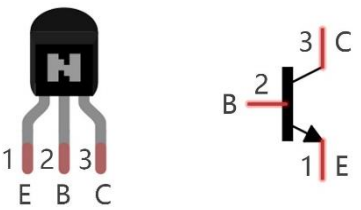
Transistor, the full name: semiconductor transistor, is a semiconductor device that controls current. Transistor

can be used to amplify weak signal, or works as a switch. It has three electrodes(PINs): base (b), collector (c) and emitter (e). When there is current passing between "be", "ce" will allow several-fold current (transistor magnification) pass, at this point, transistor works in the amplifying area. When current between "be" exceeds a certain value, "ce" will not allow current to increase any longer, at this point, transistor works in the saturation area. Transistor has two types as shown below: PNP and NPN.

PNP transistor



NPN transistor

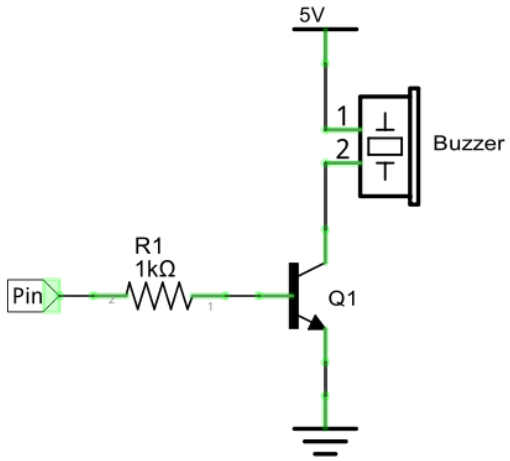
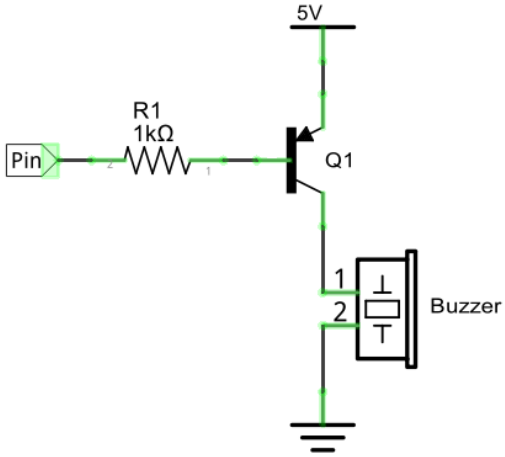


In our kit, the PNP transistor is marked with 8550, and the NPN transistor is marked with 8050.

Based on the transistor's characteristics, it is often used as a switch in digital circuits. As micro-controller's capacity to output current is very weak, we will use transistor to amplify current and drive large-current components.

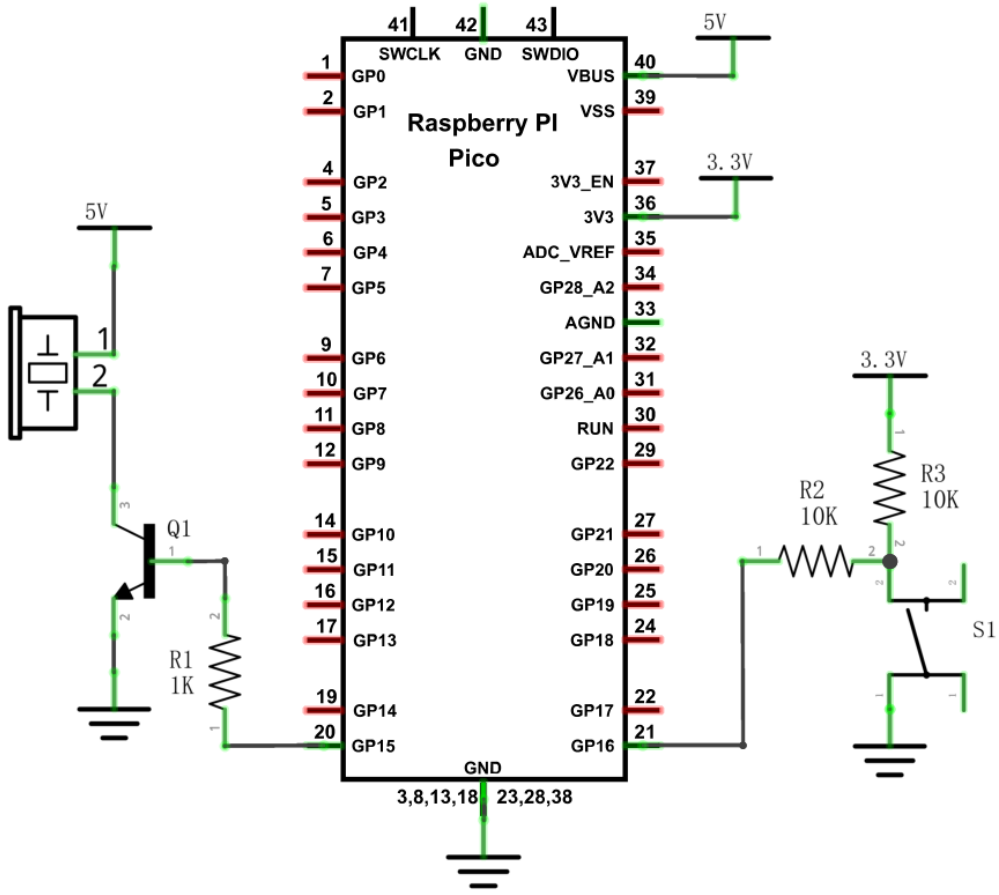
When using NPN transistor to drive buzzer, we often adopt the following method. If GP outputs high level, current will flow through R1, the transistor will be conducted, and the buzzer will sound. If GP outputs low level, no current flows through R1, the transistor will not be conducted, and buzzer will not sound.

When using PNP transistor to drive buzzer, we often adopt the following method. If GP outputs low level, current will flow through R1, the transistor will be conducted, and the buzzer will sound. If GP outputs high level, no current flows through R1, the transistor will not be conducted, and buzzer will not sound.

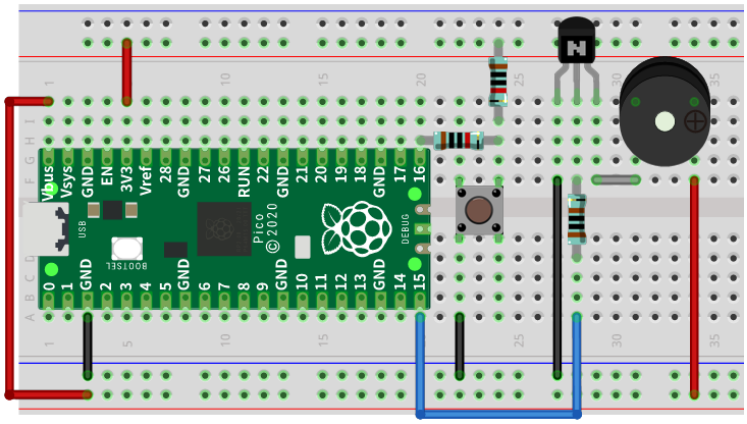
NPN transistor to drive buzzer	PNP transistor to drive buzzer
	

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Note:

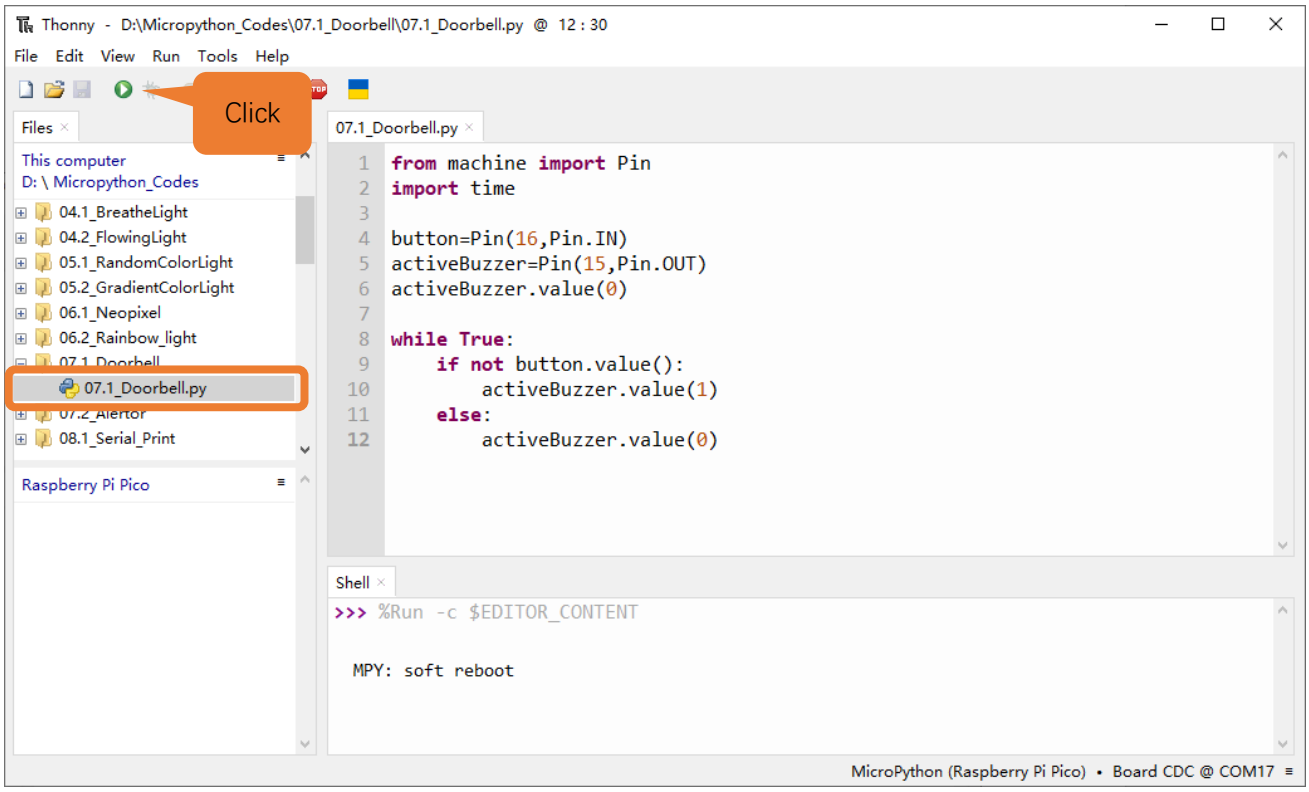
1. In this circuit, the power supply for buzzer is 5V, and pull-up resistor of the button connected to the power 3.3V. The buzzer can work when connected to power 3.3V, but it will reduce the loudness.
2. VBUS should be connect to the positive end of USB cable. If it connects to GND, it may burn the computer or Raspberry Pi Pico. Similarly, please be careful when wiring pins 36-40 of Pico to avoid short circuit.

Code

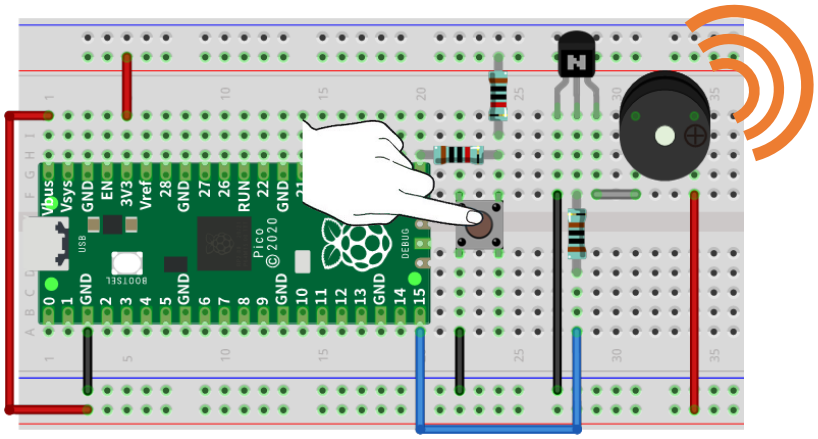
In this project, a buzzer will be controlled by a push button switch. When the button switch is pressed, the buzzer sounds and when the button is released, the buzzer stops. It is analogous to our earlier project that controlled an LED ON and OFF.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "07.1_Doorbell" and double click "07.1_Doorbell.py".

07.1_Doorbell



Click "Run current script", press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop. Press Ctrl+C or click "Stop/Restart backend" to exit program.



The following is the program code:

```
from machine import Pin
import time

button=Pin(16,Pin.IN)
activeBuzzer=Pin(15,Pin.OUT)
activeBuzzer.value(0)

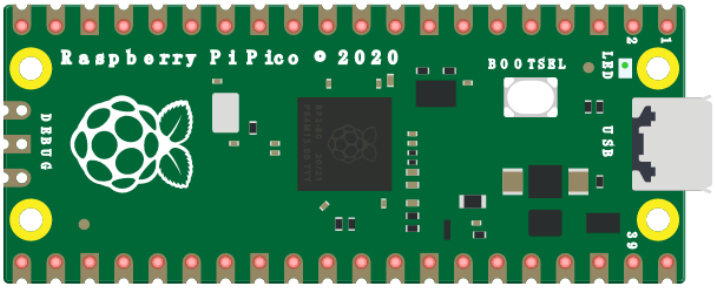
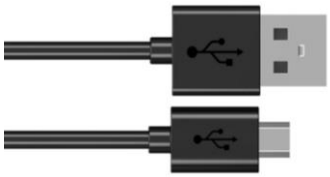
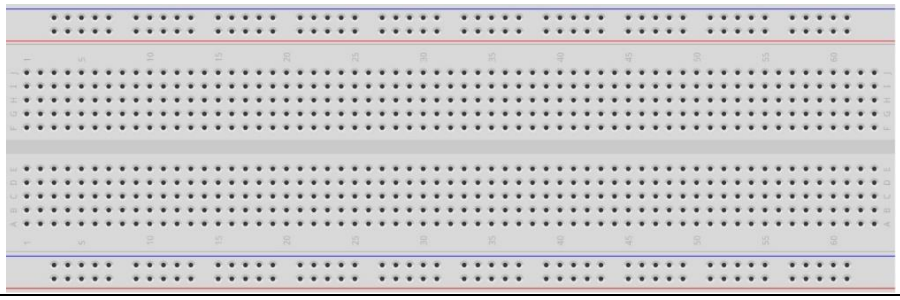
while True:
    if not button.value():
        activeBuzzer.value(1)
    else:
        activeBuzzer.value(0)
```

The code is logically the same as using button to control LED.

Project 7.2 Alertor

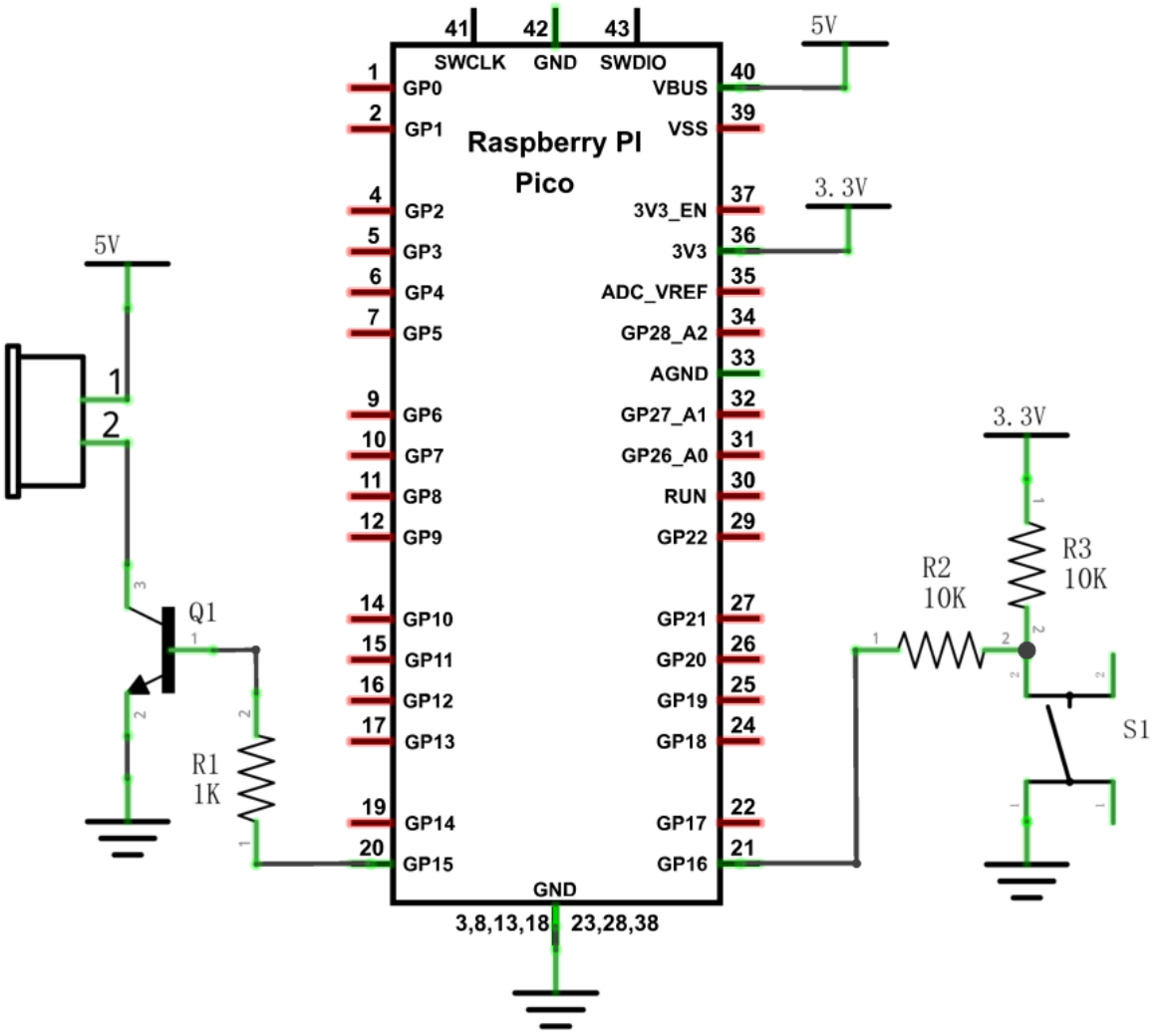
Next, we will use a passive buzzer to make an alarm.
Component list and the circuit part is similar to last section. In the Doorbell circuit, only the **active buzzer** needs to be **replaced** with a **passive buzzer.**

Component List

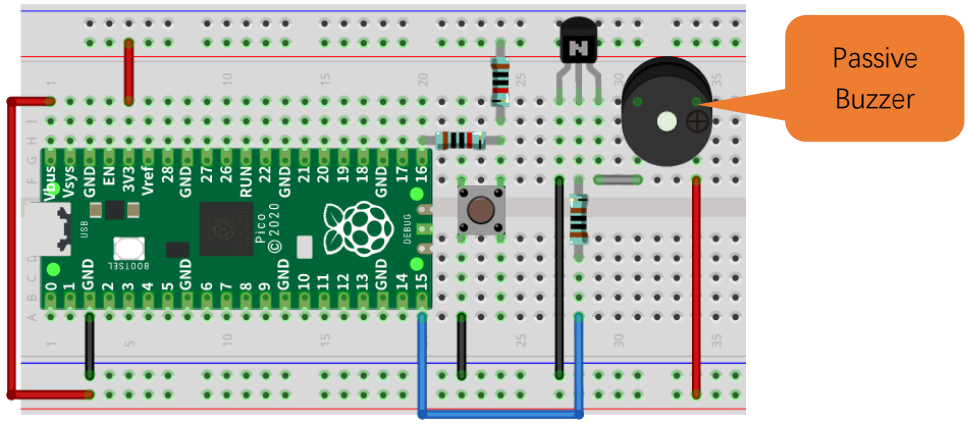
<table>
<tr><td colspan="3">Raspberry Pi Pico x1
  </td><td colspan="3">USB cable x1
  </td></tr>
<tr><td colspan="6">Breadboard x1
  </td></tr>
<tr><td colspan="6">Jumper
  </td></tr>
<tr><td colspan="2">NPN transistorx1
(S8050)
  </td><td>Passive buzzer x1
  </td><td>Push button x1
  </td><td>Resistor
1kΩ x1
  </td><td>Resistor
10kΩ x2
  </td></tr>
</table>

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

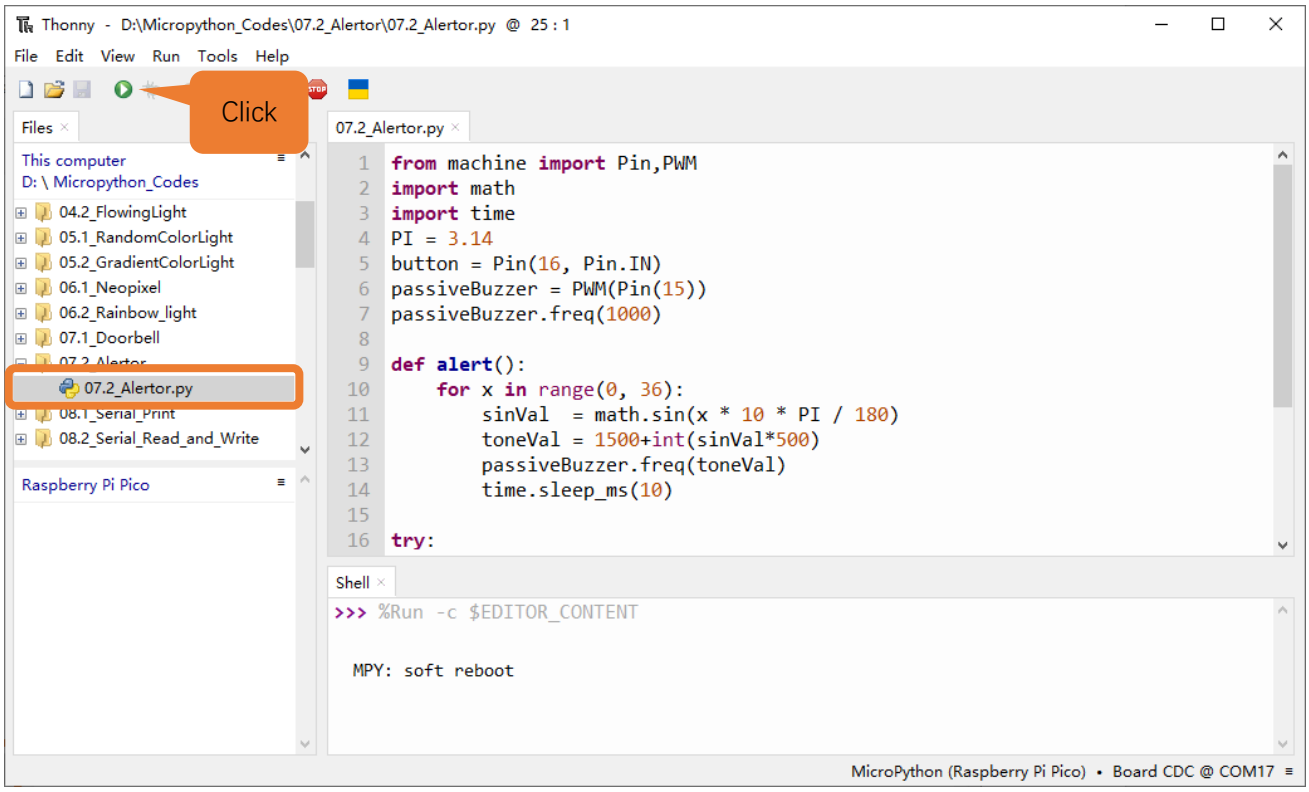


Code

In this project, the buzzer alarm is controlled by the button. Press the button, then buzzer sounds. If you release the button, the buzzer will stop sounding. Logically, it is the same as using button to control LED. As to the control method, passive buzzer requires PWM of certain frequency to sound.

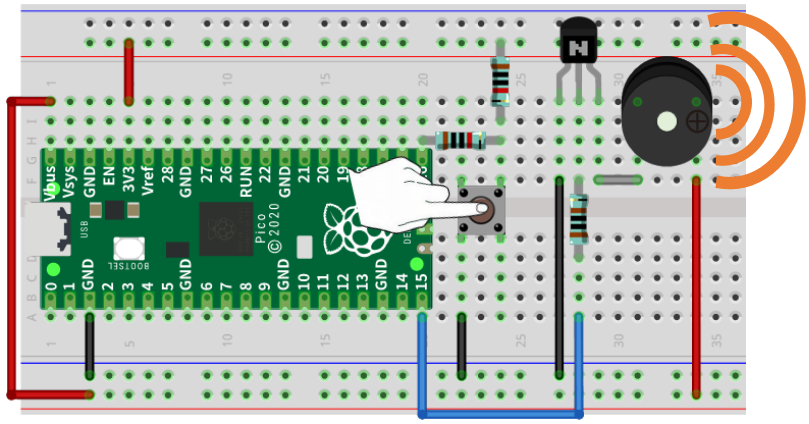
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "07.2_Alertor", and double click "07.2_Alertor.py".

07.2_Alertor



Click "Run current script", press the button, and the alarm sounds; when the button is released, the alarm will stop sounding. Press Ctrl+C or click "Stop/Restart backend" to exit program.

If the buzzer sound is too loud or too low for you, you can modify duty cycle or frequency of PWM.



The following is the program code:

```
from machine import Pin, PWM
import math
import time
PI = 3.14
button = Pin(16, Pin.IN)
passiveBuzzer = PWM(Pin(15))
passiveBuzzer.freq(1000)

def alert():
    for x in range(0, 36):
        sinVal  = math.sin(x * 10 * PI / 180)
        toneVal = 1500 + int(sinVal*500)
        passiveBuzzer.freq(toneVal)
        time.sleep_ms(10)

try:
    while True:
        if not button.value():
            passiveBuzzer.duty_u16(4092*2)
            alert()
        else:
            passiveBuzzer.duty_u16(0)
except:
    passiveBuzzer.deinit()
```

Import PWM, Pin, math and time modules.

```
from machine import Pin, PWM
import math
import time
```

Define the pins of the button and passive buzzer.

```
PI = 3.14
button = Pin(16, Pin.IN, Pin.PULL_UP)
passiveBuzzer = PWM(Pin(15))
passiveBuzzer.freq(1000)
```

Call sin function of math module to create the frequency data of the passive buzzer.

```
def alert():
    for x in range(0, 36):
        sinVal  = math.sin(x * 10 * PI / 180)
        toneVal = 1500 + int(sinVal*500)
        passiveBuzzer.freq(toneVal)
        time.sleep_ms(10)
```

When not using PWM, please turn it OFF in time.

```
    passiveBuzzer.deinit()
```

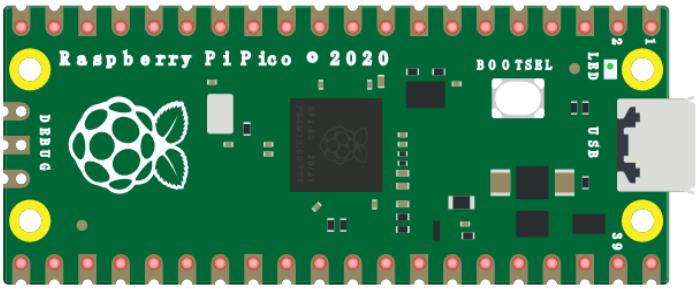
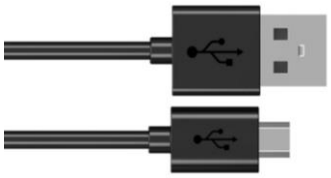
Chapter 8 Serial Communication

Serial Communication is a means of Communication between different devices. This section describes Raspberry Pi Pico Serial Communication.

Project 8.1 Serial Print

This project uses Raspberry Pi Pico serial communicator to send data to the computer and print it on the serial monitor.

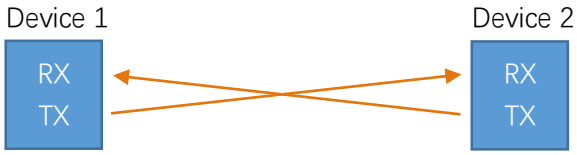
Component List

Raspberry Pi Pico x1 	USB cable x1 

Related knowledge

Serial communication

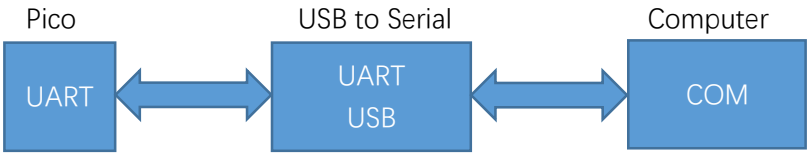
Serial communication generally refers to the Universal Asynchronous Receiver/Transmitter (UART), which is commonly used in electronic circuit communication. It has two communication lines; one is responsible for sending data (TX line) and the other for receiving data (RX line). The serial communication connections two devices use is as follows:



Before serial communication starts, the baud rate of both sides must be the same. Communication between devices can work only if the same baud rate is used. The baud rates commonly used is 9600 and 115200.

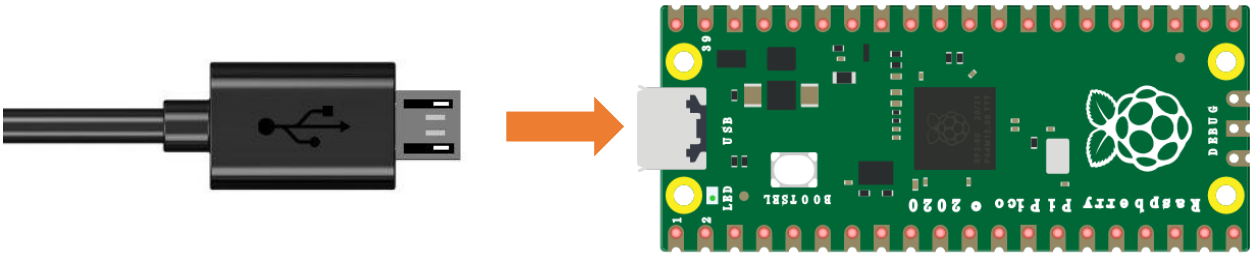
Serial port on Raspberry Pi Pico

Raspberry Pi Pico has integrated USB to serial transfer, so it could communicate with computer connecting to USB cable.



Circuit

Connect Raspberry Pi Pico to the computer with USB cable.



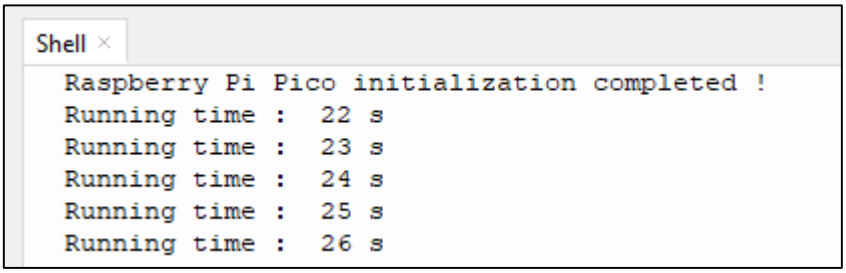
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "08.1_Serial_Print" and double "08.1_Serial_Print.py".

08.1_Serial_Print



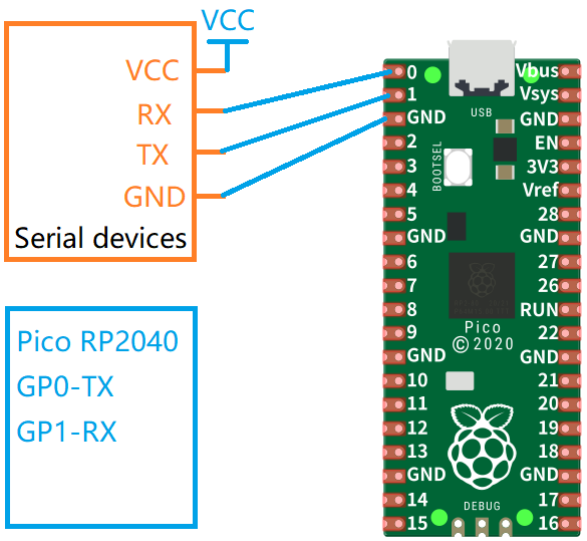
Click "Run current script" and observe the changes of "Shell", which will display the time when Raspberry Pi Pico is powered on once per second.



The following is the program code:

```
1  import time
2
3  print("Raspberry Pi Pico initialization completed !")
4
5  while True:
6      print("Running time : ", time.time()%60,"s")
7      time.sleep(1)
```

There are two serial communications on Raspberry Pi Pico: UART0 and UART1.
You can use them to communicate with serial devices.



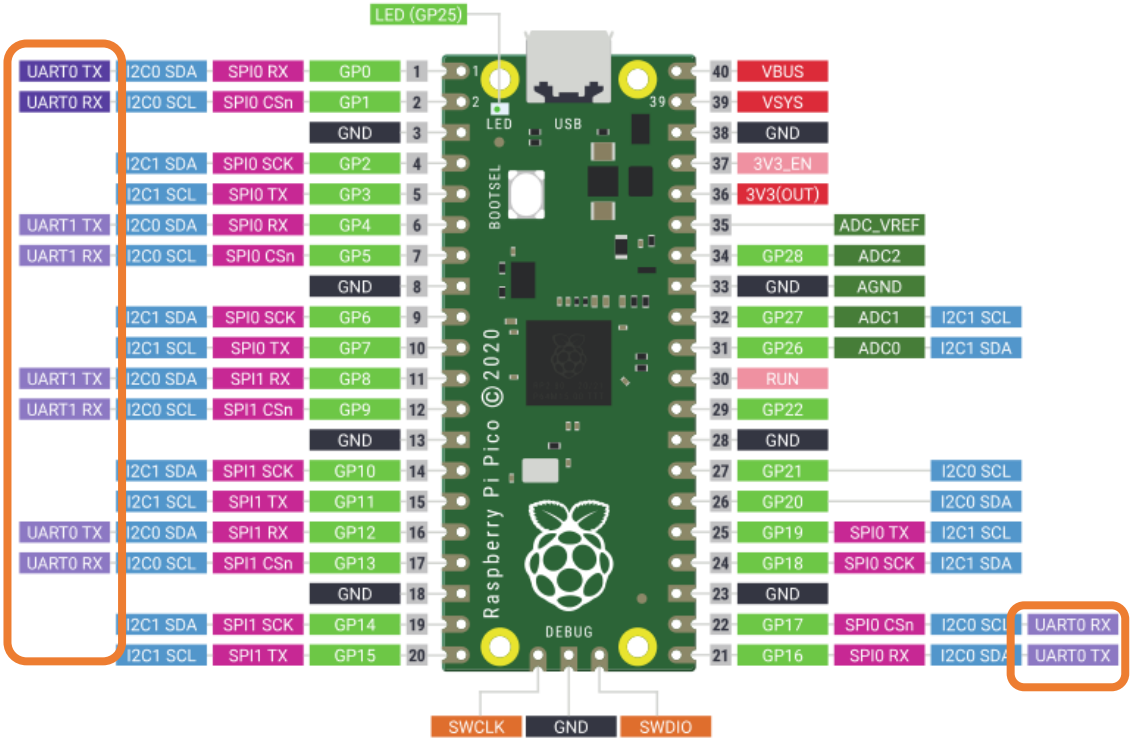
Default pin for UART0

UART0_TX	Pin 0
UART0_RX	Pin 1

Default pin for UART1

UART1_TX	Pin 4
UART1_RX	Pin 5

For details, please refer to UART, I2C, SPI default pin.
You can also change settings according to the distribution of pins.



Reference

Class UART

Before each use of **UART** module, please add the statement "**from machine import UART**" to the top of python file.

UART(id, baudrate, bits, parity, rx, tx, stop, timeout): Define serial ports and configure parameters for them.

id: Serial Number. The available serial port number is 0 or 1.

baudrate: Baud rate.

bits: The number of each character.

parity: Check even or odd, with 0 for even checking and 1 for odd checking.

rx, tx: UAPT's reading and writing pins, GP0, GP1, GP4, GP5, GP8, GP9, GP12, GP13, GP16, GP17.

stop: The number of stop bits, and the stop bit is 1 or 2.

timeout: timeout period (Unit: millisecond).

0 < timeout ⩽ 0x7FFF FFFF (decimal: 0 < timeout ⩽ 2147483647).

UART.read(nbytes): Read nbytes bytes.

UART.read(): Read data.

UART.write(buf): Write byte buffer to UART bus.

UART.readline(): Read a line of data, ending with a newline character.

UART.readinto(buf): Read and write data into buffer.

UART.readinto(buf, nbytes): Read and write data into buffer.

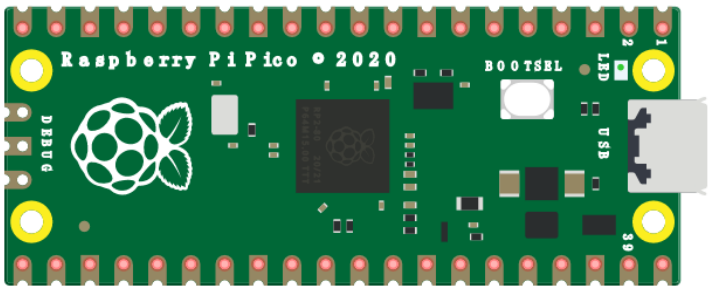
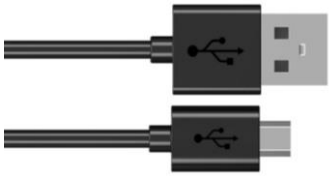
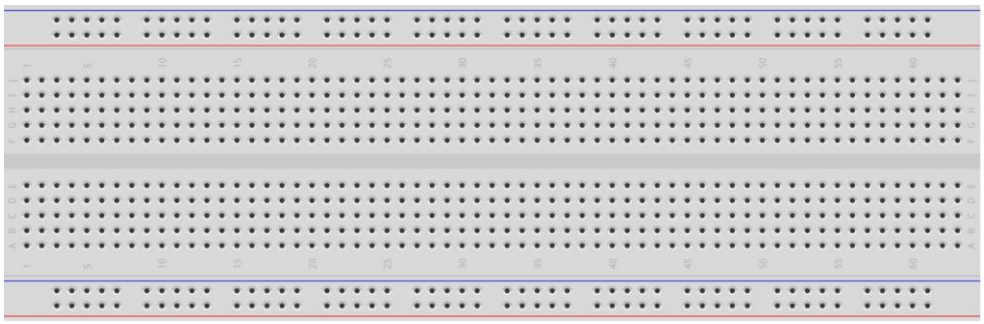
UART.any(): Determine whether there is data in serial ports. If there is, return the number of bytes; Otherwise, return 0.

Note: When usART0 or USART1 is used, it must be used with the serial port adapter board or serial port device. If not, you may not observe any symptoms.

Project 8.2 Serial Read and Write

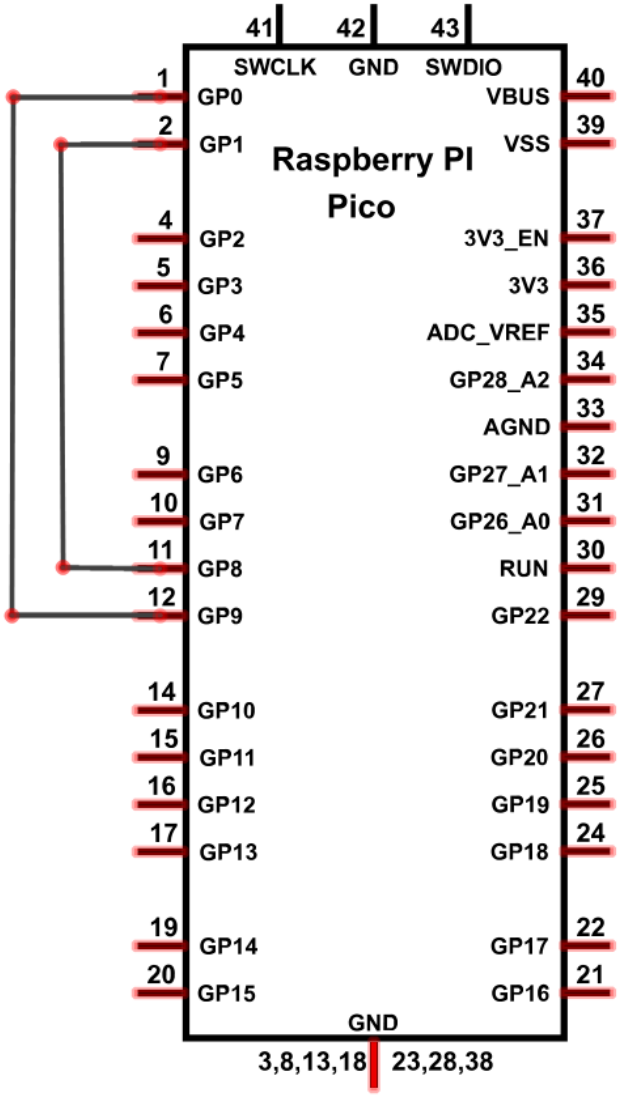
In the following example, we use Raspberry Pi Pico's UART1 to send data to UART0, and read the data received by UART0 and print it out through the "Shell".

Component List

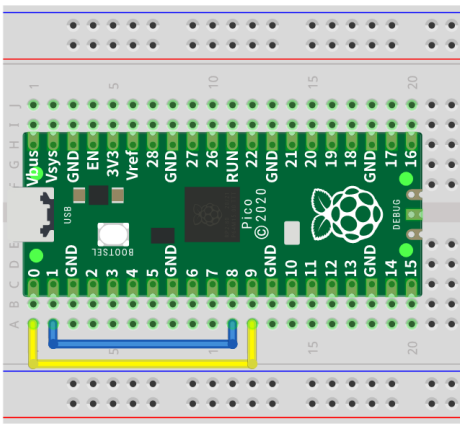
Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Jumper 	

Circuit

Schematic diagram



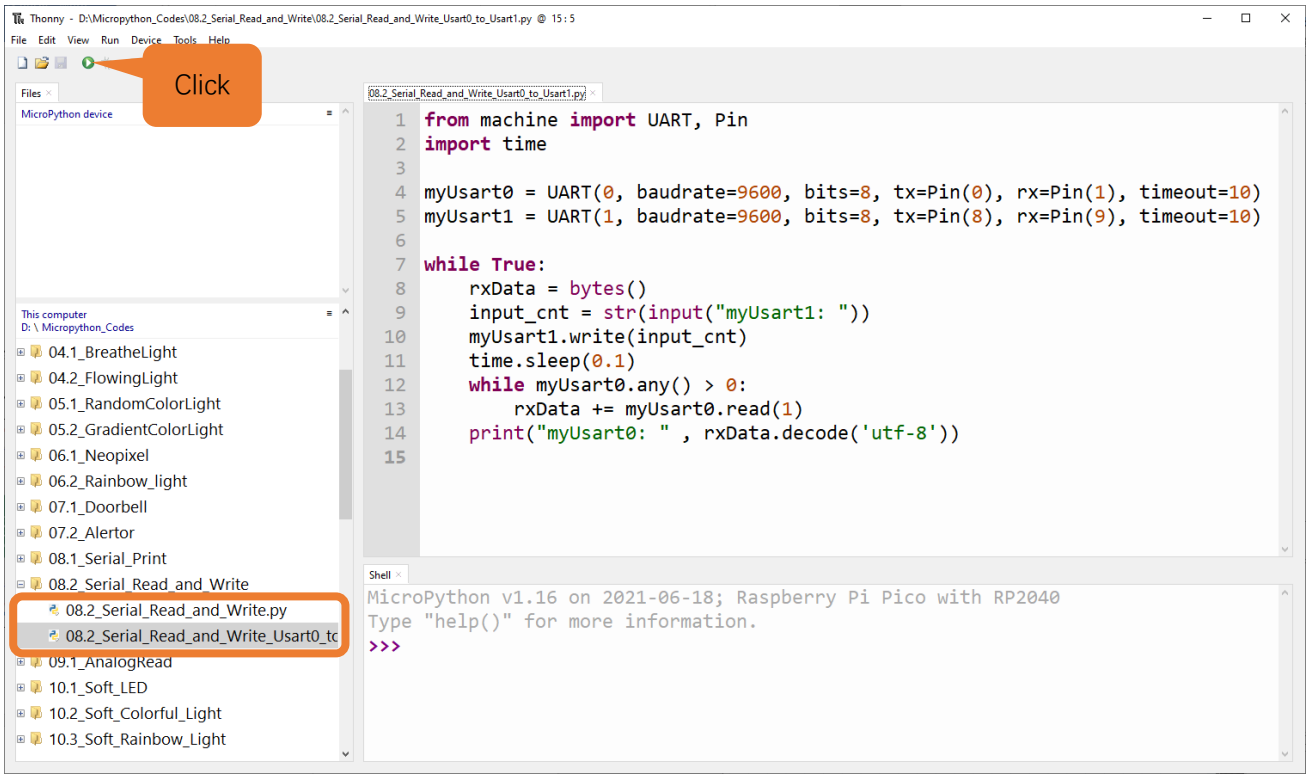
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



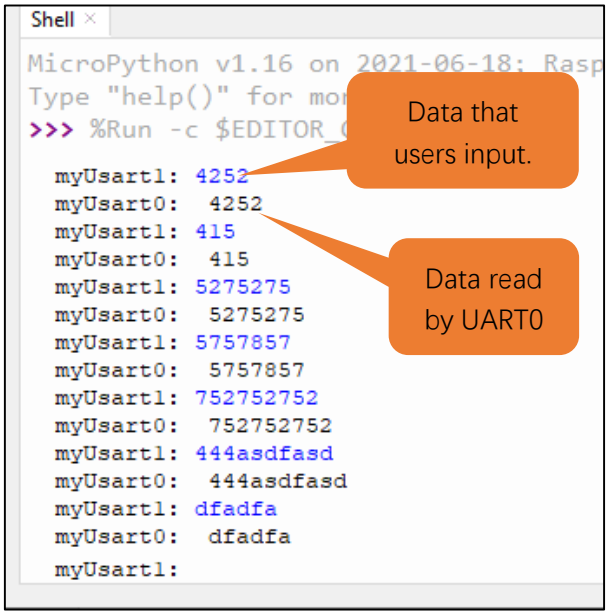
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "08.2_Serial_Read_and_Write" and double click "08.2_Serial_Read_and_Write_UART1_to_UART0.py".

08.2_Serial_Read_and_Write_UART1_to_UART0



Click "Run current script". Users can enter any data in "Shell" and press Enter. The input data will be written to UART1 and sent to UART0 to read, and printed out on "Shell". Press "Ctrl+C" or click "Stop/Restart backend" to exit the program.



The following is the program code:

```
1  from machine import UART, Pin
2  import time
3
4  myUsart0 = UART(0, baudrate=9600, bits=8, tx=Pin(0), rx=Pin(1), timeout=10)
5  myUsart1 = UART(1, baudrate=9600, bits=8, tx=Pin(8), rx=Pin(9), timeout=10)
6
7  while True:
8      rxData = bytes()
9      input_cnt = str(input("myUsart1: "))
10     myUsart1.write(input_cnt)
11     time.sleep(0.1)
12     while myUsart0.any() > 0:
13         rxData += myUsart0.read(1)
14     print("myUsart0: " , rxData.decode('utf-8'))
```

Import UART、Pin and time modules.

```
1  from machine import UART, Pin
2  import time
```

Create two UART objects and configure them as the parameters of UART0 and UART1.

```
4  myUsart0 = UART(0, baudrate=9600, bits=8, tx=Pin(0), rx=Pin(1), timeout=10)
5  myUsart1 = UART(1, baudrate=9600, bits=8, tx=Pin(8), rx=Pin(9), timeout=10)
```

Define a bytes value and assign it to rxDate.

```
8      rxData = bytes()
```

Define input_cnt to receive user input and convert it to a string format.

```
9      input_cnt = str(input("myUsart1: "))
```

myUsart1 calls write() function and writes the user input to UART1.

```
10     myUsart1.write(input_cnt)
```

myUsart0 calls the read() function to read the data sent by UART1 bit by bit and save the rxData in the received variable. When myUsart0 calls any() to determine whether UART0 has read the data, when any() returns 0, UART0 has read the data sent by UART1.

```
12     while myUsart0.any() > 0:
13         rxData += myUsart0.read(1)
```

The decode() function is called to decode the data and print it out to "Shell."

```
14     print("myUsart0: " , rxData.decode('utf-8'))
```

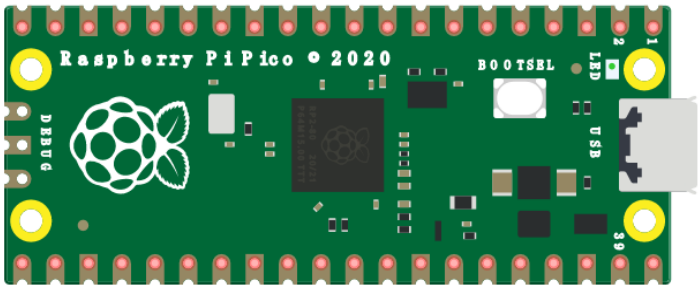
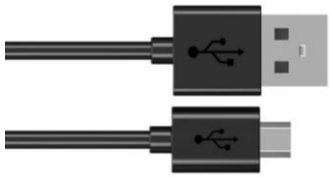
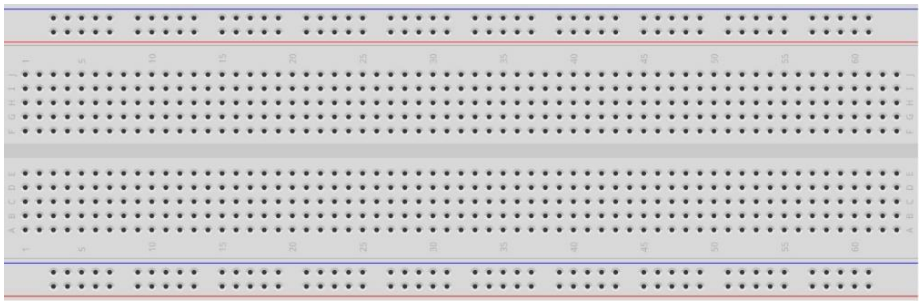
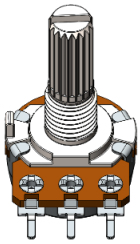
Chapter 9 AD Converter

This chapter we learn to use the ADC function of Raspberry Pi Pico.

Project 9.1 Read the Voltage of Potentiometer

In this chapter, we use ADC function of Pico to read the voltage output by potentiometer.

Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Rotary potentiometer x1 	Jumper 

Related knowledge

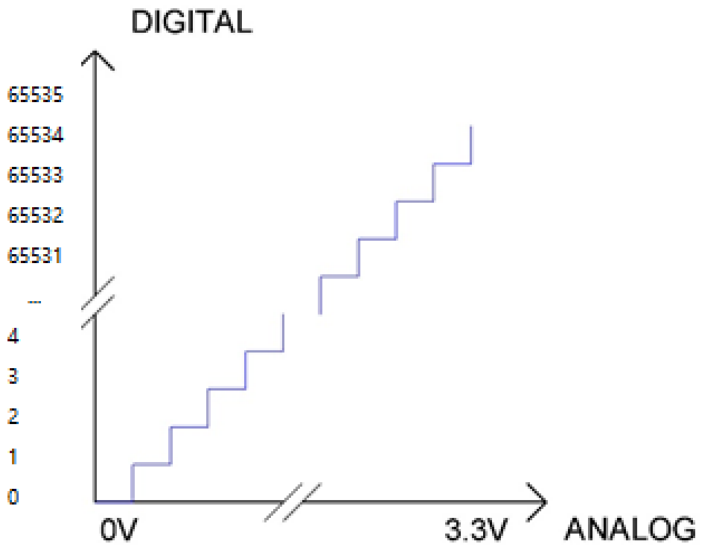
ADC

An analog-to-digital converter(ADC) converts a measured analog signal into a digital code. ADC has two key features: resolution and channels.

ADC resolution of Raspberry Pi Pico

Raspberry Pi Pico uses RP2040 chip. With a 12-bit ADC resolution, it can convert any analog signal to digital signal, ranging from 0-4095. For example, if the analog voltage range it measures is 0-3.3V, the ADC can divide it into 4096 equal parts.

However, when using Micropython firmware to call Raspberry Pi Pico ADC, the digital signal it obtains ranges from 0 to 65535. This is because Micropython internally processes Pico's ADC resolution to 16 bits, and the values is changed to 0-65535, to make it the same as the ADC of other Micropython microcontrollers. The way that ADC converts does not change but only the resolution changes. Therefore, if the measured analog voltage ranges from 0-3.3V, ADC can divide it into 65536 equal parts.



Subsection 1: the analog in a range of 0V---3.3/65535 V corresponds to digital 0;

Subsection 2: the analog in a range of 3.3/65535 V---2*3.3 /65535 V corresponds to digital 1;

...

Subsection 65535: the analog in range of 65534*3.3/65535 V---65535*3.3 /65535 V corresponds to digital 65534;

The following analog will be divided accordingly.

The conversion formula is as follows:

$$ADCValue = \frac{\text{Analog Voltage}}{3.3} * 65535$$

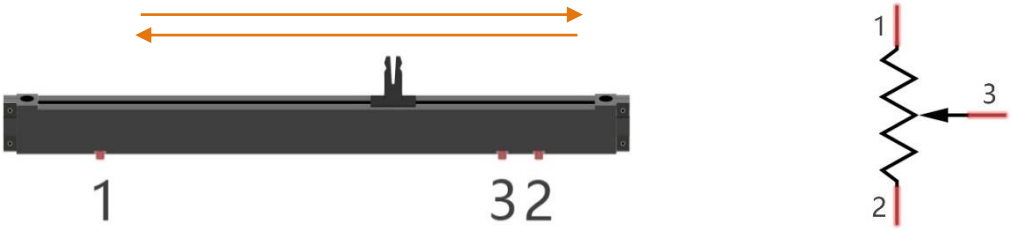
ADC ChannelsRaspberry Pi Pico

Raspberry Pi Pico has five ADC channels, which are ADC0(GP26), ADC1(GP27), ADC2(GP28), ADC3(GP29): used to measure VSYS on Pico board, and ADC4, which directly connects to the built-in temperature sensor of RP2040 chip. Therefore, there are only three generic ADC channels that can be directly used, namely, ADC0, ADC1 and ADC2.

Component knowledge

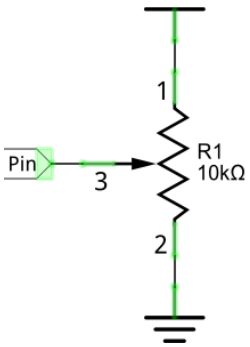
Potentiometer

Potentiometer is a resistive element with three Terminal parts. Unlike the resistors that we have used thus far in our project which have a fixed resistance value, the resistance value of a potentiometer can be adjusted. A potentiometer is often made up by a resistive substance (a wire or carbon element) and movable contact brush. When the brush moves along the resistor element, there will be a change in the resistance of the potentiometer's output side (3) (or change in the voltage of the circuit that is a part). The illustration below represents a linear sliding potentiometer and its electronic symbol on the right.



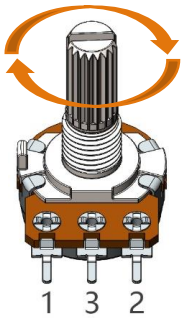
Between potentiometer pin 1 and pin 2 is the resistive element (a resistance wire or carbon) and pin 3 is connected to the brush that makes contact with the resistive element. In our illustration, when the brush moves from pin 1 to pin 2, the resistance value between pin 1 and pin 3 will increase linearly (until it reaches the highest value of the resistive element) and at the same time the resistance between pin 2 and pin 3 will decrease linearly and conversely down to zero. At the midpoint of the slider, the measured resistance values between pin 1 and 3 and between pin 2 and 3 will be the same.

In a circuit, both sides of resistive element are often connected to the positive and negative electrodes of power. When you slide the brush "pin 3", you can get variable voltage within the range of the power supply.



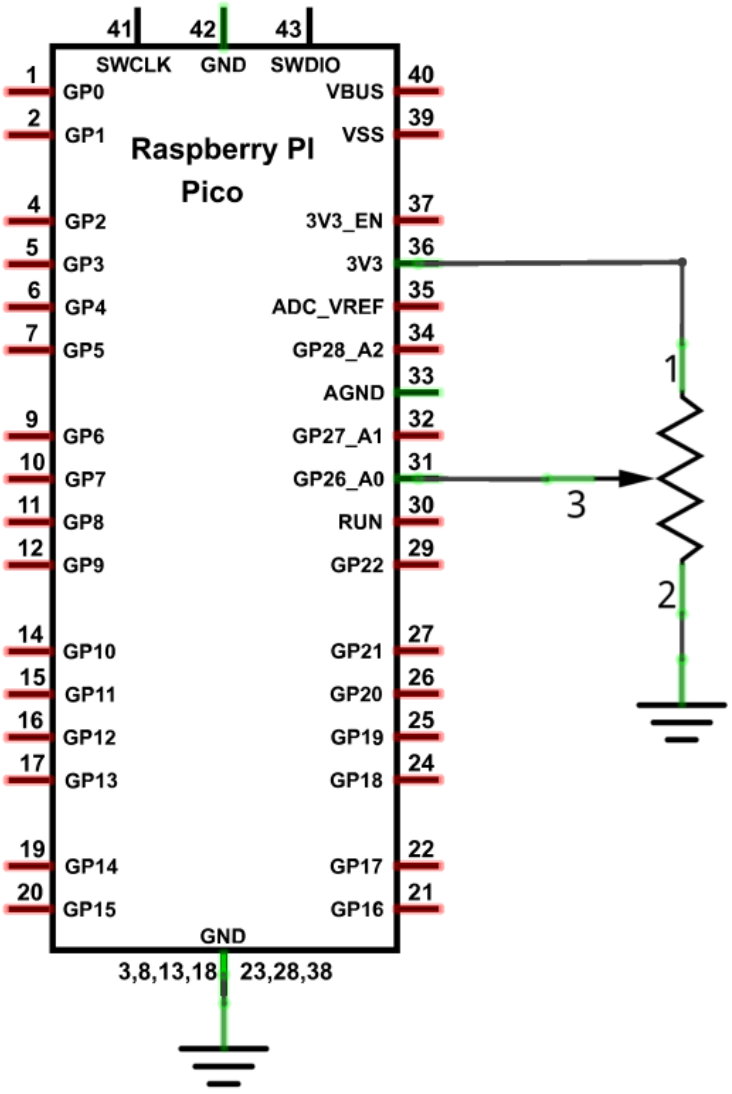
Rotary potentiometer

Rotary potentiometers and linear potentiometers have the same function; the only difference being the physical action being a rotational rather than a sliding movement.

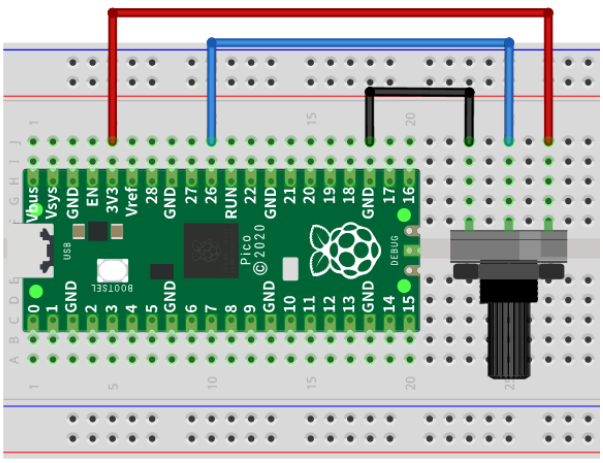


Circuit

Schematic diagram



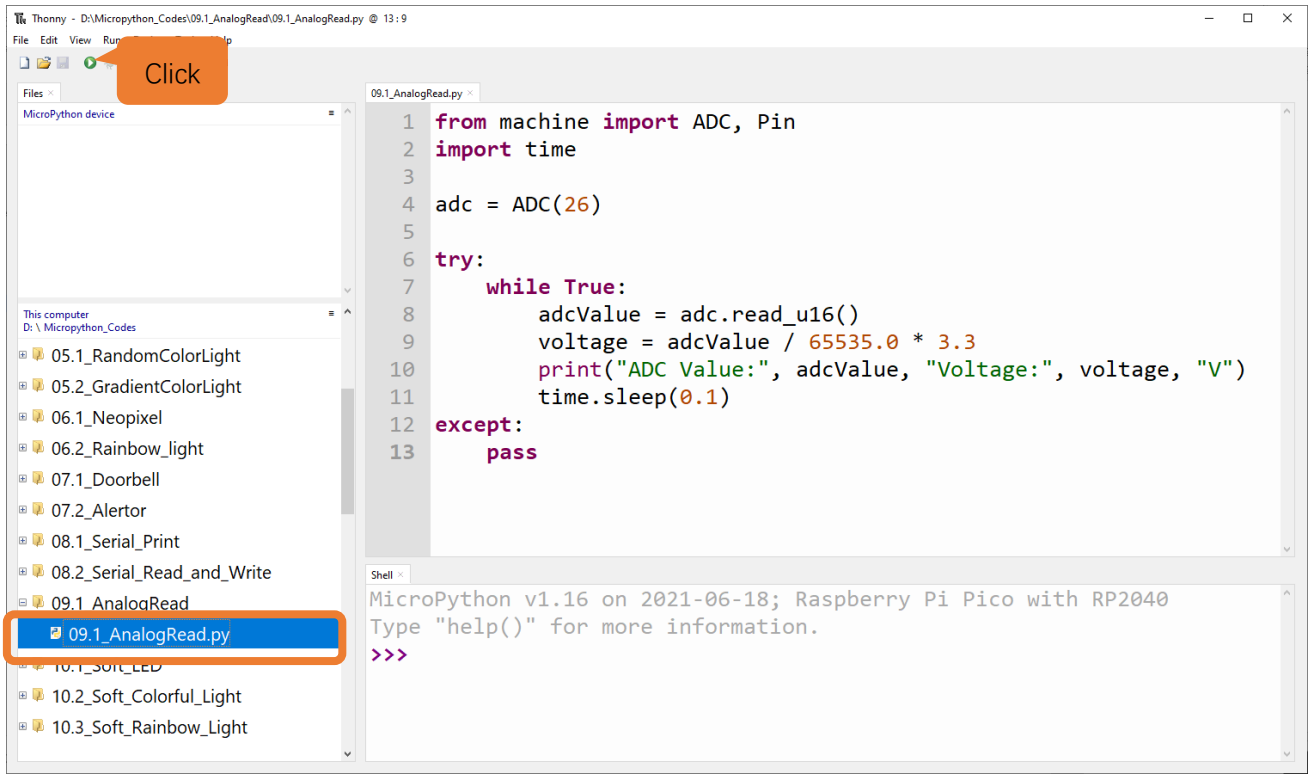
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



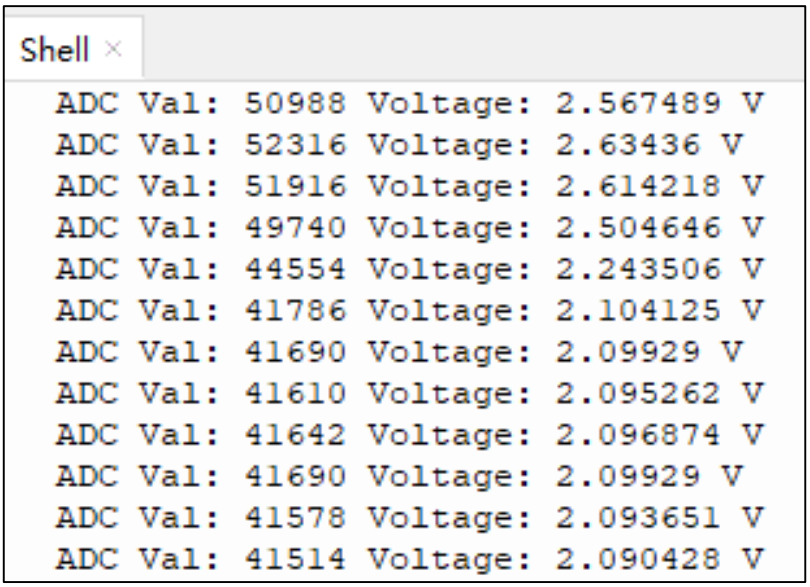
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "09.1_AnalogRead and then click "09.1_AnalogRead.py".

09.1_AnalogRead



Click "Run current script" and observe the message printed in "Shell". Press Ctrl+C or click "Stop/Restart backend" to exit the program.



The following is the code:

```
from machine import ADC, Pin
import time

adc = ADC(26)

try:
    while True:
        adcValue = adc.read_u16()
        voltage = adcValue / 65535.0 * 3.3
        print("ADC Value:", adcValue, "Voltage:", voltage, "V")
        time.sleep(0.1)
except:
    pass
```

Import Pin, ADC and time modules.

```
from machine import ADC, Pin
import time
```

Create an ADC object and connect GP26, which corresponds to ADC0 channel of Raspberry Pi Pico.

```
adc = ADC(26)
```

Read ADC value every 0.1 second. Calculate the current voltage based on the formula ADCValue= (Analog Voltage)/3.3*65535 and print it to "Shell".

```
    while True:
        adcValue = adc.read_u16()
        voltage = adcValue / 65535.0 * 3.3
        print("ADC Value:", adcValue, "Voltage:", voltage, "V")
        time.sleep(0.1)
```

Reference

Class ADC

Before each use of ADC module, please add the statement "**from machine import ADC**" to the top of the python file.

machine.ADC(pin or channel_num): Create an ADC object associated with the given pin.

pin: Available pins are:GP26,GP27,GP28,GP29.

channel_num: Available channel 0, 1, 2, 3, 4.

For example:

machine.ADC(0) = machine.ADC(26)
machine.ADC(1) = machine.ADC(27)
machine.ADC(2) = machine.ADC(28)
machine.ADC(3) = machine.ADC(29)
machine.ADC(4) Connects to the internal temperature sensor.

ADC.read_16(): reads the current ADC value and returns it, with a range of 0-65535.

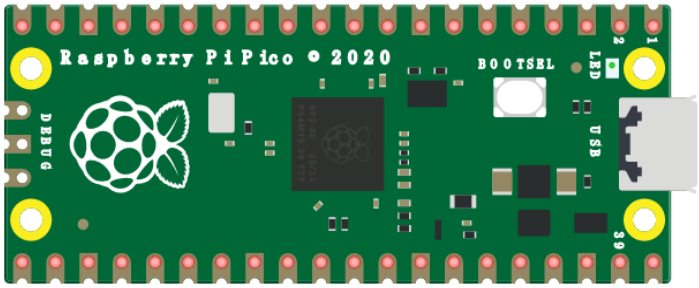
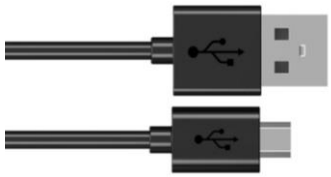
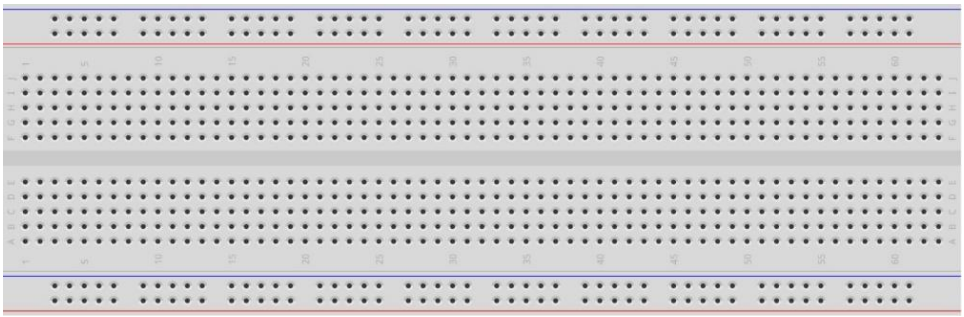
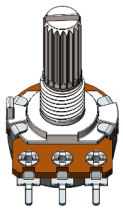
Chapter 10 Potentiometer & LED

We have learnt to use ADC in the previous chapter. In this chapter, we will combine PWM and ADC to use potentiometer to control LED, RGBLED and Neopixel.

Project 10.1 Soft Light

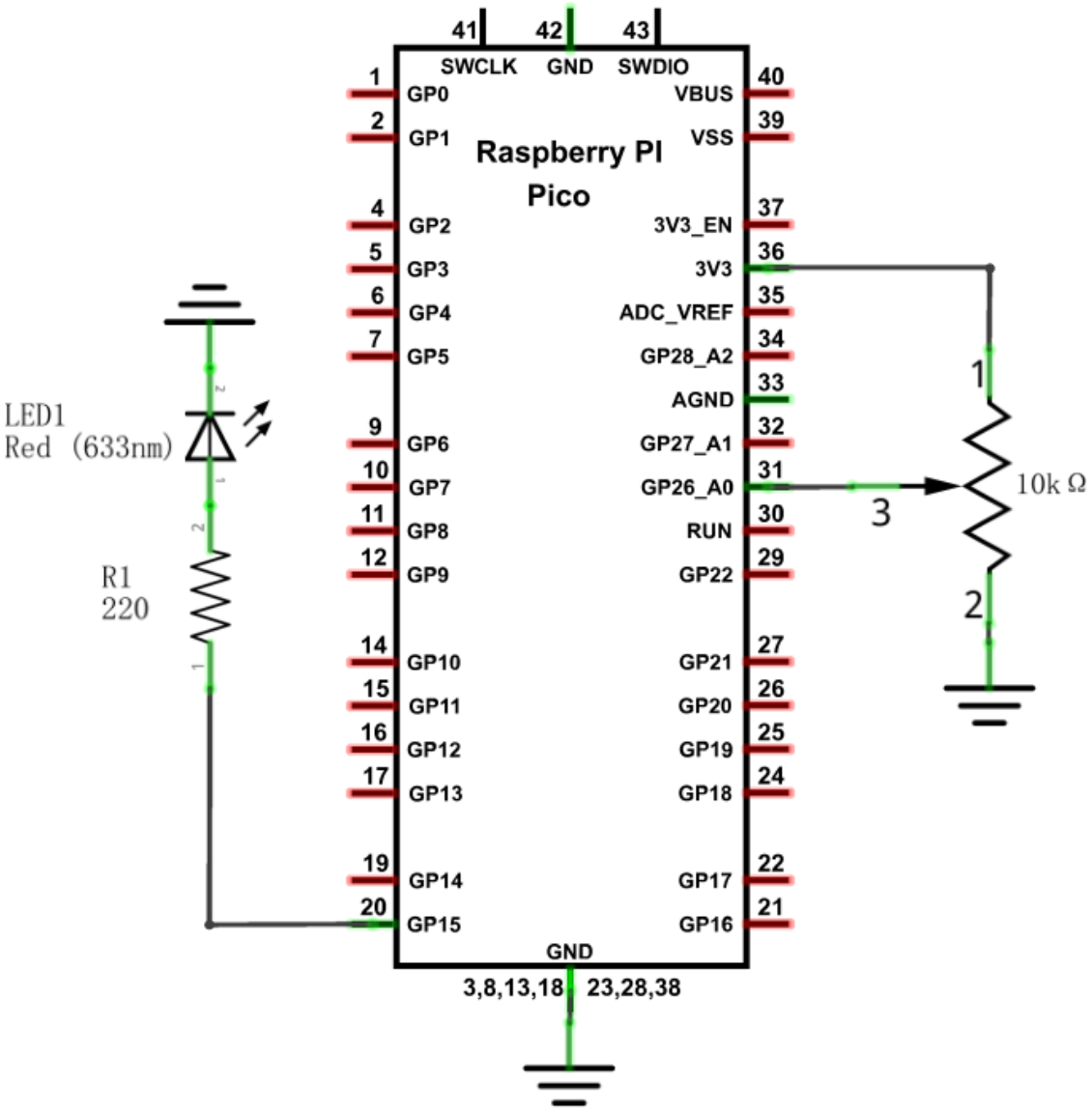
In this project, we will make a soft light. We will use an ADC Module to read ADC values of a potentiometer and map it to duty cycle of the PWM used to control the brightness of an LED. Then you can change the brightness of an LED by adjusting the potentiometer.

Component List

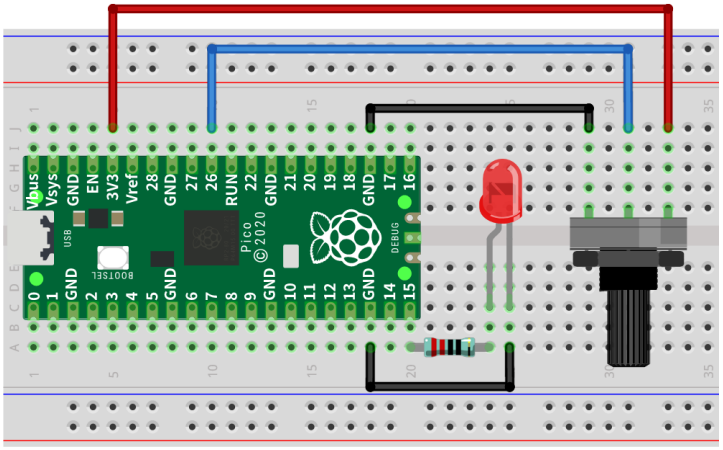
<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="4">Breadboard x1
  </td></tr>
<tr><td>Rotary potentiometer x1
  </td><td>Resistor 220Ω x1
  </td><td>LED x1
  </td><td>Jumper
</td></tr>
</table>

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



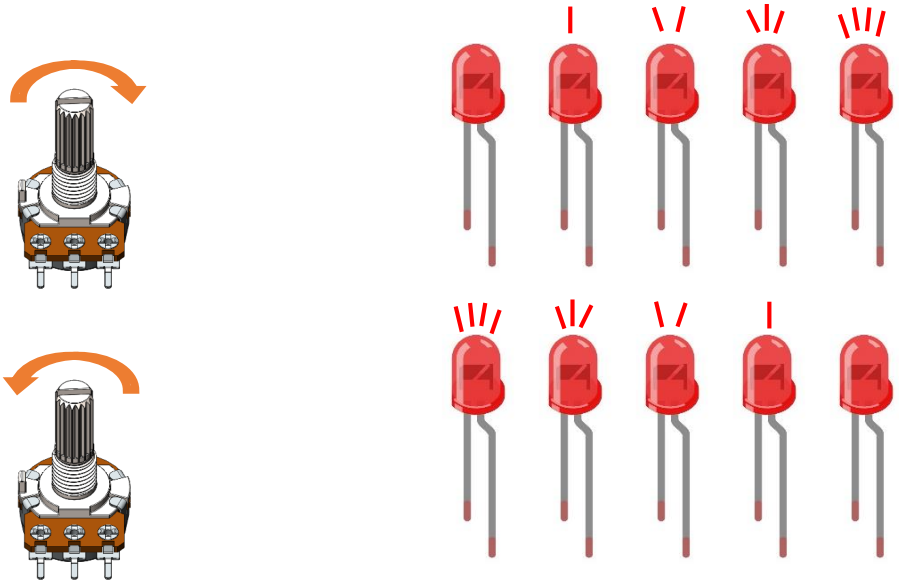
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "10.1_Soft_LED" and double click "10.1_Soft_LED.py".

10.1_Soft_LED



Click "Run current script". Rotate the handle of potentiometer and the brightness of LED will change correspondingly. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



If you have any concerns, please contact us via: support@freenove.com

Any concerns? ✉ support@freenove.com

The following is the code:

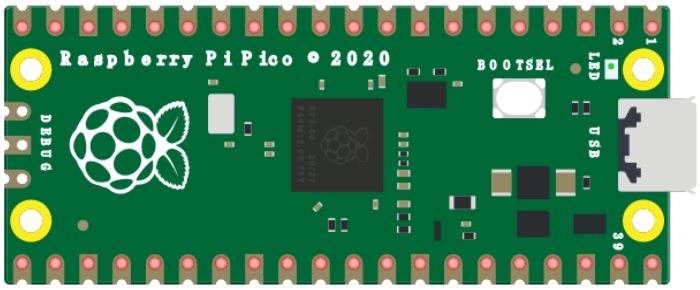
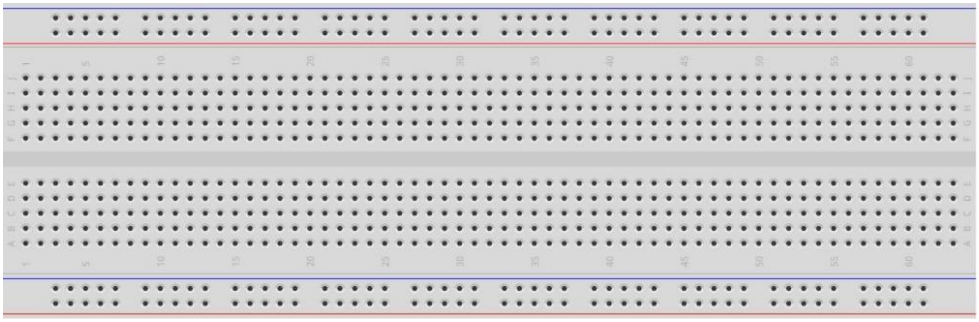
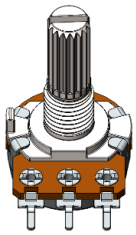
```
from machine import ADC, Pin, PWM
import time
adc = ADC(26)
pwm = PWM(Pin(15))
pwm.freq(1000)
try:
    while True:
        adcValue = adc.read_u16()
        pwm.duty_u16(adcValue)
        time.sleep(0.1)
except:
    pwm.deinit()
```

In the code, read the ADC value of potentiometer and map it to the duty cycle of PWM to control LED brightness.

Project 10.2 Soft Colorful Light

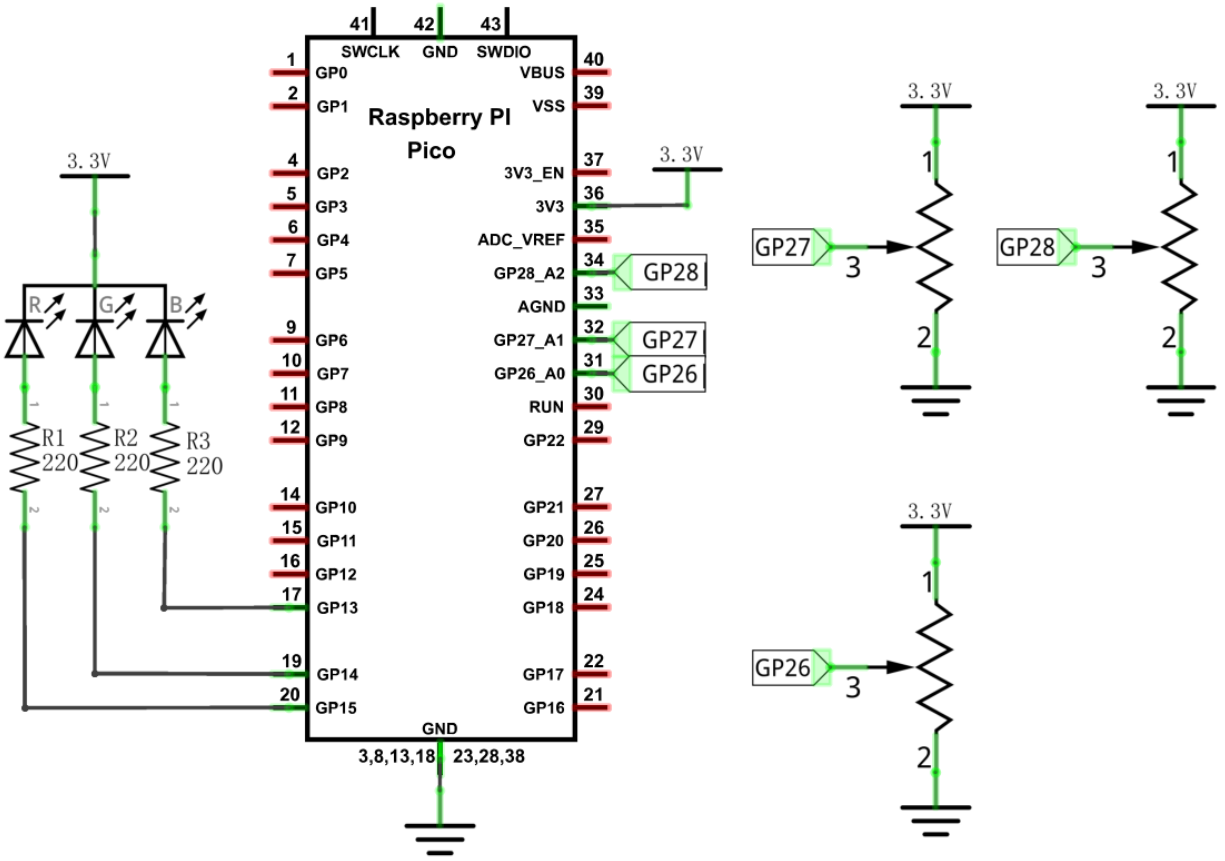
In this project, three potentiometers are used to control the RGB LED and in principle, it is the same as the Soft Light project. Namely, read the voltage value of the potentiometer and then convert it to PWM used to control LED brightness. Difference is that the original project only controlled one LED, but this project required (3) RGB LEDs.

Component List

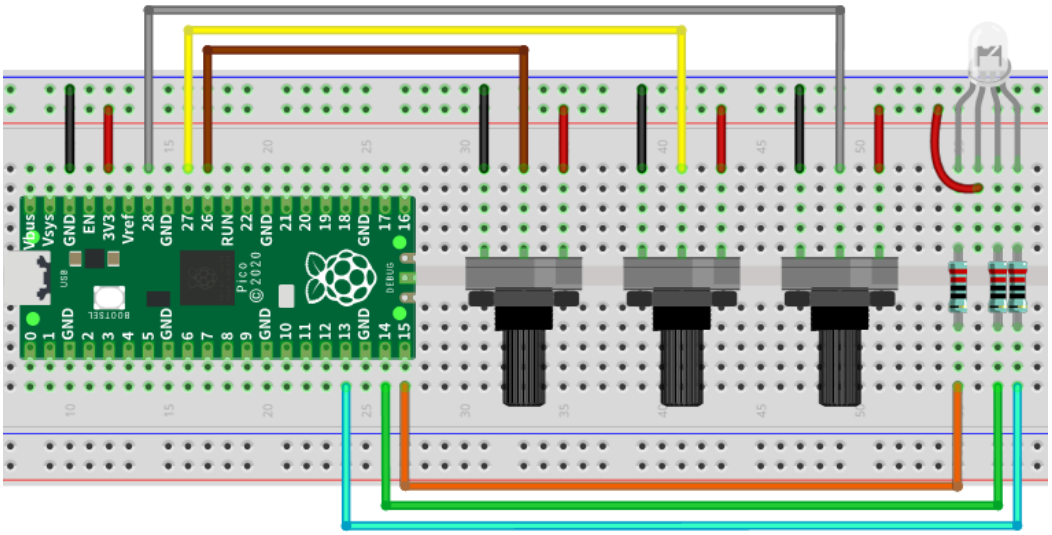
<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="4">Breadboard x1
  </td></tr>
<tr><td>Rotary potentiometer x3
  </td><td>Resistor 220Ω x3
  </td><td>RGBLED x1
  </td><td>Jumper
  </td></tr>
</table>

Circuit

Schematic diagram



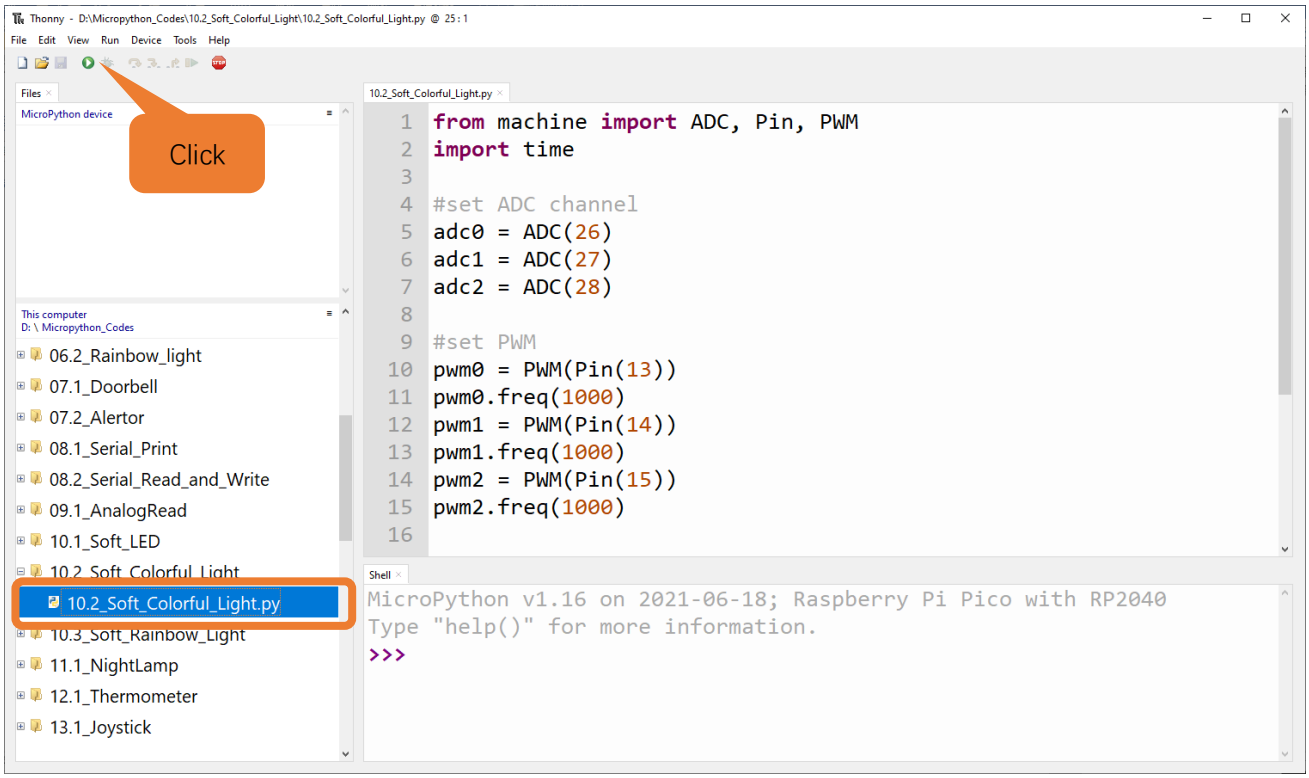
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "10.2_Soft_Colorful_Light" and double click "10.2_Soft_Colorful_Light.py".

10.2_Soft_Colorful_Light



Click "Run current script" and control the change of RGBLED color by rotating the handles of three rotary potentiometers. Press Ctrl+C or click "Stop/Restart backend" to exit the program.

The following is the program code:

```
from machine import ADC, Pin, PWM
import time

#set ADC channel
adc0 = ADC(26)
adc1 = ADC(27)
adc2 = ADC(28)

#set PWM
pwm0 = PWM(Pin(13))
pwm0.freq(1000)
pwm1 = PWM(Pin(14))
pwm1.freq(1000)
pwm2 = PWM(Pin(15))
pwm2.freq(1000)

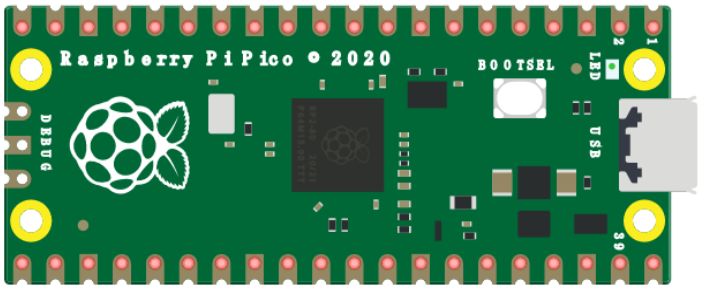
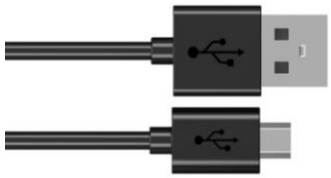
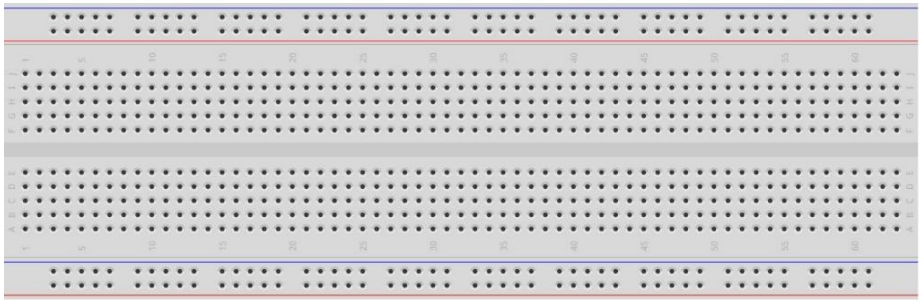
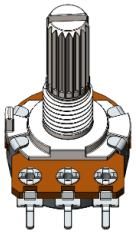
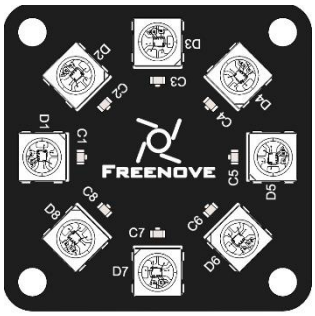
try:
    while True:
        pwm0.duty_u16(65535 - adc0.read_u16())
        pwm1.duty_u16(65535 - adc1.read_u16())
        pwm2.duty_u16(65535 - adc2.read_u16())
        time.sleep(0.1)
except:
    pwm0.deinit()
    pwm1.deinit()
    pwm2.deinit()
```

In the code, you can read the ADC values of the three potentiometers and map it into a PWM duty cycle to control the three LED elements to vary the color of their respective RGB LED.

Project 10.3 Soft Rainbow Light

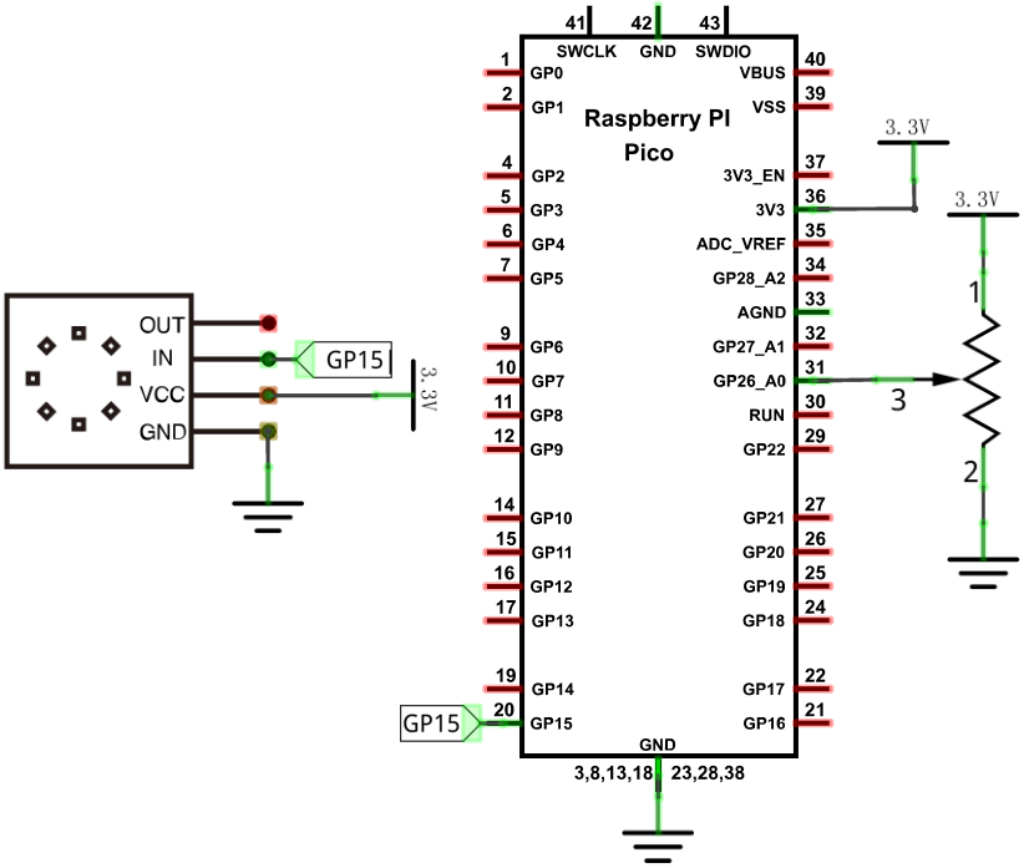
In this project, we use a potentiometer to control Freenove 8 RGBLED Module.

Component List

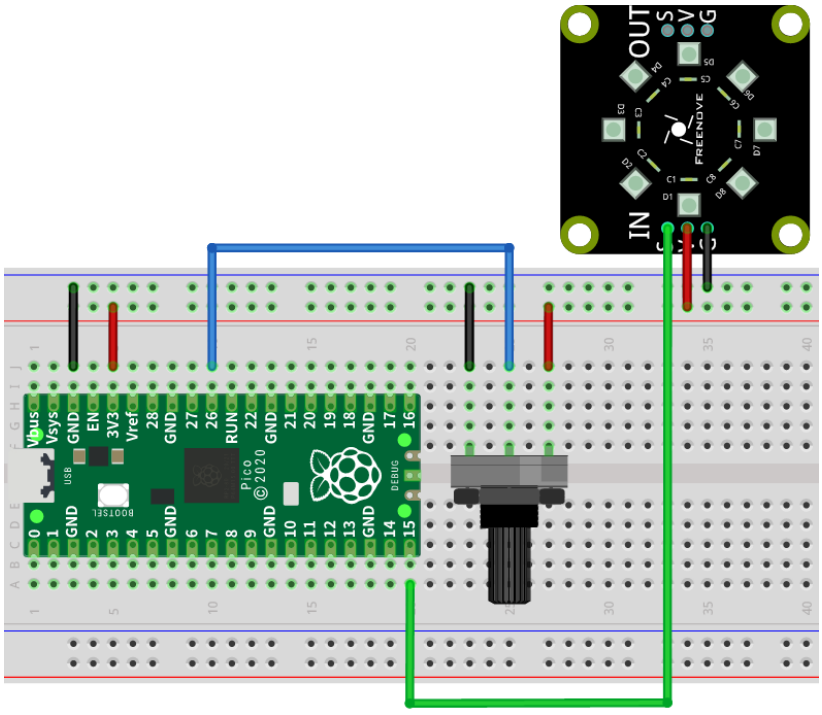
<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td>USB cable x1
  </td></tr>
<tr><td colspan="3">Breadboard x1
  </td></tr>
<tr><td>Rotary potentiometer x1
  </td><td>Freenove 8 RGB LED Module x1
  </td><td>Jumper
Jumper
  </td></tr>
</table>

Circuit

Schematic diagram



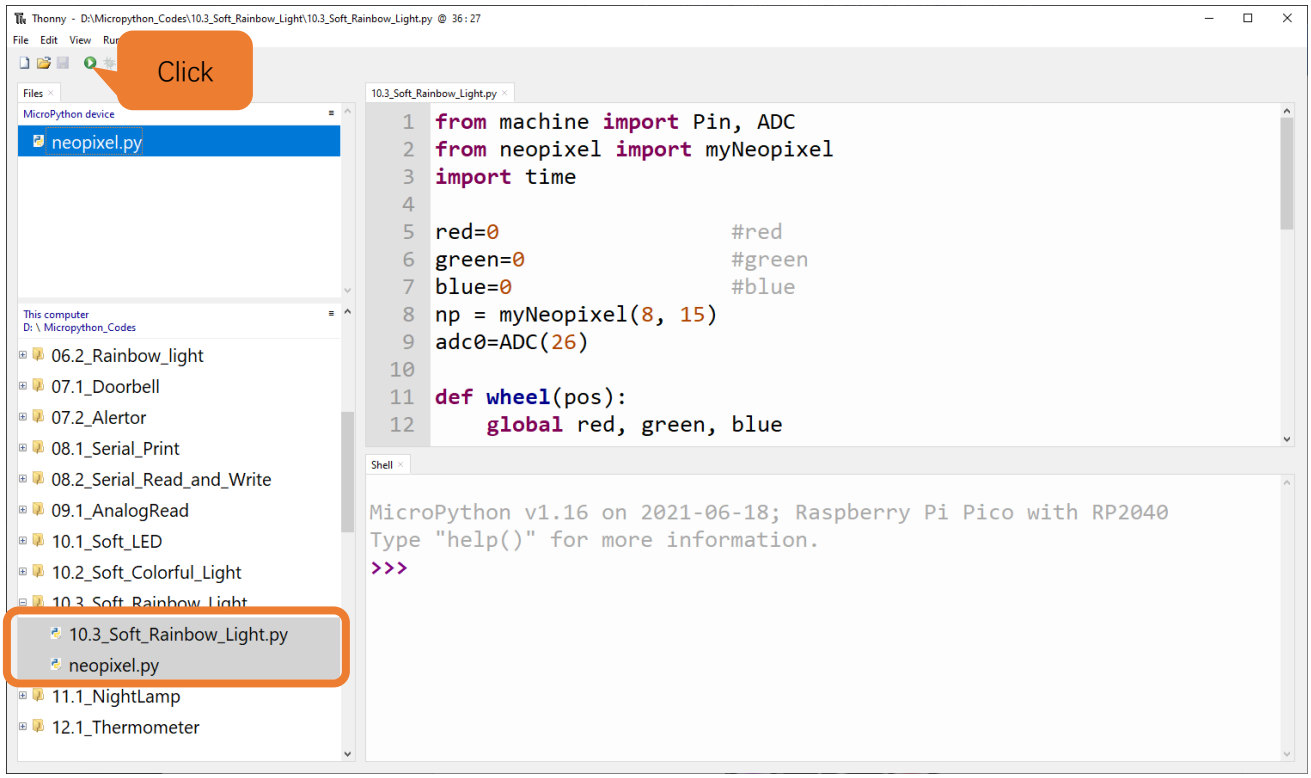
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "10.3_Soft_Rainbow_Light". Select"neopixel.py", right click to select "Upload to /", wait for "neopixel.py" to be uploaded to Raspberry Pi Pico and then double click "10.3_Soft_Rainbow_Light.py".

10.3_Soft_Rainbow_Light



Click "Run current script". Rotate the handle of potentiometer and the color of the lights will change.



If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
from machine import Pin, ADC
from neopixel import myNeopixel
import time

red=0                   #red
green=0                 #green
blue=0                  #blue
np = myNeopixel(8, 15)
adc0 = ADC(26)

def wheel(pos):
    global red, green, blue
    WheelPos=pos%255
    if WheelPos < 85:
        red = (255 - WheelPos*3)
        green = (WheelPos*3)
        blue = 0
    elif WheelPos >= 85 and WheelPos < 170:
        WheelPos -= 85
        red = 0
        green = (255 - WheelPos*3)
        blue = (WheelPos*3)
    else:
        WheelPos -= 170
        red = (WheelPos*3)
        green = 0
        blue = (255 - WheelPos*3)

np.brightness(20)
while True:
    for i in range(0, 255):
        for j in range(0, 8):
            wheel(i + j*255 // 8)
            np.set_pixel(j, red, green, blue)
        np.show()
        time.sleep_ms(1)
```

The logic of the code is basically the same as the previous project Rainbow Light. The difference is that in this code, the starting point of the color is controlled by the potentiometer.

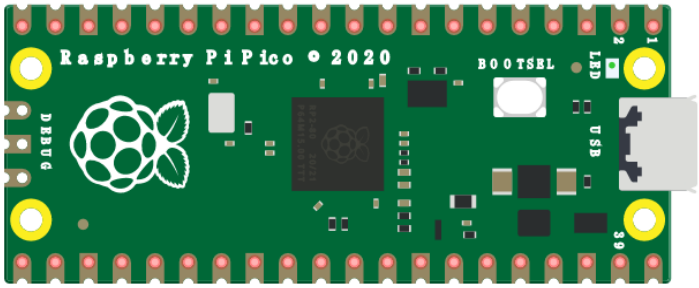
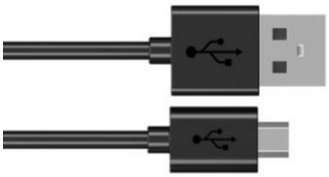
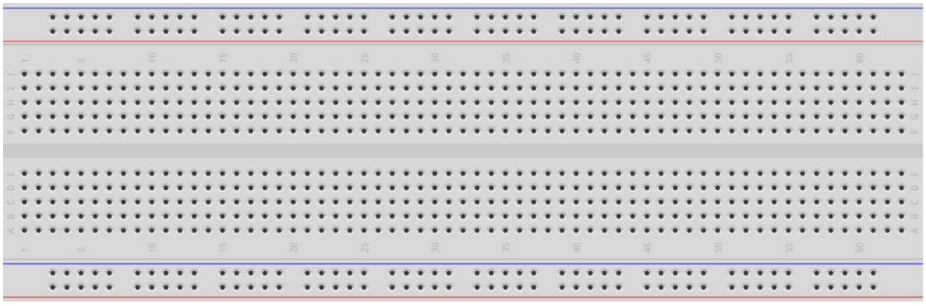
Chapter 11 Photoresistor & LED

In this chapter, we will learn how to use photoresistor.

Project 11.1 Control LED through Photoresistor

A Photoresistor is very sensitive to the amount of light present. We can take advantage of the characteristic to make a night lamp with the following function: when the ambient light is less (darker environment) the LED will automatically become brighter to compensate and when the ambient light is greater (brighter environment) the LED will automatically dim to compensate.

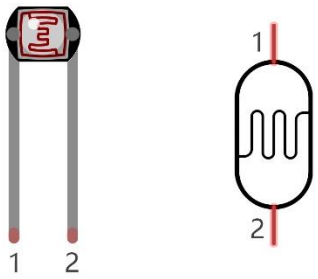
Component List

<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="4">Breadboard x1
  </td></tr>
<tr><td>Photoresistor x1
  </td><td colspan="2">Resistor
220Ω x1
 
10KΩ x1
  </td><td>LED x1
  </td><td>Jumper
</td></tr>
</table>

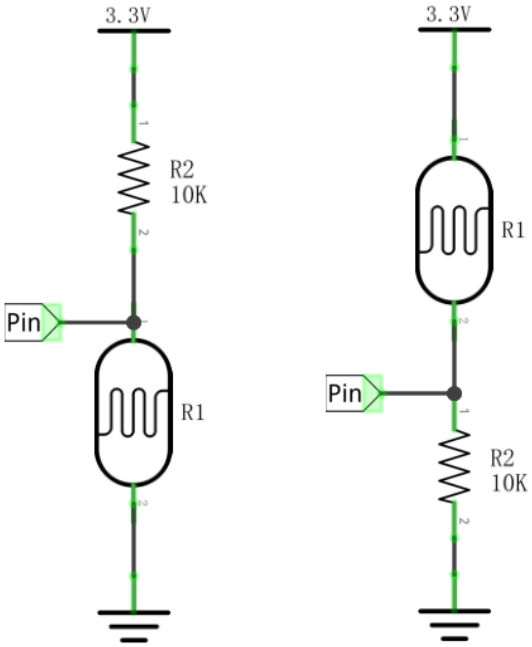
Component knowledge

Photoresistor

Photoresistor is simply a light sensitive resistor. It is an active component that decreases resistance with respect to receiving luminosity (light) on the component's light sensitive surface. Photoresistor's resistance value will change in proportion to the ambient light detected. With this characteristic, we can use a Photoresistor to detect light intensity. The Photoresistor and its electronic symbol are as follows.



The circuit below is used to detect the change of a Photoresistor's resistance value:

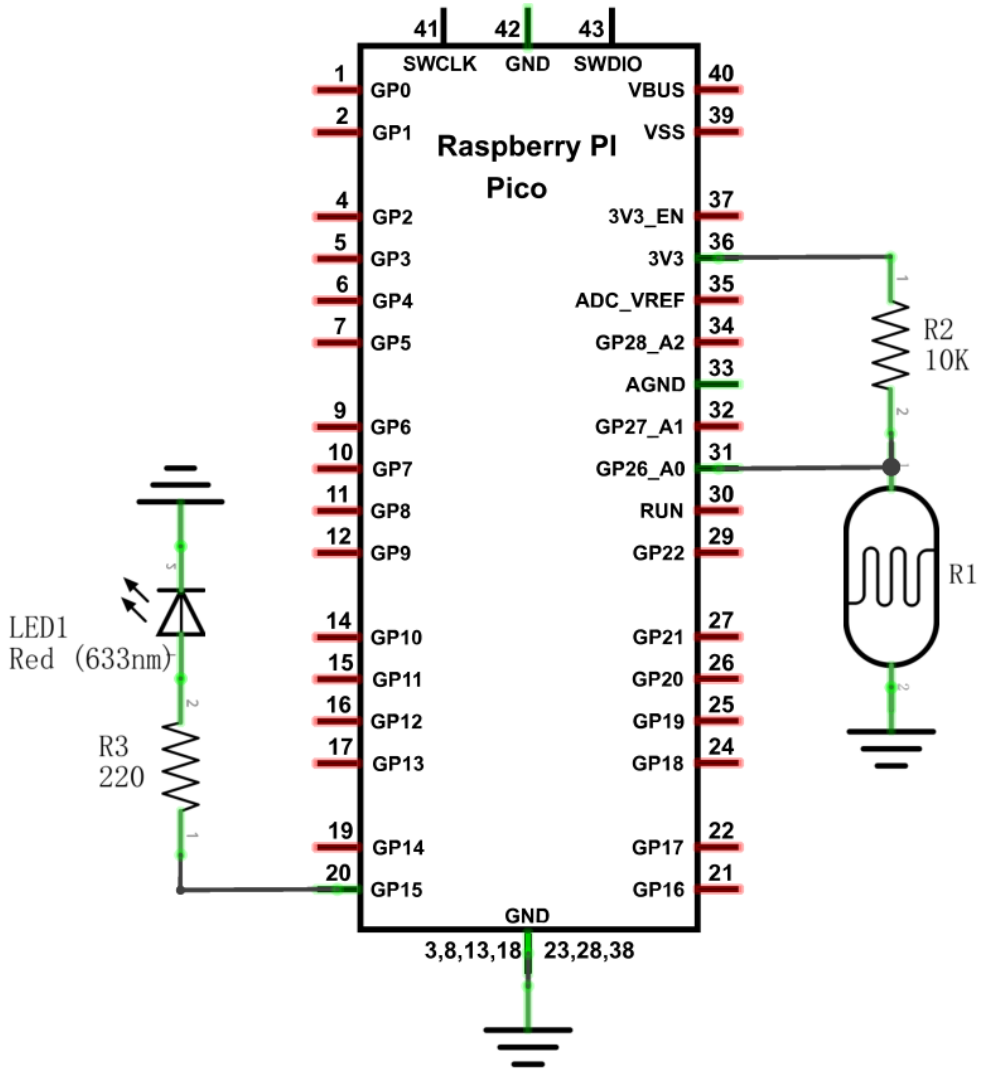


In the above circuit, when a Photoresistor's resistance value changes due to a change in light intensity, the voltage between the Photoresistor and Resistor R1 will also change. Therefore, the intensity of the light can be obtained by measuring this voltage.

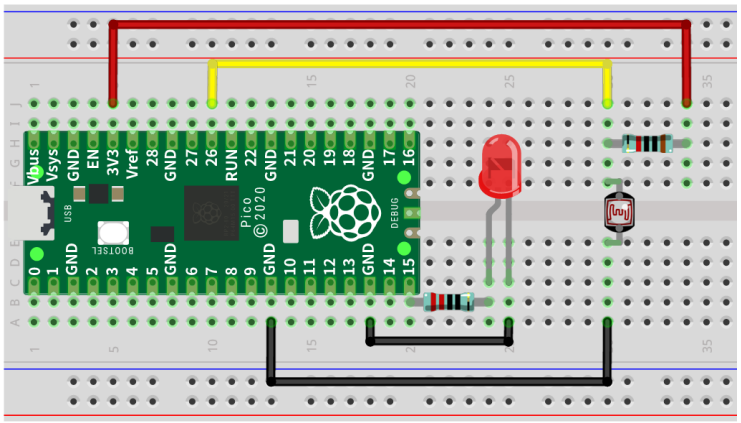
Circuit

The circuit of this project is similar to SoftLight. The only difference is that the input signal is changed from a potentiometer to a combination of a photoresistor and a resistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

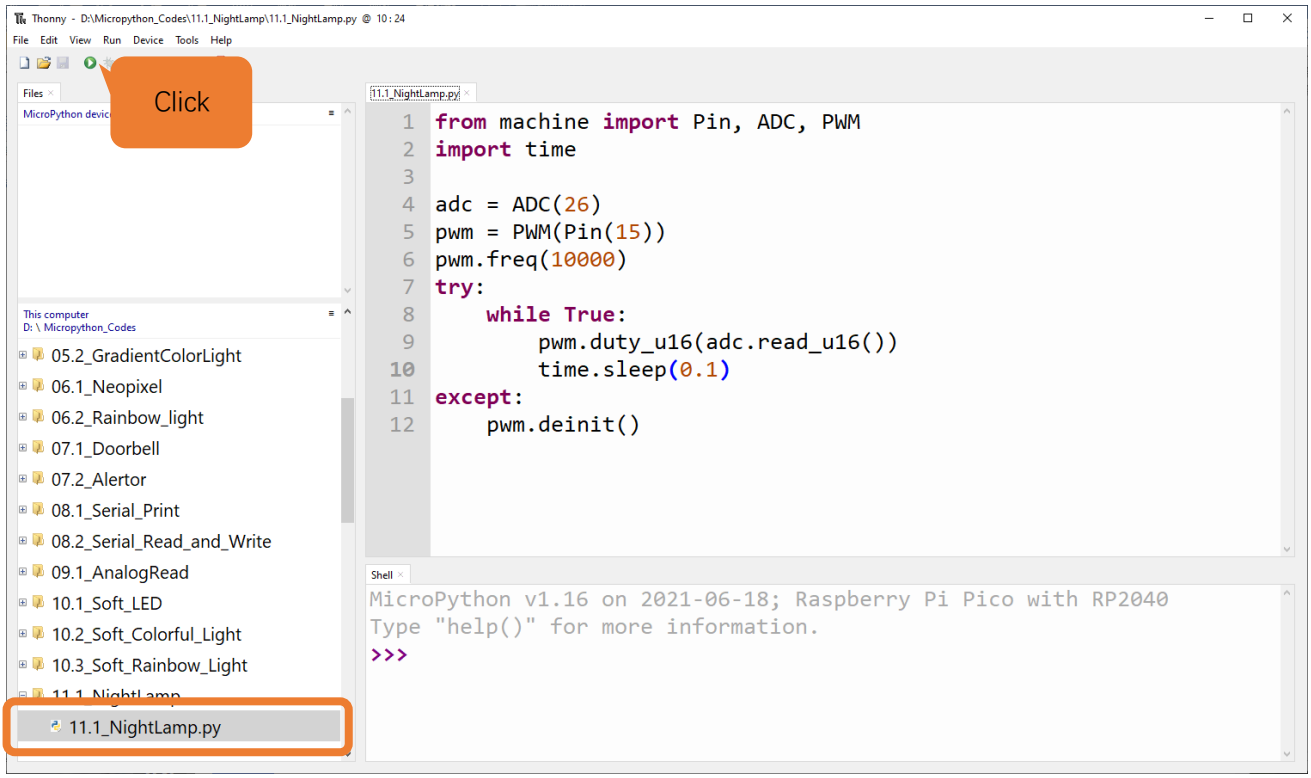


Code

Codes of this project is logically the same as the project Soft Light.

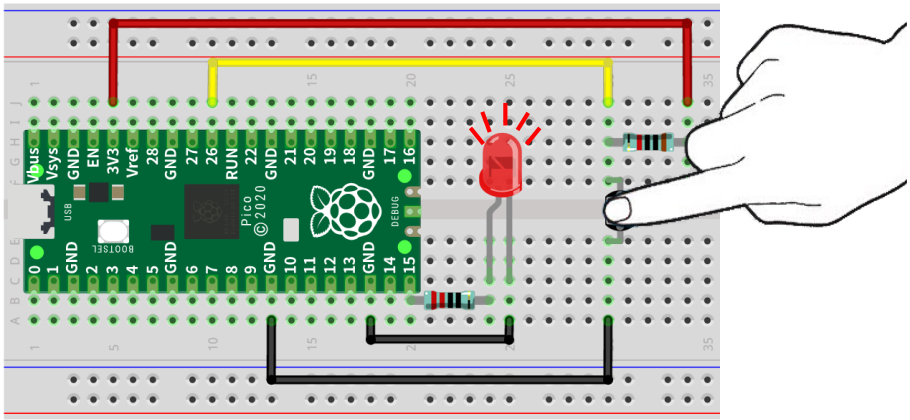
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "11.1_Photoresistor" and double click "11.1_Photoresistor.py".

11.1_Photoresistor

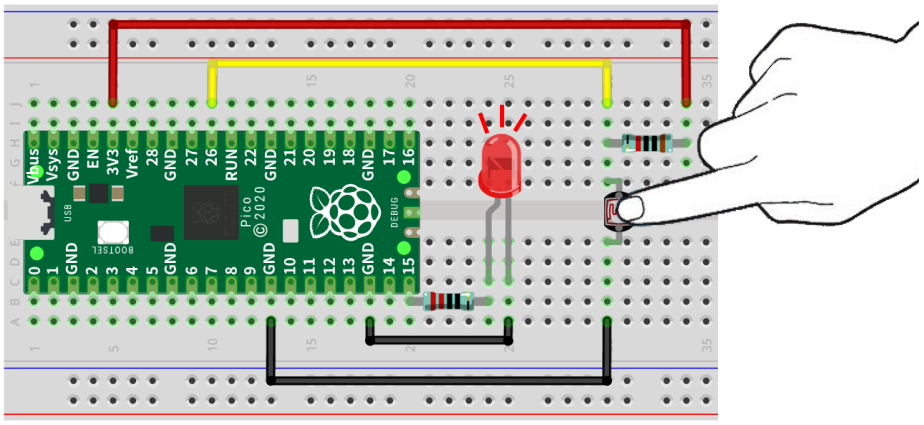


Click "Run current script". Cover the photoresistor with your hands or illuminate it with lights, the brightness of LEDs will change.

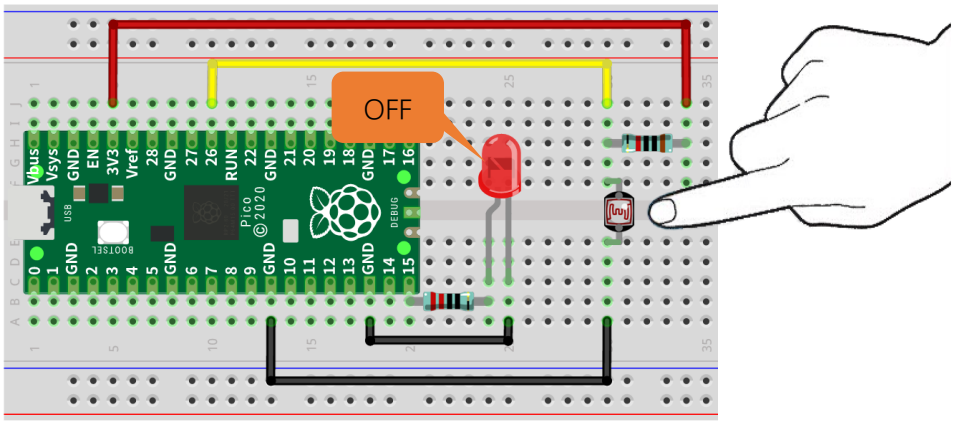
Fully cover the photoresistor:



Half cover the photoresistor:



Not cover the photoresistor:



The following is the program code:

```
from machine import Pin, ADC, PWM
import time

adc = ADC(26)
pwm = PWM(Pin(15))
pwm.freq(10000)
try:
    while True:
        pwm.duty_u16(adc.read_u16())
        time.sleep(0.1)
except:
    pwm.deinit()
```

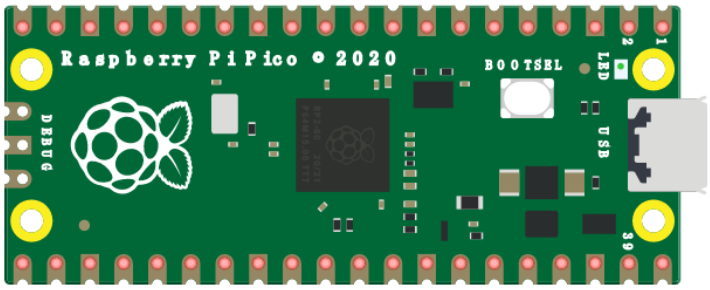
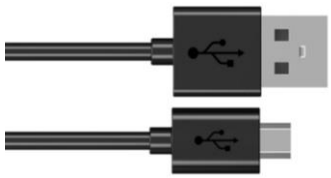
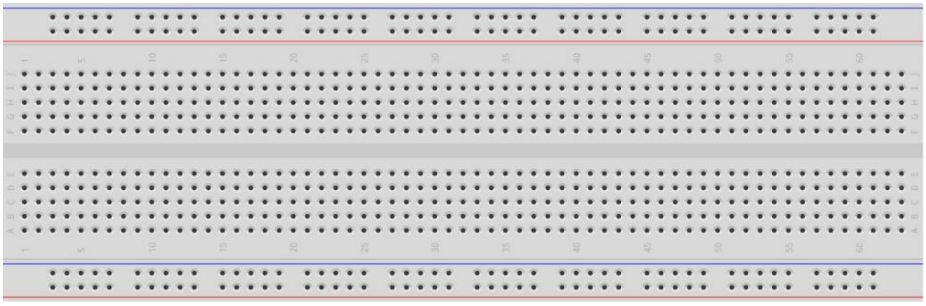
Chapter 12 Thermistor

In this chapter, we will learn about Thermistors, which are another kind of Resistor.

Project 12.1 Thermometer

A Thermistor is a type of Resistor whose resistance value is dependent on temperature and changes in temperature. Therefore, we can take advantage of this characteristic to make a Thermometer.

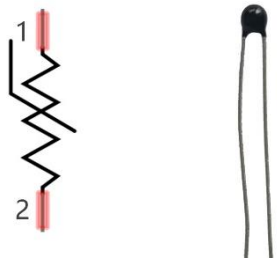
Component List

<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td>USB cable x1
  </td></tr>
<tr><td colspan="3">Breadboard x1
  </td></tr>
<tr><td>Thermistor x1
  </td><td>Resistor 10kΩ x1
  </td><td>Jumper
  </td></tr>
</table>

Component knowledge

Thermistor

A Thermistor is a temperature sensitive resistor. When it senses a change in temperature, the resistance of the Thermistor will change. We can take advantage of this characteristic by using a Thermistor to detect temperature intensity. A Thermistor and its electronic symbol are shown below.



The relationship between resistance value and temperature of a thermistor is:

$$Rt = R * EXP[\ B * (\frac{1}{T2} - \frac{1}{T1})\ ]$$

Where:

Rt is the thermistor resistance under T2 temperature;

R is the nominal resistance of thermistor under T1 temperature;

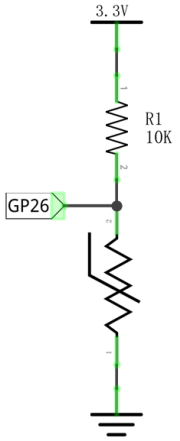
EXP[n] is nth power of e;

B is for thermal index;

T1, T2 is Kelvin temperature (absolute temperature). Kelvin temperature=273.15 + Celsius temperature.

For the parameters of the Thermistor, we use: B=3950, R=10kΩ, T1=25℃.

The circuit connection method of the Thermistor is similar to photoresistor, as the following:



We can use the value measured by the ADC converter to obtain the resistance value of Thermistor, and then we can use the formula to obtain the temperature value.

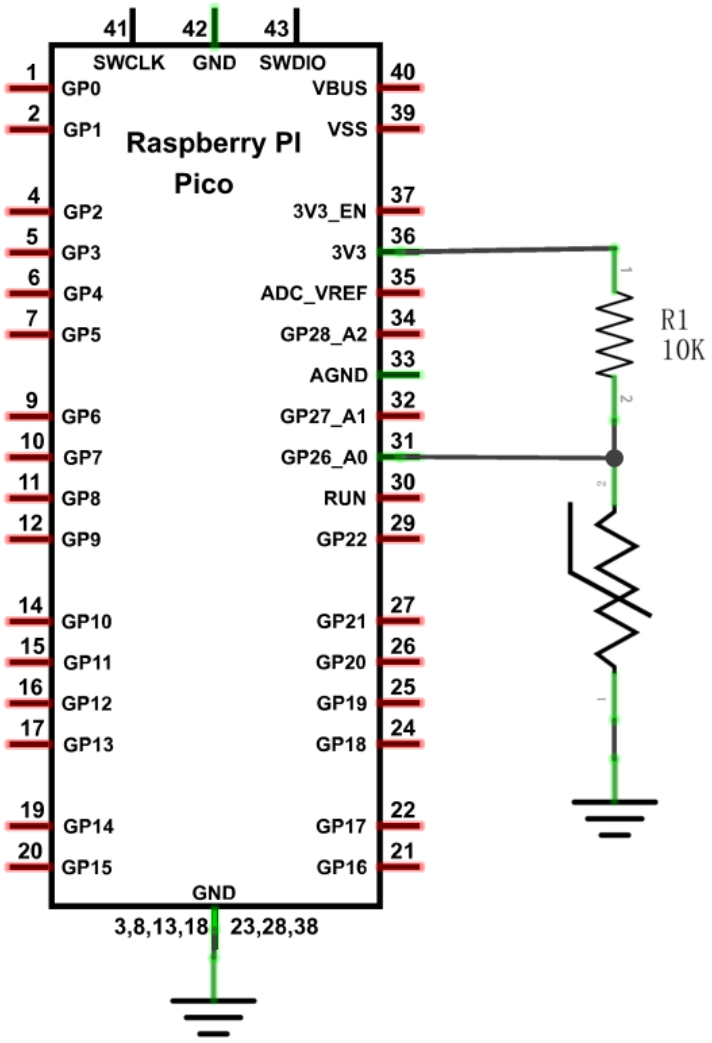
Therefore, the temperature formula can be derived as:

$$T2 = 1/(\frac{1}{T1} + \ln(\frac{Rt}{R})/B)$$

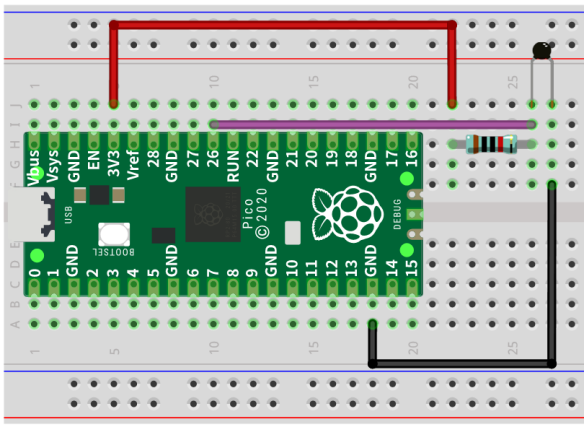
Circuit

The circuit of this project is similar to the one in the previous chapter. The only difference is that the Photoresistor is replaced by a Thermistor.

Schematic diagram



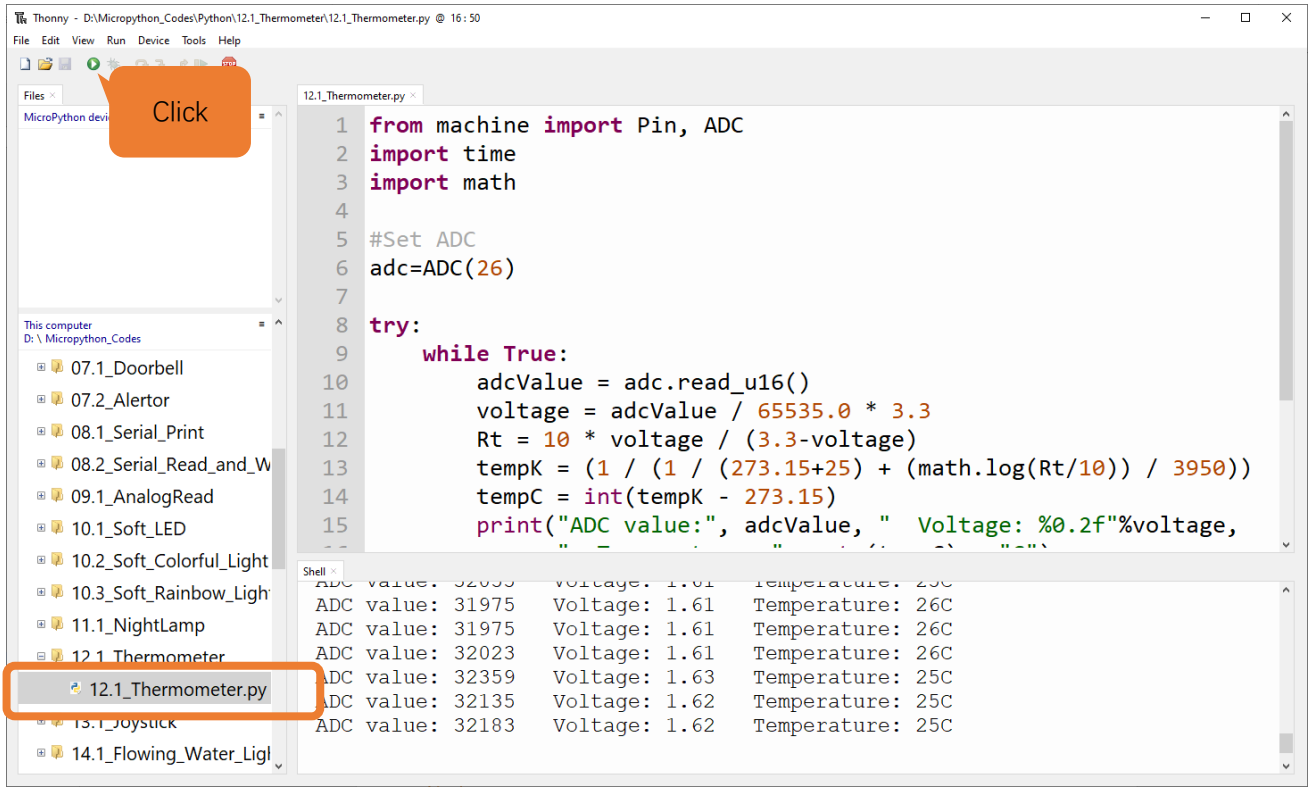
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



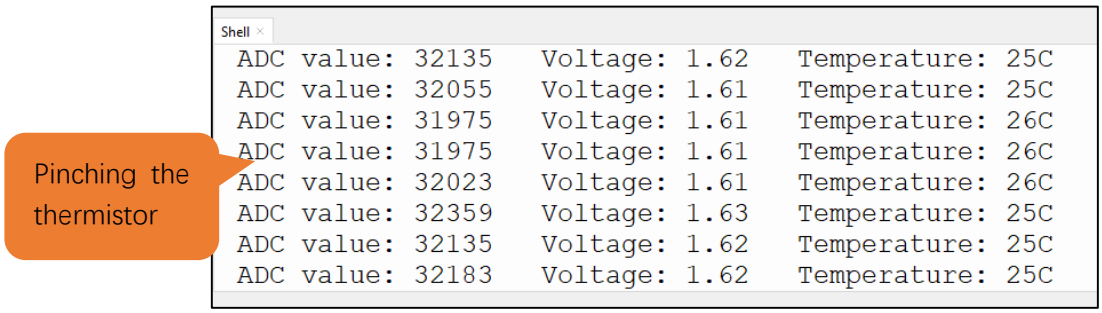
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "12.1_Thermometer" and double click "12.1_Thermometer.py".

12.1_Thermometer



Click "Run current script" and "Shell" will constantly display the current ADC value, voltage value and temperature value. Try to "pinch" the thermistor (without touching the leads) with your index finger and thumb for a brief time, you should see that the temperature value increases. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



If you have any concerns, please contact us via: support@freenove.com

The following is the code:

```
from machine import Pin, ADC
import time
import math

#Set ADC
adc=ADC(26)

try:
    while True:
        adcValue = adc.read_u16()
        voltage = adcValue / 65535.0 * 3.3
        Rt = 10 * voltage / (3.3-voltage)
        tempK = (1 / (1 / (273.15+25) + (math.log(Rt/10)) / 3950))
        tempC = int(tempK - 273.15)
        print("ADC value:", adcValue, "  Voltage: %0.2f"%voltage,
              "  Temperature: " + str(tempC) + "C")
        time.sleep(1)
except:
    pass
```

read_u16() function is called to read the value of ADC0.

```
    adcValue = adc.read_u16()
```

Convert the read ADC0 value to get the current Thermistor voltage value.

```
    voltage = adcValue / 65535.0 * 3.3
```

The current Thermistor resistance (Rt) is calculated by Ohm's law.

```
    Rt = 10 * voltage / (3.3-voltage)
```

According to the formula: $T2 = 1/(\frac{1}{T1} + \ln(\frac{Rt}{R})/B)$, where T1 = 25℃, R = 10KΩ, B = 3950 and the Rt calculated in the previous step, substitute the formula to calculate tempK(T2) . Get the value of the temperature unit K.

```
    tempK = (1 / (1 / (273.15+25) + (math.log(Rt/10)) / 3950))
```

Finally, tempK (unit: K) is converted to tempC (unit: °C). You can also convert to Fahrenheit based on your needs.

```
    tempC = int(tempK - 273.15)
```

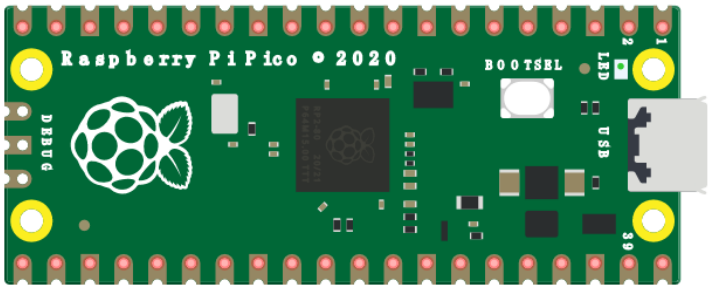
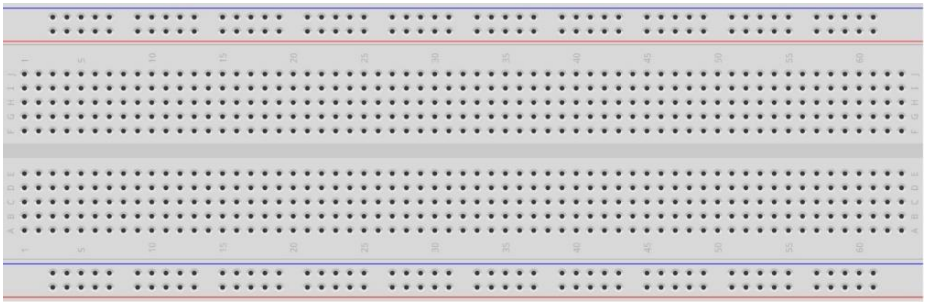
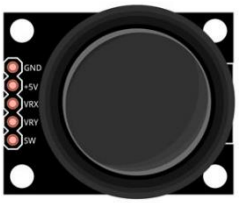
Chapter 13 Joystick

In the previous chapter, we have learned how to use rotary potentiometer. Now, let us learn a new electronic module Joystick that works on the same principle as rotary potentiometer.

Project 13.1 Joystick

In this project, we will read the output data of a Joystick and display it to the Terminal screen.

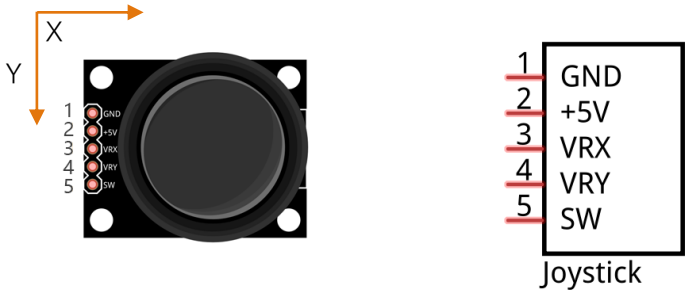
Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Joystick x1 	Jumper

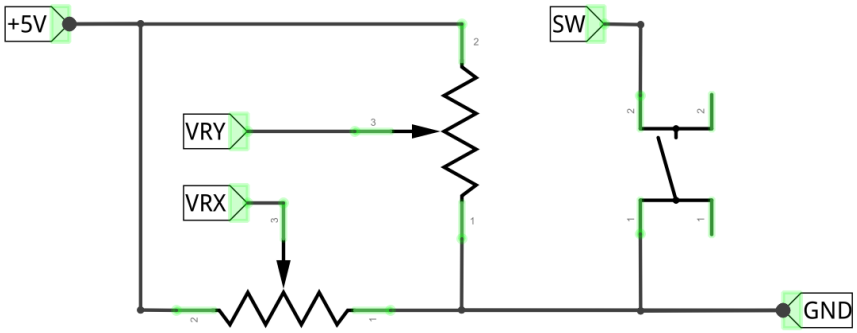
Component knowledge

Joystick

A Joystick is a kind of input sensor used with your fingers. You should be familiar with this concept already as they are widely used in gamepads and remote controls. It can receive input on two axes (Y and or X) at the same time (usually used to control direction on a two dimensional plane). It also has a third direction capability by pressing down (Z axis/direction).



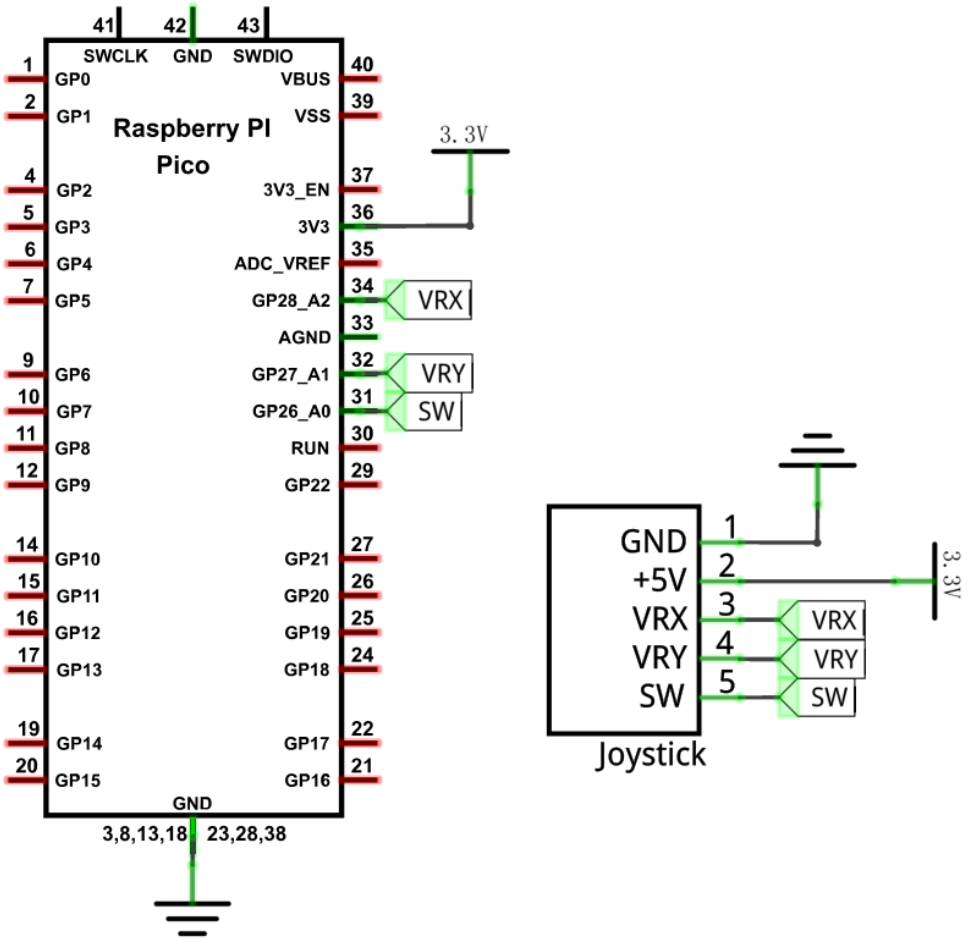
This is accomplished by incorporating two rotary potentiometers inside the Joystick Module at 90 degrees of each other, placed in such a manner as to detect shifts in direction in two directions simultaneously and with a Push Button Switch in the “vertical” axis, which can detect when a User presses on the Joystick.



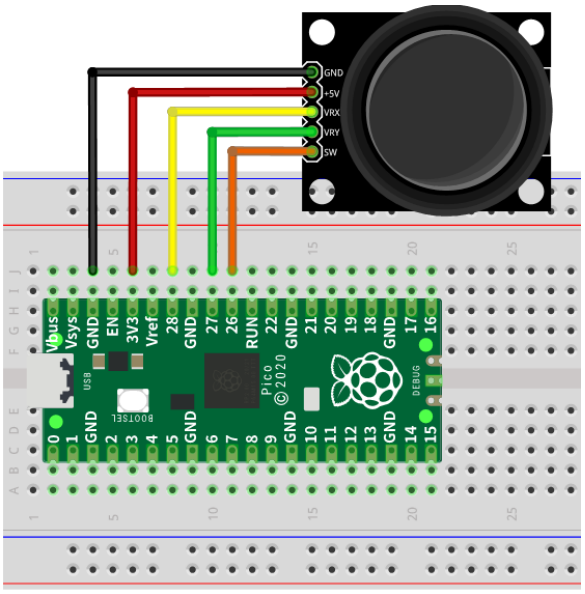
When the Joystick data is read, there are some differences between the axes: data of X and Y axes is analog, which needs to use the ADC. The data of the Z axis is digital, so you can directly use the GPIO to read this data or you have the option to use the ADC to read this.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via:support@freenove.com

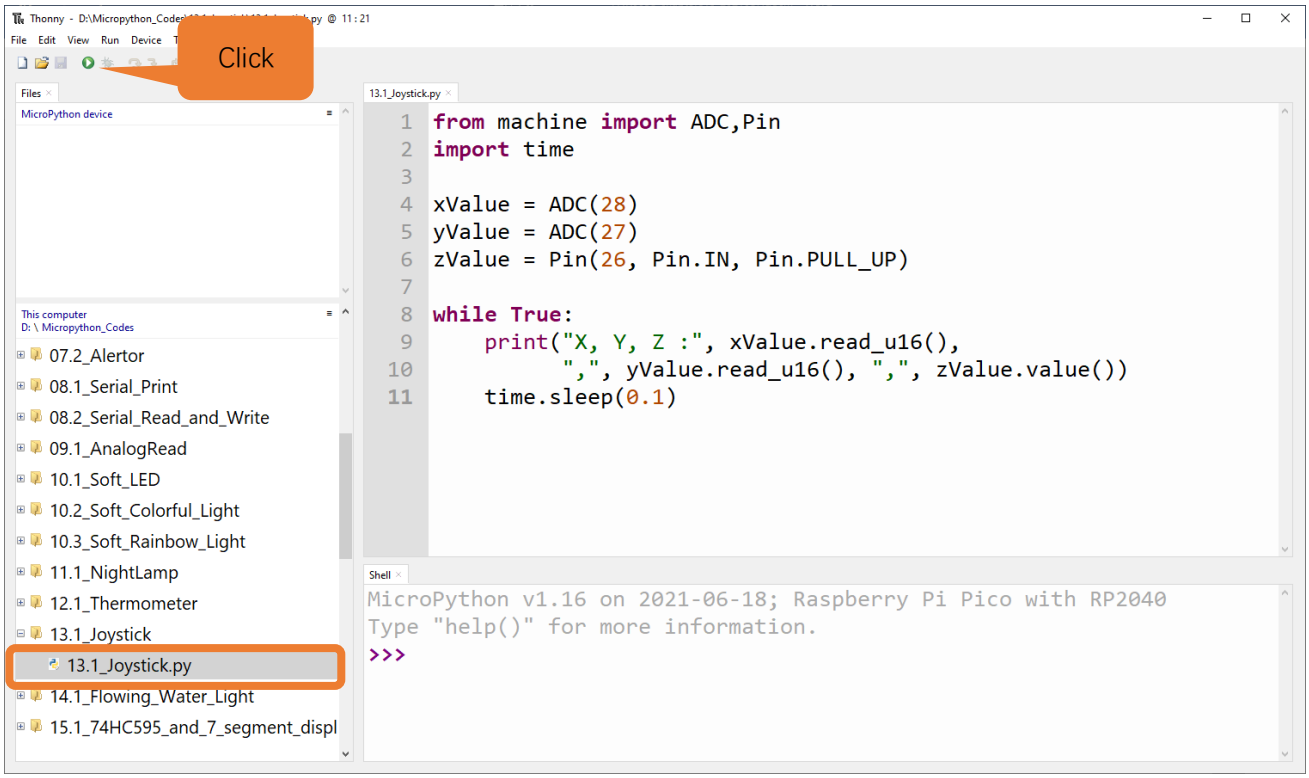


Code

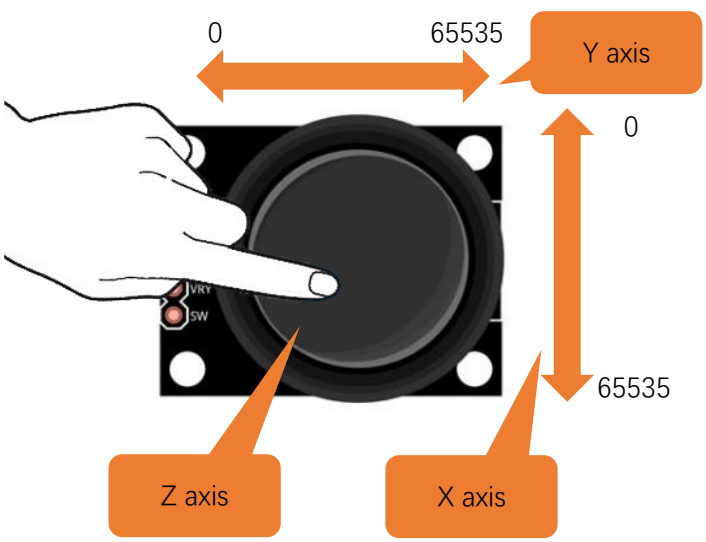
In this project's code, we will read the ADC values of X and Y axes of the Joystick, and read digital quality of the Z axis, then display these out in Terminal.

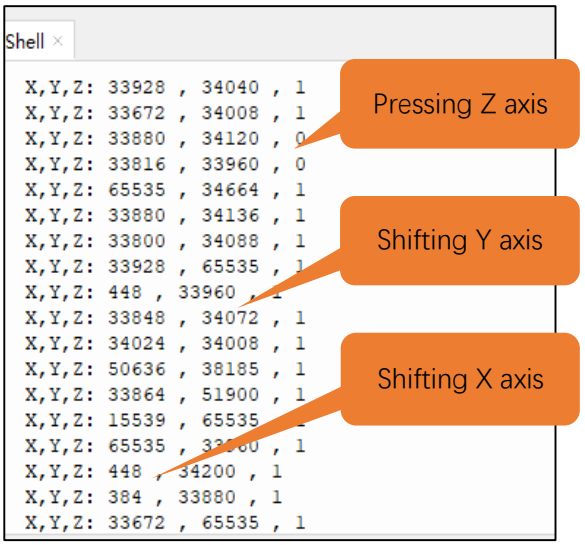
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "13.1_Joystick" and double click "13.1_Joystick.py".

13.1_Joystick



Click "Run current script". Shifting the Joystick or pressing it down will change the printed data in "Shell". Press Ctrl+C or click "Stop/Restart backend" to exit the program.





The flowing is the code:

```
from machine import ADC,Pin
import time

xValue = ADC(28)
yValue = ADC(27)
zValue = Pin(26, Pin.IN, Pin.PULL_UP)

while True:
    print("X, Y, Z :", xValue.read_u16(),
          ",", yValue.read_u16(), ",", zValue.value())
    time.sleep(0.1)
```

In the code, configure Z_Pin to pull-up input mode. In loop, use read_u16 () to read the value of axies X and Y and use value() to read the value of axis Z, and then display them.

```
    print("X, Y, Z :", xValue.read_u16(),
          ",", yValue.read_u16(), ",", zValue.value())
```

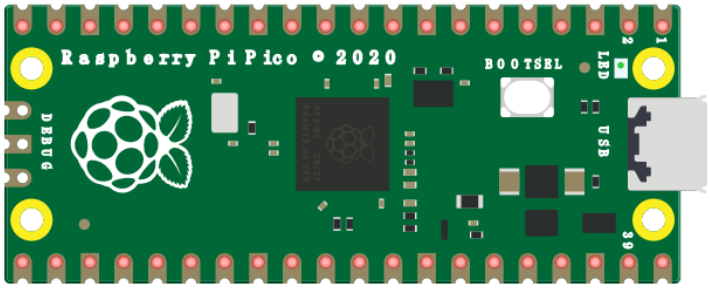
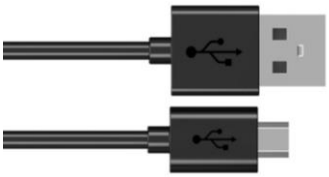
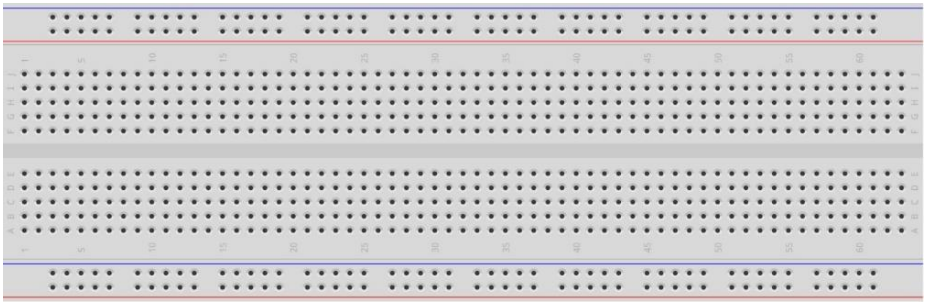
Chapter 14 74HC595 & LED Bar Graph

We have used LED Bar Graph to make a flowing water light, in which 10 GPIO ports of Raspberry Pi Pico is occupied. More GPIO ports mean that more peripherals can be connected to Raspberry Pi Pico, so GPIO resource is very precious. Can we make flowing water light with less GPIO? In this chapter, we will learn a component, 74HC595, which can achieve the target.

Project 14.1 Flowing Water Light

Now let us learn how to use the 74HC595 IC Chip to make a flowing water light using less GPIO.

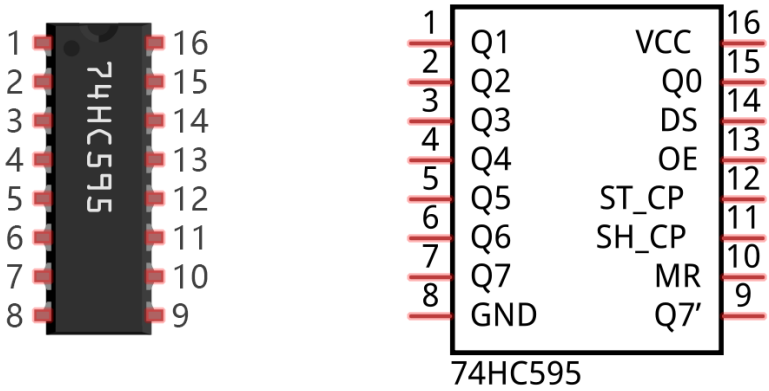
Component List

Raspberry Pi Pico x1 			USB cable x1 
Breadboard x1 			
74HC595 x1 	LED Bar Graph x1 	Resistor 220Ω x8 	Jumper 

Related knowledge

74HC595

A 74HC595 chip is used to convert serial data into parallel data. A 74HC595 chip can convert the serial data of one byte into 8 bits, and send its corresponding level to each of the eight ports correspondingly. With this characteristic, the 74HC595 chip can be used to expand the IO ports of Raspberry Pi Pico. At least three ports are required to control the eight ports of the 74HC595 chip.



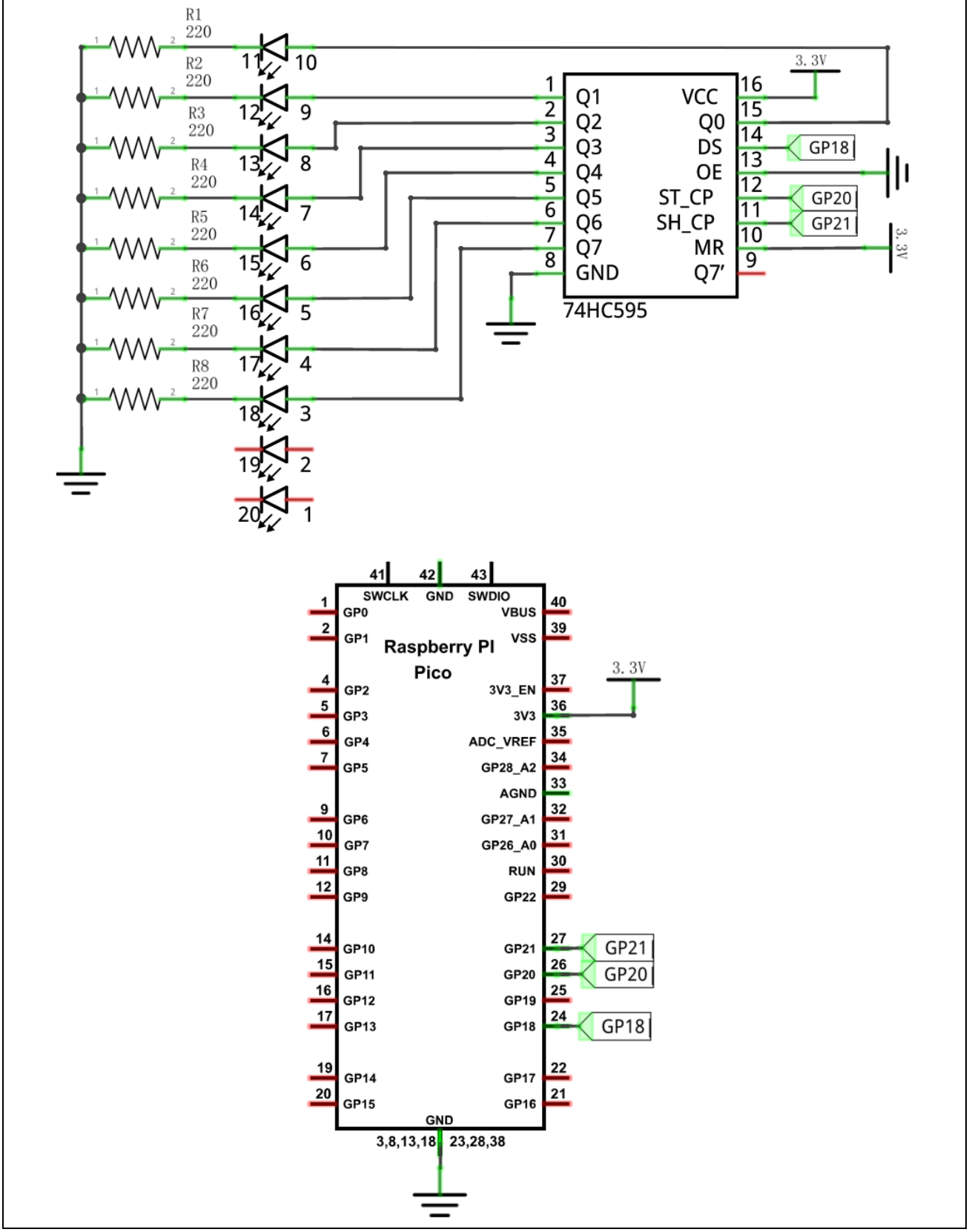
The ports of the 74HC595 chip are described as follows:

Pin name	GPIO number	Description
Q0-Q7	15, 1-7	Parallel data output
VCC	16	The positive electrode of power supply, the voltage is 2~6V
GND	8	The negative electrode of power supply
DS	14	Serial data Input
OE	13	Enable output, When this pin is in high level, Q0-Q7 is in high resistance state When this pin is in low level, Q0-Q7 is in output mode
ST_CP	12	Parallel Update Output: when its electrical level is rising, it will update the parallel data output.
SH_CP	11	Serial shift clock: When its electrical level is rising, serial data input, register would do a shift.
MR	10	Remove shift register: When this pin is in low level, the content in shift register will be cleared.
Q7'	9	Serial data output: it can be connected to more 74HC595 in series.

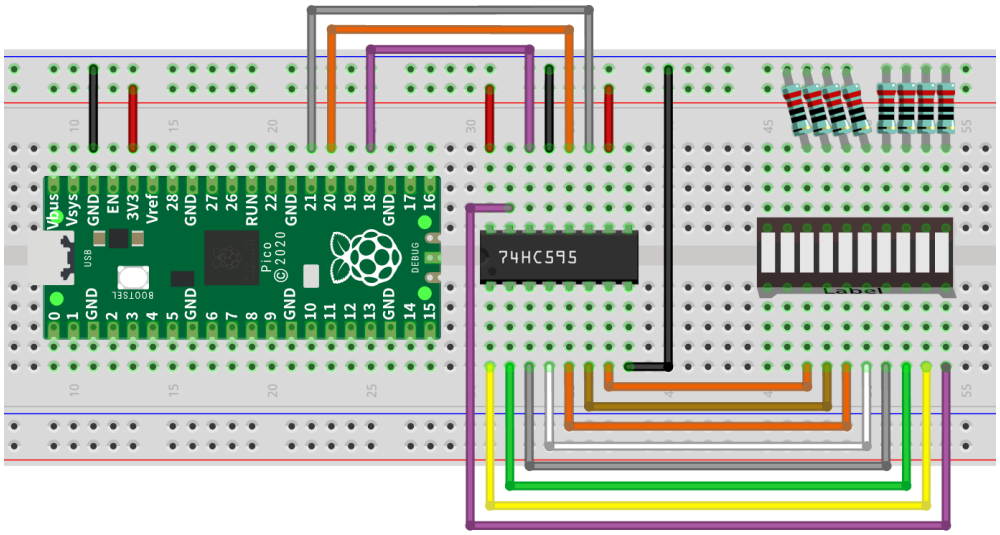
For more detail, please refer to the datasheet on the 74HC595 chip.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

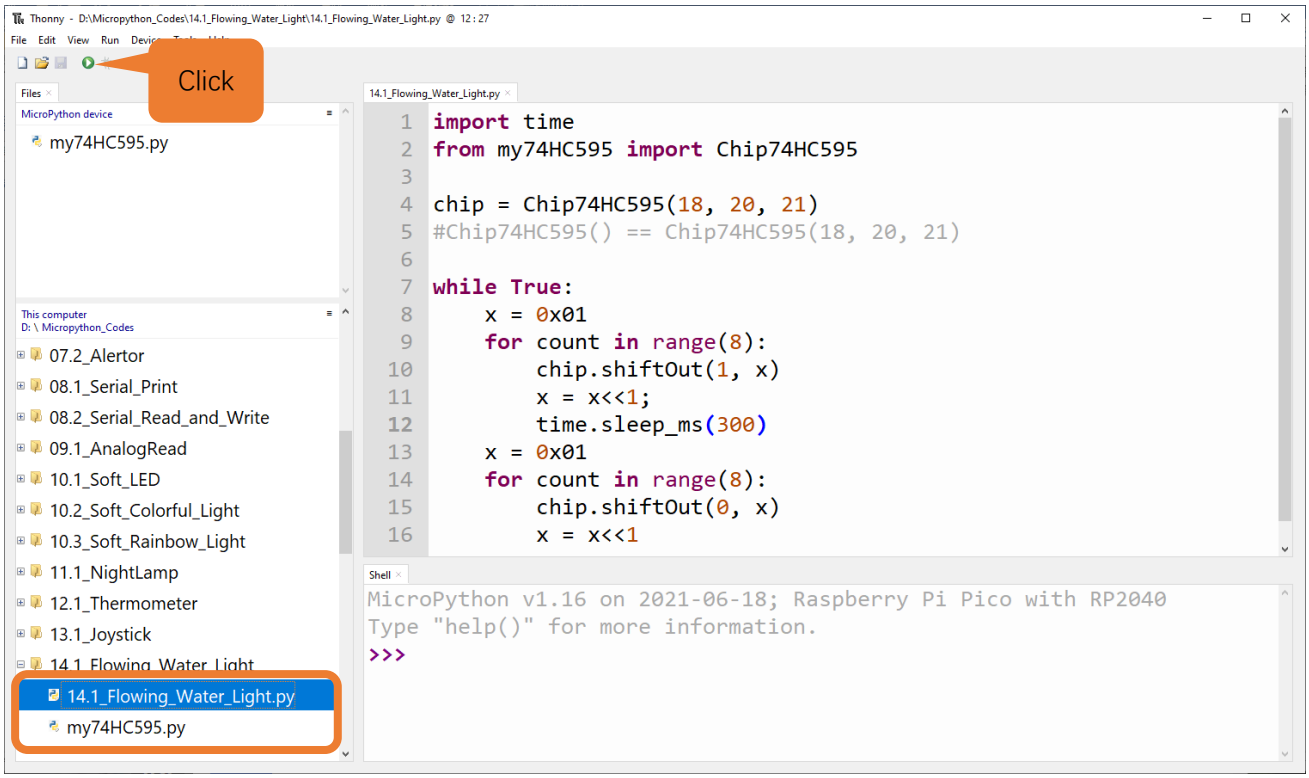


Code

In this project, we will make a flowing water light with a 74HC595 chip to learn about its functions.

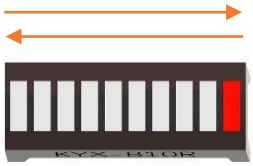
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "14.1_Flowing_Water_Light". Select "my74HC595.py", right click your mouse to select "Upload to /", wait for "my74HC595.py"to be uploaded to Raspberry Pi Pico and then double click "14.1_Flowing_Water_Light.py".

14.1_Flowing_Water_Light



Click"Run current script" and you will see that Bar Graph LED starts with the flowing water pattern blinking

from left to right and then back from right to left. If it displays nothing, maybe the LED Bar is connected upside down, please unplug it and then re-plug it reversely. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
import time
from my74HC595 import Chip74HC595

chip = Chip74HC595(18, 20, 21)
#Chip74HC595() == Chip74HC595(18, 20, 21)

while True:
    x=0x01
    for count in range(8):
        chip.shiftOut(1,x)
        x=x<<1;
        time.sleep_ms(300)
    x=0x01
    for count in range(8):
        chip.shiftOut(0,x)
        x=x<<1
        time.sleep_ms(300)
```

Import time and my74HC595 modules.

```
import time
from my74HC595 import Chip74HC595
```

Create a Chip74HC595 object and configure pins, among which, the default Raspberry Pi Pico and 74HC595 bound pins are DS(GP18), STCP(GP20), SHCP(GP21), OE(GP19). If you want to use other pins, you can change the pins by changing the arguments passed to the Chip74HC595 object.

```
chip = Chip74HC595(18, 20, 21)
#Chip74HC595() == Chip74HC595(18, 20, 21)
```

The first for loop makes LED Bar display separately from left to right while the second for loop make it display separately from right to left.

```
    x=0x01
    for count in range(8):
        chip.shiftOut(1,x)
        x=x<<1;
        time.sleep_ms(300)
    x=0x01
    for count in range(8):
```

```
15          chip.shiftOut(0,x)
16          x=x<<1
17          time.sleep_ms(300)
```

Reference

Class Chip74HC595

Before each use of the object **Chip74HC595**, make sure my74HC595.py has been uploaded to "/" of Raspberry Pi Pico, and then add the statement "**from my74HC595 import Chip74HC595**" to the top of the python file.

Chip74HC595(): An object. By default, 74HC595's DS pin is connected to GP18 of Raspberry Pi Pico, ST_CP pin is connected to Raspberry Pi Pico GP20, SH_CP pin is connected to Raspberry Pi Pico GP21, OE pin is connected to Raspberry Pi Pico GP19. If you need to modify the pins, just do the following operations.

chip=Chip74HC595() or **chip=Chip74HC595(18, 20, 21, 19).**

shiftOut(direction, data): Write data to 74HC595.

direction: When direction=1, it indicates data is sent from the highest byte to the lowest byte (left to right) in turn; direction=0 indicates data is sent from the lowest byte to the highest byte (right to left) in turn.

data: The content that is sent, which is one-byte data.

clear(): Clear the latch data of 74HC595.

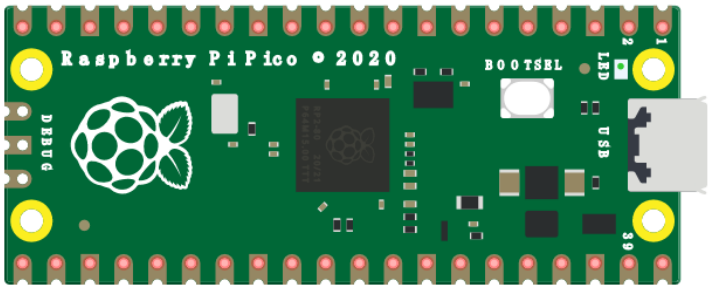
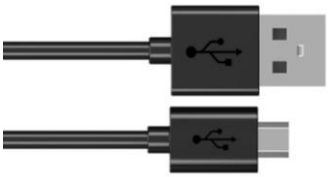
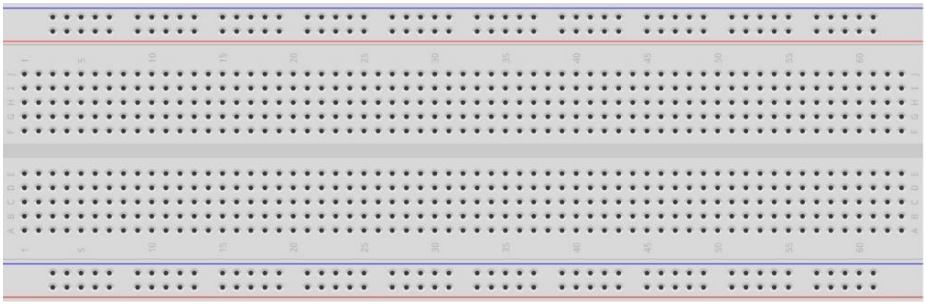
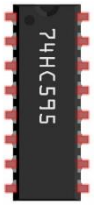
Chapter 15 74HC595 & 7-Segment Display.

In this chapter, we will introduce the 7-Segment Display.

Project 15.1 7-Segment Display.

We will use 74HC595 to control 7-segment display and make it display hexadecimal character "0-F".

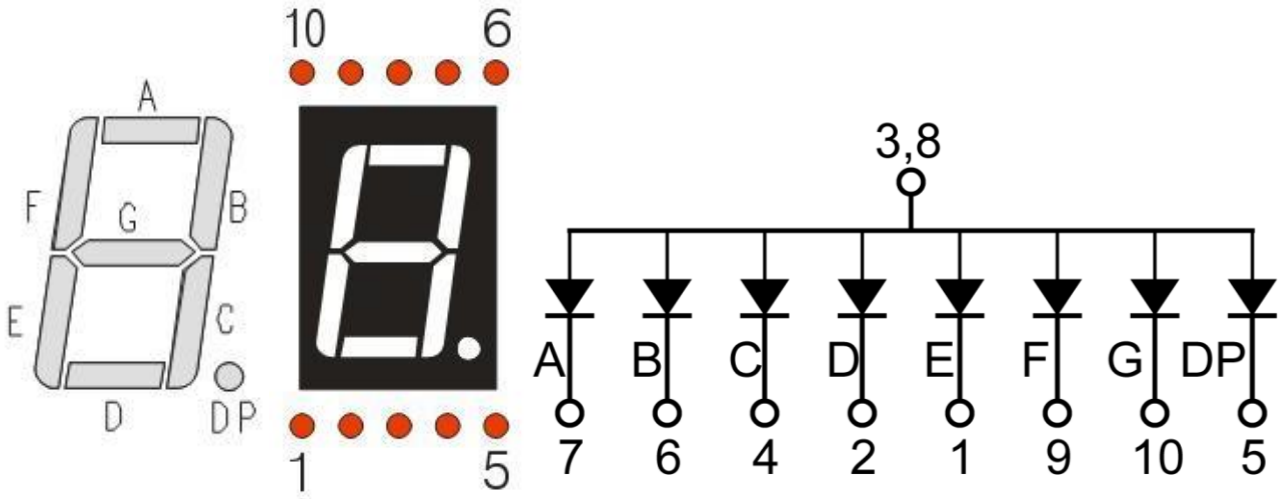
Component List

Raspberry Pi Pico x1 			USB cable x1 
Breadboard x1 			
74HC595 x1 	7-segment display x1 	Resistor 220Ω x8 	Jumper

Component knowledge

7-segment display

A 7-Segment Display is a digital electronic display device. There is a figure "8" and a decimal point represented, which consists of 8 LEDs. The LEDs have a Common Anode and individual Cathodes. Its internal structure and pin designation diagram is shown below:



As we can see in the above circuit diagram, we can control the state of each LED separately. In addition, by combining LEDs with different states of ON and OFF, we can display different characters (Numbers and Letters). For example, to display a “0”: we need to turn ON LED segments A, B, C, D, E and F, and turn OFF LED segments G and DP.



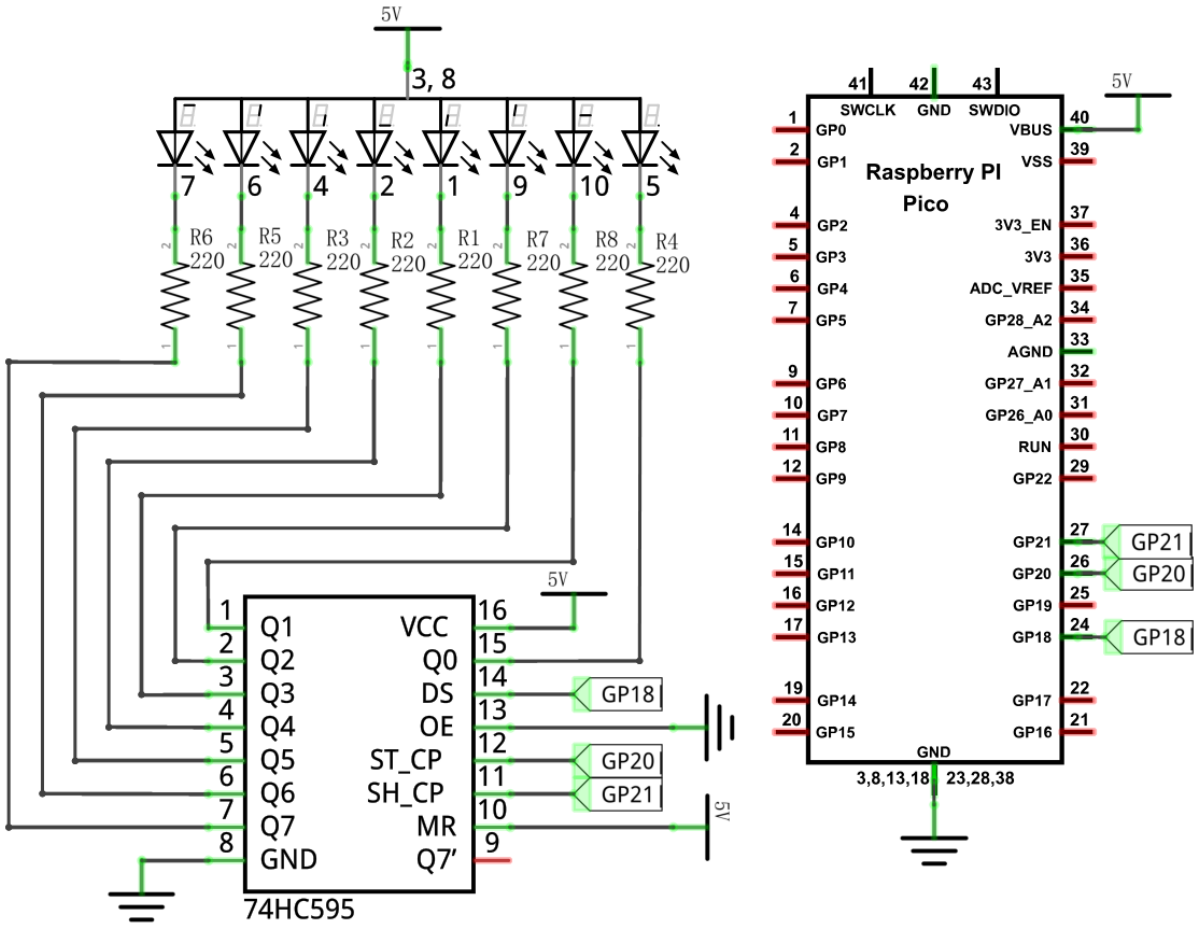
In this project, we will use a 7-Segment Display with a Common Anode. Therefore, when there is an input low level to an LED segment the LED will turn ON. Defining segment “A” as the lowest level and segment “DP” as the highest level, from high to low would look like this: “DP”, “G”, “F”, “E”, “D”, “C”, “B”, “A”. Character "0" corresponds to the code: 1100 0000b=0xc0.

For detailed code values, please refer to the following table (common anode).

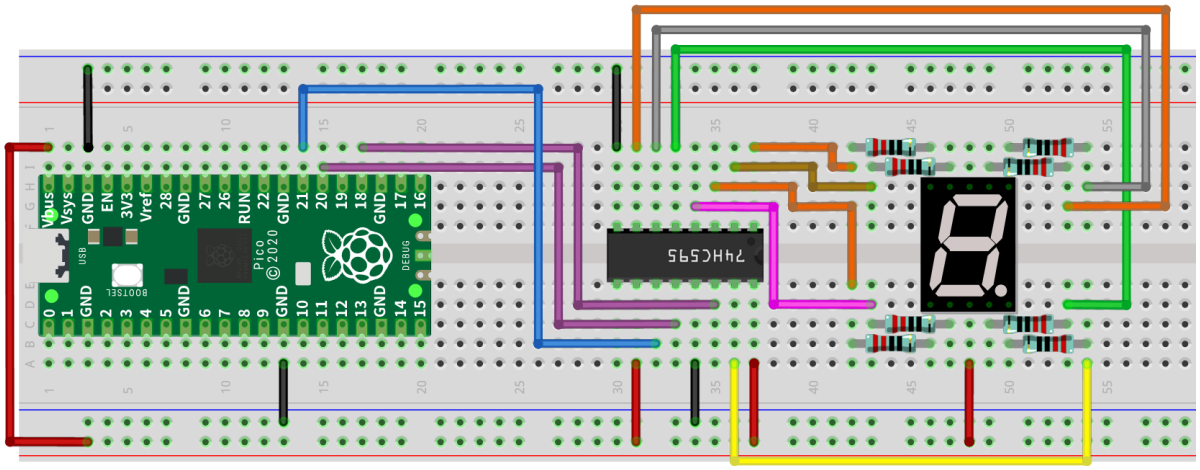
CHAR	DP	G	F	E	D	C	B	A	Hex	ASCII
0	1	1	0	0	0	0	0	0	0xc0	1100 0000
1	1	1	1	1	1	0	0	1	0xf9	1111 1001
2	1	0	1	0	0	1	0	0	0xa4	1010 0100
3	1	0	1	1	0	0	0	0	0xb0	1011 0000
4	1	0	0	1	1	0	0	1	0x99	1001 1001
5	1	0	0	1	0	0	1	0	0x92	1001 0010
6	1	0	0	0	0	0	1	0	0x82	1000 0010
7	1	1	1	1	1	0	0	0	0xf8	1111 1000
8	1	0	0	0	0	0	0	0	0x80	1000 0000
9	1	0	0	1	0	0	0	0	0x90	1001 0000
A	1	0	0	0	1	0	0	0	0x88	1000 1000
B	1	0	0	0	0	0	1	1	0x83	1000 0011
C	1	1	0	0	0	1	1	0	0xc6	1100 0110
D	1	0	1	0	0	0	0	1	0xa1	1010 0001
E	1	0	0	0	0	1	1	0	0x86	1000 0110
F	1	0	0	0	1	1	1	0	0x8e	1000 1110

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

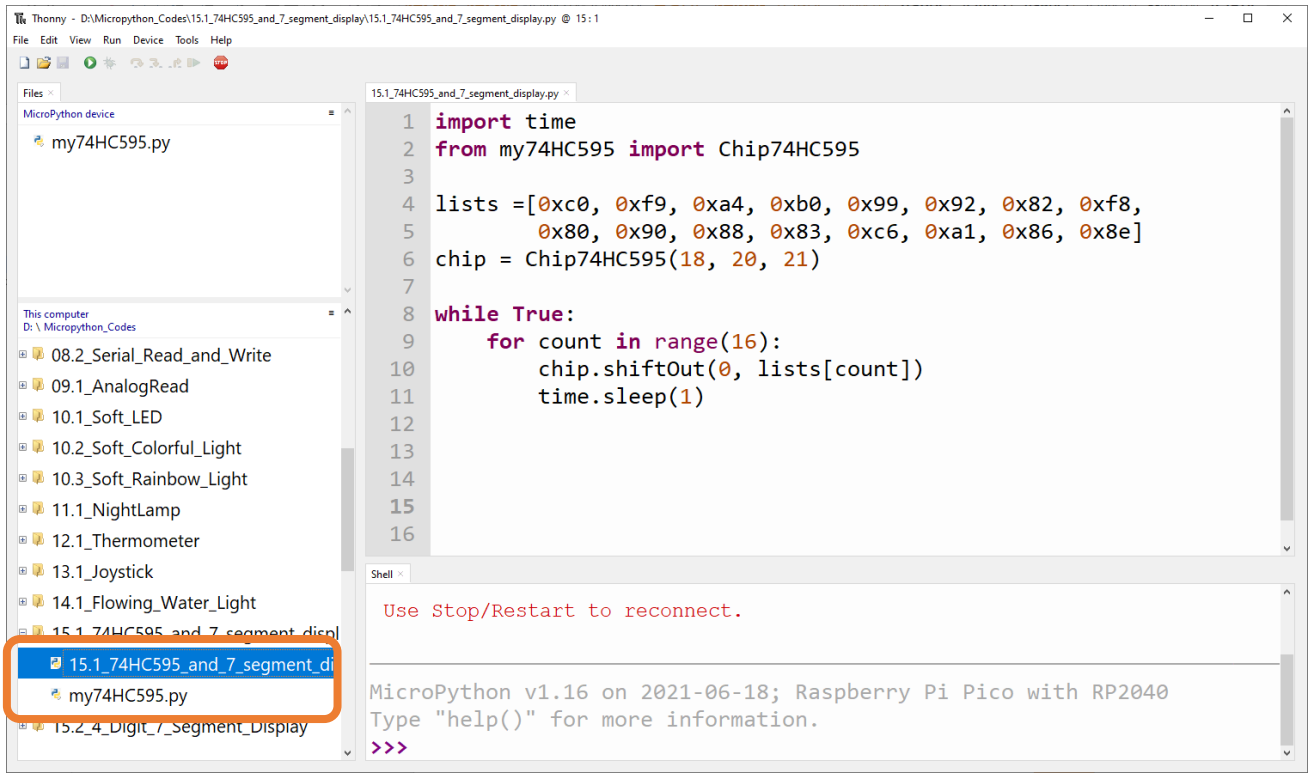


Code

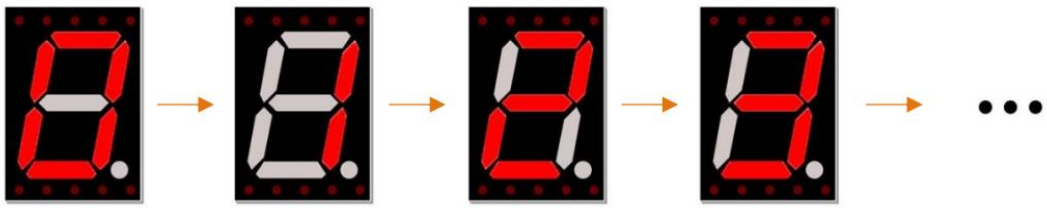
In this section, the 74HC595 is used in the same way as in the previous section, but with different values transferred. We can learn how to master the digital display by sending the code value of "0" - "F".

Open "Thonny", click "This computer"→"D:"→"Micropython_Codes"→ "15.1_74HC595_and_7_segment_display".Select "my74HC595.py", right click your mouse to select "Upload to /", wait for "my74HC595.py" to be uploaded to Raspberry Pi Pico and then double click "15.1_74HC595_and_7_segment_display.py".

15.1_74HC595_and_7_segment_display



Click "Run current script" and you'll see a 1-bit, 7-segment display displaying 0-f in a loop. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



The following is the program code:

```
1  import time
2  from my74HC595 import Chip74HC595
3
4  lists = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8,
5          0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
6  chip = Chip74HC595(18, 20, 21)
7
8  while True:
9      for count in range(16):
10         chip.shiftOut(0, lists[count])
11         time.sleep(1)
```

Import time and my74HC595 modules.

```
1  import time
2  from my74HC595 import Chip74HC595
```

Put the encoding **"0" - "F"** into the list.

```
4  lists =[0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8,
5          0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
```

Define an object, whose pins applies default configuration, to drive 74HC595.

```
6  chip = Chip74HC595(18, 20, 21)
```

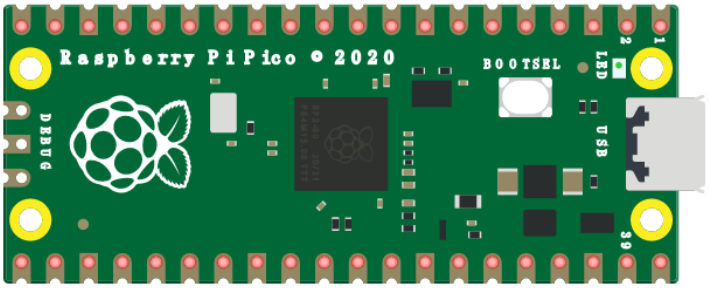
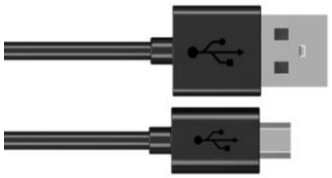
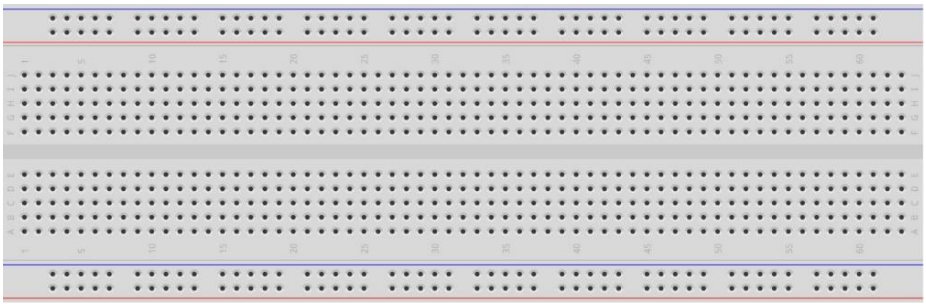
Send data of digital tube to 74HC595 chip.

```
10     chip.shiftOut(0, lists[count])
```

Project 15.2 4-Digit 7-Segment Display

Now, let us try to control a more-digit 7-segment display.

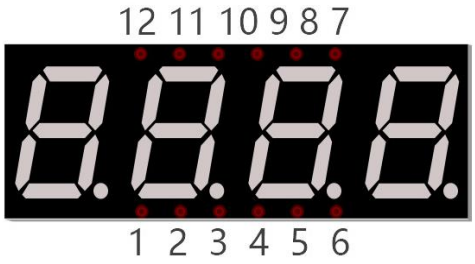
Component List

<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="4">Breadboard x1
  </td></tr>
<tr><td>74HC595 x1
  </td><td>7-segment display x1
  </td><td>Resistor 220Ω x8
  </td><td>Jumper
</td></tr>
</table>

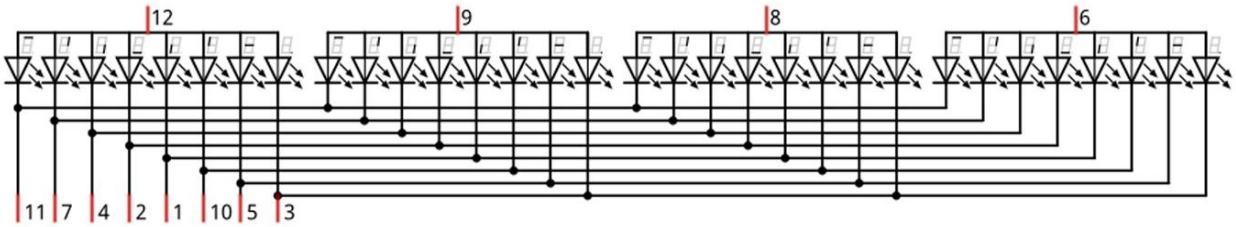
Component knowledge

4 Digit 7-Segment Display

A 4-Digit 7-segment display integrates four 7-Segment Displays into one module. Therefore, it can display more characters. All the LEDs contained have a Common Anode and individual Cathodes. Its internal structure and pin designation diagram is shown below:



The internal electronic circuit is shown below, and all eight LED cathode pins of each 7-Segment Display are connected together.

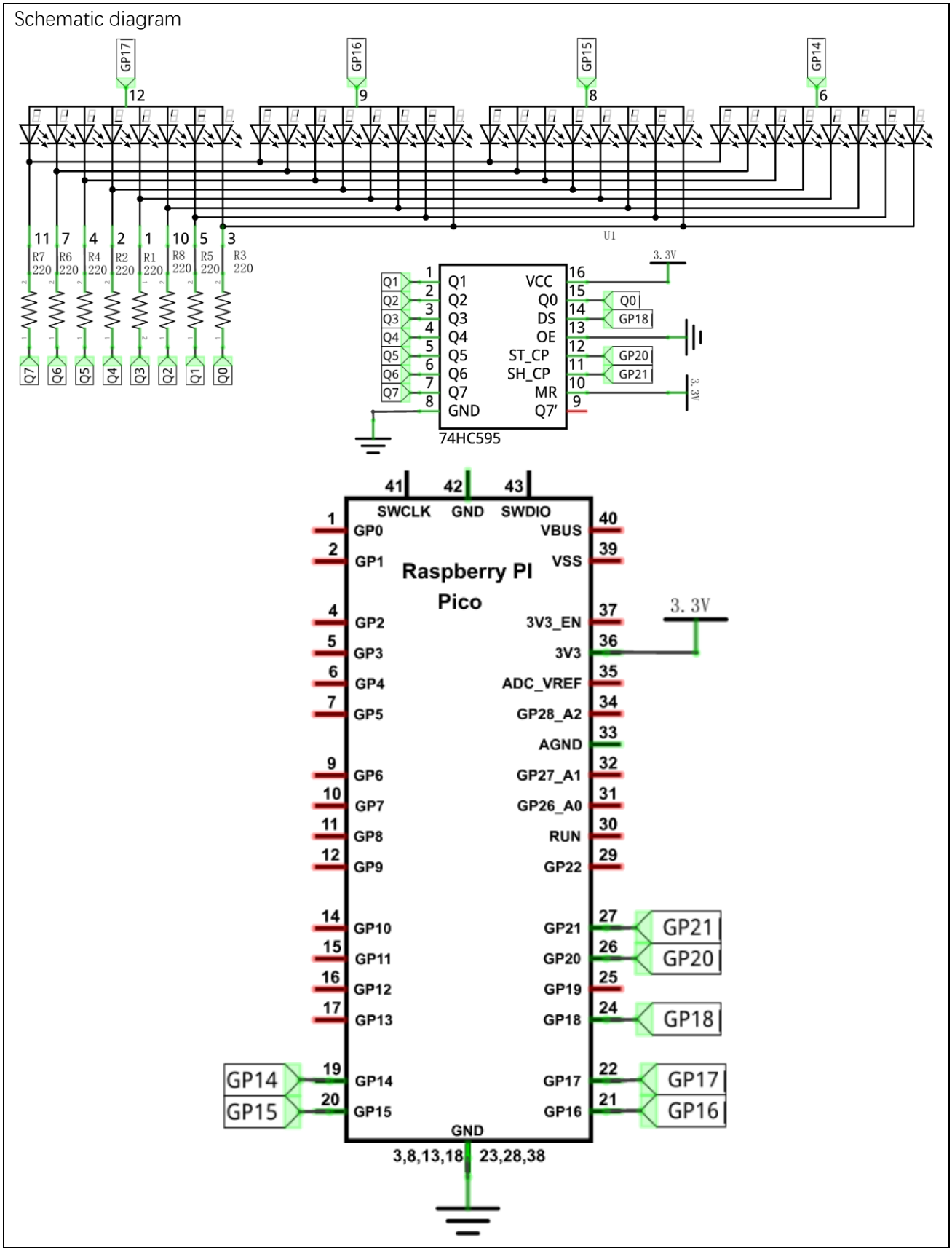


Display method of 4-Digit 7-segment display is similar to 1-Digit 7-segment display. The difference between them is that the 4-Digit displays each Digit visibly in turn, one by one and not together. We need to first send high level to the common end of the first Digit Display, and send low level to the remaining three common ends, and then send content to eight LED cathode pins of the first Digit Display. At this time, the first 7-Segment Display will show visible content and the remaining three will be OFF.

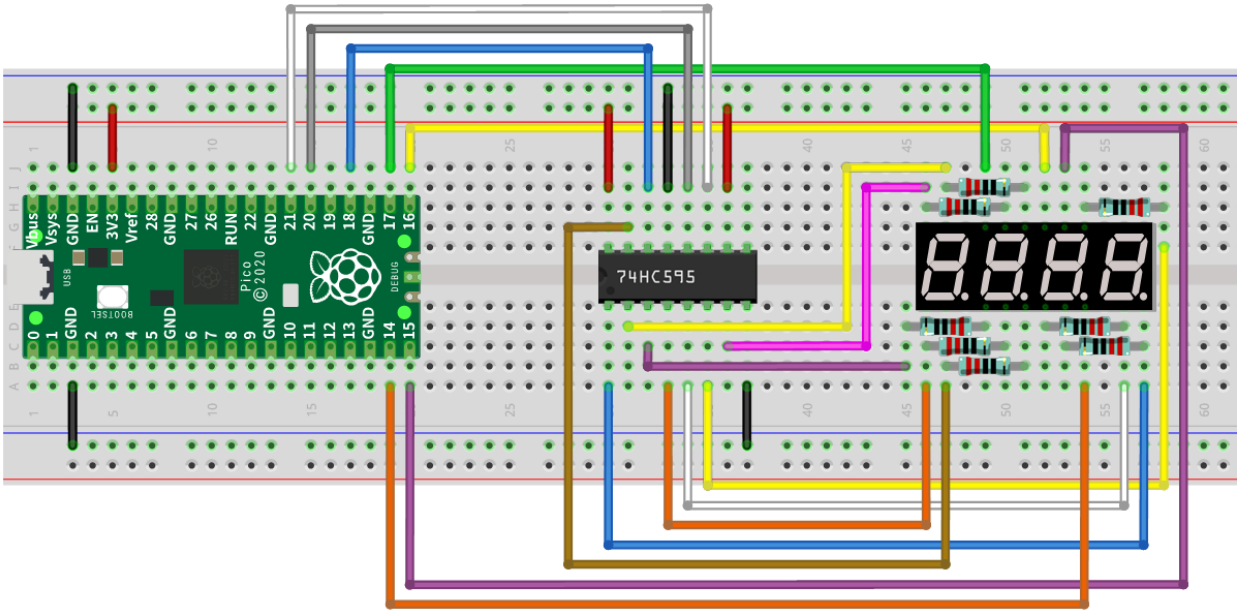
Similarly, the second, third and fourth 7-Segment Displays will show visible content in turn by scanning the display. Although the four number characters are displayed in turn separately, this process is so fast that it is imperceptible to the naked eye. This is due to the principle of optical afterglow effect and the vision persistence effect in human sight. This is how we can see all four number characters at the same time. However, if each number character is displayed for a longer period, you will be able to see that the number characters are displayed separately.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

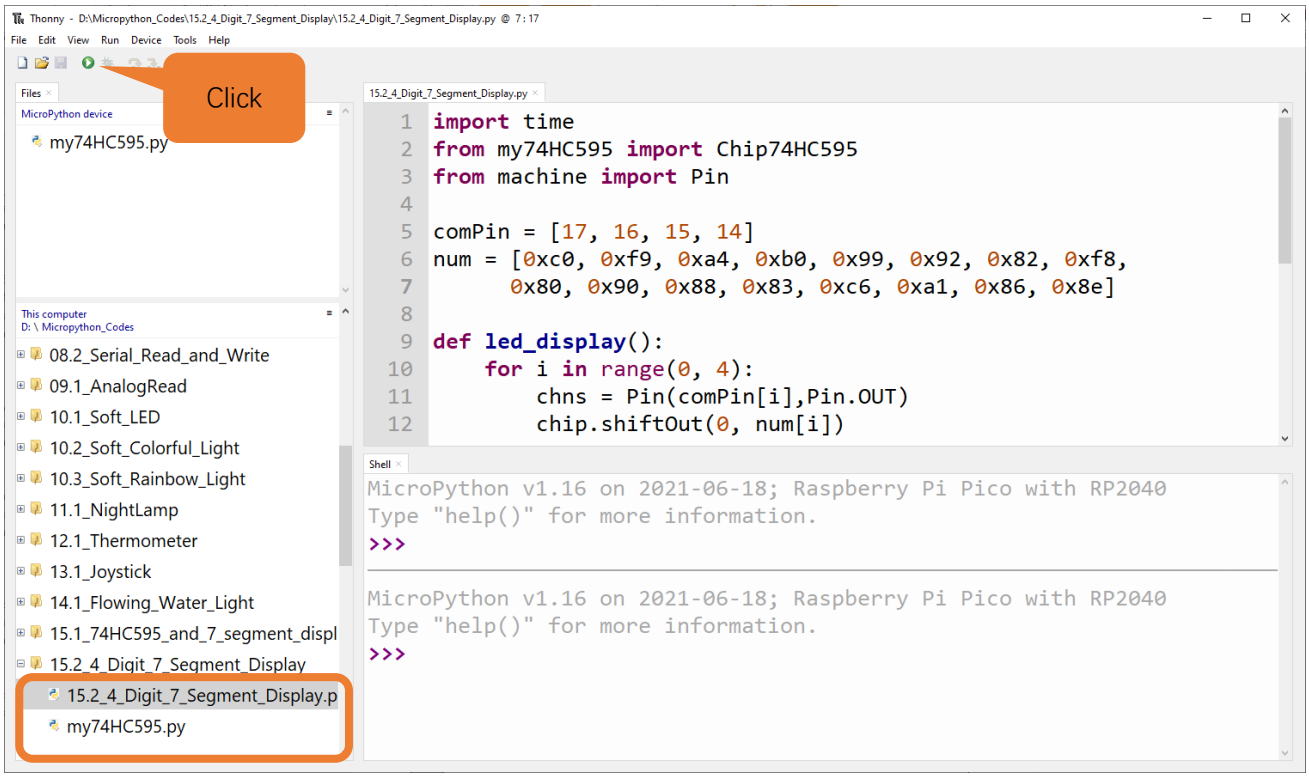


Code

In this code, we use the 74HC595 IC Chip to control the 4-Digit 7-Segment Display, and use the dynamic scanning method to show the changing number characters.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "15.2_4_Digit_7_Segment_Display". Select "my74HC595.py", right click your mouse to select "Upload to /", wait for "my74HC595.py" to be uploaded to Raspberry Pi Pico and double click "15.2_4_Digit_7_Segment_Display.py".

15.2_4_Digit_7_Segment_Display



Click "Run current script", and the Nixie tube display as shown in the image below. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



The following is the program code:

```
1  import time
2  from my74HC595 import Chip74HC595
3  from machine import Pin
4
5  comPin = [17, 16, 15, 14]
```

```
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8,
       0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]

def led_display():
    for i in range(0, 4):
        chns = Pin(comPin[i],Pin.OUT)
        chip.shiftOut(0, num[i])
        chns.value(1)
        time.sleep_ms(1)
        chns.value(0)

# Pico-GP 18: 74HC595-DS(18)
# Pico-GP 20: 74HC595-STCP(20)
# Pico-GP 21: 74HC595-SHCP(21)
chip = Chip74HC595(18, 20, 21)

try:
    while True:
        led_display()
except:
    pass
```

Import time, my74HC595 and Pin modules.

```
import time
from my74HC595 import Chip74HC595
from machine import Pin
```

Define common anode pins for digital tubes and request a list to put character encodings in it.

```
comPin = [17, 16, 15, 14]
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8,
       0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
```

Request an object to drive 74HC595 and associate pins with it.

```
# Pico-GP 18: 74HC595-DS(18)
# Pico-GP 20: 74HC595-STCP(20)
# Pico-GP 21: 74HC595-SHCP(21)
chip = Chip74HC595(18, 20, 21)
```

Make the digital tube display “0 1 2 3”.

```
def led_display():
    for i in range(0, 4):
        chns = Pin(comPin[i],Pin.OUT)
        chip.shiftOut(0, num[i])
        chns.value(1)
        time.sleep_ms(1)
        chns.value(0)
```

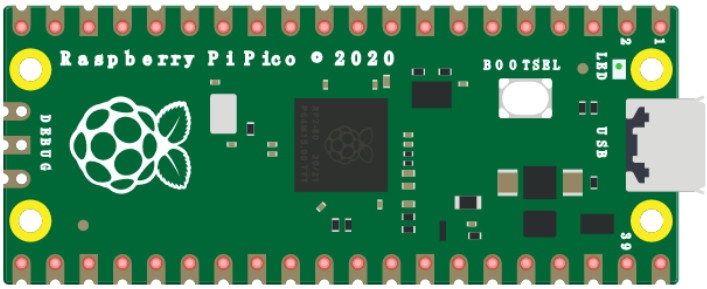
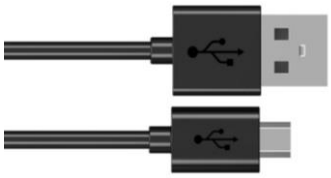
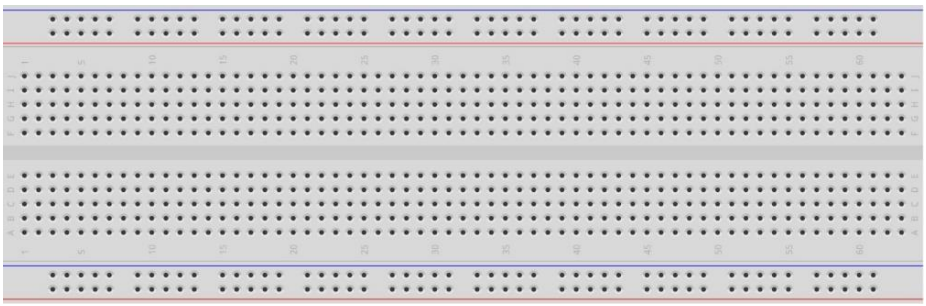
Chapter 16 74HC595 & LED Matrix

Thus far, we have learned how to use the 74HC595 IC Chip to control the Bar Graph LED and the 7-Segment Display. We will now use 74HC595 IC Chips to control an LED Matrix.

Project 16.1 LED Matrix

In this project, we will use two 74HC595 IC chips to control a monochrome (one color) (8X8) LED Matrix to make it display both simple graphics and characters.

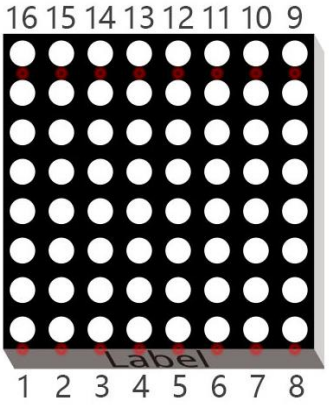
Component List

<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="4">Breadboard x1
  </td></tr>
<tr><td>74HC595 x2
  </td><td>8*8 LED Matrix x1
  </td><td>Resistor 220Ω x8
  </td><td>Jumper
</td></tr>
</table>

Component knowledge

LED matrix

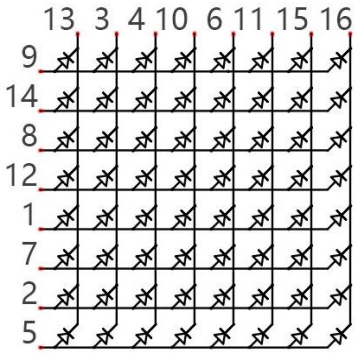
An LED Matrix is a rectangular display module that consists of a uniform grid of LEDs. The following is an 8X8 monochrome (one color) LED Matrix containing 64 LEDs (8 rows by 8 columns).



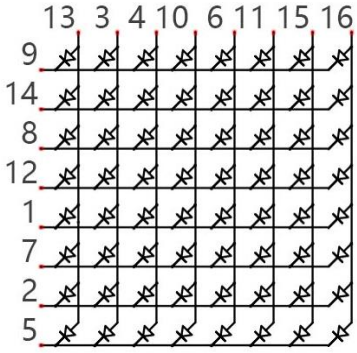
In order to facilitate the operation and reduce the number of ports required to drive this component, the Positive Poles of the LEDs in each row and Negative Poles of the LEDs in each column are respectively connected together inside the LED Matrix module, which is called a Common Anode. There is another arrangement type. Negative Poles of the LEDs in each row and the Positive Poles of the LEDs in each column are respectively connected together, which is called a Common Cathode.

The LED Matrix that we use in this project is a Common Anode LED Matrix.

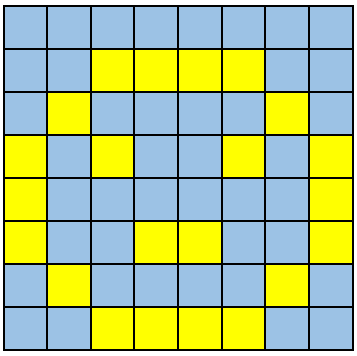
Connection mode of common anode



Connection mode of common cathode



Here is how a Common Anode LED Matrix works. First, choose 16 ports on Raspberry Pi Pico to connect to the 16 ports of LED Matrix. Configure one port in columns for low level, which makes that column the selected port. Then configure the eight port in the row to display content in the selected column. Add a delay value and then select the next column that outputs the corresponding content. This kind of operation by column is called Scan. If you want to display the following image of a smiling face, you can display it in 8 columns, and each column is represented by one byte.



1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0
0	0	1	1	1	1	0	0
0	1	0	0	0	0	1	0
1	0	1	0	0	1	0	1
1	0	0	0	0	0	0	1
1	0	0	1	1	0	0	1
0	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0

Column	Binary	Hexadecimal
1	0001 1100	0x1c
2	0010 0010	0x22
3	0101 0001	0x51
4	0100 0101	0x45
5	0100 0101	0x45
6	0101 0001	0x51
7	0010 0010	0x22
8	0001 1100	0x1c

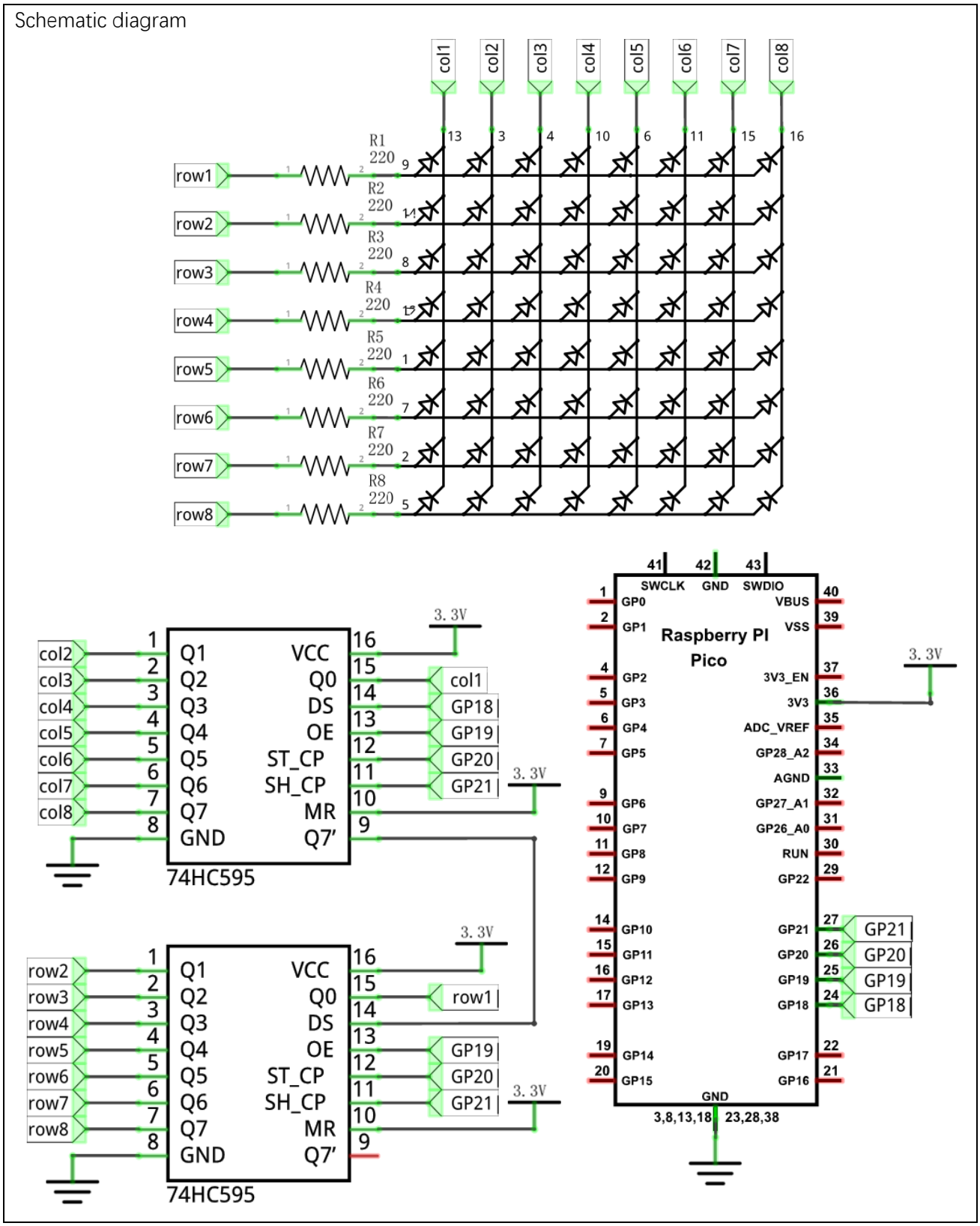
To begin, display the first column, then turn off the first column and display the second column. (and so on) .... turn off the seventh column and display the eighth column, and then start the process over from the first column again like the control of LED Bar Graph project. The whole process will be repeated rapidly in a loop. Due to the principle of optical afterglow effect and the vision persistence effect in human sight, we will see a picture of a smiling face directly rather than individual columns of LEDs turned ON one column at a time (although in fact this is the reality we cannot perceive).

Then, to save the number of GPIO, we use a 74HC595. When the first column is turned ON, set the lights that need to be displayed in the first column to "1", otherwise to "0", as shown in the above example, where the value of the first column is 0x1c. This value is sent to 74HC595 to control the display of the first column of the LEDMatrix. Following the above idea, turn OFF the display of the first column, then turn ON the second column, and then send the value of the second column to 74HC595... Until each column is displayed, the LED Matrix is displayed again from the first column.

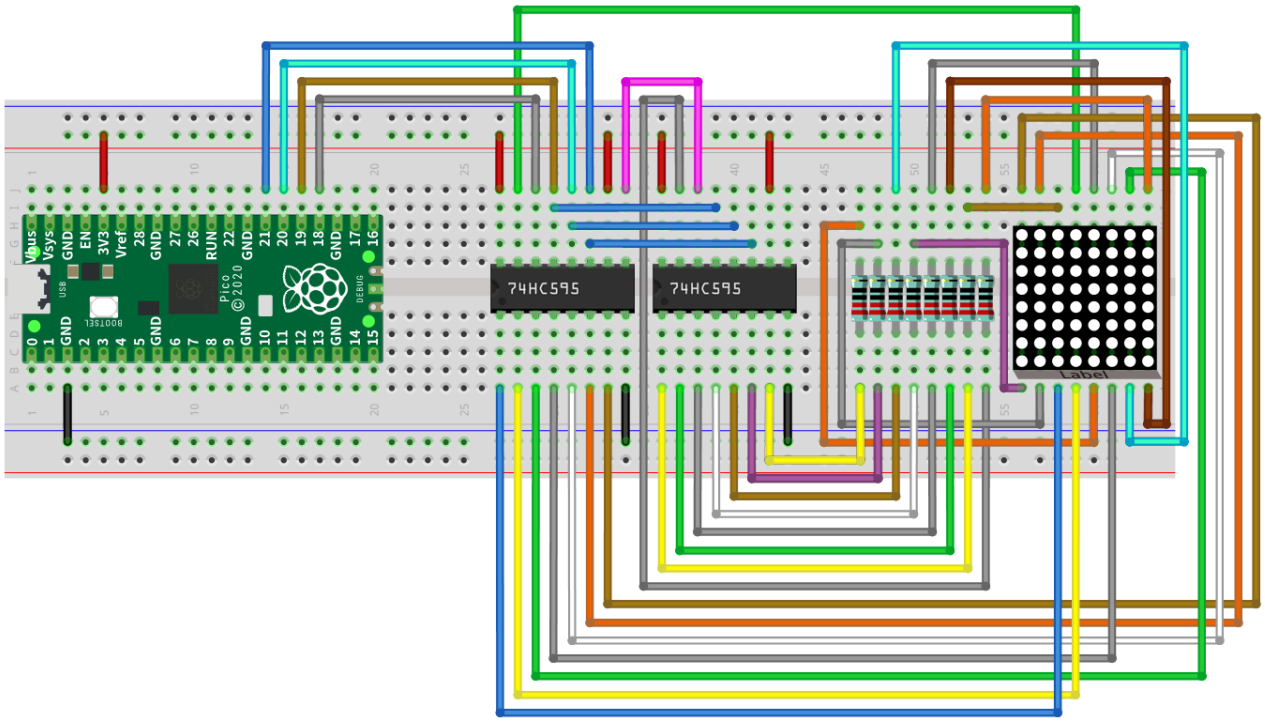
Circuit

In circuit of this project, the power pin of the 74HC595 IC Chip is connected to 3.3V. It can also be connected to 5V to make LED Matrix brighter.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via:support@freenove.com

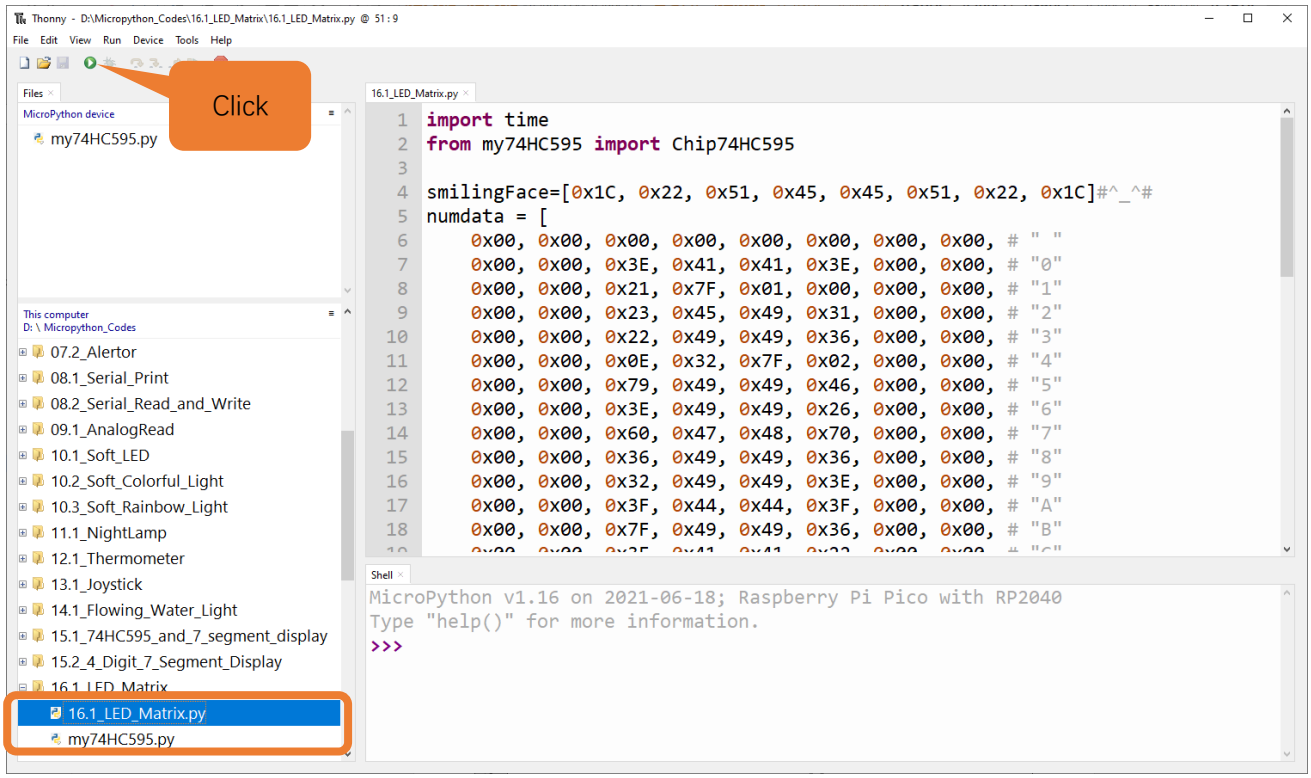


Code

The following code will make LED Matrix display a smiling face, and then display scrolling character "0-F".

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "Micropython_Codes". Select "my74HC595.py", right click your mouse to select "Upload to /", wait for "my74HC595.py" to be uploaded to Raspberry Pi Pico and double click "16.1_LED_Matrix.py".

16.1_LED_Matrix



Click "Run current script", and the LED Matrix display a smiling face, and then display characters "0 to F" scrolling in a loop on the LED Matrix.

The following is the program code:

```
import time
from my74HC595 import Chip74HC595

smilingFace=[0x1C, 0x22, 0x51, 0x45, 0x45, 0x51, 0x22, 0x1C]#^_^#
numdata = [
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, # " "
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x3E, 0x00, 0x00, # "0"
    0x00, 0x00, 0x21, 0x7F, 0x01, 0x00, 0x00, 0x00, # "1"
    0x00, 0x00, 0x23, 0x45, 0x49, 0x31, 0x00, 0x00, # "2"
    0x00, 0x00, 0x22, 0x49, 0x49, 0x36, 0x00, 0x00, # "3"
    0x00, 0x00, 0x0E, 0x32, 0x7F, 0x02, 0x00, 0x00, # "4"
    0x00, 0x00, 0x79, 0x49, 0x49, 0x46, 0x00, 0x00, # "5"
    0x00, 0x00, 0x3E, 0x49, 0x49, 0x26, 0x00, 0x00, # "6"
```

```
    0x00, 0x00, 0x60, 0x47, 0x48, 0x70, 0x00, 0x00, # "7"
    0x00, 0x00, 0x36, 0x49, 0x49, 0x36, 0x00, 0x00, # "8"
    0x00, 0x00, 0x32, 0x49, 0x49, 0x3E, 0x00, 0x00, # "9"
    0x00, 0x00, 0x3F, 0x44, 0x44, 0x3F, 0x00, 0x00, # "A"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x36, 0x00, 0x00, # "B"
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x22, 0x00, 0x00, # "C"
    0x00, 0x00, 0x7F, 0x41, 0x41, 0x3E, 0x00, 0x00, # "D"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x41, 0x00, 0x00, # "E"
    0x00, 0x00, 0x7F, 0x48, 0x48, 0x40, 0x00, 0x00, # "F"
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00  # " "
]

chip = Chip74HC595(18, 20, 21, 19)
try:
    while True:
        #smilingFace
        for j in range(100):
            cols = 0x01
            for i in range(8):
                chip.disable()
                chip.shiftOut(1, smilingFace[i])
                chip.shiftOut(0, ~cols)
                cols <<= 1
                chip.enable()
                time.sleep_us(500)
        #numdata
        for i in range(136):
            for _ in range(5):
                cols = 0x01
                for j in range(i, 8+i):
                    chip.disable()
                    chip.shiftOut(1, numdata[j])
                    chip.shiftOut(0, ~cols)
                    cols <<= 1
                    chip.enable()
                    time.sleep_us(500)
except:
    pass
```

Import time and my74HC595 modules.

```
1  import time
2  from my74HC595 import Chip74HC595
```

Use a nesting of two for loops to display a smiling face.

```
30      for j in range(100):
31          cols = 0x01
32          for i in range(8):
33              chip.disable()
34              chip.shiftOut(1, smilingFace[i])
35              chip.shiftOut(0, ~cols)
36              cols <<= 1
37              chip.enable()
38               time.sleep_us(500)
```

Use a nesting of two for loops to display “0”- “F”.

```
40      for i in range(136):
41          for _ in range(5):
42              cols = 0x01
43              for j in range(i, 8+i):
44                  chip.disable()
45                  chip.shiftOut(1, numdata[j])
46                  chip.shiftOut(0, ~cols)
47                  cols <<= 1
48                  chip.enable()
49                  time.sleep_us(500)
```

The amount of pins of Raspberry Pi Pico is limited, so we need to find ways to save pins. If we use Raspberry Pi Pico GPIO to control the LEDMatrix instead of 74HC595, we need 16 pins to drive LED matrix. In this example, we use two 74HC595 chips to drive the LED matrix, requiring only three pins, so that we could save the rest of 13 pins.

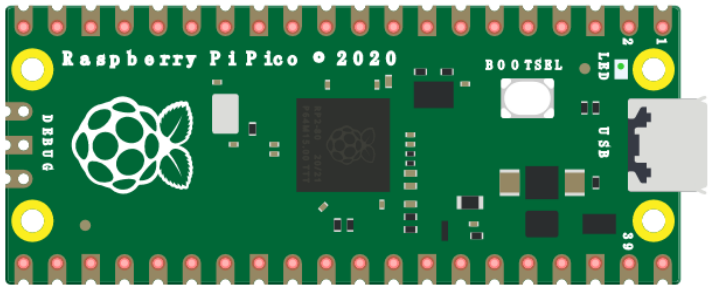
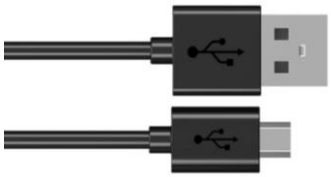
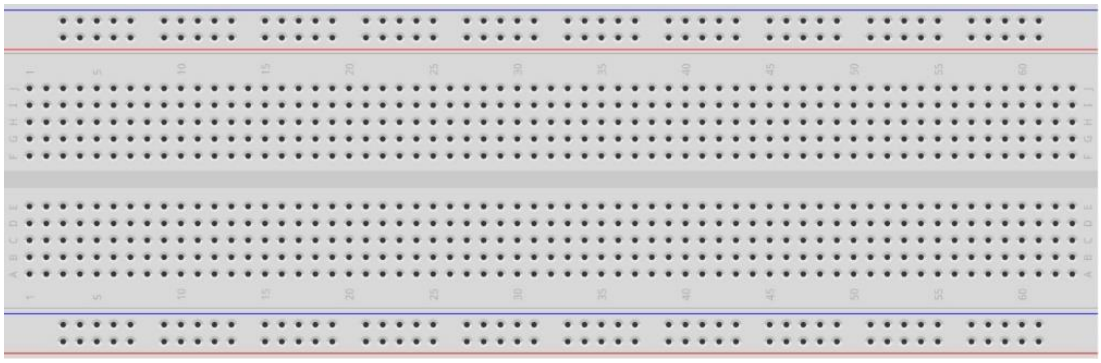
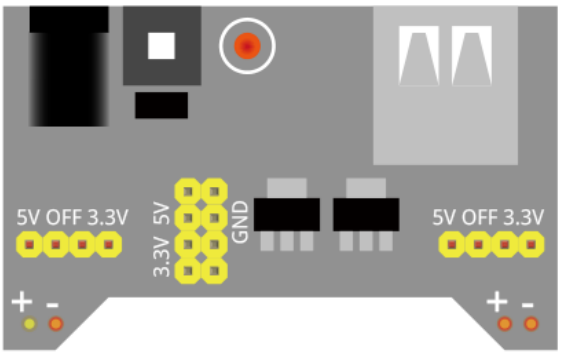
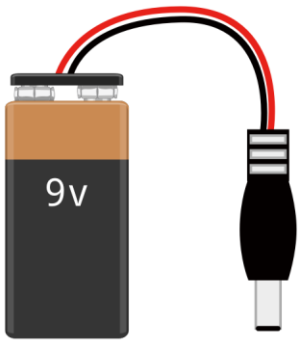
Chapter 17 Relay & Motor

In this chapter, we will learn a kind of special switch module, Relay Module.

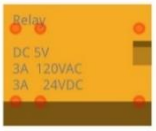
Project 17.1 Relay & Motor

In this project, we will use a Push Button Switch indirectly to control the motor via a Relay.

Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Breadboard Power x1 	9V battery (prepared by yourself) & battery line x1 

Resistor 10kΩ x2 	Resistor 1kΩ x1 	Resistor 220Ω x1 

NPN transistor x1 	Relay x1 	Motor x1 	Push button x1 	LED x1 	Diode x1 

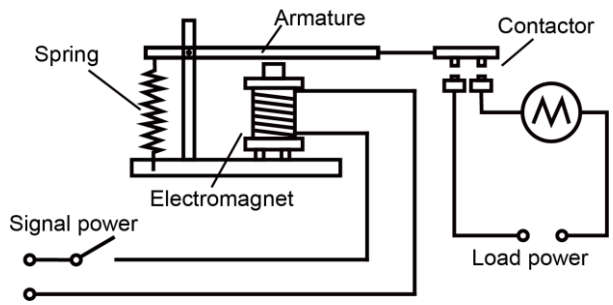
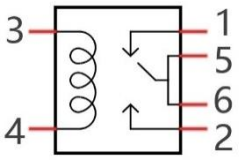
Jumper 

Component knowledge

Relay

A relay is a safe switch, which can use low power circuit to control high power circuit. It consists of electromagnet and contacts. The electromagnet is controlled by low power circuit and contacts are used in high power circuit. When the electromagnet is energized, it will attract contacts.

The following is a schematic diagram of a common relay and the feature and circuit symbol of a 5V relay used in this project:

Diagram	Feature:	Symbol
		

Pin 5 and pin 6 are connected to each other inside. When the coil pins 3 and 4 get connected to 5V power supply, pin 1 will be disconnected to pin 5&6 and pin 2 will be connected to pin 5&6. So pin 1 is called close end, pin 2 is called open end.

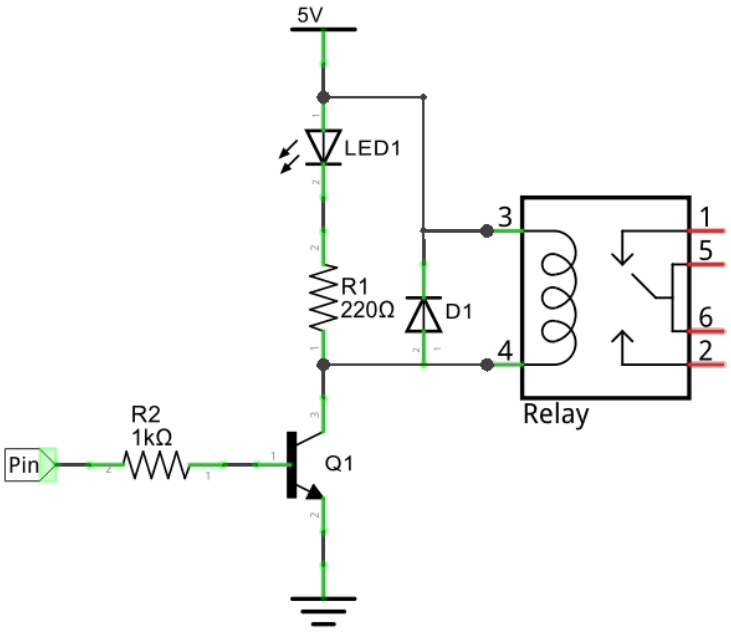
Inductor

The symbol of Inductance is "L" and the unit of inductance is the "Henry" (H). Here is an example of how this can be encountered: 1H=1000mH, 1mH=1000μH.

An inductor is an energy storage device that converts electrical energy into magnetic energy. Generally, it consists of winding coil, with a certain amount of inductance. Inductors hinder the change of current passing through it. When the current passing through it increases, it will attempt to hinder the increasing trend of current; and when the current passing through it decreases, it will attempt to hinder the decreasing trend of current. Therefore, the current passing through inductor is not transient.

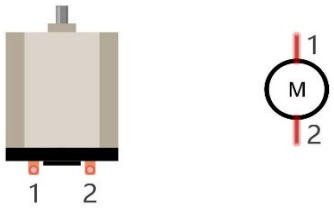


The reference circuit for relay is as follows. The coil of relays can be equivalent to that of inductors, when the transistor disconnects power supply of the relay, the current in the coil of the relay cannot stop immediately, causing an impact on power supply. Therefore, a parallel diode will be connected to both ends of relay coil pin in reversing direction, and then the current will pass through diode, avoiding the impact on power supply.

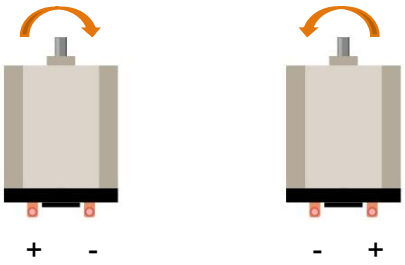


Motor

A motor is a device that converts electrical energy into mechanical energy. Motor consists of two parts: stator and rotor. When motor works, the stationary part is stator, and the rotating part is rotor. Stator is usually the outer case of motor, and it has terminals to connect to the power. Rotor is usually the shaft of motor, and can drive other mechanical devices to run. Diagram below is a small DC motor with two pins.

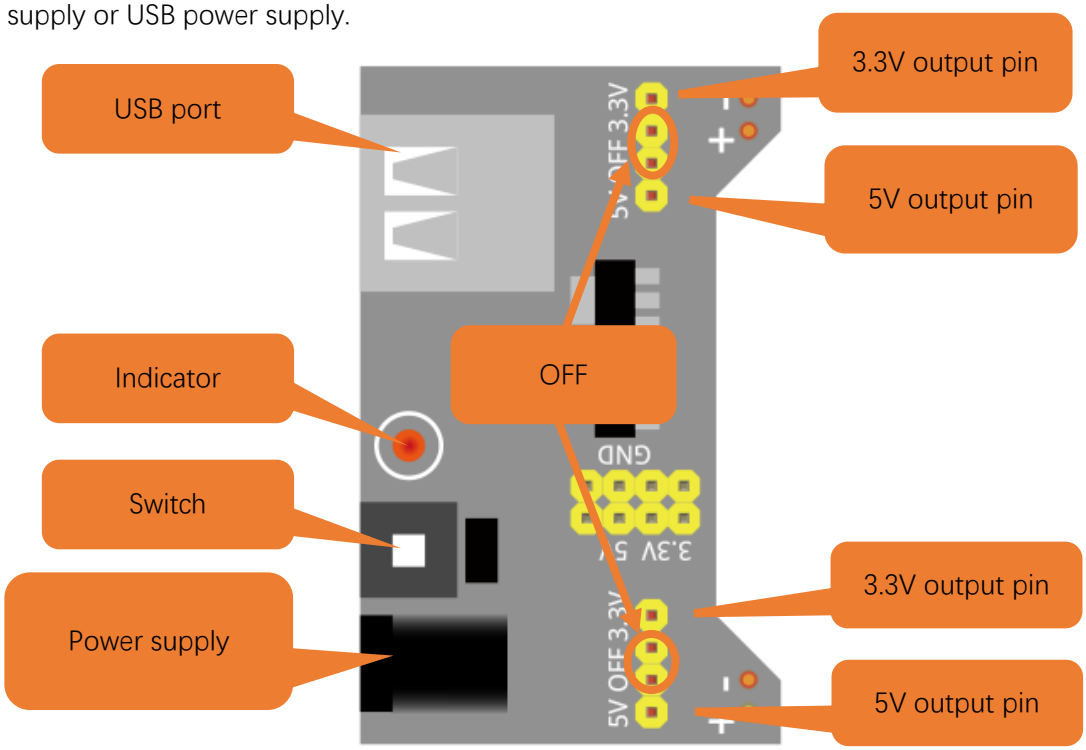


When a motor is connected to the power supply, it will rotate in one direction. Reverse the polarity of power supply, then the motor rotates in opposite direction.



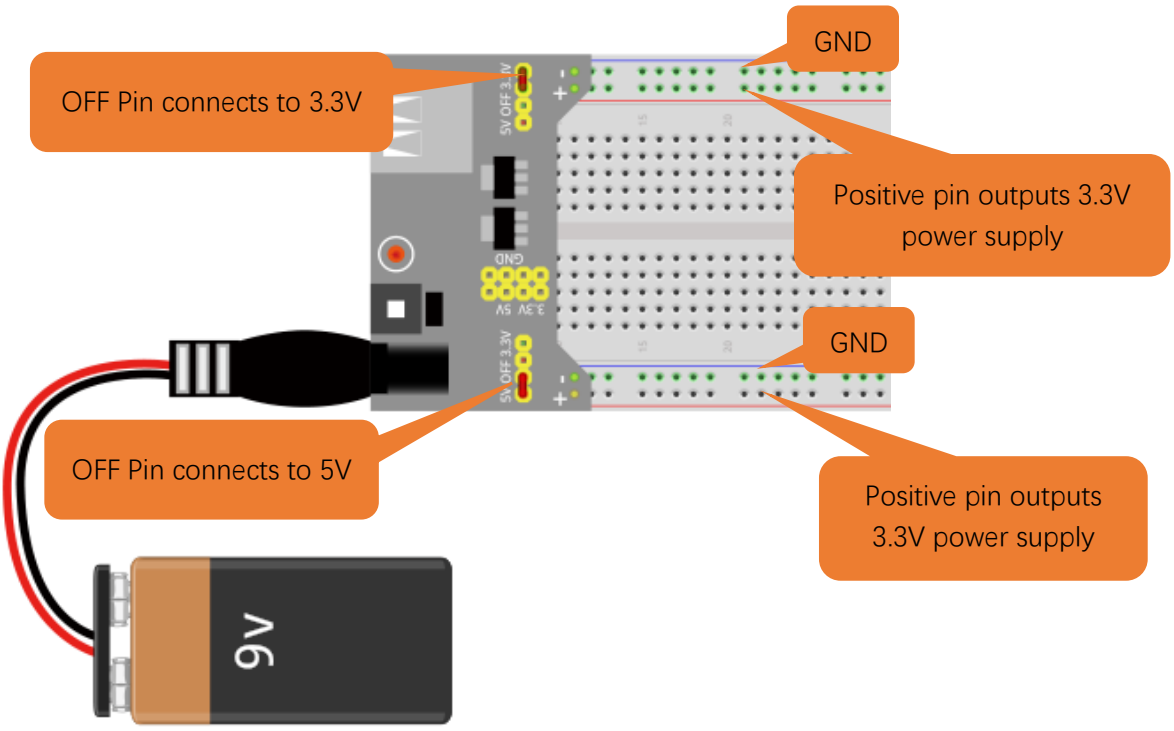
Breadboard Power

When the Raspberry Pi Pico outputs insufficient power or the power supply voltage and power consumption required by the device or module exceeds that provided by the Raspberry Pi Pico, you can choose Breadboard Power to either output 3.3V voltage source or 5V voltage source. Input power: DC 6~10V power supply or USB power supply.



Usage:

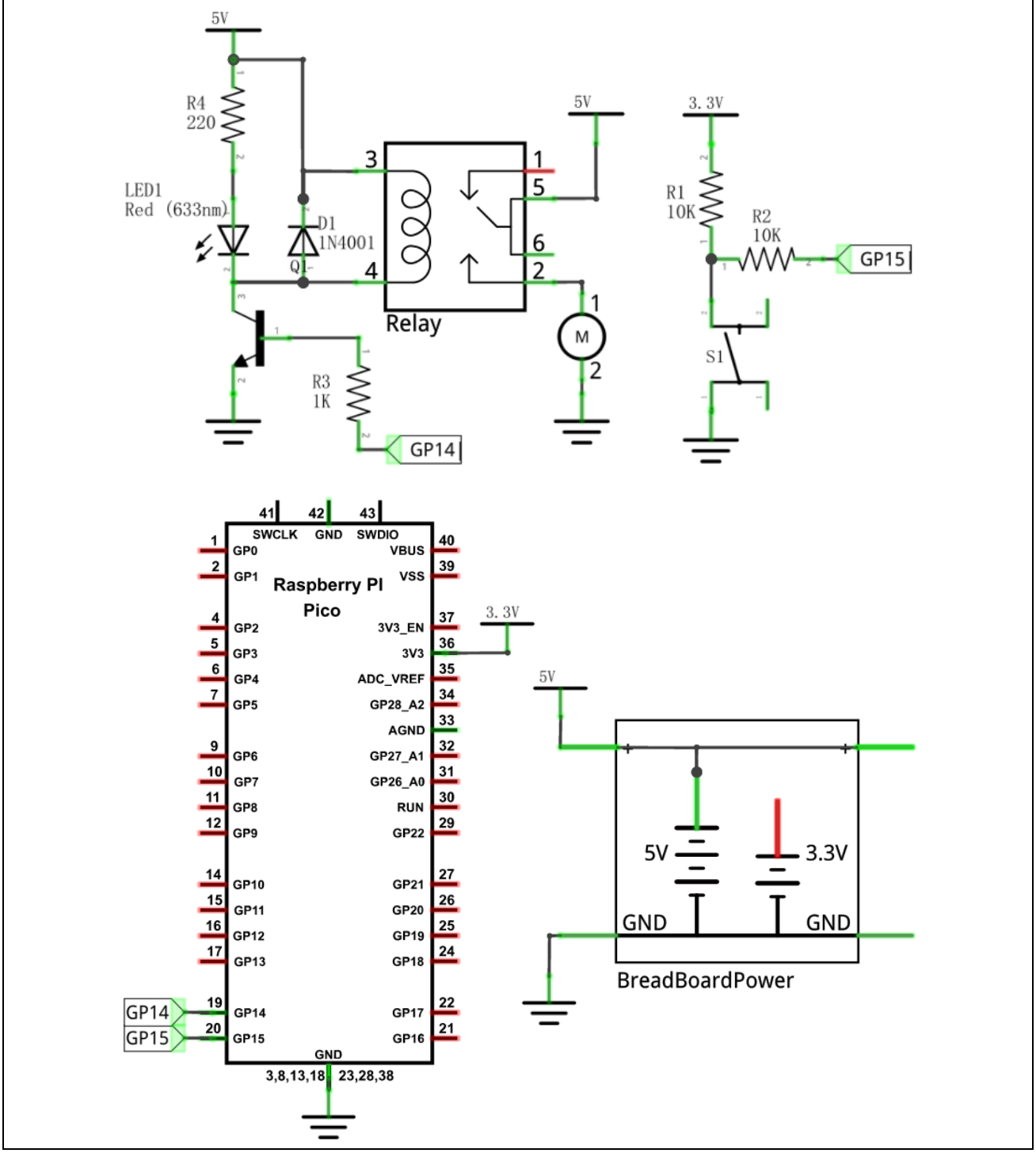
First connect Breadboard Power to the power supply, and then selectively choose the connection between OFF and 5V, 3V to generate different voltage source outputs. As the picture shows.



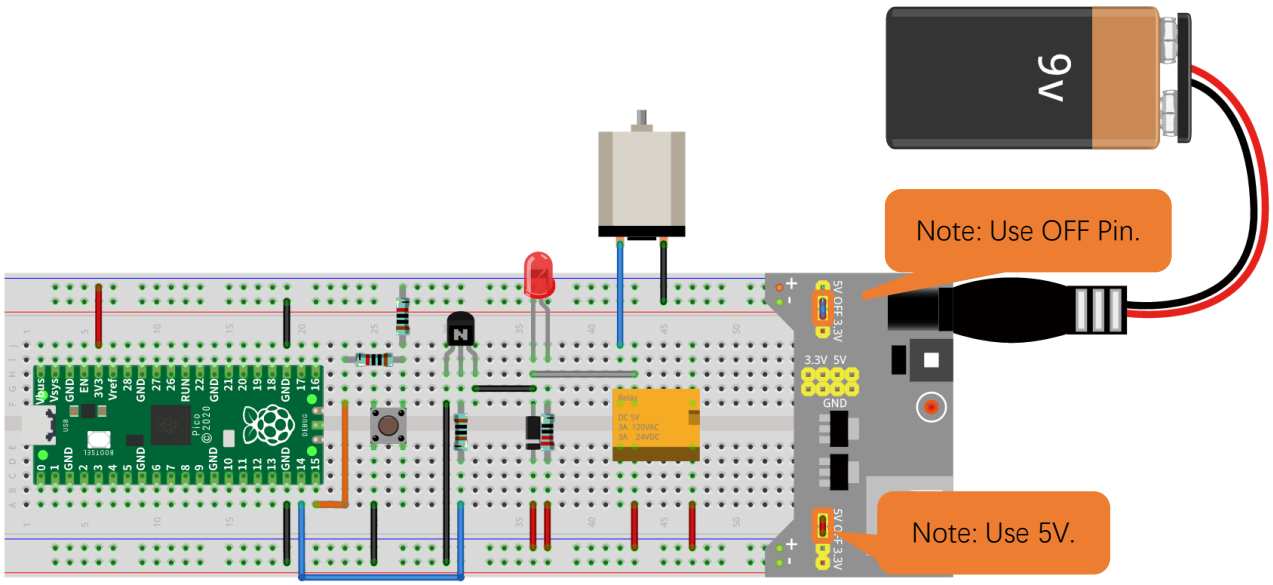
You can either select only one side of the output power supply, or choose to output 5V or 3.3V power supply at the same time on both sides, because the output circuits on both sides are separated and controlled independently and will not cause interference. Note that the GND should be connected to any GND of the Raspberry Pi Pico to form a Common ground connection.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

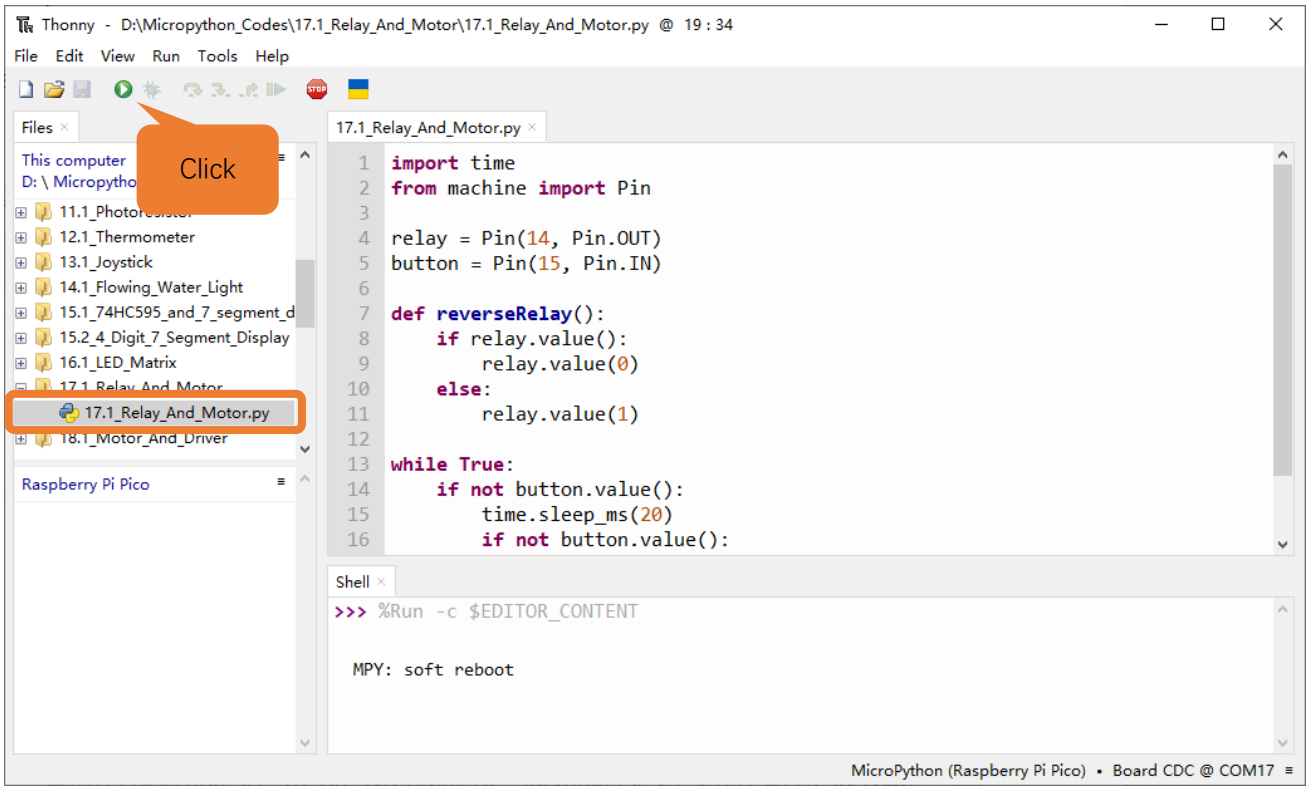


Code

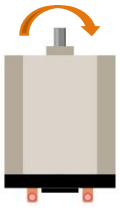
Use buttons to control the relays and motors.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "17.1_Relay_And_Motor" and double click "17.1_Relay_And_Motor.py".

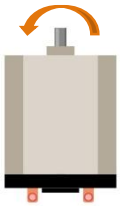
17.1_Relay_And_Motor



Click "Run current script". When the DC Motor is connected to a power supply, it will rotate in one direction. If you reverse the polarity of the power supply, the DC Motor will rotate in opposite direction. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



\+ -



\- +

The following is the program code:

```
import time
from machine import Pin

relay = Pin(14, Pin.OUT)
button = Pin(15, Pin.IN)

def reverseRelay():
    if relay.value():
        relay.value(0)
    else:
        relay.value(1)

while True:
    if not button.value():
        time.sleep_ms(20)
        if not button.value():
            reverseRelay()
            while not button.value():
                time.sleep_ms(20)
```

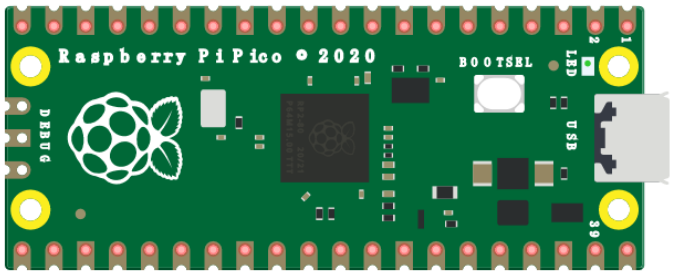
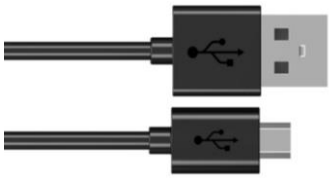
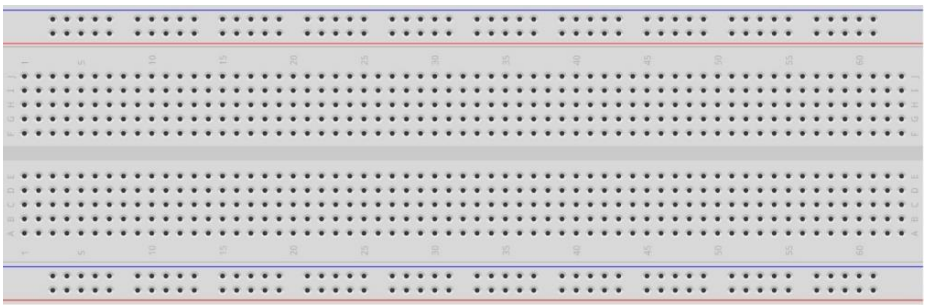
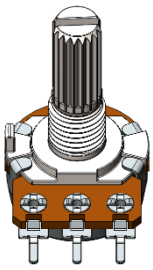
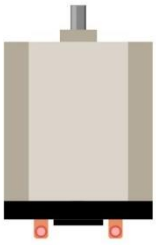
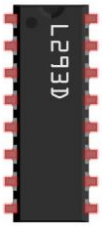
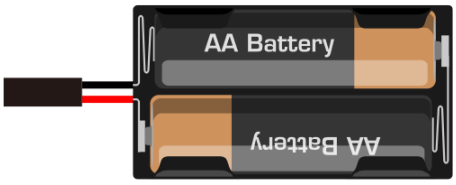
This section of code is the same as that of project Tablelamp. If you do not understand the program, you can click here to go back to the Tablelamp and study again.

Chapter 18 L293D & Motor

Project 18.1 Control Motor with Potentiometer

Control the direction and speed of the motor with a potentiometer.

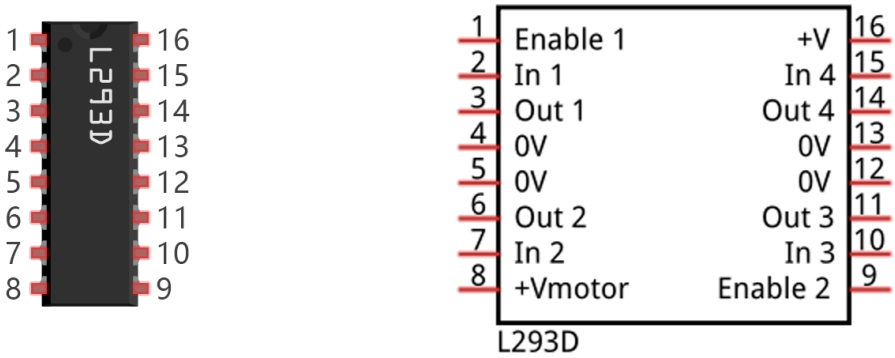
Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Rotary potentiometer x1 	Motor x1 
L293D x1 	
Jumper	Battery box x1 

Component knowledge

L293D

L293D is an IC Chip (Integrated Circuit Chip) with a 4-channel motor drive. You can drive a Unidirectional DC Motor with 4 ports or a Bi-Directional DC Motor with 2 ports or a Stepper Motor (Stepper Motors are covered later in this Tutorial).

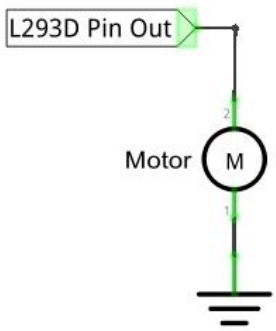


Port description of L293D module is as follows:

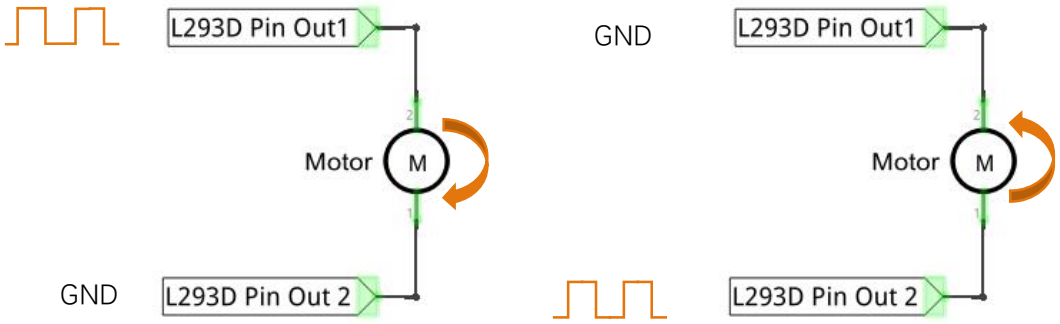
Pin name	Pin number	Description
In x	2, 7, 10, 15	Channel x digital signal input pin
Out x	3, 6, 11, 14	Channel x output pin, input high or low level according to In x pin, get connected to +Vmotor or 0V
Enable1	1	Channel 1 and channel 2 enable pin, high level enable
Enable2	9	Channel 3 and channel 4 enable pin, high level enable
0V	4, 5, 12, 13	Power cathode (GND)
+V	16	Positive electrode (VCC) of power supply, supply voltage 3.0~36V
+Vmotor	8	Positive electrode of load power supply, provide power supply for the Out pin x, the supply voltage is +V~36V

For more details, please refer to the datasheet for this IC Chip.

When using L293D to drive DC motor, there are usually two connection options.
The following connection option uses one channel of the L239D, which can control motor speed through the PWM, However the motor then can only rotate in one direction.



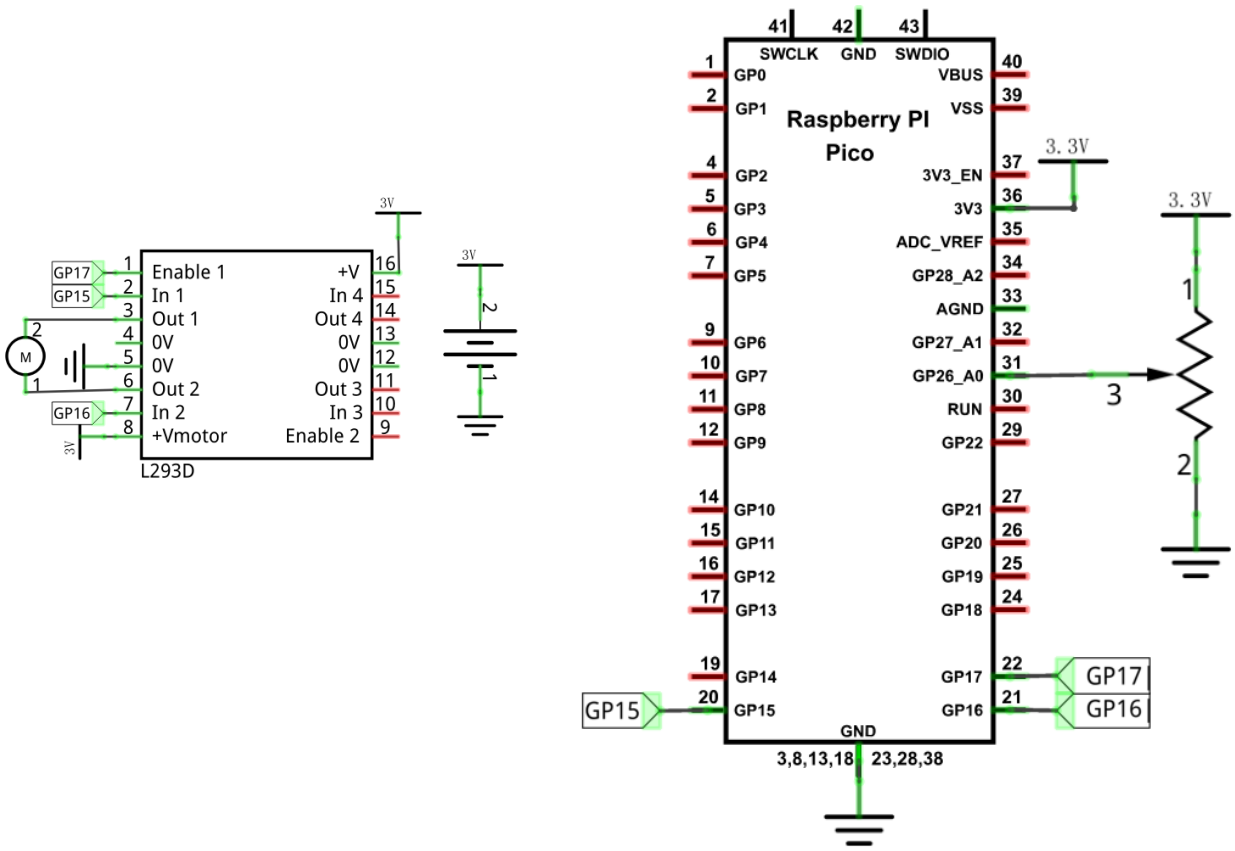
The following connection uses two channels of the L239D: one channel outputs the PWM wave, and the other channel connects to GND, therefore you can control the speed of the motor. When these two channel signals are exchanged, not only can they control the speed of motor, but also control the direction of the motor.



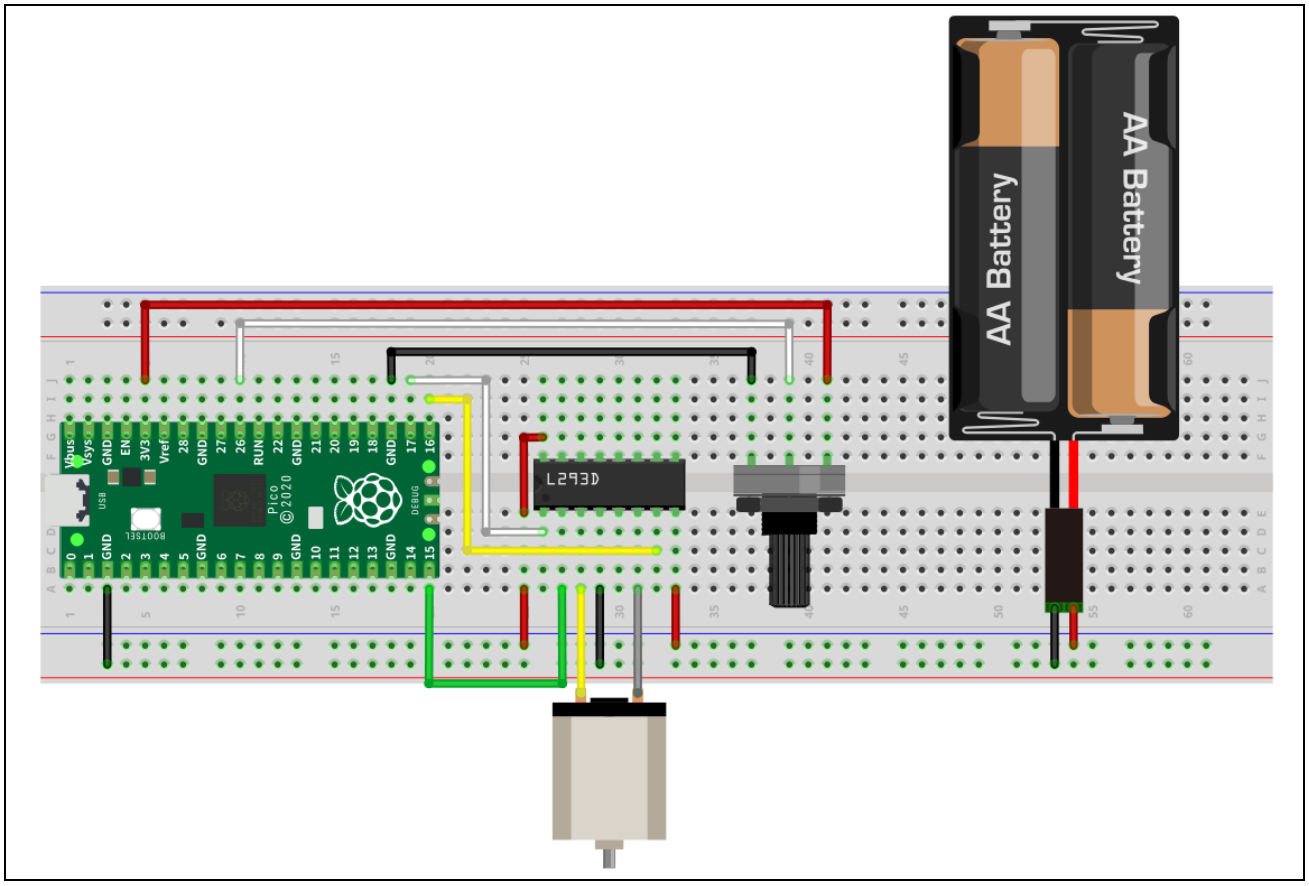
In practical use, the motor is usually connected to channels 1 and 2 by outputting different levels to in1 and in2 to control the rotational direction of the motor, and output to the PWM wave to Enable1 port to control the motor's rotational speed. If the motor is connected to channel 3 and 4 by outputting different levels to in3 and in4 to control the motor's rotation direction, and output to the PWM wave to Enable2 pin to control the motor's rotational speed.

Circuit

Schematic diagram



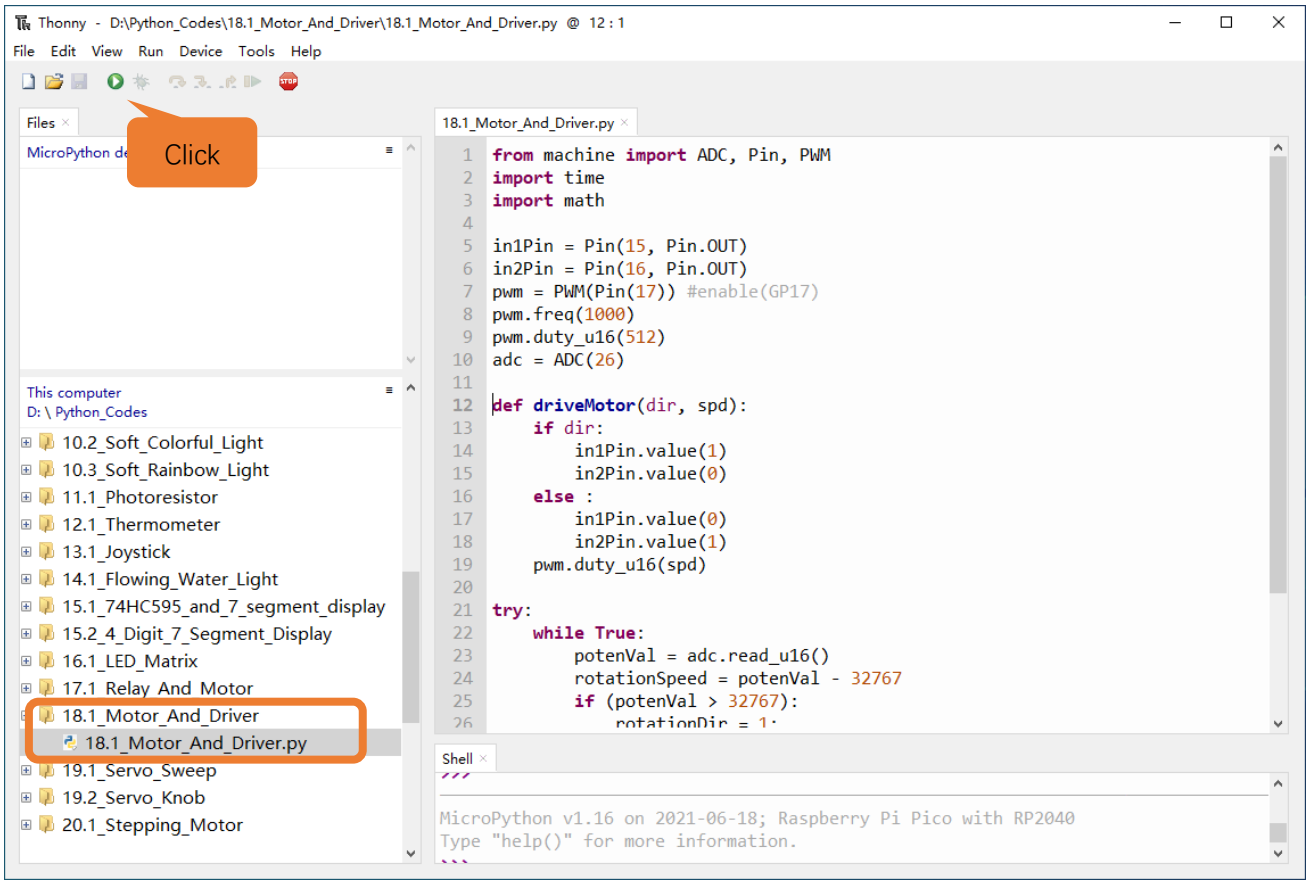
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com


AA Battery
AA Battery
L293D

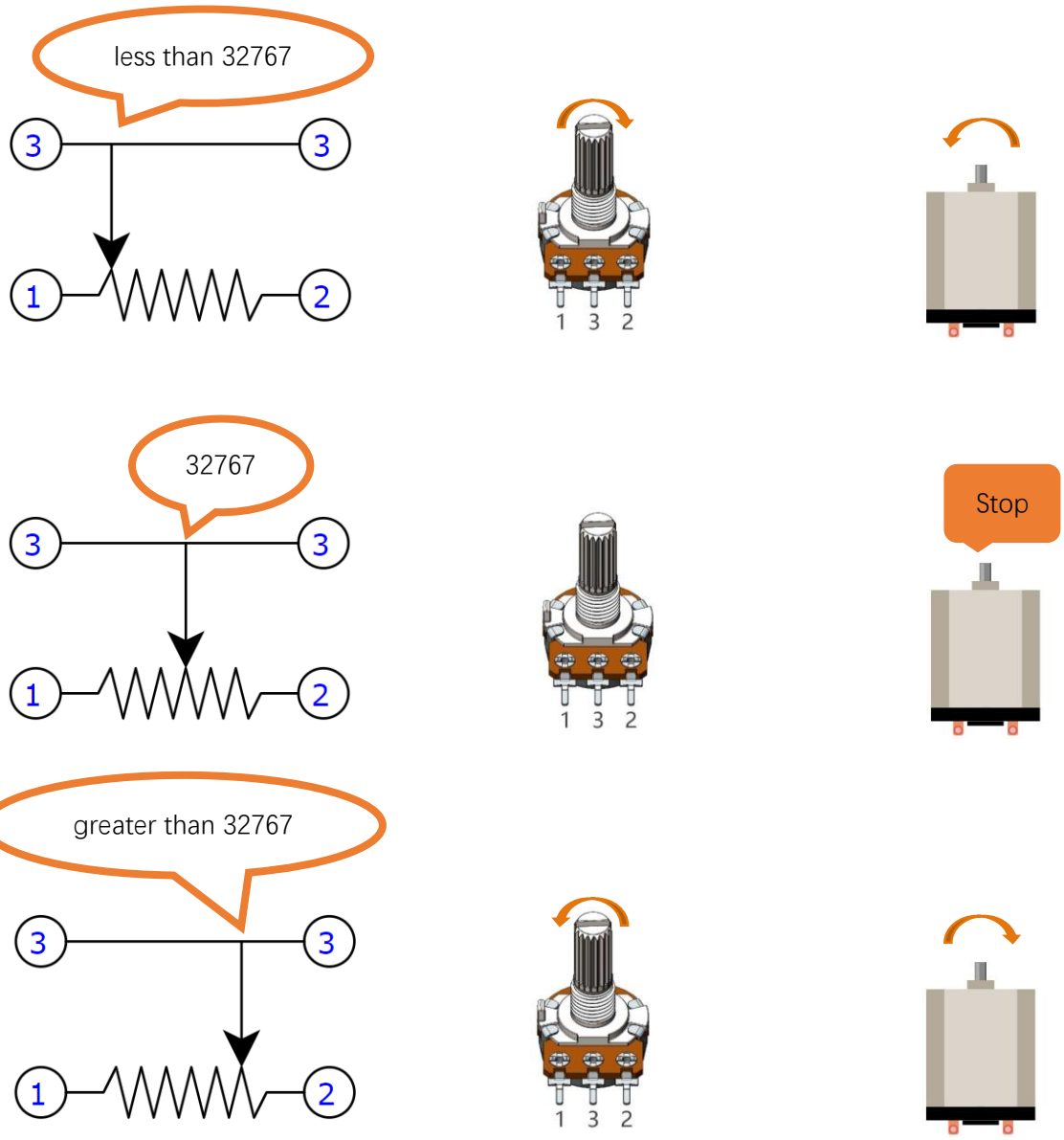
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "18.1_Motor_And_Driver" and double click "18.1_Motor_And_Driver.py".

18.1_Motor_And_Driver



Click “Run current script”, rotate the potentiometer in one direction and the motor speeds up slowly in one direction. Rotate the potentiometer in the other direction and the motor will slow down to stop. Then rotate it in the original direction to accelerate the motor. Press Ctrl+C or click “Stop/Restart backend” to exit the program.



The following is the Code:

```
from machine import ADC, Pin, PWM
import time
import math

in1Pin = Pin(15, Pin.OUT)
in2Pin = Pin(16, Pin.OUT)
pwm = PWM(Pin(17)) #enable(GP17)
pwm.freq(1000)
pwm.duty_u16(512)
adc = ADC(26)

def driveMotor(dir, spd):
    if dir:
        in1Pin.value(1)
        in2Pin.value(0)
    else :
        in1Pin.value(0)
        in2Pin.value(1)
    pwm.duty_u16(spd)

try:
    while True:
        potenVal = adc.read_u16()
        rotationSpeed = potenVal - 32767
        if (potenVal > 32767):
            rotationDir = 1;
        else:
            rotationDir = 0;
        rotationSpeed = int(math.fabs((potenVal-32767) * 2) - 1)
        driveMotor(rotationDir,rotationSpeed)
        time.sleep_ms(10)
except:
    pwm.deinit()
```

In the program, we will define 32767 as the intermediate point to adjust the potentiometer. When the ADC value is less than 32767, the motor rotates in one direction; when the ADC value is greater than 32767, the motor rotates in the opposite direction. The speed of the motor will change with the adjustment of the potentiometer. When the potentiometer is adjusted, the closer the ADC value is to the middle point, the slower the motor speed; the closer the ADC value is to 0 or 65535, the faster the motor speed.

```
    while True:
        potenVal = adc.read_u16()
        rotationSpeed = potenVal - 32767
        if (potenVal > 32767):
            rotationDir = 1;
```

```
27          else:
28              rotationDir = 0;
29          rotationSpeed = int(math.fabs((potenVal-32767) * 2) - 1)
30          driveMotor(rotationDir,rotationSpeed)
31          time.sleep_ms(10)
```

Initialize pins of L293D chip.

```
5   in1Pin = Pin(15, Pin.OUT)
6   in2Pin = Pin(16, Pin.OUT)
7   pwm = PWM(Pin(17)) #enable -> GP 17
8   pwm.freq(1000)
9   pwm.duty_u16(512)
```

Function driveMotor is used to control the rotation direction and speed of the motor. The **dir** represents direction while **spd** refers to speed.

```
12  def driveMotor(dir, spd):
13      if dir:
14          in1Pin.value(1)
15          in2Pin.value(0)
16      else :
17          in1Pin.value(0)
18          in2Pin.value(1)
19      pwm.duty_u16(spd)
```

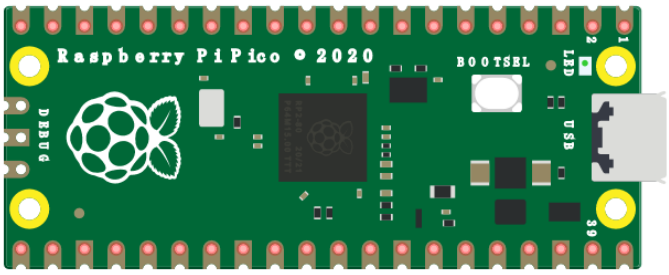
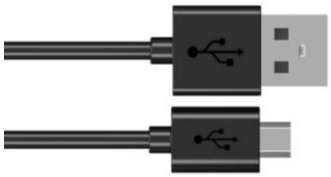
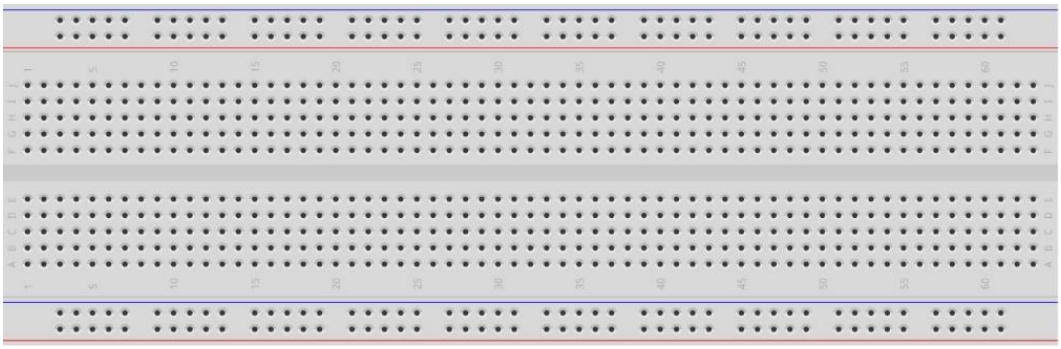
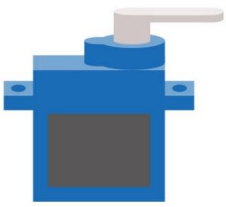
Chapter 19 Servo

Previously, we learned how to control the speed and rotational direction of a Motor. In this chapter, we will learn about servos, which are a rotary actuator type motor that can be controlled to rotate to specific angles.

Project 19.1 Servo Sweep

First, we need to learn how to make a Servo rotate.

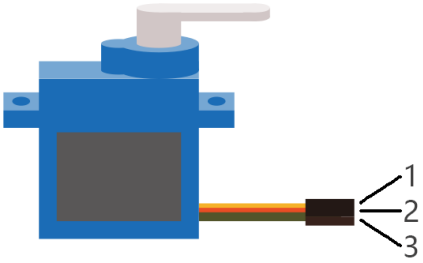
Component List

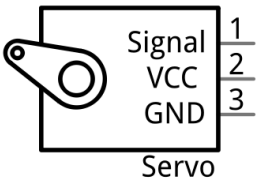
Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Servo x1 	Jumper

Component knowledge

Servo

Servo is a compact package, which consists of a DC Motor, a set of reduction gears to provide torque, a sensor and control circuit board. Most Servos only have a 180-degree range of motion via their "horn". Servos can output higher torque than a simple DC Motor alone and they are widely used to control motion in model cars, model airplanes, robots, etc. Servos have three wire leads, which usually terminate to a male or female 3-pin plug. Two leads are for electric power: Positive (2-VCC, Red wire), Negative (3-GND, Brown wire), and the signal line (1-Signal, Orange wire) as represented in the Servo provided in your Kit.





We will use a 50Hz PWM signal with a duty cycle in a certain range to drive the Servo. The time interval of 0.5ms-2.5ms of PWM single cycle high level corresponds to the Servo angle 0 degrees - 180 degrees linearly. Part of the corresponding values are as follows:

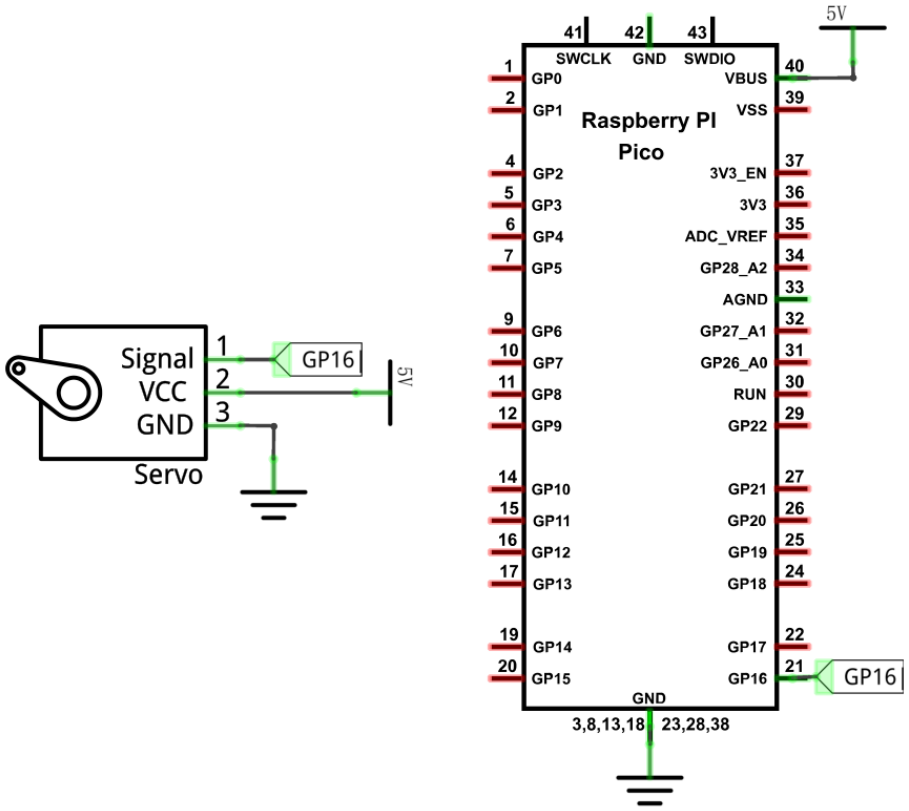
High level time	Servo angle
0.5ms	0 degree
1ms	45 degree
1.5ms	0 degree
2ms	45 degree
2.5ms	180 degree

When you change the Servo signal value, the Servo will rotate to the designated angle.

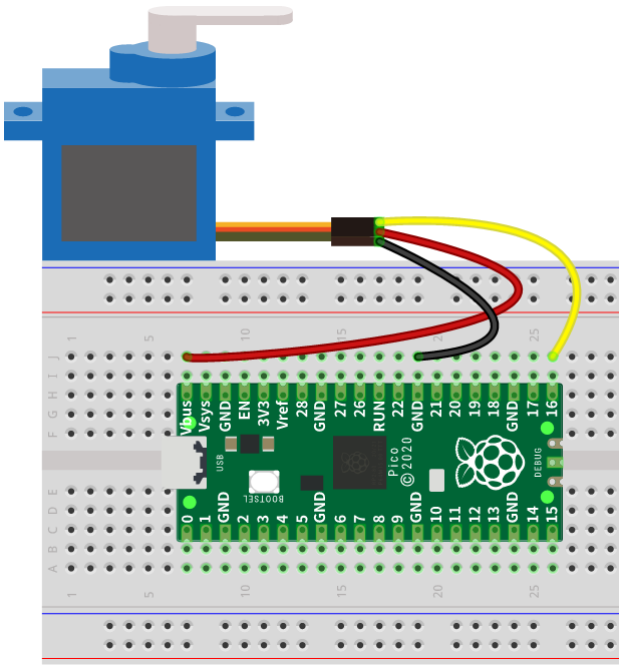
Circuit

Use caution when supplying power to the Servo, it should be 5V. Make sure you do not make any errors when connecting the Servo to the power supply.

Schematic diagram



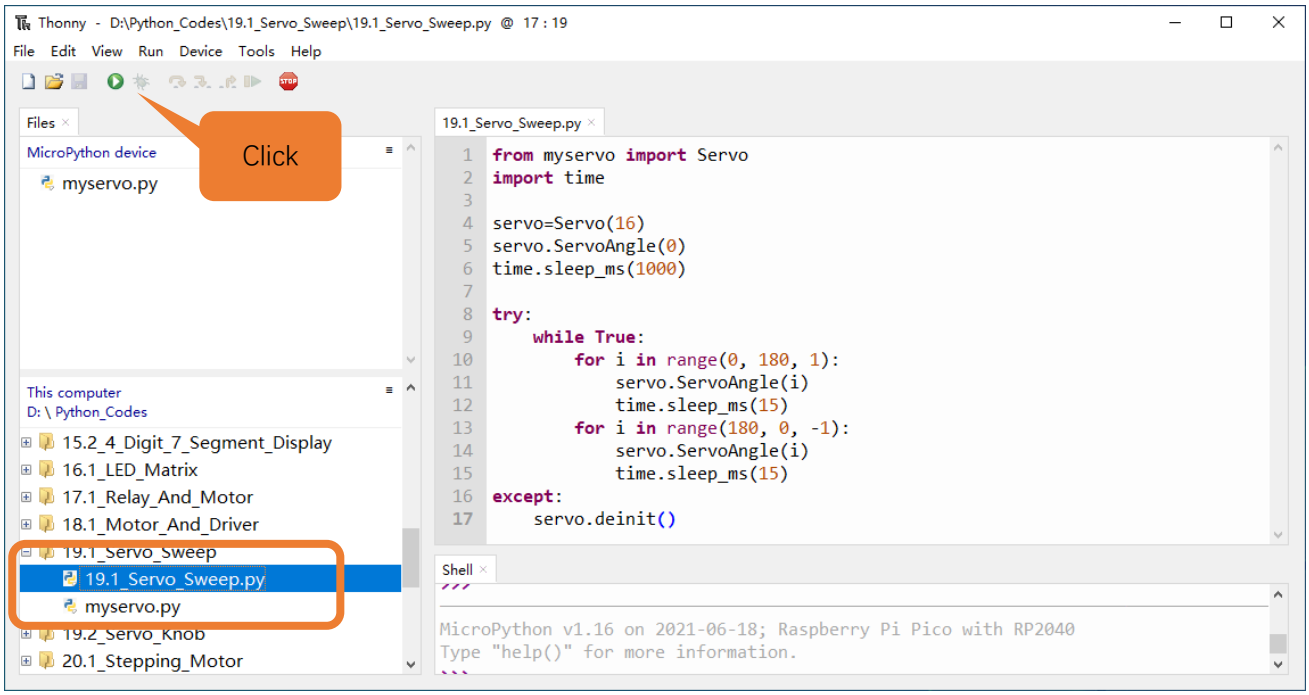
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



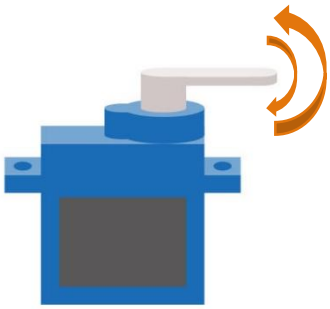
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "19.1_Servo_Sweep".Select "myservo.py", right click your mouse to select "Upload to /", wait for "myservo.py" to be uploaded to Raspberry Pi Pico and then double click "19.1_Servo_Sweep.py".

19.1_Servo_Sweep



Click "Run current script", the Servo will rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



The following is the program code:

```
from myservo import Servo
import time

servo=Servo(16)
servo.ServoAngle(0)
time.sleep_ms(1000)

try:
```

```
9       while True:
10          for i in range(0, 180, 1):
11              servo.ServoAngle(i)
12              time.sleep_ms(15)
13          for i in range(180, 0, -1):
14              servo.ServoAngle(i)
15              time.sleep_ms(15)
16  except:
17      servo.deinit()
```

Import myservo module.

```
1   from myservo import Servo
```

Initialize pins of the servo and set the starting point of the servo to 0 degree.

```
4   servo=Servo(16)
5   servo.ServoAngle(0)
6   time.sleep_ms(1000)
```

Control the servo to rotate to a specified angle within the range of 0-180 degrees.

```
9   servo.ServoAngle(i)
```

Use two for loops. The first one controls the servo to rotate from 0 degree to 180 degrees while the other controls it to rotate back from 180 degrees to 0 degree.

```
10          for i in range(0, 180, 1):
11              servo.ServoAngle(i)
12              time.sleep_ms(15)
13          for i in range(180, 0, -1):
14              servo.ServoAngle(i)
15              time.sleep_ms(15)
```

Reference

class myServo

Before each use of **Servo**, please make sure myservo.py has been uploaded to "/" of Raspberry Pi Pico, and then add the statement "**from myservo import Servo**" to the top of the python file.

Servo(): The object that controls the servo, with the default pin GP15, default frequency 50Hz.

ServoDuty(duty): The function controls the servo's rotating angle through the duty cycle.

duty: Ranges from 1638 to 8190, with 1638 corresponding to the servo's 0 degree and 8190 corresponding to 180 degrees.

ServoAngle(pos): The function passes in pos(angle) value to control the servo's rotating angle.

pos: Ranging from 0-180, corresponding the 0-180 degrees of the servo.

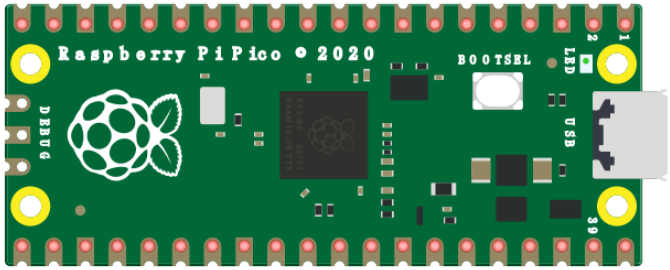
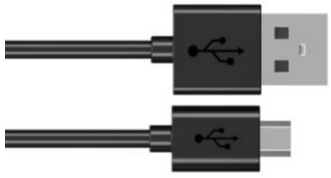
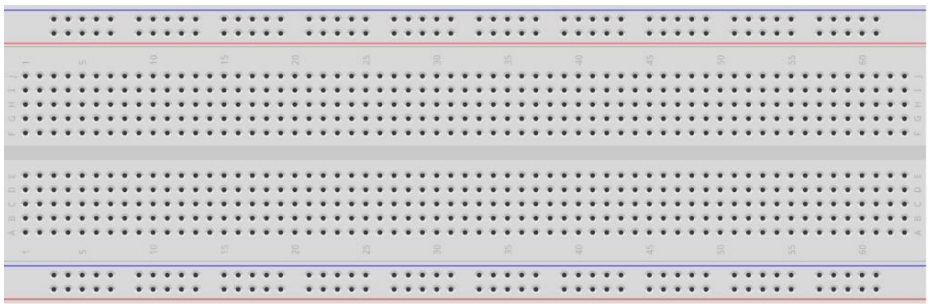
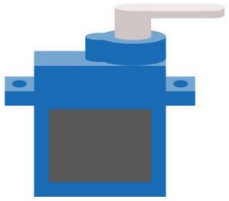
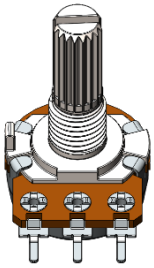
ServoTime(us): The function passes in us(time) to control the servo's rotating angle.

us: Ranges from 500-2500, with 500 corresponding to the servo's 0 degree and 2500 corresponding to 180 degrees.

Project 19.2 Servo Knob

Use a potentiometer to control the servomotor to rotate at any angle.

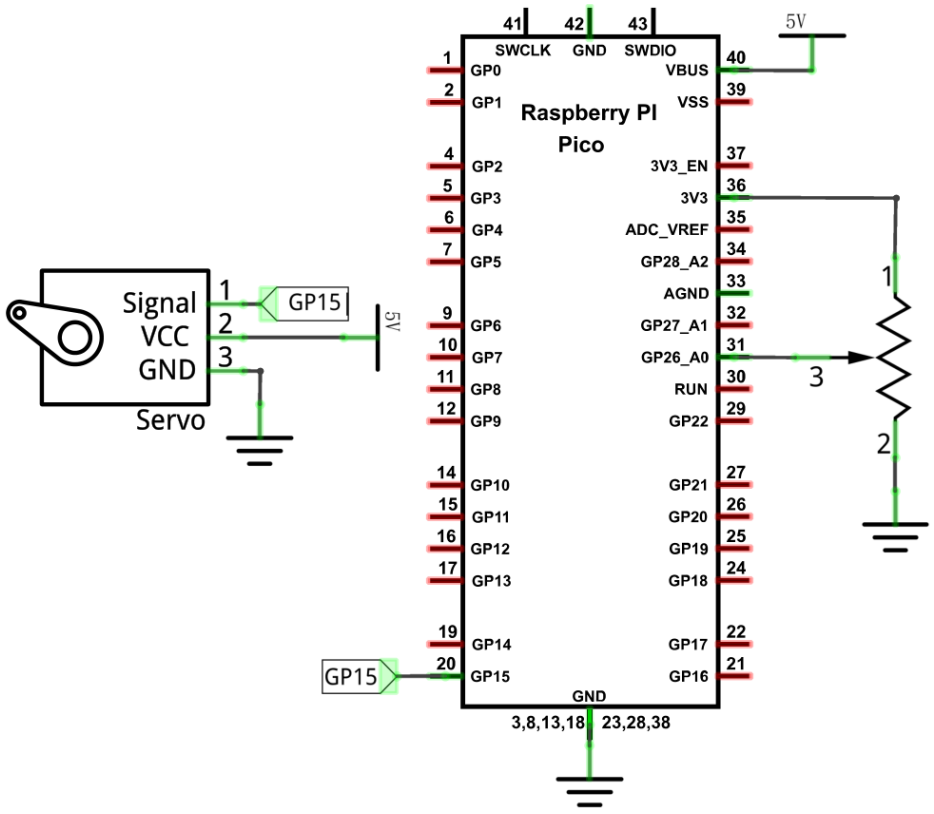
Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Servo x1 	Jumper 
Rotary potentiometer x1 	

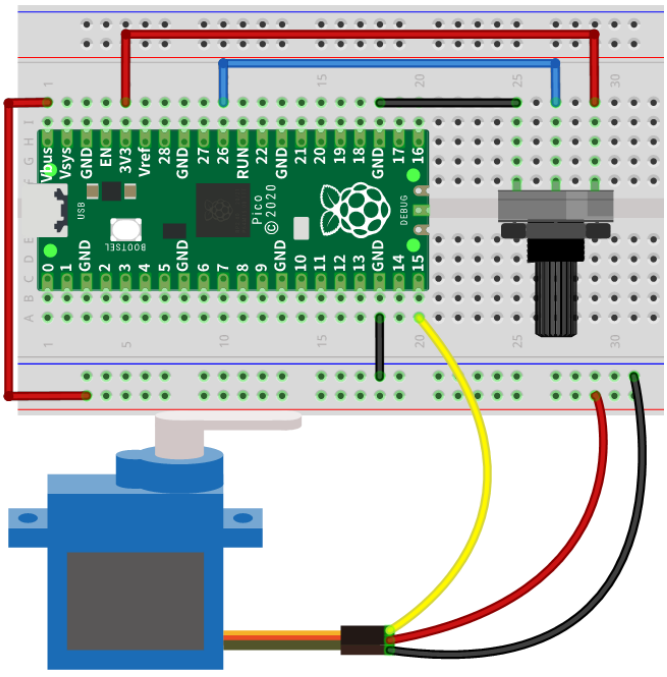
Circuit

Use caution when supplying power to the Servo, it should be 5V. Make sure you do not make any errors when connecting the Servo to the power supply.

Schematic diagram



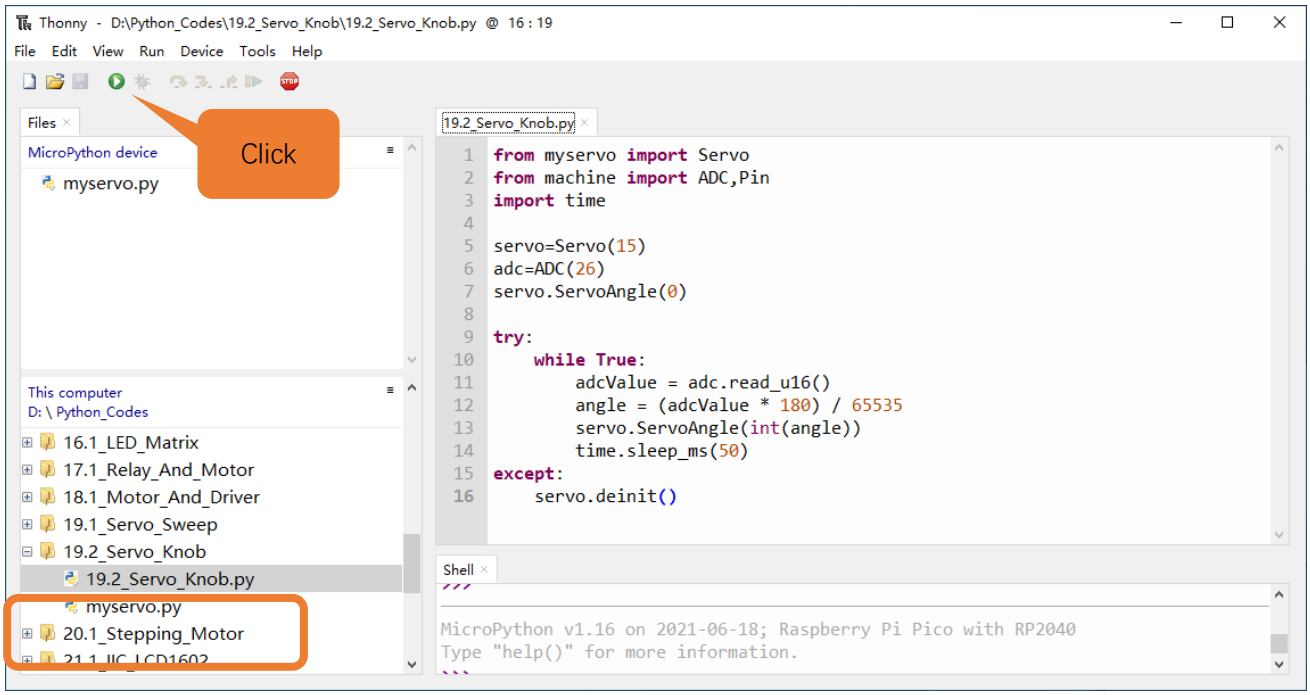
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



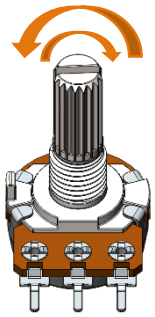
Code

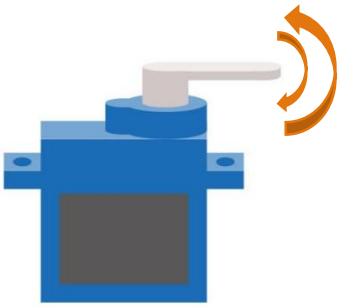
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "19.2_Servo_Knob". Select "myservo.py", right click your mouse to select "Upload to /", wait for "myservo.py" to be uploaded to Raspberry Pi Pico and then double click "19.2_Servo_Knob.py".

19.2_Servo_Knob



Click "Run current script", twist the potentiometer back and forth, and the servo motor rotates accordingly.





The following is the program code:

```
from myservo import Servo
from machine import ADC,Pin
import time

servo=Servo(15)
adc=ADC(26)
servo.ServoAngle(0)

try:
    while True:
        adcValue = adc.read_u16()
        angle = (adcValue * 180) / 65535
        servo.ServoAngle(int(angle))
        time.sleep_ms(50)
except:
    servo.deinit()
```

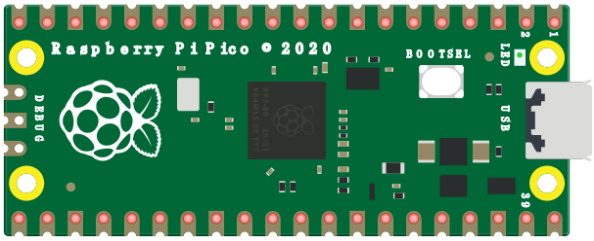
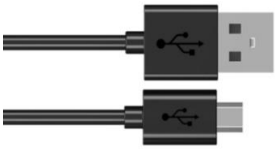
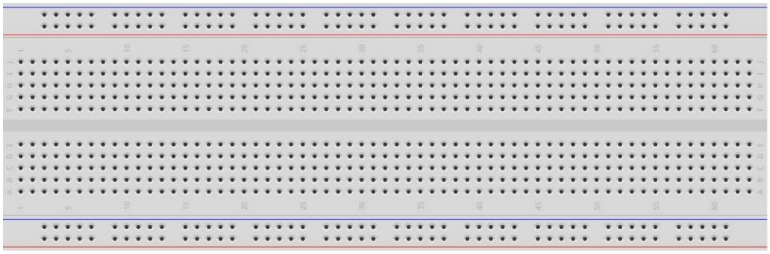
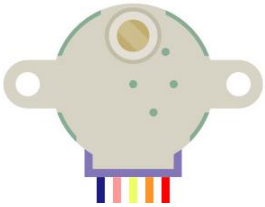
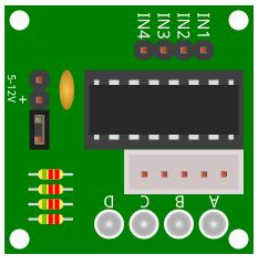
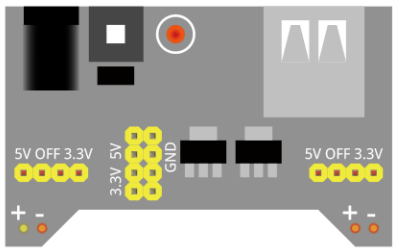
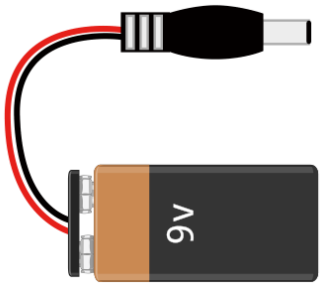
In this project, we will use GP26 of Raspberry Pi Pico to read the ADC value of the rotary potentiometer and then convert it to the angle value required by the servo and control the servo to rotate to the corresponding angle.

Chapter 20 Stepper Motor

In this project, we will learn how to drive a Stepper Motor, and understand its working principle.

Project 20.1 Stepper Motor

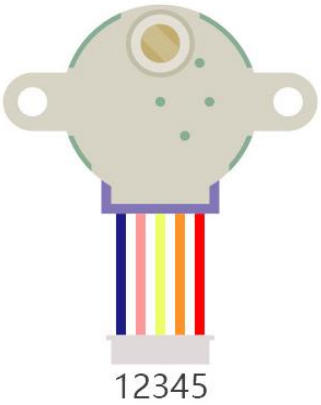
Component List

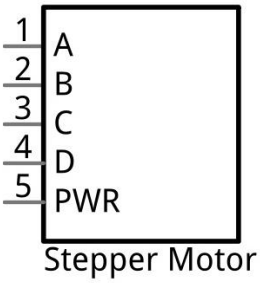
Raspberry Pi Pico x1 	USB cable x1 	
Breadboard x1 		
Stepper Motor x1 	ULN2003 Stepper motorDriver x1 	Jumper 
Breadboard Power x1 	9V battery (prepared by yourself) & battery line x1 	

Component knowledge

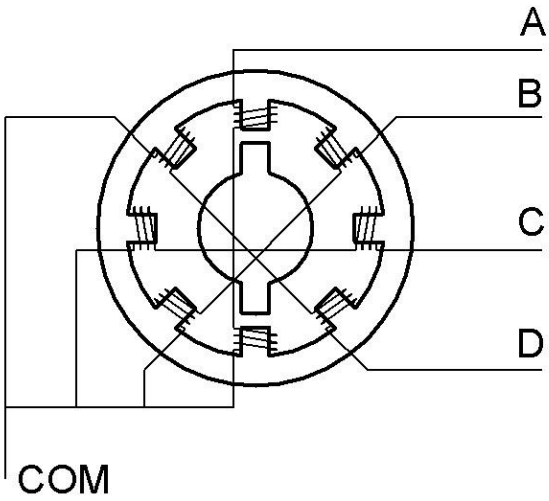
Stepper Motor x1

Stepper Motors are an open-loop control device, which converts an electronic pulse signal into angular displacement or linear displacement. In a non-overload condition, the speed of the motor and the location of the stops depend only on the pulse signal frequency as well as the number of pulses and are not affected by changes in load as with a DC Motor. A small Four-Phase Deceleration Stepper Motor is shown here:



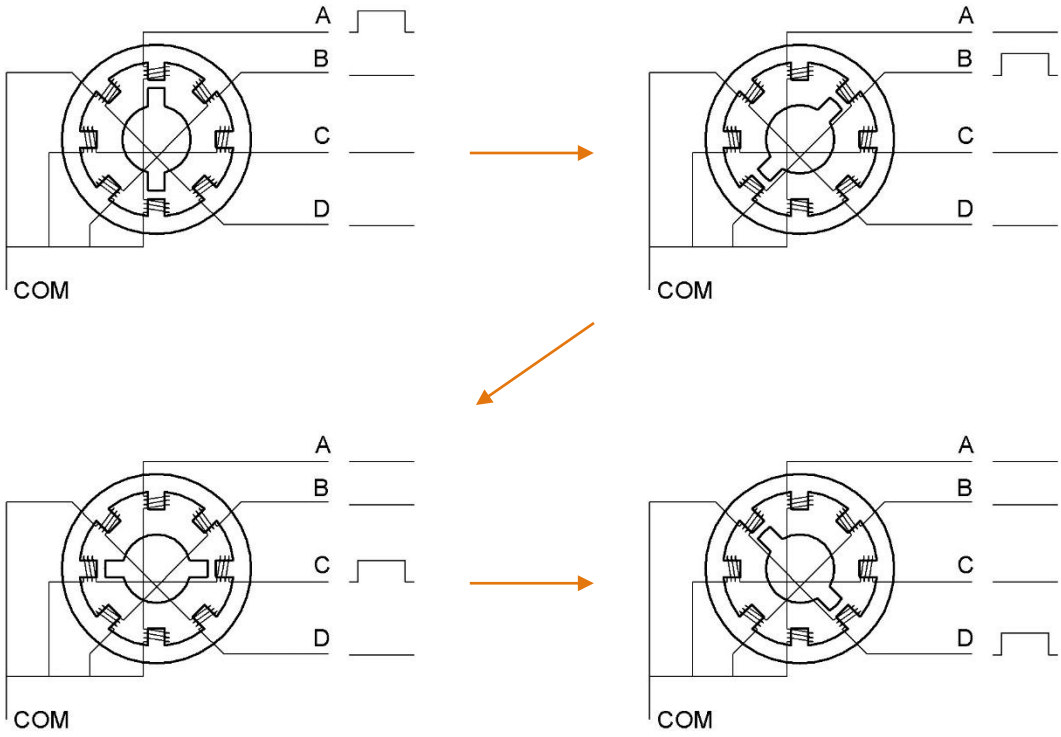


The electronic schematic diagram of a Four-Phase Stepper Motor is shown below:



The outside case or housing of the Stepper Motor is the Stator and inside the Stator is the Rotor. There are a specific number of individual coils, usually an integer multiple of the number of phases the motor has, when the Stator is powered ON, an electromagnetic field will be formed to attract a corresponding convex diagonal groove or indentation in the Rotor's surface. The Rotor is usually made of iron or a permanent magnet. Therefore, the Stepper Motor can be driven by powering the coils on the Stator in an ordered sequence (producing a series of "steps" or stepped movements).

A common driving process is as follows:



In the course above, the stepping motor rotates a certain angle once, which is called a step. By controlling the number of rotation steps, you can control the stepping motor rotation angle. By controlling the time between two steps, you can control the stepping motor rotation speed. When rotating clockwise, the order of coil powered on is: A→B→C→D→A→··· And the rotor will rotate in accordance with the order, step by step down, called four steps four pats. If the coils is powered on in the reverse order, D→C→B→A→D→···, the rotor will rotate in anti-clockwise direction.

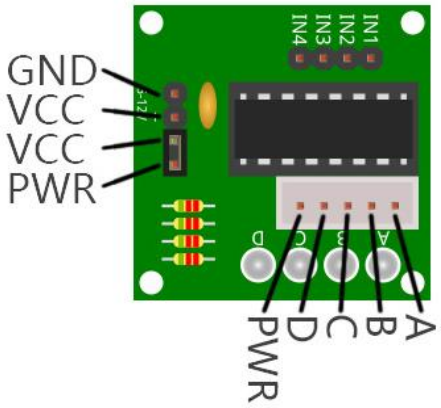
There are other methods to control Stepper Motors, such as: connect A phase, then connect A B phase, the stator will be located in the center of A B, which is called a half step. This method can improve the stability of the Stepper Motor and reduces noise. The sequence of powering the coils looks like this: A→ AB→B →BC→C →CD→D→DA→A→·····, the rotor will rotate in accordance to this sequence at a half-step at a time, called four-steps, eight-part. Conversely, if the coils are powered ON in the reverse order the Stepper Motor will rotate in the opposite direction.

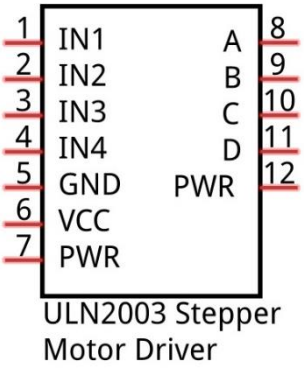
The stator in the Stepper Motor we have supplied has 32 magnetic poles. Therefore, to complete one full revolution requires 32 full steps. The rotor (or output shaft) of the Stepper Motor is connected to a speed reduction set of gears and the reduction ratio is 1:64. Therefore, the final output shaft (exiting the Stepper Motor's housing) requires 32 X 64 = 2048 steps to make one full revolution.

The time required for each step of the stepper motor must be greater than 2ms to operate normally.

ULN2003 Stepper motor driver

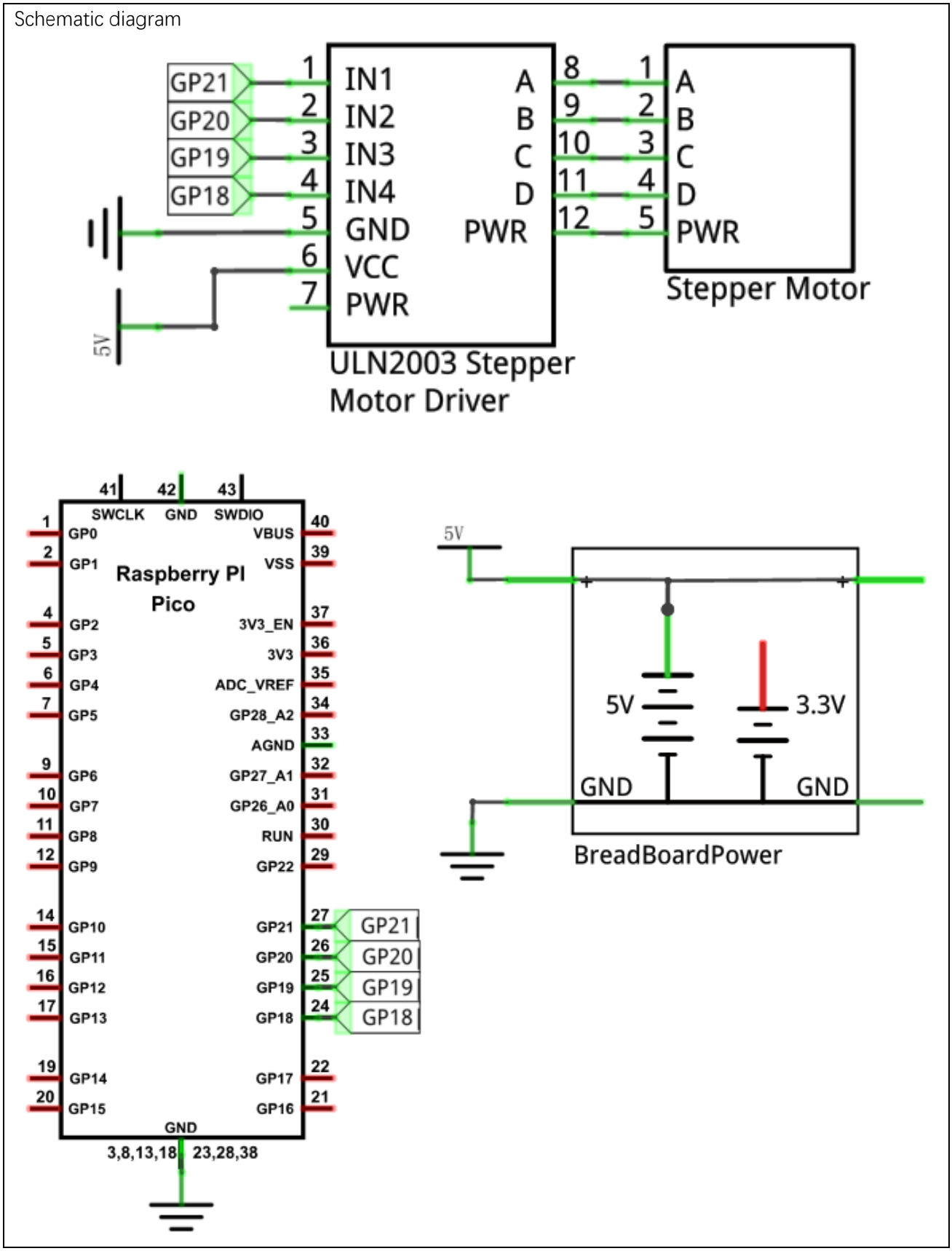
A ULN2003 Stepper Motor Driver is used to convert weak signals into more powerful control signals in order to drive the Stepper Motor. In the illustration below, the input signal IN1-IN4 corresponds to the output signal A-D, and four LEDs are integrated into the board to indicate the state of these signals. The PWR interface can be used as a power supply for the Stepper Motor. By default, PWR and VCC are connected.



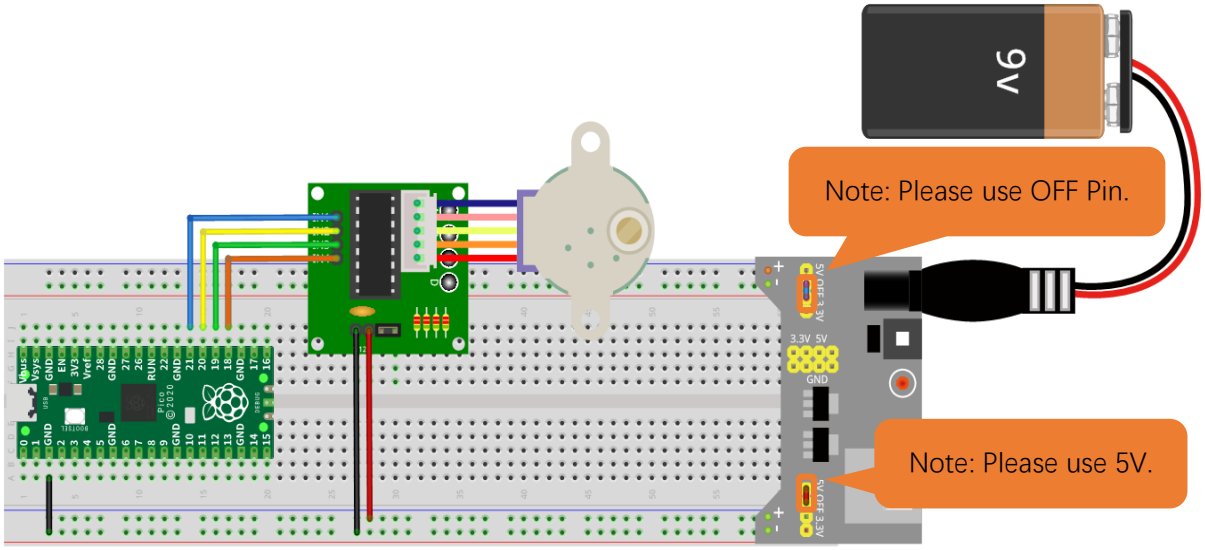


Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

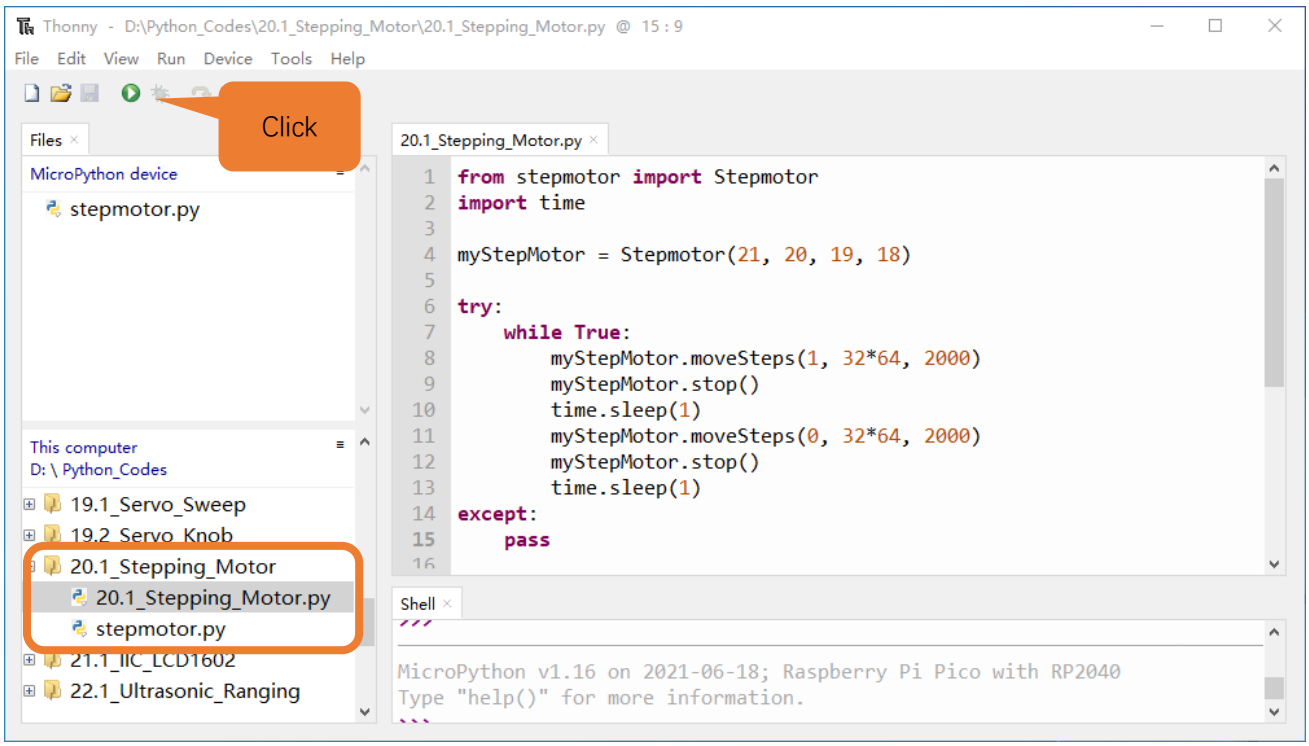


Code

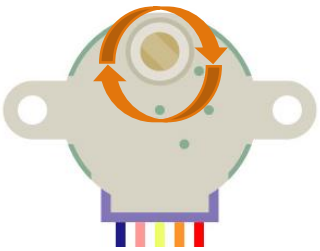
This code uses the four-step, four-part mode to drive the Stepper Motor in the clockwise and anticlockwise directions.

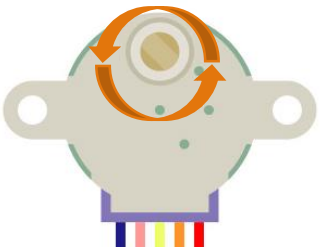
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "20.1_Stepping_Motor". Select "stepmotor.py", right click your mouse to select "Upload to /", wait for "stepmotor.py" to be uploaded to Raspberry Pi Pico and then double click "20.1_Stepping_Motor.py".

20.1_Stepping_Motor



Click "Run current script", the stepper motor will rotate 360° clockwise and stop for 1s, and then rotate 360° anticlockwise and stop for 1s. This action is repeated in an endless loop. Press Ctrl+C or click "Stop/Restart backend" to exit the program.





The following is the program code:

```
from stepmotor import Stepmotor
import time

myStepMotor=Stepmotor(21, 20, 19, 18)

try:
    while True:
        myStepMotor.moveSteps(1, 32*64, 2000)
        myStepMotor.stop()
        time.sleep(1)
        myStepMotor.moveSteps(0, 32*64, 2000)
        myStepMotor.stop()
        time.sleep(1)
except:
    pass
```

Import time and stepmotor modules.

```
from stepmotor import Stepmotor
import time
```

In this project, we define four pins to drive the stepper motor.

```
myStepMotor=Stepmotor(21, 20, 19, 18)
```

Call the function moveSteps to control the stepper motor to rotate for 360°and then call function stop() to stop it.

```
    myStepMotor.moveSteps(1,32*64,2000)
    myStepMotor.stop()
```

Repeatedly control the stepmotor to rotate 360° clockwise and then rotate 360° anti-clockwise.

```
    while True:
        myStepMotor.moveSteps(1,32*64,2000)
        myStepMotor.stop()
        time.sleep(1)
        myStepMotor.moveSteps(0,32*64,2000)
        myStepMotor.stop()
        time.sleep(1)
```

Reference

class stepmotor	

Before each use of the object **Stepmotor**, please make sure that stepmotor.py has been uploaded to "/" of Raspberry Pi Pico, and then add the statement "**from stepmotor import Stepmotor**" to the top of the python file.

Stepmotor(): The object to control the stepper motor. The default control pins are GP21, GP20, GP19, GP18.

moveSteps(direction,steps,us): Control the stepper motor to rotate a specified number of steps.

direction: The rotation direction of stepper motor.

Steps: Rotation steps of the stepper motor.

us: The time needed for stepper motor to rotate one step, which should be greater than 2000.

moveAround(direction,turns,us): Control the stepper motor to rotate a specific number of turns.

Turns: Number of turns that the stepper motor rotates.

moveAngle(direction,angles,us): Control the stepper motor to rotate a specific angle.

Angles: Rotation angles that the stepper motor rotates.

stop(): Stop the stepper motor.

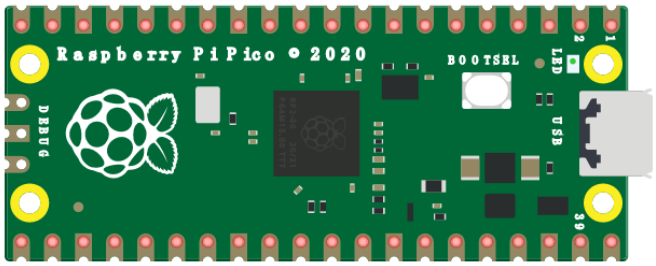
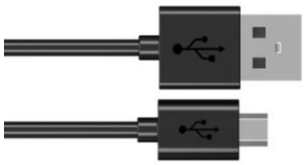
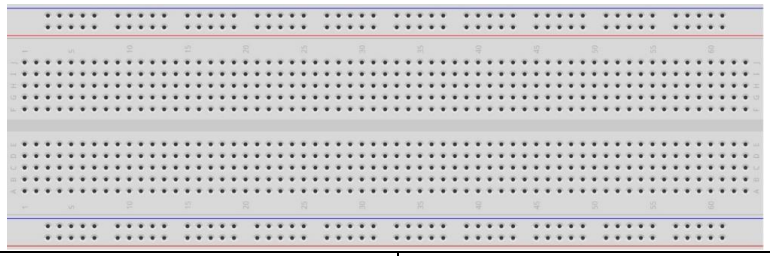
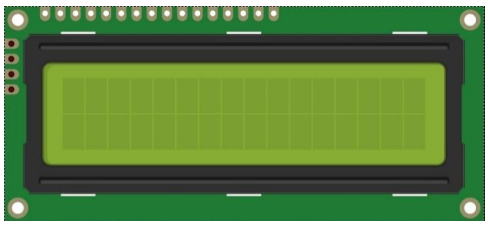
Chapter 21 LCD1602

In this chapter, we will learn about the LCD1602 Display Screen.

Project 21.1 LCD1602

In this section, we learn how to use LCD1602 to display something.

Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
LCD1602 Module x1 	Jumper 

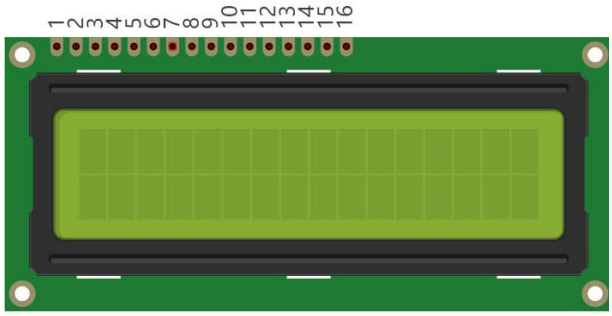
Component knowledge

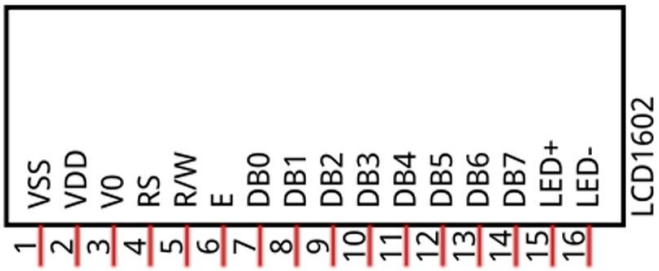
I2C communication

I2C (Inter-Integrated Circuit) is a two-wire serial communication mode, which can be used for the connection of micro controllers and their peripheral equipment. Devices using I2C communication must be connected to the serial data (SDA) line, and serial clock (SCL) line (called I2C bus). Each device has a unique address and can be used as a transmitter or receiver to communicate with devices connected to the bus.

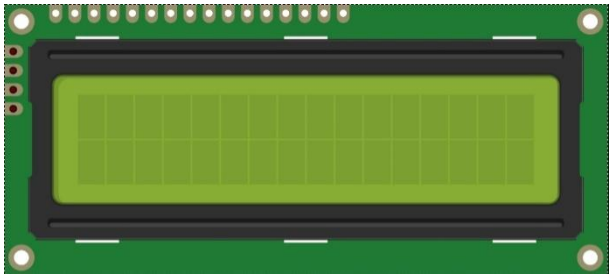
LCD1602 communication'

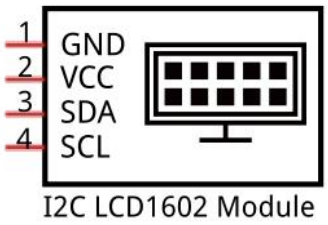
The LCD1602 Display Screen can display 2 lines of characters in 16 columns. It is capable of displaying numbers, letters, symbols, ASCII code and so on. As shown below is a monochrome LCD1602 Display Screen along with its circuit pin diagram.





I2C LCD1602 Display Screen integrates an I2C interface, which connects the serial-input & parallel-output module to the LCD1602 Display Screen. This allows us to use only four lines to operate the LCD1602.

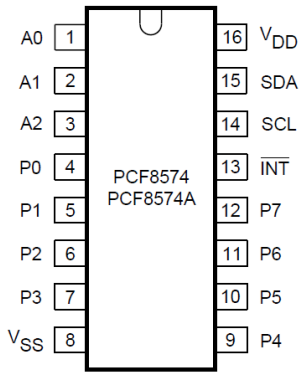




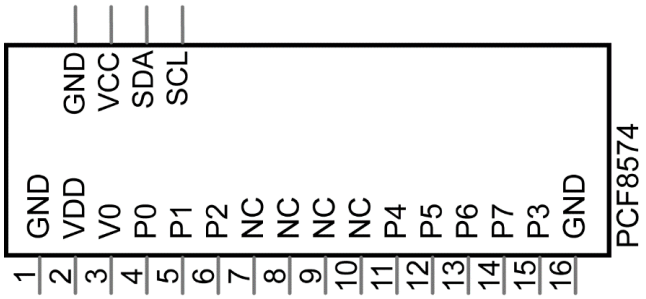
The serial-to-parallel IC chip used in this module is PCF8574T (PCF8574AT), and its default I2C address is 0x27(0x3F).

Below is the PCF8574 pin schematic diagram and the block pin diagram:

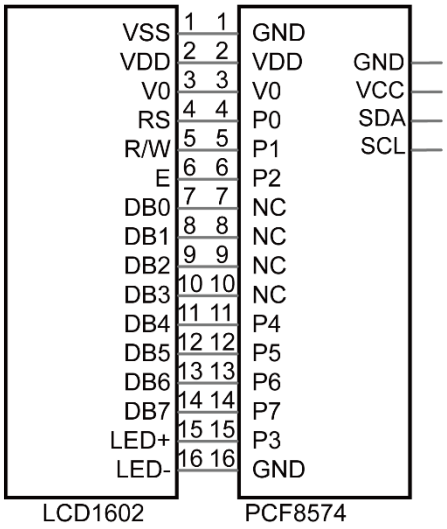
PCF8574 chip pin diagram:



PCF8574 module pin diagram



PCF8574 module pin and LCD1602 pin are corresponding to each other and connected with each other:

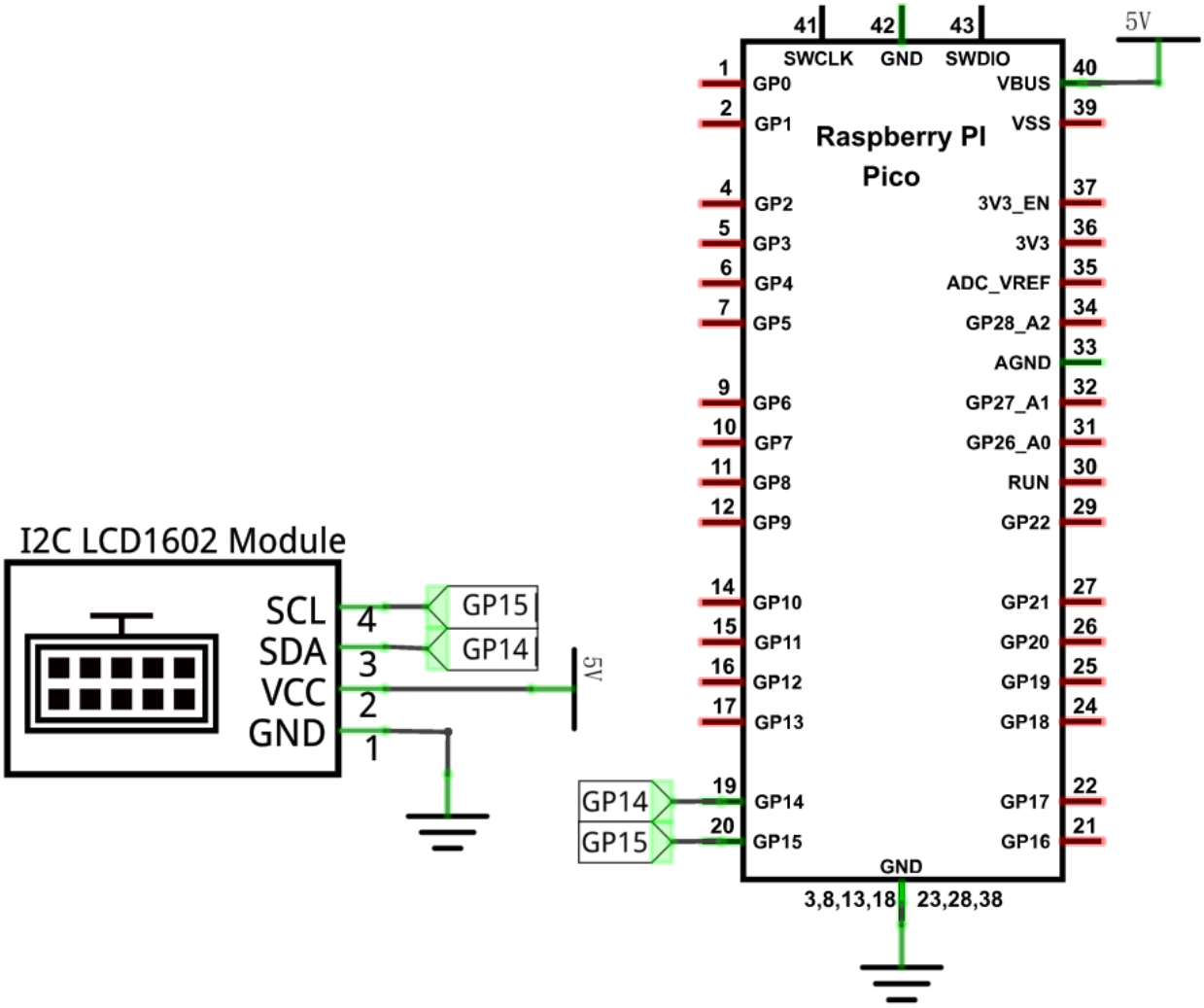


Therefore, we only need four pins to control the 16 pins of the LCD1602 Display Screen through the I2C interface.

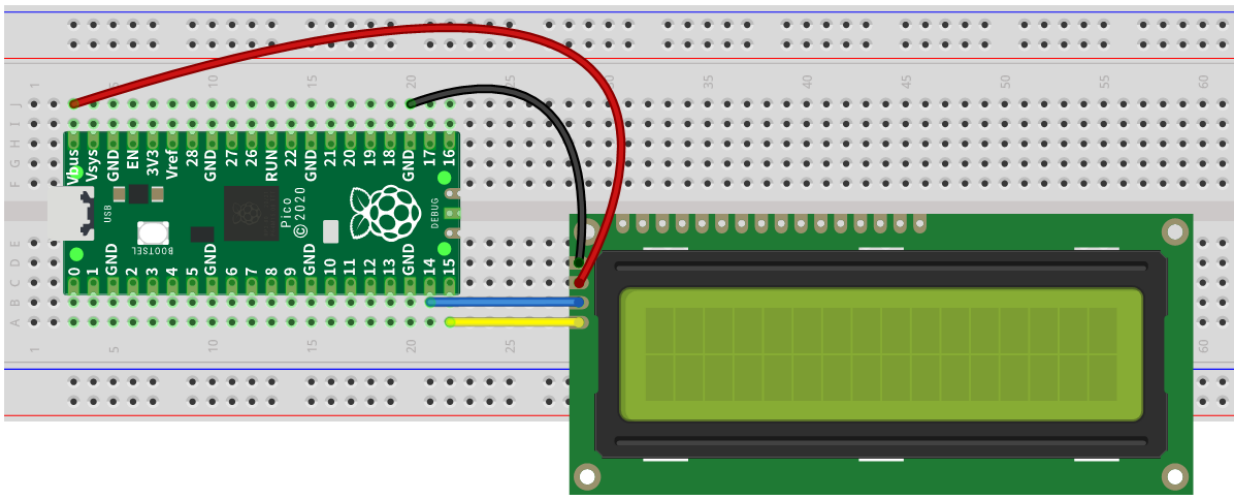
In this project, we will use the I2C LCD1602 to display some static characters and dynamic variables.

Circuit

Schematic diagram



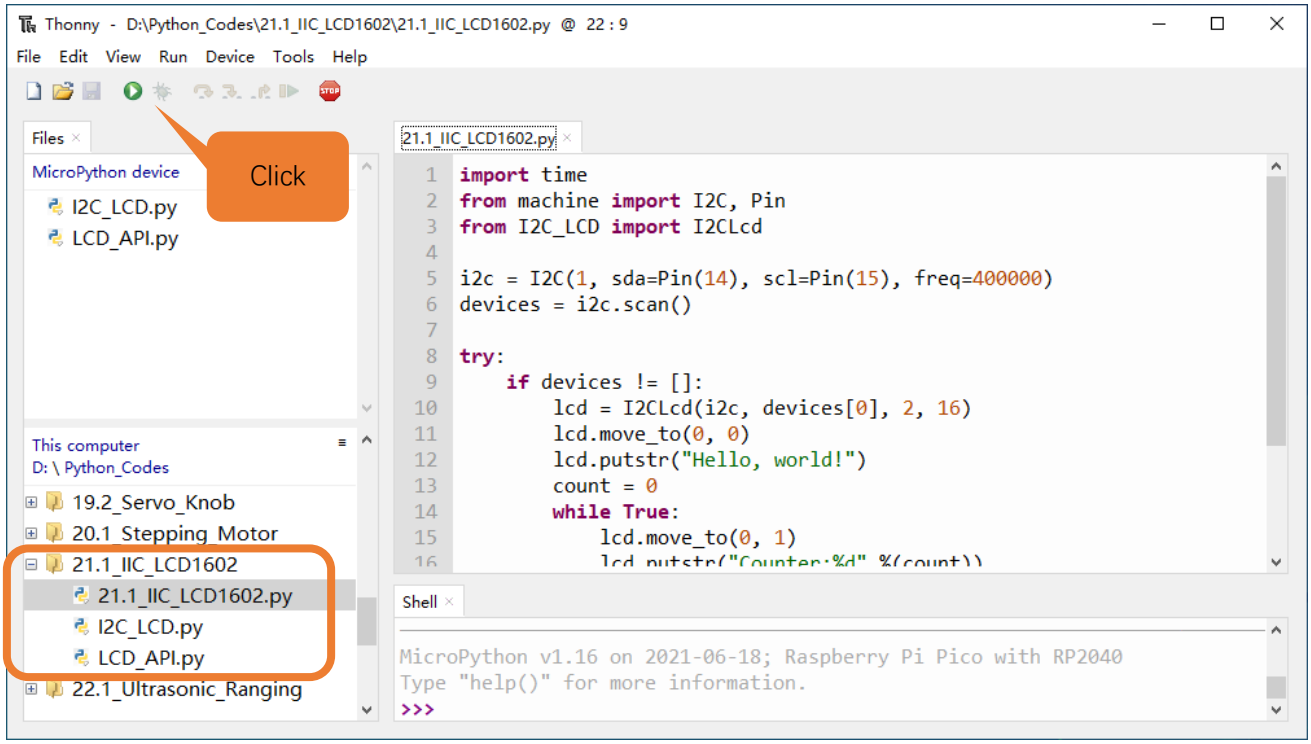
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "21.1_I2C_LCD1602". Select "I2C_LCD.py"and "LCD_API.py", right click your mouse to select "Upload to /", wait for "I2C_LCD.py" and "LCD_API.py" to be uploaded to Raspberry Pi Pico and then double click "21.1_I2C_LCD1602.py".

21.1_I2C_LCD1602



Click "Run current script" and LCD1602 displays some characters.Press Ctrl+C or click "Stop/Restart backend" to exit the program.

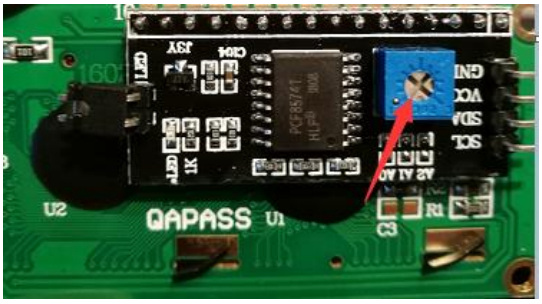


So far, at this writing, we have two types of LCD1602 on sale. One needs to adjust the backlight, and the other does not.

The LCD1602 that does not need to adjust the backlight is shown in the figure below.



If the LCD1602 you received is the following one, and you cannot see anything on the display or the display is not clear, try rotating the white knob on back of LCD1602 slowly, which adjusts the contrast, until the screen can display clearly.



The following is the program code:

```
import time
from machine import I2C, Pin
from I2C_LCD import I2CLcd

i2c = I2C(1, sda=Pin(14), scl=Pin(15), freq=400000)
devices = i2c.scan()

try:
    if devices != []:
        lcd = I2CLcd(i2c, devices[0], 2, 16)
        lcd.move_to(0, 0)
        lcd.putstr("Hello, world!")
        count = 0
        while True:
            lcd.move_to(0, 1)
            lcd.putstr("Counter:%d" %(count))
            time.sleep(1)
            count += 1
    else:
        print("No address found")
except:
    pass
```

Import time, I2C and I2C_LCD modules.

```
import time
from machine import I2C, Pin
from I2C_LCD import I2CLcd
```

Create an I2C object, initialize the I2C parameter configuration, and associate it with the LCD1602 pin. Call the scan() function to query the LCD1602 device address.

```
i2c = I2C(1, sda=Pin(14), scl=Pin(15), freq=400000)
devices = i2c.scan()
```

Use the if statement to determine whether the queried I2C device address is empty. If it is not empty, create an I2CLcd object and set the created I2C object, I2C device address, and the number of rows and columns of LCD1602; if it is empty, print out "No address" found" and the program exits.

```
9     if devices != []:
10        lcd = I2CLcd(i2c, devices[0], 2, 16)
...       ...
19    else:
20        print("No address found")
```

Move the cursor of LCD1602 to the first row, first column, and print out "Hello, world!"

```
11    lcd.move_to(0, 0)
12    lcd.putstr("Hello, world!")
```

The second line of LCD1602 continuously prints the number of seconds after the Raspberry Pi Pico program runs.

```
14    while True:
15        lcd.move_to(0, 1)
16        lcd.putstr("Counter:%d" %(count))
17        time.sleep(1)
18        count += 1
```

Reference

Class I2CLcd

Before each use of the object **I2CLcd**, please make sure that **I2C_LCD.py** and **LCD_API.py** have been uploaded to "/" of Raspberry Pi Pico, and then add the statement "**from I2C_LCD import I2CLcd**" to the top of the python file.

clear(): Clear the LCD1602 screen display.

show_cursor(): Show the cursor of LCD1602.

hide_cursor(): Hide the cursor of LCD1602.

blink_cursor_on(): Turn on cursor blinking.

blink_cursor_off(): Turn off cursor blinking.

display_on(): Turn on the display function of LCD1602.

display_off(): Turn on the display function of LCD1602.

backlight_on(): Turn on the backlight of LCD1602.

backlight_off(): Turn on the backlight of LCD1602.

move_to(cursor_x, cursor_y): Move the cursor to a specified position.

cursor_x: Column cursor_x.

cursor_y: Row cursor_y.

putchar(char): Print the character in the bracket on LCD1602.

putstr(string): Print the string in the bracket on LCD1602.

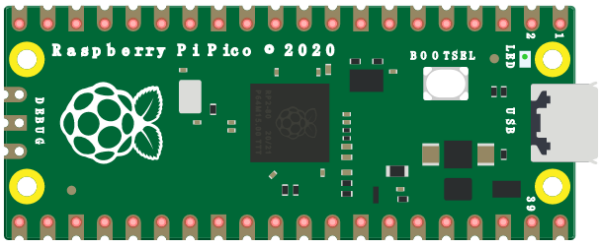
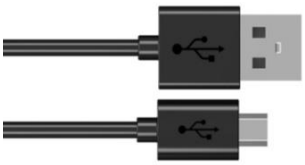
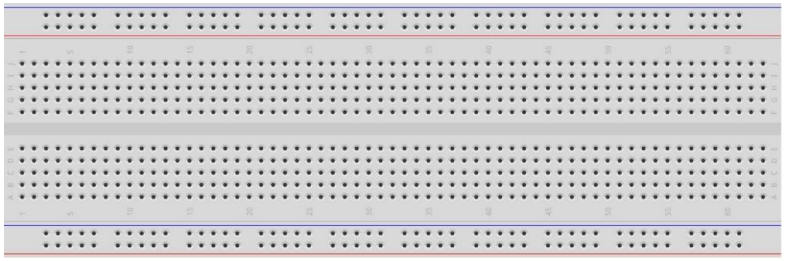
Chapter 22 Ultrasonic Ranging

In this chapter, we learn a module, which use ultrasonic to measure distance, HC SR04.

Project 22.1 Ultrasonic Ranging

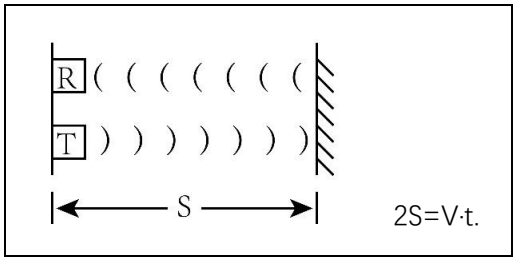
In this project, we use ultrasonic ranging module to measure distance, and print out the data in the terminal.

Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Jumper 	HC SR04 x1 

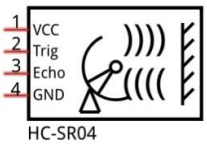
Component Knowledge

The Ultrasonic Ranging Module uses the principle that ultrasonic waves will reflect when they encounter any obstacles. This is possible by counting the time interval between when the ultrasonic wave is transmitted to when the ultrasonic wave reflects back after encountering an obstacle. Time interval counting will end after an ultrasonic wave is received, and the time difference (delta) is the total time of the ultrasonic wave's journey from being transmitted to being received. Because the speed of sound in air is a constant, and is about v=340m/s, we can calculate the distance between the Ultrasonic Ranging Module and the obstacle: s=vt/2.



The HC-SR04 Ultrasonic Ranging Module integrates both an ultrasonic transmitter and a receiver. The transmitter is used to convert electrical signals (electrical energy) into high frequency (beyond human hearing) sound waves (mechanical energy) and the function of the receiver is opposite of this. The picture and the diagram of the HC SR04 Ultrasonic Ranging Module are shown below:





Pin description:

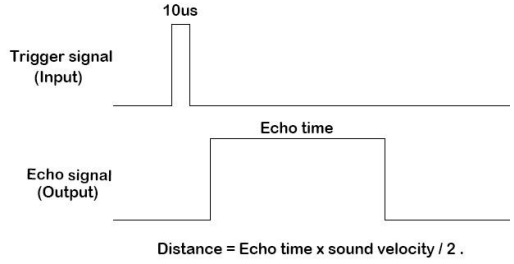
Pin	Description
VCC	power supply pin
Trig	trigger pin
Echo	Echo pin
GND	GND

Technical specs:

Working voltage: 5V Working current: 12mA

Minimum measured distance: 2cm Maximum measured distance: 200cm

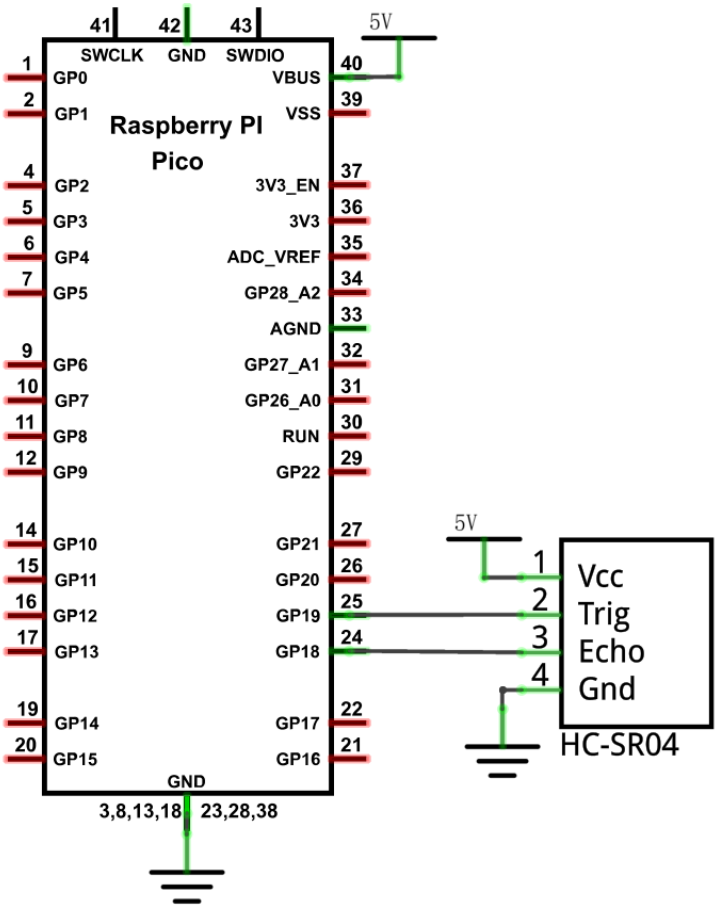
Instructions for Use: output a high-level pulse in Trig pin lasting for least 10us, the module begins to transmit ultrasonic waves. At the same time, the Echo pin is pulled up. When the module receives the returned ultrasonic waves from encountering an obstacle, the Echo pin will be pulled down. The duration of high level in the Echo pin is the total time of the ultrasonic wave from transmitting to receiving, s=vt/2.



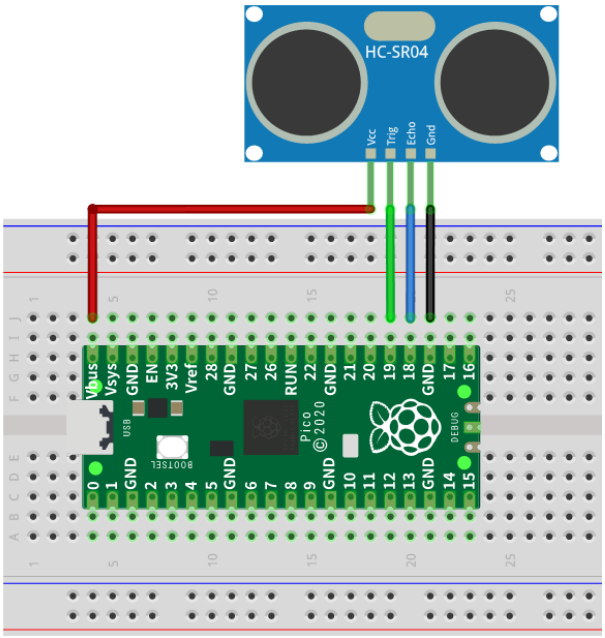
Circuit

Note that the voltage of ultrasonic module is 5V in the circuit.

Schematic diagram



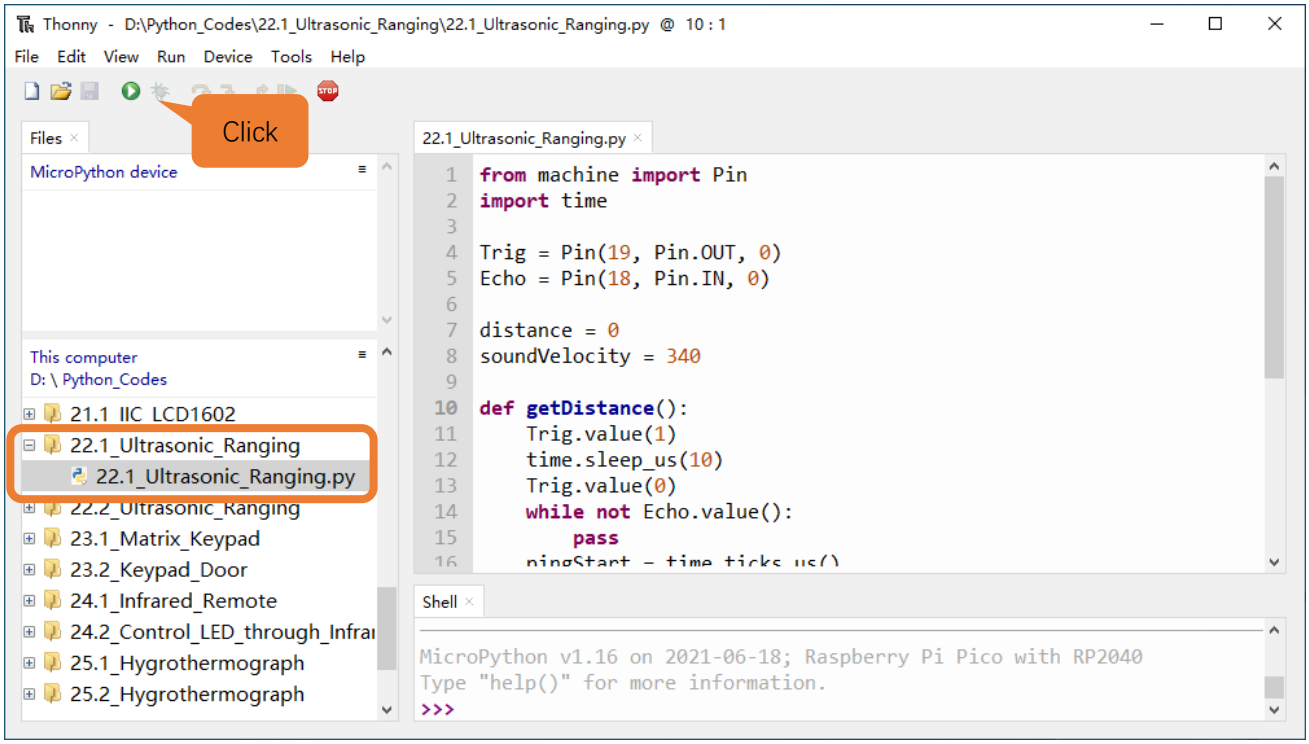
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "22.1_Ultrasonic_Ranging" and double click "22.1_Ultrasonic_Ranging.py".

22.1_Ultrasonic_Ranging



Click "Run current script", you can use it to measure the distance between the ultrasonic module and the object. As shown in the following figure. Press Ctrl+C or click "Stop/Restart backend" to exit the program.

```
>>> %Run -c $EDITOR_CONTENT

  Distance:  5 cm
  Distance:  319 cm
  Distance:  302 cm
  Distance:  4 cm
  Distance:  5 cm
  Distance:  4 cm
  Distance:  4 cm
  Distance:  66 cm
  Distance:  22 cm
```

The following is the program code:

```
from machine import Pin
import time

Trig = Pin(19, Pin.OUT, 0)
Echo = Pin(18, Pin.IN, 0)

distance = 0
soundVelocity = 340

def getDistance():
    Trig.value(1)
    time.sleep_us(10)
    Trig.value(0)
    while not Echo.value():
        pass
    pingStart = time.ticks_us()
    while Echo.value():
        pass
    pingStop = time.ticks_us()
    distanceTime = time.ticks_diff(pingStop, pingStart) // 2
    distance = int(soundVelocity * distanceTime // 10000)
    return distance

time.sleep(2)
while True:
    time.sleep_ms(500)
    distance = getDistance()
    print("Distance: ", distance, "cm")
```

Define the control pins of the ultrasonic ranging module.

```
Trig = Pin(19, Pin.OUT, 0)
Echo = Pin(18, Pin.IN, 0)
```

Set the speed of sound.

```
soundVelocity = 340
```

The getDistance() function is used to drive the ultrasonic module to measure distance. In the function, after the Trig pin keeps at high level for 10us to start the ultrasonic module, Echo.value() is used to read the status of ultrasonic module's Echo pin, and then use timestamp function of the time module to calculate the duration of Echo pin's high level. Finally, calculate the measured distance based on time and return the value.

```
def getDistance():
    Trig.value(1)
    time.sleep_us(10)
    Trig.value(0)
    while not Echo.value():
        pass
```

```
    pingStart = time.ticks_us()
    while Echo.value():
        pass
    pingStop = time.ticks_us()
    distanceTime = time.ticks_diff(pingStop, pingStart) // 2
    distance = int(soundVelocity * distanceTime // 10000)
    return distance
```

Delay for 2 seconds and wait for the ultrasonic module to stabilize. Print data obtained from ultrasonic module every 500 milliseconds.

```
time.sleep(2)
while True:
    time.sleep_ms(500)
    distance = getDistance()
    print("Distance: ", distance, "CM")
```

Project 22.2 Ultrasonic Ranging

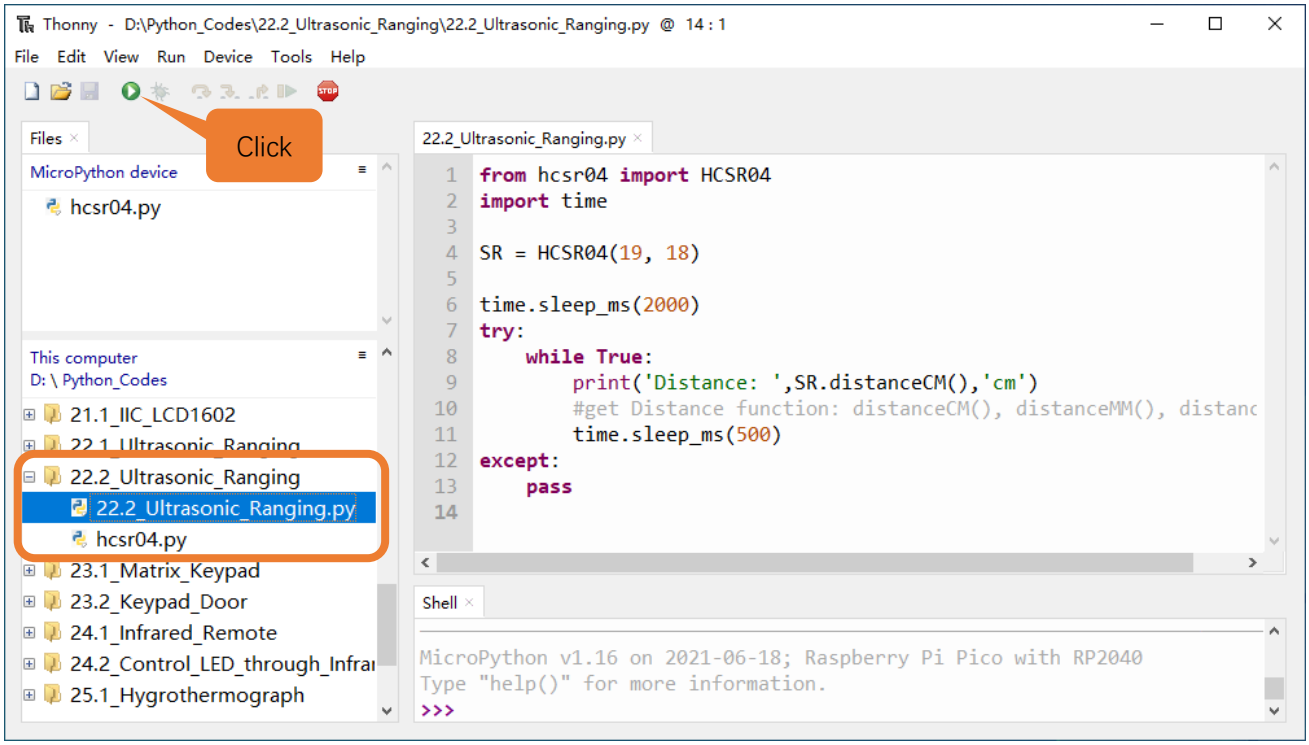
Component List and Circuit

Component List and Circuit are the same as the previous section.

Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "22.2_Ultrasonic_Ranging". Select "hcsr04.py", right click your mouse to select "Upload to /", wait for "hcsr04.py" to be uploaded to Raspberry Pi Pico and then double click "22.2_Ultrasonic_Ranging.py".

22.2_Ultrasonic_Ranging



Click "Run current script". Use the ultrasonic module to measure distance. As shown in the following figure. Press Ctrl+C or click "Stop/Restart backend" to exit the program.

```
>>> %Run -c $EDITOR_CONTENT

  Distance:  16 cm
  Distance:  17 cm
  Distance:  18 cm
  Distance:  17 cm
  Distance:  17 cm
  Distance:  16 cm
  Distance:  16 cm
  Distance:  16 cm
```

The following is the program code:

```
1  from hcsr04 import HCSR04
2  import time
3
4  SR = HCSR04(19, 18)
5
6  time.sleep_ms(2000)
7  try:
8      while True:
9          print('Distance: ',SR.distanceCM(),'cm')
10         #get Distance function: distanceCM(), distanceMM(), distanceM()
11         time.sleep_ms(500)
12 except:
13     pass
```

Import hcsr04 module.

```
1  from hcsr04 import HCSR04
```

Define an ultrasonic object and associate with the pins.

```
4  SR= HCSR04(18, 19)
```

Obtain the distance data returned from the ultrasonic ranging module.

```
9  SR.distanceCM()
```

Obtain the ultrasonic data every 500 milliseconds and print them out in “Shell”.

```
8      while True:
9          print('Distance: ',SR.distanceCM(),'cm')
10         #get Distance function: distanceCM(), distanceMM(), distanceM()
11         time.sleep_ms(500)
```

Reference

Class hcsr04

Before each use of object **HCSR04**, please add the statement “**from hcsr04 import HCSR04**” to the top of python file.

SRHC04(): Object of ultrasonic module. By default, trig pin is GP19 and echo pin is GP18.

distanceCM(): Obtain the distance from the ultrasonic to the measured object with the data type being int type, and the unit being cm.

distanceMM(): Obtain the distance from the ultrasonic to the measured object with the data type being int type, and the unit being mm.

distanceM(): Obtain the distance from the ultrasonic to the measured object with the data type being float type, and the unit being m.

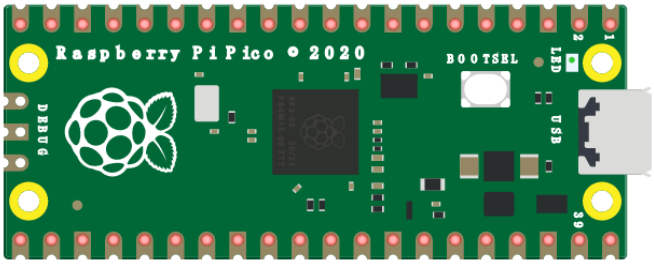
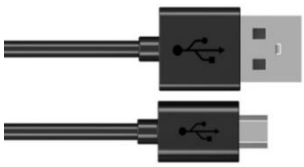
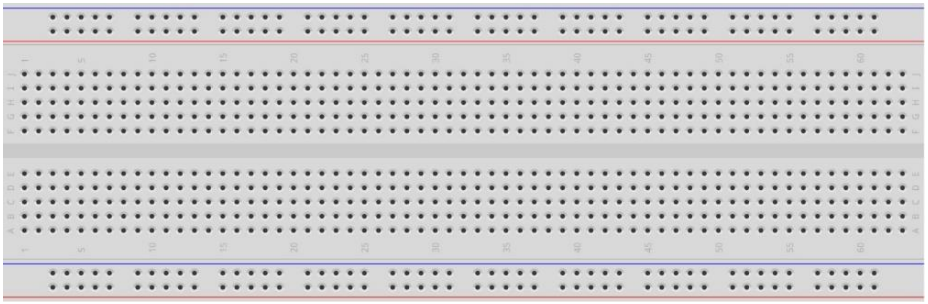
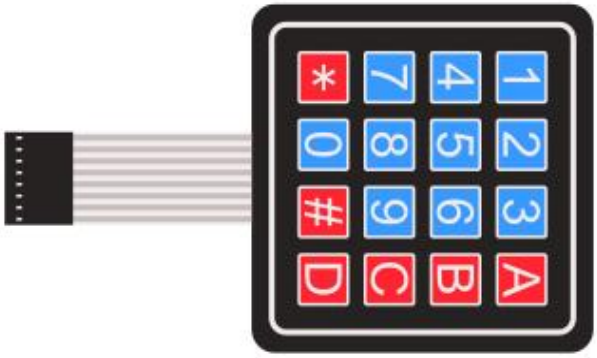
Chapter 23 Matrix Keypad

Earlier we learned about a single Push Button Switch. In this chapter, we will learn about Matrix Keyboards, which integrates a number of Push Button Switches as Keys for the purposes of Input.

Project 23.1 Matrix Keypad

In this project, we will attempt to get every key code on the Matrix Keypad to work.

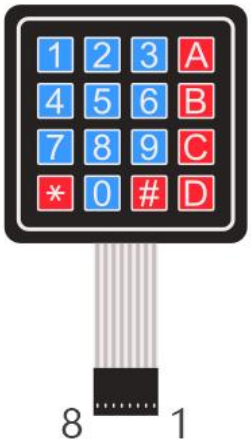
Component List

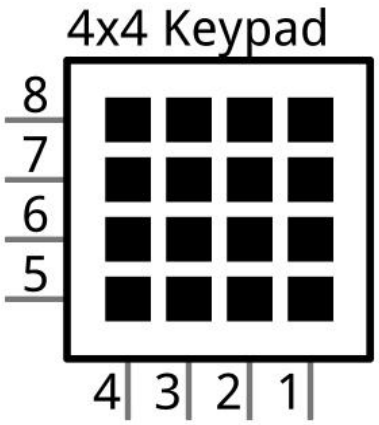
Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Jumper 	4x4 Matrix Keypad x1 

Component knowledge

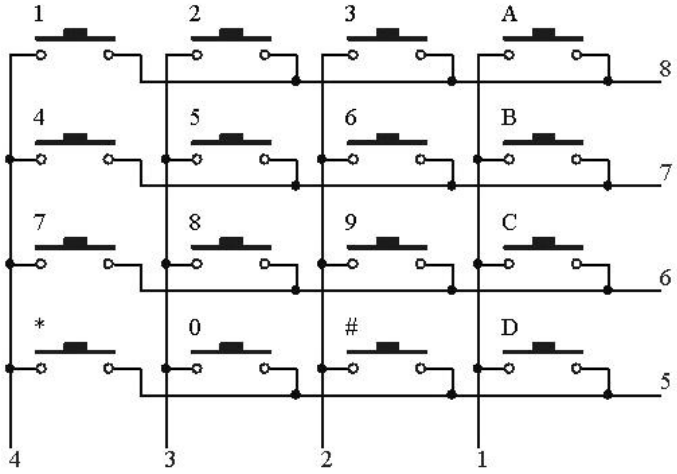
4x4 Matrix Keypad

A Keypad Matrix is a device that integrates a number of keys in one package. As is shown below, a 4x4 Keypad Matrix integrates 16 keys:





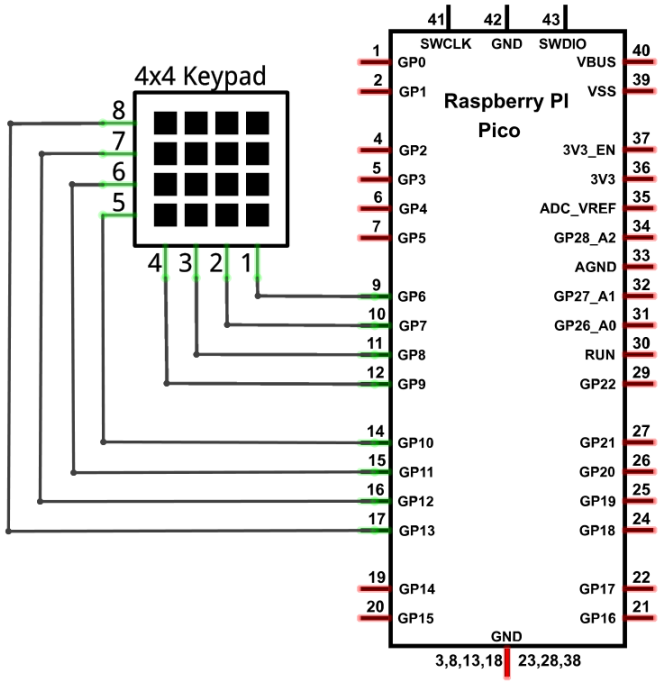
Similar to the integration of an LED Matrix, the 4x4 Keypad Matrix has each row of keys connected with one pin and this is the same for the columns. Such efficient connections reduce the number of processor ports required. The internal circuit of the Keypad Matrix is shown below.



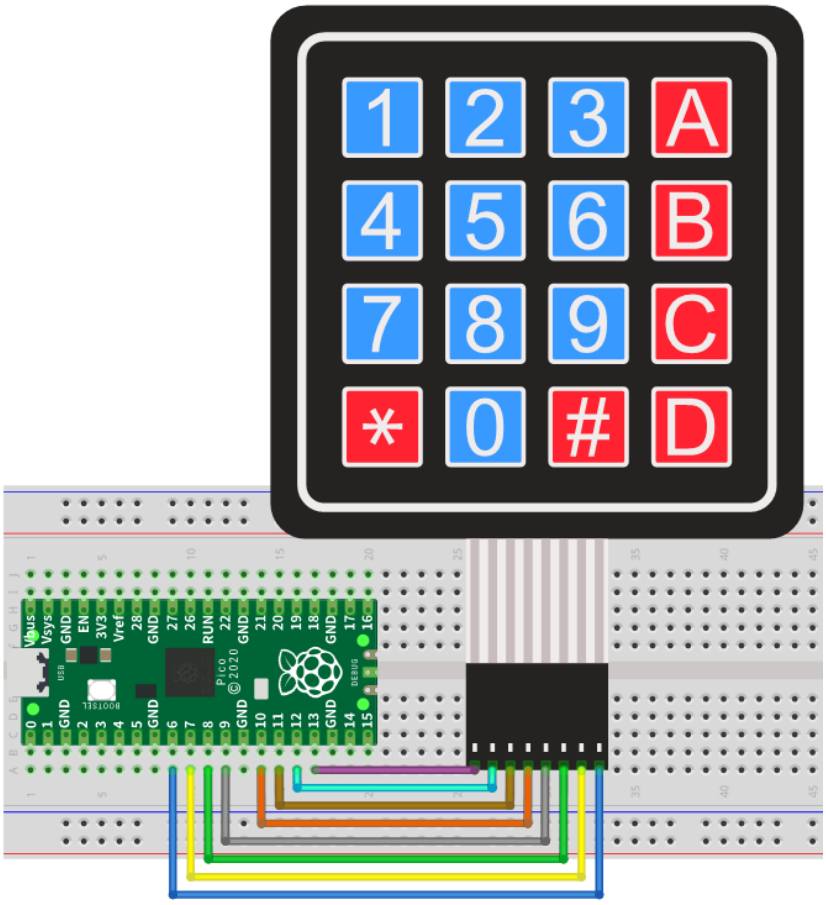
The method of usage is similar to the Matrix LED, by using a row or column scanning method to detect the state of each key's position by column and row. Take column scanning method as an example, send low level to the first 1 column (Pin1), detect level state of row 5, 6, 7, 8 to determine whether the key A, B, C, D are pressed. Then send low level to column 2, 3, 4 in turn to detect whether other keys are pressed. By this means, you can get the state of all of the keys.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

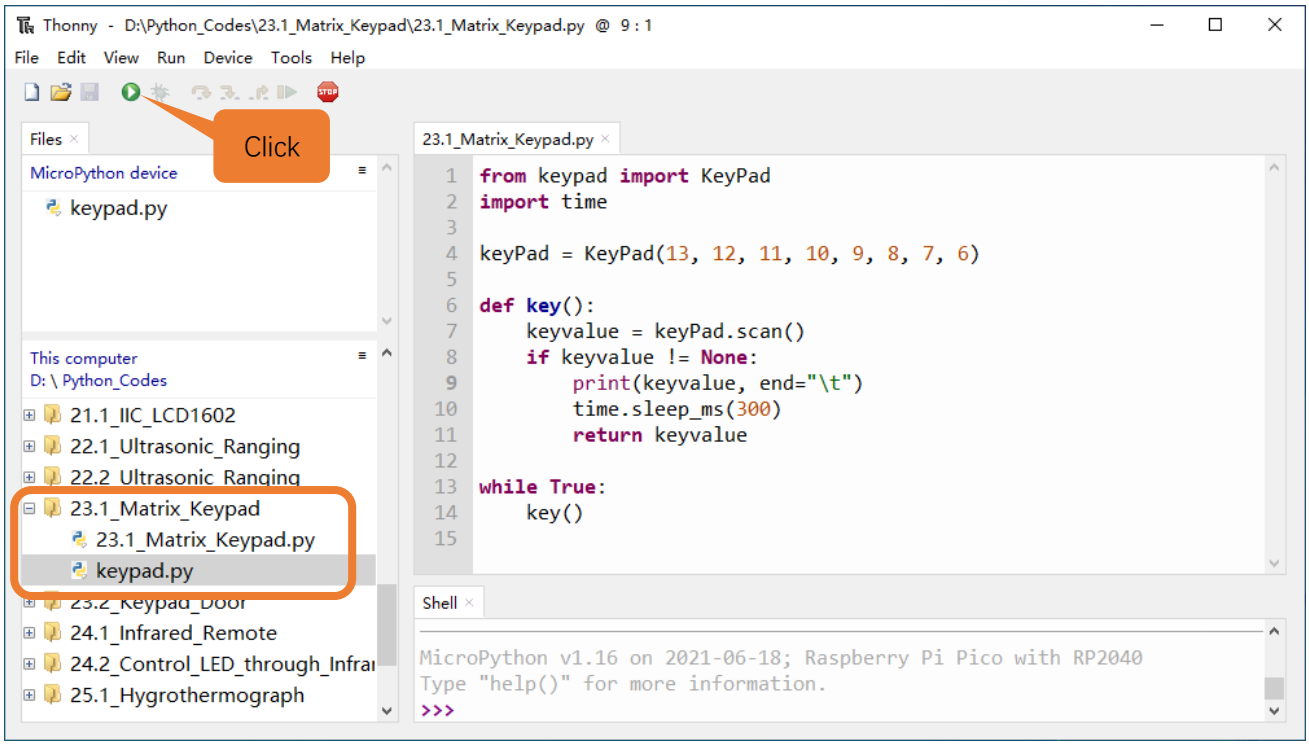


Code

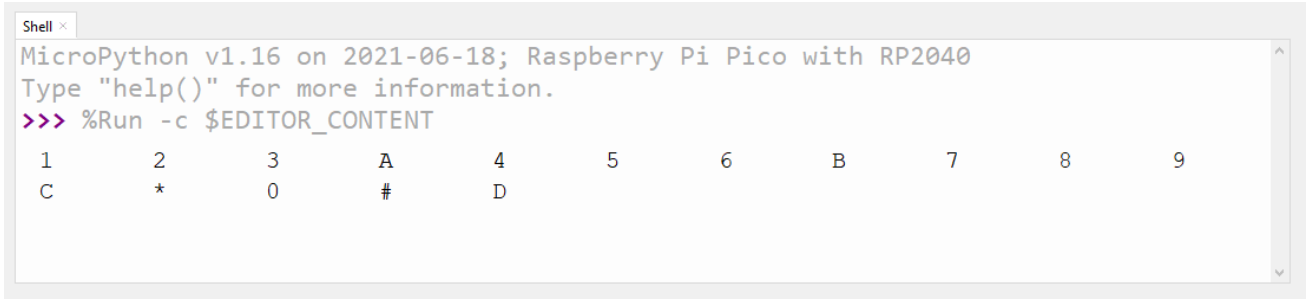
This code is used to obtain all key codes of the 4x4 Matrix Keypad, when one of the keys is pressed, the key code will be printed out via serial port.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "23.1_Matrix_Keypad". Select "keypad.py", right click your mouse to select "Upload to /", wait for "keypad.py" to be uploaded to Raspberry Pi Pico and then double click "23.1_Matrix_Keypad.py".

23.1_Matrix_Keypad



Click "Run current script", push the key board and the key value will be printed in "Shell", as shown in the illustration below. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



The following is the program code:

```
from keypad import KeyPad
import time

keyPad = KeyPad(13, 12, 11, 10, 9, 8, 7, 6)
def key():
    keyvalue = keyPad.scan()
    if keyvalue != None:
        print(keyvalue, end="\t")
        time.sleep_ms(300)
        return keyvalue

while True:
    key()
```

Import keypad module.

```
from keypad import KeyPad
```

Associate the keypad module to Raspberry Pi Pico pins.

```
keyPad = KeyPad(13, 12, 11, 10, 9, 8, 7, 6)
```

Call function keypad.scan() of the keypad module. When the keypad module detects that the key is pressed, it returns the value of the pressed key; when no key is pressed, the return value is None.

```
keyPad.scan()
```

Call function keyPan.scan() to obtain the value of the pressed key. Once it is obtained, print it out.

```
def key():
    keyvalue=keyPad.scan()
    if keyvalue!= None:
        print(keyvalue, end="\t")
        time.sleep_ms(300)
        return keyvalue
```

Reference

Class keypad

Before each use of the object **KeyPad**, please make sure **keypad.py** has been uploaded to "/" of Raspberry Pi Pico and then add the statement "**from keypad import KeyPad**" to the top of python file.

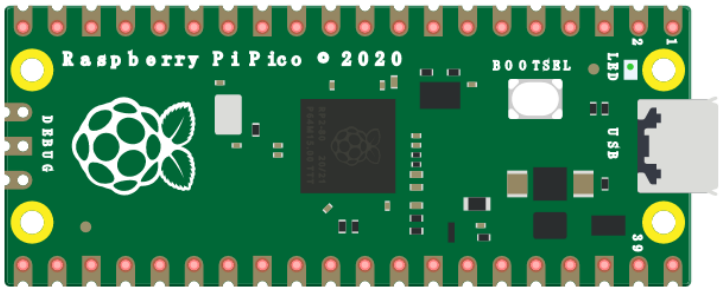
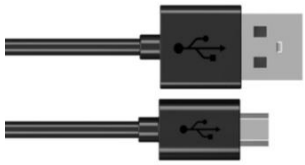
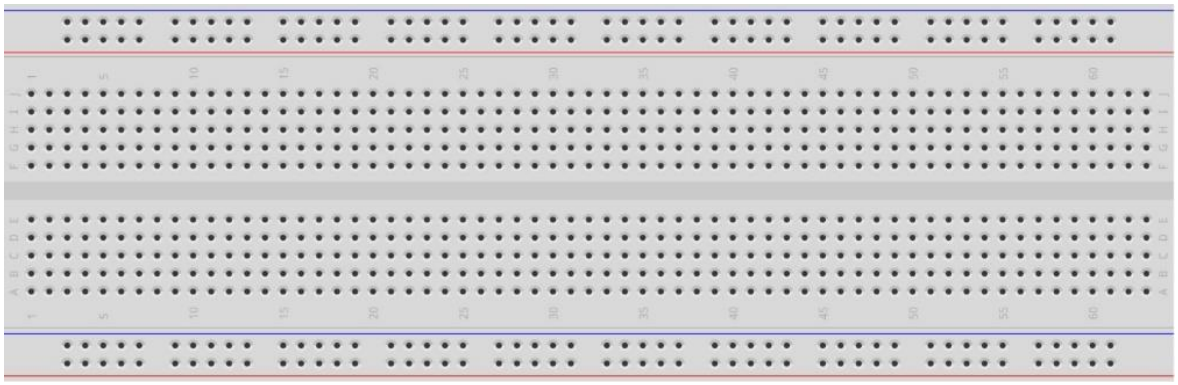
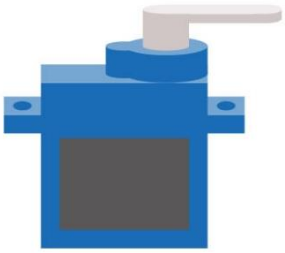
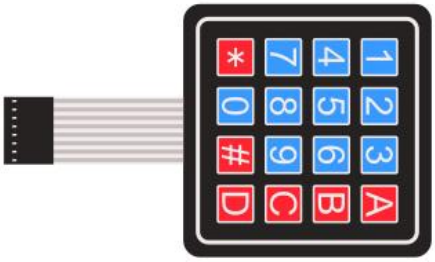
KeyPad(row1,row2,row3,row4,col1,col2,col3,col4): Initialize keypad module and associate its pins with Raspberry Pi Pico. **KeyPad() == KeyPad(13, 12, 11, 10, 9, 8, 7, 6)**.

scan(): Non-blocking keypad scan function. If no key is pressed, it returns None; Otherwise, it returns the valued of the pressed key.

Project 23.2 Keypad Door

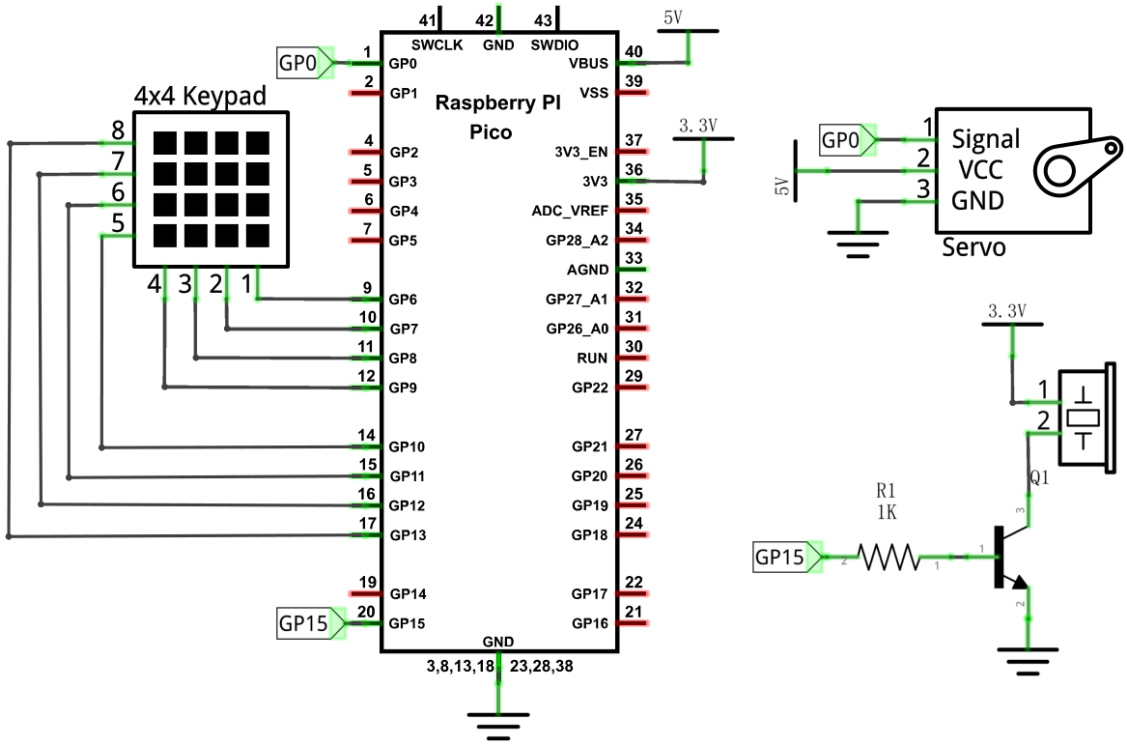
In this project, we use keypad as a keyboard to control the action of the servo motor.

Component List

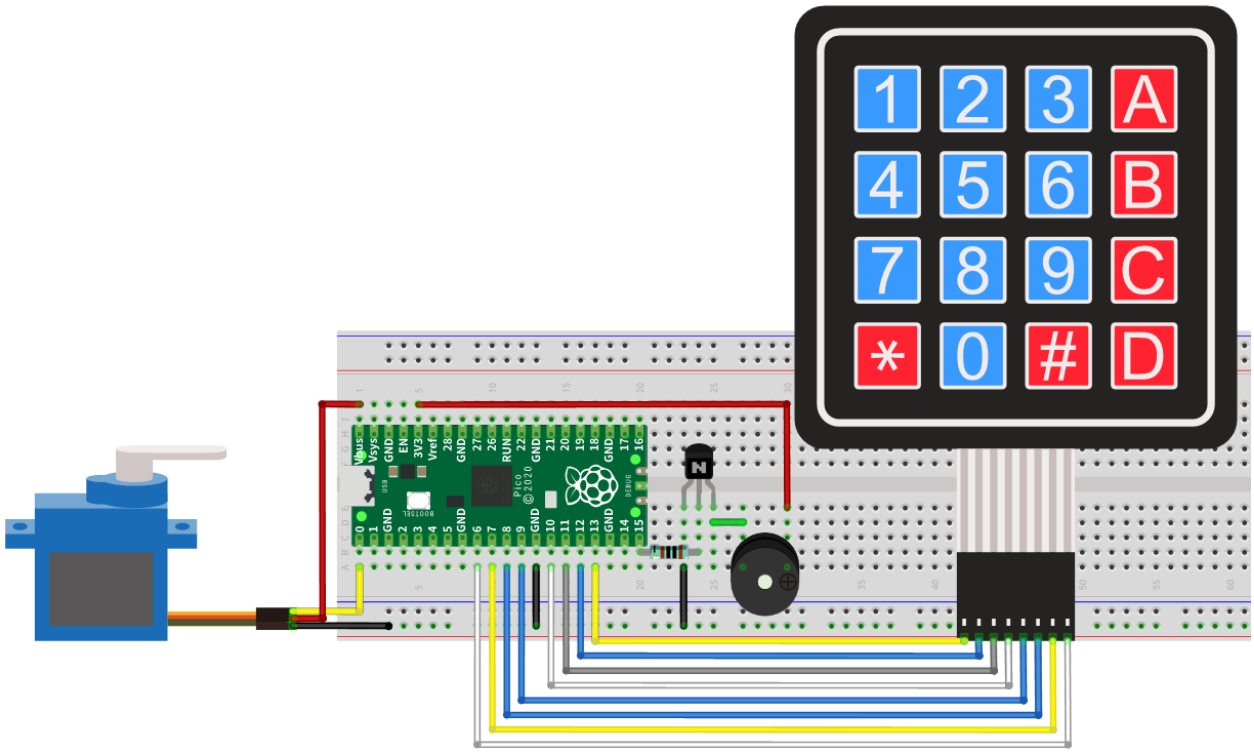
<table>
<tr><td>Raspberry Pi Pico x1
  </td><td>USB cable x1
  </td></tr>
<tr><td colspan="2">Breadboard x1
  </td></tr>
<tr><td>Jumper
  </td><td>Servo x1
  </td><td>4x4 Matrix Keypad x1
  </td></tr>
<tr><td>NPN transistorx1
(S8050)
  </td><td>Active buzzer x1
  </td><td>Resistor 1kΩ x1
  </td></tr>
</table>

Circuit

Schematic diagram



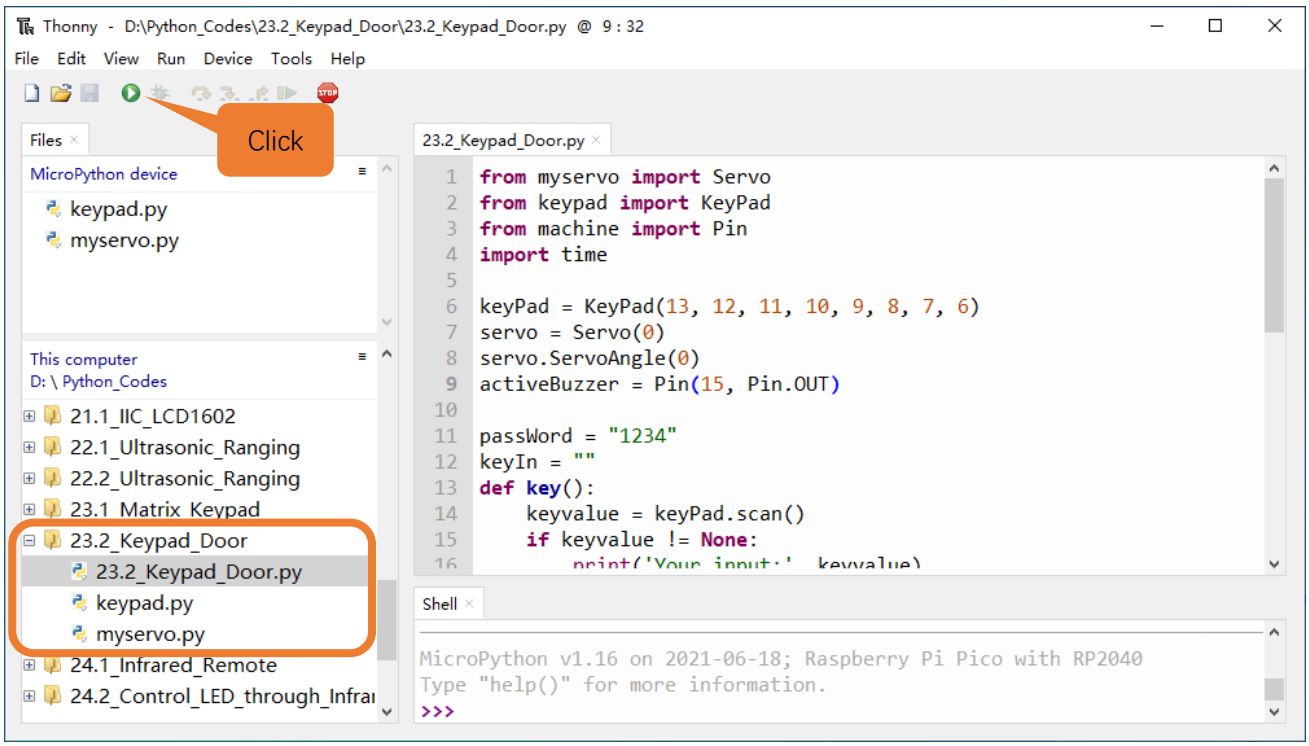
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "23.2_Keypad_Door". Select "keypad.py" and "myservo.py", right click your mouse to select "Upload to /", wait for "keypad.py" and "myservo.py" to be uploaded to Raspberry Pi Pico and then double click "23.2_Keypad_Door.py".

23.2_Keypad_Door



Click "Run current script", press the keypad to input password with 4 characters. If the input is correct, the servo will move to a certain degree, and then return to the original position. If the input is wrong, an input error will occur. Press Ctrl+C or click "Stop/Restart backend" to exit the program.

The following is the program code:

```
from myservo import Servo
from keypad import KeyPad
from machine import Pin
import time

keyPad = KeyPad(13, 12, 11, 10, 9, 8, 7, 6)
servo = Servo(0)
servo.ServoAngle(0)
activeBuzzer = Pin(15, Pin.OUT)

passWord = "1234"
keyIn = ""
def key():
    keyvalue = keyPad.scan()
    if keyvalue != None:
        print('Your input:', keyvalue)
        time.sleep_ms(200)
        return keyvalue

while True:
    keydata = key()
    if keydata != None:
        activeBuzzer.value(1)
        time.sleep_ms(100)
        activeBuzzer.value(0)
        keyIn += keydata

    if len(keyIn) == 4:
        if keyIn == passWord:
            print("passWord right!")
            servo.ServoAngle(90)
            time.sleep_ms(1000)
            servo.ServoAngle(0)
        else:
            print("passWord error!")
            activeBuzzer.value(1)
            time.sleep_ms(1000)
            activeBuzzer.value(0)
        keyIn = ""
```

Set the passWord data to a string and assign the value to the passWord variable.

```
11  passWord = "1234"
```

The buzzer makes a brief sound each time a key is pressed, and the key value is recorded in the keyIn string.

```
21      keydata = key()
22      if keydata != None:
23          activeBuzzer.value(1)
24          time.sleep_ms(100)
25          activeBuzzer.value(0)
26          keyIn += keydata
```

When pressed 4 times, keyIn records 4 key values and compares them with passWord. If they are the same, the motor rotates 90 degrees, and returns to its original position after one second. If they are inconsistent, the buzzer will sound a warning and clear the contents of the keyIn string and wait for the next input.

```
28      if len(keyIn) == 4:
29          if keyIn == passWord:
30              print("passWord right!")
31              servo.ServoAngle(90)
32              time.sleep_ms(1000)
33              servo.ServoAngle(0)
34          else:
35              print("passWord error!")
36              activeBuzzer.value(1)
37              time.sleep_ms(1000)
38              activeBuzzer.value(0)
39          keyIn = ""
```

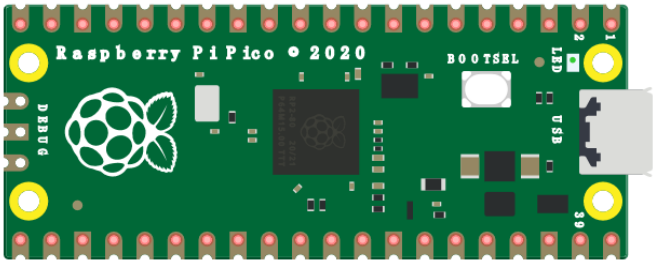
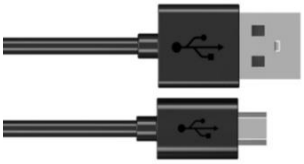
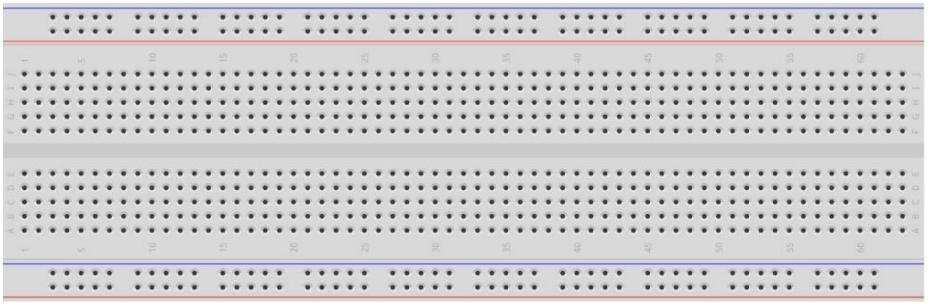
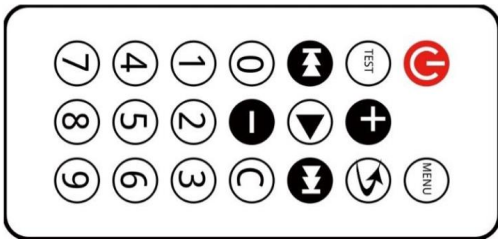
Chapter 24 Infrared Remote

In this chapter, we will learn how to use an infrared remote control, and control an LED.

Project 24.1 Infrared Remote Control

First, we need to understand how infrared remote control works, then get the command sent from infrared remote control.

Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Jumper 	Infrared Remote x1 (May need CR2025 battery x1, please check the holder) 
Infrared Remote x1 	Resistor 10kΩ x1 

Component knowledge

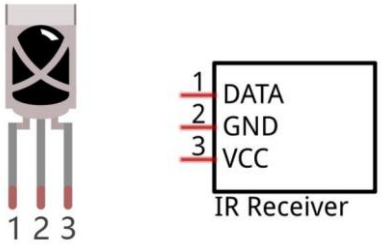
Infrared Remote

An infrared (IR) remote control is a device with a certain number of buttons. Pressing down different buttons will make the infrared emission tube, which is located in the front of the remote control, send infrared ray with different command. Infrared remote control technology is widely used in electronic products such as TV, air conditioning, etc. Thus making it possible for you to switch TV programs and adjust the temperature of the air conditioning when away from them. The remote control we use is shown below:



Infrared receiver

An infrared (IR) receiver is a component that can receive the infrared light, so we can use it to detect the signal emitted by the infrared remote control. DATA pin here outputs the received infrared signal.



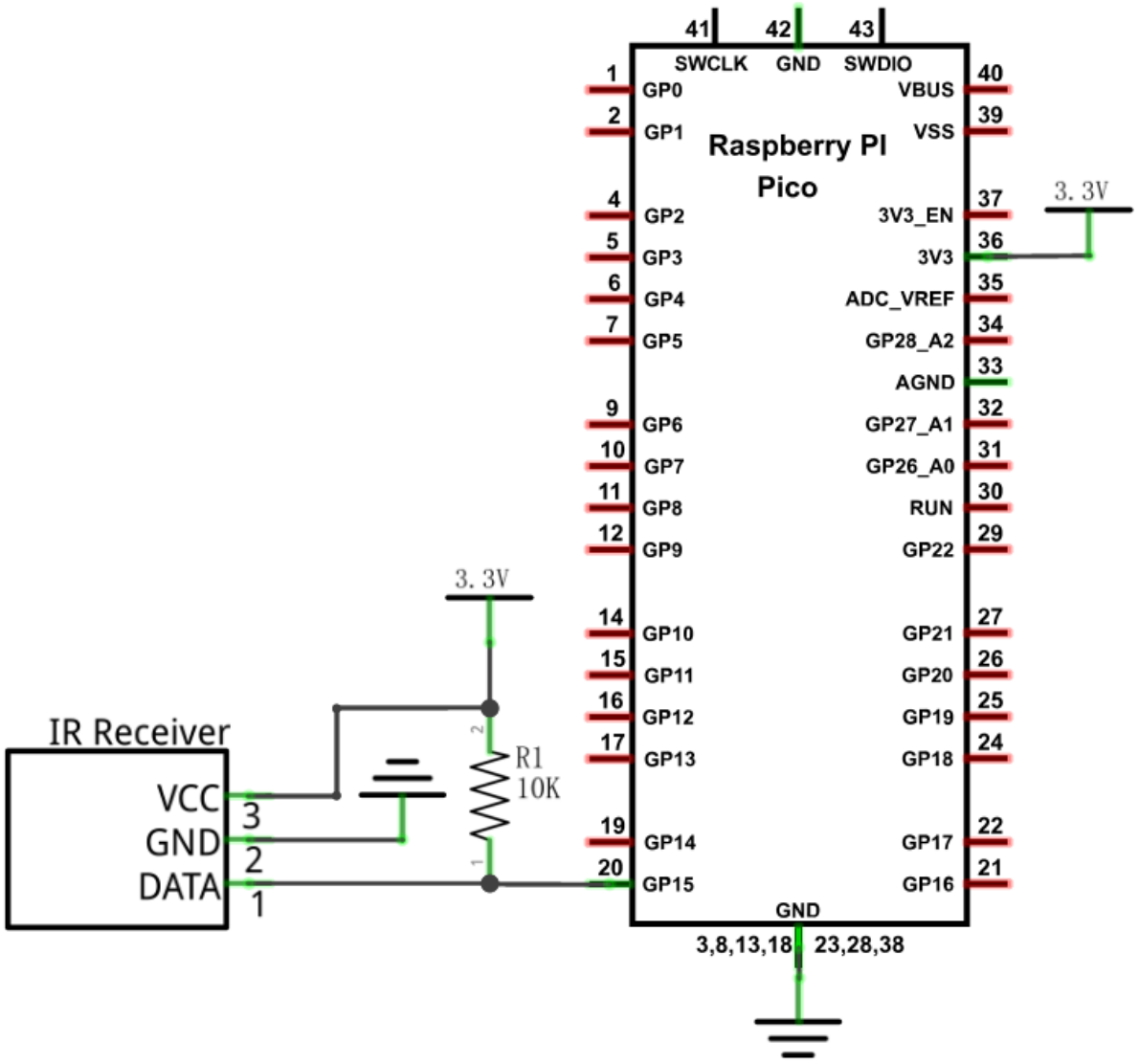
When you use the infrared remote control, the infrared remote control sends a key value to the receiving circuit according to the pressed key. We can program the Raspberry Pi Pico to do things like lighting, when a key value is received.

The following is the key value that the receiving circuit will receive when each key of the infrared remote control is pressed.

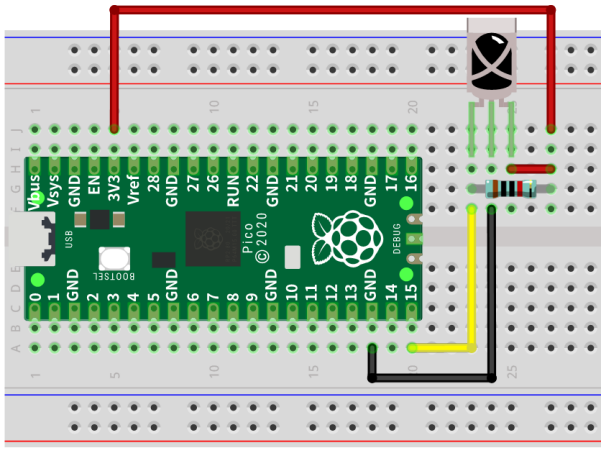
ICON	KEY Value	ICON	KEY Value
	FFA25D		FFB04F
	FFE21D		FF30CF
	FF22DD		FF18E7
	FF02FD		FF7A85
	FFC23D		FF10EF
	FFE01F		FF38C7
	FFA857		FF5AA5
	FF906F		FF42BD
	FF6897		FF4AB5
	FF9867		FF52AD

Circuit

Schematic diagram



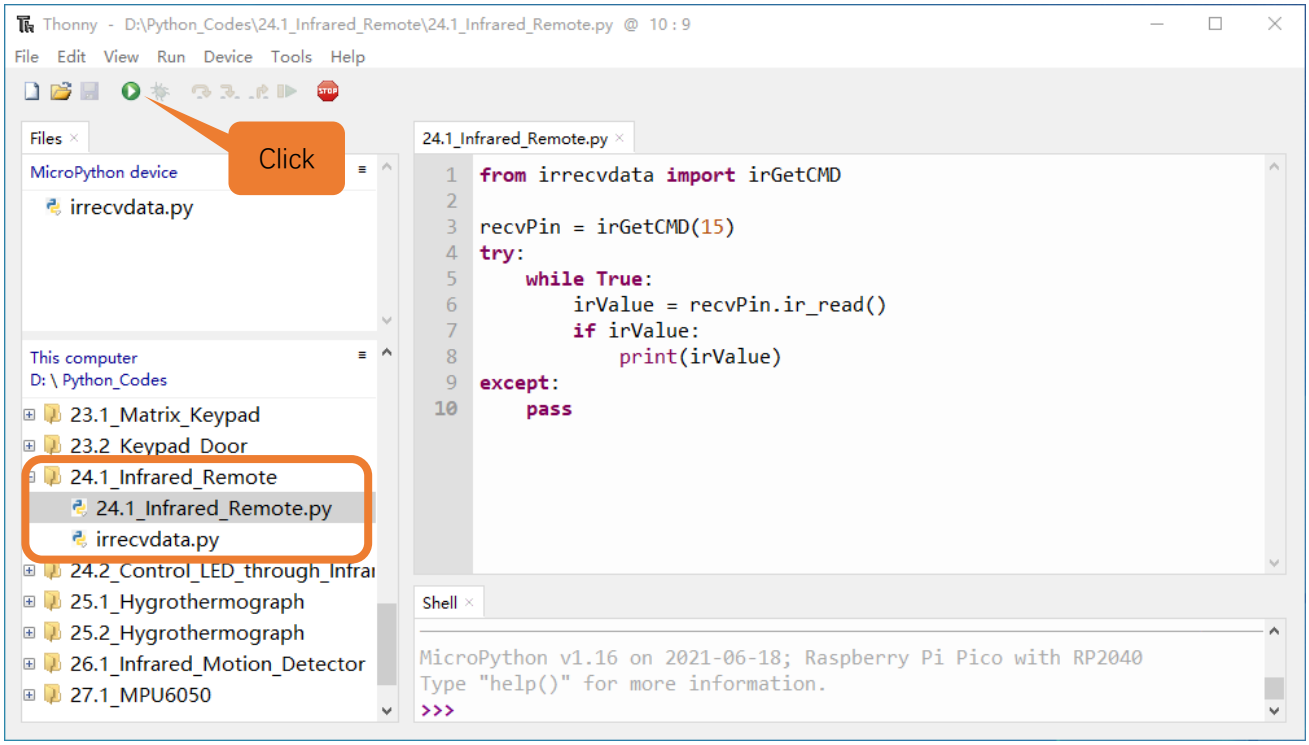
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "24.1_Infrared_Remote". Select "irrecvdata.py", right click your mouse to select "Upload to /", wait for "irrecvdata.py" to be uploaded to Raspberry Pi Pico and then double click "24.1_Infrared_Remote.py".

24.1_Infrared_Remote



Click "Run current script". Press any key of the infrared remote and the key value will be printed in "Shell", as shown in the illustration below. Press Ctrl+C or click "Stop/Restart backend" to exit the program.

```
MicroPython v1.16 on 2021-07-20; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

  0xff6897
  0xff18e7
  0xff7a85
  0xff30cf
  0xff10ef
  0xff38c7
  0xff5aa5
  0xff4ab5
  0xff42bd
  0xff4ab5
  0xff52ad
```

The following is the program code:

```
1  from irrecvdata import irGetCMD
2
3  recvPin = irGetCMD(15)
4  try:
5      while True:
6          irValue = recvPin.ir_read()
7          if irValue:
8              print(irValue)
9  except:
10     pass
```

Import the infrared decoder.

```
1  from irrecvdata import irGetCMD
```

Associate the infrared decoder with GP15.

```
6  recvPin = irGetCMD(15)
```

Call ir_read() to read the value of the pressed key and assign it to IRValue.

```
12 irValue = recvPin.ir_read()
```

When infrared key value is obtained, print it out in" Shell".

```
5      while True:
6          irValue = recvPin.ir_read()
7          if irValue:
8              print(irValue)
```

Reference

Class irrecvdata

Before each use of the object **irrecvdata**, please add the statement "**from irrecvdata import irGetCMD**" to the top of the python file.

irGetCMD(): Object of infrared encoder, which is associated with GP15 by default.

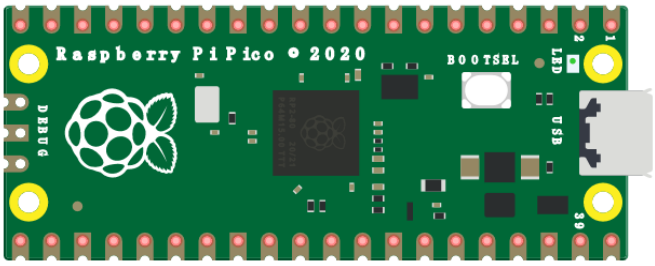
ir_read(): The function that reads the key value of infrared remote. When the value is read, it will be returned; when no value is obtained, character **None** will be returned.

Project 24.2 Control LED through Infrared Remote

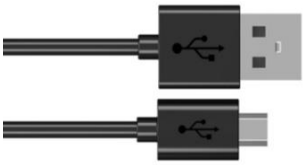
In this project, we will control the brightness of LED lights through an infrared remote control.

Component List

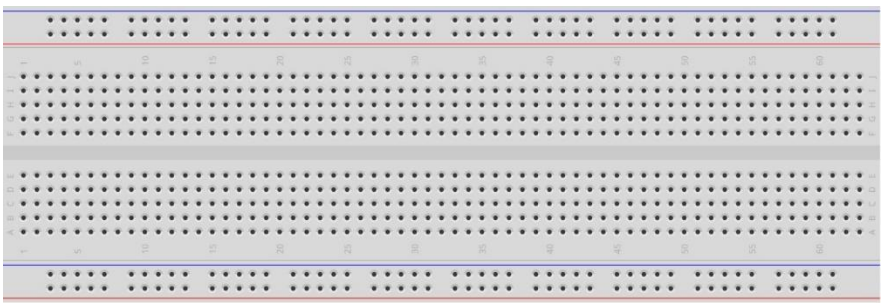
Raspberry Pi Pico x1



USB cable x1



Breadboard x1



Jumper



Infrared Remote x1
(May need CR2025 battery x1, please check the battery holder)



LED x1



Active buzzer x1



Resistor 1kΩ x2



Infrared receiver x1



NPN transistorx1
(S8050)

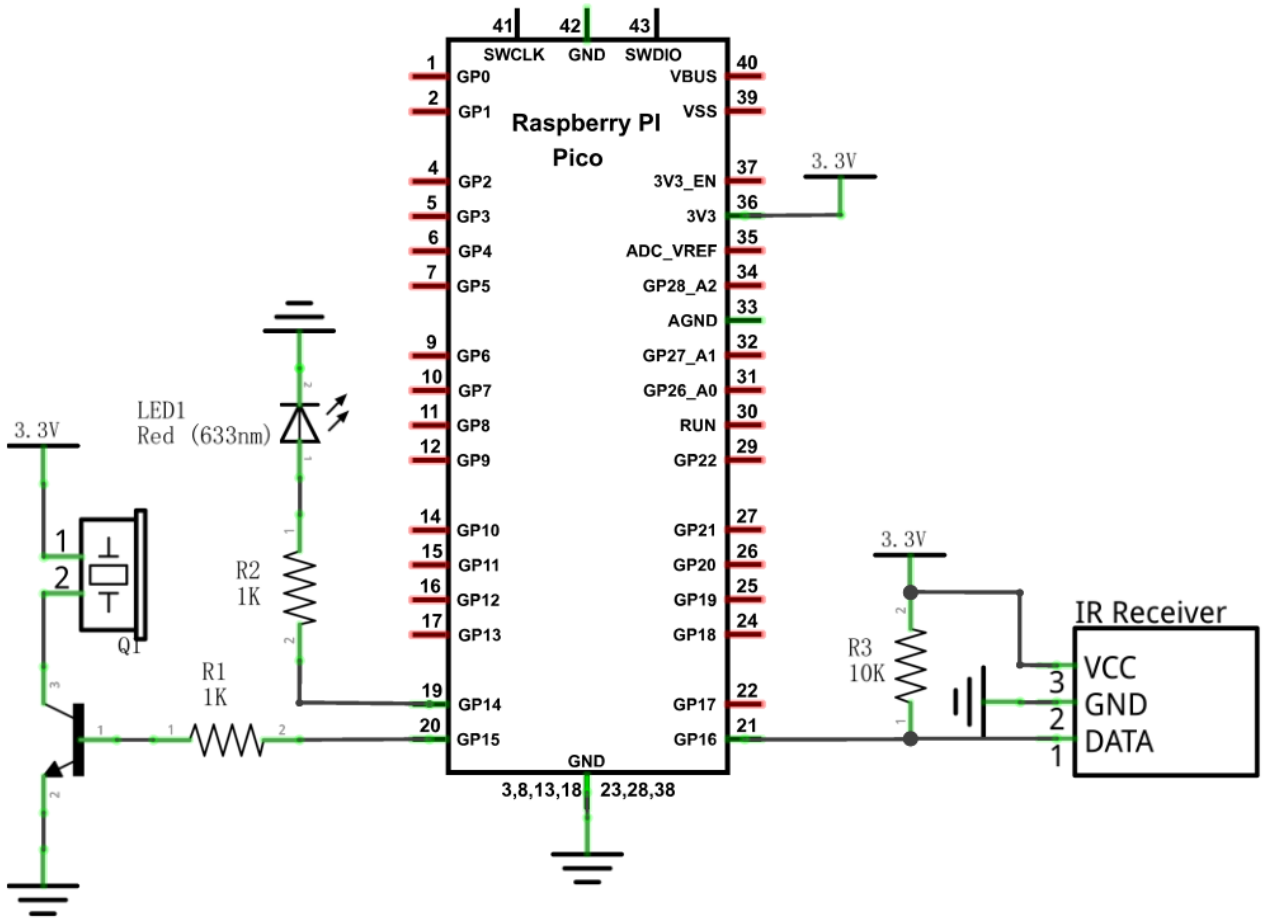


Resistor 10kΩ x1

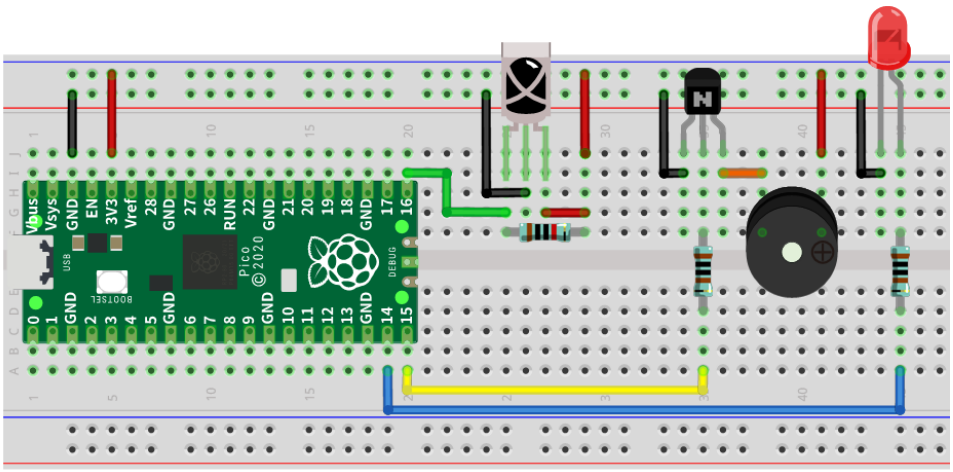


Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

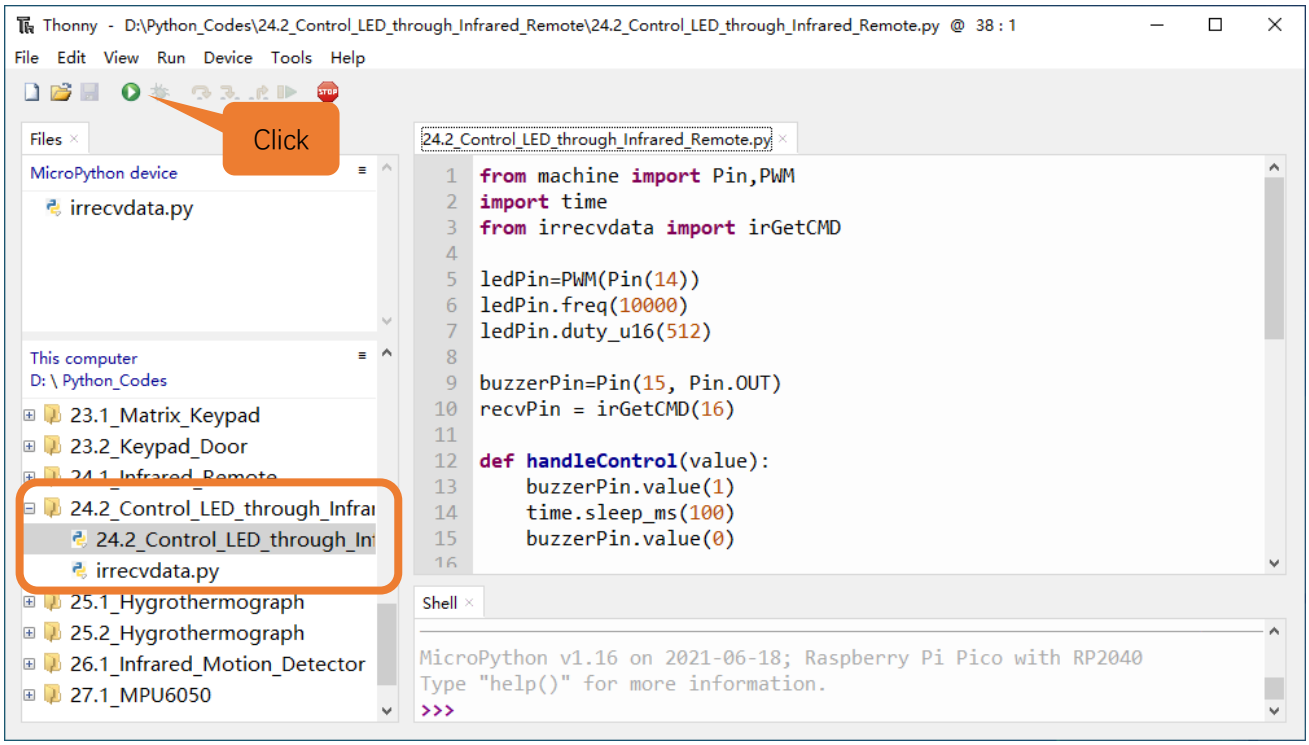


Code

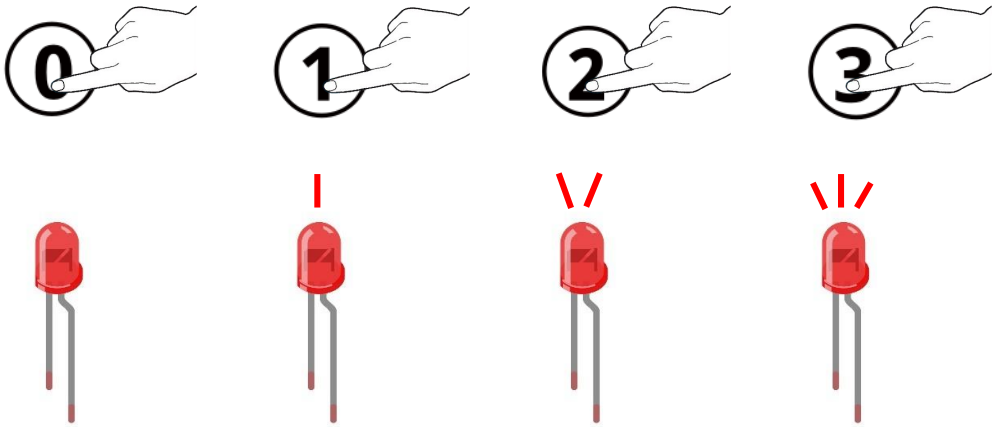
The Code controls the brightness of the LED by determining the key value of the infrared received.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "24.2_Control_LED_through_Infrared_Remote". Select "irrecvdata.py", right click your mouse to select "Upload to /", wait for "irrecvdata.py" to be uploaded to Raspberry Pi Pico and then double click "24.2_Control_LED_through_Infrared_Remote.py".

24.2_Control_LED_through_Infrared_Remote



Click “Run current script”. When pressing "0", "1", "2", "3" of the infrared remote control, the buzzer will sound once, and the brightness of the LED light will change correspondingly. Press Ctrl+C or click “Stop/Restart backend” to exit the program.



The following is the program code:

```
from machine import Pin,PWM
import time
from irrecvdata import irGetCMD

ledPin=PWM(Pin(14))
ledPin.freq(10000)
ledPin.duty_u16(512)
buzzerPin=Pin(15, Pin.OUT)
recvPin = irGetCMD(16)

def handleControl(value):
    buzzerPin.value(1)
    time.sleep_ms(100)
    buzzerPin.value(0)
    if value == '0xff6897':    #0
        ledPin.duty_u16(1)
    elif value == '0xff30cf': #1
        ledPin.duty_u16(1023)
    elif value == '0xff18e7': #2
        ledPin.duty_u16(4096)
    elif value == '0xff7a85': #3
        ledPin.duty_u16(10000)
    else:
        return

try:
    while True:
```

```
28          irValue = recvPin.ir_read()
29          if irValue:
30              print(irValue)
31              handleControl(irValue)
32  except:
33      ledPin.deinit()
```

The handleControl() function is used to execute events corresponding to infrared code values. Every time when the function is called, the buzzer sounds once and determines the brightness of the LED based on the infrared key value. If the key value is not "0", "1", "2", "3", the buzzer sounds once, but the brightness of LED will not change.

```
11  def handleControl(value):
12      buzzerPin.value(1)
13      time.sleep_ms(100)
14      buzzerPin.value(0)
15      if value == '0xff6897':    #0
16          ledPin.duty_u16(1)
17      elif value == '0xff30cf': #1
18          ledPin.duty_u16(1023)
19      elif value == '0xff18e7': #2
20          ledPin.duty_u16(4096)
21      elif value == '0xff7a85': #3
22          ledPin.duty_u16(10000)
23      else:
24          return
```

Each time the key value of IR remote is received, function handleControl() will be called to process it.

```
27      while True:
28          irValue = recvPin.ir_read()
29          if irValue:
30              print(irValue)
31              handleControl(irValue)
```

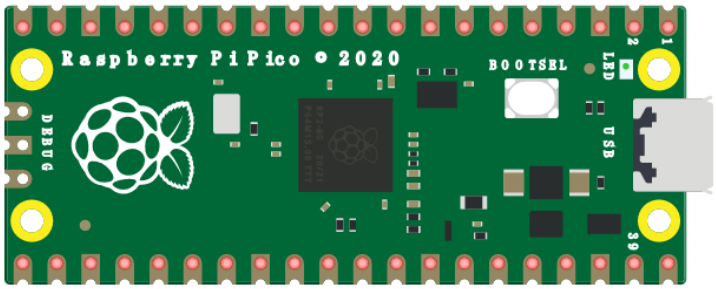
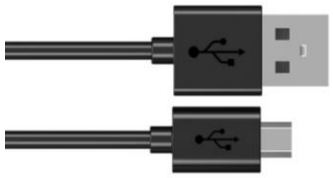
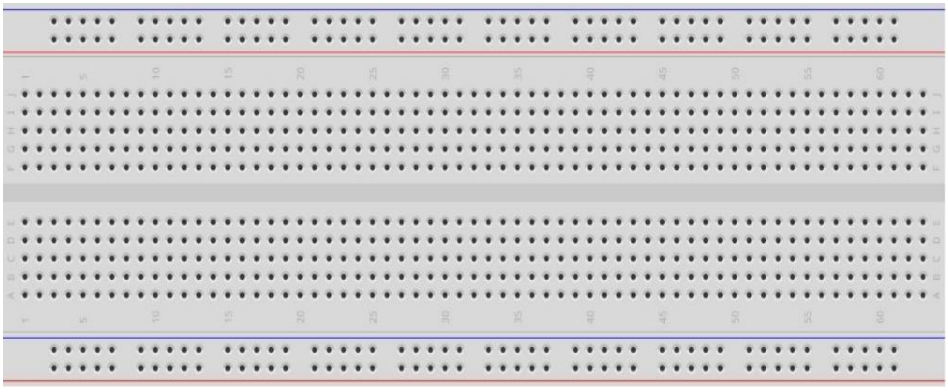
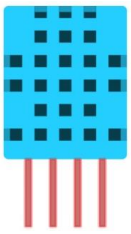
Chapter 25 Hygrothermograph DHT11

In this chapter, we will learn about a commonly used sensor called a Hygrothermograph DHT11.

Project 25.1 Hygrothermograph

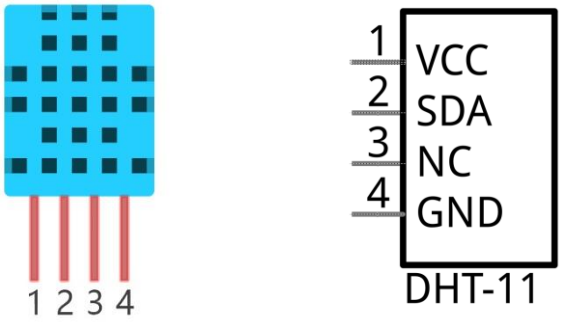
Hygrothermograph is an important tool in our lives to give us data on the temperature and humidity in our environment. In this project, we will use the Raspberry Pi Pico to read Temperature and Humidity data of the DHT11 Module.

Component List

Raspberry Pi Pico x1 	USB cable x1 	
Breadboard x1 		
Jumper 	DHT11 x1 	Resistor 10kΩ x1 

Component knowledge

The Temperature & Humidity Sensor DHT11 is a compound temperature & humidity sensor, and the output digital signal has been calibrated by its manufacturer.



DHT11 uses customized single-line communication protocol, so we can use the library to read data more conveniently.

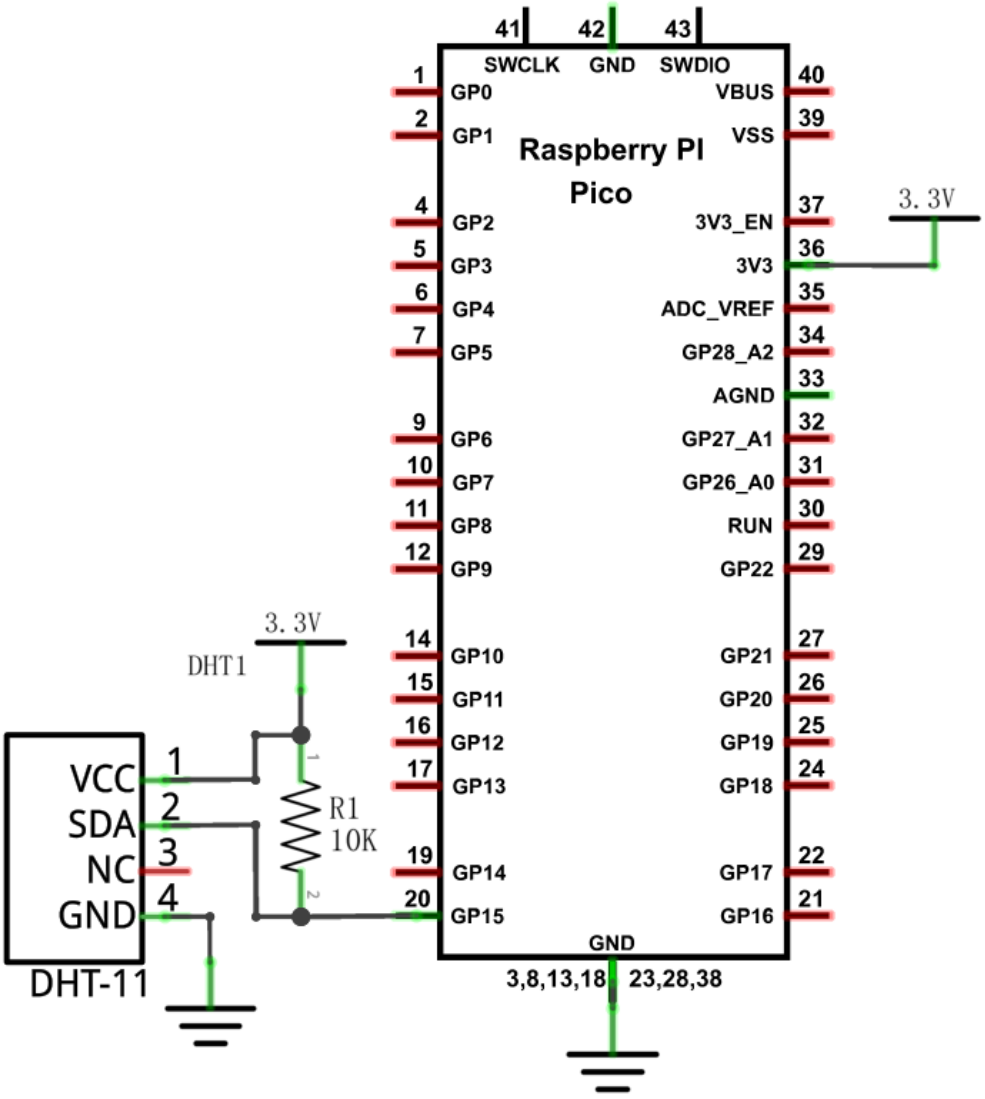
After being powered up, it will initialize in 1S's time. Its operating voltage is within the range of 3.3V-5.5V.

The SDA pin is a data pin, which is used to communicate with other devices.

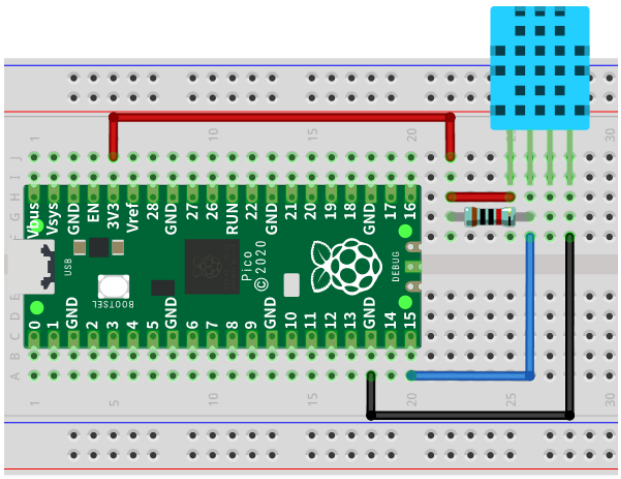
The NC pin (Not Connected Pin) is a type of pin found on various integrated circuit packages. Those pins have no functional purpose to the outside circuit (but may have an unknown functionality during manufacture and test). Those pins should not be connected to any of the circuit connections.

Circuit

Schematic diagram



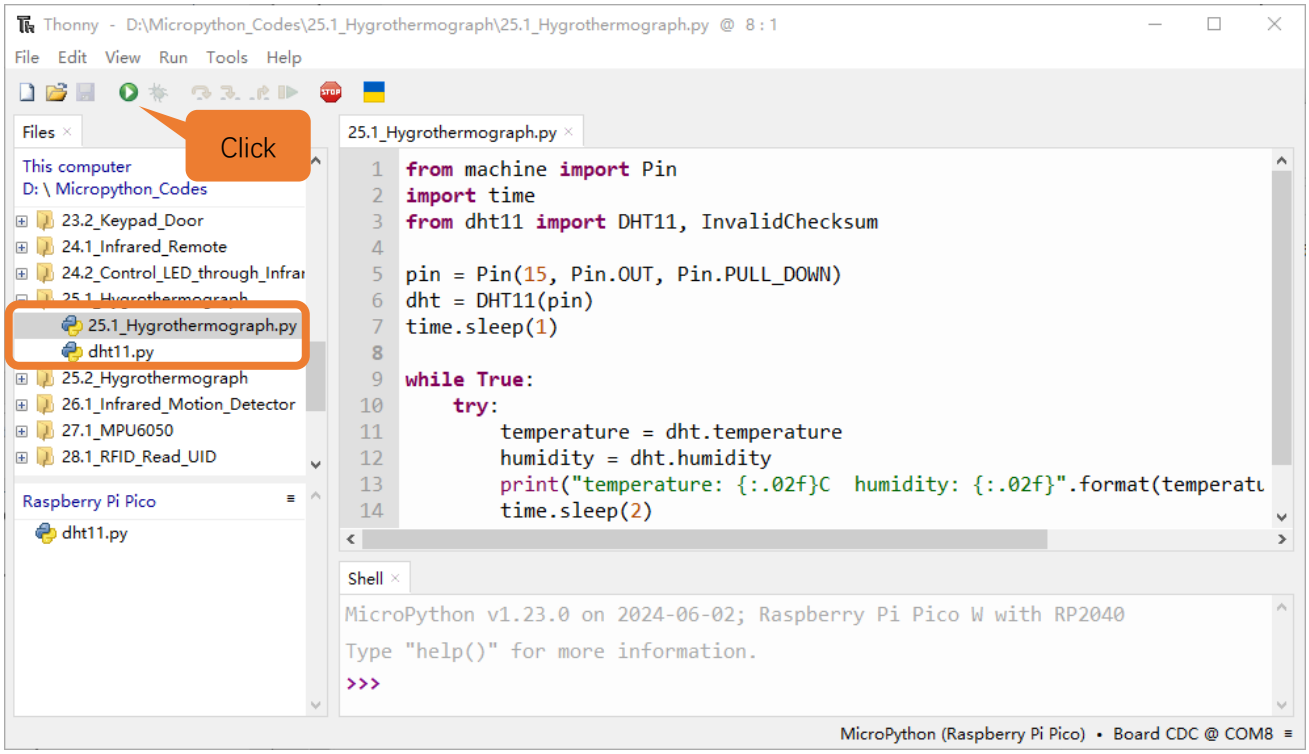
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "25.1_Hygrothermograph". Select "dht11.py", right click your mouse to select "Upload to /", wait for "dht11.py" to be uploaded to Raspberry Pi Pico and then double click "25.1_Hygrothermograph.py".

25.1_Hygrothermograph



Make sure your circuit is correctly connected and you will see the following messages printed in "Shell". Press Ctrl+C or click "Stop/Restart backend" to exit the program.

```
temperature: 26.02C  humidity: 46.00%
temperature: 26.02C  humidity: 46.00%
temperature: 26.02C  humidity: 46.00%
temperature: 26.05C  humidity: 46.00%
temperature: 26.06C  humidity: 46.00%
temperature: 26.06C  humidity: 46.00%
temperature: 26.07C  humidity: 46.00%
temperature: 26.07C  humidity: 46.00%
temperature: 26.07C  humidity: 46.00%
temperature: 26.07C  humidity: 46.00%
temperature: 26.07C  humidity: 46.00%
temperature: 26.07C  humidity: 46.00%
```

The following is the program code:

```
from machine import Pin
import time
from dht11 import DHT11, InvalidChecksum

pin = Pin(5, Pin.OUT, Pin.PULL_DOWN)
dht = DHT11(pin)
time.sleep(1)

try:
    while True:
        temperature = dht.temperature
        humidity = dht.humidity
        print("temperature: {:.02f}℃  humidity: {:.02f}".format(temperature, humidity) + "%")
        time.sleep(2)
except InvalidChecksum:
    print("DHT11 data error!")
```

Import time and dht11 modules.

```
from machine import Pin
import time
from dht11 import DHT11, InvalidChecksum
```

Create a DHT11 object and bind GP15 as the data transfer pin.

```
pin = Pin(15, Pin.OUT, Pin.PULL_DOWN)
dht = DHT11(pin)
```

Call the temperature function and humidity function to read the temperature and humidity data of DHT module and print the data to “shell”.

```
        temperature = dht.temperature
        humidity = dht.humidity
        print("temperature: {:.02f}℃  humidity: {:.02f}".format(temperature, humidity) + "%")
```

Obtain temperature and humidity data once per second and print them out.

```
    while True:
        temperature = dht.temperature
        humidity = dht.humidity
        print("temperature: {:.02f}℃  humidity: {:.02f}".format(temperature, humidity) + "%")
        time.sleep(2)
```

Reference

Class dht11

Before each use of object **dht11**, please add the statement “**from dht11 import DHT11, InvalidChecksum**” to the top of python file.

DHT11(pin): Object of DHT11, pin_name parameter is to set the data output pin connecting to DHT11.

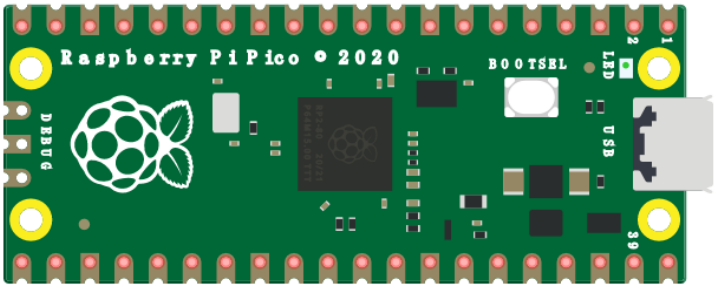
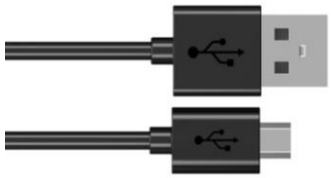
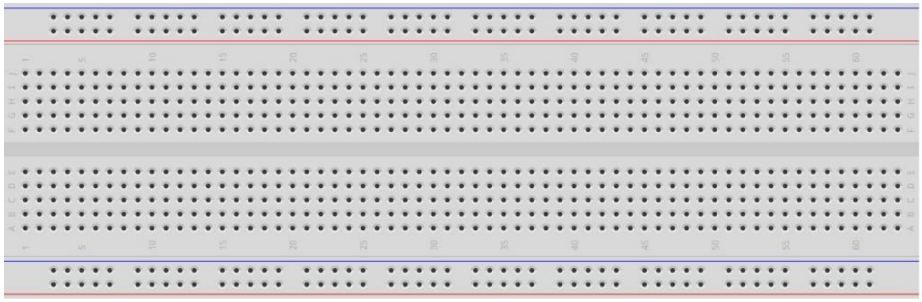
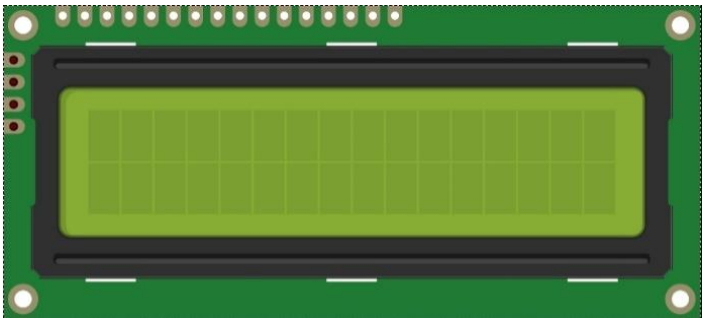
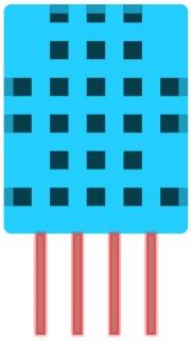
DHT11.temperature: Returns the temperature data measured by DHT11.

DHT11.humidity: Returns the humidity data measured by DHT11.

Project 25.2 Hygrothermograph

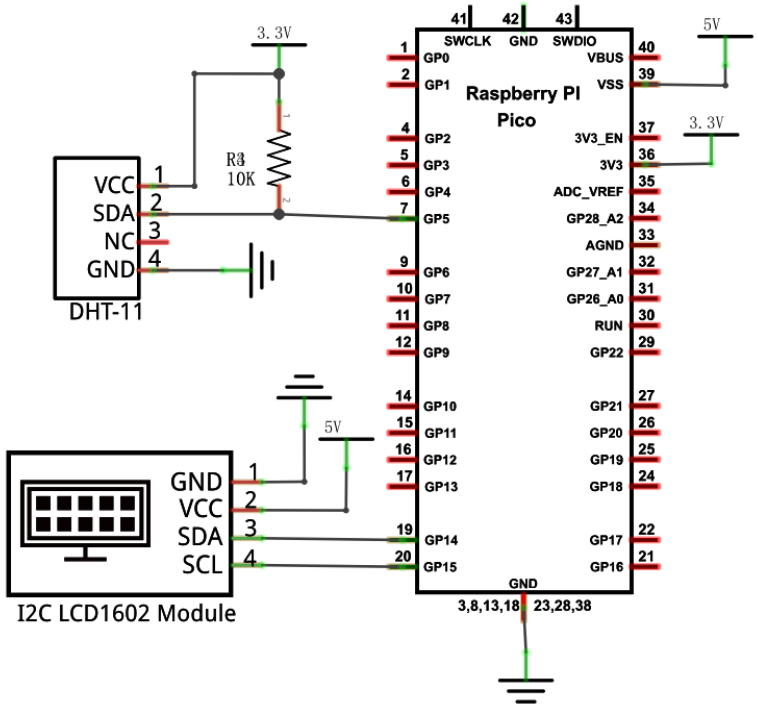
In this project, we use I2C-LCD1602 to display data collected by DHT11.

Component List

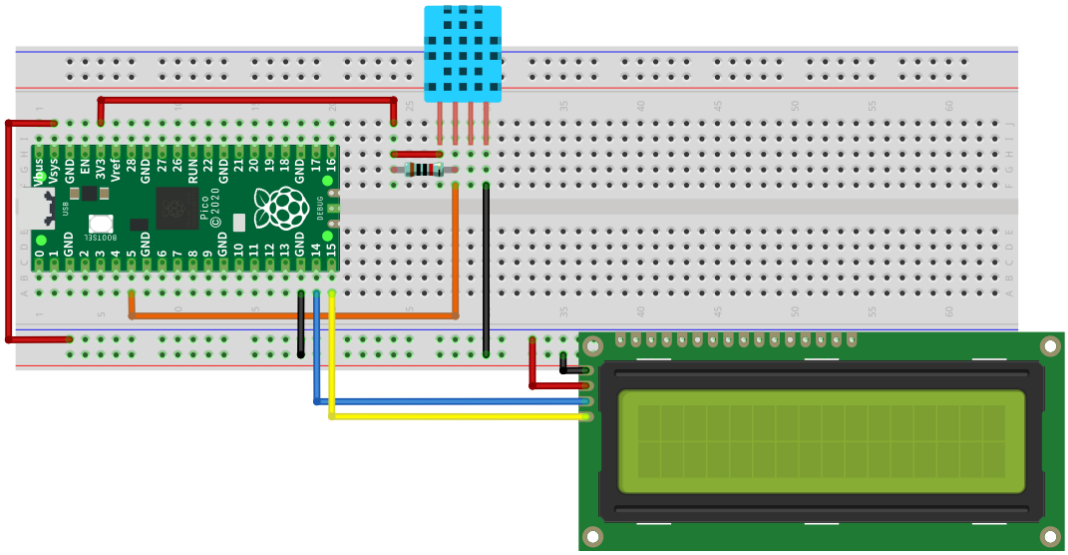
Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
LCD1602 Module x1 	Resistor 10kΩ x1 
Jumper 	DHT11 x1 

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

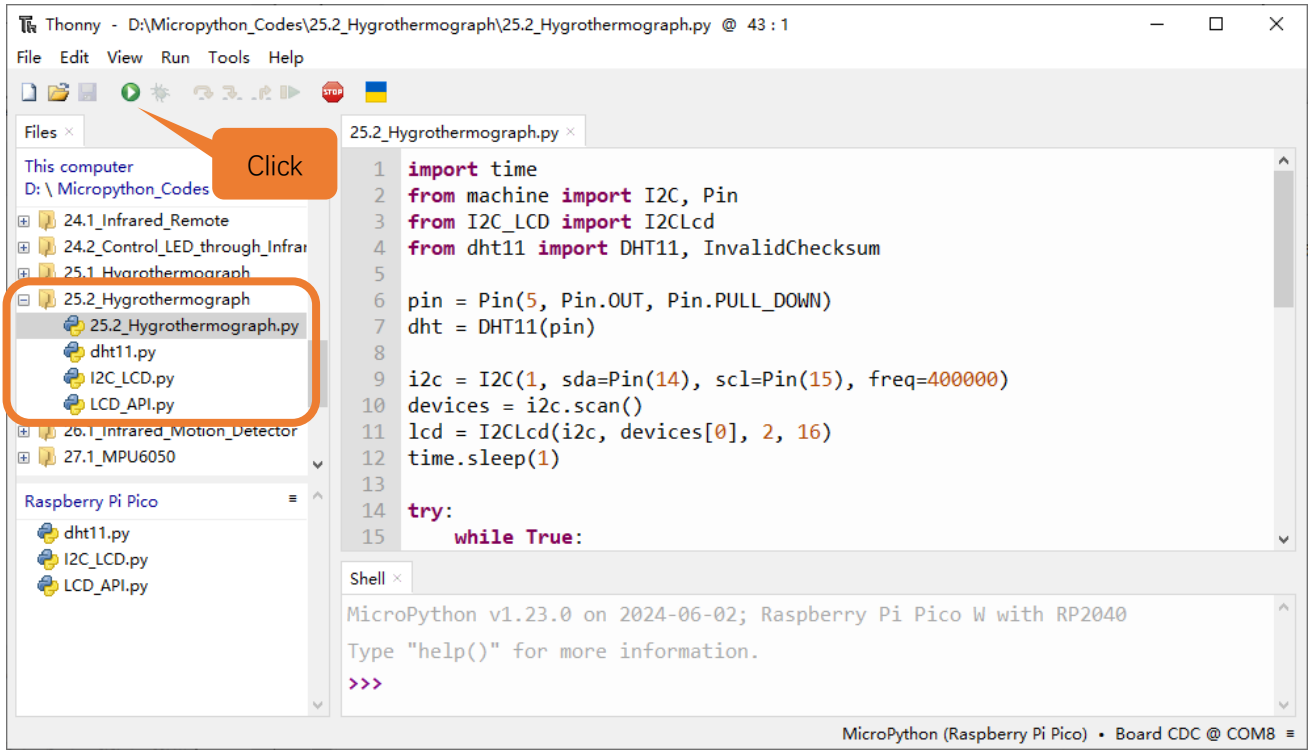


Code

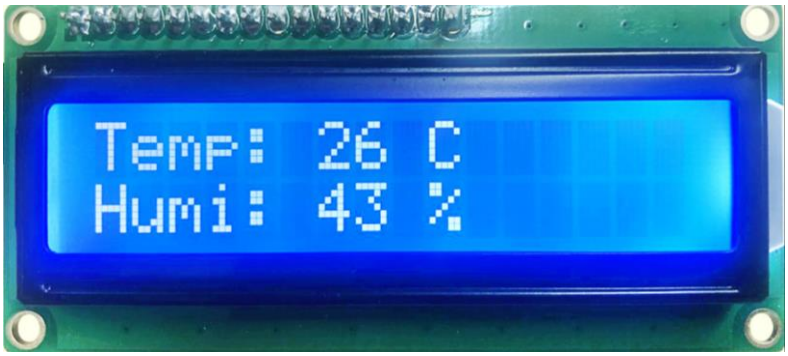
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → 25.2_Hygrothermograph". Select "dht11.py"、"I2C_LCD.py" and "LCD_API.py", right click your mouse to select "Upload to /", wait for "dht11.py"、"I2C_LCD.py" and "LCD_API.py" to be uploaded to Raspberry Pi Pico and then double click "25.2_Hygrothermograph.py".

Any concerns? ✉ support@freenove.com

25.2_Hygrothermograph



Click "Run current script". The first row of LCD1602 is temperature value and the second row is humidity. Try to "pinch" the DHT11 (without touching the leads) with your index finger and thumb for a brief time, you should see that the displayed value on LCD1602 changes. Press Ctrl+C or click "Stop/Restart backend" to exit the program.



The following is the program code:

```
import time
from machine import I2C, Pin
from I2C_LCD import I2CLcd
from dht11 import DHT11, InvalidChecksum

pin = Pin(5, Pin.OUT, Pin.PULL_DOWN)
dht = DHT11(pin)

i2c = I2C(1, sda=Pin(14), scl=Pin(15), freq=400000)
devices = i2c.scan()
lcd = I2CLcd(i2c, devices[0], 2, 16)
time.sleep(1)

try:
    while True:
        temp = int(dht.temperature)
        humi = int(dht.humidity)
        lcd.move_to(0, 0)
        lcd.putstr("Temp: ")
        lcd.putstr(str(temp))
        lcd.putstr(" C")
        lcd.move_to(0, 1)
        lcd.putstr("Humi: ")
        lcd.putstr(str(humi))
        lcd.putstr(" %")
        time.sleep(2)
except InvalidChecksum:
    print("DHT11 data error!")
```

Import dht11 and I2C LCD1602 modules.

```
Import time
from machine import I2C, Pin
from I2C_LCD import I2CLcd
from dht11 import DHT11, InvalidChecksum
```

Assign GP5 to DHT11, GP14 and GP15 to LCD1602.

```
pin = Pin(5, Pin.OUT, Pin.PULL_DOWN)
dht = DHT11(pin)

i2c = I2C(1, sda=Pin(14), scl=Pin(15), freq=400000)
devices = i2c.scan()
lcd = I2CLcd(i2c, devices[0], 2, 16)
```

Obtain data of Hygrothermograph every second and display them on LCD1602. The first line displays temperature and the second line displays humidity.

```
try:
    while True:
        temp = int(dht.temperature)
        humi = int(dht.humidity)
        lcd.move_to(0, 0)
        lcd.putstr("Temp: ")
        lcd.putstr(str(temp))
        lcd.putstr(" C")
        lcd.move_to(0, 1)
        lcd.putstr("Humi: ")
        lcd.putstr(str(humi))
        lcd.putstr(" %")
        time.sleep(2)
except InvalidChecksum:
    print("DHT11 data error!")
```

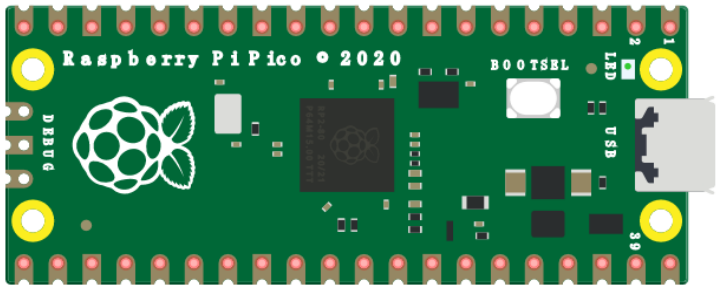
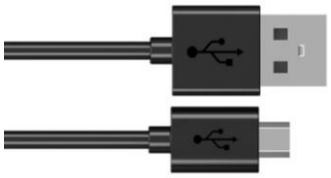
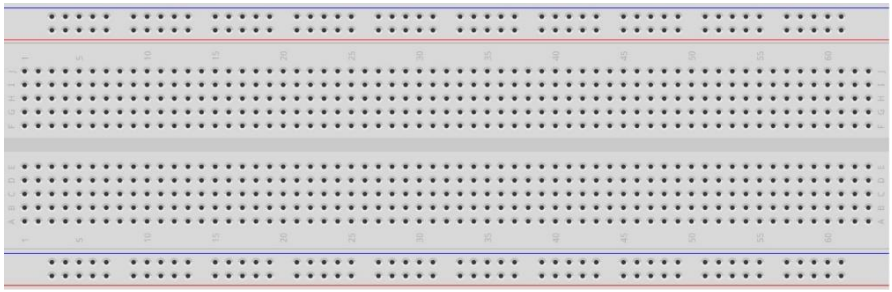
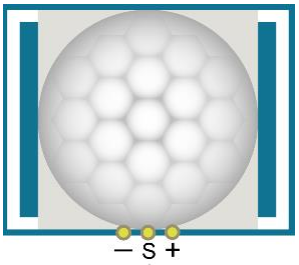
Chapter 26 Infrared Motion Sensor

In this chapter, we will learn a widely used sensor, Infrared Motion Sensor.

Project 26.1 Infrared Motion Detector with LED Indicator

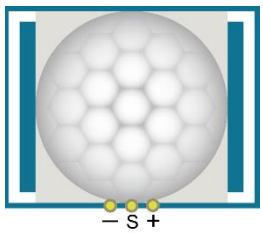
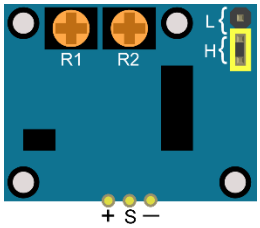
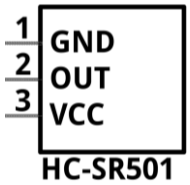
In this project, we will make a Motion Detector, with the human body infrared pyroelectric sensors. When someone is in close proximity to the Motion Detector, it will automatically light up and when there is no one close by, it will be out.
This Infrared Motion Sensor can detect the infrared spectrum (heat signatures) emitted by living humans and animals.

Component List

<table>
<tr><td colspan="2">Raspberry Pi Pico x1
  </td><td colspan="2">USB cable x1
  </td></tr>
<tr><td colspan="4">Breadboard x1
  </td></tr>
<tr><td>HC SR501 x1
  </td><td>LED x1
  </td><td>Resistor 220Ω x1
  </td><td>Jumper
  </td></tr>
</table>

Component knowledge

The following is the diagram of the infrared Motion sensor（HC SR-501）:

Top	Bottom	Schematic
		

Description:

Working voltage: 5v-20v (DC)　Static current: 65uA.

Automatic Trigger: When a living body enters into the active area of sensor, the module will output high level (3.3V). When the body leaves the sensor's active detection area, it will output high level lasting for time period T, then output low level (0V). Delay time T can be adjusted by the potentiometer R1.

According to the position of jumper cap, you can choose non-repeatable trigger mode or repeatable mode.

L: non-repeatable trigger mode. The module output high level after sensing a body, then when the delay time is over, the module will output low level. During high level time, the sensor no longer actively senses bodies.

H: repeatable trigger mode. The distinction from the L mode is that it can sense a body until that body leaves during the period of high level output. After this, it starts to time and output low level after delaying T time.

Induction block time: the induction will stay in block condition and does not induce external signal at lesser time intervals (less than delay time) after outputting high level or low level.

Initialization time: the module needs about 1 minute to initialize after being powered ON. During this period, it will alternately output high or low level.

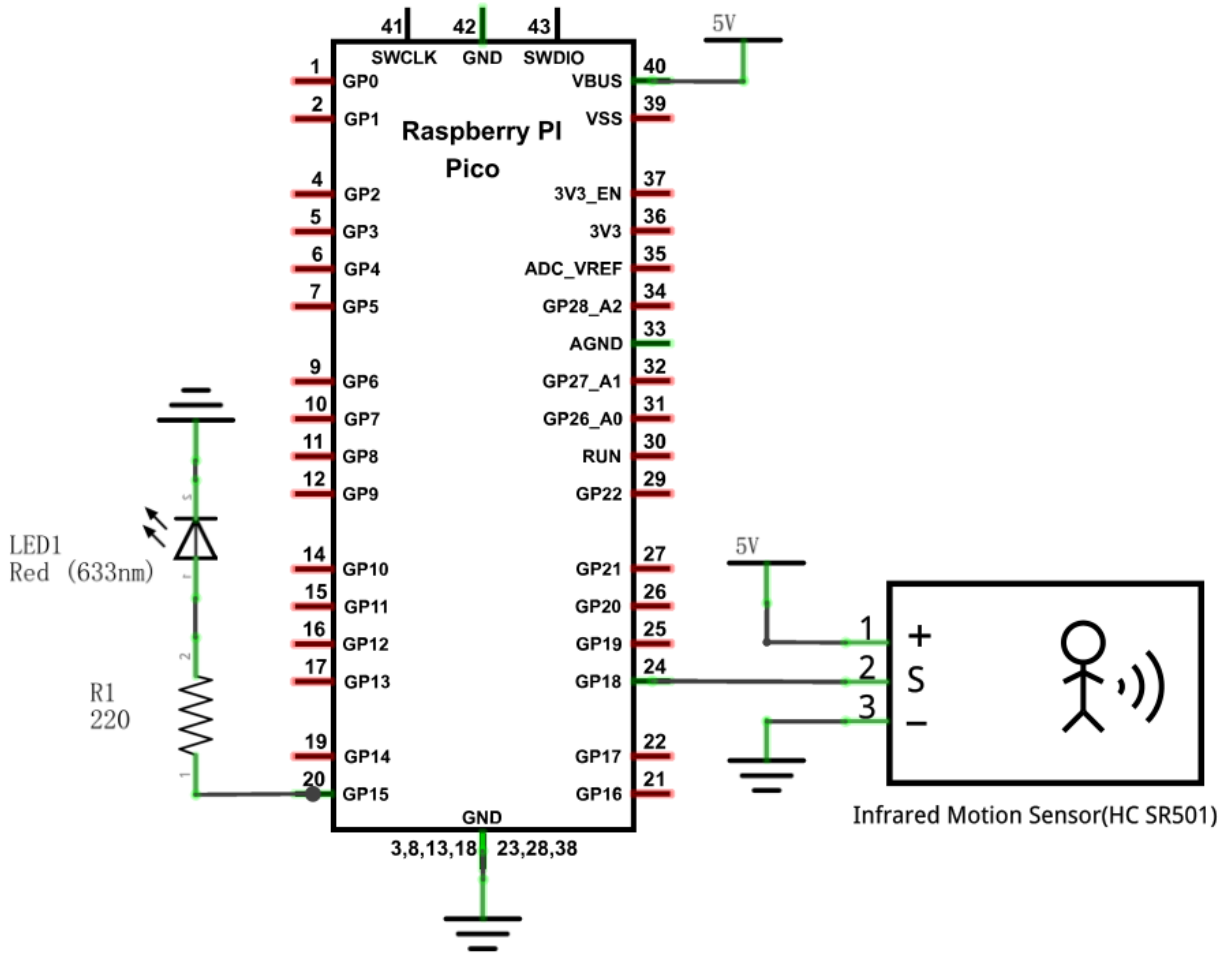
One characteristic of this sensor is when a body moves close to or moves away from the sensor's dome edge, the sensor will work at high sensitivity. When a body moves close to or moves away from the sensor's dome in a vertical direction, the sensor cannot detect well (please take note of this deficiency).

Note: The Sensing Range (distance before a body is detected) is adjusted by the potentiometer.

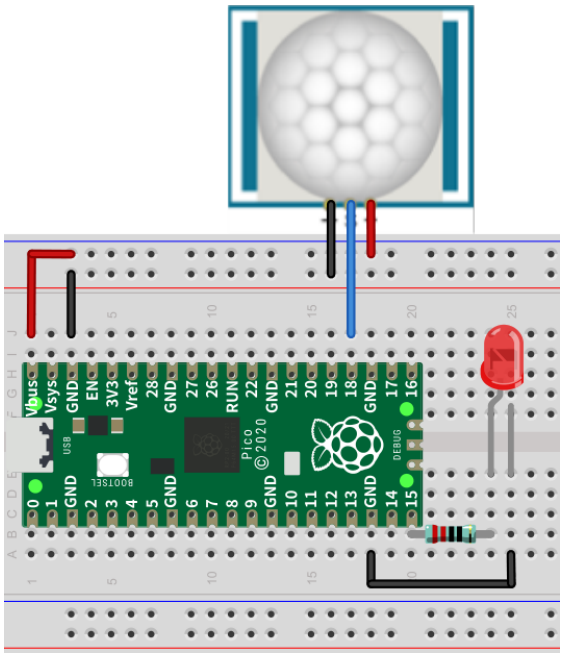
We can regard this sensor as a simple inductive switch when in use.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

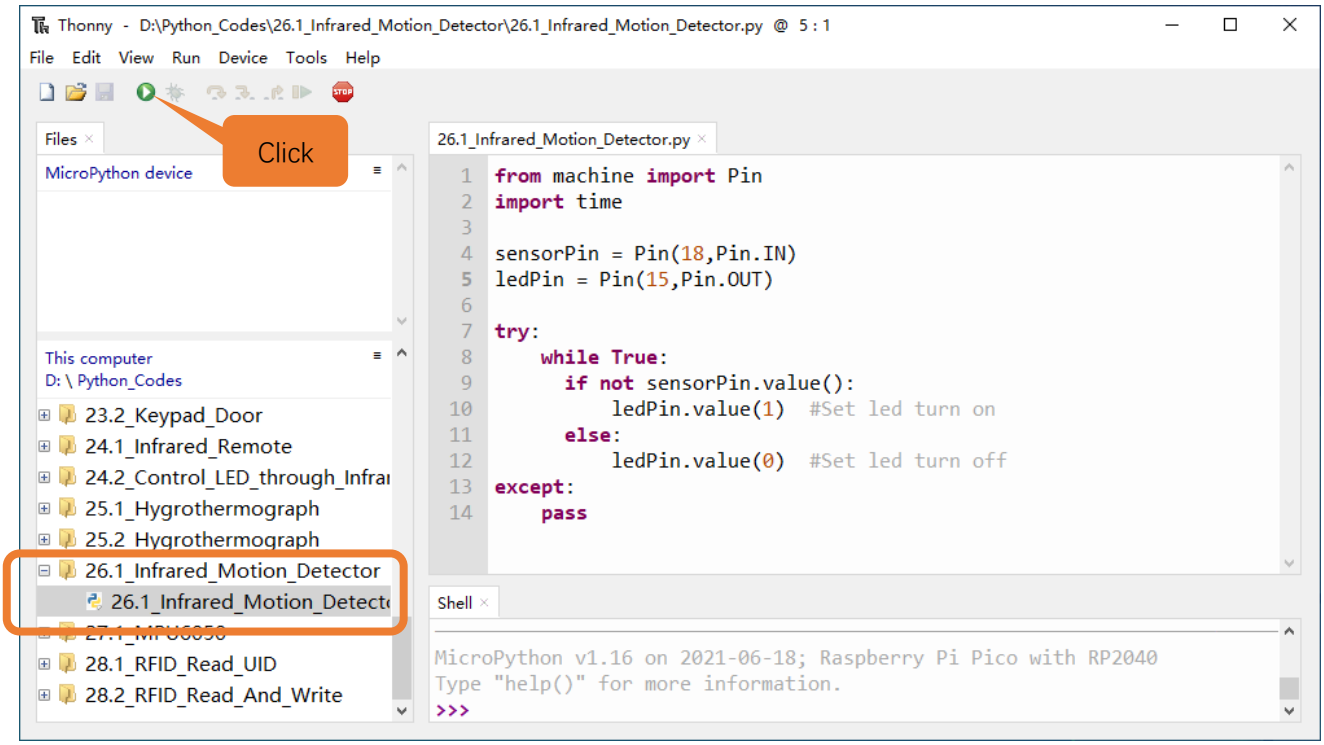


Code

In this project, we will use an infrared motion sensor to trigger an LED, essentially using the infrared motion sensor as a motion switch. Therefore, the code of this project is similar to that of project "Button & Led". The difference is when infrared motion sensor detects changes, it will out high level; when it detects nothing, it will output low level. When the sensor outputs high level, LED turns ON; Otherwise, LED turns OFF.

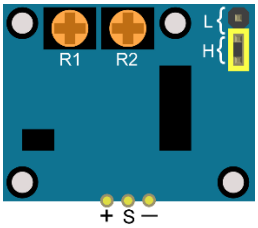
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "26.1_Infrared_Motion_Detector" and then double click"26.1_Infrared_Motion_Detector.py".

26.1_Infrared_Motion_Detector



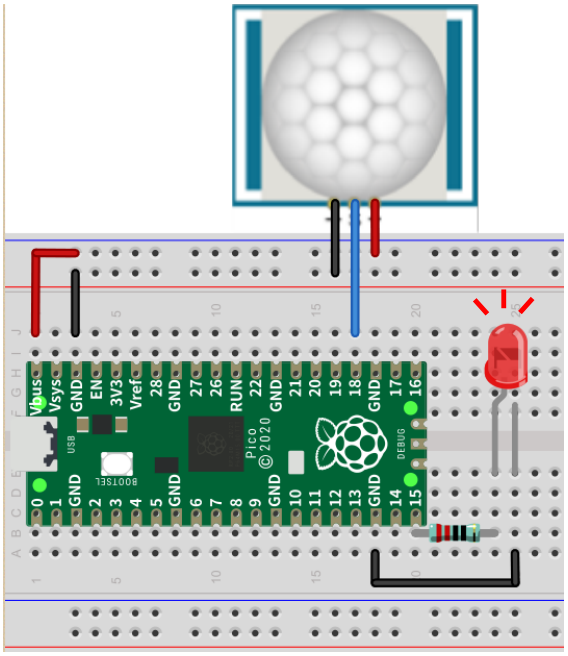
Click "Run current script". Put the sensor on a stationary table and wait for about a minute. Then try to move away from or move closer to the Infrared Motion Sensor and observe whether the LED turns ON or OFF automatically. Press Ctrl+C or click "Stop/Restart backend" to exit the program.

You can rotate the potentiometer on the sensor to adjust the detection effect, or use different modes by changing the jumper.



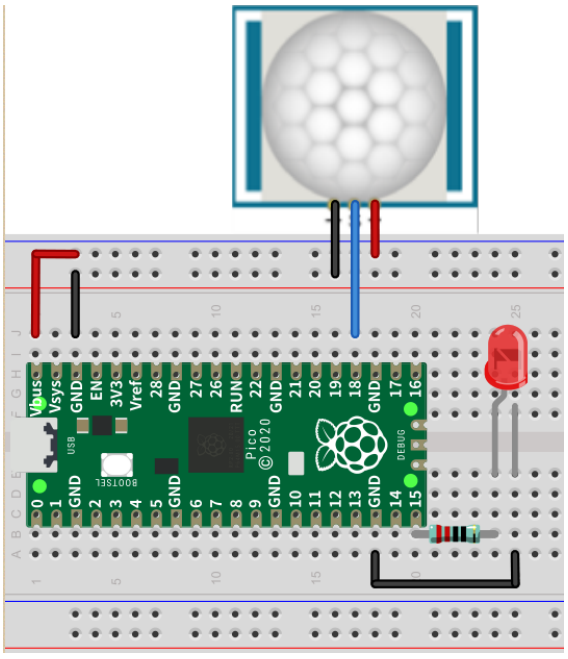
Apart from that, you can also use this sensor to control some other modules to implement different functions by reediting the code, such as the induction lamp, induction door.

Move to the Infrared Motion Sensor





Move away from the Infrared Motion Sensor





The following is the program code:

```
1  from machine import Pin
2  import time
3
4  sensorPin = Pin(18,Pin.IN)
5  ledPin = Pin(15,Pin.OUT)
6
7  try:
```

```
8       while True:
9           if not sensorPin.value():
10              ledPin.value(1)   #Set led turn on
11          else:
12              ledPin.value(0)   #Set led turn off
13  except:
14      pass
```

Configure the associated pin of the infrared Motion sensor as GP18 and set it to the input state; configure the associated pin of the LED light.

```
4   sensorPin = Pin(18,Pin.IN)
5   ledPin = Pin(15,Pin.OUT)
```

Call value() to monitor the infrared Motion sensor output for high or low level.

```
9           if not sensorPin.value():
```

When the infrared Motion sensor detects an infrared signal, the LED light is on; when there is no infrared signal, the LED light turns off.

```
9           if not sensorPin.value():
10              ledPin.value(1)   #Set led turn on
11          else:
12              ledPin.value(0)   #Set led turn off
```

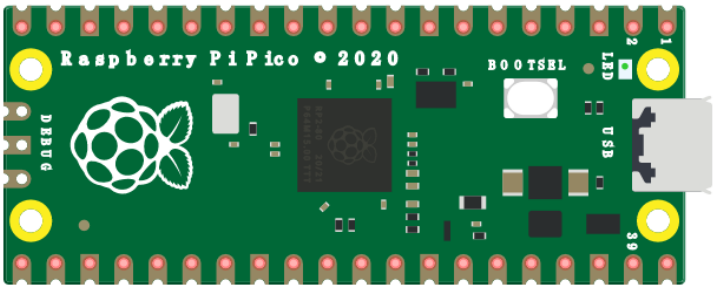
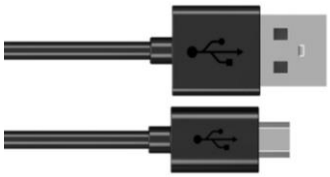
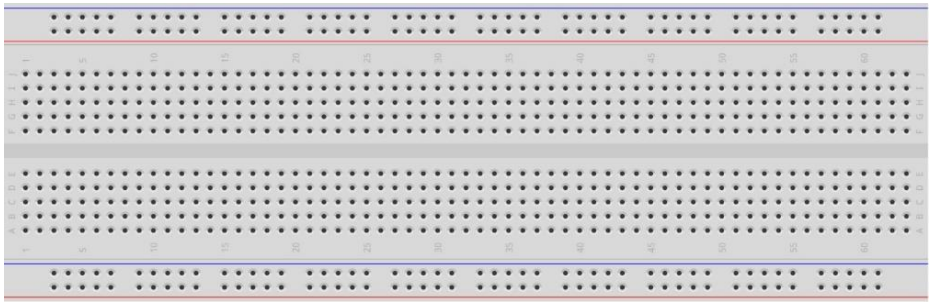
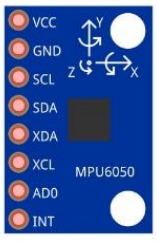
Chapter 27 Attitude Sensor MPU6050

In this chapter, we will learn about an MPU6050 Attitude Sensor, which integrates an Accelerometer and Gyroscope.

Project 27.1 Read a MPU6050 Sensor Module

In this project, we will read Acceleration and Gyroscope Data of the MPU6050 Sensor.

Component List

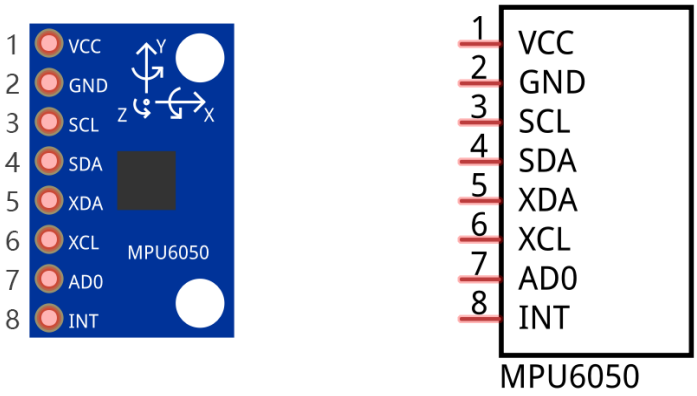
Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Jumper 	MPU6050 x1 

Component knowledge

MPU6050

MPU6050 Sensor Module is a complete 6-axis Motion Tracking Device. It combines a 3-axis Gyroscope, a 3-axis Accelerometer and a DMP (Digital Motion Processor) all in a small package. The settings of the Accelerometer and Gyroscope of MPU6050 can be changed. A precision wide range digital temperature sensor is also integrated to compensate data readings for changes in temperature, and temperature values can also be read. The MPU6050 Module follows the I2C communication protocol and the default address is 0x68.

MPU6050 is widely used to assist with balancing vehicles, robots and aircraft, mobile phones and other products which require stability to control stability and attitude or which need to sense same.



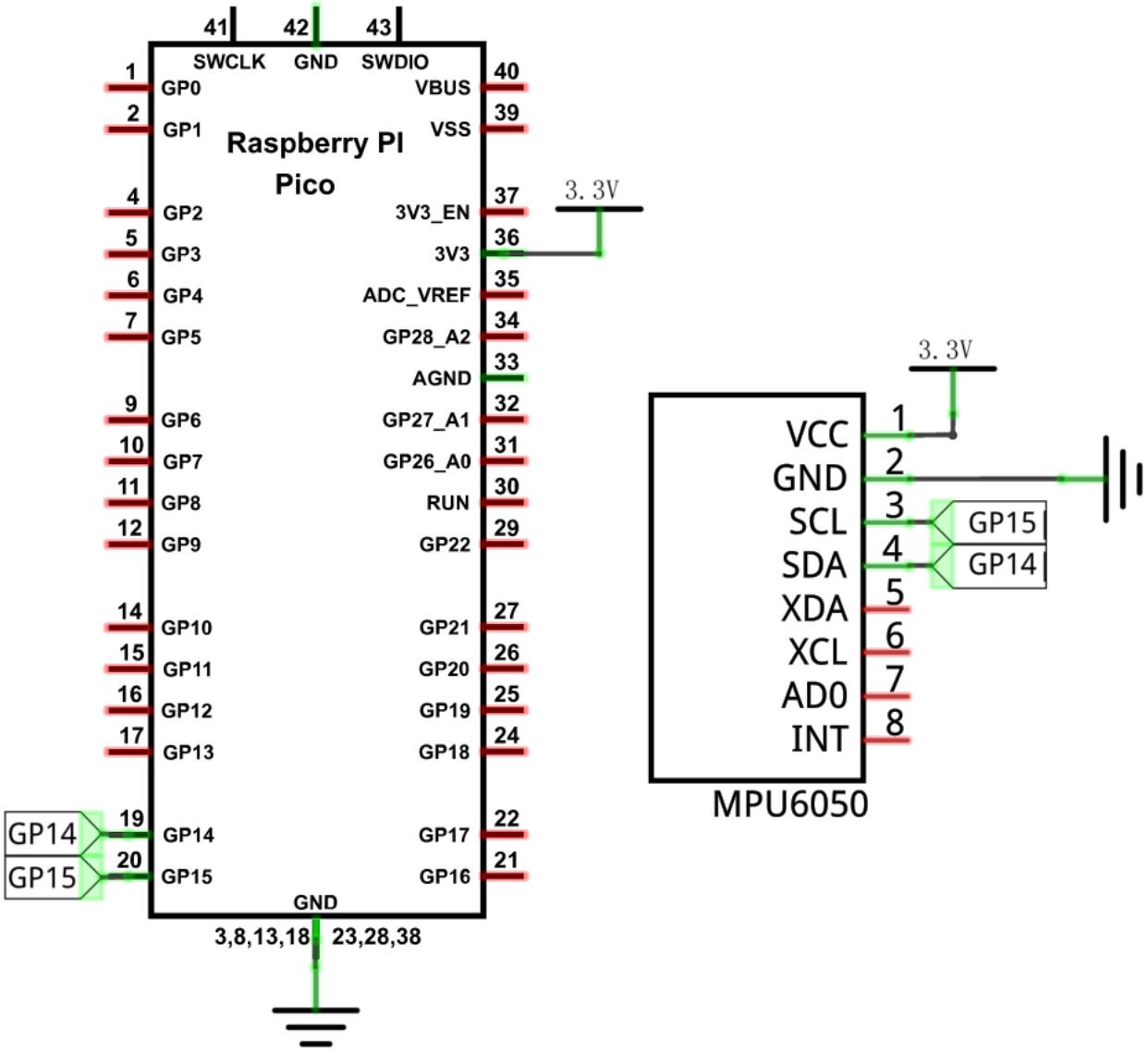
The port description of the MPU6050 module is as follows:

Pin name	Pin number	Description
VCC	1	Positive pole of power supply with voltage 5V
GND	2	Negative pole of power supply
SCL	3	I2C communication clock pin
SDA	4	I2C communication clock pin
XDA	5	I2C host data pin, which can be connected to other devices.
XCL	6	I2C host clock pin, which can be connected to other devices.
AD0	7	I2C address bit control pin. Low level: the device address is 0x68 High level: the device address is 0x69
INT	8	Output interrupt pin

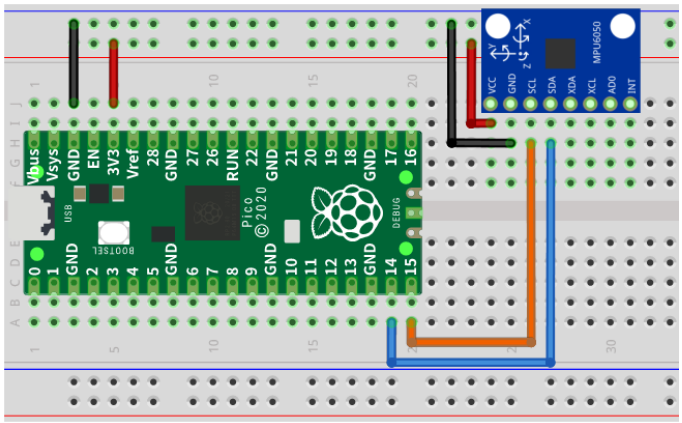
For more details, please refer to datasheet.

Circuit

Schematic diagram



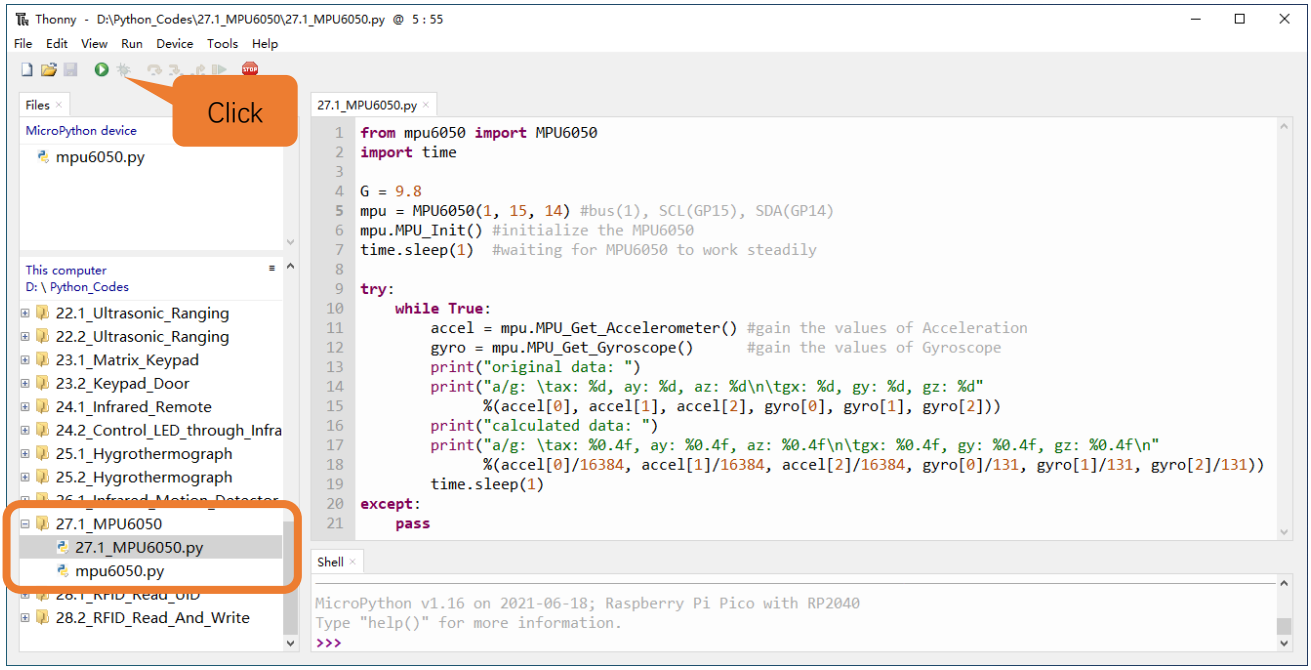
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "27.1_MPU6050". Select "mpu6050.py", right click your mouse to select "Upload to /", wait for "mpu6050.py" to be uploaded to Raspberry Pi Pico and then double click "27.1_MPU6050.py".

27.1_MPU6050



Click "Run current script". When Raspberry Pi Pico obtains acceleration data and gyroscope data of MPU605, it will print them in "Shell". Press Ctrl+C or click "Stop/Restart backend" to exit the program.

```
MicroPython v1.16 on 2021-06-18; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
  original data:
  a/g:    ax: 792, ay: -44, az: 15658
          gx: 454, gy: -55, gz: -22
  calculated data:
  a/g:    ax: 0.0483, ay: -0.0027, az: 0.9557
          gx: 3.4656, gy: -0.4198, gz: -0.1679

  original data:
  a/g:    ax: 770, ay: -28, az: 15664
          gx: 454, gy: -56, gz: -22
  calculated data:
  a/g:    ax: 0.0470, ay: -0.0017, az: 0.9561
```

Note: The data transmission of MPU6050 is very sensitive. Therefore, when using it, please make sure the jumper wire is in good contact; otherwise, the data may fail to be obtained.

The following is the program code:

```
from mpu6050 import MPU6050
import time

G = 9.8
mpu = MPU6050(1, 15, 14) #bus(1), SCL(GP15), SDA(GP14)
mpu.MPU_Init() #initialize the MPU6050
time.sleep(1)  #waiting for MPU6050 to work steadily

try:
    while True:
        accel = mpu.MPU_Get_Accelerometer() #gain the values of Acceleration
        gyro = mpu.MPU_Get_Gyroscope()      #gain the values of Gyroscope
        print("original data: ")
        print("a/g: \tax: %d, ay: %d, az: %d\n\tgx: %d, gy: %d, gz: %d"
              %(accel[0], accel[1], accel[2], gyro[0], gyro[1], gyro[2]))
        print("calculated data: ")
        print("a/g: \tax: %0.4f, ay: %0.4f, az: %0.4f\n\tgx: %0.4f, gy: %0.4f, gz: %0.4f\n"
              %(accel[0]/16384, accel[1]/16384, accel[2]/16384, gyro[0]/131, gyro[1]/131,
              gyro[2]/131))
        time.sleep(1)
except:
    pass
```

Import MPU6050 and time modules.

```
from mpu6050 import MPU6050
import time
```

Set I2C pins and associate them with MPU6050 module, and then initialize MPU6050 and wait for the initialization to complete.

```
mpu = MPU6050(1, 15, 14) #bus(1), SCL(GP15), SDA(GP14)
mpu.MPU_Init() #initialize the MPU6050
```

Obtain the acceleration data of MPU6050 and store it in accel. Obtain the gyroscope data and store it in gyro.

```
        accel = mpu.MPU_Get_Accelerometer() #gain the values of Acceleration
        gyro = mpu.MPU_Get_Gyroscope()      #gain the values of Gyroscope
```

Update and collect the original data of the gyroscope every second and print the original data and processed acceleration and angular velocity data in "Shell".

```
10      while True:
11          accel = mpu.MPU_Get_Accelerometer() #gain the values of Acceleration
12          gyro = mpu.MPU_Get_Gyroscope()      #gain the values of Gyroscope
13          print("original data: ")
14          print("a/g: \tax: %d, ay: %d, az: %d\n\tgx: %d, gy: %d, gz: %d"
15                %(accel[0], accel[1], accel[2], gyro[0], gyro[1], gyro[2]))
16          print("calculated data: ")
17          print("a/g: \tax: %0.4f, ay: %0.4f, az: %0.4f\n\tgx: %0.4f, gy: %0.4f, gz: %0.4f\n"
18                %(accel[0]/16384, accel[1]/16384, accel[2]/16384, gyro[0]/131, gyro[1]/131,
                  gyro[2]/131))
19          time.sleep(1)
```

Reference

Class mpu6050

Before each use of **mpu6050**, please add the statement "**from mpu6050 import MPU6050**" to the top of the python file.

MPU6050(bus, sclpin, sdapin): Create MPU6050 object and set the bus and pins for I2C.

MPU_Init(): Initialize MPU6050 module.

MPU_Get_Accelerometer(): Obtain original data of MPU6050's acceleration.

MPU_Get_Gyroscope(): Obtain original data of MPU6050's Gyroscope.

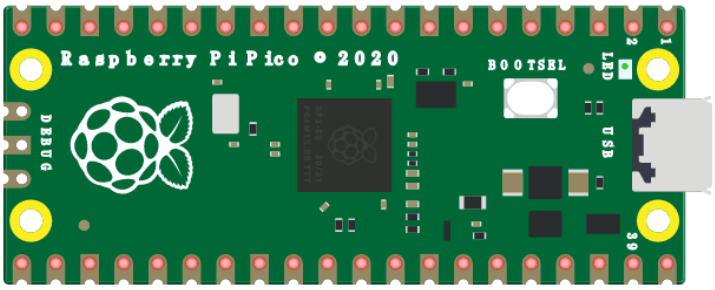
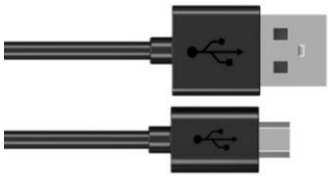
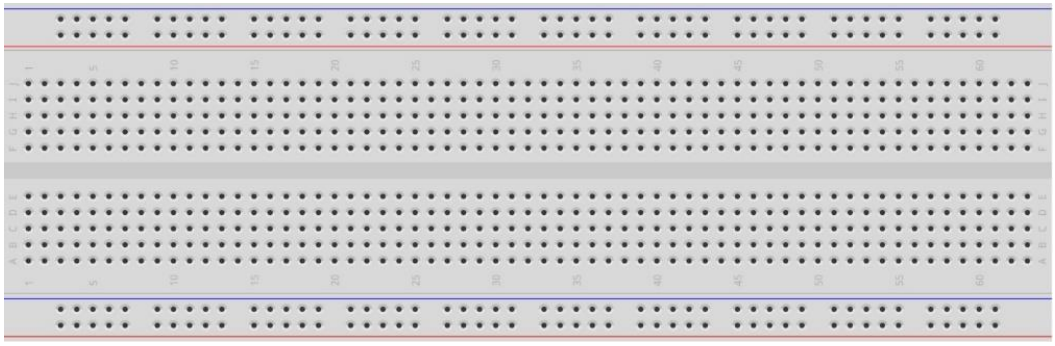
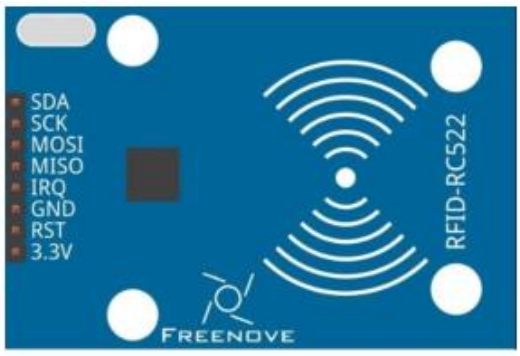
Chapter 28 RFID

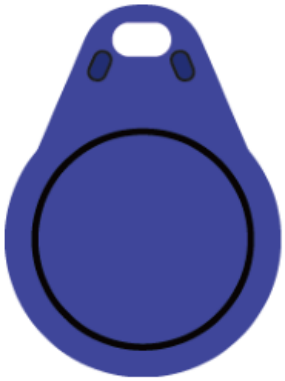
Now, we will learn to use the RFID (Radio Frequency Identification) wireless communication technology.

Project 28.1 RFID read UID

In this project, we will read the unique ID number (UID) of the RFID card, recognize the type of the RFID card and display the information through serial port.

Component List

Raspberry Pi Pico x1 	USB cable x1 
Breadboard x1 	
Jumper 	RFID Module(RC522) x1 

Mifare1 S50 Standard card x1	Mifare1 S50 Non-standard card x1
	

Component knowledge

RFID

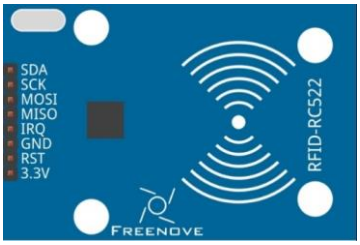
RFID (Radio Frequency Identification) is a wireless communication technology. A complete RFID system is generally composed of the responder and reader. Generally, we use tags as responders, and each tag has a unique code, which is attached to the object to identify the target object. The reader is a device for reading (or writing) tag information.

Products derived from RFID technology can be divided into three categories: passive RFID products, active RFID products and semi active RFID products. In addition, Passive RFID products are the earliest, the most mature and most widely used products in the market among others. It can be seen everywhere in our daily life such as, the bus card, dining card, bankcard, hotel access cards, etc., and all of these belong to close-range contact recognition. The main operating frequency of Passive RFID products are: 125KHZ (low frequency), 13.56MHZ (high frequency), 433MHZ (ultrahigh frequency), 915MHZ (ultrahigh frequency). Active and semi active RFID products work at higher frequencies.

The RFID module we use is a passive RFID product with the operating frequency of 13.56MHz.

MFRC522 RFID Module

The MFRC522 is a highly integrated reader/writer IC for contactless communication at 13.56MHz.
The MFRC522's internal transmitter is able to drive a reader/writer antenna designed to communicate with ISO/IEC 14443 A/MIFARE cards and transponders without additional active circuitry. The receiver module provides a robust and efficient implementation for demodulating and decoding signals from ISO/IEC 14443 A/MIFARE compatible cards and transponders. The digital module manages the complete ISO/IEC 14443A framing and error detection (parity and CRC) functionality.
This RFID Module uses MFRC522 as the control chip and SPI (Peripheral Interface Serial) as the reserved interface.

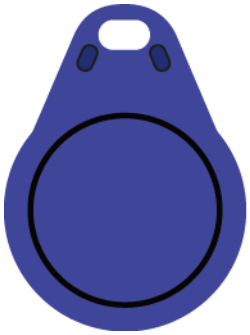


Mifare1 S50 Card

Mifare1 S50 is often called Mifare Standard with the capacity of 1K bytes. And each card has a 4-bytes global unique identifier number (USN/UID), which can be rewritten 100 thousand times and read infinite times. Its storage period can last for 10 years. The ordinary Mifare1 S50 Card and non-standard Mifare1 S50 Card equipped for this kit are shown below.



Mifare1 S50 Standard card



Mifare1 S50 Non-standard card

The Mifare1 S50 capacity (1K byte) is divided into 16 sectors (Sector0-Sector15). Each sector contains four data block (Block0-Block3. 64 blocks of 16 sectors will be numbered according to absolute address, from 0 to 63). Each block contains 16 bytes (Byte0-Byte15), 64*16=1024. As is shown in the following table:

Sector No.	Block No.	Storage area	Block type	Absolute block No.
sector 0	block 0	vendor code	vendor block	0
	block 1		data block	1
	block 2		data block	2
	block 3	Password A-access control-password B	control block	3
sector 1	block 0		data block	4
	block 1		data block	5
	block 2		data block	6
	block 3	Password A-access control-password B	control block	7
……	……	……	……	
sector 15	block 0		data block	60
	block 1		data block	61
	block 2		data block	62
	block 3	Password A-access control-password B	control block	63

Each sector has a set of independent password and access control, which are put, in the last block of each sector, and the block is also known as sector trailer, that is Block 3 in each sector. Sector 0, block 0 (namely absolute address 0) of S50 is used to store the vendor code, which has been solidified and cannot be changed, and the card serial number is stored here. In addition to the manufacturer and the control block, the rest of the cards are data blocks, which can be used to store data. Data block can be used for two kinds of applications:
(1) used as general data storage and can be operated for reading and writing.
(2) used as data value, and can be operated for initializing the value, adding value, subtracting and reading the value.

The sector trailer block in each sector is the control block, including a 6-byte password A, 4-byte access control and 6-byte password B. For example, the control block of a brand new card is as follows:

A0 A1 A2 A3 A4 A5	FF 07 80 69	B0 B1 B2 B3 B4 B5
password A	access control	password B

The default password of a brand new card is generally A0A1A2A3A4A5 for password A, B0B1B2B3B4B5 for password B, or both the password A and password B are 6 FF. Access control is used to set the access conditions for each block (including the control block itself) in a sector.

Blocks of S50 are divided into data blocks and control blocks. There are four operations, "read", "write", "add value", "subtract value (including transmission and storage)" for data blocks, and there are two operations, "read" and "write" for control blocks.

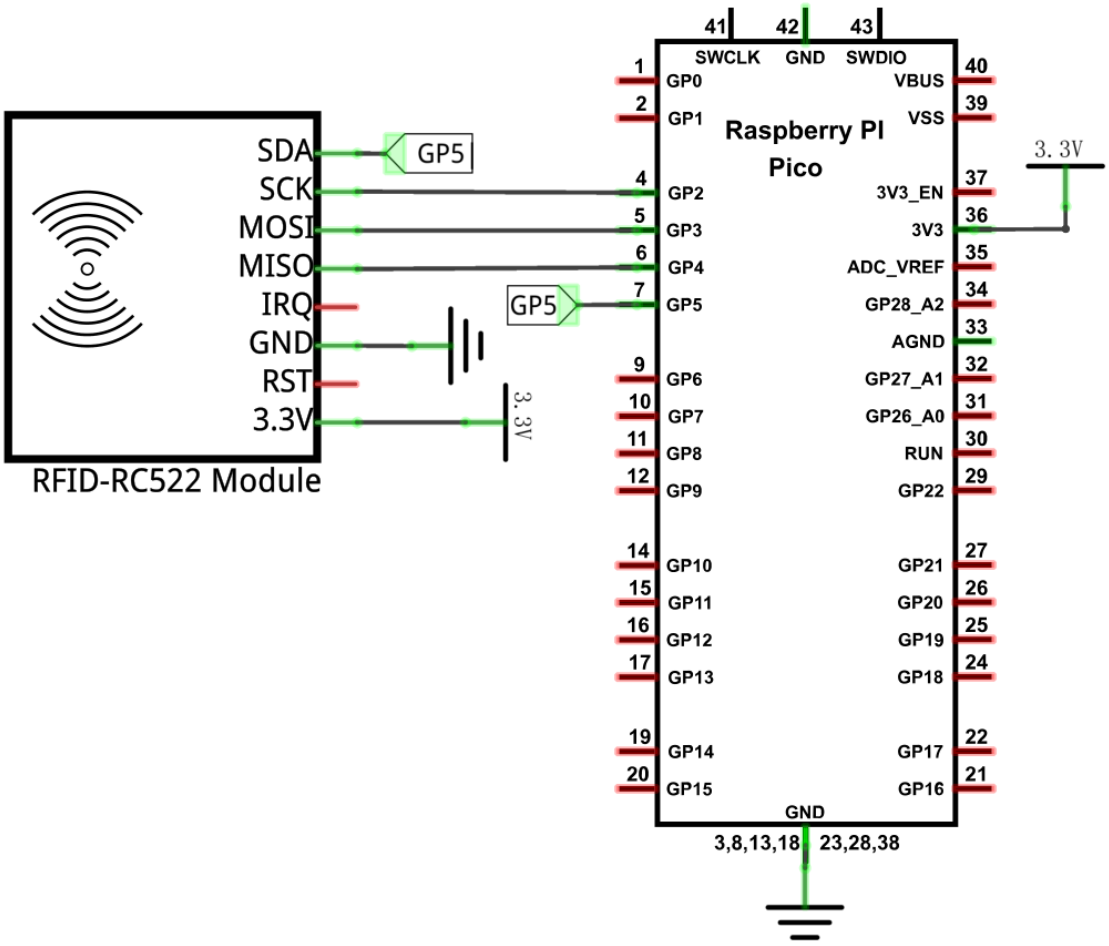
For more details about how to set data blocks and control blocks, please refer to Datasheet.

By default, after verifying password A or password B, we can do reading or writing operation to data blocks. And after verifying password A, we can do reading or writing operation to control blocks. However, password A can never be read. If you choose to verify password A and then you forget the password A, the block will never be able to read again. **It is highly recommended that beginners should not try to change the contents of control blocks.**

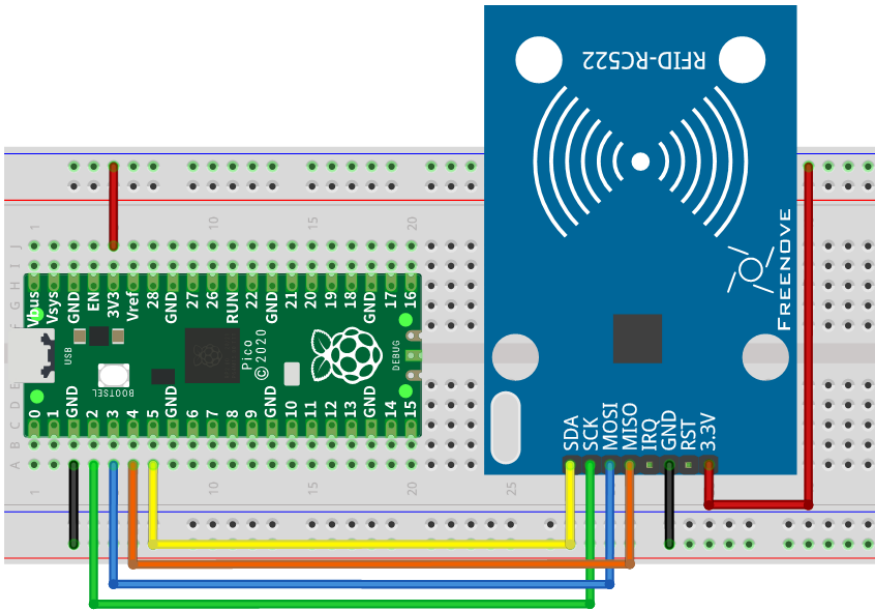
Circuit

The connection of control board and RFID module is shown below.

Schematic diagram



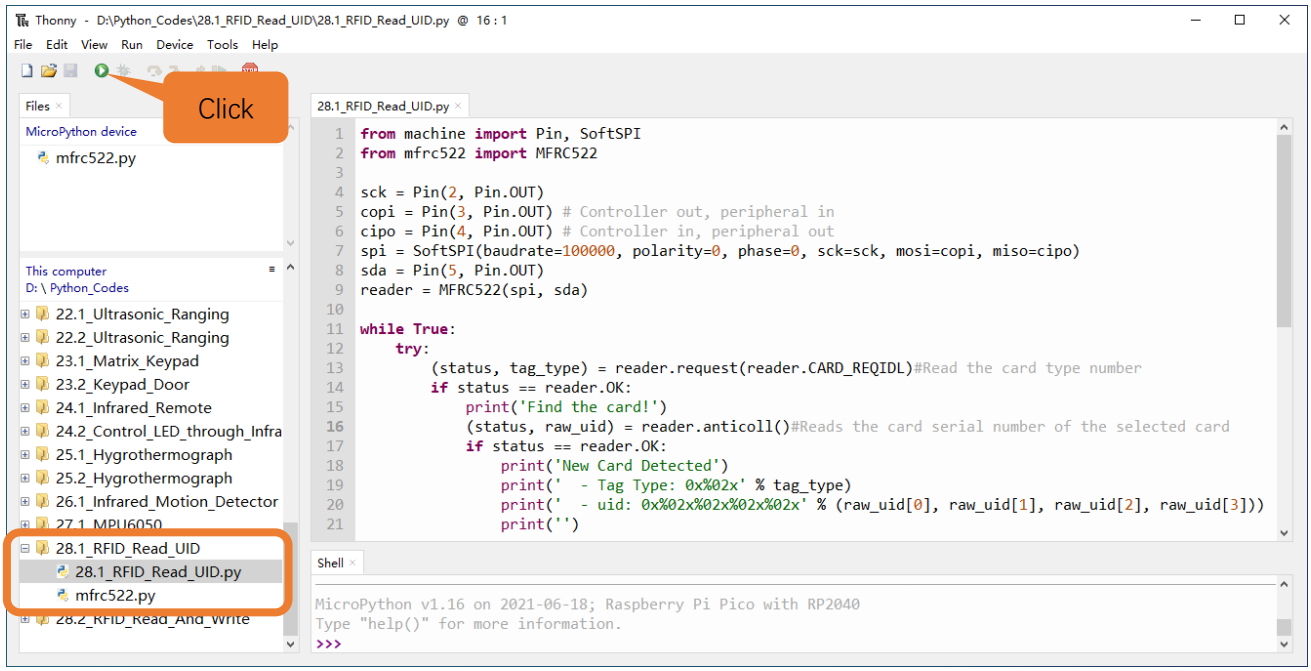
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "28.1_RFID_Read_UID". Select "mfrc522.py", right click to select "Upload to /", wait for "mfrc522.py" to be uploaded to Raspberry Pi Pico and then double click "28.1_RFID_Read_UID.py".

28.1_RFID_Read_UID



Click "Run current script", put the white or blue magnetic card near the RFID module, and Pico will print out the read information such as card type, card UID and card address data to "Shell". Press Ctrl+C or click "Stop/Restart backend" to exit the program.

```
Shell
    - Tag Type: 0x10
    - uid: 0x5cfb4117

  Address Data: bytearray(b'WelcomeFreenove!')
  Find the card!
  16
  New Card Detected
    - Tag Type: 0x10
    - uid: 0x5cfb4117

  Address Data: bytearray(b'WelcomeFreenove!')
  Find the card!
  16
  New Card Detected
    - Tag Type: 0x10
    - uid: 0x5cfb4117

  Address Data: bytearray(b'WelcomeFreenove!')
```

The following is the program code:

```
from machine import Pin, SoftSPI
from mfrc522 import MFRC522

sck = Pin(2, Pin.OUT)
copi = Pin(3, Pin.OUT) # Controller out, peripheral in
cipo = Pin(4, Pin.OUT) # Controller in, peripheral out
spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=sck, mosi=copi, miso=cipo)
sda = Pin(5, Pin.OUT)
reader = MFRC522(spi, sda)

while True:
    try:
        (status, tag_type) = reader.request(reader.CARD_REQIDL)#Read the card type number
        if status == reader.OK:
            print('Find the card!')
            print(tag_type)
            (status, raw_uid) = reader.anticoll()#Reads the card serial number of the selected card
            if status == reader.OK:
                print('New Card Detected')
                print('  - Tag Type: 0x%02x' % tag_type)
                print('  - uid: 0x%02x%02x%02x%02x%02x' % (raw_uid[0], raw_uid[1], raw_uid[2], raw_uid[3]))
                print('')
                if reader.select_tag(raw_uid) == reader.OK:#Read card memory capacity
                    key = [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF]
                    if reader.auth(reader.AUTH, 8, key, raw_uid) == reader.OK:#Verification card password
                        print("Address Data: %s" % bytearray(reader.read(8)))
                        reader.stop_crypto1()
                    else:
                        print("AUTH ERROR")
                else:
                    print("FAILED TO SELECT TAG")
    except KeyboardInterrupt:
        break
```

Import Pin, SoftSPI and MFRC522 modules.

```
from machine import Pin, SoftSPI
from mfrc522 import MFRC522
```

Set SPI Pins and associate them with RFID-RC522 module, and then set MFRC522 module.

```
sck = Pin(2, Pin.OUT)
copi = Pin(3, Pin.OUT) # Controller out, peripheral in
cipo = Pin(4, Pin.OUT) # Controller in, peripheral out
```

```
7  spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=sck, mosi=copi, miso=cipo)
8  sda = Pin(5, Pin.OUT)
9  reader = MFRC522(spi, sda)
```

Call the request() function to determine whether the RFID module detects the card. If so, read the model information of the card.

```
13      (status, tag_type) = reader.request(reader.CARD_REQIDL)#Read the card type number
```

Call anticol1() function to read serial number of the selected card and save the data to the raw_uid variable.

```
17      (status, raw_uid) = reader.anticoll()#Reads the card serial number of the selected card
```

Call select_tag() function to determine whether the memory information of the card can be read.

```
23      reader.select_tag(raw_uid)
```

Call the auth() function to verify whether the card's password information is correct.

```
25      reader.auth(reader.AUTH, 8, key, raw_uid)
```

Read the data of the card.

```
26      reader.read(8)
```

Stop reading the card.

```
27      reader.stop_crypto1()
```

The process of reading card information:

First determine whether the RFID module detects the card. If it does, read the model information and UID of the card, and then determine whether the memory information of the card is readable. If it is readable, first verify the password of the card. If the password is correct, read the address data of the card, and print out all the read information to the "Shell".

```
12 try:
13     (status, tag_type) = reader.request(reader.CARD_REQIDL)#Read the card type number
14     if status == reader.OK:
15         print('Find the card!')
16         print(tag_type)
17         (status, raw_uid) = reader.anticoll()#Reads the card serial number of the selected card
18         if status == reader.OK:
19             print('New Card Detected')
20             print('  - Tag Type: 0x%02x' % tag_type)
21             print('     - uid:  0x%02x%02x%02x%02x' % (raw_uid[0], raw_uid[1], raw_uid[2],
   raw_uid[3]))
22             print('')
23             if reader.select_tag(raw_uid) == reader.OK:#Read card memory capacity
24                 key = [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF]
25                 if reader.auth(reader.AUTH, 8, key, raw_uid) == reader.OK:#Verification card
   password
26                     print("Address Data: %s" % bytearray(reader.read(8)))
27                     reader.stop_crypto1()
28                 else:
29                     print("AUTH ERROR")
30             else:
31                 print("FAILED TO SELECT TAG")
```

Reference

Class MFRC522()

Before each use of mfrcC522 module, please add the statement "**from mfrc522 import MFRC522**" to the top of the python file.

MFRC522(spi, cs): Create an object MFRC522.

spi: instance of the SoftSPI class.

cs: Chip select.

Default variables in the MFRC522 class:

MFRC522.OK = 0

MFRC522.NO_TAG_ERR = 1

MFRC522.ERR = 2

MFRC522.CARD_REQIDL = 0x26

MFRC522.AUTH = 0x60

MFRC522.init(): Initialize MFRC522 module.

MFRC522.reset(): Reset MFRC522 module.

MFRC522.request(mode): Request to read card data. The function returns a status code and an int. The status code is used to check whether a card is being read. The data of the int type is the model data of the card.

mode: CARD_REQIDL = 0x26.

MFRC522.anticol1(): Read the serial number of the card. The function returns a list and a status code. The data in the list is the serial number of the card.

MFRC522.select_tag(): Check whether the memory information of the card can be read.

MFRC522.auth(mode, addr, sect, serial_number): The function is used to verify the card password, and returns a boolean value. If it is correct, it returns True, if it is wrong, it returns False.

mode: AUTH = 0x60.

addr: data address.

sect: password of the card.

serial_number: serial number of the card.

MFRC522.read(addr): read data in addr address.

MFRC522.write(addr, data): Function to write card data. addr is the address of the data to be written and data is the data to be written. Returns a status code.

MFRC522.Read_Data(sect, serial_number): Read all the data written to the card and print it out.

sect: password of the card.

serial_number: serial number of the card.

MFRC522.Write_Data(sect, serial_number, datas): Write datas data into the card, the original data in the card will be overwritten.

MFRC522.Clear_Data(sect, serial_number): Clear the data written to the card.

MFRC522.Write_Row_Data(sect, serial_number, addr, datas): Write the datas data to the addr address.

MFRC522.Read_Row_Data(sect, serial_number, addr): Read the data at the addr address and print it out.

MFRC522.Clear_Row_Data(sect, serial_number, addr): Clear the data of the addr address.

Class SoftSPI

Before each use of SoftSPI module, please add the statement "**from machine import SoftSPI**" to the top of the python file.

Machine.SoftSPI(id, baudrate, polarity, phase, bits, sck, mosi, miso): construct a new software SPI object. Additional parameters must be give, usually at least id, baudrate, sck, mosi, miso.

SoftSPI.init(baudrate, polarity, phase, bits, sck, mosi, miso):

Baudrate: the SCK clock rate.

Polarity: It is set as 0 or 1, which represents the level of the clock line when it is idle.

Phase: It is set as 0 or 1, to sample data on the first or second clock edge, respectively.

sck, mosi, miso: Corresponding pins of SPI bus in Raspberry Pi Pico.

SoftSPI.deinit(): Close the SoftSPI bus.

SoftSPI.read(nbytes, write=0): Read a number of bytes specified by nbytes while continuously writing the single byte given by write. Returns a bytes object with the data that was read.

SoftSPI.readinto(buf, write=0): Read into the buffer specified by buf while continuously writing the single byte given by write. Returns None.

SoftSPI.write(buf): Write the bytes contained in buf. Returns None.

SoftSPI.write_readinto(write_buf, read_buf): Write the bytes from write_buf while reading into read_buf. The buffers can be the same or different, but both buffers must have the same length. Returns None.

Project 28.2 RFID Read and Write

In this project, we will do reading and writing operations to the card.

Code

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “28.2_RFID_Read_And_Write”. Select“mfrc522.py”, right click to select “Upload to /”, wait for “mfrc522.py” to be uploaded to Raspberry Pi Pico and then double click “28.2_RFID_Read_And_Write.py”.

282_RFID_Read_And_Write

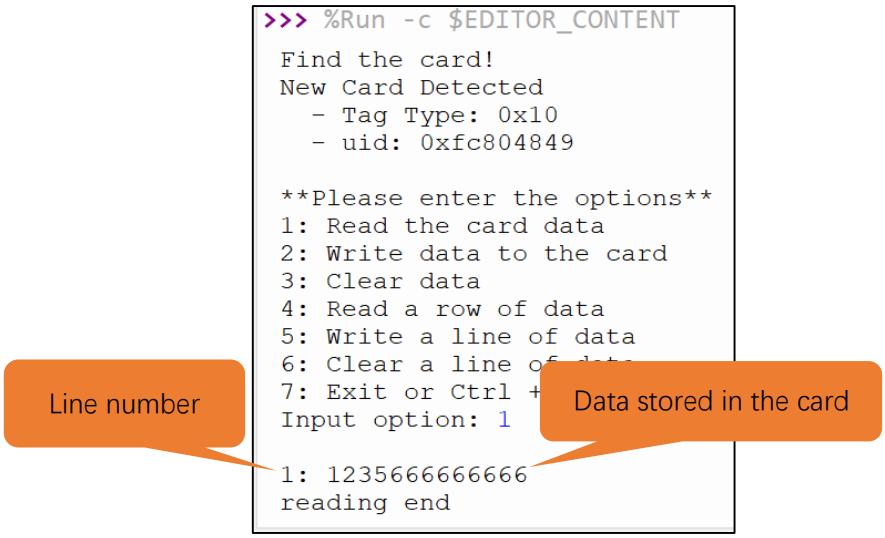


Click "Run current script", put the white magnetic card or blue magnetic card close to the induction area of the RFID module, the "Shell" window prints the read information such as card UID, card type and other data. It also provides the option to perform regular read and write operations on the card. Press Ctrl+C or click “Stop/Restart backend” to exit the program.

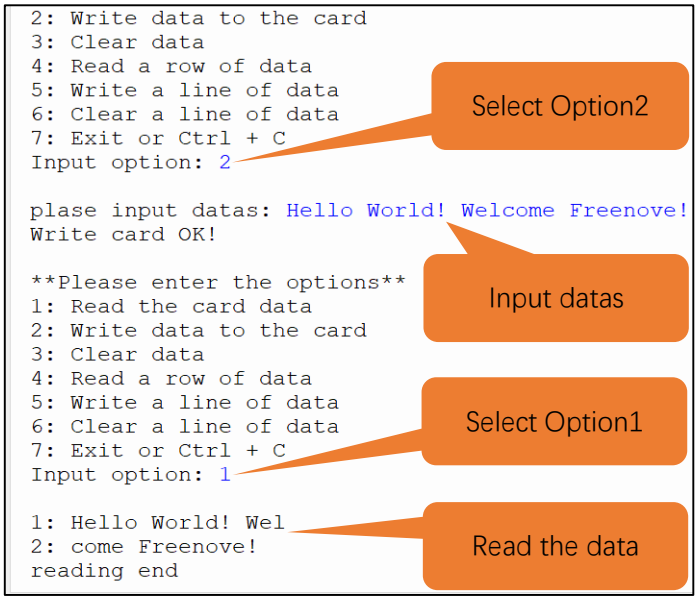
```
>>> %Run -c $EDITOR_CONTENT
 Find the card!
 New Card Detected
   - Tag Type: 0x10
   - uid: 0xfc804849

 **Please enter the options**
 1: Read the card data
 2: Write data to the card
 3: Clear data
 4: Read a row of data
 5: Write a line of data
 6: Clear a line of data
 7: Exit or Ctrl + C
```

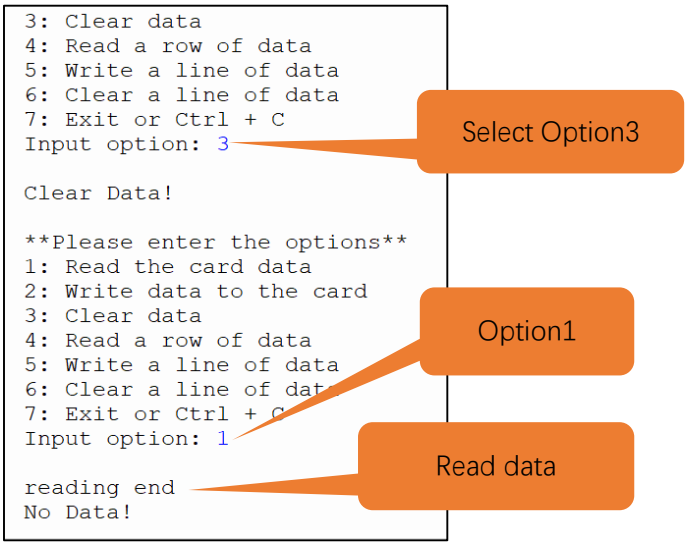
For example, Option 1 is to read all the data of the card.



Option2 is to write data to the card.



Option3 is to clear data.



The following is the program code:

```
from machine import Pin, SoftSPI
from mfrc522 import MFRC522

sck = Pin(2, Pin.OUT)
copi = Pin(3, Pin.OUT) # Controller out, peripheral in
cipo = Pin(4, Pin.OUT) # Controller in, peripheral out
spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=sck, mosi=copi, miso=cipo)
sda = Pin(5, Pin.OUT)
reader = MFRC522(spi, sda)
key = [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF]
option = 0

try:
    while True:
        (status, tag_type) = reader.request(reader.CARD_REQIDL)#Read the card type number
        if status == reader.OK:
            print('Find the card!')
            (status, raw_uid) = reader.anticoll()#Reads the card serial number of the selected card
            if status == reader.OK:
                print('New Card Detected')
                print('  - Tag Type: 0x%02x' % tag_type)
                print('  - uid: 0x%02x%02x%02x%02x%02x' % (raw_uid[0], raw_uid[1], raw_uid[2], raw_uid[3]))
                print('')
                if reader.select_tag(raw_uid) == reader.OK:#Read card memory capacity
                    while option != "7":
                        print("**Please enter the options**")
                        print("1: Read the card data")
                        print("2: Write data to the card")
                        print("3: Clear data")
                        print("4: Read a row of data")
                        print("5: Write a line of data")
                        print("6: Clear a line of data")
                        print("7: Exit or Ctrl + C")
                        option = input("Input option: ")
                        print("")
                        if option == "1":
                            reader.Read_Data(key, raw_uid)
                        elif option == "2":
                            datas = input("plase input datas: ")
                            reader.Write_Data(key, raw_uid, datas)
                        elif option == "3":
```

```
42                              reader.Clear_Data(key, raw_uid)
43                          elif option == "4":
44                              row_index = int(input("Please enter the row index: "))
45                              reader.Read_Row_Data(key, raw_uid, row_index)
46                          elif option == "5":
47                              row_index = int(input("Please enter the row index: "))
48                              datas = input("plase input datas: ")
49                              reader.Write_Row_Data(key, raw_uid, row_index, datas)
50                          elif option == "6":
51                              row_index = int(input("Please enter the row index: "))
52                              reader.Clear_Row_Data(key, raw_uid, row_index)
53                          elif option == "7":
54                              reader.stop_crypto1()
55                              print("Exit the success")
56                          else:
57                              print("Input error")
58                          print("")
59                      break
60  except KeyboardInterrupt:
61      pass
```

Print the function of each option; use the option variable to receive the options input by the user.

```
26          print("**Please enter the options**")
27          print("1: Read the card data")
28          print("2: Write data to the card")
29          print("3: Clear data")
30          print("4: Read a row of data")
31          print("5: Write a line of data")
32          print("6: Clear a line of data")
33          print("7: Exit or Ctrl + C")
34          option = input("Input option: ")
```

Read_Data() function reads data in the card (excluding the password).

```
37          reader.Read_Data(key, raw_uid)
```

Call Write_Data() to continuously write data to the card. Call the function again will overwrite data previously input.

```
40          reader.Write_Data(key, raw_uid, datas)
```

Clear_Data() function is used to clear all input data in the card.

```
42          reader.Clear_Data(key, raw_uid)
```

Call Read_Row_Data() function to read data in designated address.

```
45          reader.Read_Row_Data(key, raw_uid, row_index)
```

Write_Row_Data() function writes data to designated address.

```
49          reader.Write_Row_Data(key, raw_uid, row_index, datas)
```

Clear_Row_Data() clears data of designated address.

```
52          reader.Clear_Row_Data(key, raw_uid, row_index)
```

Based on the prompt, users can input option in "Shell" to read, write, or erase data on the card, and output the result in the "Shell". You can exit the program by inputting 7 or pressing Ctrl + C.

```
    while option != "7":
        print("**Please enter the options**")
        print("1: Read the card data")
        print("2: Write data to the card")
        print("3: Clear data")
        print("4: Read a row of data")
        print("5: Write a line of data")
        print("6: Clear a line of data")
        print("7: Exit or Ctrl + C")
        option = input("Input option: ")
        print("")
        if option == "1":
            reader.Read_Data(key, raw_uid)
        elif option == "2":
            datas = input("plase input datas: ")
            reader.Write_Data(key, raw_uid, datas)
        elif option == "3":
            reader.Clear_Data(key, raw_uid)
        elif option == "4":
            row_index = int(input("Please enter the row index: "))
            reader.Read_Row_Data(key, raw_uid, row_index)
        elif option == "5":
            row_index = int(input("Please enter the row index: "))
            datas = input("plase input datas: ")
            reader.Write_Row_Data(key, raw_uid, row_index, datas)
        elif option == "6":
            row_index = int(input("Please enter the row index: "))
            reader.Clear_Row_Data(key, raw_uid, row_index)
        elif option == "7":
            reader.stop_crypto1()
            print("Exit the success")
        else:
            print("Input error")
        print("")
    break
```

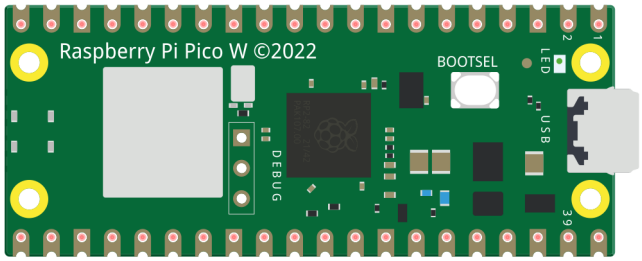
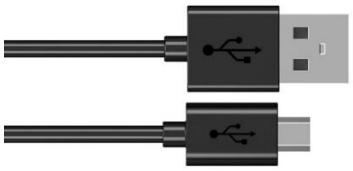
Chapter 29 WiFi Working Modes (Only for Pico W/Pico 2W)

The biggest difference between the raspberry pi pico and the raspberry pi pico W is that the raspberry pi Pico W is equipped with a WiFi function module. At the beginning of this chapter, we will learn about the WiFi function of Pico W of Raspberry Pi.

If you have Pico in your hand, please change it to Pico W before continuing to learn.

Project 29.1 Station mode

Component List

Raspberry Pi Pico W(or Pico 2W) x1 	Micro USB Wire x1 

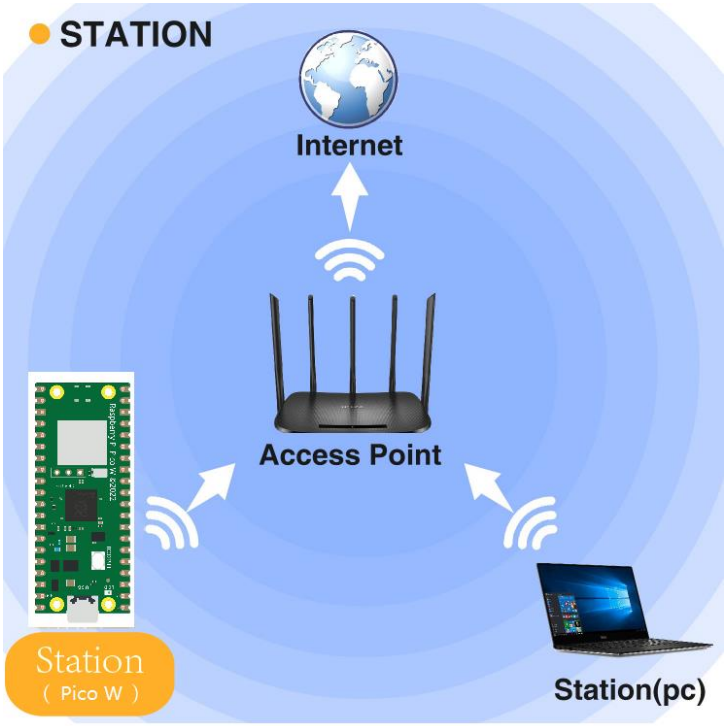
Component knowledge

Wireless

Pico W has an on-board 2.4GHz wireless interface using an Infineon CYW43439. The antenna is an onboard antenna licensed from ABRACON (formerly ProAnt). The wireless interface is connected via SPI to the RP2040.

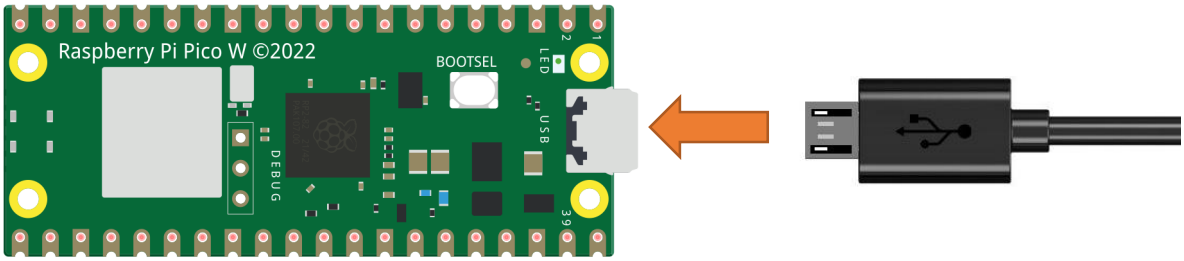
Station mode

When PICO W selects Station mode, it acts as a WiFi client. It can connect to the router network and communicate with other devices on the router via WiFi connection. As shown below, the PC is connected to the router, and if PICO W wants to communicate with the PC, it needs to be connected to the router.



Circuit

Connect Pico W to the computer using the USB cable.

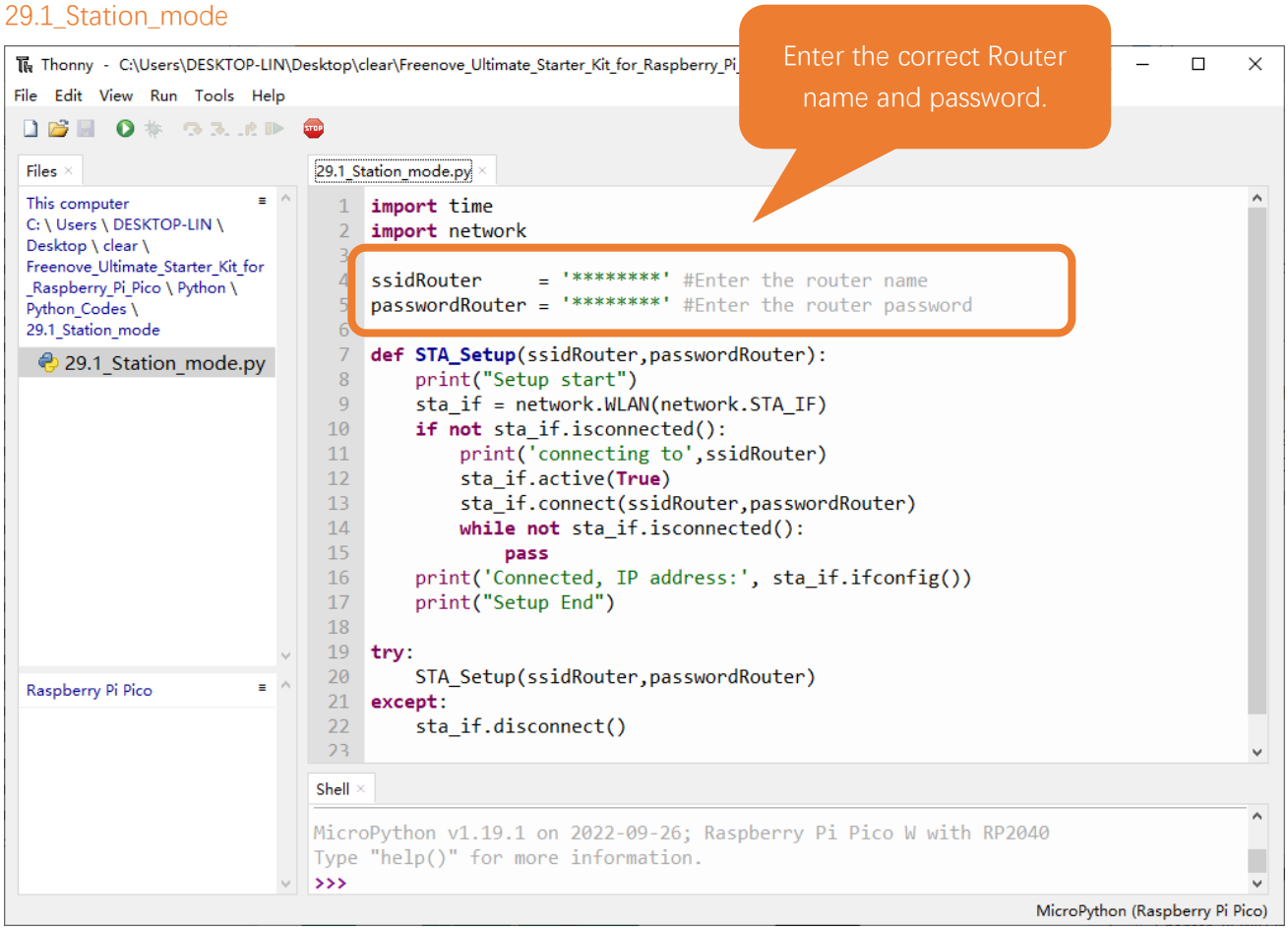


Code

Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

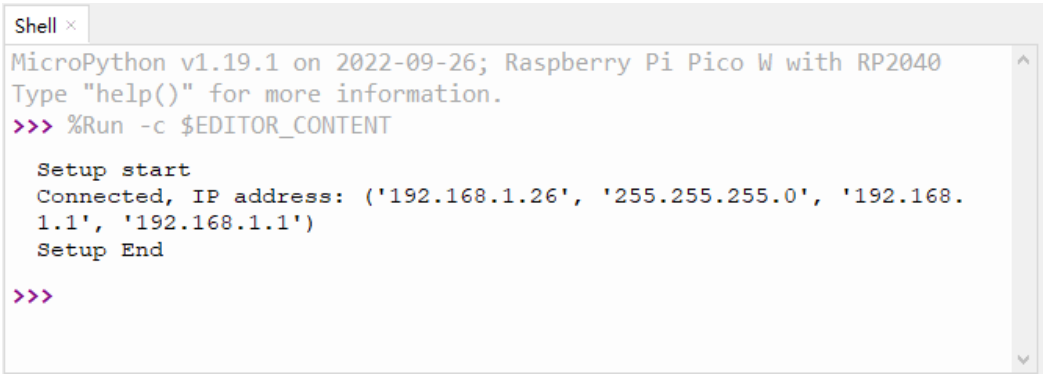
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "29.1_Station_mode" and double click "29.1_Station_mode.py".

29.1_Station_mode



Because the names and passwords of routers in various places are different, before the Code runs, users need to enter the correct router's name and password in the box as shown in the illustration above.

After making sure the router name and password are entered correctly, compile and upload codes to PICO W, wait for PICO W to connect to your router and print the IP address assigned by the router to PICO W in "Shell".



The following is the program code:

```
1  import time
2  import network
3
4  ssidRouter     = '********' #Enter the router name
5  passwordRouter = '********' #Enter the router password
6
```

```
7  def STA_Setup(ssidRouter,passwordRouter):
8      print("Setup start")
9      sta_if = network.WLAN(network.STA_IF)
10     if not sta_if.isconnected():
11         print('connecting to',ssidRouter)
12         sta_if.active(True)
13         sta_if.connect(ssidRouter,passwordRouter)
14         while not sta_if.isconnected():
15             pass
16     print('Connected, IP address:', sta_if.ifconfig())
17     print("Setup End")
18
19 try:
20     STA_Setup(ssidRouter,passwordRouter)
21 except:
22     sta_if.disconnect()
```

Import network module.

```
2  import network
```

Enter correct router name and password.

```
4  const char *ssid_Router     = "********"; //Enter the router name
5  const char *password_Router = "********"; //Enter the router password
```

Set PICO W in Station mode.

```
9    sta_if = network.WLAN(network.STA_IF)
```

Activate Pico W's Station mode, initiate a connection request to the router and enter the password to connect.

```
12         sta_if.active(True)
13         sta_if.connect(ssidRouter,passwordRouter)
```

Wait for PICO W to connect to router until they connect to each other successfully.

```
14         while not sta_if.isconnected():
15             pass
```

Print the IP address assigned to PICO W in "Shell".

```
16   Print('Connected, IP address:', sta_if.ifconfig())
```

Reference

Class network

Before each use of **network**, please add the statement "**import network**" to the top of the python file.

WLAN(interface_id): Set to WiFi mode.

network.STA_IF: Client, connecting to other WiFi access points.

network.AP_IF: Access points, allowing other WiFi clients to connect.

active(is_active): With parameters, it is to check whether to activate the network interface; Without parameters, it is to query the current state of the network interface.

scan(ssid, bssid, channel, RSSI, authmode, hidden): Scan for wireless networks available nearby (only scan on STA interface), return a tuple list of information about the WiFi access point.

bssid: The hardware address of the access point, returned in binary form as a byte object. You can use

ubinascii.hexlify() to convert it to ASCII format.

authmode: Access type

AUTH_OPEN = 0

AUTH_WEP = 1

AUTH_WPA_PSK = 2

AUTH_WPA2_PSK = 3

AUTH_WPA_WPA2_PSK = 4

AUTH_MAX = 6

Hidden: Whether to scan for hidden access points

False: Only scanning for visible access points

True: Scanning for all access points including the hidden ones.

isconnected(): Check whether PICO W is connected to AP in Station mode. In STA mode, it returns True if it is connected to a WiFi access point and has a valid IP address; Otherwise, it returns False.

connect(ssid, password): Connecting to wireless network.

ssid: WiFiname

password: WiFipassword

disconnect(): Disconnect from the currently connected wireless network.

Project 29.2 AP mode

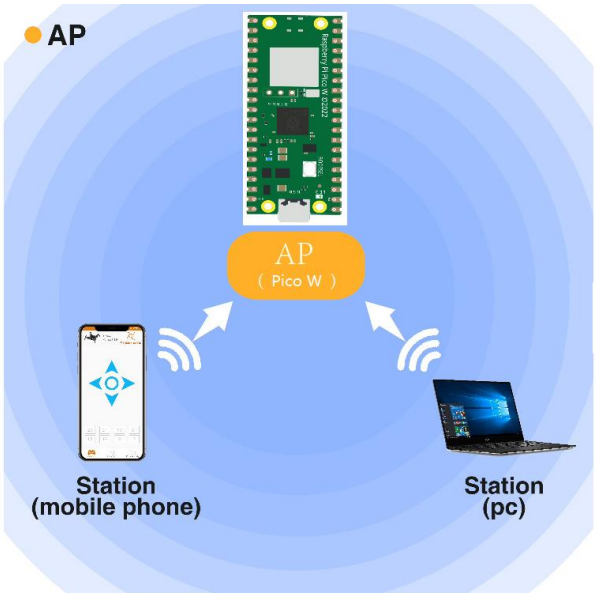
Component List & Circuit

Component List & Circuit are the same as in Section 29.1.

Component knowledge

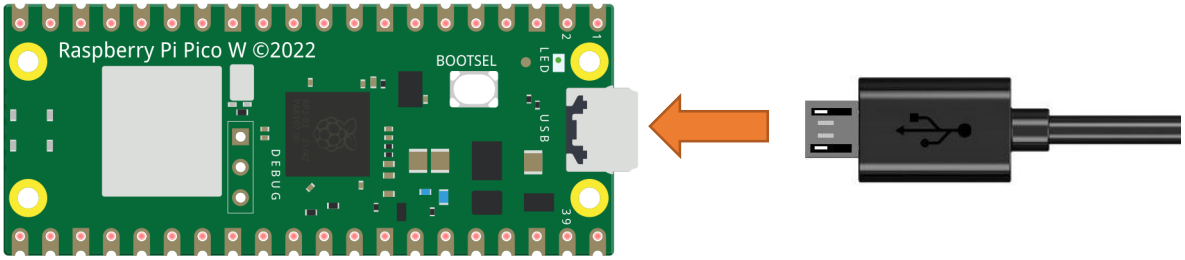
AP mode

When PICO W selects AP mode, it creates a hotspot network that is separated from the Internet and waits for other WiFi devices to connect. As shown in the figure below, PICO W is used as a hotspot. If a mobile phone or PC wants to communicate with PICO W, it must be connected to the hotspot of PICO W. Only after a connection is established with PICO W can they communicate.



Circuit

Connect Pico W to the computer using the USB cable.

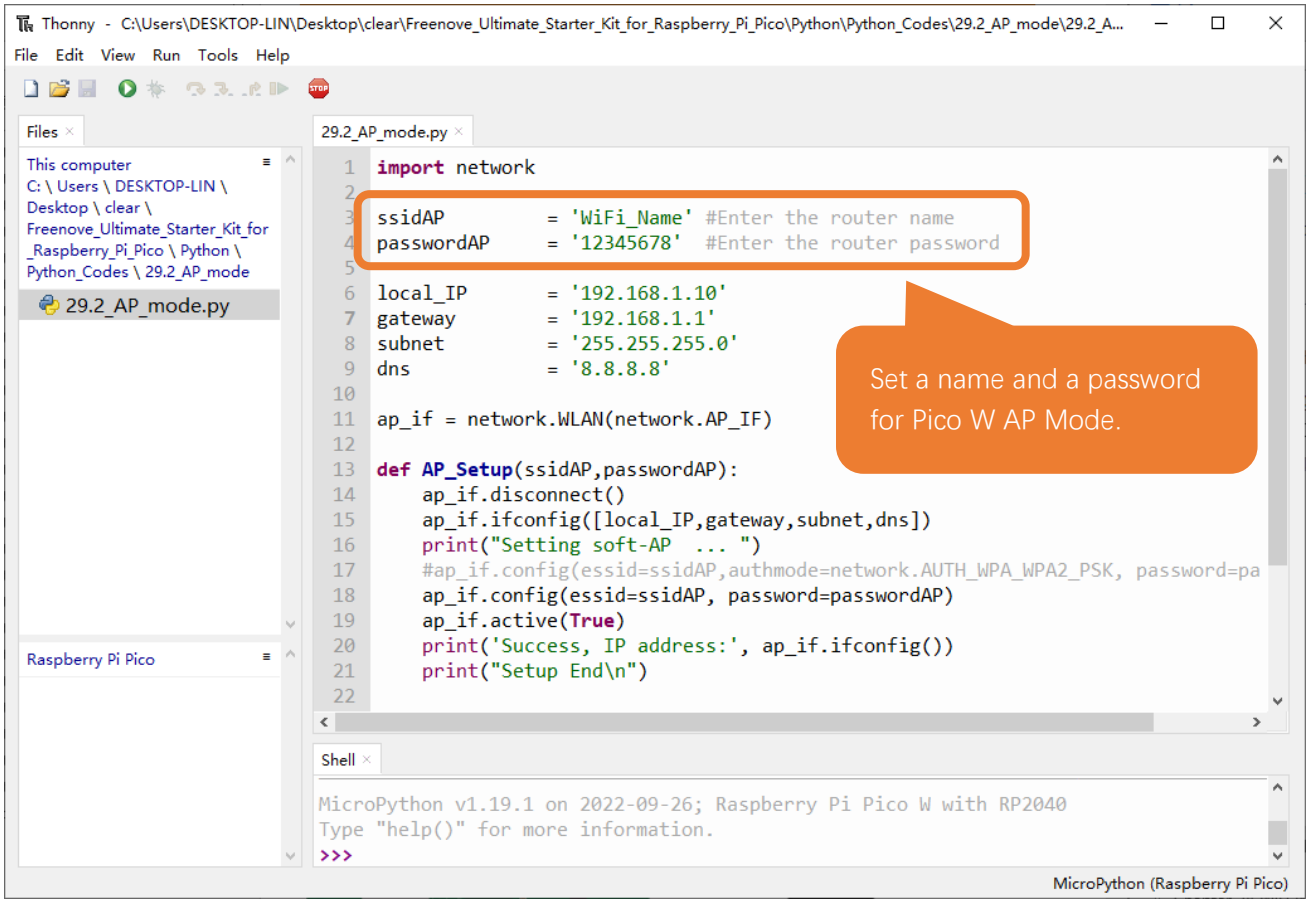


Code

Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

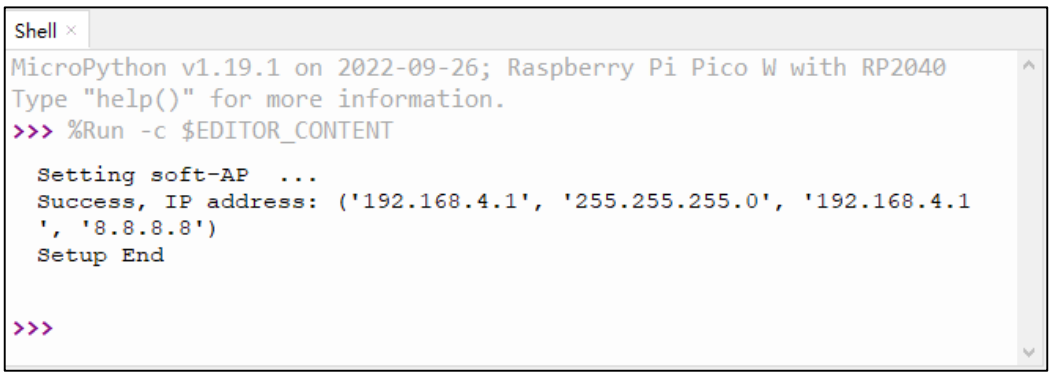
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "29.2_AP_mode". and double click "29.2_AP_mode.py".

29.2_AP_mode

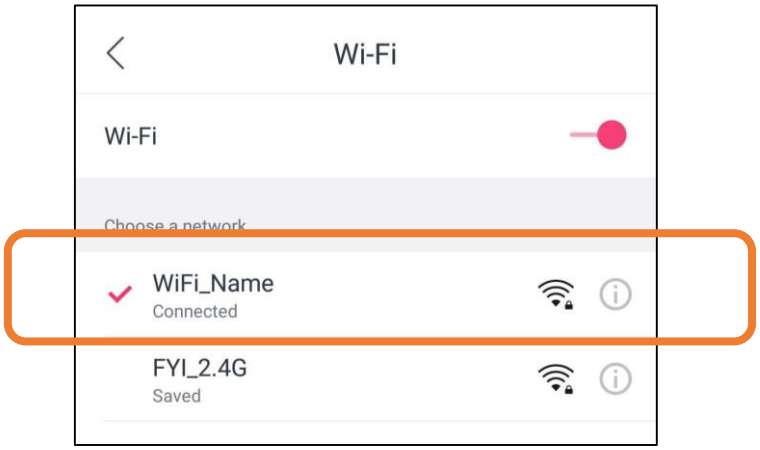


Before the Code runs, you can make any changes to the AP name and password for PICO W in the box as shown in the illustration above. Of course, you can leave it alone by default.

Click "Run current script", open the AP function of PICO W and print the access point information.



Turn on the WiFi scanning function of your phone, and you can see the ssid_AP on PICO W, which is called "WiFi_Name" in this Code. You can enter the password "12345678" to connect it or change its AP name and password by modifying Code.



The following is the program code:

```
import network

ssidAP          = 'WiFi_Name' #Enter the router name
passwordAP      = '12345678'  #Enter the router password

local_IP        = '192.168.1.10'
gateway         = '192.168.1.1'
subnet          = '255.255.255.0'
dns             = '8.8.8.8'

ap_if = network.WLAN(network.AP_IF)

def AP_Setup(ssidAP, passwordAP):
    ap_if.ifconfig([local_IP,gateway,subnet,dns])
    print("Setting soft-AP  ... ")
    ap_if.config(essid=ssidAP, password=passwordAP)
    ap_if.active(True)
    print('Success, IP address:', ap_if.ifconfig())
    print("Setup End\n")

try:
    AP_Setup(ssidAP,passwordAP)
except:
    ap_if.disconnect()
```

Import network module.

```
import network
```

Enter correct AP name and password.

```
3  ssidAP          = 'WiFi_Name' #Enter the router name
4  passwordAP      = '12345678'  #Enter the router password
```

Set PICO W in AP mode.

```
11 ap_if = network.WLAN(network.AP_IF)
```

Configure IP address, gateway and subnet mask for PICO W.

```
14 ap_if.ifconfig([local_IP,gateway,subnet,dns])
```

Turn on an AP in PICO W, whose name is set by ssid_AP and password is set by password_AP.

```
16     ap_if.config(essid=ssidAP, password=passwordAP)
17     ap_if.active(True)
```

If the program is running abnormally, the AP disconnection function will be called.

```
14     ap_if.disconnect()
```

Reference

Class network

Before each use of **network**, please add the statement "**import network**" to the top of the python file.

WLAN(interface_id): Set to WiFi mode.

network.STA_IF: Client, connecting to other WiFi access points

network.AP_IF: Access points, allowing other WiFi clients to connect

active(is_active): With parameters, it is to check whether to activate the network interface; Without parameters, it is to query the current state of the network interface

isconnected(): In AP mode, it returns True if it is connected to the station; otherwise it returns False.

connect(ssid, password): Connecting to wireless network

ssid: WiFiname

password: WiFipassword

config(essid, channel): To obtain the MAC address of the access point or to set the WiFi channel and the name of the WiFi access point.

ssid: WiFi account name

channel: WiFichannel

ifconfig([(ip, subnet, gateway, dns)]): Without parameters, it returns a 4-tuple (ip, subnet_mask, gateway, DNS_server); With parameters, it configures static IP.

ip: IPaddress

subnet_mask: subnet mask

gateway: gateway

DNS_server: DNSserver

disconnect(): Disconnect from the currently connected wireless network

status(): Return the current status of the wireless connection

Project 29.3 AP+Station mode

Component List & Circuit

Component List & Circuit are the same as in Section 29.1.

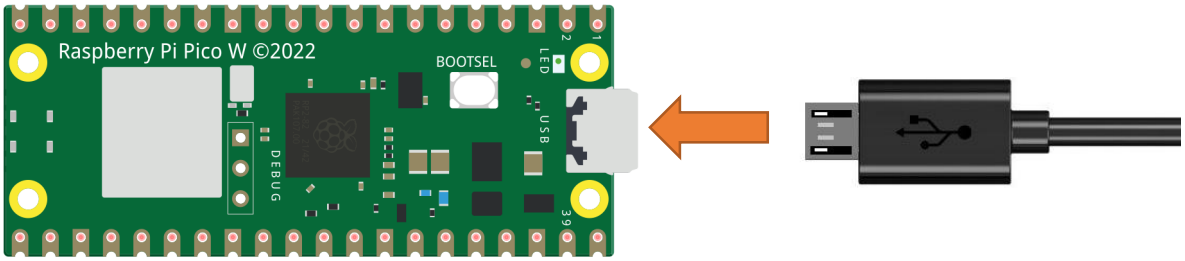
Component knowledge

AP+Station mode

PICO W currently does not support simultaneous use of AP mode and Station mode, so this section can be skipped. In the actual mode configuration, the last configured mode shall prevail.

Circuit

Connect Pico W to the computer using the USB cable.

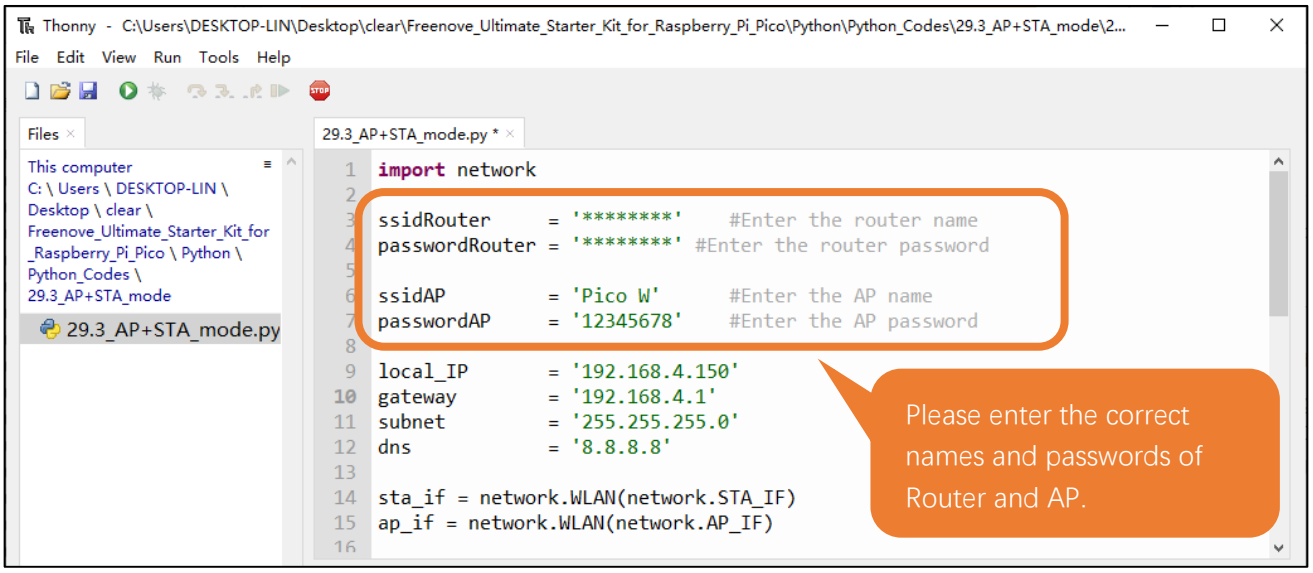


Code

Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "29.3_AP+STA_mode"and double click "29.3_AP+STA_mode.py".

29.3_AP+STA_mode



It is analogous to project 29.1 and project 29.2. Before running the Code, you need to modify ssidRouter, passwordRouter, ssidAP and passwordAP shown in the box of the illustration above.

After making sure that the code is modified correctly, click "Run current script" and the "Shell" will display as follows:

```
Shell
>>>

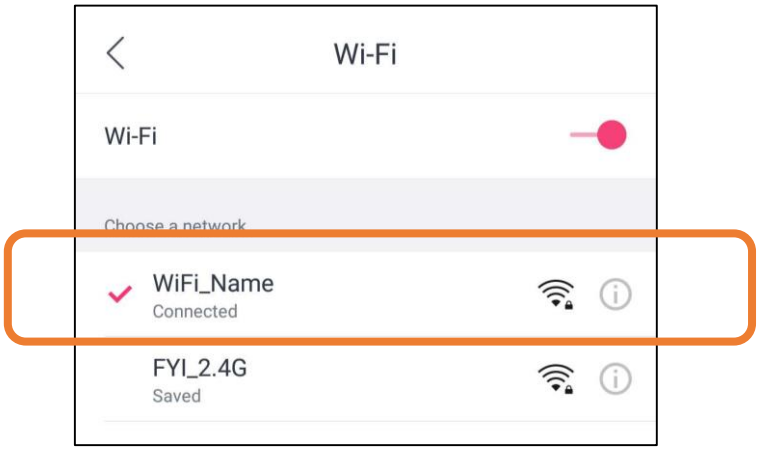
MicroPython v1.19.1 on 2022-09-26; Raspberry Pi Pico W with RP2040
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

  Setting soft-AP  ...
  Success, IP address: ('192.168.4.150', '192.168.4.1', '255.255.255.0', '8.8.8.8')
  Setup End

  Setting soft-STA  ...
  Connected, IP address: ('192.168.1.26', '255.255.255.0', '192.168.1.1', '8.8.8.8')
  Setup End

>>>
```

Turn on the WiFi scanning function of your phone, and you can see the ssidAP on PICO W.



The following is the program code:

```
import network

ssidRouter     = '********' #Enter the router name
passwordRouter = '********' #Enter the router password

ssidAP         = 'WiFi_Name'#Enter the AP name
passwordAP     = '12345678' #Enter the AP password

local_IP       = '192.168.4.150'
gateway        = '192.168.4.1'
subnet         = '255.255.255.0'
dns            = '8.8.8.8'

sta_if = network.WLAN(network.STA_IF)
ap_if = network.WLAN(network.AP_IF)

def STA_Setup(ssidRouter,passwordRouter):
    print("Setting soft-STA  ... ")
    if not sta_if.isconnected():
        print('connecting to',ssidRouter)
        sta_if.active(True)
        sta_if.connect(ssidRouter,passwordRouter)
        while not sta_if.isconnected():
            pass
    print('Connected, IP address:', sta_if.ifconfig())
    print("Setup End")

def AP_Setup(ssidAP,passwordAP):
    ap_if.ifconfig([local_IP,gateway,subnet,dns])
    print("Setting soft-AP  ... ")
```

```
31      ap_if.config(essid=ssidAP , password=passwordAP)
32      ap_if.active(True)
33      print('Success, IP address:', ap_if.ifconfig())
34      print("Setup End\n")
35
36  try:
37      AP_Setup(ssidAP,passwordAP)
38      STA_Setup(ssidRouter,passwordRouter)
39  except:
40      sta_if.disconnect()
41      ap_if.disconnect()
```

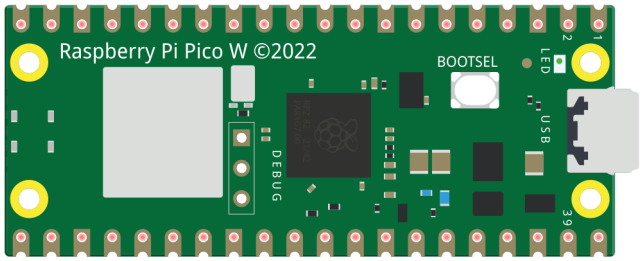
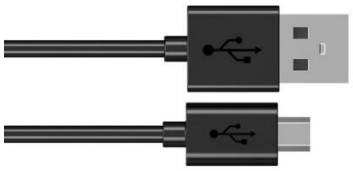
Chapter 30 TCP/IP (Only for Pico W/Pico 2W)

If you have Pico in your hand, please change it to Pico W before continuing to learn.
In this chapter, we will introduce how PICO W implements network communications based on TCP/IP protocol. There are two roles in TCP/IP communication, namely Server and Client, which will be implemented respectively with two projects in this chapter.

Project 30.1 as Client

In this section, PICO W is used as Client to connect Server on the same LAN and communicate with it.

Component List

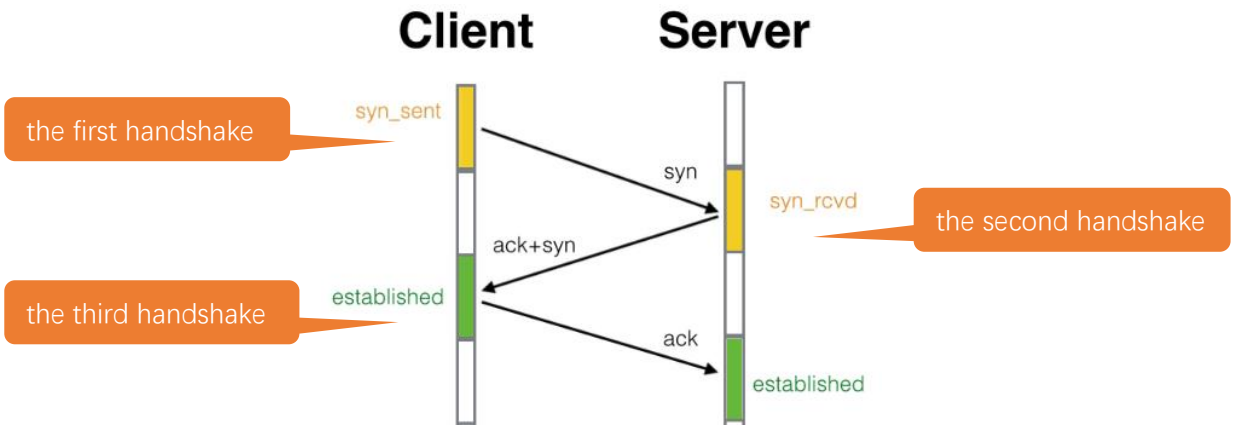
Raspberry Pi Pico W(or Pico 2W) x1 	Micro USB Wire x1 

Component knowledge

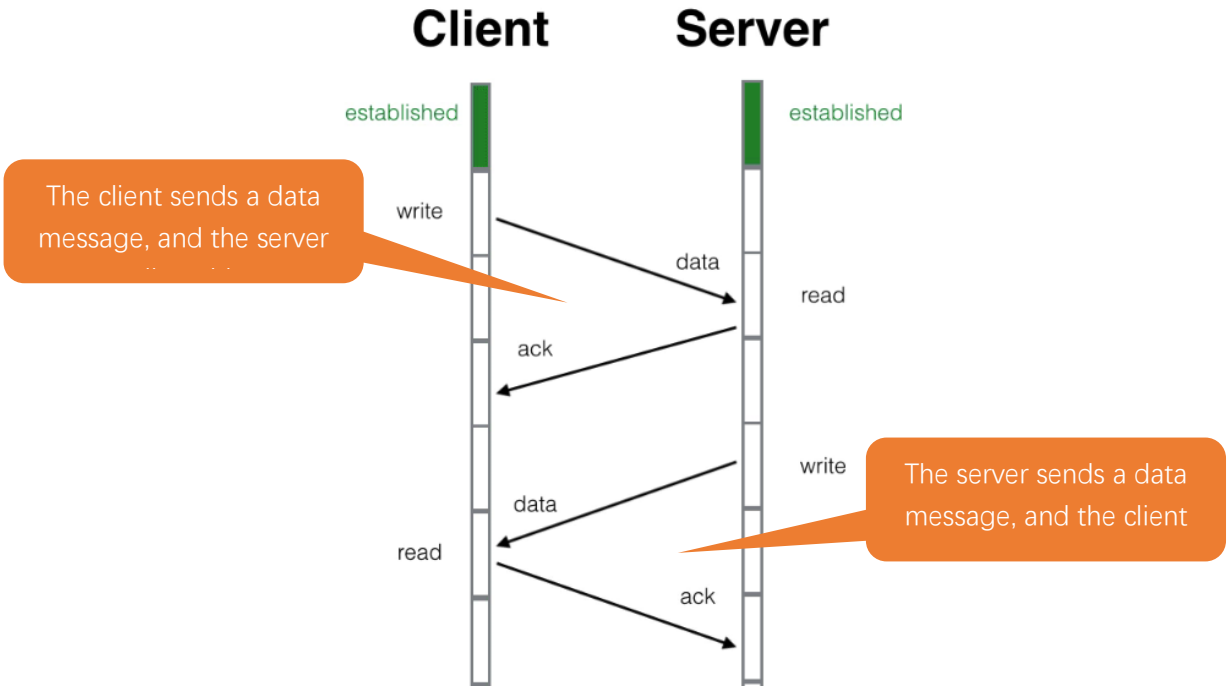
TCP connection

Before transmitting data, TCP needs to establish a logical connection between the sending end and the receiving end. It provides reliable and error-free data transmission between the two computers. In the TCP connection, the client and the server must be clarified. The client sends a connection request to the server, and each time such a request is proposed, a "three-time handshake" is required.

Three-time handshake: In the TCP protocol, during the preparation phase of sending data, the client and the server interact three times to ensure the reliability of the connection, which is called "three-time handshake".
The first handshake, the client sends a connection request to the server and waits for the server to confirm.
The second handshake, the server sends a response back to the client informing that it has received the connection request.
The third handshake, the client sends a confirmation message to the server again to confirm the connection.



TCP is a connection-oriented, low-level transmission control protocol. After TCP establishes a connection, the client and server can send and receive messages to each other, and the connection will always exist as long as the client or server does not initiate disconnection. Each time one party sends a message, the other party will reply with an ack signal.



Install Processing

In this tutorial, we use Processing to build a simple TCP/IP communication platform.

If you have not installed Processing, you can download it by clicking https://processing.org/download/. You can choose an appropriate version to download according to your PC system.



Unzip the downloaded file to your computer. Click "processing.exe" as the figure below to run this software.



Use Server mode for communication

Open the "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Codes/Micropython_Codes/30.1_TCP_as_Client/sketchWiFi/sketchWiFi.pde**". Click "Run".

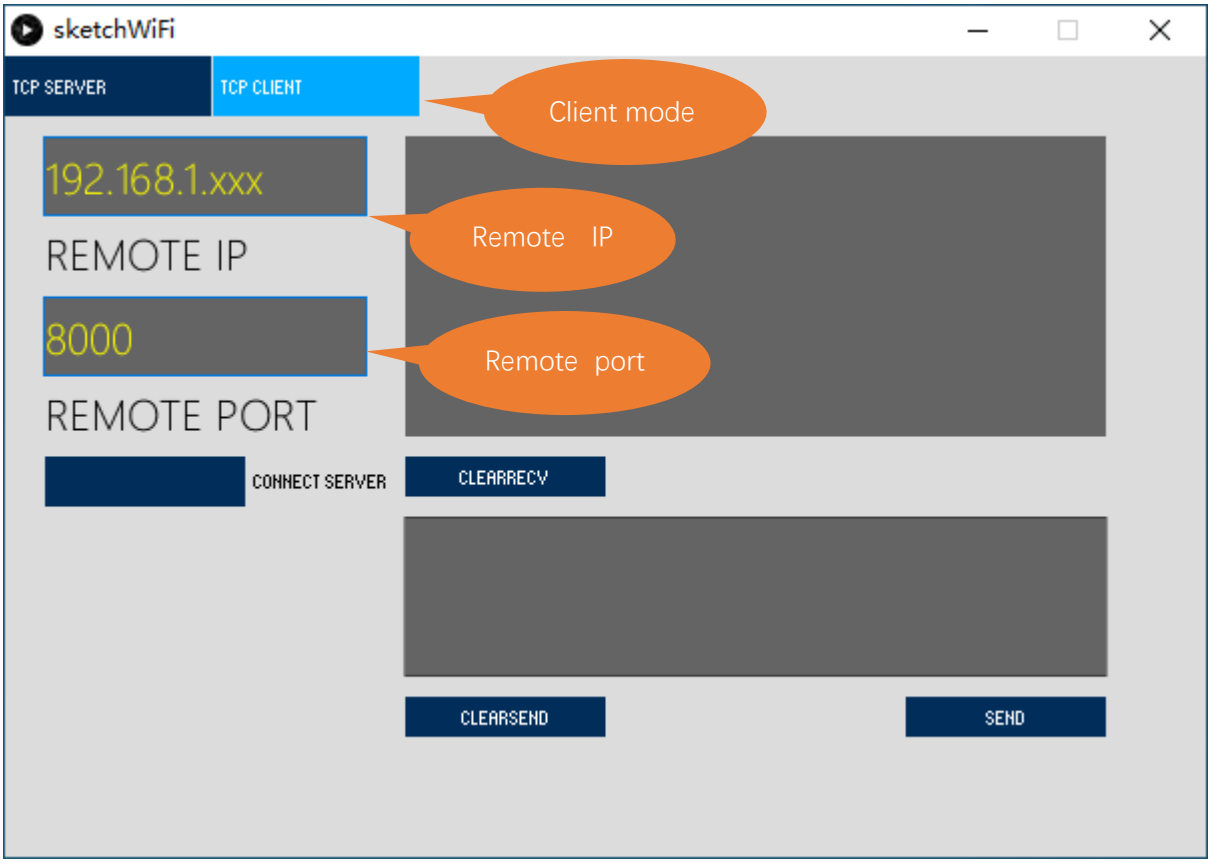


The new pop-up interface is as follows. If PICO W is used as Client, select TCP SERVER mode for sketchWiFi.



When sketchWiFi selects TCP SERVER mode, PICO W Code needs to be changed according to sketchWiFi's displaying of LOCAL IP or LOCAL PORT.

If PICO W serves as Server, select TCP CLIENT mode for sketchWiFi.



When sketchWiFi selects TCP CLIENT mode, the LOCAL IP and LOCAL PORT of sketchWiFi need to be changed according to the IP address and port number printed by the serial monitor.

Mode selection: select **Server mode/Client mode**.

IP address: In Server mode, this option does not need to be filled in, and the computer will automatically obtain the IP address.

In Client mode, fill in the remote IP address to be connected.

Port number: In Server mode, fill in a port number for client devices to make an access connection.

In client mode, fill in port number given by the Server devices to make an access connection.

Start button: In server mode, push the button, and then the computer will serve as Server and open a port number for Client to make access connection. During this period, the computer will keep monitoring.

In client mode, before pushing the button, please make sure the server is on, remote IP address and remote port number is correct; push the button, and the computer will make access connection to the remote port number of the remote IP as a Client.

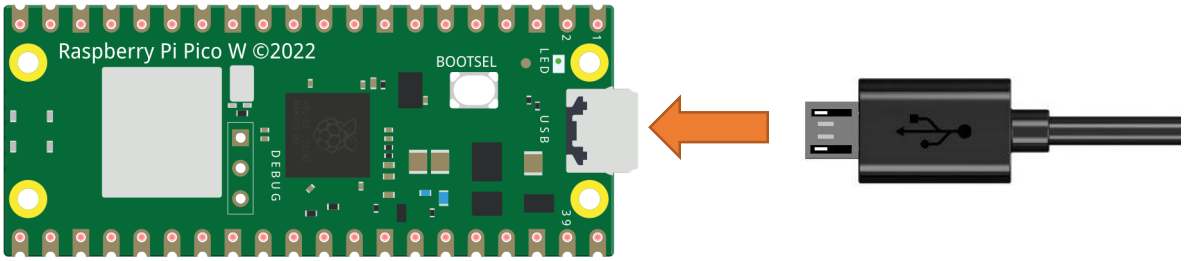
clear receive: clear out the content in the receiving text box

clear send: clear out the content in the sending text box

Sending button: push the sending button, the computer will send the content in the text box to others.

Circuit

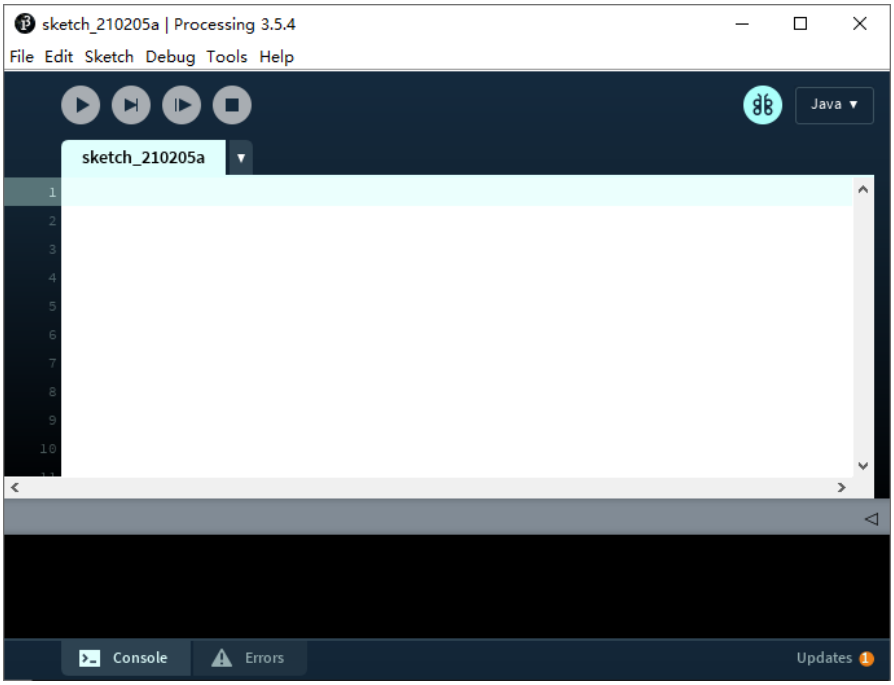
Connect Pico W to the computer using the USB cable.



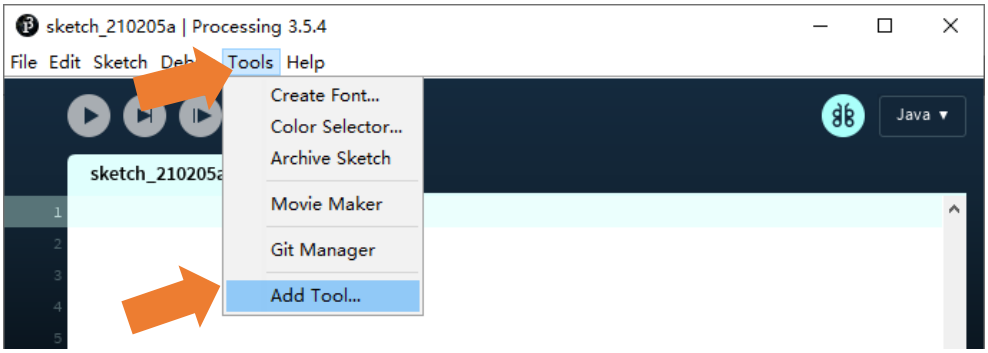
Code

If you have not installed "ControlP5", please follow the following steps to continue the installation, if you have installed, please skip this section.

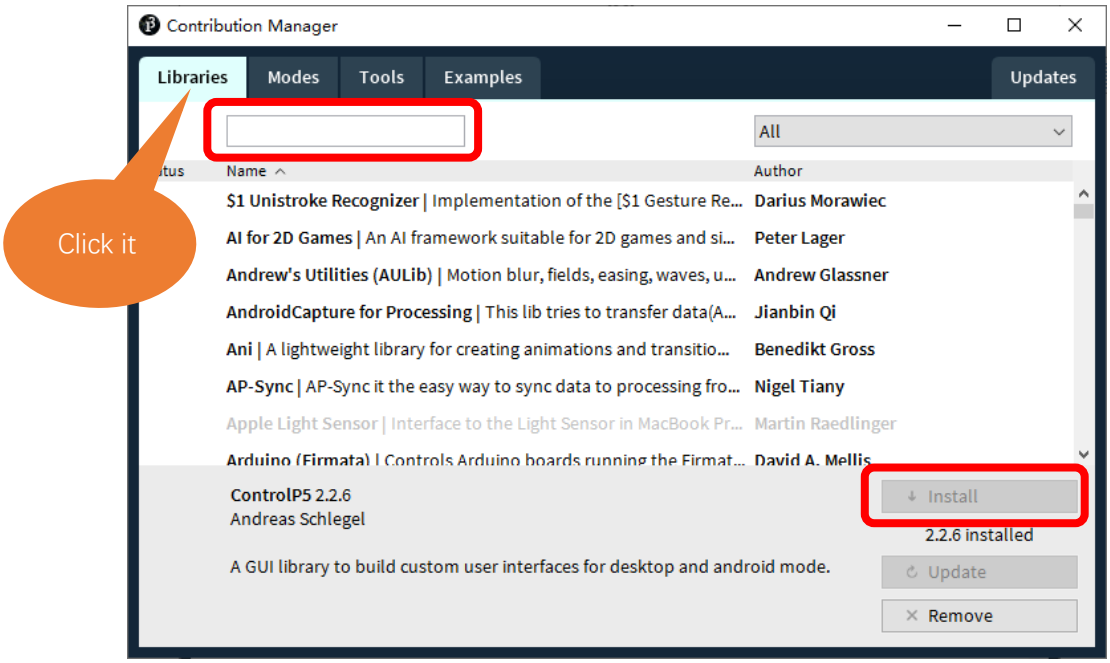
Open Processing.



Click Add Tool under Tools.

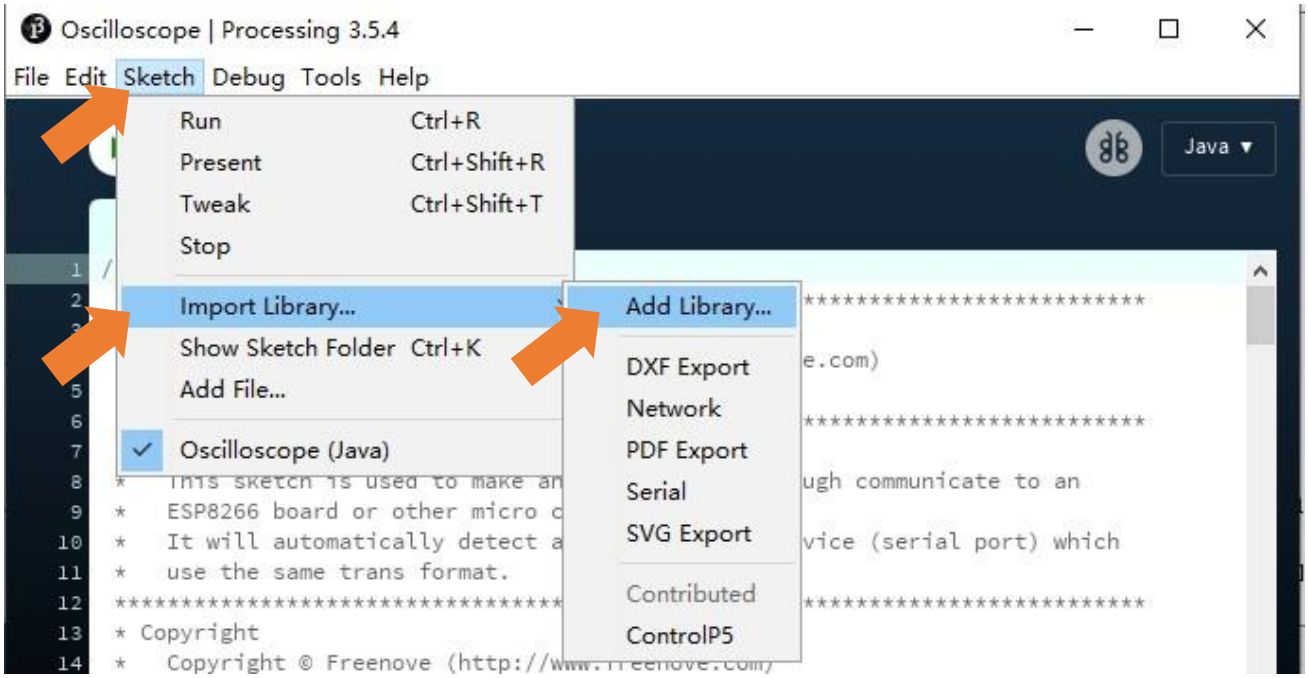


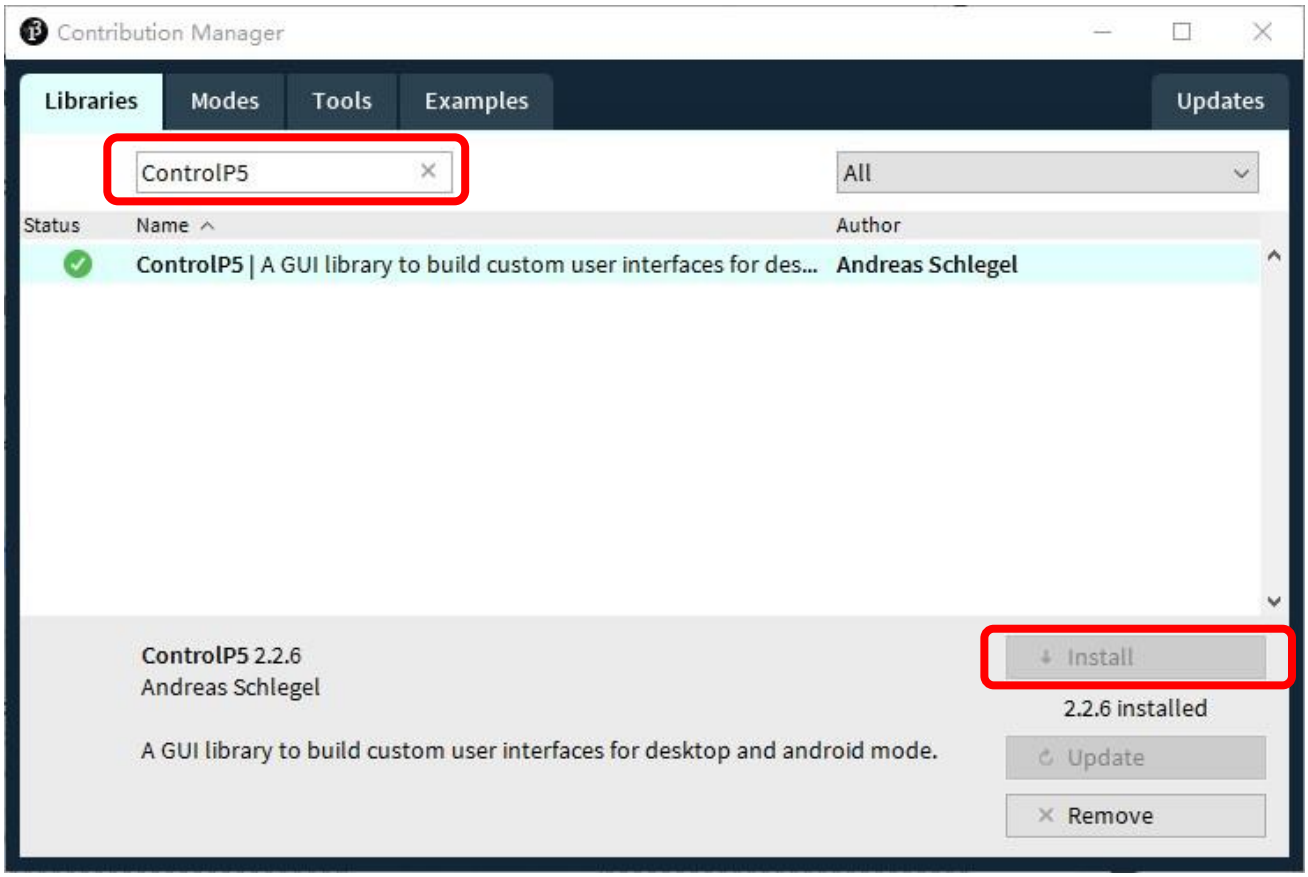
Select Libraries in the pop-up window.



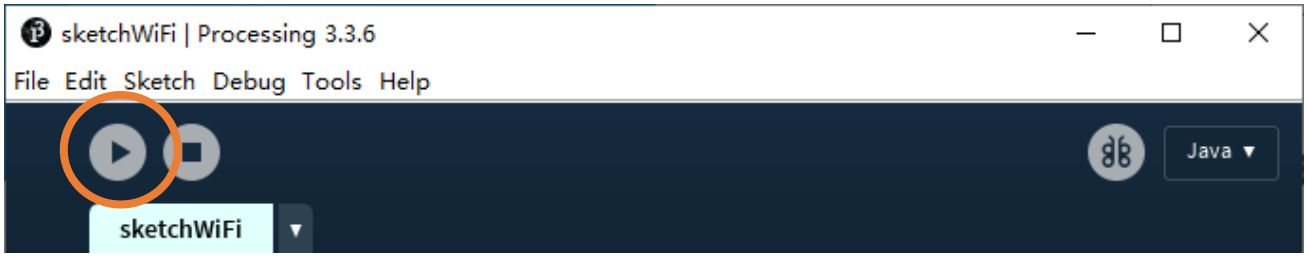
Input "ControlP5" in the searching box, and then select the option as below. Click "Install" and wait for the installation to finish.

You can also click Add Library under 'Import Library' under 'Sketch'.

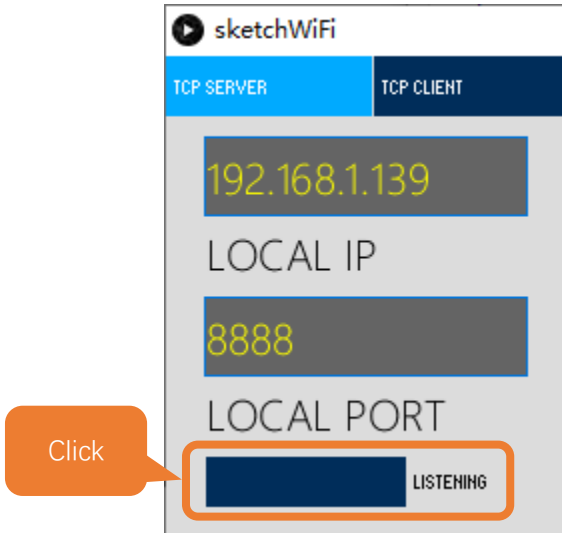




Before running the Code, please open "sketchWiFi.pde." first, and click "Run".



The newly pop up window will use the computer's IP address by default and open a data monitor port. Click "Listening"。

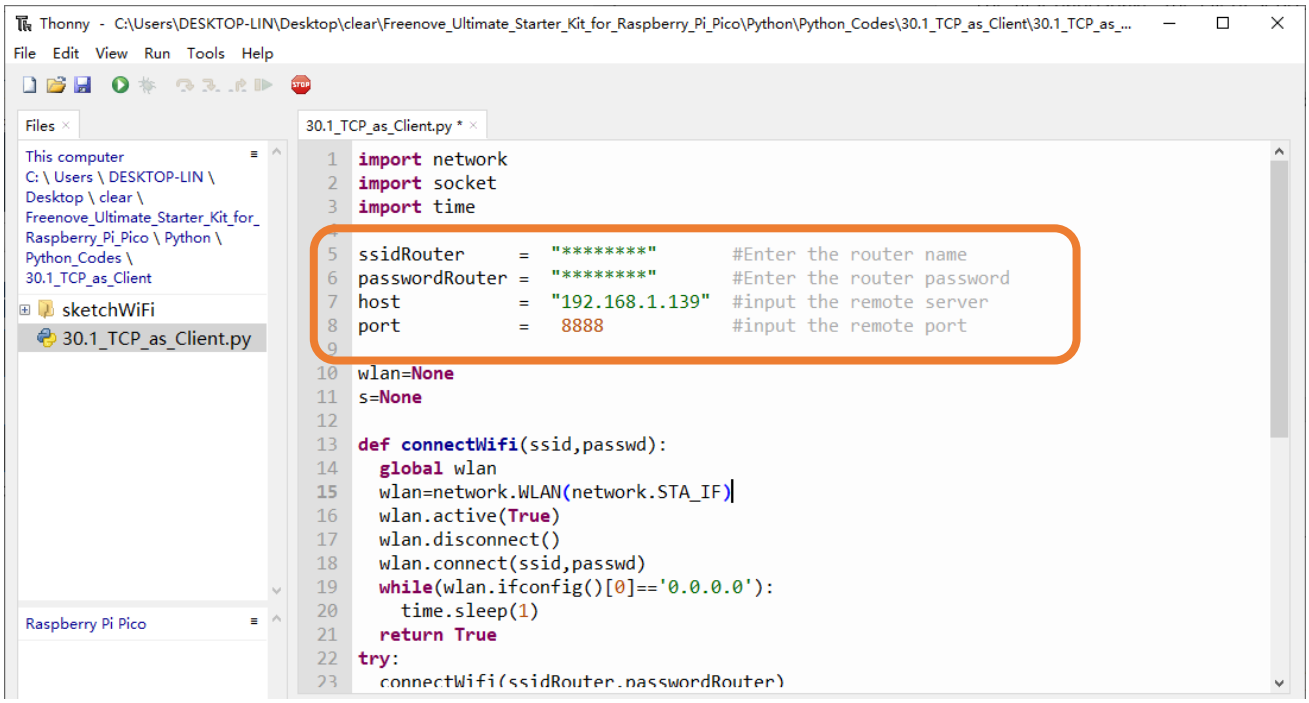


Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

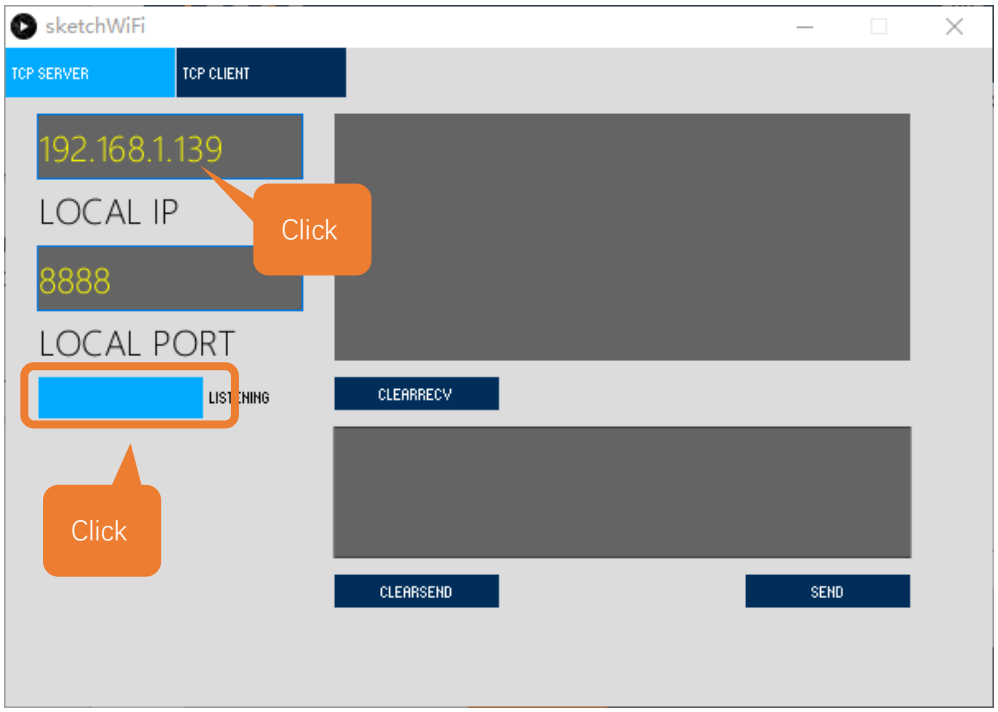
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "30.1_TCP_as_Client" and double click "30.1_TCP_as_Client.py".

Before clicking "Run current script", please modify the name and password of your router and fill in the "host" and "port" according to the **IP information in processing app** shown in the box below:

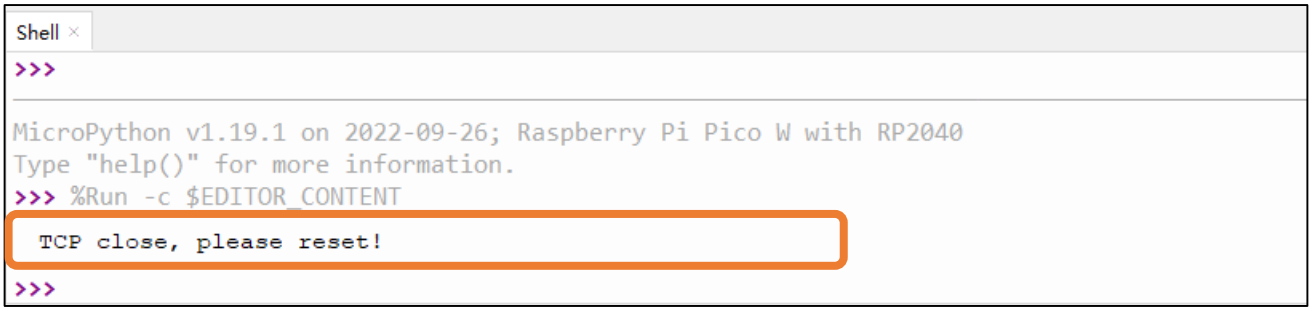
30.1_TCP_as_Client



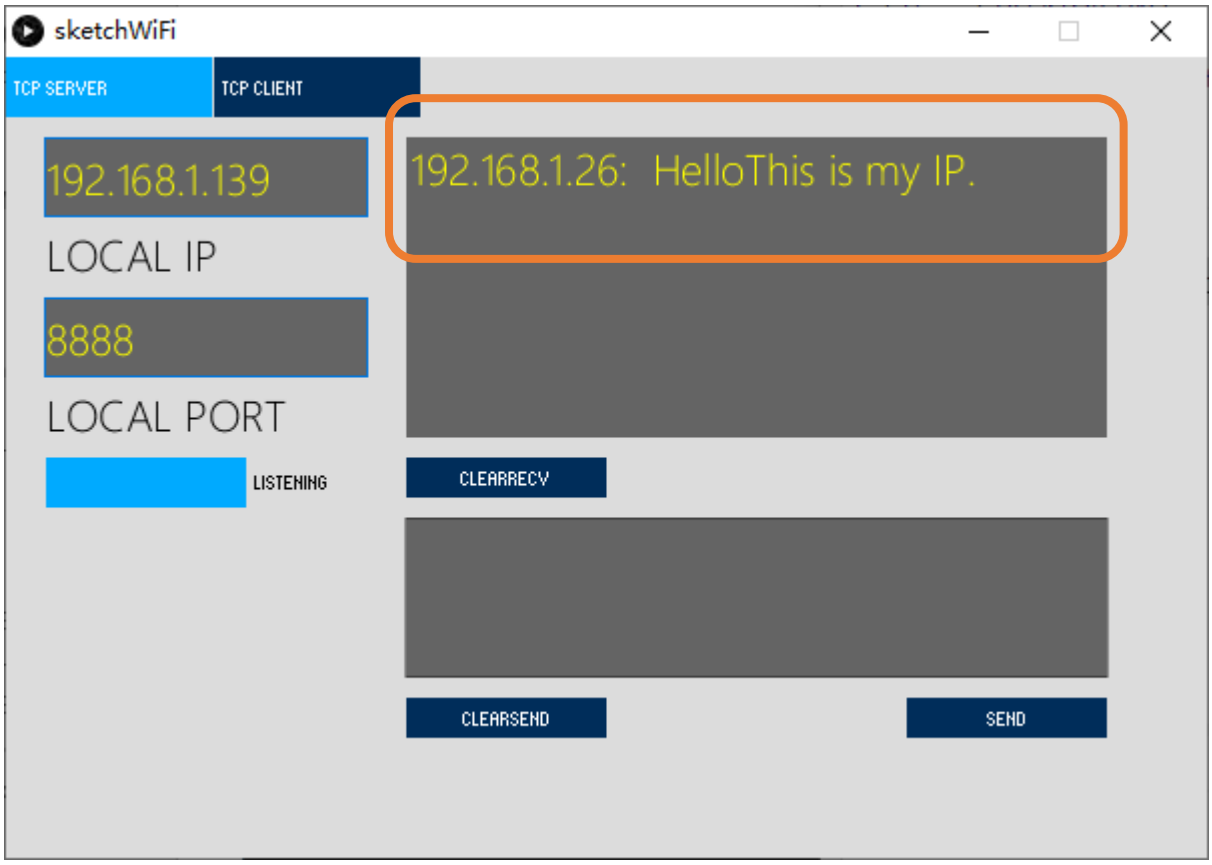
Click "Run current script" and in "Shell", you can see PICO W automatically connects to sketchWiFi.



If you do not click “Listening” for sketchWiFi, PICO W will fail to connect and will print information as follows:



PICO W connects with TCP SERVER, and TCP SERVER receives messages from PICO W, as shown in the figure below.



At this point, you can send data to Pico W through sketchWiFi. Pico W will send the received data back to sketchWiFi after receiving it.

The following is the program code:

```
1  import network
2  import socket
3  import time
4
5  ssidRouter     = "********"    #Enter the router name
6  passwordRouter = "********"    #Enter the router password
7  host           = "********"    #input the remote server
8  port           =  8888         #input the remote port
9
```

```
wlan=None
s=None

def connectWifi(ssid,passwd):
    global wlan
    wlan= network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.disconnect()
    wlan.connect(ssid,passwd)
    while(wlan.ifconfig()[0]=='0.0.0.0'):
        time.sleep(1)
    return True
try:
    connectWifi(ssidRouter,passwordRouter)
    s = socket.socket()
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.connect((host,port))
    print("TCP Connected to:", host, ":", port)
    s.send('Hello')
    s.send('This is my IP.')
    while True:
        data = s.recv(1024)
        if(len(data) == 0):
            print("Close socket")
            s.close()
            break
        print(data)
        ret=s.send(data)
except:
    print("TCP close, please reset!")
    if (s):
        s.close()
    wlan.disconnect()
    wlan.active(False)
```

Import network、socket、time modules.

```
import network
import socket
import time
```

Enter the actual router name, password, remote server IP address, and port number.

```
ssidRouter     =  "********"    #Enter the router name
passwordRouter =  "********"    #Enter the router password
host           =  "********"    #input the remote server
port           =   8888         #input the remote port
```

Connect specified Router until it is successful.

```
13  def connectWifi(ssid,passwd):
14      global wlan
15      wlan= network.WLAN(network.STA_IF)
16      wlan.active(True)
17      wlan.disconnect()
18      wlan.connect(ssid,passwd)
19      while(wlan.ifconfig()[0]=='0.0.0.0'):
20          time.sleep(1)
21      return True
```

Connect router and then connect it to remote server.

```
23      connectWifi(ssidRouter,passwordRouter)
24      s = socket.socket()
25      s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
26      s.connect((host,port))
27      print("TCP Connected to:", host, ":", port)
```

Send messages to the remote server, receive the messages from it and print them out, and then send the messages back to the server.

```
28      s.send('Hello')
29      s.send('This is my IP.')
30      while True:
31          data = s.recv(1024)
32          if(len(data) == 0):
33              print("Close socket")
34              s.close()
35              break
36          print(data)
37          ret=s.send(data)
```

If an exception occurs in the program, for example, the remote server is shut down, execute the following program, turn off the socket function, and disconnect the WiFi.

```
39      print("TCP close, please reset!")
40      if (s):
41          s.close()
42      wlan.disconnect()
43      wlan.active(False)
```

Reference

Class socket

Before each use of **socket**, please add the statement "**import socket**" to the top of the python file.

socket([af, type, proto]): Create a socket.

af: address

- **socket.AF_INET**: IPv4
- **socket.AF_INET6**: IPv6

type: type

- **socket.SOCK_STREAM** : TCP stream
- **socket.SOCK_DGRAM** : UDP datagram
- **socket.SOCK_RAW** : Original socket
- **socket.SO_REUSEADDR** : socket reusable

proto: protocol number

- **socket.IPPROTO_TCP**: TCPmode
- **socket.IPPROTO_UDP**: UDPmode

socket.setsockopt(level, optname, value): Set the socket according to the options.

Level: Level of socket option

- **socket.SOL_SOCKET**: Level of socket option. By default, it is 4095.

optname: Options of socket

- **socket.SO_REUSEADDR**: Allowing a socket interface to be tied to an address that is already in use.

value: The value can be an integer or a bytes-like object representing a buffer.

socket.connect(address): To connect to server.

Address: Tuple or list of the server's address and port number

send(bytes): Send data and return the bytes sent.

recv(bufsize): Receive data and return a bytes object representing the data received.

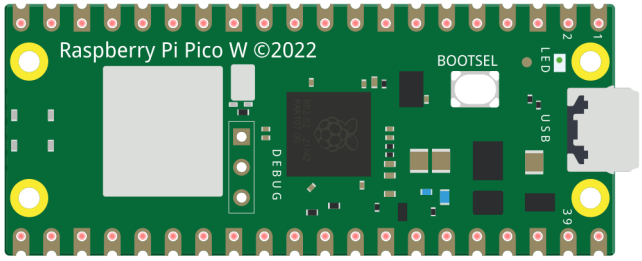
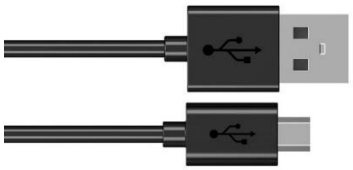
close(): Close socket.

To learn more please visit: http://docs.micropython.org/en/latest/

Project 30.2 as Server

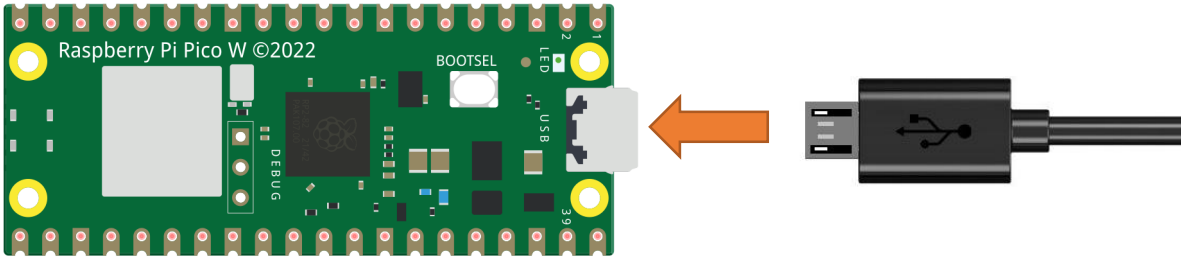
In this section, PICO W is used as a Server to wait for the connection and communication with Client on the same LAN.

Component List

Raspberry Pi Pico W(or Pico 2W) x1 	Micro USB Wire x1 

Circuit

Connect Pico W to the computer using the USB cable.



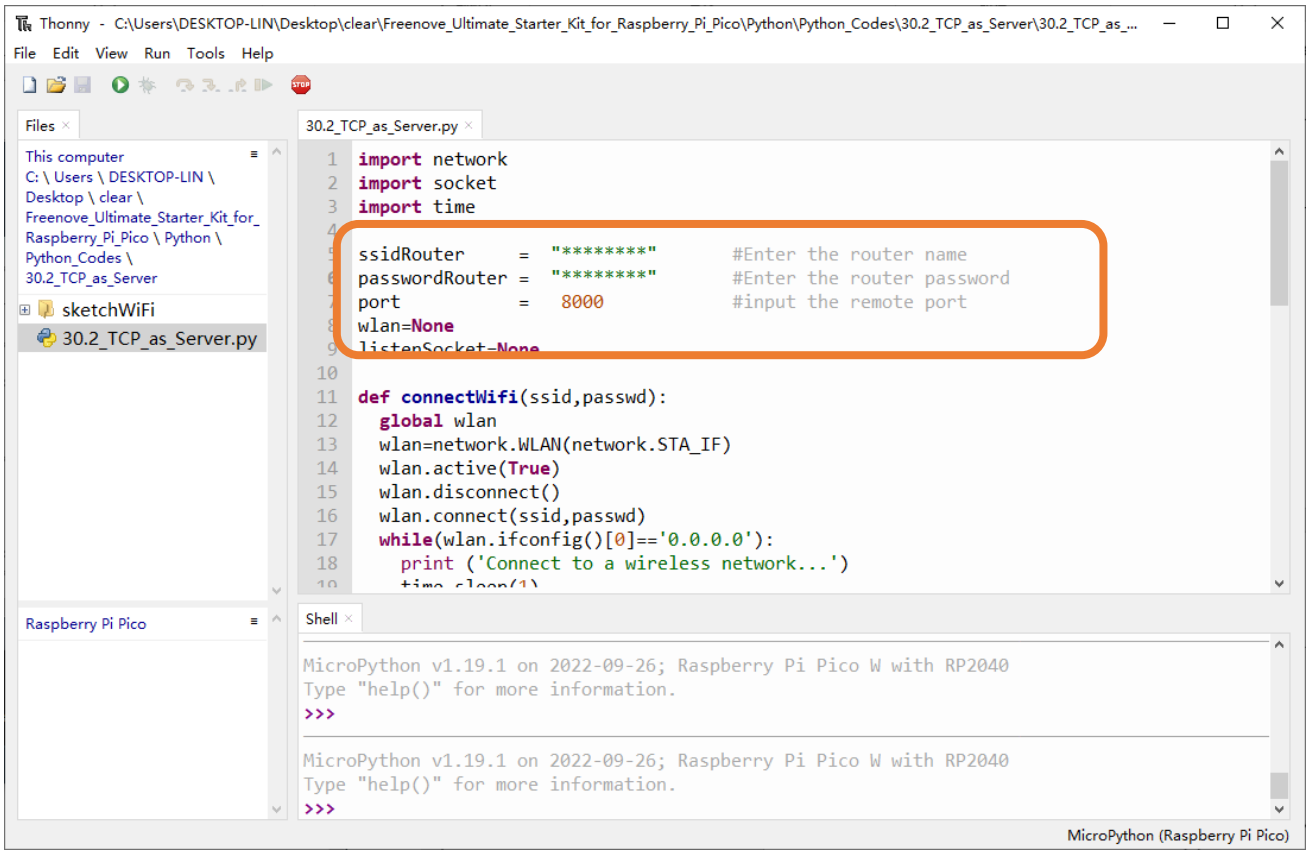
Code

Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

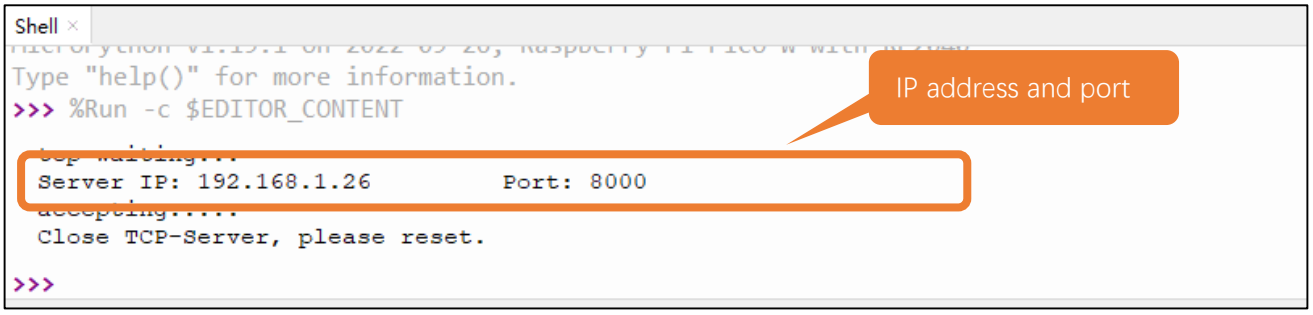
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "30.2_TCP_as_Server" and double click "30.2_TCP_as_Server.py".

Before clicking "Run current script", please modify the name and password of your router shown in the box below.

30.2_TCP_as_Server



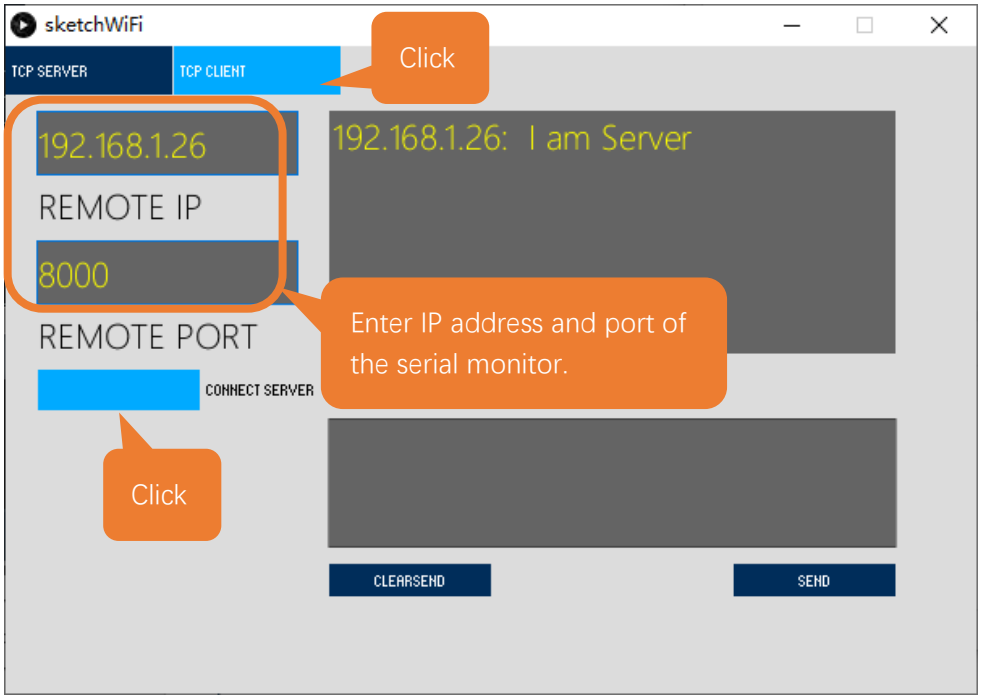
After making sure that the router's name and password are correct, click "Run current script" and in "Shell", you can see a server opened by the PICO W waiting to connecting to other network devices.



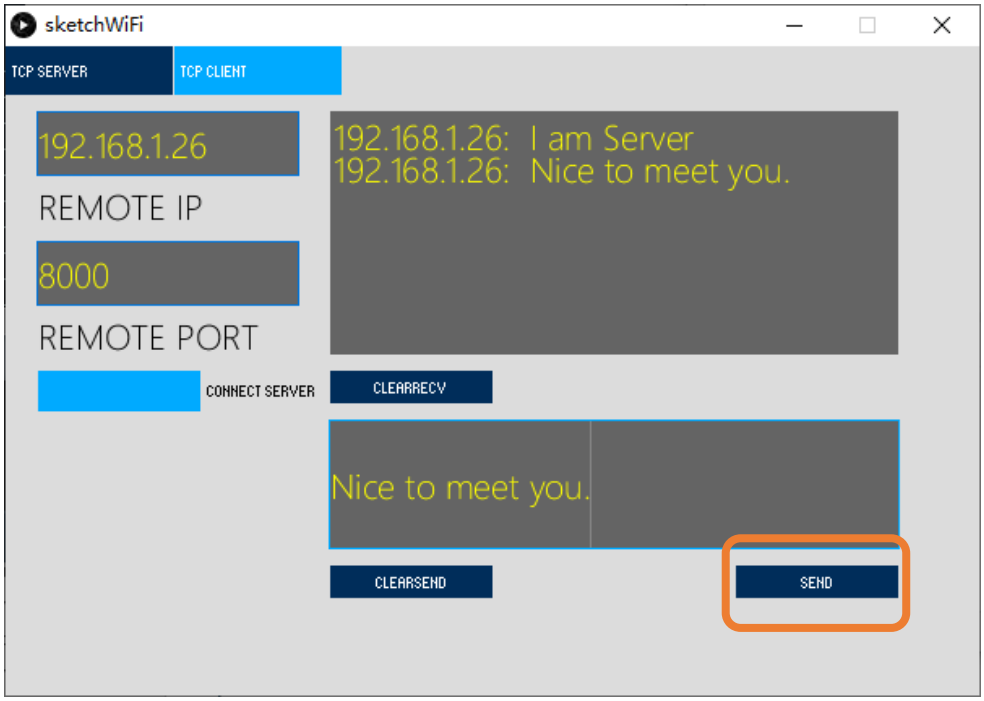
Processing:

Open the "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Codes/MicroPython_Codes/30.2_TCP_as_Server/sketchWiFi/sketchWiFi.pde**".

Based on the message printed in "Shell", enter the correct IP address and port when processing, and click to establish a connection with PICO W to communicate.



You can enter any information in the "Send Box" of sketchWiFi. Click "Send" and PICO W will print the received messages to "Shell" and send them back to sketchWiFi.



```
Shell
Close TCP-Server, please reset.
>>> %Run -c $EDITOR_CONTENT

  tcp waiting...
  Server IP: 192.168.1.26         Port: 8000
  accepting.....
  ('192.168.1.139', 59648) connected
  b'Nice to meet you.'
```

The following is the program code:

```
1  import network
2  import socket
3  import time
4
5  ssidRouter     = "********"       #Enter the router name
6  passwordRouter = "********"       #Enter the router password
7  port           =  8000            #input the remote port
8  wlan           = None
9  listenSocket   = None
10
11 def connectWifi(ssid,passwd):
12   global wlan
13   wlan=network.WLAN(network.STA_IF)
14   wlan.active(True)
15   wlan.disconnect()
16   wlan.connect(ssid,passwd)
17   while(wlan.ifconfig()[0]=='0.0.0.0'):
18     time.sleep(1)
19   return True
20
21 try:
22   connectWifi(ssidRouter,passwordRouter)
23   ip=wlan.ifconfig()[0]
24   listenSocket = socket.socket()
25   listenSocket.bind((ip,port))
26   listenSocket.listen(1)
27   listenSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
28   print('tcp waiting...')
29   while True:
30     print("Server IP:",ip,"\tPort:",port)
31     print("accepting.....")
32     conn,addr = listenSocket.accept()
33     print(addr,"connected")
34     break
35   conn.send('I am Server')
```

```
36    while True:
37      data = conn.recv(1024)
38      if(len(data) == 0):
39        print("close socket")
40        listenSocket.close()
41        wlan.disconnect()
42        wlan.active(False)
43        break
44      else:
45        print(data)
46        ret = conn.send(data)
47  except:
48    print("Close TCP-Server, please reset.")
49    if(listenSocket):
50      listenSocket.close()
51    wlan.disconnect()
52    wlan.active(False)
```

Call function connectWifi() to connect to router and obtain the dynamic IP that it assigns to PICO W.

```
22    connectWifi(ssidRouter,passwordRouter)
23    ip=wlan.ifconfig()[0]
```

Open the socket server, bind the server to the dynamic IP, and open a data monitoring port.

```
24    listenSocket = socket.socket()
25    listenSocket.bind((ip,port))
26    listenSocket.listen(1)
27    listenSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

Print the server's IP address and port, monitor the port and wait for the connection of other network devices.

```
29    while True:
30      print("Server IP:",ip,"\tPort:",port)
31      print("accepting.....")
32      conn,addr = listenSocket.accept()
33      print(addr,"connected")
34      break
```

Each time receiving data, print them in "Shell" and send them back to the client.

```
36    while True:
37      data = conn.recv(1024)
38      if(len(data) == 0):
39        print("close socket")
40        listenSocket.close()
41        wlan.disconnect()
42        wlan.active(False)
43        break
44      else:
45        print(data)
46        ret = conn.send(data)
```

If the client is disconnected, close the server and disconnect WiFi.

```
except:
  print("Close TCP-Server, please reset.")
  if(listenSocket):
    listenSocket.close()
  wlan.disconnect()
  wlan.active(False)
```

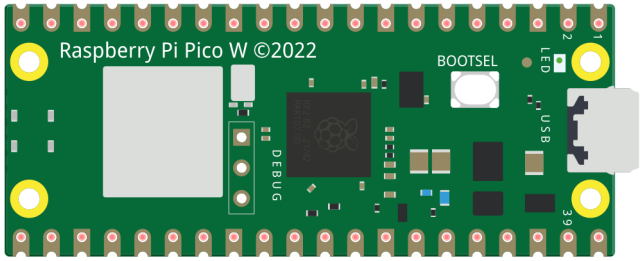
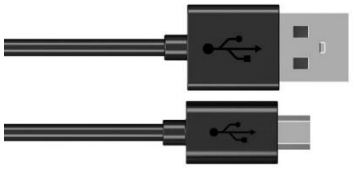
Chapter 31 Control LED with Web (Only for Pico W/Pico 2W)

If you have Pico in your hand, please change it to Pico W before continuing to learn.
In this chapter, we will use PICO W to make a simple smart home. We will learn how to control LED lights through web pages.

Project 31.1 Control_LED_through_Web

In this project, we need to build a Web Service and then use PICO W to control the LED through the Web browser of the phone or PC. Through this example, you can remotely control the appliances in your home to achieve smart home.

Component List

Raspberry Pi Pico W(or Pico 2W) x1 	Micro USB Wire x1 

Component knowledge

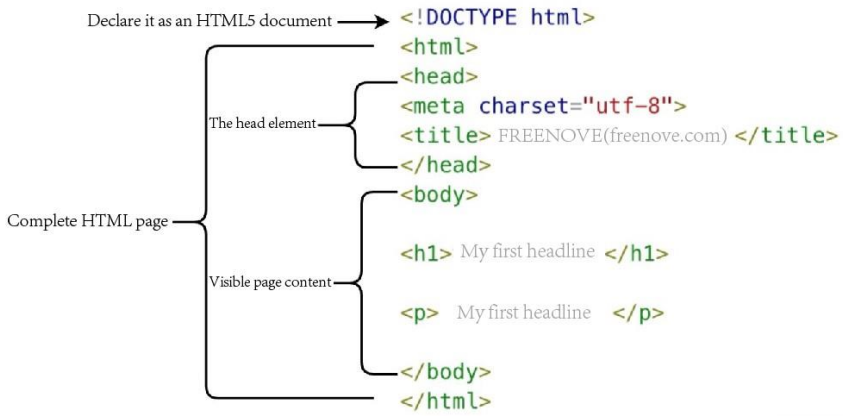
HTML

Hypertext Markup Language (HTML) is a standard Markup Language for creating web pages. It includes a set of tags that unify documents on the network and connect disparate Internet resources into a logical whole.HTML text is descriptive text composed of HTML commands that describe text, graphics, animations, sounds, tables, links, etc. The extension of the HTML file is HTM or HTML. Hypertext is a way to organize information. It uses hyperlinks to associate words and charts in Text with other information media. These related information media may be in the same Text, other files, or files located on a remote computer. This way of organizing information connects the information resources distributed in different places, which is convenient for people to search and retrieve information.

The nature of the Web is hypertext Markup Language (HTML), which can be combined with other Web technologies (e.g., scripting languages, common gateway interfaces, components, etc.) to create powerful Web pages. Thus, Hypertext Markup Language (HTML) is the foundation of World Wide Web (Web) programming, that is, the World Wide Web is based on hypertext. Hypertext Markup Language is called hypertext Markup language because the text contains so-called "hyperlink" points.

You can build your own WEB site using HTML, which runs on the browser and is parsed by the browser.

Example analysis is shown in the figure below:



<!DOCTYPE html>: Declare it as an HTML5 document

<html>: Is the root element of an HTML page

<head>: Contains meta data for the document, such as < meta charset="utf-8"> Define the web page encoding format to UTF-8.

<title>: Notes the title of the document

<body>: Contains visible page content

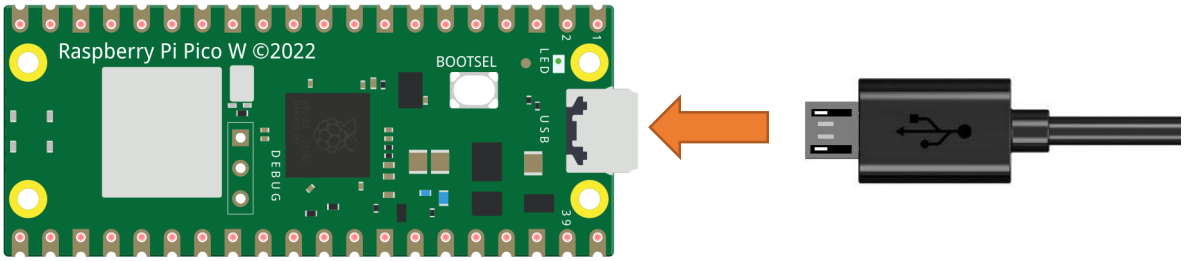
<h1>: Define a big heading

<p>: Define a paragraph

For more information, please visit: https://developer.mozilla.org/en-US/docs/Web/HTML

Circuit

Connect Pico W to the computer using the USB cable.

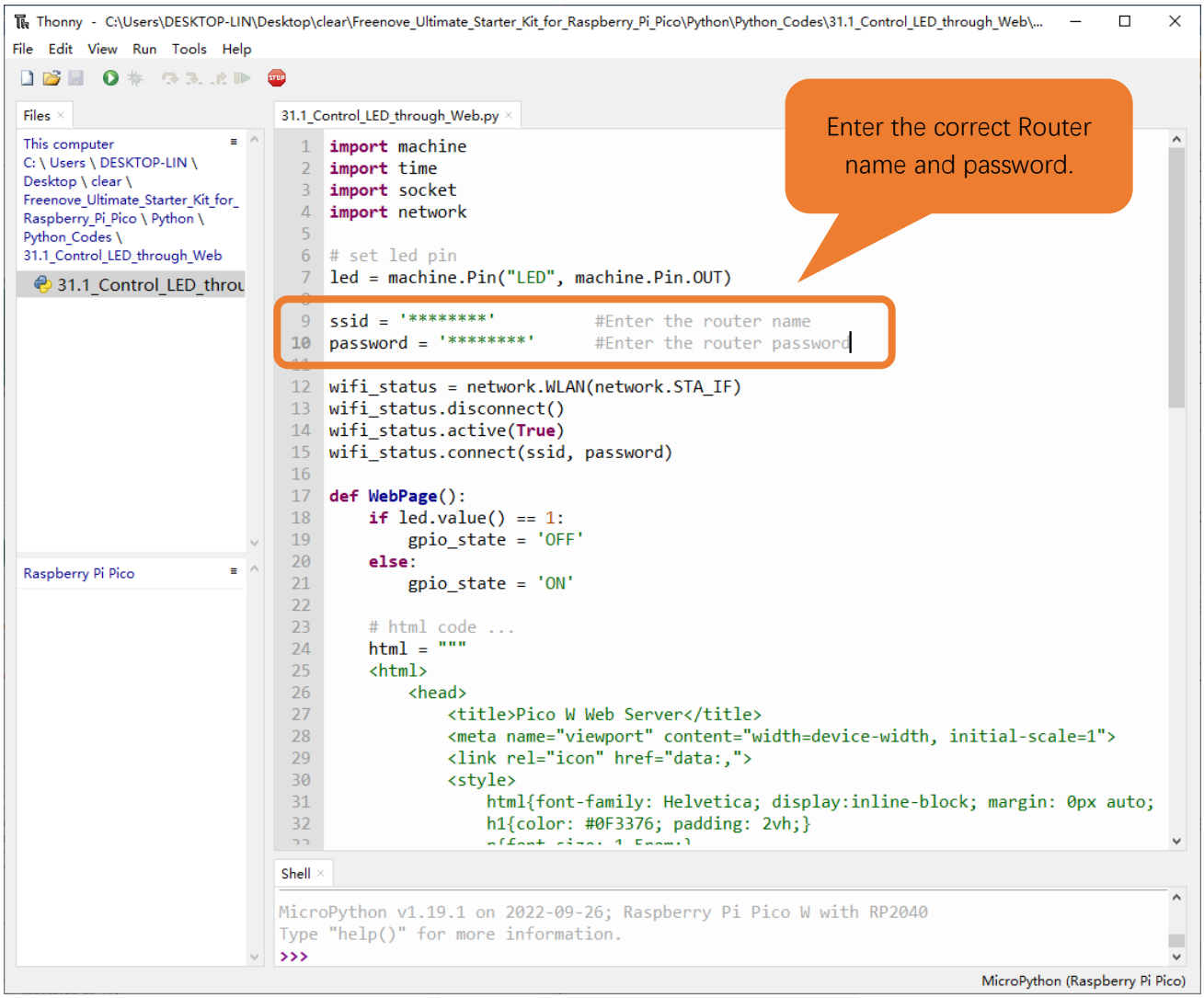


Code

Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

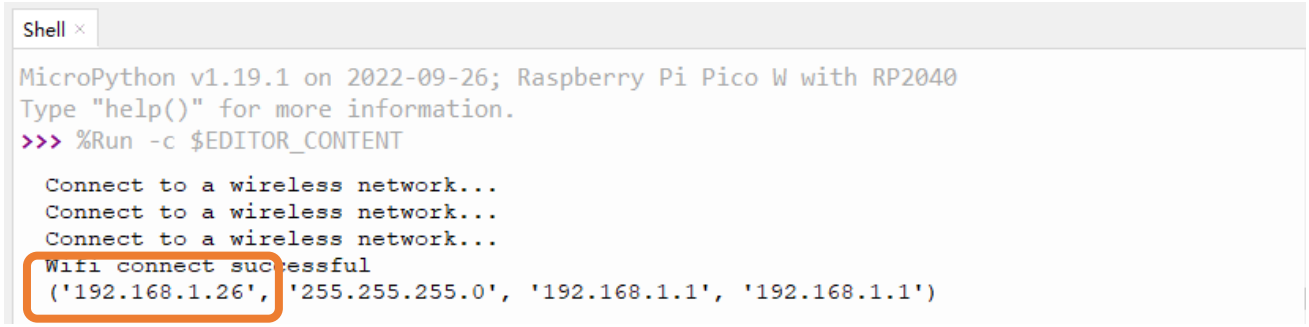
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "31.1_Control_LED_through_Web", and double click "31.1_Control_LED_through_Web.py".

31.1_Control_LED_through_Web



Because the names and passwords of routers in various places are different, before the Code runs, users need to enter the correct router's name and password in the box as shown in the illustration above.

After making sure the router name and password are entered correctly, compile and upload codes to PICO W, wait for PICO W to connect to your router and print the IP address assigned by the router to PICO W in "Shell".



When PICO W successfully connects to "ssid", "Shell" displays the IP address assigned to PICO W by the router. Access http://192.168.1.26 in a computer browser on the LAN. As shown in the following figure:



You can click the corresponding button to control the LED on and off.

The following is the program code:

```
import machine
import time
import socket
import network

# set led pin
led = machine.Pin("LED", machine.Pin.OUT)

ssid = '********'              #Enter the router name
password = '********'          #Enter the router password

wifi_status = network.WLAN(network.STA_IF)
wifi_status.disconnect()
wifi_status.active(True)
wifi_status.connect(ssid, password)

def WebPage():
    if led.value() == 1:
        gpio_state = 'On'
    else:
        gpio_state = 'OFF'

    # html code ...
    html = """
    <html>
        <head>
            <title>Pico W Web Server</title>
```

```
            <meta name="viewport" content="width=device-width, initial-scale=1">
            <link rel="icon" href="data:,">
            <style>
                html{font-family: Helvetica; display:inline-block; margin: 0px auto; text-align: center;}
                h1{color: #0F3376; padding: 2vh;}
                p{font-size: 1.5rem;}
                button{display: inline-block; background-color: #4286f4; border: none;border-radius: 4px; color: white; padding: 16px 40px; text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}
                button2{background-color: #4286f4;}
            </style>
        </head>
        <body> <h1>Pico W Web Server</h1>
          <p>GPIO state: <strong>""" + gpio_state + """</strong></p>
          <p><a href="/?led=on"><button class="button">ON</button></a></p>
          <p><a href="/?led=off"><button class="button button2">OFF</button></a></p>
        </body>
    </html>
    """
    return html

# check wifi connected
while wifi_status.isconnected() == False:
    time.sleep(1)
    print('Connect to a wireless network...')
# if connected
print('Wifi connect successful')
print(wifi_status.ifconfig())


s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 80))
s.listen(5)
try:
    while True:
        conn, addr = s.accept()
        #print('Connection: %s' % str(addr))
        req = conn.recv(1024)
        req = str(req)
        print('Connect = %s' % req)
        led_on = req.find('/?led=on')
        led_off = req.find('/?led=off')
        if led_on == 6:
```

```
69                print('LED ON')
70                led.value(1)
71            else:
72                print('LED OFF')
73                led.value(0)
74            if led.value() == 1:
75                gpio_state = 'On'
76            else:
77                gpio_state = 'OFF'
78            response = WebPage()
79            conn.send('HTTP/1.1 200 OK\n')
80            conn.send('Content-Type: text/html\n')
81            conn.send('Connection: close\n\n')
82            conn.sendall(response)
83            conn.close()
84    except:
85        conn.close()
86        s.close()
87        pass
```

Import socket module and Import network module.

```
3   import socket
4   import network
```

Enter correct AP name and password.

```
3   ssid = '********'            #Enter the router name
4   password = '********'        #Enter the router password
```

Set PICO W in Station mode and connect it to your router.

```
12  wifi_status = network.WLAN(network.STA_IF)
13  wifi_status.disconnect()
14  wifi_status.active(True)
15  wifi_status.connect(ssid, password)
```

"Shell" displays the IP address assigned to PICO W.

```
53  print(wifi_status.ifconfig())
```

Click the button on the web page to control the LED light on and off.

```
68            if led_on == 6:
69                print('LED ON')
70                led.value(0)
71            else:
72                print('LED OFF')
73                led.value(1)
74            if led.value() == 1:
75                gpio_state = 'OFF'
76            else:
77                gpio_state = 'ON'
```

Chapter 32 Bluetooth (Only for Pico W/Pico 2W)

Note: As the official MicroPtyhon firmware supports only Bluetooth Low Energy (BLE) at the time of this writing, code for classic Bluetooth is not included in this tutorial.

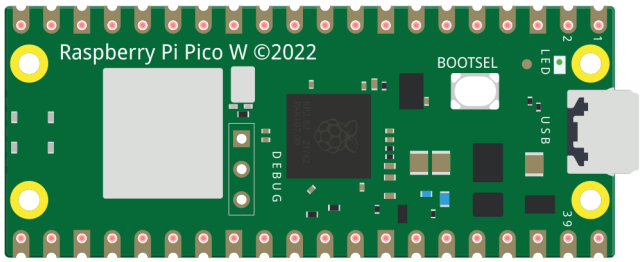
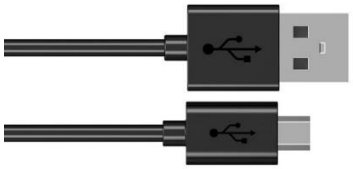
In June 2023, Raspberry Pi Official has updated to add Bluetooth support to the Pico W.

Pico W's Bluetooth 5.2 supports Bluetooth Classic and Bluetooth Low Energy (BLE) functionality. At the beginning of thie chapter, we will learn the Pico W's Bluetooth function.

If you have Pico in your hand, please change it to Pico W before continuing to learn.

Project 32.1 Bluetooth Low Energy Data Passthrough

Component List

Raspberry Pi Pico W(or Pico 2W) x1 	Micro USB Wire x1 

Component knowledge

Pico W's wireless functionality is provided by the Infineon CYW43439 device, which contains a 2.4 GHz radio providing both 802.11n Wi-Fi and Bluetooth 5.2, supporting Bluetooth Classic and Bluetooth Low Energy (BLE) functionality.

For simple data transfer, there are two modes:

Master mode

A device works in master mode can connect to one or more slave devices.

We can search and select the slave devices nearby to connect.

When a device initiates a connection request in master mode, it requires information about other Bluetooth devices, including their addresses and pairing keys.

Once the devices are paired, a direct connection can be established.

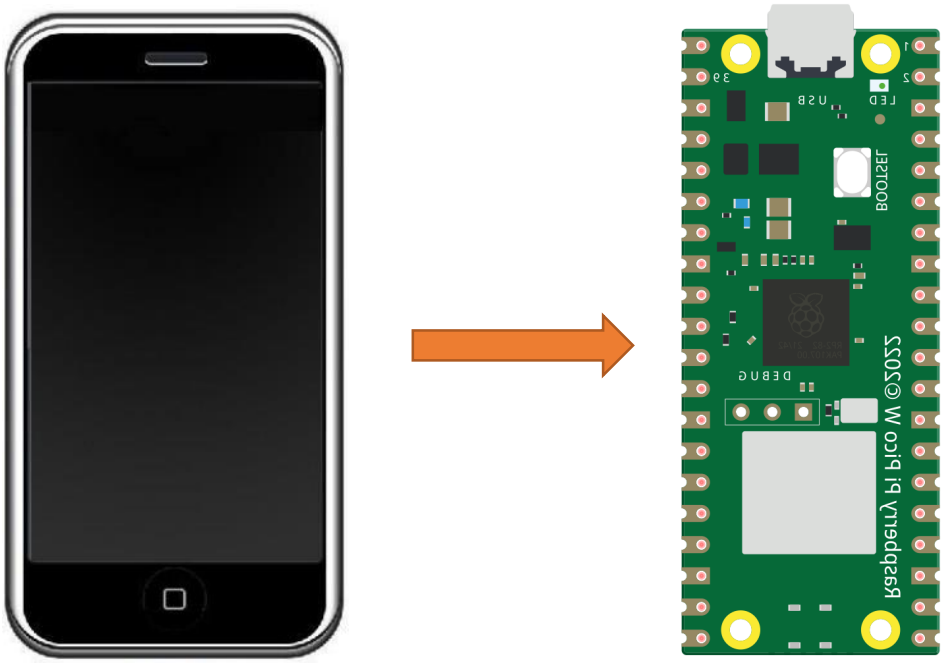
Slave mode

A Bluetooth module operating in slave mode can only receive connection requests from a master device and cannot actively initiate connections.

After establishing a connection with a master device, it can either send or receive data.

Bluetooth devices can interact with each other, with one in master mode and the other in slave mode. During data communication, the master device searches for and selects nearby devices to connect with. Once a connection is made, data can be exchanged between the devices.

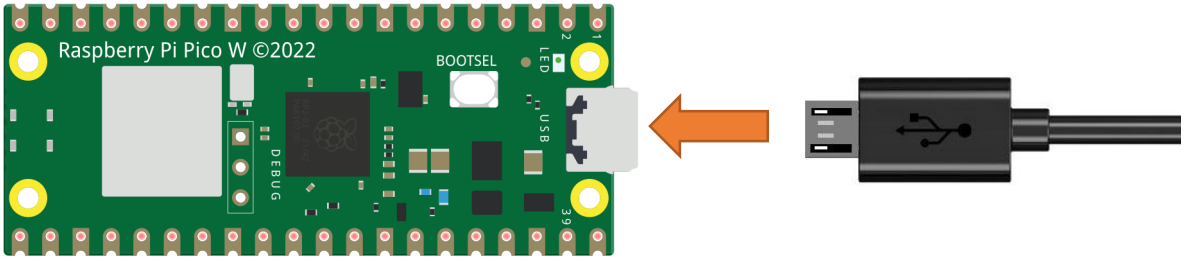
In the case of data exchange between a smartphone and a Raspberry Pi Pico W, the smartphone typically operates in master mode, while the Raspberry Pi Pico W functions in slave mode.



Master Slave

Circuit

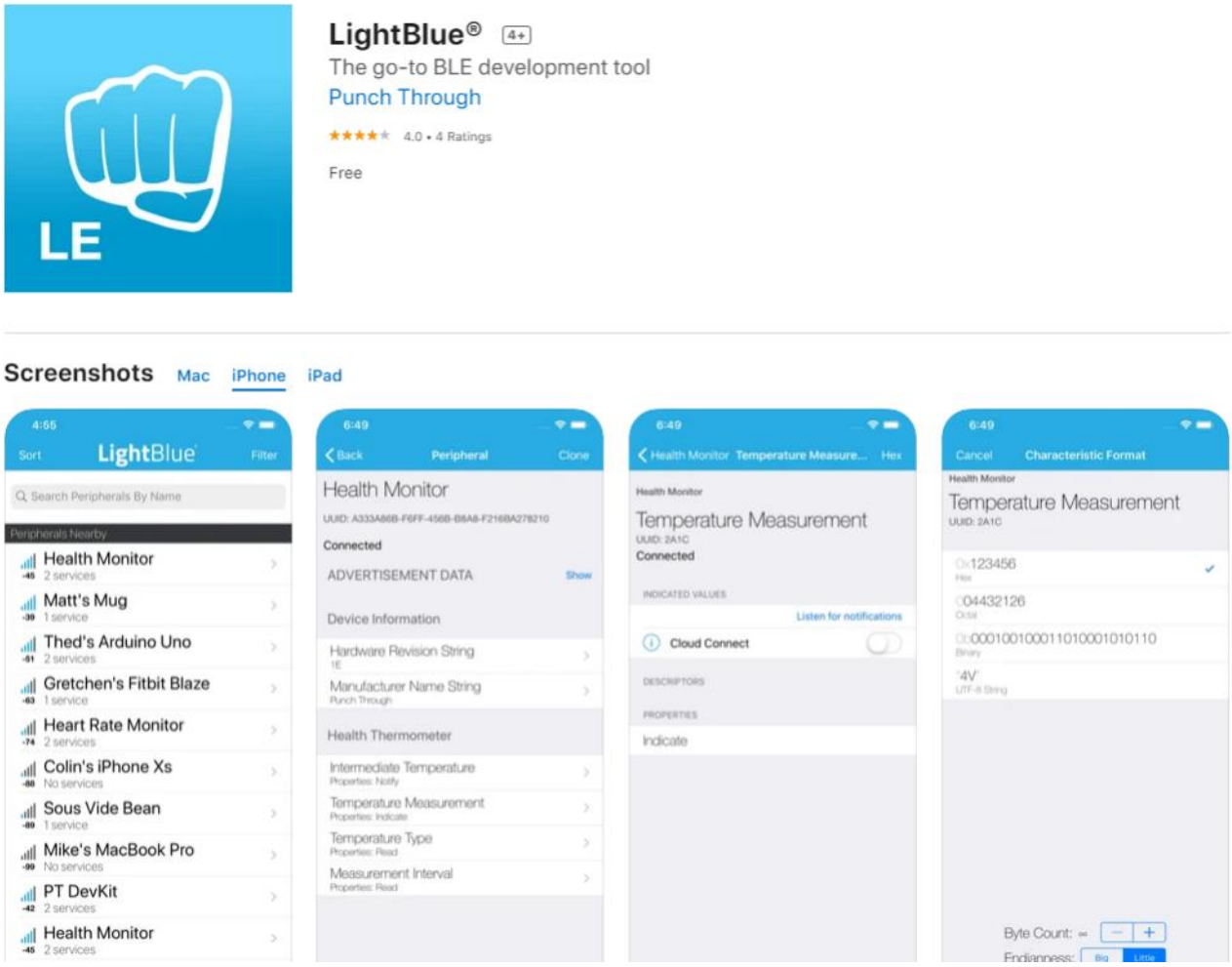
Connect Pico W to the computer using the USB cable.



LightBlue

If Serial Bluetooth cannot be installed on your phone, try LightBlue. If you do not have this software installed on your phone, you can refer to this link:

https://apps.apple.com/us/app/lightblue/id557428110?platform=iphone

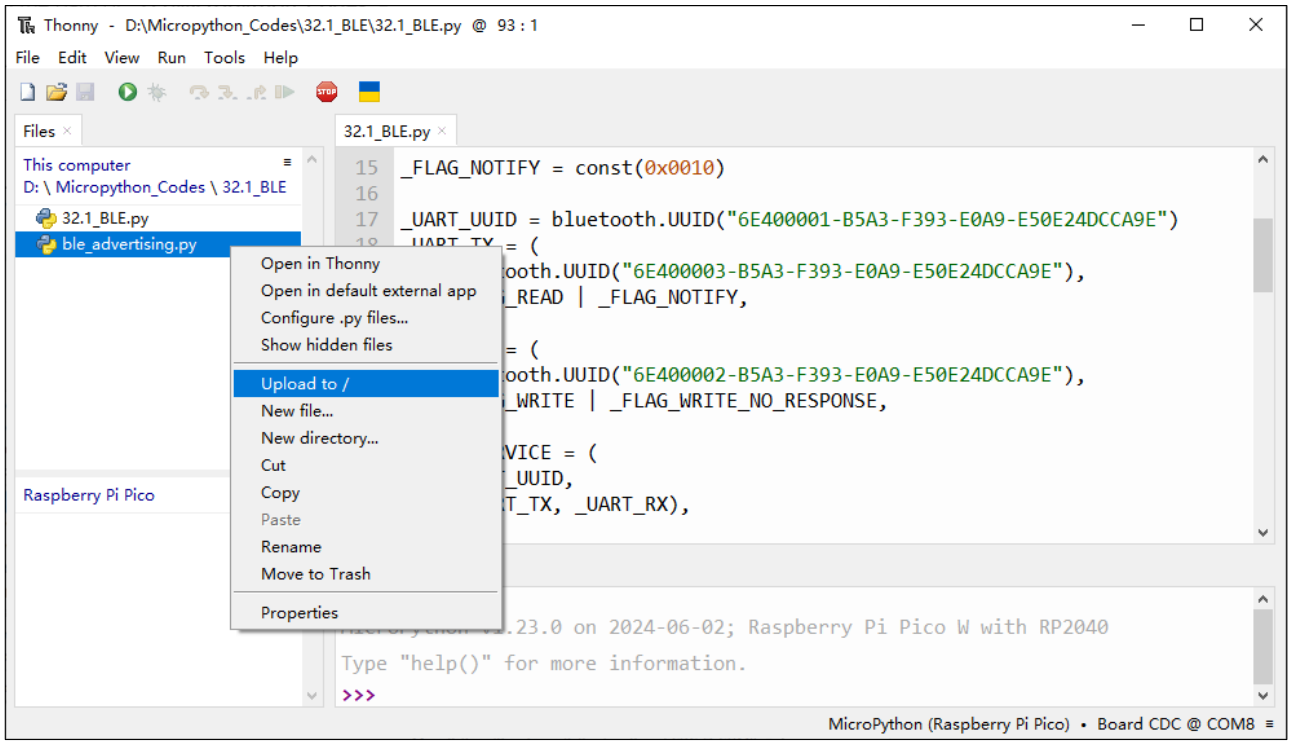


Code

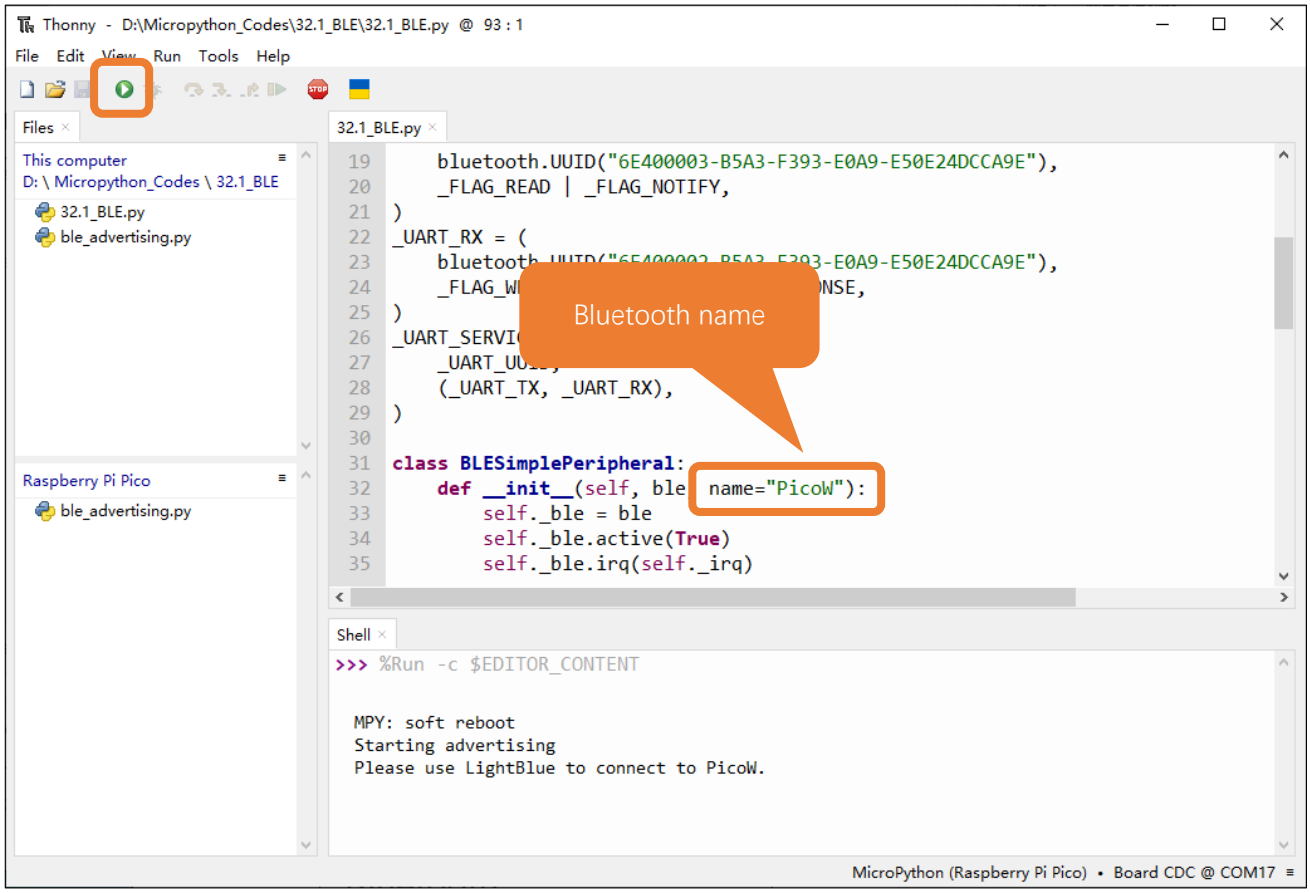
Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "32.1_BLE". Right click "ble_advertising.py" and select "Upload to /" to upload it to Pico W. Then, double click "ble.py" to open the file.

32.1_BLE



Click the Run button to run BLE.py.



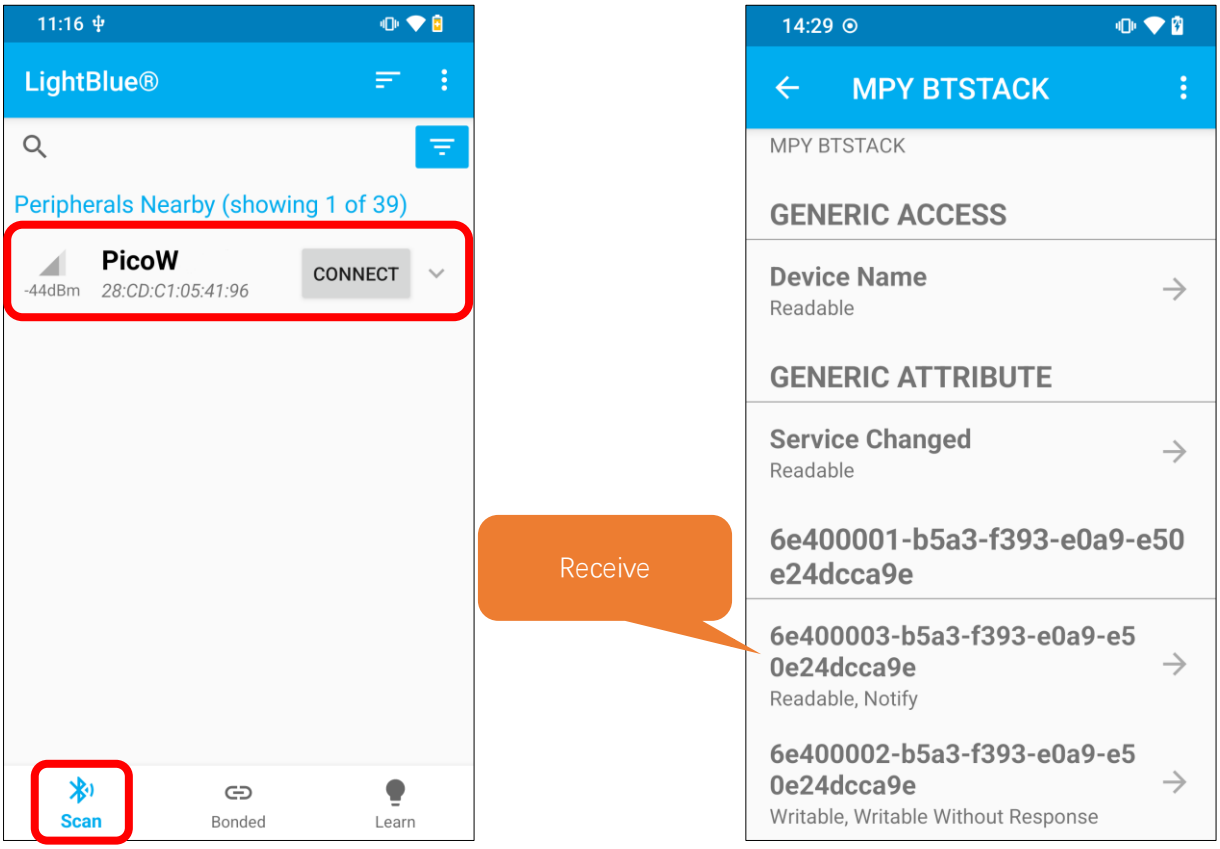
Note: If the Bluetooth name cannot be found, you can try modifying the current Bluetooth name.

Turn ON Bluetooth on your phone and open LightBlue APP.





At the device scan page, scroll down to refresh the devices nearby. Select PicoW to connect.

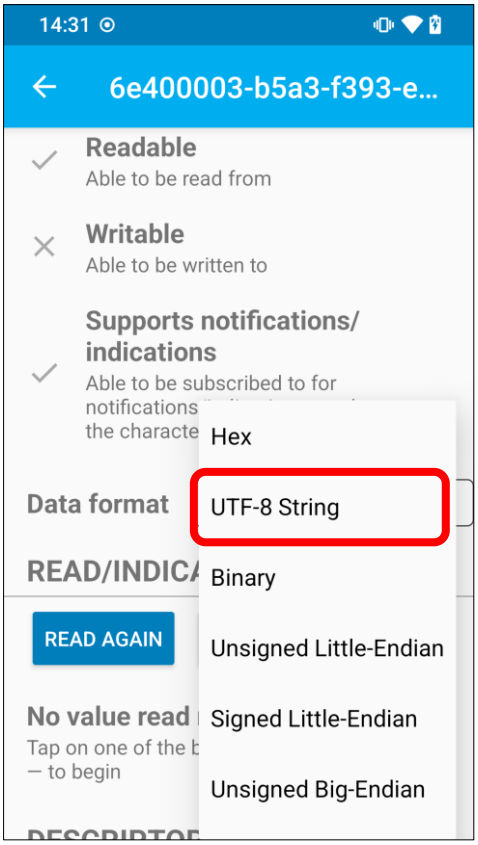


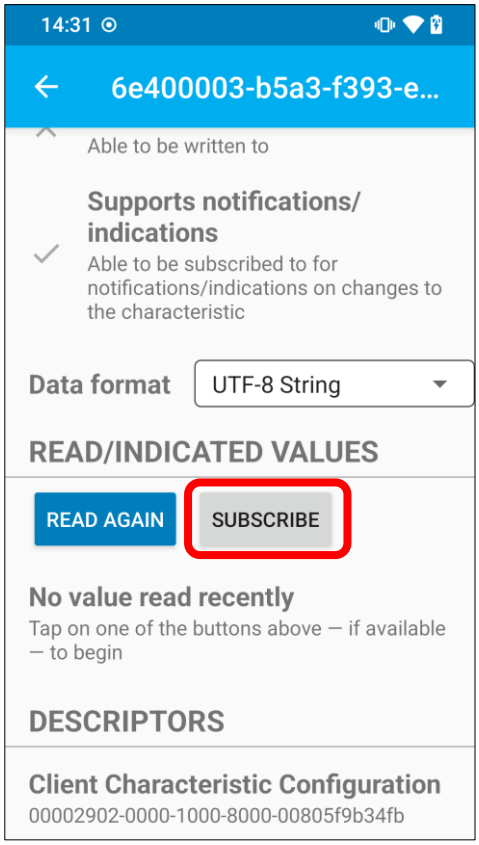
Once the Bluetooth connects successfully, it will print the messages as shown below.

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Starting advertising
Please use LightBlue to connect to PicoW.
New connection 64
Enter anything:
```

Click “Receive”. Select the corresponding data format in the box on the right side of the data format, such as HEX hexadecimal, utf-string string, Binary, etc. Then click SUBSCRIBE.



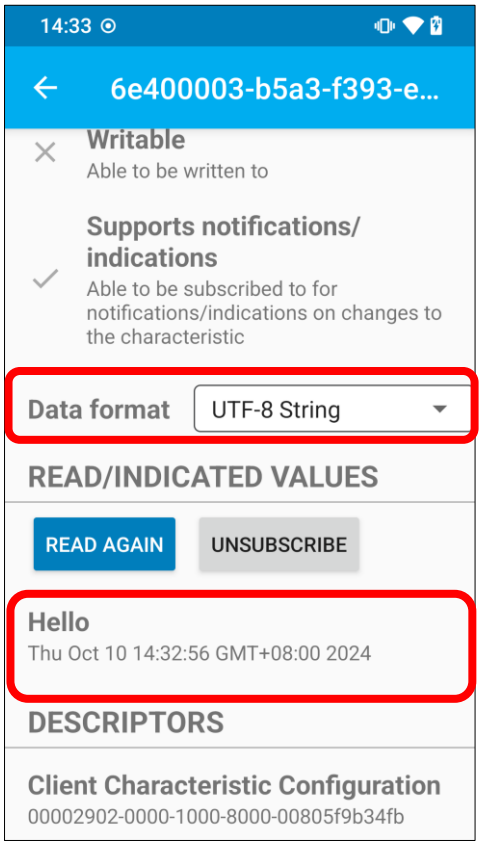


You can type "Hello" in Shell and press "Enter" to send.

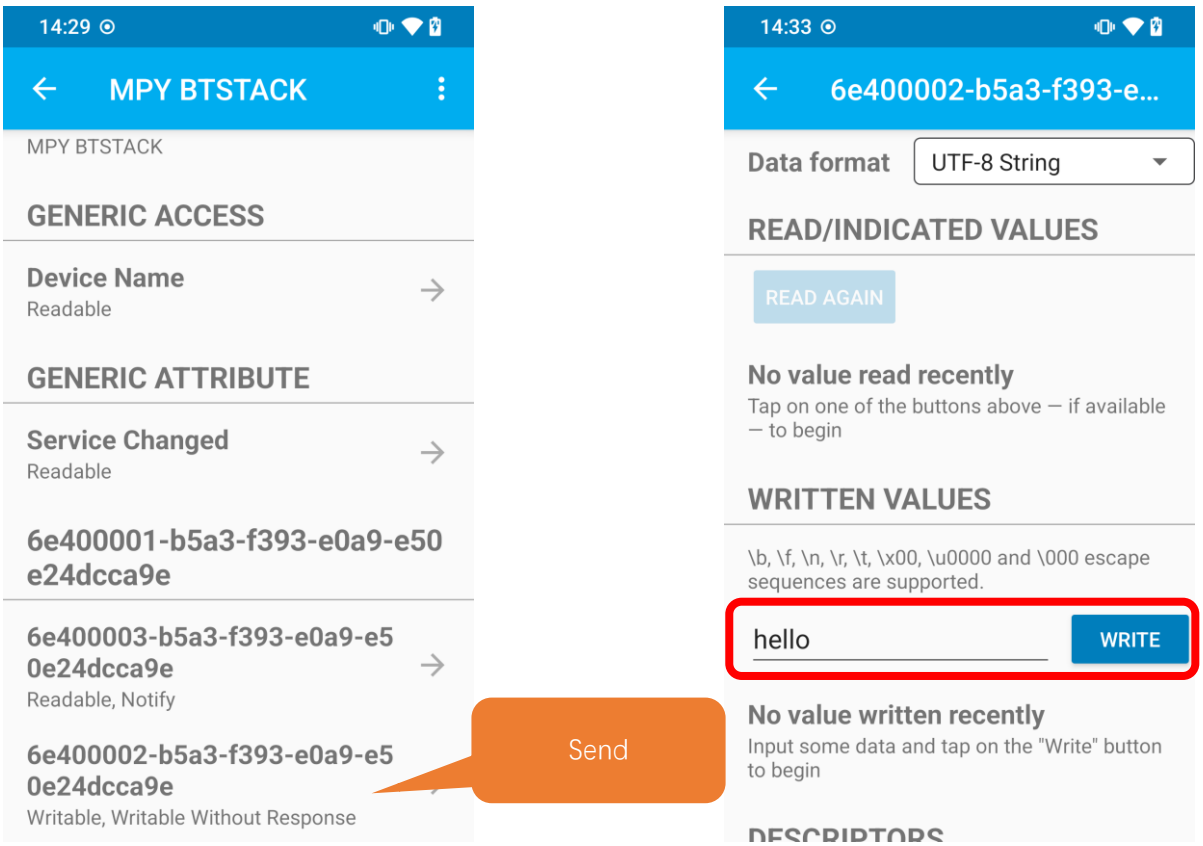
```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Starting advertising
Please use LightBlue to connect to PicoW.
New connection 64
Enter anything: Hello
Send:  Hello
Enter anything:
```

Then you can see the mobile Bluetooth has received the message.



Similarly, you can select "Send" on your phone. Set Data format, and then enter anything in the sending box and click Write to send.



You can check the message from Bluetooth in "Shell".

```
MPY: soft reboot
Starting advertising
Please use LightBlue to connect to PicoW.
New connection 64
Enter anything: Hello
Send:  Hello
Enter anything:
Receive: hello
```

The following is the program code:

```
1  import bluetooth
2  import random
3  import struct
4  import time
5  from ble_advertising import advertising_payload
6  from micropython import const
7
8  _IRQ_CENTRAL_CONNECT = const(1)
9  _IRQ_CENTRAL_DISCONNECT = const(2)
10 _IRQ_GATTS_WRITE = const(3)
11
12 _FLAG_READ = const(0x0002)
13 _FLAG_WRITE_NO_RESPONSE = const(0x0004)
14 _FLAG_WRITE = const(0x0008)
15 _FLAG_NOTIFY = const(0x0010)
16
17 _UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
18 _UART_TX = (
19     bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),
20     _FLAG_READ | _FLAG_NOTIFY,
21 )
22 _UART_RX = (
23     bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
24     _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
25 )
26 _UART_SERVICE = (
27     _UART_UUID,
28     (_UART_TX, _UART_RX),
29 )
30
31 class BLESimplePeripheral:
32     def __init__(self, ble, name="PicoW"):
33         self._ble = ble
```

```
        self._ble.active(True)
        self._ble.irq(self._irq)
        ((self._handle_tx, self._handle_rx),) =
self._ble.gatts_register_services((_UART_SERVICE,))
        self._connections = set()
        self._write_callback = None
        self._payload = advertising_payload(name=name, services=[_UART_UUID])
        self._advertise()

    def _irq(self, event, data):
        # Track connections so we can send notifications.
        if event == _IRQ_CENTRAL_CONNECT:
            conn_handle, _, _ = data
            print("New connection", conn_handle)
            self._connections.add(conn_handle)
        elif event == _IRQ_CENTRAL_DISCONNECT:
            conn_handle, _, _ = data
            print("Disconnected", conn_handle)
            self._connections.remove(conn_handle)
            # Start advertising again to allow a new connection.
            self._advertise()
        elif event == _IRQ_GATTS_WRITE:
            conn_handle, value_handle = data
            value = self._ble.gatts_read(value_handle)
            if value_handle == self._handle_rx and self._write_callback:
                self._write_callback(value)

    def send(self, data):
        for conn_handle in self._connections:
            self._ble.gatts_notify(conn_handle, self._handle_tx, data)

    def is_connected(self):
        return len(self._connections) > 0

    def _advertise(self, interval_us=500000):
        print("Starting advertising")
        self._ble.gap_advertise(interval_us, adv_data=self._payload)

    def on_write(self, callback):
        self._write_callback = callback


def demo():
    ble = bluetooth.BLE()
```

```
    p = BLESimplePeripheral(ble)

    def on_rx(rx_data):
        rx_data = rx_data.decode('utf-8').rstrip('\r\n')
        print("\nReceive:", rx_data)
    p.on_write(on_rx)
    print("Please use LightBlue to connect to PicoW.")

    while True:
        if p.is_connected():
            # Short burst of queued notifications.
            tx_data = input("Enter anything: ")
            print("Send: ", tx_data)
            p.send(tx_data)
if __name__ == "__main__":
    demo()

```

Define the specified UUID number for BLE vendor.

```
_UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
_UART_TX = (
    bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),
    _FLAG_READ | _FLAG_NOTIFY,
)
_UART_RX = (
    bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
    _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
)
```

Write a '_irq' function to manage BLE interrupt events

```
def _irq(self, event, data):
        # Track connections so we can send notifications.
        if event == _IRQ_CENTRAL_CONNECT:
            conn_handle, _, _ = data
            print("New connection", conn_handle)
            self._connections.add(conn_handle)
        elif event == _IRQ_CENTRAL_DISCONNECT:
            conn_handle, _, _ = data
            print("Disconnected", conn_handle)
            self._connections.remove(conn_handle)
            # Start advertising again to allow a new connection.
            self._advertise()
        elif event == _IRQ_GATTS_WRITE:
            conn_handle, value_handle = data
            value = self._ble.gatts_read(value_handle)
            if value_handle == self._handle_rx and self._write_callback:
```

```
59                      self._write_callback(value)
```

Initialize the BLE function and name it.

```
32  def __init__(self, ble, name="PicoW"):
```

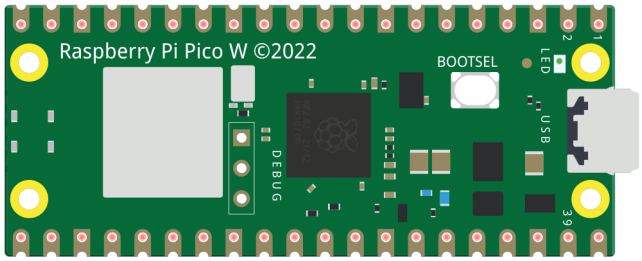
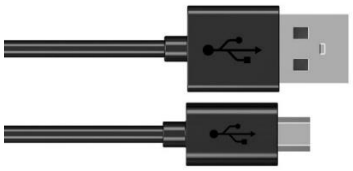
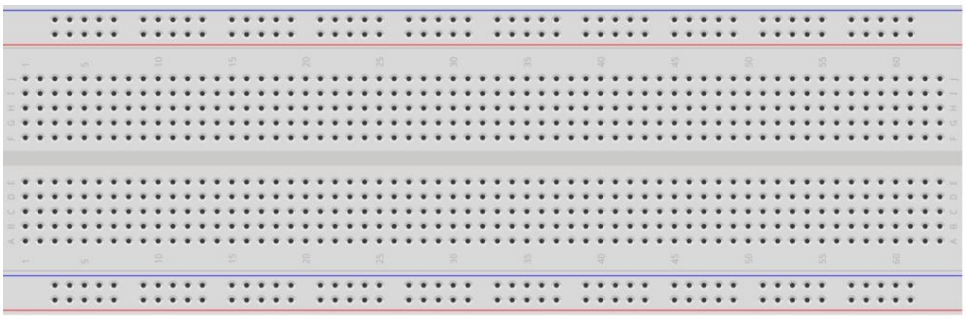
When the smartphone sends data to the Pico W via BLE Bluetooth, convert the data to utf-8 format and remove the trailing '\r\n', then print it out via the serial port. When the Pico W receives data from the serial port, it sends the data back to the smartphone via BLE Bluetooth.

```
76  def demo():
77      ble = bluetooth.BLE()
78      p = BLESimplePeripheral(ble)
79
80      def on_rx(rx_data):
81          rx_data = rx_data.decode('utf-8').rstrip('\r\n')
82          print("\nReceive:", rx_data)
83      p.on_write(on_rx)
84      print("Please use LightBlue to connect to PicoW.")
85
86      while True:
87          if p.is_connected():
88              # Short burst of queued notifications.
89              tx_data = input("Enter anything: ")
90              print("Send: ", tx_data)
91              p.send(tx_data)
```

Project 32.2 Bluetooth Low Energy Control LED

In this project, we will control an LED via Pico W's Bluetooth function.

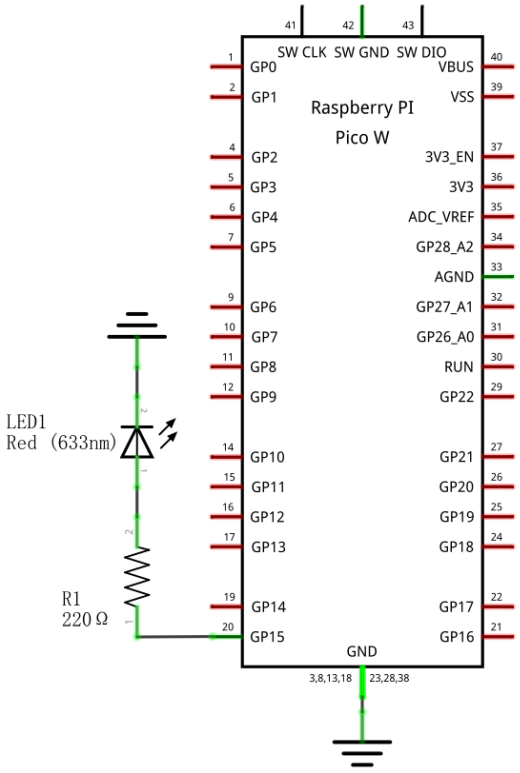
Component List

<table>
<tr><td colspan="2">Raspberry Pi Pico W(or Pico 2W) x1
  </td><td>Micro USB Wire x1
  </td></tr>
<tr><td colspan="3">Breadboard x1
  </td></tr>
<tr><td>LED x1
  </td><td>Resistor 220Ω x1
  </td><td>Jumper
  </td></tr>
</table>

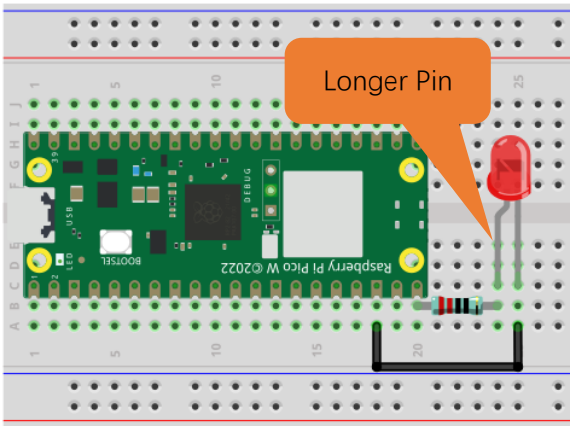
Circuit

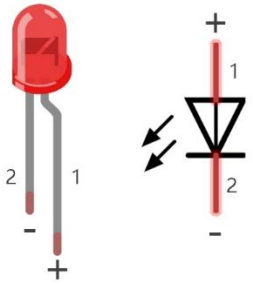
Connect Pico W to your computer with the USB cable.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com





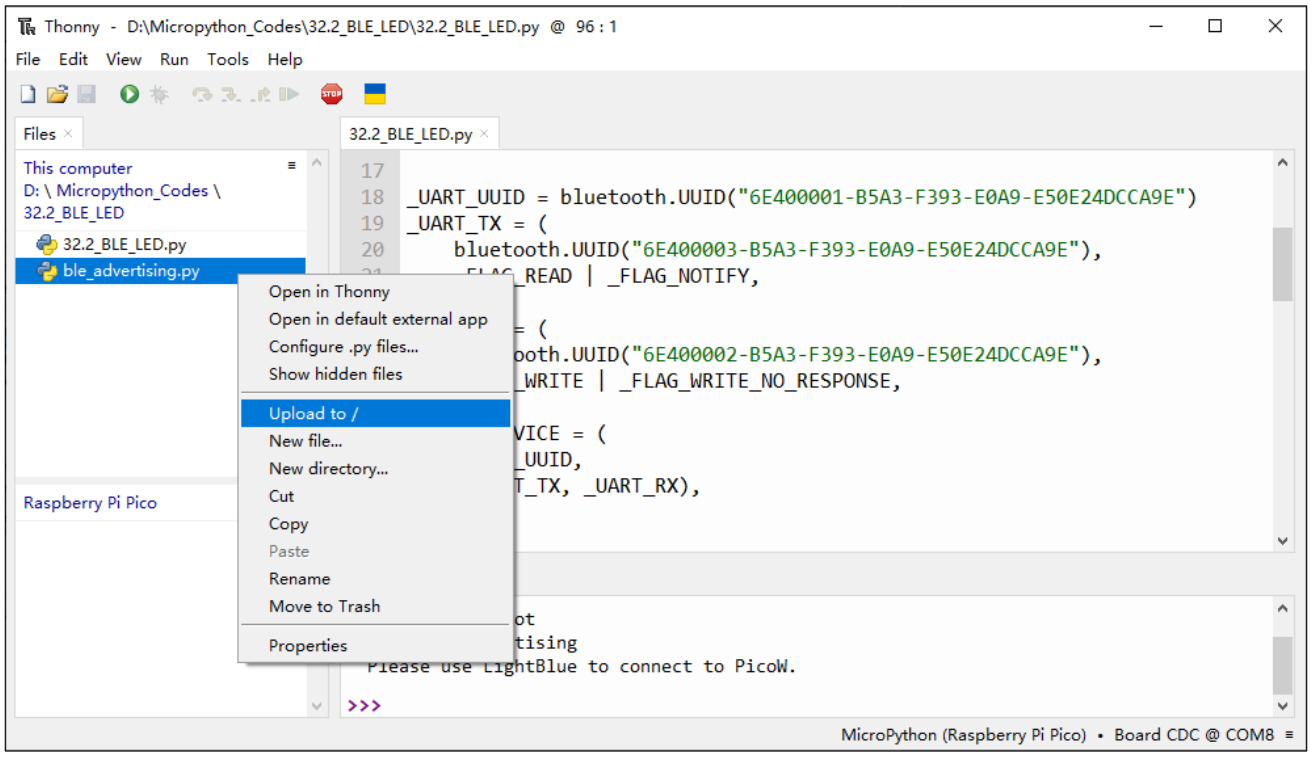
Note: To help users have a better experience when doing the projects, we have made some modifications to Pico's simulation diagram. Please note that there are certain differences between the simulation diagram and the actual board to avoid misunderstanding.

Code

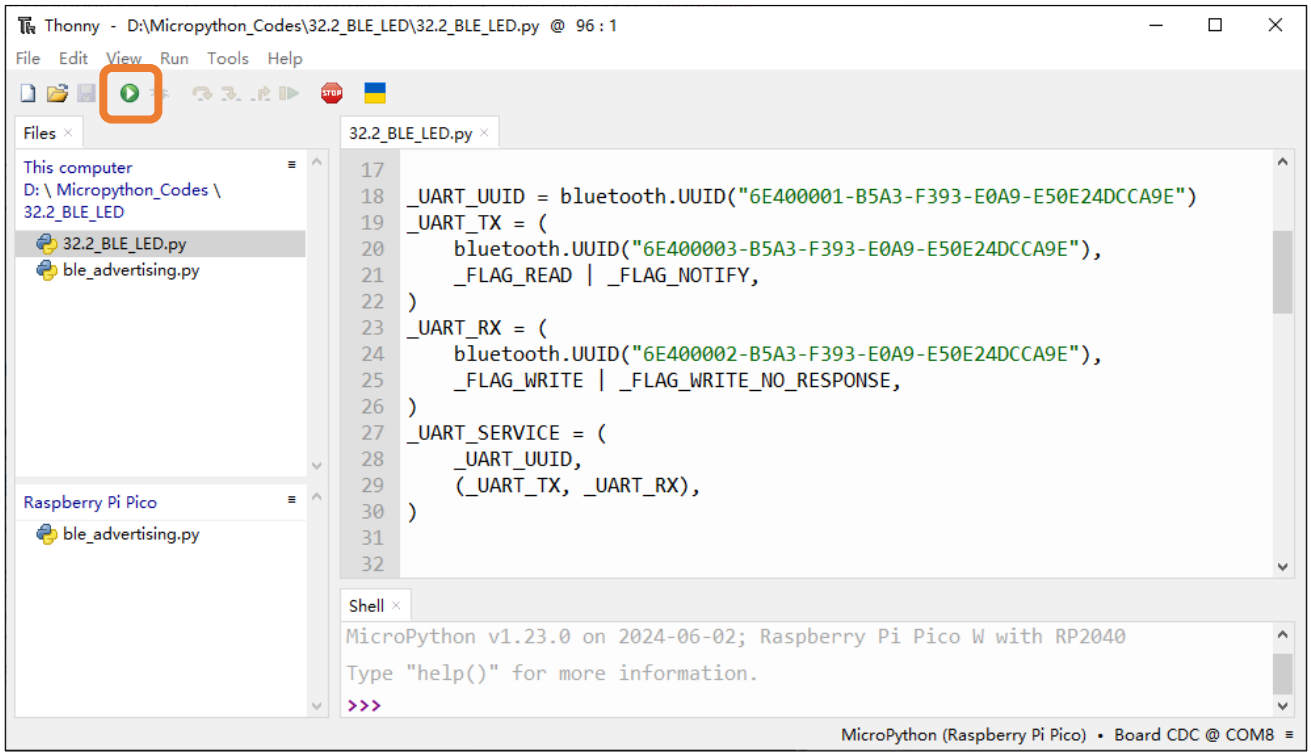
Move the program folder "**Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi_Pico/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "32.2_BLE_LED". Right click "ble_advertising.py" and select "Upload to /" to upload it to Pico W. Then, double click "ble_led.py" to open the file.

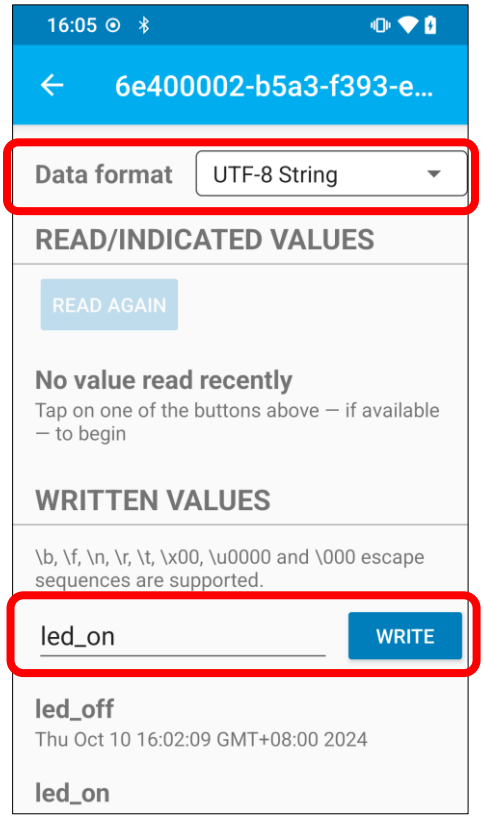
32.1_BLE_LED



Click the Run button to run BLE_LED.py.



The operation on the phone app is similar to that in project 33.1. You just need to change the sending messages to "led_on", "led_off" and "led_toggle" to control the status of the LED.



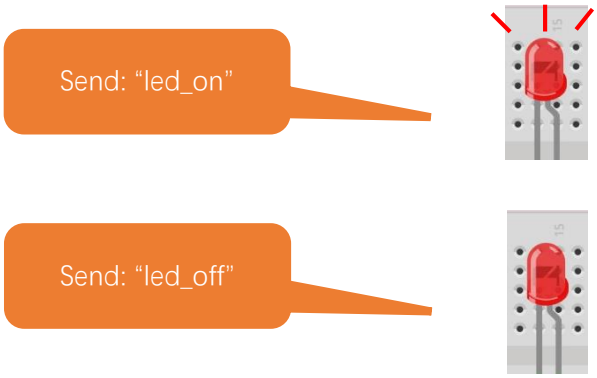
You can check the message sent by Bluetooth in "Shell".

```
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Starting advertising
  Please use LightBlue to connect to PicoW.

>>> New connection 64
  Receive: led_on
  Receive: led_off
  Receive: led_on
  Receive: led_off
```

LED Status.



Note: If the messages you send are not “led_on”, “led_off”, or “led toggle”, the status of the LED will not change.

For example, when the LED is already ON, it will remain ON unless the message “led_off”, or “led toggle” is received.

The following is the program code:

```
import bluetooth
import random
import struct
import time
from ble_advertising import advertising_payload
from machine import Pin
from micropython import const

_IRQ_CENTRAL_CONNECT = const(1)
_IRQ_CENTRAL_DISCONNECT = const(2)
_IRQ_GATTS_WRITE = const(3)

_FLAG_READ = const(0x0002)
_FLAG_WRITE_NO_RESPONSE = const(0x0004)
_FLAG_WRITE = const(0x0008)
_FLAG_NOTIFY = const(0x0010)

_UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
_UART_TX = (
    bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),
    _FLAG_READ | _FLAG_NOTIFY,
)
_UART_RX = (
    bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
    _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
)
_UART_SERVICE = (
    _UART_UUID,
    (_UART_TX, _UART_RX),
```

```
)

class BLESimplePeripheral:
    def __init__(self, ble, name="PicoW"):
        self._ble = ble
        self._ble.active(True)
        self._ble.irq(self._irq)
        ((self._handle_tx, self._handle_rx),) =
self._ble.gatts_register_services((_UART_SERVICE,))
        self._connections = set()
        self._write_callback = None
        self._payload = advertising_payload(name=name, services=[_UART_UUID])
        self._advertise()

    def _irq(self, event, data):
        # Track connections so we can send notifications.
        if event == _IRQ_CENTRAL_CONNECT:
            conn_handle, _, _ = data
            print("New connection", conn_handle)
            self._connections.add(conn_handle)
        elif event == _IRQ_CENTRAL_DISCONNECT:
            conn_handle, _, _ = data
            print("Disconnected", conn_handle)
            self._connections.remove(conn_handle)
            # Start advertising again to allow a new connection.
            self._advertise()
        elif event == _IRQ_GATTS_WRITE:
            conn_handle, value_handle = data
            value = self._ble.gatts_read(value_handle)
            if value_handle == self._handle_rx and self._write_callback:
                self._write_callback(value)

    def send(self, data):
        for conn_handle in self._connections:
            self._ble.gatts_notify(conn_handle, self._handle_tx, data)

    def is_connected(self):
        return len(self._connections) > 0

    def _advertise(self, interval_us=500000):
        print("Starting advertising")
        self._ble.gap_advertise(interval_us, adv_data=self._payload)

    def on_write(self, callback):
```

```
        self._write_callback = callback


def demo():
    ble = bluetooth.BLE()
    p = BLESimplePeripheral(ble)
    led = Pin(15, Pin.OUT)

    def on_rx(rx_data):
        rx_data = rx_data.decode('utf-8').rstrip('\r\n')
        print("Receive:", rx_data)
        if rx_data == 'led_on':
            led.value(1)
        elif rx_data == 'led_off':
            led.value(0)
        else:
            pass

    p.on_write(on_rx)
    print("Please use LightBlue to connect to PicoW.")
if __name__ == "__main__":
    demo()
```

Compare received message with "led_on" and "led_off" and take action accordingly.

```
if rx_data == 'led_on':
            led.value(1)
        elif rx_data == 'led_off':
            led.value(0)
        else:
            pass
```

What's Next?

THANK YOU for participating in this learning experience!

We have reached the end of this Tutorial. If you find errors, omissions or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us: support@freenove.com
We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you want to learn more about Arduino, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost-effective, innovative and exciting products.
https://www.freenove.com/

Thank you again for choosing Freenove products.